Configurable Multiagent System for Plug&Produce

CMAS 2.15.0 Fredrik Danielsson

1 CMAS Main Page **1**

1.1 Introduction . . . 1

1.2 Plug & Produce and Multiagent . . . 2

1.2.1 CMAS . . . 2

1.2.2 Overview of CMAS execution model . . . 2

1.2.3 Goals . . . 3

1.2.4 Skills . . . 3

1.2.5 Interfaces . . . 3

1.2.6 Resources . . . 3

1.2.7 Planner Agent . . . 4

1.2.8 Summary . . . 4

1.3 How to use CMAS . . . 4

1.3.1 Configuration . . . 4

1.3.1.1 Goals . . . 5

1.3.1.2 Process plans . . . 5

1.3.1.3 Skills . . . 6

2 Agent action **7**

2.1 State Entry Behavior . . . 7

2.2 Procedures . . . 8

2.2.1 Constructor . . . 8

2.2.2 onDepot . . . 9

2.2.3 onDelpoy . . . 9

2.2.4 Running . . . 9

2.2.5 onRun (agent) . . . 9

2.2.6 onFail . . . 10

2.2.7 onUndeploy . . . 10

2.2.8 onFinished . . . 10

2.2.9 onStop . . . 10

2.3 Preconditions . . . 10

2.3.1 Activate . . . 10

2.3.2 Failed . . . 11

2.3.3 Deactivate . . . 11

2.4 Randomized retry jitter . . . 11

2.4.1 RetryJitter . . . 11

2.4.2 RetryBackoff . . . 12

2.4.2.1 Timing illustration . . . 12

3 CMAS Command Line Arguments **13**

3.1 Command Line . . . 13

3.1.1 Preferences and Command-Line Overrides . . . 13

3.1.2 General Options . . . 13

3.1.3 Application . . . 14

3.1.4 Editor . . . 14
3.1.5 RunTime . . . 15
3.1.6 Safety . . . 16
3.1.7 PathFinder . . . 16
3.1.8 Process Plan (Java) . . . 16
3.1.9 Developer Modes . . . 17
3.1.10 Example Usage . . . 17

4 Deployment **19**
4.1 Creating and Using Batch Objects in CMAS . . . 19
4.1.1 How to Create a Batch Object . . . 19
4.1.2 Opening the Batch Editor . . . 19
4.1.3 Using the Batch Editor . . . 19
4.1.4 Running . . . 20
4.2 Creating and Using Deployment Agents in CMAS . . . 20
4.2.1 How to Create a Deployment Agent . . . 20
4.2.2 Working with Batches in a Deployment Agent . . . 20
4.2.3 Deployment Agent Properties . . . 20
4.2.3.1 **NumberOfTimes** . . . 20
4.2.3.2 **VariableTrigger** . . . 20
4.2.3.3 **DeploymentTime** . . . 20
4.2.3.4 **DeployAt** . . . 21
4.3 User defined logics for deployment . . . 21
4.3.1 Centralized Deployment Agent . . . 21
4.3.2 Local Deployment . . . 22

5 CMAS Examples **23**
5.1 Example 1 - Name tag . . . 23
5.1.1 Resource Agent 1 - Drill Bit . . . 23
5.1.2 Resource Agent 2 - Drill Station . . . 25
5.1.3 Part Agent - NameTag . . . 25
5.1.4 Process Plans . . . 25
5.1.5 Skills . . . 26
5.1.6 Deployment . . . 26
5.1.7 Errors . . . 27
5.2 Crane Simulation Example . . . 28
5.2.1 Introduction . . . 28
5.2.1.1 Table of MODBUS signals . . . 28
5.2.2 Modelling with planner . . . 28
5.2.2.1 Create Parts . . . 28
5.2.2.2 Create template variable . . . 29
5.2.2.3 Create Sources . . . 29
5.2.2.4 Create Crane . . . 30

5.2.2.5 Create Processes . . . 33
5.2.2.6 Create Sink . . . 34
5.2.2.7 Set location values . . . 35
5.2.3 Goals . . . 35
5.2.4 Running the example . . . 35
5.3 Crane-3D Simulation Example . . . 36
5.3.1 Introduction . . . 36
5.3.2 Plug and Produce . . . 36
5.3.3 Adding table resources . . . 37
5.3.3.1 Internal table variables . . . 37
5.3.3.2 Agent logic . . . 37
5.3.3.3 Table 2 . . . 40
5.3.4 Create machine resource . . . 41
5.3.4.1 Internal machine variables . . . 41
5.3.4.2 Agent logic . . . 41
5.3.5 Create the sink resource . . . 42
5.3.6 Create a process plan . . . 42
5.3.6.1 Process overview . . . 42
5.3.7 Create part agents . . . 43
5.3.8 Create the crane agent . . . 43
5.3.8.1 Internal crane variables . . . 44
5.3.8.2 Agent logic . . . 44
5.3.9 Adding interfaces . . . 45
5.3.9.1 Interface design . . . 45
5.3.9.2 Part1 and Part 2 . . . 45
5.3.9.3 Table 1 and Table 2 . . . 45
5.3.9.4 Machine 1 and Machine 2 . . . 46
5.3.9.5 Machine skills . . . 46
5.3.9.6 Spatial mapping of interfaces . . . 46
5.3.9.7 Sink . . . 47
5.3.9.8 Crane . . . 47
5.3.9.9 Crane execution logic . . . 48
5.3.9.10 Booking synchronization . . . 52
5.3.10 Running the complete manufacturing system . . . 52
5.3.10.1 Dynamic reconfiguration . . . 53
5.3.11 Summary . . . 53

6 Goal Charts **55**
6.1 The Goal Chart Editor . . . 55
6.2 Goal chart blocks . . . 56
6.2.1 Precondition . . . 56
6.2.2 Breakpoints . . . 57

6.2.3 Parallel . . . 57
6.2.4 Parallel Ordered . . . 57
6.2.5 Unordered . . . 58
6.2.6 Alternative . . . 58
6.2.7 Actions . . . 58
6.2.8 Requirement . . . 58
6.2.9 Done . . . 59
6.2.10 Fail . . . 59
6.2.11 Interrupt . . . 59
6.2.12 Label . . . 59
6.2.13 Jump . . . 59

7 Interfaces **61**
7.1 Interface action . . . 62
7.1.1 Procedures . . . 62
7.1.1.1 onAttach . . . 62
7.1.1.2 onDetach . . . 63
7.1.1.3 onTransfer . . . 63
7.1.1.4 onNegotiate . . . 63
7.1.1.5 onQueue . . . 63
7.1.1.6 onOverlap . . . 63
7.1.1.7 onBook . . . 63
7.1.1.8 onUnbook . . . 63
7.2 Negotiation . . . 64
7.2.1 Interfaces . . . 64
7.2.2 Collaboration . . . 64
7.2.3 The negotiation process . . . 64
7.2.4 Interface matching . . . 65
7.2.5 Variable matching . . . 66
7.2.6 Details . . . 66
7.2.6.1 Negotiation and Booking . . . 67
7.3 Variable bounds . . . 69
7.3.1 Integer and Real . . . 69
7.3.2 String . . . 69
7.3.3 Boolean . . . 69
7.4 Goal . . . 69
7.5 Abstract Interfaces in CMAS . . . 71
7.5.1 Concept and purpose . . . 71
7.5.2 Mental Model . . . 71
7.5.3 Plug & Produce support . . . 71
7.5.4 Declaring abstract interfaces . . . 71
7.5.4.1 Example 1: Simple abstract interfaces . . . 72

7.5.4.2 Example 2: Abstract interface with handle modifier . . . 72
7.5.5 Defining local interfaces . . . 72
7.5.5.1 Example . . . 72
7.5.6 Resource binding after negotiation . . . 72
7.5.7 Interface validation during negotiation . . . 73
7.5.8 Local abstract interfaces . . . 73
7.6 Advanced Interface Concepts . . . 73
7.6.1 Parallel Interfaces . . . 73
7.6.2 Interface Queues . . . 74
7.6.2.1 Recommended Scheduling Pattern . . . 74
7.6.2.2 Example Strategy . . . 75

8 Ontology **77**
8.1 Introduction . . . 77
8.2 How to Read the CMAS Ontology . . . 81
8.3 cmas.ontology Objects . . . 81
8.3.1 Entity . . . 81
8.3.2 Note . . . 82
8.3.3 Settings . . . 83
8.3.4 Folder . . . 83
8.4 cmas.ontology.processplans . . . 83
8.4.1 ProcessPlan . . . 83
8.4.2 ProcessPlanGoals . . . 84
8.4.3 ProcessPlanStructuredText . . . 85
8.4.4 Hazard . . . 85
8.4.5 Risk . . . 86
8.5 cmas.ontology.agent . . . 86
8.5.1 AgentAction . . . 86
8.5.2 Resource . . . 88
8.5.3 Material . . . 88
8.5.4 Part . . . 89
8.5.5 DeploymentAgent . . . 89
8.6 cmas.ontology.variables . . . 90
8.6.1 Variable . . . 90
8.6.2 Value . . . 90
8.6.3 VariableBoolean . . . 91
8.6.4 VariableInteger . . . 91
8.6.5 VariableReal . . . 91
8.6.6 VariableString . . . 92
8.6.7 VariableArray . . . 92
8.6.8 VariableObject . . . 92
8.7 cmas.ontology.interfaces . . . 93

8.7.1 InterfaceBase . . . 93
8.7.2 Interface . . . 94
8.7.3 Goal . . . 94
8.7.4 InterfaceSpecialisation . . . 95
8.7.5 InterfaceSpecialisation templates . . . 96
8.8 cmas.ontology.workflow . . . 96
8.8.1 WorkFlow . . . 96
8.8.2 WorkFlowLog . . . 96
8.8.3 Requirement . . . 96
8.9 cmas.ontology.batch . . . 97
8.9.1 Batch . . . 97
8.9.2 BatchItem . . . 97
8.10 Extended Ontology . . . 97

9 Planner **101**
9.1 Introduction . . . 101
9.1.1 Executing the Plan . . . 101
9.2 Example 1 - one robot . . . 102
9.2.1 Machine . . . 102
9.2.2 Buffer . . . 103
9.2.3 Robot . . . 103
9.2.4 Part . . . 103
9.3 Example 2 - adding an operator . . . 104
9.3.1 Updated robot . . . 104
9.3.2 Running Example 2 . . . 105
9.4 Goal-Driven Planning . . . 105
9.4.1 Skill Requirements: AND / OR Semantics . . . 105
9.4.2 Representing Skill Requirements in Structured Text . . . 106

10 Processplans **107**
10.1 Part Agent . . . 107
10.2 Resource Agent . . . 107
10.3 Global process plans . . . 107
10.4 The hierarchical structure: Goal to Execution . . . 108
10.5 Processplan action . . . 108
10.5.1 Processplan . . . 108
10.5.1.1 Process plan execution . . . 108
10.5.1.2 Processplan return value . . . 109
10.5.1.3 onNegotiate . . . 110
10.5.1.4 onRun (processplan) . . . 110
10.5.1.5 onFinished . . . 111
10.5.1.6 onOverlap . . . 111
10.5.1.7 Example . . . 111

11 Requirements **113**
11.1 Introduction . . . 113
11.2 Example requirement on a skill . . . 113
11.2.1 Model . . . 113
11.2.2 Step-by-step instructions: . . . 114
11.2.2.1 1. Buffer . . . 114
11.2.2.2 2. Machine . . . 114
11.2.2.3 3. Robot . . . 115
11.2.2.4 4. ProductA . . . 115
11.2.2.5 Run . . . 115
11.3 Example requirements in a process plan . . . 115
11.3.1 Assembly . . . 116
11.4 InterfaceSpecialisation . . . 116
11.4.1 InterfaceLocation . . . 116
11.4.2 InterfaceTransport . . . 116
11.4.3 InterfaceAssembly . . . 116

12 Structured text **117**
12.1 Introduction . . . 117
12.2 The overall structure in a program . . . 117
12.2.1 Program . . . 117
12.2.2 Directives . . . 118
12.2.2.1 Include . . . 119
12.2.2.2 Use . . . 119
12.2.2.3 Error . . . 119
12.2.3 Struct type . . . 119
12.2.4 Interface . . . 120
12.2.4.1 Asynchronous functions . . . 120
12.2.4.2 Static functions . . . 121
12.2.4.3 Abstract interface declarations . . . 121
12.2.4.4 Define interface declaration . . . 122
12.2.4.5 Static interface declarations . . . 122
12.2.5 Global variable declarations . . . 123
12.2.6 Arrays . . . 123
12.2.7 Comment . . . 123
12.2.8 Function declaration . . . 124
12.2.9 Procedure declaration . . . 124
12.2.10 Overriding ontology functions and procedures . . . 125
12.2.11 Main program block . . . 126
12.3 Control structures . . . 126
12.3.1 while . . . 126
12.3.2 repeat . . . 127

12.3.3 for . . . 127
12.3.4 foreach . . . 127
12.3.5 for dwonto . . . 128
12.3.6 if else if else . . . 128
12.3.7 case . . . 129
12.3.8 exit . . . 129
12.3.9 break . . . 130
12.3.10 error . . . 130
12.3.11 label . . . 130
12.3.12 parallel . . . 130
12.4 Operators . . . 131
12.4.1 Arithmetic . . . 131
12.4.2 Assignment . . . 131
12.4.3 Relations . . . 132
12.4.4 Logical . . . 132
12.4.5 Object . . . 133
12.4.6 Array . . . 133
12.5 Types . . . 133
12.5.1 Real . . . 133
12.5.2 Integer . . . 133
12.5.3 Boolean . . . 134
12.5.4 String . . . 134
12.5.5 Struct . . . 134
12.5.6 Objects . . . 134
12.5.6.1 this . . . 134
12.5.6.2 agent . . . 134
12.5.6.3 interface . . . 134
12.5.6.4 remoteinterface . . . 134
12.5.6.5 Goal . . . 134
12.5.6.6 ProcessPlan . . . 134
12.5.7 Reference . . . 135
12.6 Procedures . . . 135
12.6.1 Input and output . . . 135
12.6.1.1 writeln . . . 135
12.6.1.2 write . . . 135
12.6.1.3 readln . . . 136
12.6.2 Abstract Interface Functions . . . 136
12.6.2.1 abstractname . . . 136
12.6.2.2 abstractcost . . . 136
12.6.2.3 abstractshare . . . 137
12.6.2.4 abstractfind . . . 137
12.6.2.5 ignorebooked . . . 137

12.6.2.6 ignorebusy . . . 138
12.6.3 Test Functions . . . 138
12.6.3.1 isreal . . . 138
12.6.3.2 isboolean . . . 138
12.6.3.3 isinteger . . . 138
12.6.3.4 isstring . . . 138
12.6.3.5 stringisreal . . . 139
12.6.3.6 stringisinteger . . . 139
12.6.3.7 stringisboolean . . . 139
12.6.3.8 stringisempty . . . 139
12.6.4 Converting Functions . . . 139
12.6.4.1 realtoint . . . 140
12.6.4.2 inttoint . . . 140
12.6.4.3 booleantoint . . . 140
12.6.4.4 stringtoint . . . 140
12.6.4.5 realtoreal . . . 140
12.6.4.6 inttoreal . . . 140
12.6.4.7 booleantoreal . . . 141
12.6.4.8 stringtoreal . . . 141
12.6.4.9 realtoboolean . . . 141
12.6.4.10 inttoboolean . . . 141
12.6.4.11 booleantoboolean . . . 141
12.6.4.12 stringtoboolean . . . 141
12.6.4.13 realtostring . . . 141
12.6.4.14 inttostring . . . 142
12.6.4.15 booleantostring . . . 142
12.6.4.16 stringtostring . . . 142
12.6.5 String Functions . . . 142
12.6.5.1 len . . . 142
12.6.5.2 left . . . 142
12.6.5.3 right . . . 142
12.6.5.4 mid . . . 143
12.6.5.5 trim . . . 143
12.6.5.6 replace . . . 143
12.6.5.7 contains . . . 144
12.6.5.8 tolowercase . . . 144
12.6.5.9 touppercase . . . 144
12.6.5.10 split . . . 144
12.6.5.11 tostring . . . 144
12.6.5.12 chr . . . 145
12.6.5.13 ord . . . 145
12.6.5.14 space . . . 145

12.6.6 JSON functions . . . 146
12.6.6.1 GetJElement . . . 146
12.6.6.2 SetJElement . . . 146
12.6.6.3 JToStruct . . . 146
12.6.6.4 StructToJ . . . 147
12.6.6.5 IsJson . . . 147
12.6.7 Math functions . . . 147
12.6.7.1 abs . . . 148
12.6.7.2 ceil . . . 148
12.6.7.3 floor . . . 149
12.6.7.4 round . . . 149
12.6.7.5 max . . . 149
12.6.7.6 min . . . 149
12.6.7.7 exp . . . 149
12.6.7.8 log . . . 149
12.6.7.9 log10 . . . 150
12.6.7.10 sqrt . . . 150
12.6.7.11 sin . . . 150
12.6.7.12 cos . . . 150
12.6.7.13 tan . . . 150
12.6.7.14 asin . . . 150
12.6.7.15 acos . . . 150
12.6.7.16 atan . . . 151
12.6.7.17 sinh . . . 151
12.6.7.18 cosh . . . 151
12.6.7.19 tanh . . . 151
12.6.7.20 todegrees . . . 151
12.6.7.21 toradians . . . 151
12.6.7.22 random . . . 152
12.6.8 Array Functions . . . 152
12.6.8.1 length . . . 152
12.6.8.2 startindex . . . 152
12.6.8.3 endindex . . . 152
12.6.9 HMI . . . 152
12.6.9.1 messagebox . . . 152
12.6.9.2 inputdialog . . . 153
12.6.9.3 optiondialog . . . 154
12.7 System Functions . . . 154
12.7.1 Time . . . 154
12.7.1.1 sleep . . . 154
12.7.1.2 time . . . 155
12.7.1.3 beep . . . 155

12.7.1.4 memoryfree . . . 155
12.7.1.5 memorytotal . . . 155
12.7.1.6 memoryused . . . 156
12.7.2 Preferences . . . 156
12.7.2.1 preferencevalue . . . 156
12.7.3 Debug . . . 156
12.7.3.1 threadmessage . . . 156
12.7.4 File Functions . . . 157
12.7.4.1 open . . . 157
12.7.4.2 MODE . . . 157
12.7.4.3 close . . . 157
12.7.4.4 fwriteln . . . 157
12.7.4.5 freadln . . . 158
12.8 Server API Functions . . . 158
12.8.1 Pause . . . 158
12.8.1.1 pauseAll . . . 158
12.8.1.2 pausebyname . . . 159
12.8.2 Console and log . . . 159
12.8.2.1 clearconsole . . . 159
12.8.2.2 clearlog . . . 159
12.8.3 Runtime control . . . 159
12.8.3.1 isruntime . . . 159
12.8.3.2 stopsystem . . . 159
12.8.3.3 restartsystem . . . 160
12.8.3.4 getagentsbyname . . . 160
12.8.3.5 getagentsbyid . . . 160
12.8.3.6 deployandattach . . . 160
12.8.3.7 deploybyinstanceid . . . 161
12.8.3.8 deploybyid . . . 161
12.8.3.9 deploybyname . . . 161
12.8.3.10 deployall . . . 161
12.8.3.11 undeploybyname . . . 161
12.8.3.12 undeploybyid . . . 161
12.8.3.13 undeploybyinstanceid . . . 162
12.8.3.14 undeployall . . . 162
12.8.3.15 getnumberofrunningparts . . . 162
12.8.3.16 getnumberofrunningresources . . . 162
12.8.3.17 attach . . . 163
12.8.3.18 assembly . . . 163
12.8.4 Object access . . . 163
12.8.4.1 getobject . . . 163
12.8.4.2 getrelation . . . 164

12.8.4.3 getreference . . . 164
12.9 Examples . . . 165
12.9.1 Example 1 - Arrays . . . 165
12.9.2 Example 2 - Interfaces and objects . . . 165
12.9.3 Example 3 - Structures . . . 165
12.9.4 Example 4 - MessageBox User Information . . . 165
12.9.5 Example 5 - InputDialog . . . 166
12.9.6 Example 6 - OptionDialog . . . 166
12.9.7 Example 7 - JSON Get . . . 167
12.9.8 Example 8 - JSON and Struct mapping . . . 167
12.9.9 Example 9 - JSON Set . . . 168
12.9.10 Example 10 - Sort . . . 168
12.9.11 Example 11 - Foreach . . . 169
12.10 System include files . . . 170
12.10.1 ontology.inc . . . 170
12.10.2 file.inc . . . 171
12.10.3 messagebox.inc . . . 171
12.10.4 queue.inc . . . 172

13 Variables **173**
13.1 Overview . . . 173
13.2 Variable types . . . 174
13.2.1 Example: Variable Value . . . 174
13.2.2 Example: Array Variable . . . 174
13.2.3 Example: Object Variable . . . 175
13.3 Functions . . . 175
13.3.1 onBeforeRead . . . 175
13.3.2 onAfterRead . . . 176
13.3.3 onBeforeWrite . . . 176
13.3.4 onAfterWrite . . . 177
13.4 Variable adapters . . . 177
13.4.1 Introduction . . . 177
13.4.2 OPC UA . . . 178
13.4.3 REST . . . 178
13.4.4 FILE . . . 179
13.4.5 MODBUS . . . 179
13.4.6 SOCKET . . . 180
13.4.7 WEBSOCKET . . . 181
13.4.8 ZeroMQ . . . 181
13.4.9 SQL . . . 181
13.5 Variable bounds . . . 182
13.5.1 Integer and Real . . . 182

13.5.2 String . . . 184
13.5.3 Boolean . . . 184

14 Hierarchical Index **185**
14.1 Class Hierarchy . . . 185

15 Class Index **189**
15.1 Class List . . . 189

16 Class Documentation **193**
16.1 cmas.ontology.interfaces.AbstractInterface Class Reference . . . 193
16.1.1 Member Function Documentation . . . 194
16.1.1.1 getCommunicative() . . . 194
16.1.1.2 getMessageHandler() . . . 195
16.1.1.3 postMessage() . . . 195
16.2 cmas.ontology.interfaces.AbstractInterface.AbstractType Enum Reference . . . 195
16.3 cmas.ontology.agentcommunication.ACL.ACLMessage Class Reference . . . 195
16.4 cmas.ontology.action.Action Class Reference . . . 196
16.5 cmas.ontology.action.ActionInterface Interface Reference . . . 197
16.6 cmas.ontology.action.ActionJava Class Reference . . . 197
16.7 cmas.ontology.adapter.Adapter Interface Reference . . . 198
16.7.1 Member Function Documentation . . . 198
16.7.1.1 ReadData() . . . 198
16.7.1.2 setDebug() . . . 198
16.7.1.3 WriteData() . . . 199
16.8 cmas.ontology.adapter.AdapterFactory Class Reference . . . 199
16.8.1 Member Data Documentation . . . 199
16.8.1.1 adapterObjects . . . 199
16.9 cmas.ontology.adapter.Modbus.ModbusClient.ADDRESS_TYPE Enum Reference . . . 199
16.10 cmas.ontology.agents.Agent Class Reference . . . 200
16.10.1 Member Function Documentation . . . 202
16.10.1.1 copy() . . . 202
16.10.1.2 delete() . . . 202
16.10.1.3 DeSerialize() . . . 202
16.10.1.4 Factory() . . . 202
16.10.1.5 findRelationByName() . . . 203
16.10.1.6 findRelationByNameIgnoreCase() . . . 203
16.10.1.7 generateUniqueIDs() . . . 203
16.10.1.8 getAllRelations() . . . 203
16.10.1.9 getDescription() . . . 204
16.10.1.10 getID() . . . 204
16.10.1.11 getInstanceID() . . . 204
16.10.1.12 getLinkedInstanceID() . . . 204

16.10.1.13 getName() . . . 204
16.10.1.14 getRelations() . . . 204
16.10.1.15 getSpecificRelations() . . . 204
16.10.1.16 getType() . . . 205
16.10.1.17 hasRelation() . . . 205
16.10.1.18 Serialize() . . . 205
16.10.1.19 setDescription() . . . 205
16.10.1.20 setID() [1/2] . . . 205
16.10.1.21 setID() [2/2] . . . 206
16.10.1.22 setName() . . . 206
16.11 cmas.ontology.agents.AgentAction Class Reference . . . 206
16.11.1 Member Function Documentation . . . 209
16.11.1.1 copy() . . . 209
16.11.1.2 delete() . . . 210
16.11.1.3 DeSerialize() . . . 210
16.11.1.4 Factory() . . . 210
16.11.1.5 findRelationByName() . . . 210
16.11.1.6 findRelationByNameIgnoreCase() . . . 211
16.11.1.7 generateUniqueIDs() . . . 211
16.11.1.8 getAllRelations() . . . 211
16.11.1.9 getCommunicative() . . . 211
16.11.1.10 getDescription() . . . 211
16.11.1.11 getID() . . . 212
16.11.1.12 getInstanceID() . . . 212
16.11.1.13 getLinkedInstanceID() . . . 212
16.11.1.14 getMessageHandler() . . . 212
16.11.1.15 getName() . . . 212
16.11.1.16 getRelations() . . . 212
16.11.1.17 getSpecificRelations() . . . 212
16.11.1.18 getType() . . . 213
16.11.1.19 gotoLine() . . . 213
16.11.1.20 hasRelation() . . . 213
16.11.1.21 postMessage() . . . 213
16.11.1.22 Serialize() . . . 213
16.11.1.23 setDescription() . . . 213
16.11.1.24 setID() [1/2] . . . 214
16.11.1.25 setID() [2/2] . . . 214
16.11.1.26 setName() . . . 214
16.11.1.27 stopEditor() . . . 214
16.12 cmas.ontology.agents.Agent.AgentBaseTypes Enum Reference . . . 214
16.13 cmas.ontology.agents.AgentAction.AgentState Enum Reference . . . 215
16.14 cmas.ontology.interfaces.Attachment Class Reference . . . 215

16.15 cmas.ontology.batch.Batch Class Reference . . . 216
16.15.1 Member Function Documentation . . . 218
16.15.1.1 copy() . . . 218
16.15.1.2 delete() . . . 218
16.15.1.3 DeSerialize() . . . 218
16.15.1.4 Factory() . . . 218
16.15.1.5 findRelationByName() . . . 219
16.15.1.6 findRelationByNameIgnoreCase() . . . 219
16.15.1.7 generateUniqueIDs() . . . 219
16.15.1.8 getAllRelations() . . . 219
16.15.1.9 getDescription() . . . 219
16.15.1.10 getEntityDescription() . . . 220
16.15.1.11 getEntityIconFile() . . . 220
16.15.1.12 getID() . . . 220
16.15.1.13 getInstanceID() . . . 220
16.15.1.14 getLinkedInstanceID() . . . 220
16.15.1.15 getName() . . . 220
16.15.1.16 getRelations() . . . 220
16.15.1.17 getSpecificRelations() . . . 221
16.15.1.18 getType() . . . 222
16.15.1.19 hasRelation() . . . 222
16.15.1.20 openEntityEditor() . . . 222
16.15.1.21 Serialize() . . . 222
16.15.1.22 setDescription() . . . 222
16.15.1.23 setID() [1/2] . . . 223
16.15.1.24 setID() [2/2] . . . 223
16.15.1.25 setName() . . . 223
16.16 cmas.ontology.batch.BatchItem Class Reference . . . 223
16.17 cmas.ontology.agentcommunication.CommunicativeActs Interface Reference . . . 224
16.18 cmas.ontology.agents.Component Class Reference . . . 224
16.18.1 Member Function Documentation . . . 226
16.18.1.1 copy() . . . 226
16.18.1.2 delete() . . . 227
16.18.1.3 DeSerialize() . . . 227
16.18.1.4 Factory() . . . 227
16.18.1.5 findRelationByName() . . . 227
16.18.1.6 findRelationByNameIgnoreCase() . . . 228
16.18.1.7 generateUniqueIDs() . . . 228
16.18.1.8 getAllRelations() . . . 228
16.18.1.9 getCommunicative() . . . 228
16.18.1.10 getDescription() . . . 228
16.18.1.11 getID() . . . 229

16.18.1.12 getInstanceID() . . . 229
16.18.1.13 getLinkedInstanceID() . . . 229
16.18.1.14 getMessageHandler() . . . 229
16.18.1.15 getName() . . . 229
16.18.1.16 getRelations() . . . 229
16.18.1.17 getSpecificRelations() . . . 229
16.18.1.18 getType() . . . 230
16.18.1.19 gotoLine() . . . 230
16.18.1.20 hasRelation() . . . 230
16.18.1.21 postMessage() . . . 230
16.18.1.22 Serialize() . . . 230
16.18.1.23 setDescription() . . . 230
16.18.1.24 setID() [1/2] . . . 231
16.18.1.25 setID() [2/2] . . . 231
16.18.1.26 setName() . . . 231
16.18.1.27 stopEditor() . . . 231
16.19 cmas.ontology.base.Container Class Reference . . . 232
16.19.1 Member Function Documentation . . . 233
16.19.1.1 getLinkedInstanceID() . . . 233
16.20 cmas.ontology.agentcommunication.jsoncontent.Content Class Reference . . . 233
16.20.1 Member Function Documentation . . . 234
16.20.1.1 createParameters() . . . 234
16.21 cmas.ontology.agentcommunication.jsoncontent.template.ContentBusy Class Reference . . . 234
16.21.1 Member Function Documentation . . . 235
16.21.1.1 createParameters() . . . 235
16.22 cmas.ontology.agentcommunication.jsoncontent.template.ContentDone Class Reference . . . 235
16.22.1 Member Function Documentation . . . 236
16.22.1.1 createParameters() . . . 236
16.23 cmas.ontology.agentcommunication.jsoncontent.template.ContentError Class Reference . . . 236
16.23.1 Member Function Documentation . . . 237
16.23.1.1 createParameters() . . . 237
16.24 cmas.ontology.agentcommunication.jsoncontent.template.ContentInformation Class Reference . . . 237
16.24.1 Member Function Documentation . . . 238
16.24.1.1 createParameters() . . . 238
16.25 cmas.ontology.agentcommunication.jsoncontent.template.ContentNegotiate Class Reference . . . 238
16.25.1 Member Function Documentation . . . 239
16.25.1.1 createParameters() . . . 239
16.26 cmas.ontology.agentcommunication.jsoncontent.ContentParameter Class Reference . . . 239
16.26.1 Detailed Description . . . 239
16.27 cmas.ontology.agentcommunication.jsoncontent.template.ContentProgress Class Reference . . . 239
16.27.1 Member Function Documentation . . . 240
16.27.1.1 createParameters() . . . 240

16.28 cmas.ontology.agentcommunication.jsoncontent.template.ContentReplan Class Reference 240
16.28.1 Member Function Documentation 241
16.28.1.1 createParameters() 241
16.29 cmas.ontology.agentcommunication.jsoncontent.template.ContentResult Class Reference 241
16.29.1 Member Function Documentation 242
16.29.1.1 createParameters() 242
16.30 cmas.ontology.agents.DeploymentAgent Class Reference 242
16.30.1 Member Function Documentation 245
16.30.1.1 copy() 245
16.30.1.2 delete() 245
16.30.1.3 DeSerialize() 245
16.30.1.4 Factory() 245
16.30.1.5 findRelationByName() 246
16.30.1.6 findRelationByNameIgnoreCase() 246
16.30.1.7 generateUniqueIDs() 246
16.30.1.8 getAllRelations() 246
16.30.1.9 getCommunicative() 247
16.30.1.10 getDescription() 247
16.30.1.11 getEntityDescription() 247
16.30.1.12 getEntityIconFile() 247
16.30.1.13 getID() 247
16.30.1.14 getInstanceID() 247
16.30.1.15 getLinkedInstanceID() 247
16.30.1.16 getMessageHandler() 247
16.30.1.17 getName() 248
16.30.1.18 getRelations() 248
16.30.1.19 getSpecificRelations() 248
16.30.1.20 getType() 248
16.30.1.21 gotoLine() 248
16.30.1.22 hasRelation() 248
16.30.1.23 onFailed() 249
16.30.1.24 onFinished() 249
16.30.1.25 onRun() 249
16.30.1.26 onStop() 249
16.30.1.27 onUndeploy() 249
16.30.1.28 openEntityEditor() 249
16.30.1.29 postMessage() 249
16.30.1.30 Serialize() 249
16.30.1.31 setDescription() 249
16.30.1.32 setID() [1/2] 250
16.30.1.33 setID() [2/2] 250
16.30.1.34 setName() 250

16.30.1.35 stopEditor() . . . 250
16.31 cmas.ontology.agents.DeploymentAgent.DeploymentCriteria Enum Reference . . . 250
16.32 cmas.ontology.base.Entity Class Reference . . . 251
16.32.1 Constructor & Destructor Documentation . . . 252
16.32.1.1 Entity() [1/2] . . . 252
16.32.1.2 Entity() [2/2] . . . 253
16.32.2 Member Function Documentation . . . 253
16.32.2.1 copy() . . . 253
16.32.2.2 delete() . . . 253
16.32.2.3 DeSerialize() . . . 253
16.32.2.4 Factory() . . . 254
16.32.2.5 findRelationByName() . . . 254
16.32.2.6 findRelationByNameIgnoreCase() . . . 254
16.32.2.7 generateUniqueIDs() . . . 255
16.32.2.8 getAllRelations() . . . 255
16.32.2.9 getDescription() . . . 255
16.32.2.10 getID() . . . 255
16.32.2.11 getInstanceID() . . . 255
16.32.2.12 getLinkedInstanceID() . . . 255
16.32.2.13 getName() . . . 256
16.32.2.14 getRelations() . . . 256
16.32.2.15 getSpecificRelations() . . . 256
16.32.2.16 getType() . . . 256
16.32.2.17 hasRelation() . . . 256
16.32.2.18 Serialize() . . . 257
16.32.2.19 setDescription() . . . 257
16.32.2.20 setID() [1/2] . . . 257
16.32.2.21 setID() [2/2] . . . 257
16.32.2.22 setName() . . . 257
16.33 cmas.ontology.base.Entity.EntityType Enum Reference . . . 258
16.33.1 Detailed Description . . . 258
16.34 cmas.ontology.exceptions.ExceptionConstant Class Reference . . . 258
16.35 cmas.ontology.exceptions.ExceptionType Class Reference . . . 259
16.36 cmas.ontology.exceptions.ExceptionVariable Class Reference . . . 259
16.37 cmas.ontology.adapter.FILE.FileClient Class Reference . . . 260
16.37.1 Detailed Description . . . 260
16.37.2 Member Function Documentation . . . 260
16.37.2.1 close() . . . 260
16.37.2.2 createNew() . . . 260
16.37.2.3 getDefaultValue() . . . 261
16.37.2.4 getDescription() . . . 261
16.37.2.5 isValid() . . . 261

16.37.2.6 ReadData() . . . 261
16.37.2.7 setDebug() . . . 261
16.37.2.8 WriteData() [1/4] . . . 261
16.37.2.9 WriteData() [2/4] . . . 261
16.37.2.10 WriteData() [3/4] . . . 261
16.37.2.11 WriteData() [4/4] . . . 261
16.38 cmas.ontology.interfaces.Goal Class Reference . . . 262
16.38.1 Member Function Documentation . . . 264
16.38.1.1 addRequestToQueue() . . . 264
16.38.1.2 copy() . . . 265
16.38.1.3 delete() . . . 265
16.38.1.4 DeSerialize() . . . 265
16.38.1.5 Factory() . . . 265
16.38.1.6 findRelationByName() . . . 265
16.38.1.7 findRelationByNameIgnoreCase() . . . 266
16.38.1.8 generateUniqueIDs() . . . 266
16.38.1.9 getAllRelations() . . . 266
16.38.1.10 getCommunicative() . . . 266
16.38.1.11 getDescription() . . . 266
16.38.1.12 getEntityDescription() . . . 267
16.38.1.13 getEntityIconFile() . . . 267
16.38.1.14 getID() . . . 267
16.38.1.15 getInstanceID() . . . 267
16.38.1.16 getLinkedInstanceID() . . . 267
16.38.1.17 getMessageHandler() . . . 267
16.38.1.18 getName() . . . 267
16.38.1.19 getRelations() . . . 267
16.38.1.20 getSpecificRelations() . . . 268
16.38.1.21 getType() . . . 268
16.38.1.22 hasRelation() . . . 268
16.38.1.23 openEntityEditor() . . . 268
16.38.1.24 postMessage() . . . 268
16.38.1.25 Serialize() . . . 268
16.38.1.26 setDescription() . . . 269
16.38.1.27 setID() [1/2] . . . 269
16.38.1.28 setID() [2/2] . . . 269
16.38.1.29 setName() . . . 269
16.39 cmas.ontology.Hazard Class Reference . . . 270
16.39.1 Member Function Documentation . . . 272
16.39.1.1 copy() . . . 272
16.39.1.2 delete() . . . 272
16.39.1.3 DeSerialize() . . . 272

16.39.1.4 Factory() . . . 272
16.39.1.5 findRelationByName() . . . 273
16.39.1.6 findRelationByNameIgnoreCase() . . . 273
16.39.1.7 generateUniqueIDs() . . . 273
16.39.1.8 getAllRelations() . . . 273
16.39.1.9 getDescription() . . . 273
16.39.1.10 getEntityDescription() . . . 274
16.39.1.11 getEntityIconFile() . . . 274
16.39.1.12 getID() . . . 274
16.39.1.13 getInstanceID() . . . 274
16.39.1.14 getLinkedInstanceID() . . . 274
16.39.1.15 getName() . . . 274
16.39.1.16 getRelations() . . . 274
16.39.1.17 getSpecificRelations() . . . 275
16.39.1.18 getType() . . . 276
16.39.1.19 hasRelation() . . . 276
16.39.1.20 openEntityEditor() . . . 276
16.39.1.21 Serialize() . . . 276
16.39.1.22 setDescription() . . . 276
16.39.1.23 setID() [1/2] . . . 277
16.39.1.24 setID() [2/2] . . . 277
16.39.1.25 setName() . . . 277
16.40 cmas.ontology.action.IndependentClassLoader Class Reference . . . 277
16.40.1 Member Function Documentation . . . 278
16.40.1.1 getInstance() . . . 278
16.41 cmas.ontology.interfaces.Interface Class Reference . . . 278
16.41.1 Member Function Documentation . . . 280
16.41.1.1 addRequestToQueue() . . . 280
16.41.1.2 copy() . . . 281
16.41.1.3 delete() . . . 282
16.41.1.4 DeSerialize() . . . 282
16.41.1.5 Factory() . . . 282
16.41.1.6 findRelationByName() . . . 282
16.41.1.7 findRelationByNameIgnoreCase() . . . 283
16.41.1.8 generateUniqueIDs() . . . 283
16.41.1.9 getAllRelations() . . . 283
16.41.1.10 getCommunicative() . . . 283
16.41.1.11 getDescription() . . . 283
16.41.1.12 getEntityDescription() . . . 283
16.41.1.13 getEntityIconFile() . . . 284
16.41.1.14 getID() . . . 284
16.41.1.15 getInstanceID() . . . 284

16.41.1.16 getLinkedInstanceID() . . . 284
16.41.1.17 getMessageHandler() . . . 284
16.41.1.18 getName() . . . 284
16.41.1.19 getRelations() . . . 284
16.41.1.20 getSpecificRelations() . . . 285
16.41.1.21 getType() . . . 286
16.41.1.22 hasRelation() . . . 286
16.41.1.23 openEntityEditor() . . . 286
16.41.1.24 postMessage() . . . 286
16.41.1.25 Serialize() . . . 286
16.41.1.26 setDescription() . . . 286
16.41.1.27 setID() [1/2] . . . 287
16.41.1.28 setID() [2/2] . . . 287
16.41.1.29 setName() . . . 287
16.42 cmas.ontology.interfaces.InterfaceAssembly Class Reference . . . 287
16.42.1 Member Function Documentation . . . 290
16.42.1.1 addRequestToQueue() . . . 290
16.42.1.2 copy() . . . 290
16.42.1.3 delete() . . . 290
16.42.1.4 DeSerialize() . . . 290
16.42.1.5 Factory() . . . 290
16.42.1.6 findRelationByName() . . . 291
16.42.1.7 findRelationByNameIgnoreCase() . . . 291
16.42.1.8 generateUniqueIDs() . . . 292
16.42.1.9 getAllRelations() . . . 292
16.42.1.10 getCommunicative() . . . 292
16.42.1.11 getDescription() . . . 292
16.42.1.12 getEntityDescription() . . . 292
16.42.1.13 getEntityIconFile() . . . 292
16.42.1.14 getID() . . . 292
16.42.1.15 getInstanceID() . . . 292
16.42.1.16 getLinkedInstanceID() . . . 292
16.42.1.17 getMessageHandler() . . . 293
16.42.1.18 getName() . . . 293
16.42.1.19 getRelations() . . . 293
16.42.1.20 getSpecificRelations() . . . 293
16.42.1.21 getType() . . . 293
16.42.1.22 hasRelation() . . . 293
16.42.1.23 openEntityEditor() . . . 294
16.42.1.24 postMessage() . . . 294
16.42.1.25 Serialize() . . . 294
16.42.1.26 setDescription() . . . 294

16.42.1.27 setID() [1/2] . . . 294
16.42.1.28 setID() [2/2] . . . 294
16.42.1.29 setName() . . . 295
16.43 cmas.ontology.interfaces.InterfaceBase Class Reference . . . 295
16.43.1 Member Function Documentation . . . 297
16.43.1.1 addRequestToQueue() . . . 297
16.43.1.2 copy() . . . 298
16.43.1.3 delete() . . . 298
16.43.1.4 DeSerialize() . . . 298
16.43.1.5 Factory() . . . 298
16.43.1.6 findRelationByName() . . . 299
16.43.1.7 findRelationByNameIgnoreCase() . . . 299
16.43.1.8 generateUniqueIDs() . . . 299
16.43.1.9 getAllRelations() . . . 299
16.43.1.10 getCommunicative() . . . 300
16.43.1.11 getDescription() . . . 300
16.43.1.12 getID() . . . 300
16.43.1.13 getInstanceID() . . . 300
16.43.1.14 getLinkedInstanceID() . . . 300
16.43.1.15 getMessageHandler() . . . 300
16.43.1.16 getName() . . . 300
16.43.1.17 getRelations() . . . 300
16.43.1.18 getSpecificRelations() . . . 301
16.43.1.19 getType() . . . 301
16.43.1.20 hasRelation() . . . 301
16.43.1.21 postMessage() . . . 301
16.43.1.22 Serialize() . . . 301
16.43.1.23 setDescription() . . . 302
16.43.1.24 setID() [1/2] . . . 303
16.43.1.25 setID() [2/2] . . . 303
16.43.1.26 setName() . . . 303
16.44 cmas.ontology.interfaces.InterfaceLocation Class Reference . . . 303
16.44.1 Member Function Documentation . . . 306
16.44.1.1 addRequestToQueue() . . . 306
16.44.1.2 copy() . . . 306
16.44.1.3 delete() . . . 306
16.44.1.4 DeSerialize() . . . 306
16.44.1.5 Factory() . . . 306
16.44.1.6 findRelationByName() . . . 307
16.44.1.7 findRelationByNameIgnoreCase() . . . 307
16.44.1.8 generateUniqueIDs() . . . 308
16.44.1.9 getAllRelations() . . . 308

16.44.1.10 getCommunicative() . . . 308
16.44.1.11 getDescription() . . . 308
16.44.1.12 getEntityDescription() . . . 308
16.44.1.13 getEntityIconFile() . . . 308
16.44.1.14 getID() . . . 308
16.44.1.15 getInstanceID() . . . 308
16.44.1.16 getLinkedInstanceID() . . . 308
16.44.1.17 getMessageHandler() . . . 309
16.44.1.18 getName() . . . 309
16.44.1.19 getRelations() . . . 309
16.44.1.20 getSpecificRelations() . . . 309
16.44.1.21 getType() . . . 309
16.44.1.22 hasRelation() . . . 309
16.44.1.23 openEntityEditor() . . . 310
16.44.1.24 postMessage() . . . 310
16.44.1.25 Serialize() . . . 310
16.44.1.26 setDescription() . . . 310
16.44.1.27 setID() [1/2] . . . 310
16.44.1.28 setID() [2/2] . . . 310
16.44.1.29 setName() . . . 311
16.45 cmas.ontology.interfaces.InterfaceSpecialisation Class Reference . . . 311
16.45.1 Member Function Documentation . . . 313
16.45.1.1 addRequestToQueue() . . . 313
16.45.1.2 copy() . . . 314
16.45.1.3 delete() . . . 314
16.45.1.4 DeSerialize() . . . 314
16.45.1.5 Factory() . . . 314
16.45.1.6 findRelationByName() . . . 314
16.45.1.7 findRelationByNameIgnoreCase() . . . 315
16.45.1.8 generateUniqueIDs() . . . 315
16.45.1.9 getAllRelations() . . . 315
16.45.1.10 getCommunicative() . . . 315
16.45.1.11 getDescription() . . . 315
16.45.1.12 getEntityDescription() . . . 316
16.45.1.13 getEntityIconFile() . . . 316
16.45.1.14 getID() . . . 316
16.45.1.15 getInstanceID() . . . 316
16.45.1.16 getLinkedInstanceID() . . . 316
16.45.1.17 getMessageHandler() . . . 316
16.45.1.18 getName() . . . 316
16.45.1.19 getRelations() . . . 316
16.45.1.20 getSpecificRelations() . . . 317

16.45.1.21 getType() . . . 317
16.45.1.22 hasRelation() . . . 317
16.45.1.23 openEntityEditor() . . . 317
16.45.1.24 postMessage() . . . 317
16.45.1.25 Serialize() . . . 317
16.45.1.26 setDescription() . . . 318
16.45.1.27 setID() [1/2] . . . 318
16.45.1.28 setID() [2/2] . . . 318
16.45.1.29 setName() . . . 318
16.46 cmas.ontology.interfaces.InterfaceBase.InterfaceState Enum Reference . . . 318
16.47 cmas.ontology.interfaces.InterfaceTransport Class Reference . . . 319
16.47.1 Member Function Documentation . . . 321
16.47.1.1 addRequestToQueue() . . . 321
16.47.1.2 copy() . . . 321
16.47.1.3 delete() . . . 321
16.47.1.4 DeSerialize() . . . 322
16.47.1.5 Factory() . . . 322
16.47.1.6 findRelationByName() . . . 322
16.47.1.7 findRelationByNameIgnoreCase() . . . 322
16.47.1.8 generateUniqueIDs() . . . 324
16.47.1.9 getAllRelations() . . . 324
16.47.1.10 getCommunicative() . . . 324
16.47.1.11 getDescription() . . . 324
16.47.1.12 getEntityDescription() . . . 324
16.47.1.13 getEntityIconFile() . . . 324
16.47.1.14 getID() . . . 324
16.47.1.15 getInstanceID() . . . 325
16.47.1.16 getLinkedInstanceID() . . . 325
16.47.1.17 getMessageHandler() . . . 325
16.47.1.18 getName() . . . 325
16.47.1.19 getRelations() . . . 325
16.47.1.20 getSpecificRelations() . . . 325
16.47.1.21 getType() . . . 326
16.47.1.22 hasRelation() . . . 326
16.47.1.23 openEntityEditor() . . . 326
16.47.1.24 postMessage() . . . 326
16.47.1.25 Serialize() . . . 326
16.47.1.26 setDescription() . . . 326
16.47.1.27 setID() [1/2] . . . 326
16.47.1.28 setID() [2/2] . . . 327
16.47.1.29 setName() . . . 327
16.48 cmas.ontology.variables.Iterator Class Reference . . . 327

16.49 cmas.ontology.agents.Material Class Reference . . . 327
16.49.1 Member Function Documentation . . . 330
16.49.1.1 copy() . . . 330
16.49.1.2 delete() . . . 330
16.49.1.3 DeSerialize() . . . 330
16.49.1.4 Factory() . . . 330
16.49.1.5 findRelationByName() . . . 331
16.49.1.6 findRelationByNameIgnoreCase() . . . 331
16.49.1.7 generateUniqueIDs() . . . 331
16.49.1.8 getAllRelations() . . . 331
16.49.1.9 getCommunicative() . . . 331
16.49.1.10 getDescription() . . . 332
16.49.1.11 getEntityDescription() . . . 332
16.49.1.12 getEntityIconFile() . . . 332
16.49.1.13 getID() . . . 332
16.49.1.14 getInstanceID() . . . 332
16.49.1.15 getLinkedInstanceID() . . . 332
16.49.1.16 getMessageHandler() . . . 332
16.49.1.17 getName() . . . 332
16.49.1.18 getRelations() . . . 333
16.49.1.19 getSpecificRelations() . . . 333
16.49.1.20 getType() . . . 333
16.49.1.21 gotoLine() . . . 333
16.49.1.22 hasRelation() . . . 333
16.49.1.23 openEntityEditor() . . . 334
16.49.1.24 postMessage() . . . 334
16.49.1.25 Serialize() . . . 334
16.49.1.26 setDescription() . . . 334
16.49.1.27 setID() [1/2] . . . 334
16.49.1.28 setID() [2/2] . . . 334
16.49.1.29 setName() . . . 334
16.49.1.30 stopEditor() . . . 335
16.50 cmas.ontology.agentcommunication.ACL.MessageHandler Class Reference . . . 335
16.51 cmas.ontology.adapter.Modbus.ModbusClient.MODBUS_TYPE Enum Reference . . . 335
16.52 cmas.ontology.adapter.Modbus.ModbusClient Class Reference . . . 336
16.52.1 Member Function Documentation . . . 336
16.52.1.1 close() . . . 336
16.52.1.2 createNew() . . . 337
16.52.1.3 getDefaultValue() . . . 337
16.52.1.4 getDescription() . . . 337
16.52.1.5 isValid() . . . 337
16.52.1.6 ReadData() . . . 337

16.52.1.7 setDebug() . . . 337
16.52.1.8 WriteData() [1/4] . . . 337
16.52.1.9 WriteData() [2/4] . . . 337
16.52.1.10 WriteData() [3/4] . . . 337
16.52.1.11 WriteData() [4/4] . . . 338
16.53 cmas.ontology.Note Class Reference . . . 338
16.53.1 Member Function Documentation . . . 340
16.53.1.1 copy() . . . 340
16.53.1.2 delete() . . . 340
16.53.1.3 DeSerialize() . . . 340
16.53.1.4 Factory() . . . 340
16.53.1.5 findRelationByName() . . . 341
16.53.1.6 findRelationByNameIgnoreCase() . . . 341
16.53.1.7 generateUniqueIDs() . . . 341
16.53.1.8 getAllRelations() . . . 341
16.53.1.9 getDescription() . . . 342
16.53.1.10 getEntityDescription() . . . 342
16.53.1.11 getEntityIconFile() . . . 342
16.53.1.12 getID() . . . 342
16.53.1.13 getInstanceID() . . . 342
16.53.1.14 getLinkedInstanceID() . . . 342
16.53.1.15 getName() . . . 342
16.53.1.16 getRelations() . . . 342
16.53.1.17 getSpecificRelations() . . . 343
16.53.1.18 getType() . . . 343
16.53.1.19 hasRelation() . . . 343
16.53.1.20 openEntityEditor() . . . 343
16.53.1.21 Serialize() . . . 343
16.53.1.22 setDescription() . . . 343
16.53.1.23 setID() [1/2] . . . 345
16.53.1.24 setID() [2/2] . . . 345
16.53.1.25 setName() . . . 345
16.54 cmas.ontology.base.Ontology Class Reference . . . 345
16.55 cmas.ontology.base.Ontology.OntologyObject Class Reference . . . 345
16.56 cmas.ontology.base.Ontology.OntologyRelation Class Reference . . . 346
16.57 cmas.ontology.adapter.OPCUA.OPCUAClient Class Reference . . . 346
16.57.1 Member Function Documentation . . . 347
16.57.1.1 close() . . . 347
16.57.1.2 createNew() . . . 347
16.57.1.3 getDefaultValue() . . . 347
16.57.1.4 getDescription() . . . 347
16.57.1.5 getServerHost() . . . 347

16.57.1.6 getServerName() . . . 347
16.57.1.7 getServerPort() . . . 348
16.57.1.8 isValid() . . . 348
16.57.1.9 ReadData() . . . 348
16.57.1.10 setDebug() . . . 348
16.57.1.11 setServerHost() . . . 348
16.57.1.12 setServerName() . . . 348
16.57.1.13 setServerPort() . . . 348
16.57.1.14 WriteData() [1/4] . . . 349
16.57.1.15 WriteData() [2/4] . . . 349
16.57.1.16 WriteData() [3/4] . . . 349
16.57.1.17 WriteData() [4/4] . . . 349
16.58 cmas.ontology.adapter.OPCUA.OPCUAClientBase Class Reference . . . 350
16.58.1 Member Function Documentation . . . 351
16.58.1.1 getServerHost() . . . 351
16.58.1.2 getServerName() . . . 351
16.58.1.3 getServerPort() . . . 351
16.58.1.4 ReadData() . . . 351
16.58.1.5 setDebug() . . . 351
16.58.1.6 setServerHost() . . . 351
16.58.1.7 setServerName() . . . 351
16.58.1.8 setServerPort() . . . 352
16.58.1.9 WriteData() . . . 352
16.59 cmas.ontology.adapter.OPCUA.OPCUANode Class Reference . . . 352
16.60 cmas.ontology.agents.Part Class Reference . . . 352
16.60.1 Member Function Documentation . . . 355
16.60.1.1 copy() . . . 355
16.60.1.2 delete() . . . 355
16.60.1.3 DeSerialize() . . . 355
16.60.1.4 Factory() . . . 355
16.60.1.5 findRelationByName() . . . 355
16.60.1.6 findRelationByNameIgnoreCase() . . . 356
16.60.1.7 generateUniqueIDs() . . . 356
16.60.1.8 getAllRelations() . . . 356
16.60.1.9 getCommunicative() . . . 356
16.60.1.10 getDescription() . . . 356
16.60.1.11 getEntityDescription() . . . 357
16.60.1.12 getEntityIconFile() . . . 357
16.60.1.13 getID() . . . 357
16.60.1.14 getInstanceID() . . . 357
16.60.1.15 getLinkedInstanceID() . . . 357
16.60.1.16 getMessageHandler() . . . 357

16.60.1.17 getName() . . . 357
16.60.1.18 getRelations() . . . 357
16.60.1.19 getSpecificRelations() . . . 358
16.60.1.20 getType() . . . 358
16.60.1.21 gotoLine() . . . 358
16.60.1.22 hasRelation() . . . 358
16.60.1.23 onFailed() . . . 358
16.60.1.24 onFinished() . . . 358
16.60.1.25 onRun() . . . 359
16.60.1.26 onStop() . . . 359
16.60.1.27 onUndeploy() . . . 359
16.60.1.28 openEntityEditor() . . . 359
16.60.1.29 postMessage() . . . 359
16.60.1.30 Serialize() . . . 359
16.60.1.31 setDescription() . . . 359
16.60.1.32 setID() [1/2] . . . 359
16.60.1.33 setID() [2/2] . . . 359
16.60.1.34 setName() . . . 360
16.60.1.35 stopEditor() . . . 360
16.61 cmas.ontology.agentcommunication.ACL.ACLMessage.Performative Enum Reference . . . 360
16.62 cmas.ontology.processplan.ProcessPlan Class Reference . . . 360
16.62.1 Member Function Documentation . . . 362
16.62.1.1 copy() . . . 362
16.62.1.2 delete() . . . 363
16.62.1.3 DeSerialize() . . . 363
16.62.1.4 Factory() . . . 363
16.62.1.5 findRelationByName() . . . 363
16.62.1.6 findRelationByNameIgnoreCase() . . . 364
16.62.1.7 generateUniqueIDs() . . . 364
16.62.1.8 getAllRelations() . . . 364
16.62.1.9 getCommunicative() . . . 364
16.62.1.10 getDescription() . . . 364
16.62.1.11 getID() . . . 365
16.62.1.12 getInstanceID() . . . 365
16.62.1.13 getLinkedInstanceID() . . . 365
16.62.1.14 getMessageHandler() . . . 365
16.62.1.15 getName() . . . 365
16.62.1.16 getRelations() . . . 365
16.62.1.17 getSpecificRelations() . . . 365
16.62.1.18 getType() . . . 366
16.62.1.19 hasRelation() . . . 366
16.62.1.20 postMessage() . . . 366

16.62.1.21 run() . . . 366
16.62.1.22 Serialize() . . . 366
16.62.1.23 setDescription() . . . 367
16.62.1.24 setID() [1/2] . . . 368
16.62.1.25 setID() [2/2] . . . 368
16.62.1.26 setName() . . . 368
16.63 cmas.ontology.processplan.ProcessPlanGoals Class Reference . . . 368
16.63.1 Member Function Documentation . . . 370
16.63.1.1 abort() . . . 370
16.63.1.2 copy() . . . 370
16.63.1.3 delete() . . . 371
16.63.1.4 DeSerialize() . . . 371
16.63.1.5 execute() . . . 371
16.63.1.6 Factory() . . . 371
16.63.1.7 findRelationByName() . . . 371
16.63.1.8 findRelationByNameIgnoreCase() . . . 372
16.63.1.9 generateUniqueIDs() . . . 372
16.63.1.10 getAllRelations() . . . 372
16.63.1.11 getCommunicative() . . . 372
16.63.1.12 getDescription() . . . 372
16.63.1.13 getEntityDescription() . . . 372
16.63.1.14 getEntityIconFile() . . . 373
16.63.1.15 getID() . . . 373
16.63.1.16 getInstanceID() . . . 373
16.63.1.17 getLinkedInstanceID() . . . 373
16.63.1.18 getMessageHandler() . . . 373
16.63.1.19 getName() . . . 373
16.63.1.20 getRelations() . . . 373
16.63.1.21 getSpecificRelations() . . . 374
16.63.1.22 getType() . . . 375
16.63.1.23 hasRelation() . . . 375
16.63.1.24 onNegotiate() . . . 375
16.63.1.25 onOverlap() . . . 375
16.63.1.26 openEntityEditor() . . . 375
16.63.1.27 postMessage() . . . 376
16.63.1.28 resetAllAbstractInterfaces() . . . 376
16.63.1.29 run() . . . 376
16.63.1.30 Serialize() . . . 376
16.63.1.31 setDescription() . . . 376
16.63.1.32 setID() [1/2] . . . 376
16.63.1.33 setID() [2/2] . . . 376
16.63.1.34 setName() . . . 376

16.64 cmas.ontology.processplan.ProcessPlanInterface Interface Reference . . . 377
16.64.1 Member Function Documentation . . . 377
16.64.1.1 run() . . . 377
16.65 cmas.ontology.processplan.ProcessPlanJava Class Reference . . . 378
16.65.1 Member Function Documentation . . . 380
16.65.1.1 abort() . . . 380
16.65.1.2 copy() . . . 380
16.65.1.3 delete() . . . 380
16.65.1.4 DeSerialize() . . . 380
16.65.1.5 execute() . . . 380
16.65.1.6 Factory() . . . 380
16.65.1.7 findRelationByName() . . . 381
16.65.1.8 findRelationByNameIgnoreCase() . . . 381
16.65.1.9 generateUniqueIDs() . . . 381
16.65.1.10 getAllRelations() . . . 382
16.65.1.11 getCommunicative() . . . 382
16.65.1.12 getDescription() . . . 382
16.65.1.13 getEntityDescription() . . . 382
16.65.1.14 getEntityIconFile() . . . 382
16.65.1.15 getID() . . . 382
16.65.1.16 getInstanceID() . . . 382
16.65.1.17 getLinkedInstanceID() . . . 382
16.65.1.18 getMessageHandler() . . . 383
16.65.1.19 getName() . . . 383
16.65.1.20 getRelations() . . . 383
16.65.1.21 getSpecificRelations() . . . 383
16.65.1.22 getType() . . . 383
16.65.1.23 gotoLine() . . . 383
16.65.1.24 hasRelation() . . . 384
16.65.1.25 onNegotiate() . . . 385
16.65.1.26 onOverlap() . . . 385
16.65.1.27 openEntityEditor() . . . 385
16.65.1.28 postMessage() . . . 385
16.65.1.29 resetAllAbstractInterfaces() . . . 385
16.65.1.30 run() . . . 385
16.65.1.31 Serialize() . . . 385
16.65.1.32 setDescription() . . . 386
16.65.1.33 setID() [1/2] . . . 386
16.65.1.34 setID() [2/2] . . . 386
16.65.1.35 setName() . . . 386
16.65.1.36 stopEditor() . . . 386
16.66 cmas.ontology.processplan.ProcessPlan.ProcessPlanReturnValue Enum Reference . . . 386

16.67 cmas.ontology.processplan.ProcessPlanStructuredText Class Reference . . . 387
16.67.1 Member Function Documentation . . . 389
16.67.1.1 abort() . . . 389
16.67.1.2 copy() . . . 389
16.67.1.3 delete() . . . 389
16.67.1.4 DeSerialize() . . . 389
16.67.1.5 execute() . . . 390
16.67.1.6 Factory() . . . 390
16.67.1.7 findRelationByName() . . . 390
16.67.1.8 findRelationByNameIgnoreCase() . . . 390
16.67.1.9 generateUniqueIDs() . . . 391
16.67.1.10 getAllRelations() . . . 391
16.67.1.11 getCommunicative() . . . 391
16.67.1.12 getDescription() . . . 391
16.67.1.13 getEntityDescription() . . . 391
16.67.1.14 getEntityIconFile() . . . 391
16.67.1.15 getID() . . . 391
16.67.1.16 getInstanceID() . . . 392
16.67.1.17 getLinkedInstanceID() . . . 392
16.67.1.18 getMessageHandler() . . . 392
16.67.1.19 getName() . . . 392
16.67.1.20 getRelations() . . . 392
16.67.1.21 getSpecificRelations() . . . 392
16.67.1.22 getType() . . . 393
16.67.1.23 gotoLine() . . . 393
16.67.1.24 hasRelation() . . . 393
16.67.1.25 onNegotiate() . . . 393
16.67.1.26 onOverlap() . . . 393
16.67.1.27 openEntityEditor() . . . 393
16.67.1.28 postMessage() . . . 394
16.67.1.29 resetAllAbstractInterfaces() . . . 394
16.67.1.30 run() . . . 394
16.67.1.31 Serialize() . . . 394
16.67.1.32 setDescription() . . . 394
16.67.1.33 setID() [1/2] . . . 394
16.67.1.34 setID() [2/2] . . . 394
16.67.1.35 setName() . . . 395
16.67.1.36 stopEditor() . . . 395
16.68 cmas.ontology.processplan.ProcessPlan.ProcessPlanType Enum Reference . . . 395
16.69 cmas.ontology.base.Properties Class Reference . . . 395
16.69.1 Member Function Documentation . . . 396
16.69.1.1 getPropertiesList() . . . 396

16.70 cmas.ontology.base.Property< T > Class Template Reference . . . 396
16.71 cmas.ontology.variables.ValueRange.Range Class Reference . . . 397
16.72 cmas.ontology.base.Reference Class Reference . . . 398
16.72.1 Member Function Documentation . . . 400
16.72.1.1 copy() . . . 400
16.72.1.2 delete() . . . 400
16.72.1.3 DeSerialize() . . . 400
16.72.1.4 Factory() . . . 400
16.72.1.5 findRelationByName() . . . 401
16.72.1.6 findRelationByNameIgnoreCase() . . . 401
16.72.1.7 generateUniqueIDs() . . . 401
16.72.1.8 getAllRelations() . . . 401
16.72.1.9 getDescription() . . . 401
16.72.1.10 getEntityDescription() . . . 402
16.72.1.11 getEntityIconFile() . . . 402
16.72.1.12 getID() . . . 402
16.72.1.13 getInstanceID() . . . 402
16.72.1.14 getLinkedInstanceID() . . . 402
16.72.1.15 getName() . . . 402
16.72.1.16 getRelations() . . . 402
16.72.1.17 getSpecificRelations() . . . 402
16.72.1.18 getType() . . . 403
16.72.1.19 hasRelation() . . . 403
16.72.1.20 openEntityEditor() . . . 403
16.72.1.21 Serialize() . . . 403
16.72.1.22 setDescription() . . . 403
16.72.1.23 setID() [1/2] . . . 404
16.72.1.24 setID() [2/2] . . . 404
16.72.1.25 setName() . . . 404
16.73 cmas.ontology.base.Relations.RelationObject Class Reference . . . 404
16.74 cmas.ontology.base.Relations Class Reference . . . 405
16.75 cmas.ontology.workflow.Requirement Class Reference . . . 406
16.75.1 Member Function Documentation . . . 408
16.75.1.1 copy() . . . 408
16.75.1.2 delete() . . . 408
16.75.1.3 DeSerialize() . . . 408
16.75.1.4 Factory() . . . 408
16.75.1.5 findRelationByName() . . . 408
16.75.1.6 findRelationByNameIgnoreCase() . . . 409
16.75.1.7 generateUniqueIDs() . . . 409
16.75.1.8 getAllRelations() . . . 409
16.75.1.9 getDescription() . . . 409

16.75.1.10 getEntityDescription() . . . 409
16.75.1.11 getEntityIconFile() . . . 410
16.75.1.12 getID() . . . 410
16.75.1.13 getInstanceID() . . . 410
16.75.1.14 getLinkedInstanceID() . . . 410
16.75.1.15 getName() . . . 410
16.75.1.16 getRelations() . . . 410
16.75.1.17 getSpecificRelations() . . . 410
16.75.1.18 getType() . . . 411
16.75.1.19 hasRelation() . . . 411
16.75.1.20 openEntityEditor() . . . 411
16.75.1.21 Serialize() . . . 411
16.75.1.22 setDescription() . . . 411
16.75.1.23 setID() [1/2] . . . 412
16.75.1.24 setID() [2/2] . . . 412
16.75.1.25 setName() . . . 412
16.76 cmas.ontology.agents.Resource Class Reference . . . 412
16.76.1 Member Function Documentation . . . 415
16.76.1.1 copy() . . . 415
16.76.1.2 delete() . . . 416
16.76.1.3 DeSerialize() . . . 416
16.76.1.4 Factory() . . . 416
16.76.1.5 findRelationByName() . . . 416
16.76.1.6 findRelationByNameIgnoreCase() . . . 417
16.76.1.7 generateUniqueIDs() . . . 417
16.76.1.8 getAllRelations() . . . 417
16.76.1.9 getCommunicative() . . . 417
16.76.1.10 getDescription() . . . 417
16.76.1.11 getEntityDescription() . . . 417
16.76.1.12 getEntityIconFile() . . . 418
16.76.1.13 getID() . . . 418
16.76.1.14 getInstanceID() . . . 418
16.76.1.15 getLinkedInstanceID() . . . 418
16.76.1.16 getMessageHandler() . . . 418
16.76.1.17 getName() . . . 418
16.76.1.18 getRelations() . . . 418
16.76.1.19 getSpecificRelations() . . . 419
16.76.1.20 getType() . . . 420
16.76.1.21 gotoLine() . . . 420
16.76.1.22 hasRelation() . . . 420
16.76.1.23 onMessage() . . . 420
16.76.1.24 openEntityEditor() . . . 420

16.76.1.25 postMessage() . . . 420
16.76.1.26 Running() . . . 421
16.76.1.27 Serialize() . . . 421
16.76.1.28 setDescription() . . . 421
16.76.1.29 setID() [1/2] . . . 421
16.76.1.30 setID() [2/2] . . . 421
16.76.1.31 setName() . . . 421
16.76.1.32 stopEditor() . . . 421
16.77 cmas.ontology.adapter.REST.RESTClient.REST_TYPE Enum Reference . . . 422
16.78 cmas.ontology.adapter.REST.RESTClient Class Reference . . . 422
16.78.1 Detailed Description . . . 423
16.78.2 Member Function Documentation . . . 423
16.78.2.1 close() . . . 423
16.78.2.2 createNew() . . . 423
16.78.2.3 getDefaultValue() . . . 423
16.78.2.4 getDescription() . . . 423
16.78.2.5 isValid() . . . 423
16.78.2.6 ReadData() . . . 423
16.78.2.7 setDebug() . . . 423
16.78.2.8 WriteData() [1/4] . . . 424
16.78.2.9 WriteData() [2/4] . . . 424
16.78.2.10 WriteData() [3/4] . . . 424
16.78.2.11 WriteData() [4/4] . . . 424
16.79 cmas.ontology.adapter.socket.SocketTCPClient.SERVER_TYPE Enum Reference . . . 424
16.80 cmas.ontology.adapter.socket.SocketUDPClient.SERVER_TYPE Enum Reference . . . 424
16.81 cmas.ontology.Settings Class Reference . . . 425
16.81.1 Member Function Documentation . . . 427
16.81.1.1 copy() . . . 427
16.81.1.2 delete() . . . 428
16.81.1.3 DeSerialize() . . . 428
16.81.1.4 Factory() . . . 428
16.81.1.5 findRelationByName() . . . 428
16.81.1.6 findRelationByNameIgnoreCase() . . . 429
16.81.1.7 generateUniqueIDs() . . . 429
16.81.1.8 getAllRelations() . . . 429
16.81.1.9 getDescription() . . . 429
16.81.1.10 getEntityDescription() . . . 429
16.81.1.11 getEntityIconFile() . . . 429
16.81.1.12 getID() . . . 430
16.81.1.13 getInstanceID() . . . 430
16.81.1.14 getLinkedInstanceID() . . . 430
16.81.1.15 getName() . . . 430

16.81.1.16 getRelations() . . . 430
16.81.1.17 getSpecificRelations() . . . 430
16.81.1.18 getType() . . . 431
16.81.1.19 hasRelation() . . . 431
16.81.1.20 openEntityEditor() . . . 431
16.81.1.21 Serialize() . . . 431
16.81.1.22 setDescription() . . . 431
16.81.1.23 setID() [1/2] . . . 431
16.81.1.24 setID() [2/2] . . . 432
16.81.1.25 setName() . . . 433
16.82 cmas.ontology.processplan.SkillCall Class Reference . . . 433
16.82.1 Member Function Documentation . . . 435
16.82.1.1 copy() . . . 435
16.82.1.2 delete() . . . 435
16.82.1.3 DeSerialize() . . . 435
16.82.1.4 Factory() . . . 435
16.82.1.5 findRelationByName() . . . 436
16.82.1.6 findRelationByNameIgnoreCase() . . . 436
16.82.1.7 generateUniqueIDs() . . . 436
16.82.1.8 getAllRelations() . . . 437
16.82.1.9 getDescription() . . . 437
16.82.1.10 getEntityDescription() . . . 437
16.82.1.11 getEntityIconFile() . . . 437
16.82.1.12 getID() . . . 437
16.82.1.13 getInstanceID() . . . 437
16.82.1.14 getLinkedInstanceID() . . . 437
16.82.1.15 getName() . . . 437
16.82.1.16 getRelations() . . . 438
16.82.1.17 getSpecificRelations() . . . 438
16.82.1.18 getType() . . . 438
16.82.1.19 hasRelation() . . . 438
16.82.1.20 openEntityEditor() . . . 438
16.82.1.21 Serialize() . . . 439
16.82.1.22 setDescription() . . . 439
16.82.1.23 setID() [1/2] . . . 439
16.82.1.24 setID() [2/2] . . . 439
16.82.1.25 setName() . . . 439
16.83 cmas.ontology.adapter.socket.SocketTCPClient Class Reference . . . 440
16.83.1 Member Function Documentation . . . 440
16.83.1.1 close() . . . 440
16.83.1.2 createNew() . . . 441
16.83.1.3 getDefaultValue() . . . 441

16.83.1.4 getDescription() . . . 441
16.83.1.5 isValid() . . . 441
16.83.1.6 ReadData() . . . 441
16.83.1.7 setDebug() . . . 441
16.83.1.8 WriteData() [1/4] . . . 441
16.83.1.9 WriteData() [2/4] . . . 441
16.83.1.10 WriteData() [3/4] . . . 441
16.83.1.11 WriteData() [4/4] . . . 442
16.84 cmas.ontology.adapter.socket.SocketUDPClient Class Reference . . . 442
16.84.1 Member Function Documentation . . . 443
16.84.1.1 close() . . . 443
16.84.1.2 createNew() . . . 443
16.84.1.3 getDefaultValue() . . . 443
16.84.1.4 getDescription() . . . 443
16.84.1.5 isValid() . . . 443
16.84.1.6 ReadData() . . . 443
16.84.1.7 setDebug() . . . 443
16.84.1.8 WriteData() [1/4] . . . 443
16.84.1.9 WriteData() [2/4] . . . 443
16.84.1.10 WriteData() [3/4] . . . 444
16.84.1.11 WriteData() [4/4] . . . 444
16.85 cmas.ontology.base.Property< T >.SpecificPropertyType Enum Reference . . . 444
16.86 cmas.ontology.workflow.WorkFlow.SpecificTypes Enum Reference . . . 444
16.87 cmas.ontology.adapter.SQL.SQLClient Class Reference . . . 445
16.87.1 Detailed Description . . . 445
16.87.2 Member Function Documentation . . . 445
16.87.2.1 close() . . . 445
16.87.2.2 createNew() . . . 445
16.87.2.3 getDefaultValue() . . . 445
16.87.2.4 getDescription() . . . 446
16.87.2.5 isValid() . . . 446
16.87.2.6 ReadData() . . . 446
16.87.2.7 setDebug() . . . 446
16.87.2.8 WriteData() [1/4] . . . 446
16.87.2.9 WriteData() [2/4] . . . 446
16.87.2.10 WriteData() [3/4] . . . 446
16.87.2.11 WriteData() [4/4] . . . 446
16.88 cmas.ontology.variables.Variable.TypeOfVariable Enum Reference . . . 447
16.89 cmas.ontology.Types Class Reference . . . 447
16.89.1 Member Data Documentation . . . 447
16.89.1.1 AgentTypes . . . 447
16.89.1.2 BatchTypes . . . 447

16.89.1.3 DeploymentTypes . . . 448
16.89.1.4 EntityTypes . . . 448
16.89.1.5 FolderTypes . . . 448
16.89.1.6 GoalTypes . . . 448
16.89.1.7 HazardTypes . . . 449
16.89.1.8 InterfaceTypes . . . 449
16.89.1.9 ProcessPlanTypes . . . 449
16.89.1.10 RequirementTypes . . . 449
16.89.1.11 SkillTypes . . . 449
16.89.1.12 ValueTypes . . . 449
16.89.1.13 WorkFlowTypes . . . 450
16.90 cmas.ontology.base.Entity.UID Class Reference . . . 450
16.91 cmas.ontology.variables.Value Class Reference . . . 450
16.91.1 Member Function Documentation . . . 453
16.91.1.1 copy() . . . 453
16.91.1.2 delete() . . . 454
16.91.1.3 DeSerialize() . . . 454
16.91.1.4 Factory() . . . 454
16.91.1.5 findRelationByName() . . . 454
16.91.1.6 findRelationByNameIgnoreCase() . . . 455
16.91.1.7 generateUniqueIDs() . . . 455
16.91.1.8 getAllRelations() . . . 455
16.91.1.9 getCopyOfValue() . . . 455
16.91.1.10 getDescription() . . . 455
16.91.1.11 getID() . . . 455
16.91.1.12 getInstanceID() . . . 456
16.91.1.13 getLinkedInstanceID() . . . 456
16.91.1.14 getName() . . . 456
16.91.1.15 getRelations() . . . 456
16.91.1.16 getSpecificRelations() . . . 456
16.91.1.17 getStringValue() . . . 456
16.91.1.18 getType() . . . 457
16.91.1.19 getValue() . . . 457
16.91.1.20 hasRelation() . . . 457
16.91.1.21 openEntityEditor() . . . 457
16.91.1.22 Serialize() . . . 457
16.91.1.23 setDescription() . . . 457
16.91.1.24 setID() [1/2] . . . 457
16.91.1.25 setID() [2/2] . . . 458
16.91.1.26 setName() . . . 458
16.91.1.27 setValue() . . . 458
16.91.1.28 setValueUndoable() . . . 458

16.91.1.29 valueOf() . . . 458
16.92 cmas.ontology.variables.ValueRange Class Reference . . . 458
16.93 cmas.ontology.variables.Value.ValueTypes Enum Reference . . . 459
16.94 cmas.ontology.variables.Variable Class Reference . . . 459
16.94.1 Member Function Documentation . . . 461
16.94.1.1 copy() . . . 461
16.94.1.2 delete() . . . 462
16.94.1.3 DeSerialize() . . . 462
16.94.1.4 Factory() . . . 462
16.94.1.5 findRelationByName() . . . 462
16.94.1.6 findRelationByNameIgnoreCase() . . . 463
16.94.1.7 generateUniqueIDs() . . . 463
16.94.1.8 getAllRelations() . . . 463
16.94.1.9 getDescription() . . . 463
16.94.1.10 getID() . . . 463
16.94.1.11 getInstanceID() . . . 464
16.94.1.12 getLinkedInstanceID() . . . 464
16.94.1.13 getName() . . . 464
16.94.1.14 getRelations() . . . 464
16.94.1.15 getSpecificRelations() . . . 464
16.94.1.16 getType() . . . 464
16.94.1.17 hasRelation() . . . 465
16.94.1.18 openEntityEditor() . . . 465
16.94.1.19 Serialize() . . . 465
16.94.1.20 setDescription() . . . 465
16.94.1.21 setID() [1/2] . . . 465
16.94.1.22 setID() [2/2] . . . 465
16.94.1.23 setName() . . . 466
16.95 cmas.ontology.variables.VariableArray Class Reference . . . 466
16.95.1 Member Function Documentation . . . 469
16.95.1.1 copy() . . . 469
16.95.1.2 delete() . . . 469
16.95.1.3 DeSerialize() . . . 469
16.95.1.4 Factory() . . . 469
16.95.1.5 findRelationByName() . . . 470
16.95.1.6 findRelationByNameIgnoreCase() . . . 470
16.95.1.7 generateUniqueIDs() . . . 470
16.95.1.8 getAllRelations() . . . 471
16.95.1.9 getCopyOfValue() . . . 471
16.95.1.10 getDescription() . . . 471
16.95.1.11 getEntityDescription() . . . 471
16.95.1.12 getEntityIconFile() . . . 471

16.95.1.13 getID() . . . 471
16.95.1.14 getInstanceID() . . . 471
16.95.1.15 getIterator() . . . 471
16.95.1.16 getIteratorObject() . . . 471
16.95.1.17 getLinkedInstanceID() . . . 472
16.95.1.18 getName() . . . 472
16.95.1.19 getNext() . . . 472
16.95.1.20 getRelations() . . . 472
16.95.1.21 getSpecificRelations() . . . 472
16.95.1.22 getStringValue() . . . 472
16.95.1.23 getType() . . . 473
16.95.1.24 getValue() . . . 473
16.95.1.25 hasNext() . . . 473
16.95.1.26 hasRelation() . . . 473
16.95.1.27 openEntityEditor() . . . 473
16.95.1.28 Serialize() . . . 473
16.95.1.29 setDescription() . . . 473
16.95.1.30 setID() [1/2] . . . 474
16.95.1.31 setID() [2/2] . . . 474
16.95.1.32 setName() . . . 474
16.95.1.33 setValue() . . . 474
16.95.1.34 setValueUndoable() . . . 474
16.95.1.35 valueOf() . . . 474
16.96 cmas.ontology.variables.VariableBoolean Class Reference . . . 475
16.96.1 Member Function Documentation . . . 478
16.96.1.1 copy() . . . 478
16.96.1.2 delete() . . . 478
16.96.1.3 DeSerialize() . . . 478
16.96.1.4 Factory() . . . 479
16.96.1.5 findRelationByName() . . . 479
16.96.1.6 findRelationByNameIgnoreCase() . . . 479
16.96.1.7 generateUniqueIDs() . . . 480
16.96.1.8 getAllRelations() . . . 480
16.96.1.9 getCopyOfValue() . . . 480
16.96.1.10 getDescription() . . . 480
16.96.1.11 getEntityDescription() . . . 480
16.96.1.12 getEntityIconFile() . . . 480
16.96.1.13 getID() . . . 480
16.96.1.14 getInstanceID() . . . 481
16.96.1.15 getIterator() . . . 481
16.96.1.16 getLinkedInstanceID() . . . 481
16.96.1.17 getName() . . . 481

16.96.1.18 getRelations() . . . 481
16.96.1.19 getSpecificRelations() . . . 481
16.96.1.20 getStringValue() . . . 482
16.96.1.21 getType() . . . 482
16.96.1.22 getValue() . . . 482
16.96.1.23 hasRelation() . . . 482
16.96.1.24 openEntityEditor() . . . 482
16.96.1.25 Serialize() . . . 482
16.96.1.26 setDescription() . . . 482
16.96.1.27 setID() [1/2] . . . 483
16.96.1.28 setID() [2/2] . . . 483
16.96.1.29 setName() . . . 483
16.96.1.30 setValue() . . . 483
16.96.1.31 setValueUndoable() . . . 483
16.96.1.32 valueOf() . . . 483
16.97 cmas.ontology.variables.VariableInteger Class Reference . . . 484
16.97.1 Member Function Documentation . . . 487
16.97.1.1 copy() . . . 487
16.97.1.2 delete() . . . 487
16.97.1.3 DeSerialize() . . . 487
16.97.1.4 Factory() . . . 488
16.97.1.5 findRelationByName() . . . 488
16.97.1.6 findRelationByNameIgnoreCase() . . . 488
16.97.1.7 generateUniqueIDs() . . . 489
16.97.1.8 getAllRelations() . . . 489
16.97.1.9 getCopyOfValue() . . . 489
16.97.1.10 getDescription() . . . 489
16.97.1.11 getEntityDescription() . . . 489
16.97.1.12 getEntityIconFile() . . . 489
16.97.1.13 getID() . . . 489
16.97.1.14 getInstanceID() . . . 490
16.97.1.15 getIterator() . . . 490
16.97.1.16 getLinkedInstanceID() . . . 490
16.97.1.17 getName() . . . 490
16.97.1.18 getRelations() . . . 490
16.97.1.19 getSpecificRelations() . . . 490
16.97.1.20 getStringValue() . . . 491
16.97.1.21 getType() . . . 491
16.97.1.22 getValue() . . . 491
16.97.1.23 hasRelation() . . . 491
16.97.1.24 openEntityEditor() . . . 491
16.97.1.25 Serialize() . . . 491

16.97.1.26 setDescription() . . . 491
16.97.1.27 setID() [1/2] . . . 492
16.97.1.28 setID() [2/2] . . . 492
16.97.1.29 setName() . . . 492
16.97.1.30 setValue() . . . 492
16.97.1.31 setValueUndoable() . . . 492
16.97.1.32 valueOf() . . . 492
16.98 cmas.ontology.interfaces.InterfaceBase.VariableMatchingType Enum Reference . . . 493
16.99 cmas.ontology.variables.VariableObject Class Reference . . . 493
16.99.1 Member Function Documentation . . . 496
16.99.1.1 copy() . . . 496
16.99.1.2 delete() . . . 496
16.99.1.3 DeSerialize() . . . 496
16.99.1.4 Factory() . . . 496
16.99.1.5 findRelationByName() . . . 497
16.99.1.6 findRelationByNameIgnoreCase() . . . 497
16.99.1.7 generateUniqueIDs() . . . 497
16.99.1.8 getAllRelations() . . . 497
16.99.1.9 getCopyOfValue() . . . 498
16.99.1.10 getDescription() . . . 498
16.99.1.11 getEntityDescription() . . . 498
16.99.1.12 getEntityIconFile() . . . 498
16.99.1.13 getID() . . . 498
16.99.1.14 getInstanceID() . . . 498
16.99.1.15 getIterator() . . . 498
16.99.1.16 getIteratorObject() . . . 498
16.99.1.17 getLinkedInstanceID() . . . 499
16.99.1.18 getName() . . . 499
16.99.1.19 getNext() . . . 499
16.99.1.20 getRelations() . . . 499
16.99.1.21 getSpecificRelations() . . . 499
16.99.1.22 getStringValue() . . . 499
16.99.1.23 getType() . . . 500
16.99.1.24 getValue() . . . 500
16.99.1.25 hasNext() . . . 500
16.99.1.26 hasRelation() . . . 500
16.99.1.27 makeInstance() . . . 500
16.99.1.28 openEntityEditor() . . . 500
16.99.1.29 Serialize() . . . 500
16.99.1.30 setDescription() . . . 500
16.99.1.31 setID() [1/2] . . . 501
16.99.1.32 setID() [2/2] . . . 501

16.99.1.33 setName() . . . 501
16.99.1.34 setValue() . . . 501
16.99.1.35 setValueUndoable() . . . 501
16.99.1.36 valueOf() . . . 501
16.100 cmas.ontology.variables.VariableProperty Class Reference . . . 502
16.100.1 Member Function Documentation . . . 505
16.100.1.1 copy() . . . 505
16.100.1.2 delete() . . . 505
16.100.1.3 DeSerialize() . . . 505
16.100.1.4 Factory() . . . 506
16.100.1.5 findRelationByName() . . . 506
16.100.1.6 findRelationByNameIgnoreCase() . . . 506
16.100.1.7 generateUniqueIDs() . . . 507
16.100.1.8 getAllRelations() . . . 507
16.100.1.9 getCopyOfValue() . . . 507
16.100.1.10 getDescription() . . . 507
16.100.1.11 getEntityDescription() . . . 507
16.100.1.12 getEntityIconFile() . . . 507
16.100.1.13 getID() . . . 507
16.100.1.14 getInstanceID() . . . 508
16.100.1.15 getIterator() . . . 508
16.100.1.16 getLinkedInstanceID() . . . 508
16.100.1.17 getName() . . . 508
16.100.1.18 getRelations() . . . 508
16.100.1.19 getSpecificRelations() . . . 508
16.100.1.20 getStringValue() . . . 509
16.100.1.21 getType() . . . 509
16.100.1.22 getValue() . . . 509
16.100.1.23 hasRelation() . . . 509
16.100.1.24 openEntityEditor() . . . 509
16.100.1.25 Serialize() . . . 509
16.100.1.26 setDescription() . . . 509
16.100.1.27 setID() [1/2] . . . 510
16.100.1.28 setID() [2/2] . . . 510
16.100.1.29 setName() . . . 510
16.100.1.30 setValue() . . . 510
16.100.1.31 setValueUndoable() . . . 510
16.100.1.32 valueOf() . . . 510
16.101 cmas.ontology.variables.VariableReal Class Reference . . . 511
16.101.1 Member Function Documentation . . . 514
16.101.1.1 copy() . . . 514
16.101.1.2 delete() . . . 514

16.101.1.3 DeSerialize() . . . 514
16.101.1.4 Factory() . . . 515
16.101.1.5 findRelationByName() . . . 515
16.101.1.6 findRelationByNameIgnoreCase() . . . 515
16.101.1.7 generateUniqueIDs() . . . 516
16.101.1.8 getAllRelations() . . . 516
16.101.1.9 getCopyOfValue() . . . 516
16.101.1.10 getDescription() . . . 516
16.101.1.11 getEntityDescription() . . . 516
16.101.1.12 getEntityIconFile() . . . 516
16.101.1.13 getID() . . . 516
16.101.1.14 getInstanceID() . . . 517
16.101.1.15 getIterator() . . . 517
16.101.1.16 getLinkedInstanceID() . . . 517
16.101.1.17 getName() . . . 517
16.101.1.18 getRelations() . . . 517
16.101.1.19 getSpecificRelations() . . . 517
16.101.1.20 getStringValue() . . . 518
16.101.1.21 getType() . . . 518
16.101.1.22 getValue() . . . 518
16.101.1.23 hasRelation() . . . 518
16.101.1.24 openEntityEditor() . . . 518
16.101.1.25 Serialize() . . . 518
16.101.1.26 setDescription() . . . 518
16.101.1.27 setID() [1/2] . . . 519
16.101.1.28 setID() [2/2] . . . 519
16.101.1.29 setName() . . . 519
16.101.1.30 setValue() . . . 519
16.101.1.31 setValueUndoable() . . . 519
16.101.1.32 valueOf() . . . 519
16.102 cmas.ontology.variables.VariableReference Class Reference . . . 520
16.102.1 Member Function Documentation . . . 523
16.102.1.1 copy() . . . 523
16.102.1.2 delete() . . . 523
16.102.1.3 DeSerialize() . . . 523
16.102.1.4 Factory() . . . 523
16.102.1.5 findRelationByName() . . . 523
16.102.1.6 findRelationByNameIgnoreCase() . . . 524
16.102.1.7 generateUniqueIDs() . . . 524
16.102.1.8 getAllRelations() . . . 524
16.102.1.9 getCopyOfValue() . . . 524
16.102.1.10 getDescription() . . . 524

16.102.1.11 getEntityDescription() . . . 525
16.102.1.12 getEntityIconFile() . . . 525
16.102.1.13 getID() . . . 525
16.102.1.14 getInstanceID() . . . 525
16.102.1.15 getIterator() . . . 525
16.102.1.16 getIteratorObject() . . . 525
16.102.1.17 getJSONStringValue() . . . 525
16.102.1.18 getLinkedInstanceID() . . . 525
16.102.1.19 getName() . . . 525
16.102.1.20 getNext() . . . 526
16.102.1.21 getRelations() . . . 526
16.102.1.22 getSize() . . . 526
16.102.1.23 getSpecificRelations() . . . 526
16.102.1.24 getStringValue() . . . 526
16.102.1.25 getType() . . . 526
16.102.1.26 getValue() . . . 526
16.102.1.27 hasNext() . . . 526
16.102.1.28 hasRelation() . . . 527
16.102.1.29 makeInstance() . . . 527
16.102.1.30 openEntityEditor() . . . 527
16.102.1.31 Serialize() . . . 527
16.102.1.32 setDescription() . . . 527
16.102.1.33 setID() [1/2] . . . 527
16.102.1.34 setID() [2/2] . . . 527
16.102.1.35 setName() . . . 528
16.102.1.36 setValue() . . . 528
16.102.1.37 setValueUndoable() . . . 528
16.102.1.38 valueOf() . . . 528
16.103 cmas.ontology.variables.VariableString Class Reference . . . 529
16.103.1 Member Function Documentation . . . 532
16.103.1.1 copy() . . . 532
16.103.1.2 delete() . . . 532
16.103.1.3 DeSerialize() . . . 532
16.103.1.4 Factory() . . . 533
16.103.1.5 findRelationByName() . . . 533
16.103.1.6 findRelationByNameIgnoreCase() . . . 533
16.103.1.7 generateUniqueIDs() . . . 534
16.103.1.8 getAllRelations() . . . 534
16.103.1.9 getCopyOfValue() . . . 534
16.103.1.10 getDescription() . . . 534
16.103.1.11 getEntityDescription() . . . 534
16.103.1.12 getEntityIconFile() . . . 534

16.103.1.13 getID() . . . 534
16.103.1.14 getInstanceID() . . . 535
16.103.1.15 getIterator() . . . 535
16.103.1.16 getIteratorObject() . . . 535
16.103.1.17 getLinkedInstanceID() . . . 535
16.103.1.18 getName() . . . 535
16.103.1.19 getNext() . . . 535
16.103.1.20 getRelations() . . . 535
16.103.1.21 getSpecificRelations() . . . 535
16.103.1.22 getStringValue() . . . 536
16.103.1.23 getType() . . . 536
16.103.1.24 getValue() . . . 536
16.103.1.25 hasNext() . . . 536
16.103.1.26 hasRelation() . . . 536
16.103.1.27 openEntityEditor() . . . 536
16.103.1.28 Serialize() . . . 537
16.103.1.29 setDescription() . . . 537
16.103.1.30 setID() [1/2] . . . 537
16.103.1.31 setID() [2/2] . . . 537
16.103.1.32 setName() . . . 537
16.103.1.33 setValue() . . . 537
16.103.1.34 setValueUndoable() . . . 537
16.103.1.35 valueOf() . . . 538
16.104 cmas.ontology.base.Visualisation Class Reference . . . 538
16.104.1 Member Function Documentation . . . 539
16.104.1.1 getLinkedInstanceID() . . . 539
16.105 cmas.ontology.adapter.websocket.WebSocketClient Class Reference . . . 539
16.105.1 Member Function Documentation . . . 540
16.105.1.1 close() . . . 540
16.105.1.2 createNew() . . . 540
16.105.1.3 getDefaultValue() . . . 540
16.105.1.4 getDescription() . . . 540
16.105.1.5 isValid() . . . 540
16.105.1.6 ReadData() . . . 540
16.105.1.7 setDebug() . . . 540
16.105.1.8 WriteData() [1/4] . . . 541
16.105.1.9 WriteData() [2/4] . . . 541
16.105.1.10 WriteData() [3/4] . . . 541
16.105.1.11 WriteData() [4/4] . . . 541
16.106 cmas.ontology.workflow.WorkFlow Class Reference . . . 542
16.106.1 Member Function Documentation . . . 544
16.106.1.1 copy() . . . 544

16.106.1.2 delete() . . . 544
16.106.1.3 DeSerialize() . . . 544
16.106.1.4 Factory() . . . 544
16.106.1.5 findRelationByName() . . . 545
16.106.1.6 findRelationByNameIgnoreCase() . . . 545
16.106.1.7 generateUniqueIDs() . . . 545
16.106.1.8 getAllRelations() . . . 545
16.106.1.9 getDescription() . . . 546
16.106.1.10 getID() . . . 546
16.106.1.11 getInstanceID() . . . 546
16.106.1.12 getLinkedInstanceID() . . . 546
16.106.1.13 getName() . . . 546
16.106.1.14 getRelations() . . . 546
16.106.1.15 getSpecificRelations() . . . 546
16.106.1.16 getType() . . . 547
16.106.1.17 hasRelation() . . . 547
16.106.1.18 Serialize() . . . 547
16.106.1.19 setDescription() . . . 547
16.106.1.20 setID() [1/2] . . . 547
16.106.1.21 setID() [2/2] . . . 548
16.106.1.22 setName() . . . 548
16.107 cmas.ontology.workflow.WorkFlowLog Class Reference . . . 549
16.107.1 Member Function Documentation . . . 551
16.107.1.1 copy() . . . 551
16.107.1.2 delete() . . . 551
16.107.1.3 DeSerialize() . . . 551
16.107.1.4 Factory() . . . 551
16.107.1.5 findRelationByName() . . . 552
16.107.1.6 findRelationByNameIgnoreCase() . . . 552
16.107.1.7 generateUniqueIDs() . . . 552
16.107.1.8 getAllRelations() . . . 552
16.107.1.9 getDescription() . . . 552
16.107.1.10 getEntityDescription() . . . 553
16.107.1.11 getEntityIconFile() . . . 553
16.107.1.12 getID() . . . 553
16.107.1.13 getInstanceID() . . . 553
16.107.1.14 getLinkedInstanceID() . . . 553
16.107.1.15 getName() . . . 553
16.107.1.16 getRelations() . . . 553
16.107.1.17 getSpecificRelations() . . . 554
16.107.1.18 getType() . . . 555
16.107.1.19 hasRelation() . . . 555

16.107.1.20 openEntityEditor() . . . 555
16.107.1.21 Serialize() . . . 555
16.107.1.22 setDescription() . . . 555
16.107.1.23 setID() [1/2] . . . 556
16.107.1.24 setID() [2/2] . . . 556
16.107.1.25 setName() . . . 556
16.107.1.26 setTime() . . . 556
16.108 cmas.ontology.workflow.WorkFlow.WorkFlowTypes Enum Reference . . . 556
16.109 cmas.ontology.adapter.ZeroMQ.ZMQClient Class Reference . . . 557
16.109.1 Detailed Description . . . 557
16.109.2 Member Function Documentation . . . 558
16.109.2.1 close() . . . 558
16.109.2.2 createNew() . . . 558
16.109.2.3 getDefaultValue() . . . 558
16.109.2.4 getDescription() . . . 558
16.109.2.5 isValid() . . . 558
16.109.2.6 ReadData() . . . 558
16.109.2.7 setDebug() . . . 558
16.109.2.8 WriteData() [1/4] . . . 558
16.109.2.9 WriteData() [2/4] . . . 558
16.109.2.10 WriteData() [3/4] . . . 559
16.109.2.11 WriteData() [4/4] . . . 559

Index **561**

Chapter 1

CMAS Main Page

1.1 Introduction

The CMAS from University West is a software package for Multiagent control (MAS) to facilitate the implementation of Plug&Produce concepts. Plug&Produce refer to autonomous manufacturing systems that can handle changes in both hardware and requirements online. CMAS is an acronym for Configurable Multiagent System, and its fundamental characteristic is its capacity to configure the system's overall behaviour rather than traditional programming approaches.

The work is based on efforts, many years of research and knowledge at University West in the Flexible Industrial Automation research group.

This manual is divided in the following sections:

- Introduction to Plug&Produce and Multiagent
- How to use CMAS
- Examples
- Process plans
- Interfaces

- Agent actions
- Variables
- Planner
- Automating deployment
- Structured text

- Goal charts
- CMAS ontology

1.2 Plug & Produce and Multiagent

1.2.1 CMAS

CMAS is a framework for Plug & Produce that makes it possible to use configurations rather than programming to model and handle parts in a manufacturing system. The key tasks are to define goals for parts. Goals are central in the design of autonomous multi-agent systems. The final goal for a part is to become manufactured and this can be divided into several subgoals. A goal can achieve its completion by requiesting skills from available resources. Therefore, the task of defining skills for resources is important. Typically, a resource skill is established upon resource acquisition and tends to remain relatively constant throughout the resource's lifespan. It's worth noting that a skill might demand additional control logic to function effectively, hence, it is possible to develop more conventional programs for skill execution as well. Typically, before the first deployment the system integrators will carrying out the definition of low-level skills.

Goal-oriented multi-agent control is one approach to handle flexible and reconfigurable manufacturing systems. Goals are important to be able to create autonomous agents. An autonomous agent must have the possibility to decide not only how to achieve a goal, but also which goal to select. For parts that will be manufactured, a state-oriented model is adopted implying that each agent associated with a part tries to reach a final state by executing all assigned goals. When all goals are finalized, the part is ready. A part is defined as a part of a product or the product itself that will be manufactured within the boundary of the actual manufacturing system. Hence, a goal can only be fulfilled by utilizing skills on resources within this boundary. To assist part-agents to reach their goals, process plans exist that defines skills needed to reach a specific goal. Process plans can be define without specifying resources, only required skills, requirements, and abstract interfaces are specified. Abstract interfaces are connected to a resource-agent when needed during runtime, the resource-agent are selected after a negotiation process. Hence, the process plans of goals separate the resources from the goals. This makes it possible to have several potential solutions in the system for the autonomous agents to select the most suitable process plans. A goal depicts the future desired state of the system. However, the sequence of goals is important, methods to configure and execute concurrent and parallel sequence of goals. Plug & Produce systems, controlled by CMAS, enables the possibility to rapidly plug in and out resources to adapt to the actual manufacturing situation. Resources prepared with suitable skills and process plans can be plugged in without doing any software reconfiguration of the CMAS. Parallel goals can utilize resources concurrently in parallel. If only one resource is available, goals that are configured to execute in parallel will be executed in sequence instead by the single resource. This approach enables the possibility to rapidly avoid bottlenecks in the production that can appear due to for example changed customer orders just by plugging in suitable resources. No reprogramming or software reconfiguration is needed in these situations.

1.2.2 Overview of CMAS execution model

The CMAS execution model separates *what should be achieved* from *how it is executed*.
Instead of programming specific machines or robots directly, programs describe goals and required capabilities. The system then determines how to achieve them using the available resources.

This allows programs to remain independent of specific hardware while the system dynamically allocates suitable resources.

The main concepts of the CMAS execution model are:

- Goals
- Skills
- Interfaces
- Resources
- Planner Agent

1.2.3 Goals

A Goal represents a desired state or result that should be achieved in the system.

A goal does not describe how the task should be executed. Instead, it defines *what outcome is required*. The Planner Agent is responsible for determining how the goal can be achieved using the available skills and resources. Goals are used on Part Agents.

1.2.4 Skills

A Skill represents an executable operation provided by a resource agent.
Examples of skills could be:

- Move
- Pick
- Place
- Weld
- Inspect

Skills are typically implemented by agents representing machines, robots, or software components in the system.

1.2.5 Interfaces

Interfaces define the capabilities that a resource exposes to the system.
Agents interact with interfaces rather than directly with specific resources.

Abstract interfaces allow the system to dynamically find a suitable resource that provides the required skills.

Example:

```
abstract Robot : interface;

Robot.Pick();
Robot.Place();
```

The program does not specify which robot should perform the task.
Instead, the system selects a compatible resource during runtime.

1.2.6 Resources

Resources are agents representing physical or logical entities that provide skills in the system.

Examples include:

- Robots
- Machines
- Conveyors
- Software agents

Each resource exposes one or more interfaces that describe the skills and variables it provides.

1.2.7 Planner Agent

The Planner Agent coordinates the execution of goals and skills.

Its responsibilities include:

- Finding compatible resources for abstract interfaces
- Negotiating and binding interfaces to resources
- Generating execution plans
- Optimizing resource usage based on cost or availability

By combining goals, skills, and interfaces, the Planner Agent can automatically determine how a task should be executed within the system.

1.2.8 Summary

The CMAS execution model allows developers to focus on describing the required operations rather than controlling specific hardware.

Programs define goals and call skills through interfaces, while the system automatically handles:

- resource selection
- execution sequencing
- negotiation
- runtime adaptability

This approach enables flexible automation systems and supports Plug & Produce environments where resources can be added or removed during runtime.

1.3 How to use CMAS

This section aims to familiarise users with CMAS and demonstrate its practical application through some examples. CMAS serves as a powerful tool for modelling flexible Plug&Produce capable automation solutions. However, the flexibility is dependent on the modelling technique used. The following segment will delve into the fundamentals of CMAS, accompanied by illustrative examples to enhance comprehension and practical understanding.

- Configuration
- Example 1 - Name tag
- Example 2 - Crane 2D
- Example 3 - Crane 3D

1.3.1 Configuration

There are two major tasks for CMAS to handle:

1. Configuration
2. Online control

Configuration is the task to describe for CMAS what you want to achieve and is a planning activity. The configuration can mainly be divided into some basic activities:

Activity	Action	Where
Add resources	When new resources have been acquired	Modelling tree

Activity	Action	Where
Add parts	Introduction of a new type of product	Modelling tree
Specify goals	Whenever you want to add or modify a part	On parts
Specify process plans	Whenever you want to specify or modify how a specific goal is achieved	Process plans
Specify skills	When a skill is added or modified	On resources

A good configuration approach is crucial when it comes to a configurable multi-agent system. The configuration is based on an ontology model. To understand how to make a configuration we will start with a small example.

1.3.1.1 Goals

Let's say we want to transform a metal plate into a name tag, see figure below. The first step is to create a virtual representation of the raw metal plate within the system. This virtual representation is managed by an entity called a **part agent**. The part agent represents the raw metal plate and oversees its transformation into a name tag. In this example, the transformation involves smoothing the corners, drilling four holes, and printing a name on the plate.

These transformations are expressed as **goals**. Every part agent is assigned goals, which allow it to dynamically plan and schedule a sequence of actions needed to achieve its overall objective, in this case, becoming a finalized name tag. A goal defines a desirable state for the part agent. In this example, the overall aim for the part agent, "becoming a name tag", can be broken down into the following goals:

#	Goal	Precondition
1.	Get rounded corners	None
2.	Get hole in the NE corner	Goal 1 is finished
3.	Get hole in the NW corner	Goal 1 is finished
4.	Get hole in the SE corner	Goal 1 is finished
5.	Get hole in the SW corner	Goal 1 is finished
6.	Get a printed name	Goal 2 & 3 & 4 & 5 is finished

The order of the goals is important, in this scenario, goal 1 can be reached whenever as it does not depend on any other goals. Goals 2, 3, 4 and 5 can only be reached after goal 1 is completed. These goals can be completed in any order or even in parallel. Finally, goal 6 must be completed at the end, as it depends on the completion of all preceding goals. This order of goals forms a **process plan**, which can be expressed and visualised using a **goal chart**.

Note

Goals does not state how the goals should be reach only the order of them. It is also important to break down the problem in a feasible amount of goals to take advantage of the flexibility within CMAS.

1.3.1.2 Process plans

A part agent must be able to reach its goals. A process plan simply describes a sequence of skills to be used to reach a specific goal. Hence, a process plan assists in transforming a part agent from its current state to a new state where the goal is fulfilled. See the figure below for an example of a process plan to get rounded corners.

Step #	Skill
1	Grind corner NE

Step #	Skill
2	Grind corner NW
3	Grind corner SE
4	Grind corner SW

Note

A process plan only describe the main steps and skills involved to reach a goal. It does not describe what resources to use. This make the solution more independent of the real hardware and thus more flexible.

1.3.1.3 Skills

To be able to manufacture a name tag we need resources. A **resource agent** is a virtual representation of a resource in the manufacturing system. A typical resource might be a robot, a drill bit or a fixture. Every resource agent can be assigned one or several **skills** (abilities). A skill represents a way to do something (change the state of the world) and is published via an interface on the specific resource agent. Examples of skills might be transportation, painting, or drilling. A resource might have several skills, e.g., a robot could represent both painting and transportation. A skill might require other skills on other resources to be able to execute. An example might be a make hole skill on a drill bit resource that requires a drill skill on another resource. This chain of skills is a key element to get the system more flexible. A skill is described as a sequence of local actions and remote skills executed in a specific order, see figure below for an example of a skill called make hole.

Step #	Action/Skill	Resource involved
1	Move to the target position	Executed on robot
2	Start Drill motor	Executed on drill motor
3	Stop drill motor	Executed on drill motor
4	Move home	Executed on robot

Chapter 2

Agent action

An agent, e.g, a resource agent, is associated with a property *AgentState* (from AgentAction). The *AgentState* indicates which state an Agent is at during runtime. The main states are;

State name	Description
Depot	Agent is ready to be deployed
Running	Agent is running
Failed	Agent has failed
Finished	Agent is finished

These states are visualised in the runtime viewer tree during runtime. Note, that there exist more states (see state chart figure below).

2.1 State Entry Behavior

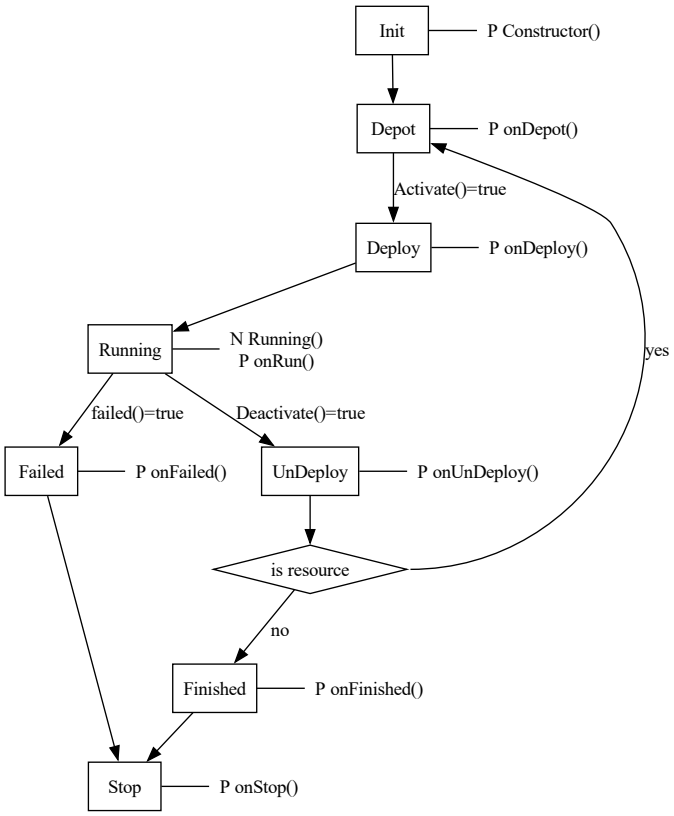
When an agent enters a state, two types of procedures can be triggered:

- **Pulse Procedure (P):** A *pulse* is generated once upon entering the state. This triggers a corresponding procedure exactly one time, for example: `onDepot()`.
- **Normal Procedure (N):** This procedure is called continuously while the agent remains in the state. It behaves like a cyclic callback.

Important

- A normal procedure must complete within a defined time limit.
- It should **not block or lock** the agent, as this may affect responsiveness and performance.

These procedures are not predefined. Instead, they can be implemented by the user within the `AgentAction` class as custom methods.



2.2 Procedures

2.2.1 Constructor

The constructor for an AgentAction is executed once when the agent starts. It runs only a single time and can be used to initialize variables, or perform other one-time setup tasks.

```
Program Part;

var runOnce: boolean;

procedure Running;
begin
    if not runOnce then
    begin
        writeln('Agent ', agent.name, ' is now Running');
        runOnce:=true;
    end;
end.

// Constructor
begin
    runOnce:=false;
end.
```

2.2.2 onDepot

The onDepot procedure is called once when the agent enters the Depot state. In the simplified example below, onDepot is used to automatically deploy a resource. Note, this logic should not be used for parts, as it would result in an infinite number of parts being generated. It also prevents the user from undeploying the resource, since it would be immediately redeployed.

```
Program Resource;

var runOnce: boolean;

procedure onDepot;
begin
    // Will deploy this resource when it is in the depot
    DeployByInstanceID(this.InstanceID);
end.
```

2.2.3 onDelpoy

The onDeploy procedure is called once when the agent enters the Deploy state.

```
procedure onDeploy;
begin
    writeln('Resource ', agent.Name, ' is now deployed.');
end.
```

2.2.4 Running

The procedure Runnning() is a normal (N) procedure, a procedure that is constantly called, i.e., for each cycle the procedure is called and is expected to return immediately.

Note

It is not a good approach to use functions, such as writeln(), to continuously produce output from the procedure Running() since it will overload the system with information to display.

The following example will use the Running to to indicate that the agent is running. In this case a better approach to write a single message might be to utilise the (P) procedure onRun() instead.

```
Program Part;

var runOnce:=false: boolean;

procedure Running;
begin
    if not runOnce then
    begin
        writeln('Agent ', agent.name, ' is now Running');
        runOnce:=true;
    end;
end.
```

2.2.5 onRun (agent)

The onRun procedure is called once when the agent enters the Running state.

```
Program Part;

procedure onRun;
begin
    writeln('Agent ', agent.name, ' is now Running');
end.
```

2.2.6 onFail

The onFail procedure is called once if the agent enters the Failed state.

```
Program Part;

procedure onFailed;
begin
    writeln('Agent ', agent.name, ' has failed');
end.
```

2.2.7 onUndeploy

The onUndeploy procedure is called once the agent is deactivated or undeployed.

```
Program Part;

procedure onUndeploy;
begin
    writeln('Agent ', agent.name, ' is undeployed');
end.
```

2.2.8 onFinished

The onFinished procedure is called once a part agent is finished.

```
Program Part;

procedure onFinished;
begin
    writeln('Agent ', agent.name, ' is done');
end.
```

2.2.9 onStop

The onStop procedure is called as the final stage of any agent's lifecycle, just before the agent stops. It can be used to perform final cleanup tasks. Note that the runtime environment may already be shutting down at this point, so interactions with other agents should be avoided.

```
Program Part;

procedure onStop;
begin
    writeln('Agent ', agent.name, ' is done');
end.
```

2.3 Preconditions

Precondition functions are boolean expressions used to determine when an agent should transition from its current state to a new one. They are written in Structured Text and evaluated continuously during execution.

2.3.1 Activate

If the agent is in the Depot state and Activate returns true the agent will be deployed.

```
Program Resource;

function Activate():boolean;
begin
    // Will deploy current resource
    Activate:=true;
end.
```

2.3.2 Failed

If the agent is in the Running state and Failed returns true the agent will be transfered to the failed state.

```
Program Resource;

function Failed():boolean;
begin
    Failed:=true;
end.
```

2.3.3 Deactivate

If the agent is in the Running state and Deactivate returns true the agent will be transfered to the UnDeploy state.

```
Program Resource;

function Deactivate():boolean;
begin
    Deactivate:=true;
end.
```

2.4 Randomized retry jitter

In CMAS there are two global settings, available in Preferences → Runtime, named **RetryBackoff** and **RetryJitter**. When enabled or disabled, they affect the behavior of all agents in the system. These settings are used when an agent attempts to interact with another agent that is currently busy or unavailable or if something goes wrong. In such cases, the normal behavior is to retry after a short waiting period (a delay).

2.4.1 RetryJitter

When **RetryJitter** is enabled, retry delays include a small random variation. This helps prevent many agents from retrying at exactly the same time, which can otherwise lead to synchronization effects, congestion peaks, and repeated resource conflicts in distributed multi-agent systems.

Example:
Without jitter:

- Agent A retry at 100 ms
- Agent B retry at 100 ms
- Agent C retry at 100 ms

With jitter enabled:

- Agent A retry at 113 ms
- Agent B retry at 147 ms
- Agent C retry at 121 ms

Disabling jitter can be useful during debugging, deterministic testing, or performance measurements where repeatable timing behavior is required. In normal operation, it should typically remain enabled.

2.4.2 RetryBackoff

When **RetryBackoff** is enabled, the retry delay increases exponentially after each failed attempt. This reduces retry pressure during overload situations or temporary failures.

Example sequence with backoff enabled:
50 ms -> 100 ms -> 200 ms -> 400 ms -> ...

When disabled, retries use a constant delay (typically 50 ms for a normal negotiation but can vary depending on type of action). Exponential backoff is commonly used in distributed systems, networking, and multi-agent coordination to improve stability and reduce contention when many agents compete for the same resources. Disabling backoff can be useful during debugging, simulations, or scenarios where constant retry intervals are preferred.

RetryBackoff has a predefined min delay of 50 ms and a max delay of 750 ms. Note, the RetryJitter might render a total value lower than 50 ms or higher than 750 ms.

2.4.2.1 Timing illustration

In CMAS, retry delays are default computed using exponential backoff combined with optional randomized jitter. In the following table the min delay of 50 ms and max of 750 ms is used as a base and the effect of Backoff and Jitter is shown.

Retry n:	0	1	2	3	4	5
No backoff:	50	50	50	50	50	50
Backoff:	50	100	200	400	750	750
With jitter:	55	120	180	430	710	600

Chapter 3

CMAS Command Line Arguments

3.1 Command Line

CMAS supports command line options in the form of **key/value pairs**, with an optional **project file name** as the last parameter.
If a file name is specified, that project file will be automatically loaded after CMAS starts. The file name can also be specified as a key/value pair (e.g., file=project.cmas).

Key/value pairs can be specified either as:

- `parameter=value` (e.g., `debug=true`)
- `parameter value` (e.g., `debug true`)

Boolean options accept any of the following forms:
`true` / `false`, `t` / `f`, or `1` / `0`.

3.1.1 Preferences and Command-Line Overrides

All settings in CMAS are by default loaded from the preferences file
`cmas-preferences.json` when the application starts.

Each command-line option can **override** the corresponding value from the preferences for that session.
The only exception is the **`loadpreferences`** option, which controls whether the preferences file is loaded at all.

When CMAS is closed, all current preference values — including those changed via command-line arguments or through the user interface —
are **saved back to the `cmas-preferences.json` file**.

> **Note:** If you start CMAS with `debug=true`, that value will be saved to the preferences.
> The next time CMAS starts, debug mode will remain enabled by default unless you explicitly set `debug=false`.

3.1.2 General Options

Option	Default	Description	Example
loadpreferences	`true`	Loads application preferences from `cmas-preferences.json`. If set to `false`, CMAS will start with default preferences.	`cmas loadpreferences=false`

Option	Default	Description	Example
file	`true`	Loads a CMAS project after start.	`cmas file=project.cmas`

3.1.3 Application

Option	Default	Description	Example
LibrarySave	`true`	When saving a project, also save any changes to libraries.	`cmas LibrarySave=false`
ConsoleMirror	`false`	Mirror console output (requires restart to take effect).	`cmas ConsoleMirror=true`
LogMirror	`false`	Mirror log output (requires restart to take effect).	`cmas LogMirror=true`
Debug	`false`	Debug mode. When enabled this sets the `Debug` preference. (Persisted to preferences on exit.)	`cmas debug=true`
ShowMemoryUsage	`false`	Show memory usage in the title bar. Changing this triggers `Main::setMemoryTrack`.	`cmas ShowMemoryUsage=true`
ShowConnections	`true`	Show connections in the model viewer.	`cmas ShowConnections=false`
ShowModelViewer	`ALL` (enum)	What to show in the model viewer (enum `SHOW`).	`cmas ShowModelViewer=ALL`
InitialDirectory	`""` (empty)	Default open/save directory for file dialogs.	`cmas InitialDirectory=/home/user/projects`
ShowRibbon	`true`	Show/hide the main toolbar (ribbon). Changing triggers `Main::setRibbon`.	`cmas ShowRibbon=false`
TitleBar	`true`	Enable the modern title bar. Requires restart to take effect.	`cmas TitleBar=false`
ShowSplash	`true`	Show splash screen during start of CMAS.	`cmas showsplash=false`

3.1.4 Editor

Option	Default	Description	Example
EditorAutoComplete	`true`	Enable/disable editor auto-complete.	`cmas EditorAutoComplete=false`
EditorAutoIndent	`true`	Enable/disable auto-indent.	`cmas EditorAutoIndent=false`
EditorAutoSuggestion	`true`	Enable/disable auto-suggestions.	`cmas EditorAutoSuggestion=false`
EditorMarkErrors	`true`	Mark errors in the editor.	`cmas EditorMarkErrors=false`
EditorSyntaxHighlighting	`true`	Enable syntax highlighting.	`cmas EditorSyntaxHighlighting=false`

Option	Default	Description	Example
EditorHoverInformation	`true`	Show hover popup info when hovering over words in the editor.	`cmas EditorHover↩ Information=false`
EditorHoverShowWebHelp	`true`	When hovering over system functions show extra web help content.	`cmas EditorHoverShow↩ WebHelp=false`
EditorWrapText	`true`	Wrap long lines to next line in editor view.	`cmas EditorWrap↩ Text=false`
EditorShowRightMargin	`false`	Show a vertical guideline to indicate preferred line length (visual only).	`cmas EditorShowRight↩ Margin=true`
EditorRightMargin	`80`	Position (column) of the vertical guideline.	`cmas EditorRight↩ Margin=100`
EditorTabSize	`4`	Number of spaces a tab represents in the editor.	`cmas EditorTabSize=2`
EditorUseDefaultColors	`true`	Use built-in editor colors instead of user-defined ones. Changing triggers `Main↩ Action::setColorUser`.	`cmas EditorUse↩ DefaultColors=false`

3.1.5 RunTime

Option	Default	Description	Example
ACLTimeOut	`0` (ms)	Timeout in milliseconds for skill calls on other agents. `0` = no timeout.	`cmas ACLTimeOut=5000`
ClearConsoleOnRun	`true`	Clear console window when starting a run.	`cmas ClearConsoleOn↩ Run=false`
ClearLogOnRun	`false`	Clear log window when starting a run.	`cmas ClearLogOn↩ Run=true`
DeployResourcesOnStart	`true`	Deploy resources when runtime starts.	`cmas Deploy↩ ResourcesOn↩ Start=false`
DeployDeploymentAgents↩ OnStart	`true`	Deploy deployment agents when runtime starts.	`cmas Deploy↩ DeploymentAgents↩ OnStart=false`
DeployPartsOnStart	`false`	Deploy parts on start.	`cmas DeployPartsOn↩ Start=true`
StopOnPartFailed	`false`	Stop runtime if a part fails.	`cmas StopOnPart↩ Failed=true`
RetryOnBusy	`true`	If a required skill is busy, retry later; if `false`, the caller fails immediately.	`cmas RetryOn↩ Busy=false`
SelectSkillBasedOnCostOnly	`false`	Select skill purely on cost even if that skill is busy.	`cmas SelectSkill↩ BasedOnCostOnly=true`
WarnOnDuplicatedIDs	`false`	Warn at runtime when two entities share the same ID.	`cmas WarnOn↩ DuplicatedIDs=true`

Option	Default	Description	Example
CheckForSafetyOverlaps	`false`	When interfaces have overlapping hazards, booking one will automatically book the other to prevent unsafe execution.	`cmas CheckForSafety↩ Overlaps=true`

3.1.6 Safety

Option	Default	Description	Example
HighRisk	`5`	Threshold value for what is considered high risk; used to define safety limits.	`cmas HighRisk=7`

3.1.7 PathFinder

Option	Default	Description	Example
PathFinderBook	`false`	If `true`, the PathFinder will attempt to book all resources before execution.	`cmas PathFinder↩ Book=true`
PathFinderStopOnError	`false`	If PathFinder cannot find a solution, determines whether runtime should stop.	`cmas PathFinderStopOn↩ Error=true`
ShowPathFinderResults	`false`	Print PathFinder info to console to help diagnose errors.	`cmas ShowPathFinder↩ Results=true`
ShowMapResults	`false`	Print a PathFinder map to console.	`cmas ShowMap↩ Results=true`
SolverDebug	`false`	Debug printing for the solver.	`cmas SolverDebug=true`

3.1.8 Process Plan (Java)

Option	Default	Description	Example
ProcessPlanJavaPath	`./ProcessPlans↩ Java`	Path where Java process plans are found and compiled. Visibility/enabled state is controlled by `Process↩ PlanJava.ENABLED` in code.	`cmas Process↩ PlanJava↩ Path=/opt/processplans/java`

3.1.9 Developer Modes

Option	Default	Description	Example
debug	`false`	Enables or disables debug mode. When set to `true`, the `Debug` property in preferences is enabled.	`cmas debug=true`
mirror	`false`	Enables console and log mirroring by setting the `Console↩ Mirror` and `LogMirror` preferences to `true`.	`cmas mirror`

3.1.10 Example Usage

This example starts CMAS in debug mode, disables the modern title bar, and automatically loads the project file `project.cmas` after startup.
cmas debug=true titlebar=false project.cmas

Chapter 4

Deployment

There are several strategies to automate deployment in CMAS.

1. Batches
2. Deployment Agents
3. User defined logics for deployment

4.1 Creating and Using Batch Objects in CMAS

In CMAS, you can create a **batch** object to specify which agents to deploy. You can also define whether each deployed agent should be **attached** to a specific interface during deployment.
Attachments are particularly useful when using the Planner agent, as the planner utilizes these attachments to coordinate transport planning.

4.1.1 How to Create a Batch Object

1. Open the **Modelling** tab.
2. In the **Others** group, click **Batch** object.

This will create a new batch object in the modelling tree.

4.1.2 Opening the Batch Editor

- To open the **Batch Editor**, double-click the new batch object in the modelling tree, or
- Select the batch object and choose **Batch Set** in the **Property Viewer**.

4.1.3 Using the Batch Editor

- **Left Side**: Displays available agents for deployment.
- **Right Side**: Displays the current batch configuration.

To add an agent to the batch:

1. Drag an agent from the left side and drop it into the batch on the right.
2. You can add the same agent multiple times to create multiple instances.

To specify an attachment:

- Double-click an agent in the batch list to open its settings and assign an attachment.

4.1.4 Running

When CMAS is in **Run** mode, you can deploy a batch object from the **Depot**. If the batch contains multiple agents, all of them will be deployed. If any attachments were specified during setup, those agents will be attached accordingly.

> **Note:** You can deploy a batch as many times as needed. Each deployment creates new instances of the agents defined in the batch.

4.2 Creating and Using Deployment Agents in CMAS

Deployment Agents provide more options and control than a standard **Batch** object.

4.2.1 How to Create a Deployment Agent

1. Open the **Modelling** tab.
2. In the **Agents** group, click the **DeploymentAgent** object.

This will add a new deployment agent to the modelling tree.

4.2.2 Working with Batches in a Deployment Agent

A deployment agent has a relation called **Batches**. Expand the deployment agent node in the modelling tree to view and manage its associated batches.
You can create as many batches as needed here. Batches are created the same way as described in the batches section.

Note

a deployment agent must at least have one associated batch object to work.

4.2.3 Deployment Agent Properties

Deployment behavior is controlled using properties available in the **Property Viewer**.

4.2.3.1 NumberOfTimes

Defines how many times the deployment process should be repeated across all batches. Once the specified number of repetitions is completed, the deployment agent will automatically undeploy itself.

- `0` means infinite repetitions. In this case, the deployment will continue until the deployment agent is either manually undeployed or stopped.

4.2.3.2 VariableTrigger

The name of an agent variable that triggers deployment.

- The variable can be of type boolean, integer, or real.
- Deployment is triggered when the variable's value changes to something other than false (for booleans) or 0 (for numeric types).
- If left empty, the variable trigger will be ignored.

4.2.3.3 DeploymentTime

The deployment time is relative to the start or the previous deployment (if multiple).

- If time is < 0, the time is ignored and only the `VariableTrigger` is used.
- By default, the time value is interpreted in **milliseconds**. You can also specify units:
  - Use `h` for hours (e.g., `1h`)
  - Use `m` for minutes (e.g., `30m`)
  - Use `s` for seconds (e.g., `1s`)

4.2.3.4 DeployAt

Determines when deployment should occur. Available options:

- **TIME** – Deployment waits for the specified time or trigger condition before each repetition (up to *NumberOf↩ Times*).
- **INTERVAL** – Deployment happens immediately, then repeats based on the time or condition (up to *Number↩ OfTimes*).

Note

Deployment of **DeploymentAgents** is controlled by the RunTime\DeployDeploymentAgentsOnStart setting in the preferences. If this value is set to false, DeploymentAgents will not be deployed automatically at startup and must be deployed manually or by some logic.

4.3 User defined logics for deployment

If the default behavior of the **Deployment Agent** is not sufficient, there are several ways to implement custom deployment logic. It is hard to cover all possibilities but two main strategies are explained:

- **Centralized**, managed by a dedicated deployment agent that coordinates deployment across other agents.
- **Local**, applied directly to a specific agent's behavior.

4.3.1 Centralized Deployment Agent

By creating a deployment agent with an agent variable and setting **VariableTrigger** to that variable, you can control deployment directly from within the agent's user-defined functions.
Example: Create a deployment agent with a boolean agent variable named `trigger`, and set **VariableTrigger** to `trigger`.
To disable the default deployment logic, set **DeploymentTime** to `0` and **NumberOfTimes** to `0`.
Next, add agent actions by double-clicking the deployment agent. The following example will ask the user for the batch size and by default triggers five deployments, each separated by 2 seconds. This replicates the behavior of setting **DeploymentTime** to `2s` and **NumberOfTimes** to `5`. However, using a variable trigger allows for more complex, customized deployment logic.

> Note: The trigger variable can also be connected to an adapter to receive signals from external systems.

```
program DeploymentAgent;

procedure onRun;
var i, size : integer;
var result : string;
begin

    result:=InputDialog('Batch size', '5');
    if not StringIsEmpty(result) and StringIsInteger(result) then
        size:=stringtoint(result);
    else
        error 'Wrong batch size';

    for i:=1 to size do
    begin
        agent.trigger:=true;
        agent.trigger:=false;
        sleep(2000);
        writeln('deployed');
    end;

    // Make sure to undeploy the deployment agent
    undeploybyinstanceid(this.InstanceID);
end.
```

4.3.2 Local Deployment

The simplest way to apply local deployment behavior is to open an agent's **Actions Editor** and implement the `onDepot` procedure.

Example 1: In this example, we will implement a local deployment behavior on a 'Part' agent. Whenever a `Part` agent is placed in the depot, the `onDepot` procedure is triggered. The agent begins by waiting for 2 seconds, then deploys itself by using the `deployByInstanceID()` function. When a part is deployed, a new part is created in the depot, which repeats the process resulting in continuous, timed deployments.

```
program Part;

procedure onDepot;
begin
    sleep(2000);
    deployByInstanceID(this.InstanceID);
end.
```

Example 2: In this example, a dialog is created when the Part agent is placed in the depot. If the user presses the Deploy button, the agent will deploy itself and close the dialog. A new agent will then be created automatically, which in turn shows a new dialog continuing the cycle. If the user presses Stop, the entire runtime environment will shut down.

```
program Part;

procedure onDepot;
var Selection:=-1:integer;
begin
    OptionDialog(1, Selection, 'Press a button', 'Selection', 'Deploy', 'Stop');

    // Wait for user input
    repeat
        sleep(50);
    until Selection=-1;

    // Close dialog
    OptionDialog(1, false);

    if Selection=1 then
        deploybyinstanceid(this.InstanceID);
    else if Selection=2 then
        stopsystem();
end.
```

Chapter 5

CMAS Examples

This section provides a set of example models included in the CMAS distribution.
All examples are delivered as ready-to-use model files and are intended to demonstrate core concepts of the CMAS framework, from basic agent structure to full-scale simulation environments. The examples are organized in increasing complexity:

Name	Description	File name
Example 1 - Name tag	A simple example to understand the core concepts of CMAS.	Example1_in_help.cmas
Example 2 - Crane 2D	A 2D crane example that utilizes the integrated planner.	Example2_in_help_crane2d - planning.↩ cmas
Example 3 - Crane 3D	A 3D crane example demonstrating planner integration and full Plug & Produce behavior.	Example3_in_help_crane3d.cmas

Note

For Example 2 (the 2D crane), there are additional models distributed with CMAS that do not utilize the planner. However, they are still valid examples of how to implement a solution to the crane problem.

5.1 Example 1 - Name tag

To perform a real configuration for the scenario described in the introduction how to use CMAS, let's create a simplified example focusing on the goals for drilling holes in the name tag. The manufacturing process involves a drill station and its associated drill bits. We will begin by designing a basic model of the agents needed. The resources are divided into two distinct resource agents: *DrillBit* and *DrillStation*. Additionally, a Part Agent is required to represent the name tag being manufactured.
Now start the CMAS tool, it should look something like this.

5.1.1 Resource Agent 1 - Drill Bit

The first step is to add a **resource agent**. In CMAS, you can use the **Modeling Tree** viewer located on the left side. This is the primary interface for modeling and displays all entities within the model.
At the top of the CMAS window, you will find the main toolbar, organized into different categories across various tabs. Begin by switching to the **Modeling** tab. Each tab is further divided into groups. Within the Modeling tab, locate the **Agents** group. Click the [resource] button to create a resource agent, which will then appear in the **Modelling Tree** viewer.

Note

You can also click the **Add new** button in the modelling tree view. A dialogue window will then ask you what type of entity to add.

When the new entity is created it is automatically added to the **Modelling Tree** and selected. You can see all properties of the selected entity to the right in the **Properties** viewer. To be able to find our new resource in the system we need to give it a friendly **Name**, lets name it *DrillBit*. You can change the name by navigating to the **Properties** viewer and replace the automatically generated name with *DrillBit*. Further, it is also possible to give a more detailed description of the resource in the **Description**. The **Entity ID** is a unique *string* used by the platform to keep track of our agents during runtime. It is possible to give **Entity IDs** manually but usually it is best to let the system decide an **Entity ID** automatically.

Note

An **Entity ID** should be unique in the entire configuration.

The next step is to assign interfaces to our drill bit agent. An **interface** is a defined way for agents to communicate or interact with each other. An **interface** also represents physical and logical compatibility. Our drill bit needs to interact with two other agents, the *name tag* and the *drill station*. Hence, we need two interfaces, the first to be able to drill a hole in a part and the second to be able to utilise a drill station for rotation of the bit (our self).

#	Interface name	Description
1	Hole	Enables the drill bit to create holes in the name tag.
2	Chuck_B10	Allows the drill bit to interact with the drill station for rotation of the bit .

First, create the **Interface** to interact with parts. To do this expand the *DrillBit* resource in the modelling view and select the relation **Interfaces**. When the **Interfaces** is selected press the [interface] button in the interface group in the tool bar. While the new interface is selected in the modelling tree view go to the properties view and change the name to *Hole*.
To reflect the size of the hole we need a variable. On the *Hole* interface let's create an **Interface variable** named *Diameter*. To add variables to an **Interface** expand the interface in the modelling view and select the relation **Variables**. When the relation variables is selected press the [variablereal] button in tool bar to create a new variable. Now go to the properties view and change the variable name to *Diameter*, also set the **UpperBound** and **Lower↩ Bound** to *5.0* to represent a diameter of exactly 5 mm. Our drill bit **interface** is now compatible with holes that have a diameter of 5.

Note

Interface information is used to check for compatibility. I.e., the **Name** of the **interfaces** must match and then all the **Variables** types plus their range (**UppeBound** / **LowerBound**), for more details see negotiation.

Further, a hole also has a position lets create two more **Interface variable** called *x* and *y*. We can use the same type *VariableReal* to represent a real number. We do not give a range here, hence, any range will be compatible with these variables. With this modelling strategy it is now possible to create several drill bits with the same interfaces and name but with different diameters.
We also need to create a **Skill** on the *Hole* **interface**. A skill represents an action that other agents can use to change the state of the world, in this case to make a hole by the use of our drill bit **resource agent**. Now select the **Interface** *Hole* and select the relation **Skills**. Click the [processplanstructuredtext] button on the tool bar. Go to the properties view and name the new skill *MakeAHole*. We can leave the other skill details for now and come back to them later.
The *DrillBit* by itself cannot make a hole since it cannot rotate nor fixate. Therefore, it must utilise another resource, a drill station of some kind. To be able to interact with a drill station we need another **interface**. Let us, therefore, create a new **interface** named *Chuck_B10* on the resource. Remember that the *DrillBit* agent knows the position of the hole through the first interface but cannot make use of it. Therefore, the hole position must be transferred to the drill. For that reason, add two **Interface variables** to the *Chuck_B10* interface namely, x and y. The *DrillBit* agent is now configured. It is now possible to create many drill bits in the same way representing different chucks or hole diameters. Our configuration can now be visualised in a figure.
This figure can also be viewed in CMAS using the Schema Viewer. The Schema Viewer visualizes all agents and their interfaces, and for part agents it also displays their goals. Compatible interfaces between agents are represented by connecting lines.

5.1.2 Resource Agent 2 - Drill Station

Now create a drill station as a **Resource Agent** with a compatible interface to our drill bit *Chuck_B10* with two **Interface variables** named x and y. Also, create an empty skill named *Drill* on the same **interface**.

Note

This chain of agents with matching interfaces is an important key to flexibility. It is now possible to duplicate resources or replace resources as long as they are compatible with each other (i.e. compatible interfaces). We will see later how this work.

5.1.3 Part Agent - NameTag

Now when the resources are ready the next step is to create a **Part agent** that will be the virtual representation of the product, name tag. To add a part agent select the root in the modelling tree and press the [part] button in the tool bar. Change the name to *NameTag* for our new part.
On the **Part agent** we need to create four holes each hole will be expressed as a **Goal**. A **Goal** is something a **Part agent** will try to fulfil by utilising **Process plans** and/or **Skills**. To add a **Goal** expand the **Part agent** in the modelling view and select the relation **Goals** and click the [goal] button on the tool bar. Change the name of the new **Goal** to *Hole1*. To reach the goal we need to name a good **Process plans**. To do this select the button [...] behind the ProcessPlanName to get a list of available process plans and select *MakeAHole*. Now we have a **Goal** to get a hole but we also need to give information on where we want the hole.
To do this we need to create **Goal variables**. Expand the *Hole1* in the modelling tree and select **Variables** and the click the [variablereal] button on the tool bar. Now create two **VariableReal** variables and name them x and y. For x set the **Value** to 1 and y to 1. Further, also create a variable called *Diameter* and set the **Value** to 5.0. In this way, we create a demand for our **Part agent** to only consider a hole with a specific diameter of 5.0.
In the same way, as we added **Interfaces** to our **Resources** we have now created an interface on our goal. The main difference is the minor adjustment needed while designating the interface name in the context of a **Goal**. In the properties view for the goal click the button [...] behind **Interface** property. You should now get a list of available interface names. Select the *Hole* interface in the list. This is a convenient way to set the name of the interface.
Now, in the same way create three more goals with the same variables but with different values. This can be done by selecting *Hole1* in the modelling view and press [Duplicate] button in the modelling view. You will now get a copy of the **Goal** *Hole1* and you can easily rename it to *Hole2* and set new values for x and y. To easier see the difference between all goals you can give more details in the **Description** field. The **Part agent** is now ready and should look something like this.
To make unique positions for each hole change the x and y variables for goal 2, 3 and 4 to:

Goal	x	y
Hole1	1	1
Hole2	10	1
Hole3	10	10
Hole4	1	10

5.1.4 Process Plans

A **Process plan** is a description on how to reach goals. **Process plans** can be created on many different levels but for now let's focus on the part *NameTag*. Select the *NameTag* in the modelling view and in the properties view you can open the **ProcessPlan** editor by pressing [Edit].
A new editor will open.
The start and the end of the **ProcessPlan** is indicated in the editor. It is now possible to drag the **Goals** from the goal view in to the editor. However, since there is no specific order of the 4 holes we can say to the system that any order is valid. To do this drag the **Parallel** statement from the statement view and drop it on the start statement. The parallel statement will then be inserted after the start statement.
The **Parallel** statement is now visible as some parallel lines. Now drag each *Goal* in the goal list and drop them on the top line of the **Parallel** statement.
The editor now shows all 4 goals in parallel. It is also possible to see and change the name of each goal and the interface name and the requested process plan name in the editor. Now close the editor.

5.1.5 Skills

We also need to define the **skill** for *MakeAHole* on the *DrillBit* agent. To do this select the skill *MakeAHole* in the modelling view and in the properties view press the button Actions. The process plan for the **skill** is now visible, this is a Structure Text process plan. We now have the flexibility to describe in Structure Text what we want to achieve. This is an alternative to the **Goal** oriented process plan used for the part.

```
Program Processplan;

function onRun() : integer;
begin
    // Add actions here

    // Return value - running, done, failed or interrupted
    onRun:=processplan.done;
end.
```

What we need to do is to add actions on how to drill with the bit. The drill bit need another resource to be able to make a hole, i.e. the drill bit by it self cannot make a hole. That resource must have an **interface** that is compatible with our **interface**. However, we do not care about which resource it is at this planning stage we just want it to be a resource **interface** that we can use. This unknown but required **interface** we can refer to as an **abstract interface** and all **abstract interfaces** must be declared at the top of a process plan. To be able to refer to this **abstract interface** in our process plan we give it a name when we declare it. It is possible to declare as many **abstract interfaces** as we wish in a process plan but for this plan one is enough. In the main block in-between **Begin** and **End** we need to give the details on what we want to accomplish and how to make us of our abstract interface.
The position of the hole is something that we need to give when we ask for the **skill** *MakeHole*. We can do that by assigning the position from the **goal** (*Goal.x*) to the skills interface variable. However, since we have declared all variable on the interfaces all values are transfered automatically.
An abstract interface is a way to describe what kind of resource we need without a hard link to a specific resource. An abstract interface is converted to a real interface by a negotiation process during runtime.
When defined, it is possible to use the abstract interface and request a skill by using the abstract interface.

```
Program ProcessplanMakeAHole;

abstract aDrillResource : interface;

function onRun() : integer;
begin
    // Add actions here
    aDrillResource.Drill();

    // Return value - running, done, failed or interrupted
    onRun:=processplan.done;
end.
```

You can now close the editor for the skill *MakeAHole*.
Now we need to do the same for the Drill station to define a skill.

The action for the **skill** *Drill* on the *DrillStation* is to now move the spindle to the right position and and start the spindle. To simplify we only write some messages on the console with *writeln*. Note, that it is indeed possible to control a real drill station within the skill or to send messages/set signals to an external controller.

```
Program ProcessplanDrill;

function onRun() : integer;
begin
    // Add actions here
    writeln('Will drill a hole at position ', Interface.x, ' ', Interface.y);

    // Return value - running, done, failed or interrupted
    onRun:=processplan.done;
end.
```

5.1.6 Deployment

Our configuration should now look something like this and this is now our flexible control system.
We are now ready to test our configuration in the runtime environment. By pressing the [Run] button on the tool bar (first switch tab to Run), instances of our configuration will be transfered to the runtime environment. A new runtime tree viewer will open to visualise the runtime environment. The tree is divided in four different areas the first is called *Depot*. This is an area where agents that are ready to be deployed will remain until they are deployed manually, automatically by CMAS or by user logic.

Note

CMAS can be configured to automatically deploy all resource agents at runtime by setting the **Deploy↩ ResourcesOnStart** option to **true** in the **Preferences** viewer. If this option is enabled, your *DrillBit* and *DrillStation* agents will appear in the Running folder after start.

We are now ready to deploy our agents and can do that by pressing the [Deploy all] button in the runtime view. If you want to deploy each agent individually right click on the agent a chose deploy or make use of the main tool bar. When an agent is deployed it will move to the *Running* area. When a **part** agent is deployed and move to the *Running* area a new part is generated in the **Depot** ready to be deployed.

Note

If you get a compiler error you must go back to the configuration and correct the error. To test again you must restart the runtime environment.

In bottom of CMAS you should have a console window that display user outputs from agents. With our *writeln* call inside the skill *Drill* we should now be able to see that the drill will produce 4 holes. When a part agent has reached all goals it will be moved to the *Finished* or *Failed* area depending on the result. You can expand the *Finished* area to see the part and check that all goals has become ready and green.
It is now possible to produce as many name tags as desired by deploying new *NameTag* agents with the [Deploy all] button. To stop the runtime environment press the [Stop] button.
You can also view the history of a finished part. To do so, select the part you want to examine from the **Finished** folder. Right-click on the part to open the context menu and choose **Show History**. A new window will open, displaying the complete history of the part.
Of course it is possible to add print outs for other activities to see what´s going on. Print outs is one way to see activities in the system. An example of the process plan *GetHole* with print outs:

```
Program ProcessplanMakeAHole;

abstract aDrillResource : interface;

function onRun() : integer;
begin
    // Add actions here
    writeln('Start of process plan ', this.Name, ' with a diameter of ', interface.Diameter);
    aDrillResource.Drill();
    writeln('End of process plan ', this.Name);

    // Return value - running, done, failed or interrupted
    onRun:=processplan.done;
end.
```

5.1.7 Errors

Sometimes, identifying errors or mistakes in the model can be challenging, as they are typically discovered during runtime. If you encounter a compiler error (such as a mistake in a Structured Text skill), try following the prompts in the dialog boxes. These will guide you to the correct editor and highlight the line that caused the error. Additionally, note that the editor will display a red underline to help you identify errors as you code.
In some cases, you may see multiple error dialogs due to a single mistake. As in this example, the primary error is a missing semicolon (;) at the end of the statement on line 6 in the **ProcessPlanDrill** program. However, when the **ProcessPlanMakeAHole** attempts to use the skill, it will also fail due to the problem to execute the skill.
Another common issue is when nothing happens after deploying all your agents. To troubleshoot and see what is happening in the system, you can open the **Agent Thread** viewer.

This viewer shows the negotiation status for all agents. In this example, you can observe that the busy **Process↩ PlanMakeAHole** on the **Hole** interface of the **DrillBit** resource is unable to find any matching interfaces during the negotiation process.
To check interface compatibility, you can use the **Schema** viewer. At the top of the **Schema** viewer, change the **Show** dropdown menu to **Show folder or selection content**. Then, select the agent you want to check. The schema viewer will display the agent, and all compatible agents will be connected by lines. If any connections are missing, it indicates a problem with the interfaces. For compatibility, the interfaces must match in both name and variables.
It is also possible to select a specific interface in the model viewer. Right-click on the interface to open the context menu and choose **Verify Interface**. This will display a list of all compatible interfaces.
Another way to check for errors is by using **Project Verification**. Change the console view to **Verification** and click the **Verify Project** button. CMAS will then attempt to detect any possible errors. Note that, since we are not running

the system, it can be challenging to check complex, abstract interfaces during the modeling phase. E.g, interface variables are only updated during runtime if they are depend on external signals.

5.2 Crane Simulation Example

5.2.1 Introduction

Start the crane simulation program, it should look like the picture below. Within the simulation, there's a crane capable of motions along the *x y* plane. Equipped with a vacuum gripper, the crane can affix products to itself and move them around. Within the simulation model, there exist two sources: **source 1** for **part A** and **source 2** for **part B**. Additionally, two processes are available for part processing. Upon completion, a part can be released into the sink for removal from the system.
The manufacturing environment includes sensors and control signals that can be reach through MODBUS TCP.

5.2.1.1 Table of MODBUS signals

Name	Type	Direction	Address	Explanation
setX	Integer	Write	1	Crane set value for x
setY	Integer	Write	2	Crane set value for y
vacuum	Boolean	Write	3	Crane lift mechanism Vacuum on/off
startp1	Boolean	Write	4	Start signal to process 1
startp2	Boolean	Write	5	Start signal to process 2
atX	Integer	Read	15	Crane current position in x
atY	Integer	Read	16	Crane current position in y
source1	Boolean	Read	17	Proximity sensor in source 1
source2	Boolean	Read	18	Proximity sensor in source 2
p1running	Boolean	Read	19	True if process 1 is running
p2running	Boolean	Read	20	True if process 2 is running
p1sensor	Boolean	Read	21	Proximity sensor in process 1
p2sensor	Boolean	Read	22	Proximity sensor in process 2
button1	Boolean	Read	23	Button 1 state
button2	Boolean	Read	24	Button 2 state

5.2.2 Modelling with planner

In this example, we'll develop a CMAS model that utilises the integrated planner. This planner can be used to automate transportation operations, i.e, handling the movement of parts or resources. By employing this approach, we can avoid specific transportation goals and concentrate solely on manufacturing planning for parts. Additionally, this technique can be applied to resources, such as robot tools, e.g, the planner can then automatically swap tools on the robot as necessary. The planner requires; **location interfaces**, **transportation interfaces** and skill **requirements**. A location is a specialised interface that can serve as a start or end point. While a transportation is a interface that can be used to transport between two locations.

Begin by creating a new empty model in CMAS.

5.2.2.1 Create Parts

In this step we will create the two parts **ProductA** and **ProductB**. To better organise the project start by adding a new folder to the modelling tree, click the folder„ and rename it to **Products**. Select the folder and enable the property *MakeInstances* by setting it to true.

Note

If *MakeInstances* is **false** the entity inside that folder will not be instantiated during run.

Inside that folder create two *Part* agents and name them **ProductA** and **ProductB**. For each product set *AutoDeploy* to **false** to make sure that products are not deployed automatically during the start. We will later create our own logic for determining when a part can be deployed.

5.2.2.1.1 Interface

Open the *Interfaces* folder and create a new Interface for each product and name it **InterfaceSource**. This interface is to make sure that we can place a part in the **Source** resource later on. In the same way also create an *Interface* named **Crane** to enable the crane to handle a part.

Note

It is not necessary to create interfaces for the two processes as they will be handled by the goals.

5.2.2.2 Create template variable

Create a new folder in the modelling root and name it **Templates** make sure that *MakeInstances* property is set to **false**. Inside the **Templates** folder create a new *VariableObject* and name it **Location**. This will be used as a base for any location in the system. Open the **Location** variable and inside *Items* create two new *VariableInteger* and name them **x** and **y**.

5.2.2.3 Create Sources

Now create a new folder named **Resources** and set the property *MakeInstances* to true. Inside that folder create a new *Resource* agent and name it **Source1**. We want to make use of the proximity sensor to automatically deploy a new product agent when a product enters the source. The first step is to make sure that **Source1** can host a product by creating a new *InterfaceSpecialisation* named **InterfaceSource**. To be able to use the interface as a location change the property *SpecialisationName* to *Location*. Now we need a location variable, drag the template variable **Location** from the **Template** folder to the new interface folder called *Specialisation* and select *Copy*.

Note

It is also possible to manually create the *VariableObject* inside *Specialisation* but it can be more convenient to work with templates. If you select *Reference* instead of *Copy* a reference is made, if you then change the template all referenced objects will be updated accordingly. A template based variable can only have one value during modelling, e.g., an integer value can be 1, even if you create many references they will point to the same variable. However, during runtime each variable will become an unique instance and they can then have different values.

We also need to create two specialisation skills for the *Interface*. Create a new *Skill* inside the *Skills* folder, name it **pickPart** and change the *PostAction* to *DETACHED*. Also create a *Skill* named **placePart** with *PostAction* set to *ATTACHED*. These skills can now be used by the internal AI planner or by other resources to attach or detach from the source. For each skill we can add logic required to do the real attach/detach, however, for the source we can leave it empty since there is no specific logic required to attach/detach.

Note

It is also possible to create location interface directly by using *interfacelocation*. A *interfacelocation* will automatically get attach/detach skills and an empty *Specialisation* object.

The second step is to detect when a new part arrives, for that we can utilise the proximity sensor inside the source. To read the value of the sensor we need to add **sensor1** as a variable. Inside the *Variables* folder on **Source1** create a new *VariableBoolean* and name it **sensor1**. Select the new variable **sensor1** and set the property *source* to **modbus.tcp:** and change the *address* to **17**. The **sensor1** variable is now connected via MODBUS to source1 inside the simulation. Set the *ReadOnly* property to **true** since the sensor is an input signal and it is not possible to write to it. Now we need to write the logic for the sensor. Open *Actions* editor for **Source1** and write the following program that will automatically deploy a part when the proximity sensor change state from **false** to **true**.

5.2.2.3.1 Source1 Action

```
program Source1;

var newPart:boolean;

// A general function to deploy a part and attach it
procedure Deploy(Name:string);
begin
    if not deployandattach(Name) then
        error 'Unable to attach ' + agent.Name +' to ' + Name;
end.

// Called when the Source is deployed
procedure onDeploy;
begin
    newPart:=false;
end.

// Called cyclic
procedure Running;
begin
    if not agent.sensor1 then
        newPart:=false;
    else if agent.sensor1 and not newPart then begin
        // Part sensor at the source is true, lets deploy a new agent
        // that will represents the new product that arrived.
        Deploy('ProductA');
        newPart:=true;
    end;

    Sleep(50);
end.
```

5.2.2.3.2 Source 1 test run

It should now be possible to test the model by pressing run„ in CMAS.
Place a new a new Product inside the source by pressing the [generate] button inside Source1 in the simulation. This will create a new part and activates the proximity sensor. Inside CMAS a new part agent *ProductA* should be deployed according to the logic in *Source1*. The product will be finished immediately and transfered to the *Finished* folder since it has no goals yet. You can see the part by expanding the run time tree and the *Finished* folder.

5.2.2.4 Create Crane

Inside the **Resources** folder create a new *Resource* agent named **Crane**. Add five *VariableInteger* in the *Variables* folder and map them to **setX**, **setY**, **atX**, **atY** and **vacuum** according to the modbus table. By setting **setX** and **setY** we can make the crane move and by reading **atX** and **atY** we can see the current position. The **vacuum** variable will turn on the vacuum cup to be able to pick up products. To control the crane we need to create two new *VariableInteger* named **targetX** and **targetY** They will be used internally within the agent, hence, there is no need to map them to MODBUS. Now we need to write the logic for the **Crane** controller. Open the *Actions* editor for **Crane** and write the following program.

5.2.2.4.1 Crane Action

```
program Crane;

var oldx, oldy : integer;

// During the first cycle in the run time
// we initialise the variables
procedure onRun;
begin
    agent.targetX:=agent.atX;
    agent.targetY:=agent.atY;

    oldx:=agent.targetX;
    oldy:=agent.targetY;
end.

// Main controller for the crane motion
// It will listen for changes of targetX and targetY
procedure running;
begin
    if oldx<>agent.targetX then begin
        agent.setX:=agent.targetX;
```

```
        oldx:=agent.targetX;
    end;

    if oldy<>agent.targetY then begin
        agent.setY:=agent.targetY;
        oldy:=agent.targetY;
    end;

    sleep(50);
end.
```

5.2.2.4.2 Crane test run

It should now be possible to test the model by pressing run in CMAS. The runtime three should look like this.
In the runtime tree expand the *Crane* and the *Variables* folder and select the *targetX* variable. In the *Properties* window you should see the value for current position of the *Crane* (crane default position at start is 500). Change the value to 300 and press the *[TAB]* key. The crane should now start to move to 300 in the x direction.

Note

> The values within the runtime viewer are not updated continuously. This is to prevent overloading the user interface with new values and to avoid slowing down agents. However, the values are updated each time you select a new entity.

Press stop to stop the runtime environment.

5.2.2.4.3 Crane transport Interface

In the next step we need to create an interface that can be used by other agents to command the crane to do transportation. The planner can also utilise the crane if we create a specialised transportation interface. To achieve this create an *InterfaceTransport* inside *Interfaces* and rename the new interface to **InterfaceCrane**. The new interface will include three *Specialisation* variables, **FromLocation**, **ToLocation** and **Step**. We must now make sure that the from and to variables are compatible with our location variables. Go to the **Templates** folder and expand *Items* inside the variable **Location**. Then drag **x** and **y** to the **FromLocation** variable in the new interface and drop them on *Items*. Now the **FromLocation** have a **x** and a **y** variable. Do the same for the **ToLocation**.
The **Step** variable is an integer that is used to detect if the transportation is approaching a pick (*Step*=1) or a leave position (*Step*=2).

5.2.2.4.4 Modify the transport skill

Now select the **Transport** skill inside *Skills* and rename it to **transportPart**. Double click it to open the skill editor.
The code now includes a function called `onNegotiate`. This function is invoked when the planner needs to verify whether the skills can manage transportation from the **FromLocation** to the **ToLocation**. It must return an **integer** value representing the cost to do the transportation, and **-1** if unable. By examining the coordinates on **remoteinterface.Specialisation.FromLocation** and **remoteinterface.Specialisation.ToLocation**, logic can be written to consider crane reachability. However, for this example, we can leave it blank as long as we return a cost of 0 (we can also use this.cost to get the cost of the skill). To observe the negotiation process, a `writeln` statement can be added inside the `onNegotiate` function.

```
function onNegotiate(Planning : boolean): integer;
begin
    writeln('onNegotiate for ', agent.name, '.transportPart() from ',
        remoteinterface.Specialisation.FromLocation,
        ' to ',
        remoteinterface.Specialisation.ToLocation);

    onNegotiate:=0;
end.
```

The next step is to create functions to support the reading of the signals. Enter the following functions in the skill. To make these functions available for all other functions write them at the top directly after the *program* statement.

```
function isMowing(): boolean;
begin
    isMowing:=NOT (agent.targetX=agent.atX AND agent.targetY=agent.atY);
end.

function isAtPosition(x, y:integer): boolean;
```

```
begin
    isAtPosition:= x=agent.atX AND y=agent.atY;
end.

function isAtTarget(): boolean;
begin
    isAtTarget:= agent.targetX=agent.atX AND agent.targetY=agent.atY;
end.
```

The next step is to create the transportation skill. In this example, a **SAFE_HEIGHT** parameter is used to ensure the crane's motion is safe and free from collisions. Note that there are many ways to implement the transportation skill.
Regardless of the method, it is crucial to remember that the OnRun function should not take too long to execute due to its real-time nature, as this could compromise real-time performance. Therefore, we need a strategy to execute the skill, as the crane requires time to complete the transportation. If the skill requires additional time to complete, OnRun should continuously return *processplan.running* until the task is finished. CMAS will then call the function cyclically. Once the skill is complete, **OnRun** can return *processplan.done* to indicate the task's completion.
In this example, the function **pickOrPlace** tracks the sequence of actions and performs only one step at a time. This allows the **OnRun** function to iteratively call **pickOrPlace**, maintaining real-time performance.

```
Program transportPart;

var constant SAFE_HEIGHT:=150 : integer;

var bussy:=false : boolean;

function isMowing(): boolean;
begin
    isMowing:=NOT (agent.targetX=agent.atX AND agent.targetY=agent.atY);
end.

function isAtPosition(x,y:integer): boolean;
begin
    isAtPosition:= x=agent.atX AND y=agent.atY;
end.

function isAtTarget(): boolean;
begin
    isAtTarget:= agent.targetX=agent.atX AND agent.targetY=agent.atY;
end.

function onNegotiate(Planning : boolean): integer;
begin
    if bussy then begin
        writeln('onNegotiate for ', agent.name, ' is bussy');
        onNegotiate:=-1;
        exit;
    end;

    writeln('onNegotiate for ', agent.name, '.transportPart() from ',
        remoteinterface.Specialisation.FromLocation,
        ' to ',
        remoteinterface.Specialisation.ToLocation);

    onNegotiate:=0;
end.

var picStep:=0:integer;
function picorplace(posX, posY :integer; vacuum:boolean) : boolean;
begin
    picorplace:=false; // Default value

    case picStep of
    0:
        if isMowing() then error 'crane is mowing';

        // Check if we need to move to a safe height or not
        if agent.atY>=SAFE_HEIGHT then
            picStep:=30;
        else begin
            agent.targety:=SAFE_HEIGHT;
            picStep:=10;
        end;

    10: // Wait until moving
        if isMowing() then picStep:=20;

    20: // Check if we are at a safe height
        if isAtTarget() then picStep:=30;

    30: // Start to move in x direction
```

```
        if isAtPosition(posX, agent.atY) then
            picStep:=70;
        else begin
            agent.targetX:=posX;
            agent.targetY:=agent.atY;
            picStep:=50;
        end;

    40: // Wait until crane starts to move
        if isMowing( ) then picStep:=50;

    50:  // Wait until move x is ready
        if isAtTarget() then picStep:=70;

    70:  // Check if we need to move y direction
        if isAtPosition(agent.atX, posY) then
            picStep:=100;
        else
            picStep:=80;

    80: // Move in y direction
        agent.targetx:=agent.atX;
        agent.targetY:=posY;
        if isMowing( ) then picStep:=90;

    90:  // Wait until move y is ready
        if isAtTarget() then picStep:=100;

    100:  // Activate vacuum
        agent.vacuum:=booleantoint(vacuum);
        picStep:=110;

    110:  // Check if we need to move to a safe height or not
        if agent.atY>=SAFE_HEIGHT then
            picStep:=140;
        else
            picStep:=120;

    120: // Move to a safe height before we end
        agent.targety:=SAFE_HEIGHT;
        if isMowing() then picStep:=130;

    130: // Check if we are at a safe height
        if isAtTarget() then picStep:=140;

    140:  // We are done
        picorplace:=true;
        picStep:=0;
    else:
        error 'wrong state: ' + inttostring(picStep);
    end;
end.

function onRun() : integer;
begin
bussy:=true;
    onRun:=processplan.done;
    if interface.Specialisation.step=1 then begin
        if picorplace(interface.Specialisation.FromLocation.x, interface.Specialisation.FromLocation.y,
      true) then
            onRun:=processplan.done;
        else
            onRun:=processplan.running;
    end;
    else if interface.Specialisation.step=7 then begin
        if picorplace(interface.Specialisation.ToLocation.x, interface.Specialisation.ToLocation.y, false)
      then
            onRun:=processplan.done;
        else
            onRun:=processplan.running;
    end;

    bussy:=false;

end.
```

5.2.2.5 Create Processes

Inside the *Resources* folder create a new *Resource* agent named **Process1**. Now, add three agent variables as boolean,

1. the proximity sensor for process 1, name it **proximitySensor** and connect it to address 21
2. the start signal for process 1, name it **startProcess** and connect it to address 4

3. the running indicator for process 1, name it **processRunning** and connect it to address 19

5.2.2.5.1 Interfaces

Process 1 need to be compatible with the two products and to do that we need to add an interface. Select *Interface Part* for *Source1* and right click in the modelling tree and select *copy*. Now select *Interfaces* inside *Process1* and right click in the tree and select *paste* and rename it to **InterfaceProcess**.
The next step is to create a skill on *Process1* that will start the process. Select *Skills* and create a new *Skill Structured Text* and name it **runProcess1**. Edit the *skill* and enter the following program.

```
Program runProcess1;

var processState:=0: integer;

function onRun() : integer;
begin
    // Set onRun return value to running to indicate that we want more time
    // to execute the skill. This will be used as the default return value.
    onRun:=processplan.running;
    sleep(50);

    // Execute the sequence of the skill
    case processState of
    0:  // Check that it is ok to run
        writeln('Start to run ' + agent.name);
        if agent.proximitySensor and not agent.processRunning then begin
            agent.startProcess:=true;
            processState:=10;
        end;
        else
            error 'process not ready missing a part';

    10:   // Wait for feedback that the process is running
        if agent.processRunning then begin
            agent.startProcess:=false;
            processState:=20;
        end;

    20:   // Wait until process is done
        if not agent.processRunning then processState:=30;

    30:   // We are done
        processState:=0;
        writeln(agent.name + ' is done');
        // Change the default return value to done to signal that the skill is ready
        onRun:=processplan.done;

    else:
        error 'wrong state: ' + inttostring(processState);
    end;
end.
```

To be able to trigger the built in AI planner to start we need to add a requirement to the *skill*. Select the folder *Requirements* in the skill **runProcess1** and add a new. Name the new requirement **MustBeAttached** and change the *WorkFlowType* to *ATTACHED*. This indicates that any one that want to run the skill **runProcess1** must first be attached to the interface to do so.
Now create **Process2** in the same way and map it to the simulation.

5.2.2.6 Create Sink

Inside the **Resources** folder create a new *Resource* agent named **Sink**. For the **Sink** create a new *Interface Specialisation* named **InterfaceSink**. In the same way we did it for the source or processes change the property *SpecialisationName* to *Location* and drag the template variable **Location** from the **Template** folder to the new interface folder called *Specialisation* and select *Copy*. Also make sure that you have a pick and place skills in the same way as before.
Now, create a new *Skill* on the interface and name it **packing**. Since the sink won't perform any actions, we can leave the skill unchanged. However, do not forget to add an *ATTACHED* requirement on the **packing** skill.

5.2.2.7 Set location values

The model is almost done but we need to set coordinates for all our locations. The locations are used by the planner to command the crane to do transportations. Start by opening the **Location** *Specialisation* variable on the **Source1** interface **InterfaceSource** and change the **x** value to 55 and the **y** value to 82.

Entity	Variable	Value
Source1	Location.↩ x	55
Source1	Location.↩ y	82
Process1	Location.↩ x	440
Process1	Location.↩ y	82
Process2	Location.↩ x	640
Process2	Location.↩ y	82
Sink	Location.↩ x	945
Sink	Location.↩ y	82

5.2.3 Goals

Double-click on **ProductA** to open the **Goal Chart** view. Now create three goals, right-click in the top-left box and select **Add New Goal**.
For each goal, set the following values:

Name	Interface name	Process plan name
GoalProcess1	InterfaceProcess	runProcess1
GoalProcess2	InterfaceProcess	runProcess2
GoalPacking	InterfaceSink	packing

Now create a similar goal chart for ProductB.

Note

We have now created process plans for **ProductA** and **ProductB** that focus solely on the objectives related to the products themselves. This approach allows us to disregard skills, such as transportation, that are not relevant from the product's perspective.

5.2.4 Running the example

When you press **Run** to start the model, CMAS will connect to the simulation based on the variables you have defined. If you place a **ProductA** in **Source1** by clicking the **[Generate]** button in the simulation, the **Source1** agent in CMAS will detect that and deploy a new **ProductA** agent. This agent will then begin executing its goals sequentially.

For example, the first goal, **GoalProcess1**, will initiate a negotiation process to find a resource capable of completing the task **runProcess1**. In this case, **Process1** will respond, "I can do it, but I have a requirement that you must first be physically attached to me".

This requirement will trigger the planner to search for a solution to attach **ProductA** to **Process1**. The planner begins by creating a map of all possible movements for products, materials, and resources. This map is then used to identify a solution that satisfies the requirement.

You can view the map generated by the planner in the console by enabling **PathFinder.ShowMapResults** in the preferences. Additionally, you can see the solution identified by the planner by enabling **ShowPathFinderResults**. This can be useful for identifying errors or mistakes in the model.

5.3 Crane-3D Simulation Example

5.3.1 Introduction

Start the Crane-3D simulation program. The simulation should appear similar to the figure below.
The simulation contains a crane capable of movement along the *x*, *y*, and *z* axes. The crane is equipped with a vacuum gripper that can attach products to the crane and transport them between different locations in the system. The simulated manufacturing environment consist of:

- **Table 1** containing **Part 1**
- **Table 2** containing **Part 2**
- **Machine 1** and **Machine 2** for processing operations
- A **Sink** where completed parts can be removed from the system
- A **Crane** that can transport **Part1** and **Part2**

The manufacturing environment also exposes sensors and control signals through MODBUS TCP.
In the left window of the simulation program, all available MODBUS signals are listed in a table. The table contains three columns:

Column	Description
Name	Describes the owner and purpose of the MODBUS signal
Index	The MODBUS address
Value	The current value or state of the signal

These signals can be used by CMAS agents to interact with and control the simulated manufacturing environment.

5.3.2 Plug and Produce

The simulation supports dynamic Plug & Produce behavior.
The resources in the system can be inserted or removed during runtime using the buttons:

- **Machine 1**
- **Machine 2**
- **Table 1**
- **Table 2**

When a resource is removed, the corresponding capability should disappears from the system. When it is inserted again, the capability should becomes available for negotiation and planning. This type of dynamic reconfiguration is one of the core concepts supported by CMAS. The first task in this example is therefore to describe how resources can be dynamically added and removed while the system continues to operate.

5.3.3 Adding table resources

Let us start by creating a new folder named *Resources*.
This folder will be used to organize all resource agents in the CMAS model.
Inside the *Resources* folder, create a new resource agent and name it **Table 1**.
The next step is to map signals from the simulated manufacturing environment into the resource agent.
Open the **Table 1** agent and select the *Variables* folder.
Create a new *VariableInteger* named *Sensor*.
This variable will be connected to the proximity sensor of the table to sense when a part is present or not.
For the *Sensor* variable set:

- *Source* = `modbus.tcp:127.0.0.1:502`
- *Address* = `35`

The address must match the *Index* column shown in the simulation signal list.
Next, create three additional integer variables for the table position with the same *Source*:

- *x*
- *y*
- *z*

Map the address to:

- `1:36:hrs`
- `1:37:hrs`
- `1:38:hrs`

The `hrs` modifier specifies a *signed* MODBUS holding register. These variables represent the base position of the table. If the table is removed from the simulation, all coordinate values become `0`.
Now create another variable *VariableInteger* named *CreatePart* and map it to address `30` without any modifiers. This signal will later be used to trigger creation of new parts in the simulation.

5.3.3.1 Internal table variables

The table agent also requires a few internal variables that are not mapped to MODBUS.
Create the following variables:

Name	Type	Purpose
PartName	VariableString	Name of the part to create
Auto	VariableBoolean	Enables automatic part generation
Pause	VariableBoolean	Temporarily pauses automatic behavior

Set the variable values to:

- *PartName* = `Part1`
- *Auto* = `true`
- *Pause* = `false`

5.3.3.2 Agent logic

The simulation contains two table resources that share the same behavior.
Instead of duplicating the logic for each table, CMAS allows the behavior to be shared between multiple resource agents.
Start by creating a new folder in the modeling root and name it *Include*. Inside this folder, create a new **Process↩ PlanStructuredText** named *TableFunctions*. This Structured Text object will contain the reusable logic shared by all table resources.

5.3.3.2.1 Auto deploy

Open the new *TableFunctions*.
We begin by overloading the *Activate()* and *Deactivate()* functions. These functions are used by the agent runtime to determine when a resource should be automatically deployed or undeployed. By monitoring the *PosX* and *PosY* coordinates, the agent can determine whether the table currently exists in the simulation or has been removed. If both coordinates are `0`, the table is considered disconnected and the resource will be undeployed automatically.

```
// Handles automatic deploy logic
function Activate() : boolean;
begin
    if Agent.PosX = 0 AND Agent.PosY = 0 then
        Activate := false
    else
        Activate := true;
end.

// Handles automatic undeploy logic
function Deactivate() : boolean;
begin
    if Agent.PosX = 0 AND Agent.PosY = 0 then
        Deactivate := true
    else
        Deactivate := false;
end.
```

Note

This mechanism allows the resource agent to automatically react to Plug & Produce events in the simulation. When a table is inserted into the system, the agent becomes available for negotiation and planning. When removed, the capability automatically disappears from the system.

5.3.3.2.2 Startup behavior

In the next step, we will override the *onRun()* function, which is executed once the table resource has been deployed. We must handle the situation where the table already contains a physical part when the resource becomes active. In this case, a corresponding *part* agent must also be deployed so that the part can participate in planning and goal execution. We can use the *deployandattach()* function together with the agent variable *PartName* to determine which type of part agent should be created.

```
procedure Deploy;
var Message:String;
begin
    if not deployandattach(agent.PartName) then begin
        Message:='unable to deploy ' +
            agent.PartName +
            ' and attach to ' +
            agent.Name;
        writeln(Message);
        error   Message;
    end;
    else begin
        writeln('New ', agent.PartName, ' added to ' + agent.Name);
    end;
end.

procedure onRun;
begin
    // If a part is already on the table during start
    // create a new agent part for that part
    if agent.Sensor<>0 then Deploy();
    agent.CreatePart:=0; // Make sure that the signal is 0
end.
```

5.3.3.2.3 Normal behavior

The next step is to describe the normal runtime behavior of the table resource. This behavior is implemented as a simple sequence-based state machine. The current execution state is stored in the variable *Sequence*, and the agent continuously evaluates and updates the sequence during runtime.
The table supports two operating modes:

- *Auto mode*
  Automatically creates and deploys new parts whenever the table becomes empty.
- *Manual mode*
  Waits for externally inserted parts and deploys corresponding part agents when a part arrives.

The logic continuously monitors:

- The proximity sensor (*Sensor*)
- The automatic mode flag (*Auto*)
- The pause flag (*Pause*)

In automatic mode, the table requests creation of a new physical part through the MODBUS signal *CreatePart*. Once the part has physically appeared on the table, a corresponding digital part agent is deployed and attached to the table resource. The sequence also verifies that the previously attached part agent has been detached before generating a new part. This is important because the physical sensor alone does not guarantee that the digital attachment state has been updated. This type of sequence-based logic is commonly used in CMAS resource agents to create reactive and event-driven behaviors while still maintaining a deterministic execution flow.

```
procedure setSequence(newSequence:String);
begin
    Sequence:=newSequence;

    // Debug messages
    threadmessage(newSequence);
    //writeln(agent.Name, ' changed sequence to ', newSequence);
end.

procedure running;
begin
    sleep(500);

    // This sequence will continuously generate a new part
    // whenever the table is empty if not paused
    case Sequence of
    Start:
        if not agent.Auto then setSequence('ManualMode');
        else setSequence('AutoMode');

    // Auto mode will create new parts when possible
    AutoMode:   // Check if table is empty or if we should pause
        if agent.Pause then setSequence('Start');
        if agent.Sensor=0 then setSequence('AutoNewPart');

    AutoNewPart:    // Request a new part at the table
        agent.CreatePart:=1;
        setSequence('AutoWaitForPart');

    AutoWaitForPart:// Check if part has arrived at the table
        if agent.Sensor<>0 then setSequence('AutoDeployPart');

    AutoDeployPart: // Deploy a new Part agent to represent the new part
        Deploy();
        setSequence('AutoWait');

    AutoWait:  // Wait for the interface to be free (the part has left)
        // Even if the sensor is off it is not sure that the agent is
        // dettached yet, i.e., we need to check the AttachedTo property
        if getobject(agent.Name+agent.PartName, TableInterface) then
        begin
            if TableInterface.AttachedTo=" then
                setSequence('AutoDone');
        end;

    AutoDone:  // Restart
        agent.CreatePart:=0;
        setSequence('Start');

    // Manual mode
    ManualMode:  // Wait for table to become empty
        if agent.Pause then setSequence('Start');
        if agent.Sensor=0 then setSequence('ManualWaitForPart');

    ManualWaitForPart:  // Wait for new part to arrive
        if agent.Sensor<>0 then setSequence('ManualDeployPart');

    ManualDeployPart:  // Deploy and attach new part
        Deploy();
        setSequence('ManualDone');

    ManualDone:  // Restart
        setSequence('Start');

    else:
        error 'Unkown sequence step ' + Sequence;
    end;
end.
```

Note

The sequence is evaluated continuously during runtime through the running() procedure. Each sequence step represents a small reactive behavior that may transition to another step depending on sensor values, agent state, or external events.

The separation between physical state (Sensor) and logical state (AttachedTo) is important in distributed systems, since physical events and digital synchronization may occur at slightly different times.

In the top of the editor add the following to create the *TableInterface* and *Sequence* variables. *InterfaceBase* type is defined in the include file.

```
// Inlcude system ontology definitions
$include 'system:ontology.inc'

// Creat an Interface object based on the onotology
var TableInterface : InterfaceBase;

// Main sequence
var Sequence:='start':String;
```

5.3.3.2.4 Link the logic to Table 1

In the final step, we connect the reusable logic defined in *TableFunctions* to the **Table 1** agent.
Open the *Table 1* agent action. Instead of duplicating the logic inside the agent, we simply include the shared implementation file. This ensures that multiple table agents can reuse the same behavior while still operating as independent resources. The include mechanism binds the previously defined functions (*Activate*, *Deactivate*, *on↩ Run*, and runtime sequence logic) directly into the agent.

```
program Table1;

$use 'TableFunctions'
```

This means that the Table 1 agent now inherits all behavior defined in *TableFunctions*, including Plug & Produce handling, deployment logic, and runtime sequence execution.

Note

This pattern allows CMAS models to scale by separating behavior (logic) from instantiation (agent definition). Multiple agents can share the same logic while maintaining individual state and runtime identity.

5.3.3.2.5 Test run

Before testing the logic of *Table 1*, we must ensure that the CMAS runtime does not automatically deploy the agent. This is done by setting the *Table 1* agent property *AutoDeploy* to *false*. In this mode, CMAS will only use the explicitly defined deployment logic provided in *TableFunctions*.
Start the runtime environment by pressing run.
If the model is correct, the *Table 1* agent will appear in the *Running* folder inside the *Runtime Tree* window. The *Running* folder contains all currently deployed agents.
When you press *Table 1* in the simulation to remove the physical table, you should observe that the corresponding *Table 1* agent is moved from the *Running* branch to the *Depot* branch.
If you press the button again, the *Table 1* agent will be deployed again and returned to the *Running* state.

5.3.3.3 Table 2

To create an agent representing *Table 2*, select the *Table 1* agent and use the *Duplicate* function. Rename the new agent to *Table 2*.
Next, update all mapped variables so that they correspond to the physical MODBUS addresses for Table 2.
Make sure to update the following:

- *PartName* must be set to `Part2`
- All sensor and coordinate mappings (*Sensor*, *x*, *y*, *z*, *CreatePart*) must match the Table 2 signal indices in the simulation

Finally, update the agent action definition. Change:
`program Table1;`
to:
`program Table2;`

This ensures that the runtime instance is correctly identified as Table 2 while still reusing the same shared logic defined in *TableFunctions*. You are now ready to test run with two tables.

5.3.4 Create machine resource

Now we can create the machine resources for *Machine 1* and *Machine 2*. We will reuse the same design pattern that was introduced for the table agents. Create a new resource inside *Resources* and name it *Machine 1*. In the same way as for the tables, add all relevant MODBUS signals to the agent *Variables* section, mapping them to the corresponding physical machine signals in the simulation.

5.3.4.1 Internal machine variables

Each machine provides two processing positions for workpieces. This means that two independent interfaces must be supported. The coordination between these interfaces is handled through three internal agent variables.
Create the following local variables (type *VariableBoolean*):

Name	Type	Purpose
Interface1Run	VariableBoolean	Controls execution permission for processing slot 1
Interface2Run	VariableBoolean	Controls execution permission for processing slot 2
RunDone	VariableBoolean	Indicates when processing has completed

These variables are used to synchronize the execution state between the two interfaces.
The variables *Interface1Run* and *Interface2Run* are controlled by the two machine interfaces. They represent execution permissions issued by the interfaces. Only when both interfaces are active, the machine is allowed to start processing.

5.3.4.2 Agent logic

In the same way as before, create a new *ProcessPlanStructuredText* inside the *Include* folder and name it *Machine↩ Functions*.
The MODBUS signal *Run* is used to start the physical machine, while *Running* is a feedback signal indicating that the machine is actively executing a process. Note that *Running* becomes false when the operation is completed.
The machine is controlled by two independent interfaces, and both must grant execution permission before processing can begin. The internal state machine ensures synchronized execution and safe reset between cycles. This behavior reflects how the physical machine operates in the real system.

```
// Main machine sequence
var MachineSequence:String;

procedure setSequence(newSequence:string);
begin
    MachineSequence:=newSequence;

    // Debug messages
    threadmessage(newSequence);
    // writeln('New MachineSequence=', MachineSequence);
end.

// Will handle deploy logic
function Activate():boolean;
begin
    if Agent.PosX=0 AND Agent.PosY=0 then
        Activate:=false;
    else
    Activate:=true;
end.

// Will handle undeploy activities, i.e., the user remove the resource
// we should then deactivate and move to the depot
function Deactivate():boolean;
begin
    if Agent.PosX=0 AND Agent.PosY=0 then
    Deactivate:=true;
    else
    Deactivate:=false;
end.

procedure onDeploy();
```

```
begin
    setSequence('Ready');

    // Reset signals
    agent.Interface1Run:=false;
    agent.Interface2Run:=false;
    agent.RunDone:=false;
end.

procedure running();
begin
    case MachineSequence of
    Ready:
        agent.Run:=0;
        if agent.Running=0 then
            setSequence('WaitForStart');

    WaitForStart:
        if agent.Interface1Run and agent.Interface2Run then begin
            setSequence('StartRun');
            writeln(agent.Name, ' start processing');
        end;

    StartRun:
        agent.Run:=1;
        if agent.Running<>0 then begin
            agent.Run:=0;
            setSequence('WaitForRunDone');
        end;

    WaitForRunDone:
        if agent.Running=0 then begin
            agent.RunDone:=true;
            setSequence('RunDone');
            writeln(agent.Name, ' processing done');
        end;

    RunDone:
        if (not agent.Interface1Run) and (not agent.Interface2Run) then begin
            agent.RunDone:=false;
            setSequence('Ready');
        end;
    else:
        error 'unknown machine sequence';
    end;

    Sleep(250);
end.
```

Note

Make sure to include the *MachineFunctions* in the agent action logic for each machine.

5.3.5 Create the sink resource

Now create a resource that represents the sink and map the position variables to the sink. The sink is a static resource in the simulation. It cannot be removed and does not require any runtime control logic or execution behavior. Because of this, the sink agent does not need any internal state machine or process logic. Its role is purely structural: it acts as a terminal location where completed parts are delivered and removed from the system. Only the position variables need to be mapped so that the Planner Agent can include the sink as a valid destination in transport planning.

5.3.6 Create a process plan

A process plan describes how a goal can be achieved. It can operate on a low level, where individual skills are specified, or on a higher level, where complete goal structures are defined. In this step, we create a process plan in Structured Text inside the modeling root and name it *ProducePart* to define how parts are manufactured in the system.

5.3.6.1 Process overview

We assume a simple production sequence:

1. Process the part in Machine 1
2. Process the part in Machine 2

3. Move the part out of the system via the sink

To achieve this we need 3 abstract interfaces that will represents the required resources. In this process plan we will make sure to use the built in support for booking and unbooking by setting the global *PathFinderBook* to true. This can also be done manually by opening preferences (F11).

```
Program Processplan;

abstract machine1, machine2, sink : interface;

procedure AutoBooking();
begin
    // This solution assumes that the path planner
    // handles all booking and unbooking of resources.
    // The process plan only defines the required sequence of skills.

    preferencevalue('PathFinderBook', true);

    // Production sequence
    machine1.Process1();
    machine2.Process2();
    sink.MoveOut();
end;

function onRun() : integer;
begin
    AutoBooking();
    onRun := processplan.done;
end;
```

Note

The abstract interfaces *machine1*, *machine2*, and *sink* are placeholders used to express the resource requirements of the process plan. They do not refer to specific physical resources. For example, the call *machine1.Process1()* means that the Planner Agent must find a resource that provides the *Process1()* skill. Likewise, *machine2.Process2()* may be mapped to any other resource that provides the *Process2()* skill. The same resource may also provide multiple required skills. If *Process1()* and *Process2()* should be executed by the same resource, the same abstract interface can be used for both operations, for example *machine1.*↩ *Process1()* followed by *machine1.Process2()*. If they should be performed by different resources, separate abstract interfaces can be used. This separation allows the process plan to describe the required manufacturing capabilities without being tied to specific physical resources. The actual resource mapping is determined by the Planner Agent during negotiation and planning.

5.3.7 Create part agents

Now we will create an agent for *Part 1*. Create a new part agent in the modeling root and name it *Part 1*. Open the goal chart editor for the part and add a new goal named *Goal1*. Drag *Goal1* into the goal chart workspace.
Double-click *Goal1* to define its associated process plan. Set the process plan to *ProducePart*. This links the part agent to the production process, meaning that when the goal is activated, the Planner Agent will automatically generate and execute a plan based on the *ProducePart* process plan.
Now repeat the same steps for *Part 2*, ensuring that it uses the same goal structure but represents a separate instance in the system.

Note

It is possible to define separate process plans for *Part 1* and *Part 2*. In more advanced configurations, multiple process plans with the same name, such as *ProducePart*, can coexist representing different candidate solutions. In this case, the Planner Agent will evaluate all available process plans and select the one with the lowest cost.

5.3.8 Create the crane agent

Now we can create a new resource agent representing the crane. As described earlier, all relevant MODBUS signals are mapped to the crane agent.
In addition to the external signals, the crane also requires internal agent variables to support execution control and coordination.

5.3.8.1 Internal crane variables

Create the following internal variables:

Name	Type	Purpose
commandX	VariableInteger	Target X position for crane movement
commandY	VariableInteger	Target Y position for crane movement
commandZ	VariableInteger	Target Z position for crane movement
Booked	VariableBoolean	Internal flag used to book/reserve the crane during execution

The *commandX*, *commandY*, and *commandZ* variables define the target position for crane motion, while *Booked* ensures that only one skill call can control the crane at a time.
These variables are not mapped to MODBUS, but are used internally by the crane agent logic to coordinate movement and access control.

5.3.8.2 Agent logic

Since the crane is a single shared resource, there is no need to separate the implementation into an external include file as before. All control logic is therefore contained directly within the agent.

```
program Crane3D;

// Global variables to remember the position
var oldx, oldy, oldz : integer;

procedure onDepot;
begin
    // A small delay before we start
    sleep(1000);
end.

// During the first cycle in the run time
// we initialise the variables
procedure onRun;
begin
    agent.commandX:=agent.AtX;
    agent.commandY:=agent.AtY;
    agent.commandZ:=agent.AtZ;

    oldx:=agent.commandX;
    oldy:=agent.commandY;
    oldz:=agent.commandZ;
end.
// Main controller for the crane motion
// It will listen for changes of commandX and commandY
procedure running;
begin
    if oldx<>agent.commandX then begin
        agent.targetX:=agent.commandX;
        oldx:=agent.commandX;
    end;
    if oldy<>agent.commandY then begin
        agent.targetY:=agent.commandY;
        oldy:=agent.commandY;
    end;

    if oldz<>agent.commandZ then begin
        agent.targetZ:=agent.commandY;
        oldy:=agent.commandY;
    end;
    sleep(50);
end.
```

Note

The crane action logic operates in a reactive loop where command variables are continuously translated into motion targets. This decouples planning decisions from low-level actuator execution. This will later be trigger by skills on the crane interfaces.

5.3.9 Adding interfaces

We now have a basic CMAS model based on agent representations. However, there is currently no way for the agents to interact with each other. To enable interaction, we need to introduce interfaces on each agent.
In this example, we want to utilise the Planner Agent to plan and execute all transportation activities. As seen in the *ProducePart* process plan, no explicit transport logic is defined between tables, machines, or the sink. This is intentional, since the process plan should focus on manufacturing logic rather than routing or logistics.
To enable planning, we therefore need not only interfaces for interaction, but also *Interface Specialisation* to support transportation semantics.

5.3.9.1 Interface design

We will define two interfaces for the entire model:

- One interface compatible with *Part 1*
- One interface compatible with *Part 2*

This allows us to separate the handling of Part 1 and Part 2. The interfaces for Part 1 and Part 2 will be specialised so that the Planner Agent can distinguish between different part flows, positions and select appropriate resources during planning.

5.3.9.2 Part1 and Part 2

To add interface requirements for a part the easiest way is to add the required interface name to a specific goal. Open *Goal1* for *Part1* and set 'Interface name for goal' to Part1. Do the same for *Part2* but set the interface name to Part2.

5.3.9.3 Table 1 and Table 2

Go to the *Table 1* Interfaces folder and create a new interface location and name it *Part1*.
This interface acts as a single point of interaction with *Table 1* and is specialised as a spatial location (a position in the physical environment).
When a location interface is created, two default skills are automatically exposed: *Attach()* and *Detach()*. These define how objects connect to and disconnect from the location.
For the table, there is no need to modify the behavior of these skills. Instead, the purpose of this interface is to expose the physical position so that other agents can interact with it.

5.3.9.3.1 Position specialisation

Inside the new interface location *Part1* open the *Specialisation* folder and locate the variable *Position*. Inside the *Position Items* folder, add three variableintegers: *x*, *y*, and *z*.
These variables define the spatial coordinates of the table.
Since the simulation provides these values through MODBUS, no manual configuration is required. Simply map:

- $x \rightarrow$ 1:36:hrs
- $y \rightarrow$ 1:37:hrs
- $z \rightarrow$ 1:38:hrs

This allows the table position to be directly synchronized with the physical simulation model.

Note

In most real manufacturing scenarios, spatial coordinates are not available as direct sensor signals. In such cases, the values must either be calibrated manually or computed within the agent logic based on system configuration or external measurement procedures.

Now do the same interface structure for part 2 but name it *Part2*

5.3.9.4 Machine 1 and Machine 2

To enable a machine to process both *Part 1* and *Part 2*, two separate interfaces are required.
Go to the *Machine 1* Interfaces folder and create two new interface locations named *Part1* and *Part2*.
Each interface represents a distinct interaction channel to the same physical machine, allowing the Planner Agent to differentiate between part-specific workflows.

5.3.9.5 Machine skills

Each interface must expose a processing skill in order to utilise the machine.
In the *Part1* interface, create a new skill in the *Skills* folder and name it *Process1*.

5.3.9.6 Spatial mapping of interfaces

The machine has a single physical position, but we need to represent two logical interface locations. This is achieved by transforming the base machine position into two interface-specific locations.

Note

It is also possible to define two logical positions for each physical location in order to handle both parts at the exact same spatial point. However, this option is not considered in this example.

This transformation is performed during negotiation, where the interface position is updated dynamically. The negotiation process reports the current machine position, allowing the interface to reflect runtime changes in spatial configuration.
This mechanism also enables dynamic reconfiguration of the system during execution. In the simulation, enabling move mode allows all resources to be repositioned interactively while maintaining correct interface bindings. Open the skill *Process1* and add the following negotiation behavior.

```
Program ProcessplanMachine;

var InterfaceSequence:='Ready' :String;

procedure setSequence(newSequence:string);
begin
    InterfaceSequence:=newSequence;

    // Debug messages
    threadmessage(newSequence);
    // writeln('New InterfaceSequence=', InterfaceSequence);
end.

function onNegotiate(Planning : boolean): integer;
begin
    interface.Specialisation.Location.x:=agent.posX;
    // Offset for interface 1
    interface.Specialisation.Location.y:=agent.posY+300;
    interface.Specialisation.Location.z:=agent.posZ;

    onNegotiate:=0;
end.
```

To execute the actual skill, we override the *onRun()* function inside *Process1*. The implementation uses the internal agent variables defined earlier to coordinate execution with the machine agent action. For a skill call we make the default outcome, *onRun := processplan.running* to signal that we are still executing the process and when we reach step *ReadyDone* we signal done by setting *onRun := processplan.done*.

```
function onRun() : integer;
begin
    onRun:=processplan.running;

    case InterfaceSequence of
    Ready:
        if agent.Running=0 and not agent.Interface1Run then
```

```
            setSequence('Start');

    Start:
        agent.Interface1Run:=true;
        setSequence('Wait');

    Wait:
        if agent.RunDone then
            setSequence('Done');

    Done:
        agent.Interface1Run:=false;
        setSequence('WaitRelease');

    WaitRelease:
        if not agent.RunDone then
            setSequence('ReadyDone');

    ReadyDone:
        onRun:=processplan.done;
        setSequence('Ready');

    else:
        error 'machine sequence';
    end;
end.
```

Now repeat the same structure but for interface *Part2* but keep in mind that you need another location for that interface and that the size of *Part2* in not the same as *Part1* .

```
interface.Specialisation.Location.x:=agent.posX;

// Offset interface for Part1 / Part2 separation
interface.Specialisation.Location.y:=agent.posY-300;

// Extra lift for part2
interface.Specialisation.Location.z:=agent.posZ-115;
```

Now you can do the same for *Machine2*.

5.3.9.7 Sink

For the sink, we also need two interfaces: one for *Part 1* and one for *Part 2*. Repeat the same pattern as before and create a location interface for each part.
For each interface, add a new skill called *MoveOut()*. No execution logic is required inside the skill, since the sink does not perform any processing. However, the skill name is important, as it is used by the process plan to identify and invoke the sink during execution.
The *MoveOut()* call in the process plan ensures that the Planner Agent can consistently treat the sink as the final destination in the system.
The interface position can be defined in two ways:

- Directly mapped to MODBUS position variables, as shown in previous examples
- Or computed inside the *onNegotiate()* function, where the position is derived or transformed before being exposed to the planner

Both approaches are executed during negotiation. The difference is only whether the position is taken directly from external signals or calculated dynamically within the negotiation step.

5.3.9.8 Crane

For the crane, we repeat the same pattern and create two interfaces: one for *Part 1* and one for *Part 2*. However, unlike previous interfaces, the crane interfaces are not location interfaces. Instead, they are defined as transport interfaces that expose transportation capabilities. These interfaces provide three key elements: *FromLocation*, *To↩ Location*, and *Step*. The *FromLocation* and *ToLocation* values cannot be predefined, since they are determined by the Planner Agent at runtime during plan execution. To ensure compatibility with other agents, the required data structure must still be defined. Therefore, we create three *integer* variables for both *FromLocation* and *ToLocation*: *x*, *y*, and *z*, without assigning any initial values. These variables act as placeholders that will be populated by the Planner Agent during execution.
Now we add the logic for the skill *Transfer()* for each interface.

```
program ProcessplanTransport;

$use 'CraneFunctions'

function onNegotiate(Planning : boolean): integer;
```

```
begin
    // Reject if already booked
    if isBooked() then
        onNegotiate:=-1;
    else
        onNegotiate:=this.cost;
end.

function onRun() : integer;
begin
    if interface.Specialisation.step=2 then begin
        onRun:=pick(
                interface.specialisation.fromLocation.x,
                interface.specialisation.fromLocation.y,
                interface.specialisation.fromLocation.z
                );
    end;
    else if interface.Specialisation.step=7 then begin
        onRun:=place(
                interface.specialisation.toLocation.x,
                interface.specialisation.toLocation.y,
                interface.specialisation.toLocation.z
                );
    end;

    Sleep(10);
end.
```

Note

The *step* variable indicates which transportation step to execute, see planner for all steps. In our case we just need to separate pick and place and we use step 2 and 7 for that. It is possible to implement a more advanced solution that follows all steps according to the planner sequence.

5.3.9.9 Crane execution logic

The detailed crane motion logic is implemented inside the shared include file *CraneFunctions* (a *ProcessPlan*↩ *StructuredText*). The purpose of this file is to encapsulate all low-level crane behavior such as:

- Motion control
- Pickup and placement operations
- Vacuum handling
- Collision-safe movement patterns and sequencing
- Resource booking and synchronization

This allows the transport interfaces to remain compact and focused on planner interaction, while the low-level execution details are isolated inside reusable helper functions in *CraneFunctions*.
The *CraneFunctions* implementation internally uses a sequence-based state machine to coordinate the crane motion. Functions such as *pick()* and *place()* execute several coordinated movement steps before returning *processplan.done*. Since we have two interfaces, one for each part, we can handle the different height of the parts in the *Negotiation* logic and later in the *Transfer()* logic. In addition to this, we also utilize the sensor on the crane and move slowly down until we have contact with an object. Hence, each interface gives a good starting point in the z direction but the final adjustment is a sensor based approach.
The synchronization between the interfaces for *Part1* and *Part2* is handled through a local interface named *Booking*. By defining a local abstract interface inside the crane logic, the agent can internally coordinate parallel access to the physical crane resource using the standard CMAS booking and unbooking mechanisms. In this example, the interface is accessed through a local interface reference:

```
define local Booking : interface;
```

However, several alternative approaches are possible. One common solution is to use a static interface mapping:

```
static Booking:='idBooking' : interface;
```

This approach utilizes the fact that the ID of the local interface is already known.
The only requirement is that the interface exposes at least one skill so that the interface can participate in negotiation and booking. In this example, a *dummy()* skill is used for this purpose. The skill itself performs no operation and exists solely to support synchronization through the CMAS booking mechanism.

```
// Crane limits
var constant MinX := -1200 : integer;
var constant MaxX := 1200  : integer;
var constant MinY := -4400 : integer;
var constant MaxY := 4400  : integer;
var constant MinZ := 0     : integer;

// Crane main sequnce
var CraneSequence:='ReadyForPick':String;
var z: integer;

// Create an interface to connect to our own Booking interface.
// This interface acts as a booking service for the crane and
// prevents different skill calls to the crane from interfering with each other.
define local Booking : interface;
begin
    function dummy;  // We need one skill
end;

function isBooked():boolean;
begin
    isBooked:=agent.Booked;
end.

function Book():boolean;
begin
    if agent.Booked then
        Book:=false;
    else begin
        // This is the safe way to make a booking by using the system.
        Booking.agreeandbook();

        // This local agent variable is used to faster
        // check that we have booked without asking another interface
        agent.Booked:=true;
        Book:=true;
    end;
end.

procedure unBook();
begin
    Booking._unbook();

    if not isBooked() then
        error 'Unable to unbook, crane is not booked: ' + CraneSequence;

    agent.Booked:=false;
end.

procedure checkBooked();
begin
    if not isBooked() then
        error 'Unable to move, crane is not booked: ' +  CraneSequence;
end.

procedure setSequence(newSequence:string);
begin
    CraneSequence:=newSequence;

    // Debug messages
    threadmessage(newSequence);
//  writeln('New CraneSequence=', CraneSequence);
end.

function gotoX(x:integer) : boolean;
begin
    checkBooked();
    agent.TargetX:=x;

    if agent.AtX=x then
        gotoX:=true;
    else
        gotoX:=false;
end.

function gotoY(y:integer) : boolean;
begin
    checkBooked();
    agent.TargetY:=y;

    if agent.AtY=y then
        gotoY:=true;
    else
        gotoY:=false;
end.

function gotoZ(z:integer) : boolean;
```

```
begin
    checkBooked();
    agent.TargetZ:=z;

    if agent.AtZ=z then
        gotoZ:=true;
    else
        gotoZ:=false;
end.

function gotoXY(x, y:integer) : boolean;
begin
    checkBooked();
    agent.TargetX:=x;
    agent.TargetY:=y;

    if agent.AtX=x AND agent.AtY=y then
        gotoXY:=true;
    else
        gotoXY:=false;
end.

function gotoXYZ(x, y, z:integer) : boolean;
begin
    checkBooked();
    agent.TargetX:=x;
    agent.TargetY:=y;
    agent.TargetZ:=z;

    if agent.AtX=x AND agent.AtY=y AND agent.AtZ=z then
        gotoXYZ:=true;
    else
        gotoXYZ:=false;
end.

function isMoving(): boolean;
begin
    isMoving:=NOT ( agent.commandX=agent.atX AND
            agent.commandY=agent.atY AND
            agent.commandZ=agent.atZ);
end.
function isAtPosition(x, y, z:integer): boolean;
begin
    isAtPosition:= x=agent.atX AND y=agent.atY AND z=agent.atZ;
end.
function isAtTarget(): boolean;
begin
    isAtTarget:= NOT isMoving();
end.

function pick(posX, posY, posZ :integer) : integer;
begin
    pick:=processplan.running;

    case CraneSequence of
    ReadyForPlace:
        // We can end up here if no booking strategy is applied
        // for the planner.  We need to handle this case in
        // code instead. If we answer 'replan',
        // the planner will replan the situation.

        pick:=processplan.replan;


    ReadyForPick:
        if NOT isBooked() then begin
            // Book the crane to prevent anyone else from using it.
            // The booking is held until the part has been placed.
            if Book() then begin
                agent.Vacuum:=0;
                writeln(agent.Name, ' will pick at',
                        ' x=', posX,
                        ' y=', posY,
                        ' z=', posZ);
                setSequence('StartGotoSafeHeight');
            end;
        end;

    StartGotoSafeHeight:
        if gotoZ(MinZ) then
            setSequence('StartGotoSafeX');

    StartGotoSafeX:
        if gotoX(MinX) then
            setSequence('GotoPickupY');

    GotoPickupY:
```

```
        if gotoY(posY) then
            setSequence('GotoPickupX');

    GotoPickupX:
        if gotoX(posX) then
            setSequence('GotoPickupZ');

    GotoPickupZ:
        if gotoZ(posZ) then
            setSequence('PickupCheckSensor');

    PickupCheckSensor:
        // If sensor is true then we touch the part
        if agent.Sensor<>0 then
            setSequence('PickupVacuumOn');
        else begin
            // Go down slowly in increment until the sonsor is true
            z:=agent.AtZ+4;
            setSequence('PickupMoveDown');
        end;

    PickupMoveDown:
        if gotoZ(z) then
            setSequence('PickupCheckSensor');

    PickupVacuumOn:
        agent.Vacuum:=1;
        setSequence('PickupWaitVacuumOn');

    PickupWaitVacuumOn:
        if agent.VacuumOn<>0 then
            setSequence('PickupLiftZ');

    PickupLiftZ:
        if gotoZ(MinZ) then
            setSequence('PickupMoveX');

    PickupMoveX:
        if gotoX(MinX) then
            setSequence('Done');

    Done:
        setSequence('ReadyForPlace');
        pick:=processplan.done;

    else:
        writeln('Unkown crane sequence step ' + CraneSequence);
        error 'Unkown crane sequence step ' + CraneSequence;
    end;
end.

function place(posX, posY, posZ :integer) : integer;
begin
    place:=processplan.running;

    case CraneSequence of
    ReadyForPlace:
        writeln(agent.Name, ' will place at',
                    ' x=', posX,
                    ' y=', posY,
                    ' z=', posZ);
        setSequence('GotoPlaceY');

    GotoPlaceY:
        if gotoY(posY) then
            setSequence('GotoPlaceX');

    GotoPlaceX:
        if gotoX(posX) then
            setSequence('PlaceMoveDown');

    PlaceMoveDown:
        if gotoZ(posZ) then
            setSequence('PlaceVacuumOff');

    PlaceVacuumOff:
        agent.Vacuum:=0;
        setSequence('PlaceWaitVacuumOff');

    PlaceWaitVacuumOff:
        if agent.VacuumOn=0 then
            setSequence('PlaceLiftZ');

    PlaceLiftZ:
        if gotoZ(MinZ) then
            setSequence('PlaceMoveAway');
```

```
    PlaceMoveAway:
        if gotoXYZ(MinX, agent.AtY,MinZ) then
            setSequence('Done');

    Done:
        setSequence('ReadyForPick');
        unBook(); // unbook crane for any one else to use
        place:=processplan.done;

    else:
        writeln('Unkown crane sequence step ' + CraneSequence);
        error 'Unkown crane sequence step ' + CraneSequence;
    end;
end.
```

5.3.9.10 Booking synchronization

The crane exposes two transport interfaces: one for *Part 1* and one for *Part 2*. Even though these interfaces are logically independent, they both control the same physical crane. To prevent conflicting commands, an additional internal interface named *Booking* is used as a synchronization mechanism.
The *Booking* interface contains a dummy skill named *dummy()*. The skill itself performs no operation, but it allows the standard CMAS booking system to be used for synchronization.
By booking the *Booking* interface before motion starts, the crane guarantees that only one transport operation can control the crane at a time.
This pattern makes it possible to:

- Keep multiple logical transport interfaces
- Maintain safe exclusive access to the physical resource
- Reuse the standard negotiation and booking mechanisms already provided by CMAS

The transport operation keeps the booking during the entire pick-and-place sequence and releases it only after the part has been placed successfully.

Note

> The booking interface acts as an internal synchronization resource and is not intended for direct interaction by other agents.

To add the *Booking* interface, create a new interface (without specialization) on the crane agent and name it *Booking*. Inside the *Booking* interface, create a new skill named *Dummy()*. The skill does not require any execution logic. Its sole purpose is to expose a skill that can participate in the standard CMAS negotiation and booking mechanism. This allows the interface to act as an internal synchronization resource for the crane.
static Booking:='idBooking' : interface;

5.3.10 Running the complete manufacturing system

The system model is now complete and all required resources, interfaces, skills, and process plans have been connected. When the runtime environment is started, the following sequence will occur automatically:
Resource agents are deployed when physical resources are available in the simulation

- Table agents create and deploy part agents
- Part goals become active
- The Planner Agent searches for valid process plans to solve the goals
- Required transport operations are inserted automatically
- The crane negotiates transport paths between interfaces
- Machines execute their processing operations
- Finished parts are transported to the sink

The process plan itself only describes the manufacturing sequence:

- Machine 1

- Machine 2
- Sink

All transportation between resources is automatically generated by the Planner Agent through interface specialization and transport negotiation.

5.3.10.1 Dynamic reconfiguration

Because all resources support Plug & Produce behavior, the manufacturing system can dynamically reconfigure during execution.
For example:
Removing Machine 1 makes its processing capability unavailable Moving Table 1 will update the position for the planner Reconnecting a resource automatically restores its capabilities
The Planner Agent continuously adapts to the currently available resources and interfaces.

Note

> This separation between process intent and transport planning is one of the central concepts in CMAS. Process plans describe what should be accomplished, while the planner determines how the system can execute the task using the currently available resources.

However, this example intentionally assumes a relatively controlled execution environment. The implemented logic does not handle all possible runtime disturbances or recovery situations. Handling such situations typically requires additional coordination logic, fault handling, replanning strategies, or recovery behaviors inside the CMAS model. It should also be noted that completely arbitrary runtime reconfiguration is rarely desirable in real industrial systems. In practice, Plug & Produce is typically used for controlled and planned reconfiguration scenarios, such as:

- Scheduled maintenance
- Resource replacement
- Planned reconfiguration
- Introduction of new equipment or parts
- Controlled startup and shutdown procedures

This is reflected in the model through the *Pause* signal in *Table 1* and *Table 2*. When *Pause* is set to true, the table stops generating new parts. This mechanism can be used as part of a controlled shutdown procedure, allowing the system to gradually reach an empty state where no active parts remain in circulation. Once all parts have left the manufacturing system, resources can be safely added, removed, or repositioned without interfering with active production. After reconfiguration is completed, *Pause* can be set to false again, allowing production to resume.
Removing a critical resource such as Machine 1 during active production is therefore mainly a theoretical demonstration of dynamic capability handling rather than a realistic production scenario.
The purpose of this example is to demonstrate the architectural principles behind:
Plug & Produce Interface-based planning Resource negotiation Distributed execution
rather than implementing a complete fault-tolerant production system. Hence, reconfiguration is only valid in quiescent system states where no active process execution depends on the modified resource set.

5.3.11 Summary

In this example we demonstrated:

- Dynamic Plug & Produce deployment
- Shared reusable resource logic
- Interface specialization
- Transport planning
- Resource synchronization and booking

- Goal-driven execution
- Distributed process coordination

The example also illustrates the separation between:

- Physical simulation
- Resource abstraction
- Planning
- Execution
- Transport coordination

That separation is one of the key architectural principles in CMAS.

Chapter 6

Goal Charts

A goal represents a desired outcome that an agent aims to achieve. Each part agent maintains a goal chart that defines the overall objectives for that specific agent. In addition, goal charts may also be defined as reusable process plans that can be utilized by other agents when appropriate.
In practical terms, a goal often corresponds to a manufacturing objective, such as achieving a smooth surface finish or applying a specific colour (e.g., black). In this model, each part agent is responsible for driving the part toward a defined final state by executing its assigned goals.
A part is considered complete when all goals in its goal chart have been successfully achieved. At this point, the part has reached its intended final state and is ready for subsequent use or delivery.
Consider a part agent that aims at a painted metal component:

```
Goal: Achieve a black, high-gloss surface finish

    Operation (Abstract task): Apply [black] surface coating

        Skill: Spray painting [color]

            Action: Position spray gun

            Action: Activate [color]

            Action: Execute spray pattern
```

Once the operation have been successfully completed, the Goal is fulfilled. For more information about the used structure see more process plan structure.
The Goal Chart represents an ordered sequence of Goals. Goals are executed from top to bottom according to their vertical order in the chart. When the final Goal in the sequence has completed successfully, the part agent declares the part ready.
This ordered sequence of Goals, together with its blocks for branching and execution, constitutes the Goal Chart.

6.1 The Goal Chart Editor

To open the Goal Chart Editor, create a part agent in the modelling tree and double-click the agent. Alternatively, select the agent and, in the Property View, press the Edit button for Process Plan.
The editor is divided into the following areas:
(1) Goal List (top left): Displays available but unused Goals for the selected agent. The list is empty if no unused Goals exist.
(2) Building Blocks (bottom right): Contains the available blocks used to construct a goal chart.
(3) Goal Chart Canvas (right): Displays the active goal chart. Each goal chart always contains a predefined [start] block and [end] block that define the execution boundaries.
If the Goal List is empty, a new Goal can be created by clicking the + Add new goal button.
An unused Goal can be edited by double-clicking it in the Goal List. This opens the Goal editor, where properties such as:

- Name
- Description

- Interface
- Variables

and other configuration parameters can be defined.
Multiple Goals can be created simultaneously by right-clicking in the Goal List and selecting Add multiple goals.

Note

Creating a Goal in the Goal Chart Editor is equivalent to locating the part agent's goals subfolder in the modelling view and adding a Goal there. Both methods operate on the same underlying model.

6.2 Goal chart blocks

The primary building blocks of a goal chart are the Goals. A Goal can be added by dragging it from the Goal List into the Goal Chart Canvas and dropping it onto an existing block or connection. Other building blocks can be added in the same way by dragging them from the Building Blocks list into the chart. The vertical order of blocks and Goals defines the execution sequence: the topmost block is executed first, followed by subsequent blocks in descending order until the sequence is complete. To remove a goal or a block from the chart right click on it and select remove. The following building blocks are available:

- Parallel
- ParallelOrdered
- Alternative
- Unordered
- Action
- Requirement
- Done
- Fail
- Interrupt
- Label
- Jump

Note

- The blocks from Parallel to Unordered are composite blocks and may contain other blocks.
- The vertical order of blocks and Goals in the chart represents a sequence: the top block is executed first, and subsequent blocks are executed in order from top to bottom.

6.2.1 Precondition

A precondition is represented by a horizontal line placed before a block or Goal in the Goal Chart. A precondition is a logical expression that evaluates to true or false. It is implemented as a function in structured text that returns a Boolean value.
Double-click the precondition to view or modify its logic. If the precondition evaluates to false, execution of the corresponding block or Goal will wait until the expression becomes true. Once execution has started, the precondition is no longer evaluated. That is, if it later becomes false, the execution of the block or Goal will continue without interruption.

```
program GoalPreCondition;

function Precondition() : boolean;
begin
    // Will wait until agent variable signal1 or signal2 becomes true
    Precondition:=agent.Signal1 OR agent.Signal2;
end.
```

Although it is generally recommended to define the execution order by placing Goals in the desired sequence in the Goal Chart, execution order can also be controlled by combining parallel goals with preconditions. This allows Goals to be placed in parallel while still enforcing execution dependencies.

```
program GoalPreCondition;

function Precondition() : boolean;
begin
    // Wait until the goals hole1 and hole2 have been fulfilled
    Precondition:=hole1.fulfilled and hole2.fulfilled;
end.
```

6.2.2 Breakpoints

It is possible to add a breakpoint to most blocks in a Goal Chart.

- To add or remove a breakpoint, double-click in the top-left corner of a block, where the cursor changes to a hand.
- When execution reaches a breakpoint at runtime, the execution of that specific branch is paused until the user continues.
- Breakpoints are useful for debugging the behavior of the Goal Chart and the system.
- A breakpoint is visualized as a red-filled circle in the top-left corner of the block.
- Use the Step or Continue button in the editor to resume execution after a breakpoint.

Note

A breakpoint in one branch will not halt other parallel branches; they continue executing independently.

6.2.3 Parallel

A Parallel block executes all its branches concurrently and independently of each other. Each branch may have its own precondition. A branch starts execution only when its precondition evaluates to true. Once execution has started, the precondition is no longer evaluated; i.e., if it later becomes false, the branch continues executing. The Parallel block is considered complete when all parallel branches have finished successfully.

Note

All branches are scheduled according to the priority property of the first Goal in each branch. The branch with the highest priority is started first, and there is a short delay before the next branch begins execution. In most cases, this ordering has no observable effect. However, in certain scenarios, the execution order may be relevant.

To add a sequence within a Parallel block:
(1) Drag a block or Goal and drop it on the Parallel lines in the editor.
(2) To add the next step in the sequence, drop a new block or Goal on top of the previous block in the sequence.
(3) To insert a block before an existing block, drop it on the line above the target block.
(4) To add a new branch, drop the block on the parallel lines again.

6.2.4 Parallel Ordered

A Parallel Ordered block executes one branch at a time, based on branch preconditions.

- The first available branch whose precondition evaluates to true is selected for execution.
- Once a branch has completed, the next available branch is selected.
- The block is considered complete when all branches have been executed.

6.2.5 Unordered

A Unordered block contains a set of Goals that are all placed at the same hierarchical level and have no defined execution priority.

- All contained Goals are considered independent of each other.
- Execution order is not predefined and may vary between executions.
- Each Goal may have its own precondition, and execution of a Goal starts when its precondition evaluates to true.
- The block is considered complete when all contained Goals have been executed successfully.

6.2.6 Alternative

An Alternative block executes only one branch based on preconditions.

- The first branch whose precondition evaluates to true is selected for execution.
- Once that branch has completed, the Alternative block is considered finished, and no other branches are executed.

6.2.7 Actions

An Action block represents a single executable statement (or a small set of statements) written in Structured Text.

- Actions are the lowest-level building blocks in the Goal → Operation → Skill → Action hierarchy.
- Each Action performs a specific task and does not contain other blocks.
- Execution of an Action corresponds directly to the associated Structured Text statement.

```
writeln('Executing an action');
```

This Action writes the text "Executing an action" to the console when executed. An Action can utilize variables defined in the agent. Once the Action completes, execution continues with the next block in the Goal Chart, according to the defined sequence or branching logic.

Note

It is possible to write more complex code inside an Action, including multiple statements, iterations, and selections based on agent variables. However, it is good practice to avoid complex logic and extended execution inside an Action block.

```
for agent.a := 1 to 5 do begin
agent.b[agent.a] := 1; // Initialize array variable b
end;
```

6.2.8 Requirement

A Requirement is similar to a Goal in that it specifies a desired state of the system. The key difference is that a Requirement automatically triggers the planner to find a solution to satisfy it. Requirements are typically conditions that must be fulfilled before a specific Goal can start.
A common example is the ATTACHED requirement, which states that a part must be attached to a specific interface before the sequence can continue. Example: To paint a part, it must first be attached to the paint fixture. The planner will then determine how the part can be moved and attached to the fixture automatically.
This approach is often more flexible than a fixed process plan, as the planner can explore multiple ways, more complex patterns and optimize the performance to satisfy the requirement. However, the modeling technique is more complex since all resources must be able to interact with the planner. You can read more about the planner in this example or in the planner chapter.

6.2.9 Done

A Done block indicates that a plan or a sequence within a Goal Chart has been successfully completed.
It is a terminal block that marks the end of execution for a branch. Done will not end the execution of other branches in the Goal Chart.

Note

The Done block does not perform any actions itself, it serves solely as a signal that the sequence has been completed.

6.2.10 Fail

A Fail block indicates that the Goal Chart has failed. It is a terminal block that marks the end of execution due to an error. When a Fail block is reached, the system considers the entire Goal Chart unsuccessful and the execution stops. Fail blocks are typically used in combination with preconditions, error handling, or validation checks to clearly define failure states.

Note

The Fail block does not perform any actions itself, it serves solely as a signal that the execution has failed.

6.2.11 Interrupt

An Interrupt block indicates that execution of a plan, branch, or sequence has been interrupted. It can be used to stop execution under specific conditions, such as resource unavailability or higher-priority tasks.

6.2.12 Label

A Label block is used to name a specific location in a Goal Chart. Labels do not perform any actions and are not executed like other blocks. They serve as reference points for Jump blocks or for documentation purposes within the Goal Chart.
Each Label should have a unique name to allow unambiguous references.

Note

Labels are purely organizational and do not affect the execution sequence unless explicitly targeted by a Jump block.

6.2.13 Jump

A Jump block directs the execution flow to a specific Label elsewhere in the Goal Chart.
When a Jump block is executed, the sequence immediately continues from the target Label, bypassing any intervening blocks. Jumps can be used to implement loops, conditional branches, or dynamic execution paths within a Goal Chart. The target Label must exist in the same Goal Chart and branch, otherwise, execution may result in an error.

Chapter 7

Interfaces

Every agent possesses interfaces through which all interactions between agents occur. An interface comprises variables and skills. Agents can have multiple interfaces, and the purpose of an interface is to facilitate communication solely with compatible agents. In other words, for two agents to interact, they must define an identical basic interface, with matching variable names. For example, a robot might have an interface with three variables: x, y, and z, along with two skills: pickAtLocation and leaveAtLocation. Any other agent that intends to utilize these skills must provide a compatible interface that defines x, y, and z. The compatibility of variables is determined by matching the name, variable type, and range. By incorporating the range, it becomes feasible to restrict x, y, and z to the robot's workspace. The treatment of variables across all interfaces can be fine-tuned. To achieve this, select the interface in the modeling tree and access the properties view to modify the "VariableMatching" property. The following options are available:

Option	Description
Required	All variables must exits and match to be compatible
Optional	If an variable exist on both interfaces it must match, otherwise it is ignored
Ignored	All variables are ignored

Note

When matching two interfaces A and B, both sides are taken into account. Consequently, if interface A is set to "Ignore" while interface B is set to "Required," the "Required" option will be considered for the matching process.

Defining goals is crucial for a part agent, and each goal serves as an interface to interact with resources. Hence, there might be a need to include variables in a goal and establish specific values for those variables. The name of the goal interface can be set in the properties view or in the goal chart viewer.

Note

A goal on a part cannot define a skill.

When a goal is defined with three variables (x=1, y=2, and z=3) and requires the skill "pickAtLocation," the part agent will then search for all compatible process plans that match these criteria. If more than one process plan is found the one with the lowest cost is selected.
For more information about interfaces see:

- Interface action
- Negotiation
- Variable bounds
- Goal
- Abstract Interfaces in CMAS
- Advanced Interface Concepts

7.1 Interface action

When an agent requires assistance from another agent at runtime, a search is conducted to locate a suitable remote agent interface. To ensure the appropriateness and compatibility of the sought agent interface, a negotiation process starts between the agents. This negotiation process entails agents striving to establish an agreement on how to effectively cooperate and synchronize their respective skills. An interface serves as a means of depicting compatibility, i.e., when two interfaces match they can be treated as compatible.
The functions of an interface, all are optional:

Function	Description
onNegotiate(planning:boolean; agentObject↩ :reference; interfaceObject:reference) : integer	Called during negotiation, if planning is true this is done during planning and all runtime variables might not be updated
onBook(a:reference; i:reference) : boolean	Called when asked to book
onOverlap : boolean	Called to check for overlap
onQueue() : boolean	Called before negotiation to preprocess the queue

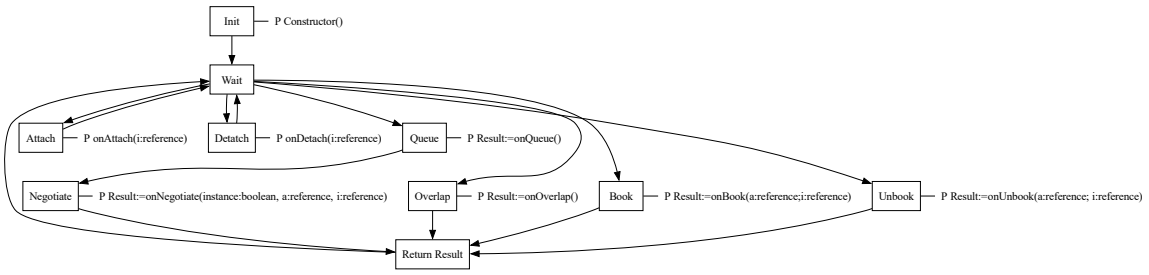
Note

onNegotiate on an interface is called before onNegotiate is called on each skill on the same interface. If on↩ Negotiate does not return a value >=0 the call to each skill is ignored. The returned value from onNegotiate for an interface is included in total cost during negotiation.

The procedures of an interface (they must be declared as procedures) and are optional:

Procedure	Description
onAttach(i:reference)	Called when an interface has been attached
onDetach(i:reference)	Called when an interface has bee detached
onTransfer(i:reference)	Called when post action transfered is executed
onUnbook(a:reference; i:reference)	Called when asked to unbook

Each function or procedure can be overloaded to replace the built in default mechanism.



7.1.1 Procedures

7.1.1.1 onAttach

The onAttach procedure is called when the interface is attached to another interface.

```
procedure onAttach(i:reference);
begin
    writeln('Interface ', this.name, ' is now attached to ', this.attachedto);
end.
```

7.1.1.2 onDetach

The onDetach procedure is called when the interface detached from another interface.

```
procedure onDetach(i:reference);
begin
    writeln('Interface ', this.name, ' is now detached from ', i.name);
end.
```

7.1.1.3 onTransfer

The onTransfer procedure is called when a transfer operation is done.

```
procedure onTransfer(i:reference);
begin
    writeln('Transfer on ', this.name, ' is now done.');
end.
```

7.1.1.4 onNegotiate

onNegotiate is called if another agent ask for a negotiation. It is possible to return -1 to state that the interface is not compatible or >=0 to accept, a value >=0 indicate a cost. The reference variables agentObject and interfaceObject are references to the agent and the specific interface that wants to negotiate. Note, the reference variables are only used during negotiation with an interface. For skills they do not exist.

```
function onNegotiate(Planning : boolean;  agentObject : reference; interfaceObject : reference) : integer;
begin
    onNegotiate:=this.cost;  // Accept by using the value in the cost property
end.
```

7.1.1.5 onQueue

By setting the interface property QueueRequest to *true*, the global interface variable `queue` is enabled and the onQueue() function is called before onNegotiate(), allowing the agent to inspect and manage incoming requests, see advanced interface concepts.

```
function onQueue():boolean;
begin
    queue[startindex(queue)].value:=remoteinterface.cost;
end.
```

7.1.1.6 onOverlap

The onOverlap function is called to check for overlaps. If it returns true the interface is booked to avoid overlap actions.

7.1.1.7 onBook

onBook is called if another agent interface want to book the interface. It is possible to return false to state that the interface do not agree on the booking request. The reference variables a and i are references to the agent and the specific interface that wants to book.

```
function onBook(a : reference, i : reference) : boolean;
begin
    writeln('Booked by ', a.name, ':', i.name);
    onBook:=true;
end.
```

7.1.1.8 onUnbook

onUnBook is called if another agent interface want to unbook the interface. The reference variables a and i are references to the agent and the specific interface that wants to book.

```
procedure onUnBook(a : reference, i : reference);
begin
    writeln('Interface is no longer booked by ', a.name, ':', i.name);
end.
```

7.2 Negotiation

7.2.1 Interfaces

A skill represents an action or task executable by a resource agent during runtime. Typically, a skill demands additional information, represented through variables. In CMAS, an agent's interface has the role of maintaining both a set of variables and a set of skills as one unit. This unit serves as a connection point and all collaborations are performed through interfaces.
For an interface to be operational it must at least possess one skill.

Note

An interface serves as a means of depicting compatibility, i.e., when two interfaces match, they can be treated as compatible.

7.2.2 Collaboration

When an agent requires assistance from another agent at runtime, a search is conducted to locate a suitable remote agent interface. To ensure the appropriateness and compatibility of the sought agent interface, a negotiation process starts between the agents. This negotiation process entails agents striving to establish an agreement on how to effectively cooperate and synchronize their respective skills. During the modelling phase, the ability to reference the utilization of compatible remote interfaces is essential, this without the need to specify any particular agent or interface. This is achieved through abstract interfaces in CMAS. Hence, an abstract interface represents a desired remote interface during the modelling process and lacks any direct connection to any specific remote interface. However, during runtime, it will be linked with the best available remote interface and transformed into a real interface during the negotiation process. A linked interface can then be used to interact with other agents, e.g., to request a skill to start and send variable values.

The following abstract interface types exist:

Typ	Description
Abstract	A pure abstract interface. All local interfaces and all remote interfaces are considered during the negotiation process to find the best pair of one local and one remote interface.
Define	Only one local interface is used (the defined one) to find the best compatible remote interface
Static	A remote interface is static mapped to any local interface. The remote interface must be known by ID at the modelling phase.

7.2.3 The negotiation process

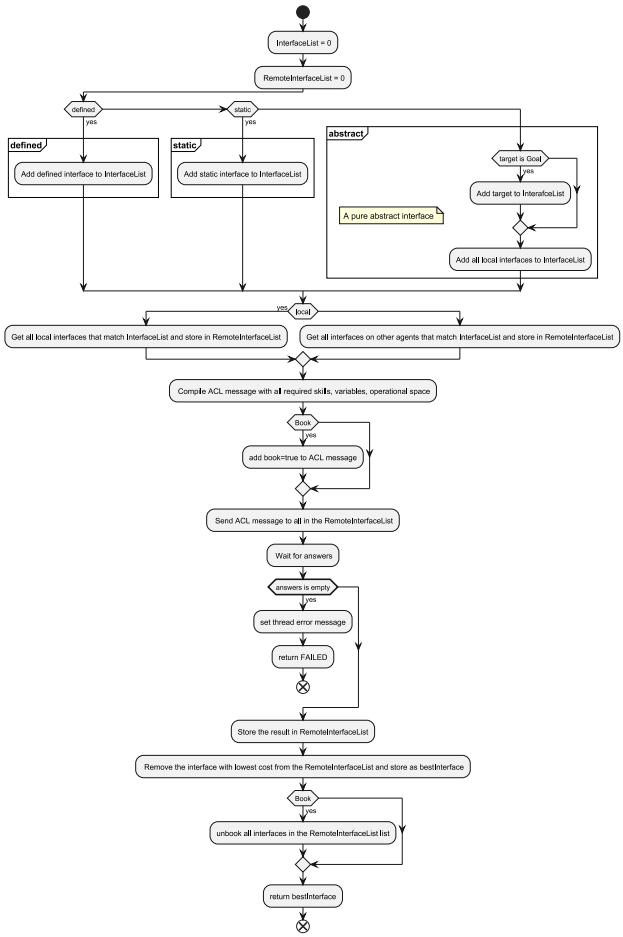
A negotiation is performed the first time a variable on a declared abstract interface is accessed (this step is ignored if handle is specified for the abstract interface) or if a skill is called on an abstract interface. It is also possible to do it manually by calling the negotiate() function on an abstract interface. For more details see, how to define and use abstract interfaces.
The negotiation process is conducted through the following sequential steps, for all local interfaces:

1. Compile a ACL message of required skills and the corresponding interface variables.
2. Generate a list of all remote interfaces that are compatible with any of the local interfaces through interface matching
3. If ignore busy is true remove all busy interfaces from the list. Ignore busy is default set to true but can be change by the function ignorebusy.
4. If ignore booked is true remove all booked interfaces from the list. Ignore booked is default set to true but can be change by the function ignorebooked.
5. Dispatch an ACL query, "negotiate", to all compatible remote interfaces, enclosing the list of obligatory skills and variables. This will trigger the onNegotiate() call on all remote interfaces.

6. In the event of an interface responds with a value >=0 (AGREE with a cost) that interface is appended to a list of potential interface combinations, i.e., saving the local and remote interfaces as a pair with a cost.
7. If an interface responds with a negative number (DISAGREE) it is disregarded.
8. If CheckForSafetyOverlaps is true, try to book all overlapping interfaces. If it fails for an interface unbook it and ignore it.
9. From the list of potential interface pairs (local, remote) select the pair that generates the lowest cost.
10. If the list is empty, depending on the abstract type and the settings, retry from step 1 or fail.

Note

If the interface is a `defined` abstract interface the negotiation process is only conducted on the define interface and the rest of the local interface will be ignored.



7.2.4 Interface matching

The process of identifying matching interfaces is carried out through the following steps:

1. Gather a list of active and deployed agents within the domain.
2. For each active agent, extract all interfaces that share the same name as the local interface.
3. Verify the alignment of variables (variable matching) between the local and remote interfaces; any mismatched interface is removed from the list.

7.2.5 Variable matching

The procedure for checking the compatibility of two variables within an interface is performed as follows:

1. Verify that the names are identical.
2. Validate that the types match.
3. Ensure that the local value falls within the bounds of the remote variable.

Note

Regarding the third step for bound verification:

- This verification is ignored if both upper and lower bounds are empty.
- It can be fixed to a specific value by setting the upper and lower bounds to the same value, e.g., 5.
- Alternatively, it can be defined within a range by setting distinct upper and lower bounds, e.g, 1 and 5.
- Another option is to establish a bound expression.
- The behaviour is also influenced by the "VariableMatching" property of the interface.

7.2.6 Details

For an abstract interface the following applies

- If the program that declares the abstract interface is called as a skill on an interface: all interface values will be copied (if possible) to the abstract interface as an automatic transfer of data. This step is ignored if the abstract interface is declared as handle.
- A negotiation will be triggered if a variable on the declared abstract interface is accessed. This step is ignored if handle is specified for the abstract interface.
- If a skill is called on the abstract interface and the interface is unnegotiated a negotiation process is triggered.
- If an integer variable is given as a parameter to a skill call on an abstract interface the skill will be started in parallel and the program will not wait for the result. Instead, the integer parameter will reflect the status of the skill call.
- If handle is specified all abstract interfaces remain static mapped after the first negotiation this can be avoided by calling the negotiate() function on the abstract interface.
- It is possible to trigger a negotiation by calling <interface>.negotiate() or <interface>.agree() A call to negotiate() will try to negotiate with available resources to find a suitable match if it fail the execution will stop with an error. If agree() is called instead, the same negotiation process will start and it will retry until an valid agreement or search forever.
- It is possible to book or unbook an interface by calling the <interface>._book() or <interface>.unbook(). A booked interface will remain booked until it is unbooked.
- If a skill is called and that skill fails the execution will be aborted and an error is thrown. This step is ignored if the interface is declared as handle and instead a return value is returned.
- It is possible to combine negotiation and booking in one call by <interface>.negotiateandbook() or <interface>.agreeandbook()

For an abstract interface the following static local functions exist:

Function	Book
negotiate()	no
negotiateandbook()	yes

Function	Book
agree()	no
agreeandbook()	yes

The negotiate() or negotiateandbook() will make a single attempt to locate a compatible remote interface from the accessible resources. If this proves unsuccessful, the execution will halt with an error unless it was declared with the modifier handle. When handle is specified the return value will be failed or done. The agree() or agreeeandbook() will try forever to locate a compatible remote interface from the accessible resources. This is good approach if resources can be pluged in or out as in the Plug & Produce concept. When an interface is booked it must be unbooked when no longer needed. To unbook an interface call the _unbook() function on the abstract interface. The behaviour of negotiate and agree on an interface can be modified with ignorebusy() or ignorebooked().

Note

If a remote skill is called on an uninitialised abstract interface an agree() is automatically triggered. If the interface is declare with the handle modifier a negotiate() call is triggered instead. This can be avoided by calling any of negotiate() or agree() functions before. It is also possible to re-negotiate an abstract interface later on by calling negotiate() or agree() again.

7.2.6.1 Negotiation and Booking

It is possible to use an abstract interface directly. In the following example an abstract interface a is declared. When the skill call doSomeThing() is called it will trigger a negotiation. That negotiation process will try to map any local interface to a compatible remote interface before executing the skill.

```
Program example;
abstract a : interface;

function onRun() : integer;
begin
    a.doeSomeThing();
    a.doeSomeMore();
end.
```

During the call to doSomeThing() the remote interface will be busy and no other calls can be made to the same interface during that time. However, it is possible for some one else to access the remote interface before we reach the doeSomeMore() call. If we want to avoid anyone else to access the interface in between we need to book the interface. We can keep the remote interface booked as long as we want to but it is necessary to unbook it when we are done to let other agents to use the same interface.

```
Program example;
abstract a : interface;

function onRun() : integer;
begin
    a.agreeandbook();  // Will negotiate and book (will never fail, but might take some time)
    a.doeSomeThing();
    a.doeSomeMore();
    a._unbook();
end.
```

For an abstract interface mapped to a remote interface all remote skills are available. A call to a remote skill is considered as asynchronous and is running in parallel. However, a remote interface also have some static synchronous functions:

Function	Description
_book	Will try to book the remote interface
_unbook()	Will try to unbook the remote interface

Function	Description
_isbookedbyme()	Will return done if the remote interface is booked by the interface
_isbooked()	Will return done if the remote interface is booked by any one

It is possible to call _book() manually to try to book an interface. Usually it is a better approach to use the local negotiateandbook() or agreeandbook() function to book instead. However, if _book is used it is possible to write your own logic, e.g., how many retries and when to give up. The global parameter RetryOnBusy and the abstract interface modifier handle will affect the behaviour of _book(). RetryOnBusy can be found in the Preferences of CMAS or can be read or altered by preferencevalue().

Function	RetryOnBusy	Handle	Description
_book()	true	*	will never fail (see note)
_book()	false	yes	will return done or failed
_book ()	false	no	will fail and stop the execution on an error

Note

a call to _book() can fail if the negotiation process fails. To overcome this it is possible to first call negotiate() and check if it is done before calling _book().

Example:

```
Program Processplan;

define handle  RemoteInterface : interface;
begin
    var A : integer;
end;

procedure BookAndAgree();
var negotiated:=false, booked:=false : boolean;
var counter:integer;
begin
    writeln('RetryOnBusy=', preferencevalue('RetryOnBusy', false));

    repeat
        // Will try to negotiate forever with all compatible interfaces
        repeat
            if RemoteInterface.negotiate()=processplan.done then
                negotiated:=true;
            else
                sleep(500); // Wait some time before we try again

        until not negotiated;

        // Will try to book five times after that give up
        for counter:=1 to 5 do
        begin
            if RemoteInterface._book()=processplan.done then begin
                booked:=true;
                break;
            end;
            else
                sleep(500);
        end;
    until not booked;

end.

function onRun() : integer;
begin
    BookAndAgree();

    RemoteInterface.a:=2;
    RemoteInterface.s2();  // Call a skill on the remote interfcae
    RemoteInterface._unbook();

    onRun:=processplan.done;
end.
```

7.3 Variable bounds

Variables can be used to handle values inside an agent. With variable bounds it is possible to limit the values of a variable of type integer, real and string.
Any variable in the modelling tree can have a range expressed as upper and lower bounds. This behaviour is useful when the variable is part of an interface since the range is part of the negotiation process. If two agents negotiate a variable value must match the remote variable range.

7.3.1 Integer and Real

If upper bound is set to 5 and lower bound to 1 the value can only be within the range 1-5, i.e. 1<= x <=5, and the variable will only accept a value in that range during negotiation.

Note

At creation of a variable, the lower and upper bounds are empty and there is no limitation on the variable.

It is possible to write more complex expressions for upper and lower bound.
The basic syntax for a range expression:

Example	Name	Description
1	fixed value	only 1 is accepted
<100	open range lt	all values lower than 100 are accepted
>50	open range gt	all values greater than 50 are accepted
!2	not	all values except 2 is accepted
[>100 <200]	AND	the set of all the elements inside [] must be true
1 | 2 | 3 | >10	OR	one of the element in the set separate by | must be true

Example 1:

```
[<100 >50 !70]      Will give a range less than 100 AND greater than 50 AND NOT the value 70)
```

Example 2:

```
[<10 >5] | [<20 >15]  A dual range example, value must be between 5 AND 10 OR 15 AND 20
```

7.3.2 String

An string can only handle the or | operator when it comes to range expressions.
Example:

```
start | stop  | pause      Will only accept strings with a value of 'start', 'stop' or 'pause'
```

7.3.3 Boolean

Boolean does not implement complex expressions, however it is possible to limit the value by setting upper or lower bound to true or false.

7.4 Goal

In CMAS, a Goal serves a dual purpose:

- Internal Purpose: It represents a objective that an agent seeks to fulfill through collaboration with other agents.
- Interactive Purpose: It also acts as an interface for interacting with other agents. This means that a goal must have an interface name and process plan name to determine how it can be achieved.

Each goal includes built-in behavior that defines a default strategy for how the goal seeks to be fulfilled. This strategy is based on an abstract interface, which enables the agent to initiate negotiations with other agents to find the most suitable remote interface for achieving the goal. During the negotiation process, the agent uses the goal interface name, the desired process plan name, and the goal variables to match the abstract interface with a compatible

implementation on another agent. Once a suitable match is found and negotiated, the requested process plan is executed remotely via the mapped interface.
This behavior is executed locally by the agent and below is the logic for the built-in behavior:

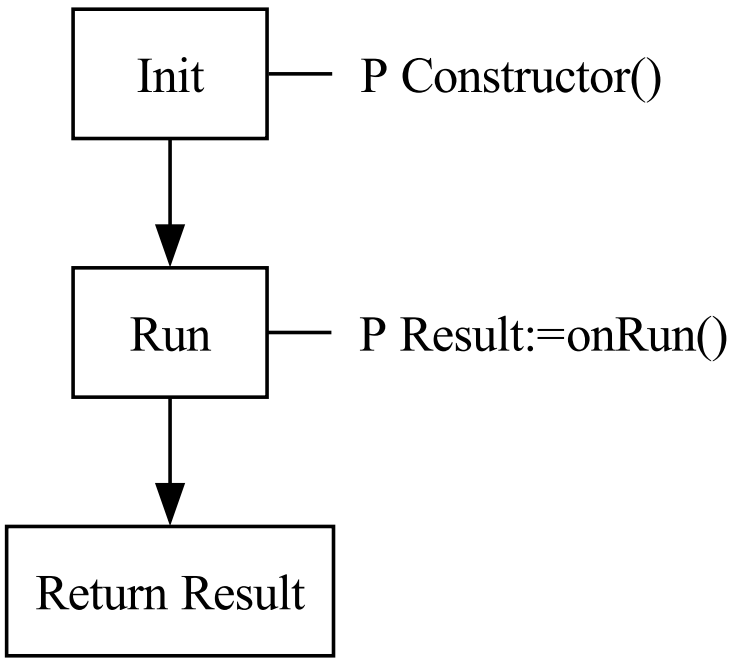
```
Program ProcessplanGoal;
abstract resource : interface;

function onRun() : integer;
begin
    resource.agreeandbook();
    tonRun:=resource.getProcessPlanName();
    resource._unbook();
end.
```

Note

The onRun function can be overridden to replace the built-in default mechanism. It must return the integer value processplan.DONE (2) when the goal is successfully achieved. However, in most cases, the default behavior is sufficient and recommended.

Function	Description
onRun() : integer	Called to execute the goal



7.5 Abstract Interfaces in CMAS

7.5.1 Concept and purpose

An abstract interface represents a capability or set of skills that can be provided by one or more resources in the system, without specifying which exact resource will execute them.
The main idea is to separate what should be done from who performs it. This allows the Planner Agent to dynamically locate and allocate a suitable resource during runtime.
By using abstract interfaces, developers can write code that is resource-independent and goal- or skill-driven, while the system automatically handles resource selection, negotiation, and execution sequencing.
An abstract interface can be seen as a virtual handle or placeholder for a capability. When a skill of an abstract interface is accessed for the first time, the system negotiates with all compatible resources and selects the most suitable one based on cost, availability, and other constraints.
It is also possible to explicitly trigger this negotiation using:
abstract_name.negotiate()

Note

If negotiation is not triggered explicitly, the system performs negotiation automatically the first time the interface is accessed.

7.5.2 Mental Model

A useful way to think about an abstract interface is as a request for a capability rather than a specific resource.
Instead of calling a particular robot or machine directly, the program simply requests a capability through the interface.
For example:

```
Robot.Pick();
Robot.Leave();
```

The program does not specify which robot should perform the operation.
During negotiation, the system searches for available resources that provide the required skills and selects a suitable one.
Once a resource has been selected, the abstract interface acts as a handle to that resource. All subsequent calls through the interface will be executed on the same resource until a new negotiation is performed. This makes it possible to write programs that focus on the required operations while the system automatically manages resource selection and allocation.

7.5.3 Plug & Produce support

Abstract interfaces also support the concept of Plug & Produce.
Since programs are written against capabilities rather than specific resources, new resources can be deployed or undeployed during runtime without requiring changes to the program logic. When a new resource becomes available, it can participate in the negotiation process and may be selected if it provides compatible interfaces with a lower cost or better availability.
Similarly, if a resource is removed or becomes unavailable, the system can renegotiate the abstract interface and bind it to another compatible resource. This makes it possible to build flexible automation systems where resources can be added, replaced, or removed dynamically while the running programs continue to operate.

7.5.4 Declaring abstract interfaces

You declare an abstract interface in Structured Text using the following syntax:
abstract [handle] <name1> [, <name2>, . . .] : interface;
name1, name2 . . . are the names of the abstract interfaces.
Adding the optional handle modifier allows special management of the interface (see interface declaration details).
After declaration, skills can be accessed through the abstract interface, for example:
name.MoveTo(); name.Use();
When the interface is accessed for the first time, the Planner Agent will automatically negotiate and bind the interface to a suitable resource.

7.5.4.1 Example 1: Simple abstract interfaces

```
Program AbstractExample;

{ Abstract remote interfaces }
abstract Robot, Resource : interface;

Begin
    Robot.MoveTo();
    Resource.Use();
    Resource.Signal := TRUE;
    Resource.Motor();
End.
```

7.5.4.2 Example 2: Abstract interface with handle modifier

```
Program AbstractHandleExample;

abstract handle transport : interface;
var returnValue: integer;

function onRun() : integer;
begin
    transport.negotiateandbook();
    transport.moveRight(returnValue);

    repeat
        sleep(100);
    until returnValue = processplan.running;

    transport._unbook();
    onRun := returnValue;
end.
```

7.5.5 Defining local interfaces

You can also define local interfaces to describe the variables and skills required for matching with remote interfaces. This is useful when the interface is not pre-configured in the system.
Syntax 1: Define a new local interface
define [handle] <name> : interface; begin var x, y, z : real; function move; end;

7.5.5.1 Example

```
Program DefineExample1;

define aRobot : interface;
begin
    var x, y, z : real;
    function move;
end;

begin
    aRobot.x := 2;
    aRobot.y := 3;
    aRobot.z := 0;

    aRobot.move();
end.
```

Note

For more details about declaring abstract interfaces see interface declaration details.

7.5.6 Resource binding after negotiation

Once an abstract interface has been negotiated and bound to a resource, all subsequent skill calls on that interface will be executed on the same resource.
For example, if an abstract interface name represents a robot:

```
name.Pick();
name.Leave();
```

The Planner Agent ensures that both Pick() and Leave() are executed on the same physical robot.
This guarantees consistent state and coordination, which is essential for tasks that require multiple sequential operations on the same resource.

The resource binding remains valid until a new negotiation is explicitly triggered or the interface is reset by the system.

7.5.7 Interface validation during negotiation

During negotiation, the Planner Agent verifies that a candidate resource is compatible with the abstract interface.
This compatibility check includes:
Variable matching
All variables defined on the abstract interface must exist on the candidate remote interface. Their types and value ranges must also be compatible.
Skill availability
All skills that are called through the interface must exist on the remote interface and be executable.
Only resources that satisfy these conditions are considered valid candidates.
If multiple compatible resources are found, the Planner Agent selects the resource with the lowest cost. This ensures that abstract interfaces always map to resources that can fully satisfy the expected capabilities and constraints.

7.5.8 Local abstract interfaces

The primary purpose of an abstract interface is to enable dynamic binding to the most suitable remote interface. However, in some situations, it is useful to bind to a local interface within the same agent.
This pattern can be used when multiple external interfaces act as receivers of skill calls, while the actual execution logic is centralized in a single internal interface. The internal interface then acts as the owner of the real implementation, ensuring consistent behavior and simplified coordination.
By routing all calls through a local internal interface, it is possible to:

- Centralize execution logic
- Maintain consistent state handling
- Coordinate multiple incoming requests in a more controlled manner

This pattern is particularly useful in combination with interfaces that expose a ParallelCapacity greater than one. In such cases, multiple interface instances may receive concurrent skill calls, while the internal interface ensures that execution is properly synchronized and scheduled according to the resource agent's logic.
Using the `define local` modifier is a convenient pattern to explicitly create an interface that is intended to resolve locally rather than through remote negotiation.

```
Program Processplan;

var counter := 0 : integer;

define local internal : interface;
begin
    var PartName : String;
    function run;
end;

function onRun() : integer;
begin
    internal.PartName := inttostring(agent.counter);
    internal.run();
    onRun := processplan.done;
end.
```

Note

When used together with ParallelCapacity, multiple external interface instances may invoke the same internal interface concurrently. The resource agent is then responsible for defining how these requests are prioritized and executed.

7.6 Advanced Interface Concepts

7.6.1 Parallel Interfaces

An interface can serve one remote interaction at a time. When booking is used (which is the preferred approach in many situations), the interface is reserved and other agents must wait until it is unbooked. In most cases, this

behavior is sufficient since other agents will simply wait until the interface becomes available. However, in scenarios where multiple requests must be handled concurrently, it is possible to expose multiple identical interface instances. This can be achieved by setting the ParallelCapacity property on a resource interface. The ParallelCapacity defines how many parallel instances of the interface can be active at the same time. During runtime, the agent will create multiple identical interface instances based on this value. Each instance behaves as an independent interface and can be negotiated and booked separately. This allows multiple agents to interact with the same type of resource in parallel, while still maintaining controlled access to each individual interface instance.

7.6.2 Interface Queues

When multiple agents attempt to access a single interface, the onNegotiate() function is called to decide whether a request should be accepted or rejected. By default, requests are processed in a first-come, first-served manner. However, the resource agent can implement custom logic to control how requests are prioritized and executed. By setting the interface property QueueRequest to *true*, the global interface variable `queue` (as defined in queue.inc, see structure text include) is enabled. The members of the queue are listed in the table below. When enabled, the onQueue() function is called before onNegotiate(), allowing the agent to inspect and manage incoming requests. The onNegotiate() function can then use this information to make more informed decisions.

Name	Type	Description
agentreference	reference	Reference to the agent that initiated the negotiation process
interfacereference	reference	Reference to the interface on *agentreference* that is requesting access
time	integer	Relative time (ms) when the negotiation request was created. Can be used to compare requests
value	integer	Default value is 0. Can be used freely to support custom prioritization logic

This mechanism allows the resource agent to determine execution order based on priority, cost, timing, or other criteria. To avoid resource conflicts and ensure consistent execution, all accepted requests can be routed to a local internal interface that coordinates the execution.The queue grows dynamically as more agents attempt to negotiate up to a max size. The current negotiation attempt is always placed at the top of the queue.

Note

The negotiation mechanism is reactive and operates on the current request only. While the queue provides visibility of multiple pending requests, the onNegotiate() function can only accept or refuse the request currently being evaluated. Multiple negotiation attempts from the same agent and interface do not create duplicate entries in the queue. Instead, the existing entry is updated and may move within the queue based on the new request.

It is not possible to directly select or accept a request at another position in the queue then the first one (top). For example, if a more suitable request exists deeper in the queue, the current request must be refused. The preferred request must then initiate a new negotiation attempt in order to be reconsidered.
This implies that prioritization strategies must be implemented indirectly by:

- Refusing lower-priority requests
- Allowing higher-priority requests to retry and reach the top of the queue

As a result, scheduling behavior emerges over time rather than being explicitly controlled in a single decision step. This behavior reflects the real-time nature of the system, where decisions are made incrementally as requests arrive.

Note

Queue entries are not explicitly removed when a request is refused. Instead, entries become outdated over time. The `time` field can be used to determine the age of a request and to ignore stale entries in scheduling logic.

7.6.2.1 Recommended Scheduling Pattern

Due to the reactive nature of the negotiation mechanism, scheduling decisions must be implemented incrementally. Since onNegotiate() can only accept or refuse the current request, it is not possible to directly select a specific

request from the queue.
A recommended pattern is therefore to combine queue inspection, selective acceptance, and retry-based prioritization.

7.6.2.1.1 Pattern Overview

1. Enable queue inspection
   Set QueueRequest to *true* to access the global queue and allow onQueue() to review incoming requests.
2. Evaluate priority
   In onQueue() and/or onNegotiate(), evaluate all pending requests using available data such as:
   - time (arrival order)
   - value (custom priority)
   - agentreference or interfacereference (context-specific logic)
3. Accept only the best candidate
   In onNegotiate(), compare the current request with the rest of the queue:
   - If the current request is the preferred one → accept
   - Otherwise → refuse
4. Rely on retry behavior
   Refused agents will retry negotiation. Over time, higher-priority requests will reach the top of the queue and be accepted.
5. Coordinate execution
   Route accepted requests through a local internal interface to ensure consistent execution and avoid resource conflicts.

7.6.2.2 Example Strategy

A simple priority strategy could be:

- Prefer the request with the highest cost (stored in `value`)
- Consider only negotiations within a timeout time
- Wait until there is a queue of agents

This pattern allows the resource agent to implement flexible scheduling strategies within the constraints of the negotiation model. By combining queue visibility with selective acceptance and retry-based prioritization, it is possible to approximate optimal execution order in a fully distributed and reactive system.

7.6.2.2.1 Example: Queue-based scheduling with cost selection

In the following example, a resource interface waits until three agents have initiated negotiation. Each agent provides a cost value, which is stored in the queue.
When all three requests have arrived within a given time window, the resource agent selects the request with the highest cost and accepts it, while the others are refused and must retry.

```
$include 'system:queue.inc'
program Queue;

var constant TIMEOUT:=1050 : integer;

procedure printQueue;
var i:=1:integer;
begin
    writeln('queue ', startindex(queue), ' - ', endindex(queue));
    writeln('|INDEX | Object name         | Interface ID                                                | Time
      | Value  |');
    writeln('|------|---------------------|-------------------------------------------------------------|---------|--------|');
    foreach q in queue do
    begin
        write('| ', space(5, inttostring(i)), '| ');
        write(space(20, q.agentreference.name + '.' + q.interfacereference.name), '| ');
```

```
        write(space(55, q.interfacereference.ID), ' | ');
        write(space(8, inttostring(q.time)), '|');
        write(space(8, inttostring(q.value)), '|');
        writeln();
        i:=i+1;
    end;
end.

// Wait for num agents that negotiate with us
function waitForAll(num:integer) : boolean;
var counter:=0, items:=0: integer;
var now : integer;
begin
    now:=queue[startindex(queue)].time;
    foreach item in queue do
    begin
        items:=items+1;
        if item.value<>0 AND (item.time + TIMEOUT)>now then
            counter:=counter+1;
    end;

    if items = num AND counter = items then begin
        waitForAll:=true;
    end;
    else waitForAll:=false;
end.

// Check if the first item has the highest cost (best)
function isFirstBest() : boolean;
var  firstCost: integer;
begin
    firstCost:=queue[startindex(queue)].value;
    foreach item in queue do
    begin
        if item.value>firstCost then begin
            isFirstBest:=false;
            exit;
        end;
    end;

    isFirstBest:=true;
end.

function onNegotiate(Planning : boolean; a: Reference; i:Reference): integer;
begin
    // Wait for all 3 parts to be in the queue
    if waitForAll(3) then begin
        // Check if the one that negotiataes now is the best one
        if isFirstBest() then begin
            // Accept the negotiation
            onNegotiate:=1;
            queue[1].value:=-1;
            exit;
        end;

    end;

    // printQueue();

    // Refuse
    onNegotiate:=-1;
end.

// Executed just before onNegotiate
function onQueue():boolean;
begin   // Save remote interface variable value cost in the queue list as value
    queue[startindex(queue)].value:=remoteinterface.cost;
end.
```

Note

This example demonstrates how scheduling decisions can be implemented using queue inspection and selective acceptance. Since only the current request can be accepted, lower-priority requests are refused and must retry until the preferred request reaches the top of the queue.

The TIMEOUT is used to ensure that only recent requests are considered. This prevents outdated requests from blocking the scheduling logic in a real-time system.

Chapter 8

Ontology

8.1 Introduction

Object-oriented data modelling is a technique used in CMAS to represent and organize data as a collection of objects. Overall, object-oriented data modelling is a powerful approach for designing systems that closely align with real-world entities and relationships. Every object within CMAS comprises a combination of *properties*, *relationships* and *methods*, alongside inherited properties and relationships. As an example, if you have an Agent object in the CMAS system, it will have all the properties and relationships that are defined for the more general Entity object. This is a common concept in object-oriented data modelling, where specialised objects (like Agent) inherit attributes from more general classes (like "Entity"). A property represents a value that can be used by the object it self or the user, while a relation denotes a link to another object. Relations can handle new relations dynamically if they are compatible with the specific type of connection. A relation can be a direct relation to a child object or a reference relation to any compatible object in the system. A method is a predefined program or sequence of instructions that can be executed during runtime to perform a specific task. All methods defined in the ontology can be overridden, meaning their default behaviour can be redefined using Structured Text.
All objects in a model are visible in the Modelling tree viewer in CMAS and all properties are visible in the properties viewer. Relations categories are visible as sub-folders in the modelling tree holding child objects. The available objects in CMAS are based on a classes defined in the CMAS ontology. A class serves as a blueprint or template for creating objects and it defines the structure that objects of that class will have.
For a brief overview of the CMAS ontology see the figure below (or see the inheritance diagram for classes Entity). The diagram in Figure 1 provides a high-level overview of the CMAS ontology. It highlights the foundational `Entity` class and the main abstract pillars of the model: agents, interfaces, variables, and process plans.
All objects in the CMAS ontology inherit from `Entity`, which defines the common identity, naming, and lifecycle-related properties. The overview intentionally omits detailed relationships and concrete specialisations in order to present a clear conceptual structure of the system.

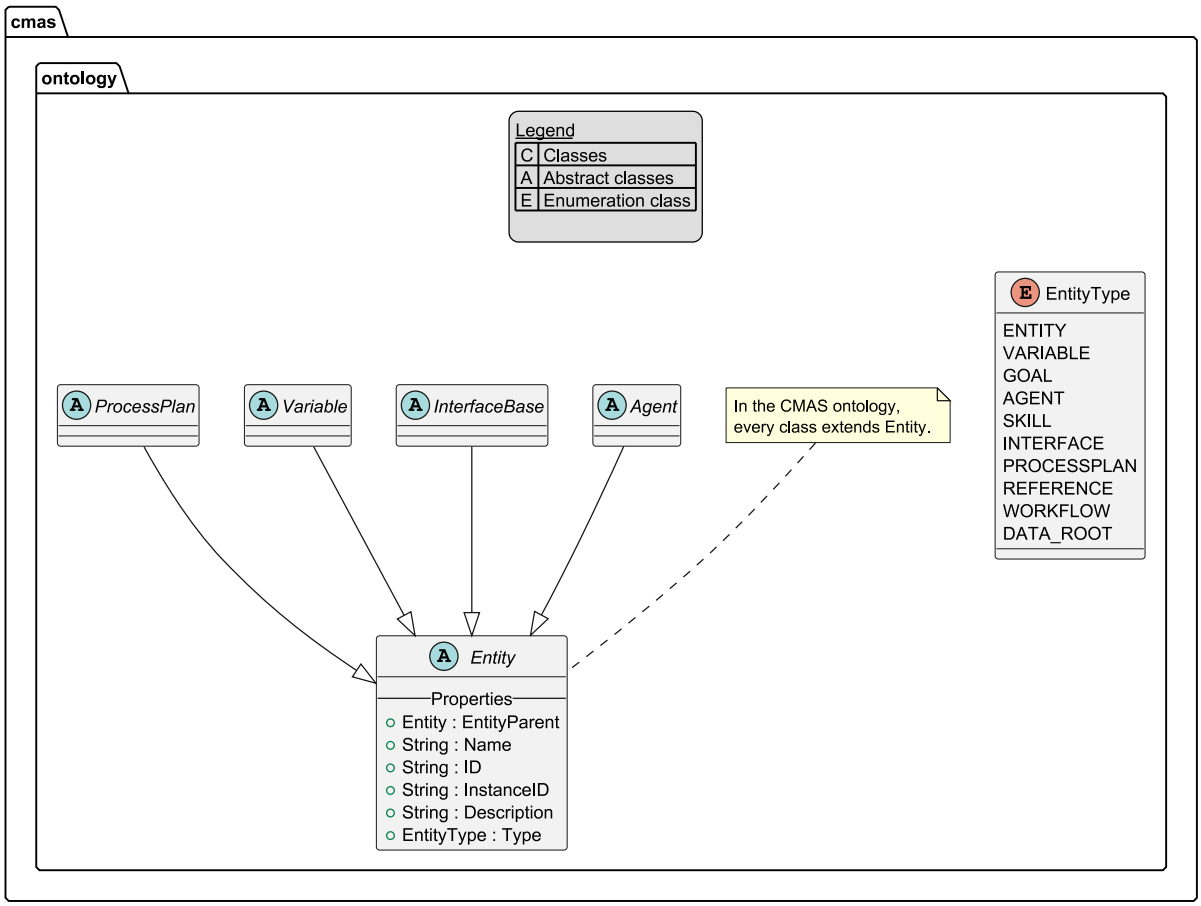


Figure 1. CMAS ontology overview.

Figure 2 describes the agent and interface hierarchy in the CMAS ontology. Agents represent active or passive entities that participate in execution, while interfaces define the interaction points between agents, resources, and goals.

The diagram shows how agent behavior is structured through inheritance, lifecycle states, and actions, as well as how interfaces and goals enable negotiation, coordination, and task execution within the system.

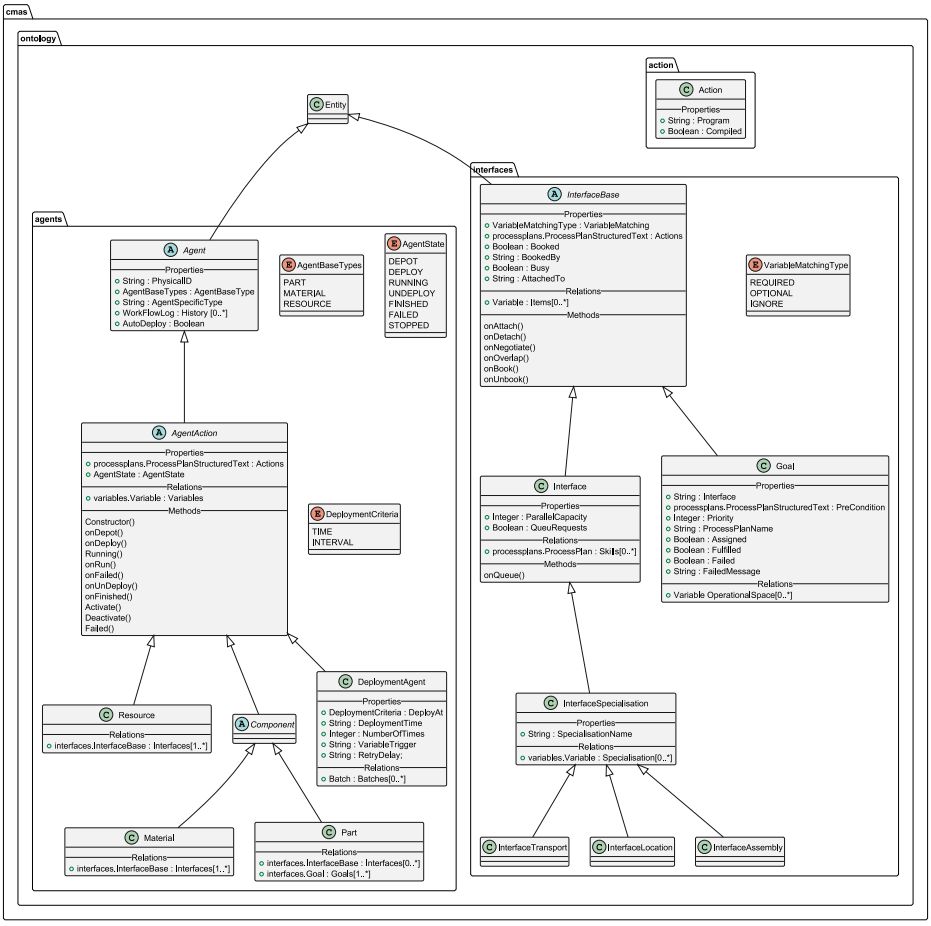


Figure 2. CMAS ontology.

The diagram in Figure 3 illustrates the hierarchy of process plans and variables used in CMAS to describe executable behavior and data exchange.
Process plans define how work is performed, either through structured text, goal-based charts, or external implementations, while variables provide a typed and extensible mechanism for storing and exchanging values during modelling and runtime.
Together, process plans and variables form the core building blocks for decision-making, planning, and execution logic in CMAS.

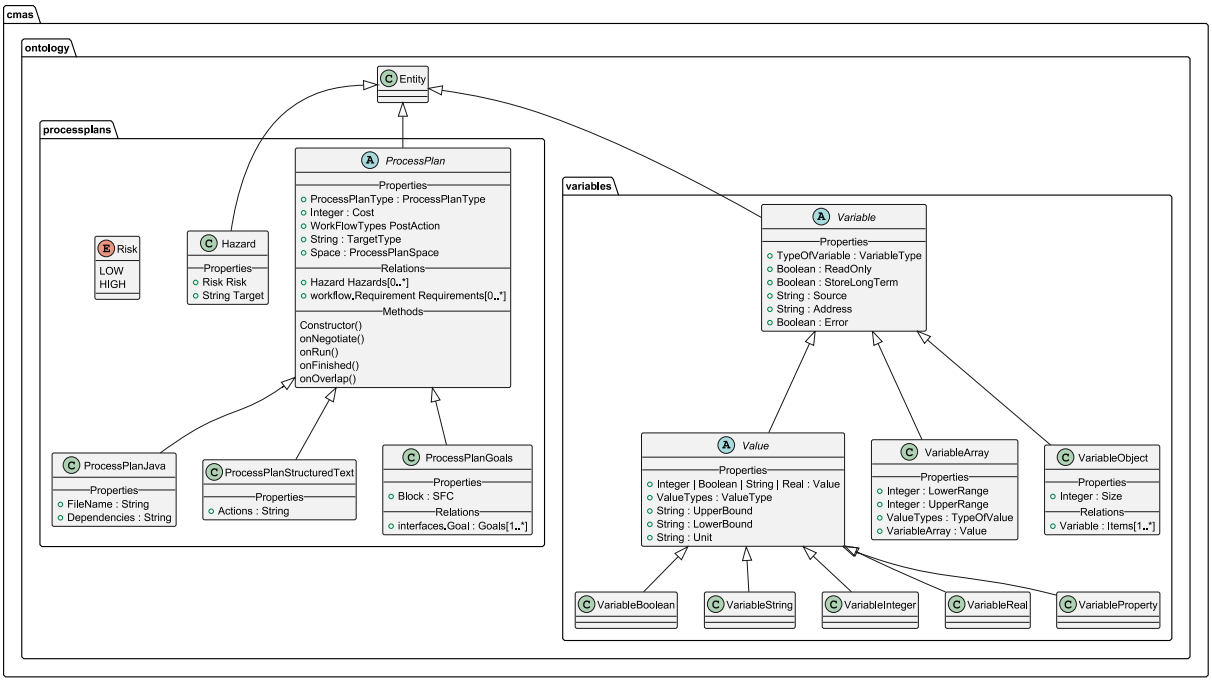


Figure 3. CMAS ontology.

Figure 4 presents supporting ontology objects related to workflow, deployment, configuration, and documentation.
Workflow objects capture execution history and requirements, while batch and deployment-related objects define how entities are instantiated and deployed at runtime. Additional objects such as notes and settings are used to document and configure the model but do not directly affect execution logic.

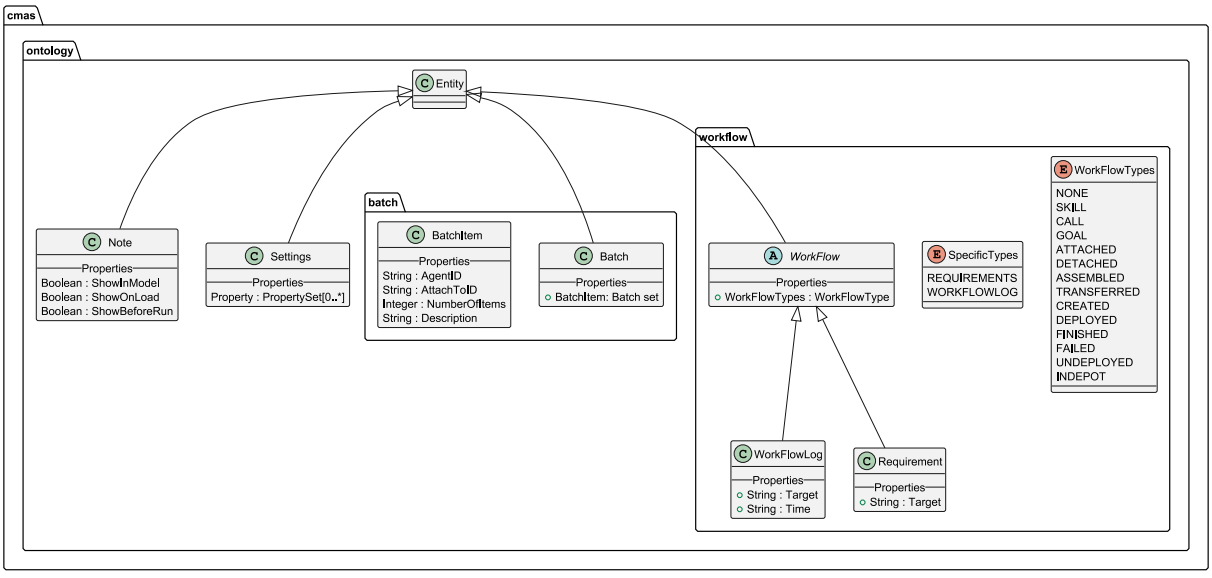


Figure 4. CMAS ontology.

The diagram in Figure 5 shows runtime-specific classes used by the CMAS execution environment.
These classes are not part of the modelling ontology but represent runtime services, execution domains, and internal program structures used to interpret and execute process plans. They are included here for completeness and to illustrate the separation between modelling concepts and runtime implementation but lack details.

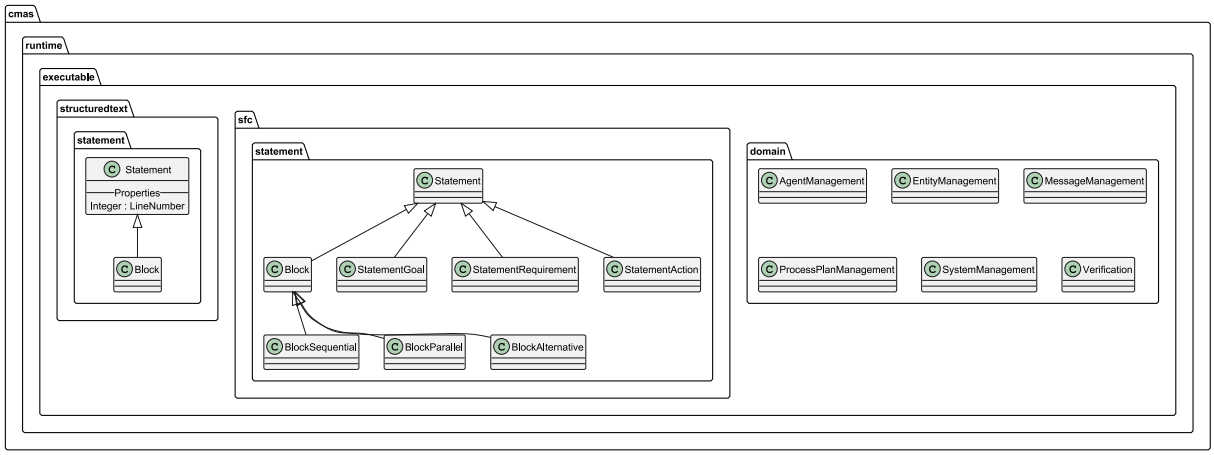


Figure 5. CMAS ontology

8.2 How to Read the CMAS Ontology

- Classes describe modelling-time objects.
- Properties are persisted values editable by the user.
- Relations are typed links to other entities.
- Runtime properties are read-only and only valid during execution.
- Abstract classes cannot be instantiated.

8.3 cmas.ontology Objects

All main objects in the CMAS ontology and their specific properties are described in the following section.

8.3.1 Entity

The *Entity* class is the foundation of the CMAS ontology and defines the common modeling structure used by all objects in CMAS. Every object in the ontology inherits from Entity, including agents, interfaces, variables, process plans, workflow objects, and configuration objects. An entity consists of two fundamental elements:
Properties – named values that define the behaviour, configuration, or state of an object. *Relations* – typed collections of other objects that define how entities are connected.
Properties are predefined by the ontology and can be edited by the user during modeling. By changing property values, the behavior and configuration of an object can be customized without modifying the underlying object definition.
Relations act as typed containers that organize connections to other entities. Each relation only accepts specific object types defined by the ontology. For example, a Resource contains an Interfaces relation that can hold one or more interface objects, while a Part contains a Goals relation that can hold goal objects. Relations therefore define both the structure of the model and the valid connections between objects.
Because every object in CMAS is an *Entity*, the same modeling principles apply throughout the system. Regardless of whether the object represents a resource, a process plan, a variable, or a goal, it is configured through properties and connected to other objects through relations.
Entity itself is an abstract class and cannot be instantiated directly.

Table Entity class attributes

Property	Description	Type
Entity::EntityParent	A parent entity object. If parent is set it is possible to inherit property values from that parent. When parent is set to a valid entity a check box will be visible for all properties that can be inherited. If the check box is checked the local value will be override by the parent value.	Entity
Entity::Name	Name is a user friendly name of an entity. It is possible to refer to an entity by name, however, a name must not be unique two entities can share the same name.	String
Entity::ID	An ID is a unique identifier of an entity during modelling time. The ID is automatically generated by CMAS when a new entity is created. However, it is possible to set the ID to a more readable form to be able to refer an ID but note that it must be unique. You can use the Verify IDs to check if all IDs are unique and also to automatically generate unique IDs.	String
Entity::InstanceID	An InstanceID is an identifier of an entity during runtime and must be unique in the system. The InstanceID is automatically generated by CMAS during runtime and can not be set by the user.	ReadOnly String
Entity::Description	A user defined description of the entity. The description is not used by the system but it is visualised in many dialogs in CMAS and can help to better document specific entities.	String
Entity::Type	The base type of the object. The value is set automatically when an object is created.	ReadOnly EntityType

8.3.2 Note

A Note is used to document important information or highlight considerations within a model. It serves as a user-facing notification and is ignored by the system during runtime. Notes are typically used for providing additional context, explanations, or reminders to the users working with the model.

Table Note class attributes

Property	Description	Type
Note::ShowInModel	If true the note will be visible in the schema viewer.	Boolean
Note::ShowOnLoad	If enabled, this note will be shown in a popup window when the model is loaded.	Boolean
Note::ShowBeforeRun	If enabled, this note will be shown in a popup window before the model run begins.	Boolean

8.3.3 Settings

The *Settings* object provides a centralized view of selected properties from across the model. Its purpose is to simplify configuration by allowing important parameters to be collected and edited from a single location.
Rather than navigating through the model to locate individual objects and their properties, the user can add references to selected properties in the *Settings* object. This creates a convenient configuration interface for frequently adjusted values.
For example, a model may contain several process plans located in different parts of the model hierarchy. By adding their Cost properties to the *Settings* object, all costs can be viewed and modified from one place without needing to locate each individual process plan.
The *Settings* object does not introduce new behavior and does not store independent values. Instead, it provides direct access to properties that already belong to other entities in the model. Any changes made through Settings are immediately reflected in the original objects.
Typical uses include:
Collecting configurable process parameters. Exposing planning and optimisation settings. Creating a simplified configuration view for end users. Grouping frequently modified properties in a single location.
The *Settings* object is intended purely as a modeling and configuration aid and has no direct impact on runtime execution beyond the property values it exposes.

Table Settings class attributes

Property	Description	Type
Settings::PropertySet	The set of properties that can be configured.	Property []

8.3.4 Folder

A folder is an organizational entity used to structure the model in a way that enhances readability and understanding. It helps group related objects together, making it easier to navigate and manage complex models.

Table Folder class attributes

Property	Description	Type
Folder::MakeInstances	If true all items in the folder will be create as instances during runtime.	Boolean

---

8.4 cmas.ontology.processplans

8.4.1 ProcessPlan

A ProcessPlan defines executable behavior in CMAS. It describes how a task, operation, or activity can be performed and serves as the common base class for all process plan implementations.
Process plans are used throughout the system to represent skills, agent actions, goal execution logic, and other executable functionality. During planning and negotiation, process plans can be evaluated based on properties such as cost, requirements, and hazards.
ProcessPlan is an abstract class and cannot be instantiated directly. Instead, executable behaviour is implemented using one of its specialised subclasses, such as ProcessPlanStructuredText, ProcessPlanGoals, or ProcessPlan↩ Java.

Table ProcessPlan class attributes

Property	Description	Type
ProcessPlan::ProcessPlanType	Process plan type.	processplans.ProcessPlanType
ProcessPlan::Cost	The cost to execute the plan.	Integer
ProcessPlan::PostAction	The outcome of executing the process plan, this is used by the built in planner.	workflow.WorkFlowTypes
ProcessPlan::TargetType	If there are any risks what is the target type.	String
ProcessPlan::Space	Required space.	ProcessPlanSpace

Table ProcessPlan class relations

Relation	Description	Type	Cardinality
ProcessPlan::Hazards	All related hazards associated with the execution of the process plan.	Hazard	[0..∗]
ProcessPlan::Requirements	All requirements that must be fulfilled before the process plan can be executed.	workflow.Requirement	[0..∗]

Table ProcessPlan class methods

Method	Description
ProcessPlan::Constructor()	Called on the creation of the runtime object
ProcessPlan::onNegotiate()	Called when someone wants to negotiate
ProcessPlan::onRun()	Called when someone wants to execute the process plan
ProcessPlan::onFinished()	Called when the process plan is done
ProcessPlan::onOverlap()	Called to check if the process plan overlaps another process plan in the terms of risks

8.4.2 ProcessPlanGoals

A ProcessPlanGoals is a process plan that defines behavior through a collection of goals organized in a goal chart. Rather than describing execution as a sequence of programming statements, the process plan specifies what goals should be achieved and in what order they may be executed. The execution logic is represented by an SFC-like (Sequential Function Chart) structure that can contain goals, actions, conditions, parallel branches, and alternative paths. This allows complex workflows to be modeled graphically while keeping the execution logic easy

to understand and maintain.
Each goal in the process plan represents a desired outcome that must be fulfilled. During execution, the CMAS planner determines how goals can be achieved by negotiating with available resources and selecting suitable process plans. *ProcessPlanGoals* is particularly useful when modeling workflows, production sequences, and coordination logic where the focus is on achieving objectives rather than implementing detailed algorithms.

Table ProcessPlanGoals class attributes

Property	Description	Type
ProcessPlanGoals::SFC	A process plan chart describing the order of the goals and actions.	runtime.executable.sfc.statement.Block

Table ProcessPlanGoals class relations

Relation	Description	Type	Cardinality
ProcessPlanGoals::Goals	All ProcessPlanGoals must at least have one goal. The Process↩ PlanGoals::SFC property describes how to execute all goals in the relation.	interfaces.Goal	[1..∗]

8.4.3 ProcessPlanStructuredText

A ProcessPlanStructuredText defines a process plan written in Structured Text, a high-level programming language. Compared to ProcessPlanGoals, Structured Text offers more advanced and flexible programming features, enabling the implementation of more complex process logic.

Table ProcessPlanStructuredText class attributes

Property	Description	Type
ProcessStructuredText::actions	A structured text program.	String

8.4.4 Hazard

A hazard object represents a potential hazard associated with a process plan. Each hazard has a target, which is a user-defined name. Hazards are considered by the planner or can be manually interpreted by agent actions. Hazards are considered during negotiation and planning.

Table Hazard class attributes

Property	Description	Type
Hazard::Risk	A risk.	Risk
Hazard::Target	A user defined target for the risk.	String

8.4.5 Risk

A risk represents a potential risk associated with a Hazard.

Table Risk enum values

Property	Description	Type
Risk::LOW	A low risk is ignored by the planner and during negotiation.	LOW
Risk::HIGH	A high risk is considered by the planner and during negotiation.	HIGH

8.5 cmas.ontology.agent

All agents in CMAS ontology are based on the object Agent.

Table Agent class attributes

Property	Description	Type
Agent::PhysicalID	A user defined physical ID of the hardware that the agent represents. The PhysicalID is not used by the system but can be used for identifications or in programs.	String
Agent::AgentBaseType	The agent base type.	ReadOnly AgentBaseTypes
Agent::AgentSpecificType	The agent specific type is user defined string that can be used to determine what type of agent its. This property is not used by the system but can be utilised in a program to make decisions.	String
Agent::AutoDeploy	If this is false the agent will not be deployed at run or when deployed all is pressed.	Boolean

8.5.1 AgentAction

AgentAction implements an overall controller and states for all agents according to actions. Agent action has a relation named Variables that accept all ValueTypes.

Table AgentAction class attributes

Property	Description	Type
AgentAction::Actions	A structure text program that implements actions.	processplans.ProcessPlanStructured↩ Text
AgentAction::AgentState	Agent current state during runtime, i.e., INIT, DEPOT, DEPLOY, RUNNING, UNDEPLOY, FINISHED, FAILED, STOPPED.	AgentState

Table AgentAction class relations

Relation	Description	Type	Cardinality
AgentAction::Variables	Agent variables than can be access from all skills and actions.	variables.Variable	[0..∗]

Table AgentAction class methods

Method	Description
AgentAction::Constructor()	Called on the creation of the runtime object
AgentAction::onDepot()	Called when the runtime object enters the Depot
AgentAction::onDeploy()	Called when the runtime object is deployed
AgentAction::Running()	Called constantly when the runtime object is running
AgentAction::onRun()	The onRun procedure is called once when the runtime object enters the Running state
AgentAction::onFail()	The onFail procedure is called once if the agent enters the Failed state
AgentAction::onUndeploy()	The onUndeploy procedure is called once the agent is deactivated or undeployed
AgentAction:: onFinished()	The onFinished procedure is called once a part agent is finished
AgentAction:: Activate()	If the agent is in the Depot state and Activate returns true the agent will be deployed
AgentAction:: Deactivate()	If the agent is in the Running state and Deactivate returns true the agent will be transfered to the UnDeploy state
AgentAction:: Failed()	If the agent is in the Running state and Failed returns true the agent will be transfered to the failed state

8.5.2 Resource

A `Resource` agent is a passive entity within the system, designed to be accessed and used as a resource. It can interact with other components through its defined interfaces and skills. The `Resource` agent has a relationship called `Interfaces`, which can accept any type of `InterfaceType`. This allows for flexible interaction with other entities and agents in the system.

Table Resource class attributes

Property	Description	Type

Table Resource class relations

Relation	Description	Type	Cardinality
Resource::Interfaces	All interfaces that can be used for negotiation and that can expose skills.	interfaces.Interface	[1..∗]

8.5.3 Material

A Material agent is a passive agent that can be used as a subpart when it comes to assembly. A Material agent has a relation named Interfaces that accept all InterfaceTypes.

Table Material class attributes

Property	Description	Type

Table Material class relations

Relation	Description	Type	Cardinality
Material::Interfaces	All interfaces that can be used for negotiation.	interfaces.Interface	[1..∗]

8.5.4 Part

A Part agent is an active agent that will try to fulfil all goals. A Part agent has a relation named Goals that accept all GoalTypes.

Table Part class attributes

Property	Description	Type
Part::ProcessPlan	A goal chart that express the order of the goals.	ProcessPlanGoals

Table DeploymentAgent class relations

Relation	Description	Type	Cardinality
Part::Interfaces	All interfaces that can be used for negotiation.	interfaces.Interface	[0..∗]
Part::Goals	All goals that can be used in the goal chart of the agent.	interfaces.Goal	[1..∗]

8.5.5 DeploymentAgent

The deployment agent will deploy all batches based on *DeploymentTime* or *VariableTrigger*. If *VariableTrigger* is empty, it will be ignored, and if *DeploymentTime* is less than 0, it will also be ignored. Otherwise, deployment occurs when the time is reached or the trigger variable is changed to true. For more complex trigger logic, you can utilise the Agent action and the Running procedure to modify the trigger variable's value.

Table DeploymentAgent class attributes

Property	Description	Type
DeploymentAgent::DeployAt	If set to TIME, the first batch will be deployed after DeploymentTime, with each subsequent batch deployed at intervals of DeploymentTime. If set to INTERVAL the first batch will be deployed at time 0 with each subsequent batch deployed at intervals of DeploymentTime.	DeploymentCriteria
DeploymentAgent::DeploymentTime	The time used by DeployAt. If no suffix is provided, the time is interpreted in milliseconds (e.g., 500 means 500 milliseconds). Valid suffixes include: h (hour), m (minutes), s (seconds), and ms (milliseconds). For example, 20s represents 20 seconds. If set to a negative value the time will be ignored.	String
DeploymentAgent::NumberOfTimes	Number of times to repeat the deployment, (0 = infinite number of times).	Integer

Property	Description	Type
DeploymentAgent::VariableTrigger	The name of a local agent variable that, when it becomes true, will trigger a deployment. If it is left empty the TriggerVariable will be ignored.	String
DeploymentAgent::RetryDelay	Retry delay after a failed deployment or attachment attempt. A failure may occur due to an error or because the target resource is temporarily unavailable (for example, already occupied by another item). If the value is greater than 0, the deployment will be retried after the specified delay when such a failure occurs. If the value is 0 or less, the deployment will stop immediately and report the failure. The default unit is milliseconds, but h (hours) or m (minutes) may be used as suffixes.	String

Table DeploymentAgent class relations

Relation	Description	Type	Cardinality
DeploymentAgent::Batches	The batch objects specifies all agents to be deploy.	Batch	[0..∗]

8.6 cmas.ontology.variables

8.6.1 Variable

Table Variable class attributes

Property	Description	Type
Variable::VariableType	Type of variable.	ReadOnly TypeOfVariable
Variable::ReadOnly	If the variable is read only during run time.	Boolean
Variable::StoreLongTerm	If the variable value is stored in an external database or not.	Boolean
Variable::Source	Adapter source field, see adapters.	String
Variable::Address	Adapter address field, see adapters.	String
Variable::Error	If an error has occurred in the adapter during runtime.	ReadOnly Boolean

8.6.2 Value

Value is an abstract class to store values for variables.

Table Value class attributes

Property	Description	Type
Value::Value	The variable value.	Integer | Boolean | String | Real
Value::ValueType	The variable value type.	ValueTypes
Value::UpperBound	The value UpperBound, see variable bounds.	String
Value::LowerBound	The value LowerBound, see variable bounds.	String
Value::Unit	The value unit of measurement.	String

8.6.3 VariableBoolean

A boolean variable.

Table VariableBoolean class attributes

Property	Description	Type

8.6.4 VariableInteger

An integer variable.

Table VariableInteger attributes

Property	Description	Type

8.6.5 VariableReal

A real variable.

Table VariableReal attributes

Property	Description	Type

8.6.6 VariableString

A string variable.

Table VariableString attributes

Property	Description	Type

8.6.7 VariableArray

An array variable.

Table VariableArray attributes

Property	Description	Type
VariableArray::LowerRange	The lower range of the array.	Integer
VariableArray::UpperRange	The upper range of the array.	Integer

8.6.8 VariableObject

An variable object is a user defined type that has a relation named Items that accept all ValueTypes.

Table VariableObject attributes

Property	Description	Type
VariableObject::Size	Number of items in the relation Items.	ReadOnly Integer

Table VariableObject class relations

Relation	Description	Type	Cardinality
VariableObject::Items	All items that belong to the object. An item can be of any Variable type, e.g., VariableInteger or VariableReal, it can even be another VariableObject.	Variable	[1..∗]

8.7 cmas.ontology.interfaces

8.7.1 InterfaceBase

Table InterfaceBase attributes

Property	Description	Type
InterfaceBase::VariableMatching	Variable matching for the interface, see interfaces.	VariableMatchingType
InterfaceBase::Actions	Interface actions, see interface action.	processplans.ProcessPlan↩ StructuredText
InterfaceBase::Booked	Runtime property.	ReadOnly Boolean
InterfaceBase::BookedBy	Runtime property.	ReadOnly String
InterfaceBase::Busy	Runtime property.	ReadOnly Boolean
InterfaceBase::AttachedTo	Runtime property.	ReadOnly String

Table InterfaceBase class relations

Relation	Description	Type	Cardinality
InterfaceBase::Variables	Interface variables than can be access from all skills and actions and that are used during negotiations.	variables.Variable	[0..∗]

Table InterfaceBase class methods

Method	Description
InterfaceBase::onAttach()	Called when an interface has been attached
InterfaceBase::onDetach()	Called when an interface has bee detached
InterfaceBase::onTransfer()	Called when post action transfered is executed

Method	Description
InterfaceBase::onUnbook()	Called when asked to unbook
InterfaceBase::onUnbook()	Called when asked to unbook
InterfaceBase::onNegotiate()	Called during negotiation
InterfaceBase::onBook()	Called when asked to book
InterfaceBase::onOverlap ()	Called to check for overlap

8.7.2 Interface

An interface object that has a relation named Skills that accept all SkillTypes.

Table Interface attributes

Property	Description	Type
Interface::ParallelCapacity	Number of parallel instances created during runtime	Integer
Interface::QueueRequests	All negotiation requests are store in a queue and onQueue() is used	Boolean

Table Interface class relations

Relation	Description	Type	Cardinality
Interface::Skills	All published skills through the interface.	processplans.ProcessPlan	[0..∗]

8.7.3 Goal

A goal object.

Table Goal attributes

Property	Description	Type
Goal::Interface	A interface name used for a goal during negotiation.	String
Goal::PreCondition	Precondition to enable the goal during runtime.	processplans.ProcessPlanStructured↩ Text
Goal::Priority	Goal priority.	Integer

Property	Description	Type
Goal::ProcessPlanName	The name of a process plan (or skill) that can assist to reach the goal.	String
Goal::Assigned	Runtime variable to indicate if the goal is assigned to a process plan instance that can solve the goal.	Boolean
Goal::Fulfilled	Runtime variable true if goal is fulfilled.	Boolean
Goal::Failed	Runtime variable true if goal failed.	Boolean
Goal::FailedMessage	Runtime variable a string explaining the reason for a failure.	ReadOnly String

Table Goal class relations

Relation	Description	Type	Cardinality
Goal::OperationalSpace	Operational space.	variables.Variable	[0..∗]

8.7.4 InterfaceSpecialisation

An interface object that can be used to implement specialisation tasks.

Table Interface attributes

Property	Description	Type
InterfaceSpecialisation::SpecialisationName	Name of the specialisation.	String

Table InterfaceSpecialisation class relations

Relation	Description	Type	Cardinality
InterfaceSpecialisation::Variables	Specialisation variables than can be access from all skills and actions. Note these variables are not used during normal negotiation but can be a requirement for the planner.	variables.Variable	[0..∗]

8.7.5 InterfaceSpecialisation templates

The listed InterfaceSpecialisation objects below are not new object types but rather templates based on the InterfaceSpecialisation class. These templates are designed to simplify the modelling of specific specialisations by predefining certain relations and property values.

- InterfaceLocation
- InterfaceTransport
- InterfaceAssembly

8.8 cmas.ontology.workflow

8.8.1 WorkFlow

An abstract WorkFlow object.

Table WorkFlow attributes

Property	Description	Type
WorkFlow::SpecificType	The specific workflow type.	ReadOnly SpecificTypes
WorkFlow::WorkFlowType	The workflow type.	WorkFlowTypes

8.8.2 WorkFlowLog

A WorkFlowLog object to log the work flow on agents.

Table WorkFlowLog attributes

Property	Description	Type
WorkFlowLog::Target	Target for the log.	String
WorkFlowLog::Time	Time stamp for the log.	String

8.8.3 Requirement

A Requirement object to add requirements on skills.

Table Requirement attributes

Property	Description	Type
Requirement::Target	Target for the requirement.	String

The meaning of the Target depends on the WorkflowType.
For WorkflowType = SKILL, the Target must be specified as either: skill_name – the name of the skill that must be executed as a requirement. interface_name:skill_name – specifies that the skill must be executed by a resource implementing a particular interface.

Note

interface_name identifies the abstract interface that provides the skill. If interface_name is left empty, the Planner Agent will attempt to match any compatible local interface for that skill. The Planner will then select the resource that satisfies the requirement with lowest cost.

For WorkflowType = GOAL: The Target must specify the name of a Goal to be achieved.
For WorkflowType = DEPLOY or UNDEPLOY: The Target specifies the name of the agent to deploy or undeploy.
For WorkflowType = ATTACH or DETACH: The Target has no specific purpose.

8.9 cmas.ontology.batch

8.9.1 Batch

A batch represents a set of entities that will be deployed at the same time.

Table Batch attributes

Property	Description	Type
Batch::Batch set	The set of entities that will be deployed	BatchItem

8.9.2 BatchItem

Table BatchItem attributes

Property	Description	Type
BatchItem::AgentID	An ID of an agent that will be deployed	String
BatchItem::AttachToID	An ID of an interface to attach to	String
BatchItem::NumberOfItems	Number of items to deploy	Integer
BatchItem::Description	Description	String

8.10 Extended Ontology

It is possible to extend the existing ontology in CMAS by creating new objects based on the existing ones. During the startup of CMAS, if the file **cmas-ontology.json** exists, it is read and interpreted.

Note

Normally, an extension should be achieved by simply adding new variables or modifying existing ones, rather than defining a new ontology with new object types.

This JSON file can contain instructions on how to extend the current ontology. The file consists of an array of object definitions, where each object must include the following information:

- `Name`: The name of the new object to define.
- `Description`: A description of the object.
- `Category`: The object's category.
- `Extends`: The base class the object extends from. Must be an existing CMAS object.

You can also define new properties that are added to the object, with the supported types being Integer, String, Boolean, and Real. A property must include:

- `Name`: The property's name.
- `Type`: The property's type (Integer, Real, Boolean or String).
- `Description`: A description of what the property represents.
- `LocalValue`: A default value for the property.
- `ClassName`: The fully qualified class name of the object the property belongs to. This is typically the base class domain plus the object name.
- `CanInherit`: A boolean indicating whether the property value can be inherited from a parent
- `Override`: A boolean specifying whether a inherited property value is overridden by a local value, must be false
- `ReadOnly`: A boolean indicating whether the property is read-only
- `ParentID`: An identifier of the parent object, must an empty string
- `PropertyType`: How the property can be utilized in the property viewer.

Additionally, you can define new relations for the object. A relation object must include:

- `Name`: The name of the relation.
- `AcceptedObjects`: An array of object names in CMAS that the relation can host.

Example of two new objects Temperature and Operator in the **cmas-ontology.json** file:

```
[
    {
        "Name": "Temperature",          /* Object Name */
        "Extends": "VariableReal",      /* Base class in CMAS */
        "Description": "",              /* Description of the Object */
        "Category": "Value",            /* Object category */

        "Properties": [],                  /* No new properties */
        "Relations" : []                   /* No new relations */
    },

    {
    "Name": "Operator",          /* Object Name */
    "Extends": "Resource",       /* Base class in CMAS */
    "Description": "",           /* Description of the Object */
    "Category": "Agent",         /* Object category */

    "Properties": [               /* An array of new properties */
      {
        "Name": "Time",           /* Property name */
        "Type": "Integer",        /* Property type */
        "Description": "The max time to work",
        "LocalValue": 10,         /* Default value */

        "ClassName": "cmas.ontology.variables.VariableInteger",
        "CanInherit": true,
        "ReadOnly": false,
```

```
        "ParentID": "",
        "Override": true,
        "PropertyType": "[]"
      }
    ],

    "Relations" : [         /* An array of new relations */
        {
            "Name": "Demands",                      /* Name of the relation */
            "AcceptedObjects" : ["Temperature"]     /* Name of the accepted objects that the relation can
      host */
        }
    ]
  }
]
```

The category can be one of the following values:

- "value"
- "skills"
- "interface"
- "goal"
- "executable"
- "hazard"
- "processplan"
- "agent"
- "batch"
- "deployment"
- "workflow"
- "requirement"

The PropertyType can be a combination of:

- INHERITANCE Possible to set inheritance
- REFERENCEFILE Only show if a reference file
- PARENTEDITOR Possible to edit parent
- IDEDIT Opens an ID editor
- TEXTEDITOR Open a text editor for the value
- STRINGEDIT Open a simple string editor for the value
- VARIABLEEDITOR Open an array editor
- SFCEDITOR Open graphical SFC editor
- PROCESSPLANEDITOR Possible to select a process plan
- BATCHEDITOR Open an editor for batches
- INTERFACEBROWSER Possible to browse for interface name
- ADDRESSBROWSE Possible to browse for an adapter address
- HISTORYSHOW Open a window to show the history log
- HIDE Don't show at all
- RUNTIME Only show during runtime
- MODELLING Only show during modelling
- HIDEBASIC Only show during normal

Chapter 9

Planner

9.1 Introduction

Assume that a product must execute process A in order to achieve a specific goal. The first step is to locate all resources capable of performing this process. In CMAS this can be achieved through negotiation with abstract interfaces to identify a suitable resource.
However, once a resource has been selected, the system must also determine how the product can reach that resource. This involves identifying a valid path through the system and selecting appropriate transportation resources. In a production system this can quickly becomes complex, as multiple resources, routes, and constraints may exist. The Planner Agent in CMAS can automatically generate solutions to achieve a goal or satisfy a requirement. This reduces the need for the modeler to write explicit transportation and routing logic. Instead, the system designer models the capabilities and interfaces of the system, allowing the Planner Agent to derive a valid plan automatically. The planner operates according to the following algorithm:

```
PathFinder(Domain, targetInterface, destinationInterface)
    map:=GenerateMap(Domain);
    solver := Solver( SMT | PDDL)
    solver.generateProblem(Map, targetInterface, destinationInterface)
    Plan := solver.findBestPlan()
    execute(Plan)
```

9.1.1 Executing the Plan

The *execute(Plan)* step represents the coordinated execution of the generated plan. A generated plan typically consists of a sequence of transportation and execution operations required to move a part from a source location to a target location. Each transportation operation is internally divided into a sequence of execution steps. The Planner Agent executes each step in order and waits for completion before continuing to the next step.
The following table illustrates the transportation steps:

Step	Name	Target	Description
0	Init	Transporter:OnRun()	To initialize the transporter
1	Move to pickup	Transporter:OnRun()	Move transporter to the source "FromLocation"
2	Pickup from source	Transporter:OnRun()	Pickup from the source "FromLocation"
3	Detach from source	Transporter:OnRun()	Detach from the source
4	Attach	*None*	Attach to the transporter
5	Attach	Transporter:onRun()	Attach to the transporter
6	Move to leave	Transporter:onRun()	Move transporter to the target "ToLocation"
7	Leave to target	Transporter:onRun()	Leave at the target
8	Attach to target	Transporter:onRun()	Attach to target
9	End	Transporter:onRun()	Finalize transportation

Some steps may only update the internal planning state and therefore do not require execution on a physical device.

Other steps might involve motion or execution of skills on remote resources.
To react to a specific execution step, check the value of *interface.specialisation.step*.

```
function onRun() : integer;
begin
    if interface.specialisation.step = 0 then
        writeln('The init step');

    onRun := processplan.done;
end.
```

Note

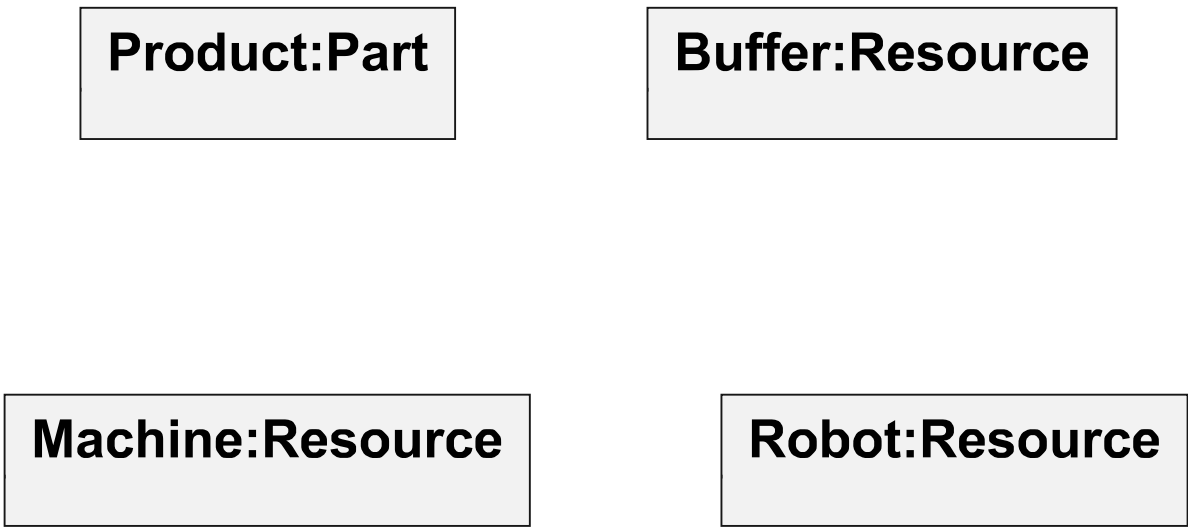
For all execution steps, the onRun() function must return *processplan.done* for the Planner Agent to continue executing the plan.

Returning *processplan.failed* aborts the current execution.

Returning *processplan.replan* aborts the current plan and triggers a replanning of the problem. This allows the Planner Agent to dynamically generate a new plan if resources become unavailable or execution constraints change during runtime.

9.2 Example 1 - one robot

In this example we have four Agents, one Part called Product and three resources Buffer, Machine and Robot.



Let's create a model that can make use of the Planning Agent.

9.2.1 Machine

We start by building a **Resource** agent to represent the Machine. This resource will serve as a provider for process *A*. To model process *A*, we need to establish an interface. This interface will function as a destination for the part, meaning the part must be connected to the interface's location in order for process *A* to run.
To achieve this, we create a **LocationInterface**, which is essentially a standard interface with a specialization to act as a location as well. We can name this interface InterfaceProcessA. If we expand the interface, we will notice it already includes two skills that were automatically generated when the location interface was created: Detach and Attach. These skills can be filled with logic to define how to attach and detach from the interface. For instance, attaching might require clamping. However, for now, we can leave these skills empty as long as they return the value *processplan.done*. Note, that these two skills also have the **PostAction** property set to *ATTACHED* and *DETATCHED*.
Additionally, we will find a Specialization folder within the interface that contains a **VariableObject** named *Location*. This variable represents the geometrical location of the interface, but it does not contain any specific variables. It's

up to the designer to determine how a location will be represented in the system. You could use real variables like *x, y, z, rx, ry, rz* for a full robot representation, or simply use an integer *position* to represent discrete locations, such as location 1, 2, 3, etc. Whatever approach we choose, it's crucial that all agents in the system follow the same notation for location. In this example, a discrete number will suffice. We can achieve this by expanding the Position object and add a new **VariableInteger** named *Position* and give it a value of 2. This value will be a known position for the system to represent process interface A. We also make it a read only value by setting **ReadOnly** to true. Now we can create process *A* by adding a new **processplannstructuredtext** to **Skills** on the interface and the skill can look like this:

```
Program ProcessplanA;

function onRun() : integer;
begin
    writeln('Running process A');
    onRun:=processplan.done;
end.
```

To run process A, we have a requirement that the part requesting the process must first be attached to it. We can model this requirement by selecting the **Requirements** folder for skill *A* and adding a new **Requirement**. Name this requirement *RequirementAttached* and set the **WorkFlowType** property to *ATTACHED*. This ensures that anyone wanting to execute skill *A* must first be attached to the interface.

9.2.2 Buffer

Just like we created the *Machine*, we can now create the *Buffer* as a **Resource**. For the *Buffer*, we also need a **LocationInterface**, but this time, we can set the *position* to 1 instead. In this example, no additional specific skills are required on the *Buffer*, except for the one that was automatically created for the specialization. We can name the interface **InterfacePart**. To make it easier to distinguish the buffer skill, we can rename the skill **Attach** to **AttachOnBuffer**. We will need this later.

9.2.3 Robot

Now, let's create a **Resource** agent and name it *Robot*. On the *Robot*, create a new specialization interface called **InterfacePart**. This interface will be specialized for transportation tasks.
We can name the interface *InterfacePart*.
On a transportation interface, we will find three predefined variables: **FromLocation**, **ToLocation**, and **Step**. For both the **FromLocation** and **ToLocation**, we need to add an integer variable called **Position**. However, we won't set a specific value for this variable, nor will we make it read only.
Now open the skill predefined skill **Transport**.

```
program ProcessplanTransport;

function onNegotiate(Planning : boolean, a:reference, i:reference): integer;
begin
    // Only accept moving forward
    if remoteinterface.specialisation.fromlocation.position >
       remoteinterface.specialisation.tolocation.position then
        onNegotiate:=-1; // reject
    else
        onNegotiate:=this.cost; // use cost value
end.

function onRun() : integer;
begin
    if interface.specialisation.step=2 then
        writeln('Robot will pick up from ',
            interface.specialisation.fromlocation,
            ' and transport to ',
            interface.specialisation.tolocation);
    else if interface.specialisation.step=7 then
        writeln('Robot will place at ',
            interface.specialisation.tolocation);

    onRun:=processplan.done;
end.
```

9.2.4 Part

Now, create a **Part** agent and name it *Part*. On the **Part**, create a new **Interface** and name it **InterfacePart**. This will make the **Part** compatible with the **buffer**, **robot**, and **machine**. When the **Part** is deployed, it needs to be attached to the **buffer** in the system. To accomplish this, we need to define the logic for the deployment process.

Select the **Part** and open the **Actions** editor. Here, we can use the dedicated skill **AttachOnBuffer** to ensure the **Part** attaches to the buffer, rather than to the machine or robot.

```
program Part;

abstract aResource : interface;

procedure onDeploy;
begin
    aResource.AttachOnBuffer();
end.
```

Now, open the process plan editor for the **Part** and create a new **Goal**. Edit the **Goal** by setting the process plan name to *A* and the interface to *InterfacePart*. Next, drag the goal to the goal chart and drop it on the start. The **Part** is now ready.

9.3 Example 2 - adding an operator

Now, we would like to add an operator to the system to handle situations when the robot is out of order, but only in those cases. We begin by adding a new **Resource** and naming it *Operator*. For the operator, we duplicate the interface of the *Robot*, which means creating an **InterfaceLocation** and naming it **InterfacePart**. Next, we update the two variables, *FromLocation* and *ToLocation*, by adding an integer variable named *position*, in the same way we did earlier for the *Robot*. Now, we need to implement the *Transport* skill. We start by renaming the default *Transport* skill to *ManualTransport*. This helps distinguish it as a manual task. Then, we set the cost for the *ManualTransport* skill, for example, 5. As long as the cost is higher than the robot's, any value will work. After that, we can implement the logic for the *ManualTransport* skill :

```
program ProcessplanManualTransport;

function onNegotiate(Planning : boolean, a:reference, i:reference): integer;
begin
    // Only accept moving forward
    if remoteinterface.specialisation.fromlocation.position >
       remoteinterface.specialisation.tolocation.position then
        onNegotiate:=-1; // reject
    else
        onNegotiate:=this.cost; // use cost value
end.

function onRun() : integer;
begin
    // Write a message for the operator on the HMI
    // This can be done with, e.g, OPC UA connected variables
    if interface.specialisation.step=2 then
        writeln('Operator must pick up from ',
            interface.specialisation.fromlocation,
            ' and transport to ',
            interface.specialisation.tolocation);
    else if interface.specialisation.step=7 then
        writeln('Operator must place at ',
            interface.specialisation.tolocation);

    onRun:=processplan.done;
end.
```

9.3.1 Updated robot

Now, we need to update the *Robot* agent to handle errors. We begin by adding an agent variable to the *Robot*. This can be a boolean variable named *RobotError*. At this point, it's possible to map it to a physical signal using adapters, but for this example, we will leave it as is. The *Transport* skill must now check the *RobotError* signal and refuse to participate by rejecting any negotiation attempts when *RobotError* is true. This can be easily done by adding an **OR** *RobotError* condition to the rejection part of the onNegotiate function.

```
program ProcessplanTransport;

function onNegotiate(Planning : boolean): integer;
begin
    // Only accept moving forward
    if remoteinterface.specialisation.fromlocation.position >
       remoteinterface.specialisation.tolocation.position OR
       agent.RobotErrror
    then
        onNegotiate:=-1; // reject
    else
        onNegotiate:=this.cost; // use cost value
```

```
end.

function onRun() : integer;
begin
    if interface.specialisation.step=1 then
        writeln('Robot will pick up from ',
            interface.specialisation.fromlocation,
            ' and transport to ',
            interface.specialisation.tolocation);
    else
        writeln('Robot will place at ',
            interface.specialisation.tolocation);

    onRun:=processplan.done;
end.
```

9.3.2 Running Example 2

When running the example and deploying a part, we can see in the console that the planner selects the **Robot** for transportation. This is because the robot's cost is lower than the operator's.

```
Robot will pick up from {"Position":"1"} and transport to {"Position":"2"}
Robot will place at {"Position":"2"}
Running process A on Machine
```

If we deploy several parts, the system will always select the robot. However, if we open the running tree, access the robot agent, and change the *RobotError* signal to true during runtime, we will see that the next part is planned with the operator instead. For more complex scenarios, note that the planner will always attempt to optimize the total cost of the plan. We can observe the same effect if we undeploy the robot because we no longer need it. To do this, simply select the robot in the running tree and click "Undeploy." When we want to use the robot again, we can deploy it again from the depot.

9.4 Goal-Driven Planning

The example above illustrates a case where the planner must determine a transportation path to fulfill the requirement ATTACHED. However, the planning capabilities in CMAS extend beyond simple routing. In CMAS, planning is goal-driven and it is possible to state other requirements such as Skill requirement.
For example:
A $\rightarrow$ required by B B $\rightarrow$ required by C
A Skill Requirement defines a dependency between skills. If a skill requires another skill, the Planner Agent will ensure that the required skill is executed before the dependent skill can start. If a user specifies that the goal is to achieve C, the Planner Agent can automatically derives the required execution chain:
A $\rightarrow$ B $\rightarrow$ C
The planner then determines where each process can be executed and generates the transportation steps needed to move the product between resources. This means that the modeler specifies *what* should be achieved, while the Planner Agent determines *how* it can be achieved within the available system capabilities.

9.4.1 Skill Requirements: AND / OR Semantics

In CMAS, a Skill Requirement defines a dependency between skills. Understanding how these dependencies interact with resource selection is key to using the Planner Agent effectively.
Logical AND Across Skills
All skills listed as requirements for a skill must be executed before the dependent skill can run. This establishes a logical AND between the required skills.
For example: Skill C requires $\rightarrow$ Skill B Skill B requires $\rightarrow$ Skill A1, Skill A2
Each skill may be implemented by one or more resources compatible with its abstract interface. When multiple resources can execute a skill, the Planner Agent chooses the resource with the lowest cost or highest efficiency. This introduces a practical OR at the resource allocation level, while the logical dependencies between skills remain an AND.
Example:
Skill A can be executed by: R1 (cost 3) R2 (cost 5) Planner selects R1.
A skill requirement states a WorkFlowType and a Target value. For WorkflowType = SKILL, the Target must be specified as either: skill_name – the name of the skill that must be executed as a requirement. interface_name$\hookleftarrow$:skill_name – specifies that the skill must be executed by a resource implementing a particular interface.

Note

interface_name identifies the abstract interface that provides the skill. If interface_name is left empty, the Planner Agent will attempt to match any compatible local interface for that skill. The Planner will then select the resource that satisfies the requirement with lowest cost.

It is also possible to specify other requirements such as goals.

9.4.2 Representing Skill Requirements in Structured Text

The concept of Skill Requirements in CMAS can also be expressed in Structured Text using abstract interfaces inside a Skill.
First, an abstract interface is declared:
abstract *interfacename*;
Later in the code, a skill can be invoked through the interface just like in the previous example with Skill Requirements:
interfacename.B;
This has the same effect as specifying Skill B as a requirement in CMAS:
The Planner Agent will resolve the full dependency chain (e.g., A $\rightarrow$ B $\rightarrow$ C).
The Planner will select an appropriate resource implementing the abstract interface.
Execution order is determined automatically according to the dependencies.

Note

Using abstract interfaces in Structured Text allows the developer to write goal- or skill-driven code, while the Planner Agent manages resource selection and execution sequencing behind the scenes.

Chapter 10

Processplans

In CMAS there exist something called process plans. Process plans can be created for different purposes. The flexibility of CMAS comes when you're putting together all process plans in one system. A process plan can be expressed as a goal chart (GC) or as structured text (ST). A process plan is executed according to the actions described in the section process plan actions and can be defined for the following entities:

Entity	Description	Modelling language
Part	a description on how a part can reach all its goals to become finalised	GC
Resource	a description of a skill on an interface available for other agents to utilize	ST or GC
Global Process Plan	defined in the root of the modelling tree and can be used by parts or resources	ST or GC

10.1 Part Agent

Before commencing production, process planning is essential, outlining the necessary steps to manufacture or assemble the end product. This crucial stage establishes the prerequisites and the sequence of operations. This must be expressed as a sequence in goal chart for a part agent. A goal serves as a bridge between product design and manufacturing and describes something that we need to reach. An example of a goal might be to get hole in the centre of the part. The process requirements are reflected in the order or sequence of the goals. For example, it might be more useful to first create a hole and then deburr it, rather than doing it the other way around. In CMAS all part agents must have at least one goal and a process plan that utilise at least one goal. To reach the part process plan select the part in the modelling tree and in the property window select processplan Edit button you can also double click on the part in the tree to open the process plan.

10.2 Resource Agent

A resource will publish available skills through interfaces. An interface describes requirements as variables and skills as process plans. A skill is a process plan describing the required steps needed to perform the skill. A skill can be used to fulfil a goal or as a required step in another skill. A well-crafted skill allows for adaptability and flexibility. To edit a skill double click on it in the modelling tree.

10.3 Global process plans

A global process plan is defined in the root of the modelling tree and can be used by parts when they are searching for a way to fulfil a goal. When used to fulfil a goal the process plan will be executed by the part agent it self.
It is also possible to use global process plans to describe skills on resources. To do this drag the global process plan to a resource interface and drop it and select reference.

For more information see:

- Structured text
- Process plan actions
- Interfaces

10.4 The hierarchical structure: Goal to Execution

This section outlines the structure and hierarchy of elements involved in task execution and planning. Each element plays a distinct role in organizing and coordinating activities to achieve system goals efficiently. The framework is divided into five primary categories: Goals, Operations, Process Plans, Skills, and Actions, each serving specific functions and working together to enable seamless execution of complex tasks.

1. Goal or requirement (System objective level)
    - Role: Defines what the system must achieve.
    - Scope: Broad, user-defined outcome, originates from the design.
    - Properties: Triggers planning or execution processes. It may be decomposed into one or more operations / skills. Goals are typically defined for parts and requirements for resource skills.
2. Operations (Abstract task)
    - Role: Represent abstract tasks such as "transport item to station".
    - Scope: Broad in scope, they are domain-relevant and understandable at the planning level.
    - Properties: Serve as units (edges) of planning in high-level graphs and can not be created manually.
3. Process Plans (General Executable capabilities)
    - Role: Implement general operations by organizing the logic and flow of execution.
    - Scope: Can include actions and delegation to remote skills (recursive structure).
    - Properties: Serve as reusable, parameterizable execution strategies that is not associated with any specific agent. The agent that executes the Process Plan will be consider the target agent. Define coordination logic (e.g., sequences, selections, iterations, synchronizations).
4. Skills (Agent specific Executable capabilities)
    - Role: Implement agent associated operations by organizing the logic and flow of execution.
    - Scope: Can include local actions and delegation to remote skills (recursive structure).
    - Properties: Serve as reusable, parameterizable execution strategies. Define coordination logic (e.g., sequences, selections, iterations, synchronizations).
5. Actions (Low-Level, Atomic Behaviors)
    - Role: Directly executable local functions or commands (e.g., “move arm”, “start conveyor motor”).
    - Scope: Minimal scope; cannot be broken down further.
    - Properties: Require no external coordination or resources beyond the executing agent.
    - Must be deterministic, safe, and bounded in execution.

10.5 Processplan action

10.5.1 Processplan

10.5.1.1 Process plan execution

A process plan will be executed according to the following steps:

State name	Description	Overload
Init	Constructor is called	optional

State name	Description	Overload
Negotiate	onNegotiate is called	onNegotiate is optional but if defined it must return an integer based on the Processplan return value
Running	onRun is called	onRun must be defined and return an integer
Finished	onFinished is called	onFinished is optional
Overlap	onOverlap is called	onOverlap is optional

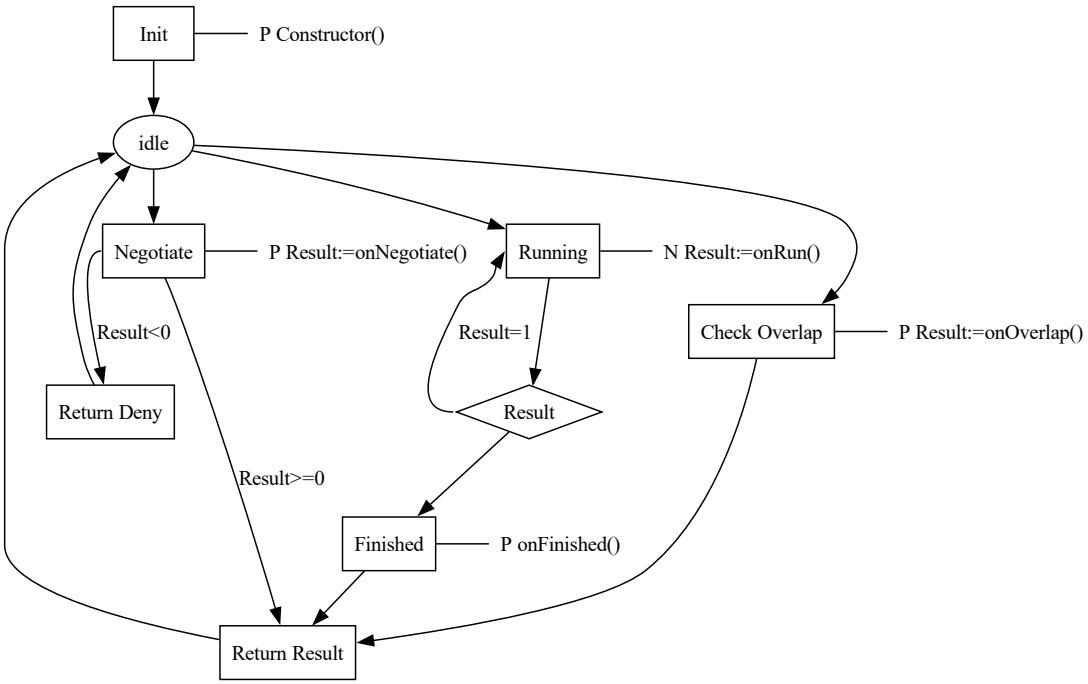
10.5.1.2 Processplan return value

The function onRun() is a user-defined function that must always exists. The onRun function must be of type *integer* and the return values from the function must be one of the following:

Integer value	Predefined constant	Explanation
1	processplan.running	Still running, the onRun() function will be called again at the next cycle
2	processplan.done	Ongoing activities are done
3	processplan.failed	Indicates a failure and ongoing running will be aborted
4	processplan.interrupted	Indicates an interruption
5	processplan.replan	Indicates that we need to replan. The calling instance must replan the solution
6	processplan.redo	Indicates that the plan must be executed again without replanning

Note

For a goal chart process plan it is only possible to describe the onRun part with the goal chart.



In the following example a process plan is created to make two holes. It also demonstrate how to make loop where each iteration must return back the control to the system to avoid time out.

10.5.1.3 onNegotiate

The onNegotiate function is called when another agent requests a negotiation.
It may return **–1** to indicate that the process plan is not compatible, or a value **0** to accept the negotiation.
A value 0 represents the cost of executing the process plan.
If onNegotiate is not defined, the default behavior is to return the value of the *cost* property of the process plan.
In the example below, the onNegotiate function checks whether the negotiated locations are reachable.
If both locations are reachable, it returns **1** to accept; otherwise, it returns **–1** to refuse. Note, for a skill there are no reference variables for agent and interface.

```
// Example: robot negotiation
//
// Check if it is possible to reach
// remoteinterface.Specialisation.FromLocation and
// remoteinterface.Specialisation.ToLocation
//
function onNegotiate(Planning : boolean): integer;
begin
    if remoteinterface.Specialisation.FromLocation.x <= 2 AND
        remoteinterface.Specialisation.ToLocation.x <= 2 then
        onNegotiate := 1;
    else
        onNegotiate := -1;
end.
```

10.5.1.4 onRun (processplan)

onRun must implement and represents the main action of a process plan. onRun must return an integer based on the Processplan return value.

Note

It is not a good approach to block the onRun() function with an extensive loop or to wait for a specific value. This will generate a time out in the system and the process plan will be aborted. This can be turned off in preferences by setting timeout to 0 (use it for debug purposes only). See the example at the end on how to handle extensive tasks.

10.5.1.5 onFinished

onFinished is called when the process plan is done. It is optional and can be used for e.g., cleanup.

10.5.1.6 onOverlap

onOverlap is called to check for overlaps. onOverlap is a boolean function and optional. If it returns true the overlapping interface is booked to prevent dangerous situations.

10.5.1.7 Example

```
Program Processplan;

abstract aResource : interface;
var steps: integer;

function onNegotiate(Planning : boolean) : integer;
begin
    if remoteinterface.diameter=5.0 then
        onNegotiate:=this.cost;  // Accept by using the value in the cost property
    else
        onNegotiate:=-1; // Refuse
end.

function onOverlap() : boolean;
begin
    onOverlap:=true;
end.


function onRun() : integer;
begin
    if steps=0 then
    begin
        aResource.x:=10.0;
        aResource.MakeAHole();
        onRun:=processplan.running;
        steps:=steps+1;
    end;
    else if steps=1 then
    begin
        aResource.x:=20.0;
        aResource.MakeAHole();
        onRun:=processplan.running;
        steps:=steps+1;
    end;
    else
    begin
        steps=0;
        onRun:=processplan.done;
    end;
end.

// Constructor
begin
    steps:=0;
end.
```

Chapter 11

Requirements

11.1 Introduction

Requirements can serve as criteria representing necessary conditions that must be met prior to the execution of a skill or a goal. For instance, if a robot possesses the skill "MoveRobot," an ATTACHED requirement can specify that a part must be affixed to the robot before it can start to move. CMAS has a built in AI planner that can autonomously generate plans outlining how to fulfil requirements and then execute them.

11.2 Example requirement on a skill

In the following example, a part is placed in a buffer. The part has only one goal to run a process. The process is located on a machine that requires the part to be there. In between the buffer and the machine, there's a robot that can do transportation. By modelling the system with a requirement and specialised interfaces CMAS can find a path for the part to reach the process without explicitly setting it as goals.

11.2.1 Model

The object illustration below demonstrates how the example can be modelled. The crucial elements in the model are the specialised interfaces paired with relevant skills and their associated PostActions, the AI planner will utilise these to find solutions.
In this example we will use two templates for interface specialisation:

- InterfaceTransport - An interface that can do transportation
- InterfaceLocation - An interface that can be a destination or a start position

Buffer:Resource							
Name	Buffer						
Interfaces	1	Name	BufferInterface				
		SpecialisationName	Location				
		Skills	1	Name	pickBuffer		
				PostAction	DETACHED		
			2	Name	placeBuffer		
				PostAction	ATTACHED		
		Specialisation	Location	Name	Position		
				ValueType	Integer		
				Value	1		
				ReadOnly	true		

Machine:Resource								
Name	Machine							
Interfaces	1	Name	ProcessInterface					
		SpecialisationName	Location					
		Skills	1	Name	placeProcess			
				PostAction	ATTACHED			
			2	Name	runProcess			
				Requirements	1	Name	MustBeAttached	
						WorkFlowType	ATTACHED	
		Specialisation	Location	Name	Position			
				ValueType	Integer			
				Value	2			
				ReadOnly	true			

Robot:Resource							
Name	Robot						
Interfaces	1	Name	GripperInterface				
		SpecialisationName	Transport				
		Skills	1	Name	Transport		
				PostAction	TRANSFERED		
		Specialisation	1	FromLocation	Name	Position	
					ValueType	Integer	
				ValueType	Object		
			2	ToLocation	Name	Position	
					ValueType	Integer	
				ValueType	Object		
			3	Step	Name	Position	
					ValueType	Integer	

ProductA:Part			
Name	ProductA		
Goals	1	Name	GoalGetProcessed
		ProcessPlan	runProcess
		Interface	ProcessInterface
Interfaces	1	Name	BufferInterface
	2	Name	GripperInterface

11.2.2 Step-by-step instructions:

11.2.2.1 1. Buffer

- 1.1 Create a resource and name it Buffer. The buffer will serve as a source for our parts later on.
  1.1. On Buffer create a new InterfaceLocation and name it BufferInterface. An InterfaceLocation is a specialized interface that can serve as a location, e.g. for pick or place.
  1.2. On BufferInterface there is a folder called Specialisation that has one item called Location. Location is a variable object. Open the location folder and add an VariableInteger to items. Name the new integer Position, and set the value to 1 and mark ReadOnly. This variable is now our location it is possible to add more variables as x, y and z to represent more complex locations if required.
  1.3. Now create a new Skill as Structured Text and name it pickBuffer.
  1.4. Select the new skill and change PostAction to DETACHED. This indicates that if you run this skill you will be detached.
  1.5. Create another Structured Text skill and name it placeBuffer.
  1.6. Select the new skill and change PostAction to ATTACHED.

11.2.2.2 2. Machine

- 2.1 Create a resource and name it Machine.
  2.2. On Machine create a new InterfaceLocation and name it ProcessInterface.
  2.3 In the same way as for BufferInterface add a Position variable and set the value to 2 and read only.
  2.4 Create a Structure Text skill, name it placeProcess and set PostAction to ATTACHED.
  2.5 Create a Structure Text skill and name it runProcess.
  2.6 Open the folder for runProcess and add a new Requirement. Name it MustBeAttached.
  2.7 Select MustBeAttached and change WorkFlowType to ATTACHED. This requirement will indicate that a Part that wants to run the skill must be attached to the interface first.
  2.8 Open Actions for placeProcess skill and add the following line to onRun() function:

```
writeln('Running the process');
```

11.2.2.3 3. Robot

- 3.1 Create a resource and name it Robot.
  3.2 Create a new InterfaceTransport and name it GripperInterface. This will serve as a interface that can do transportations.
  3.3 Open the Specialisation folder.
  3.4 On FromLocation add a new IntegerVariable named Position. Do not make it read-only.
  3.5 On ToLocation add a new IntegerVariable named Position. Do not make it read-only.
  3.6 Double-click on the existing skill Transport to edit.
  3.7 In side the onRun() function add

```
if interface.specialisation.step = 1 then
   writeln('Robot transport from ',
       remoteinterface.Specialisation.FromLocation.Position,
       ' to ',
       remoteinterface.Specialisation.ToLocation.Position,
       ' Step ', remoteinterface.Specialisation.Step);
```

11.2.2.4 4. ProductA

- 4.1 Add a new Part
  4.2 Double-click on the new part.
  4.3 Right click in the top left window and select Add new goal.
  4.4 Double-click on the new goal.
  4.5 Set interface name to ProcessInterface.
  4.6 Set process plan name for the goal to runProcess.
  4.7 Close the goal.
  4.8 Drag the goal and drop it on Start.
  4.9 Close the editor.
  4.10 Add a new interface and name it BuffertInterface. This ensures that the part can be placed in a buffer.
  4.11 Add one more interface and name it GripperInterface. This ensures that the part can be placed in a gripper.
  4.12 Select the part and click on Actions.
  4.13 Inside the empty editor write the following program (this program will put the part inside a buffer when deployed):

```
program Part;
abstract aBuffer : interface;

procedure onDeploy();
begin
    aBuffer.placeBuffer();
end.
```

11.2.2.5 Run

1. Press Run to start the system (if auto deploy is not on deploy everything).
   You should see the following output:

```
Robot transport from 1 to 2 Step 1
Running the process
```

11.3 Example requirements in a process plan

In this example we will look at requirements in process plans for *Parts*.

11.3.1 Assembly

Similar to how we specified ATTACHED as a requirement in the previous example, we can also specify ASSEMBLY as a requirement. While ASSEMBLY is closely related to ATTACHED, there is a key difference: when an agent, typically a material agent, is assembled with another agent, the assembled agent is marked as finished and can no longer be used. The modelling pattern involves representing the main structure as a *Part* agent and all subassemblies as *Material* agents. This allows the planner to generate solutions for how to *assemble materials* and *attach parts*.
The material to be assembled must have a specialized interface *InterfaceAssembly*. Similarly, the main structure must have a specialized interface *InterfaceLocation*, that is compatible with the assembly interface, to tell where to assembly the material.

11.4 InterfaceSpecialisation

An InterfaceSpecialisation resembles a regular Interface but includes an additional relation called Specialisation, which can incorporate variables. It also have a property called SpecialisationName that indicates the purpose of the interface. Unlike the standard interface relation Variables, the Specialisation relation is disregarded during the negotiation process and in determining the compatibility of two interfaces. The specialisation variables are utilised by the internal AI planner to gather information for plans or can be used in applications to introduce variables that don't interfere with the negotiation process.

11.4.1 InterfaceLocation

InterfaceLocation is a template for creating InterfaceSpecialisation for locations. It automatically creates a InterfaceSpecialisation with a VariableObject named Location and SpecialisationName set to Location. It also assigns SpecialisationName to a read only value "Location".

11.4.2 InterfaceTransport

InterfaceTransport is a template for creating an InterfaceSpecialisation tailored for transport. It automatically generates an InterfaceSpecialisation that includes two VariableObjects named FromLocation and ToLocation. Additionally, it creates a Skill named Transport with the PostAction configured as TRANSFERED and assigns Specialisation↩ Name to a read only value "Transport".

11.4.3 InterfaceAssembly

InterfaceAsembly is a template for creating an Interface that can be used for assembly. It automatically creates a InterfaceSpecialisation with the SpecialisationName set to Assembly.

Note

It is possible to manually create a specialisation interface by setting the correct values and adding the VariableObjects manually on an *InterfaceSpecialisation*.

Chapter 12

Structured text

12.1 Introduction

Structured text can be used to describe skills and process plans. We will start with the famous Hello World in structured text.

```
Program HelloWorld;

{ Program to write Hello World on the console }
Begin
  writeln('Hello World');
End.
```

12.2 The overall structure in a program

A program consist of the following sections:

- Program name (optional)
- Directives (optional)
- Struct type (optional)
- Interfaces (optional)
- Global variables declarations (optional)
- Comments (optional)
- Functions declarations (optional)
- Procedures declarations (optional)
- Main program block or constructor (optional)

12.2.1 Program

Section: Program name
Syntax: `Program <Name>;`
Description: <Name> is a descriptive text string that defines the name of the program.
Example:

```
Program HelloWorld;

{ Program to write Hello World on the console }
Begin
  writeln('Hello World');
End.
```

12.2.2 Directives

The following directive exists

Directive	Value	Description
$I	<files_string>	Include file

Directive	Value	Description
$INCLUDE	<file_string>	Include file
$U	<ID>	Include resource by ID
$USE	<ID>	Include resource by ID
$ERROR	<message_string>	Stop compiler and give an error

12.2.2.1 Include

Section: directives
Syntax: `$include <filename> | $i <filename>`
Description: include will read another file and include it in the compilation. If <filename> starts with *system:* a predefined system file can be included, see more about system files in system include files.
Example:

```
Program IncludeFile;

{ Directive to include the file standard.inc }
$INCLUDE 'standard.inc'

{ Will include a system file }
$INCLUDE 'system:file.inc'

Begin
  // Will print the object MODE defined in system:file.inc
  writeln(MODE);
End.
```

12.2.2.2 Use

Section: directives
Syntax: `$use <id> | $u <id>`
Description: use will include another entity based on the id. The id is defined in the modeling tree.
Example:

```
Program IncludeResource;

{ Directive to include a program defined in another entity }
$USE 'aResource'

Begin
  // Will call a function defined in another entity
  start();
End.
```

12.2.2.3 Error

Section: directives
Syntax: `$error <message>`
Description: will stop the compiler and show the error message.
Example:

```
Program DoNotCompile;

Begin
  // Will prevent any one to compile and use this resource
  $error 'Do not use this resource yet it is not ready!'
End.
```

Note

$error will generate an error when compiling for run time errors use error instead

12.2.3 Struct type

Section: Struct type
Syntax: `struct <name> [extends <base>]; Begin <variable declarations> End;`

Description: Definition of new structure types. It is possible to extend a previously defined structure by the use of extends.
Example:

```
Program StructExample;

{ Structure definition }
Struct Pos2D;
Begin
    Var x, y : Real;
End;

{ Structure definition }
Struct Pos3D extends Pos2D;
Begin
    Var z : Real;
End;

{ Variable declaration using a structure definition }
Var a: Pos2D;

// Variable declaration with an initialisation
Var b:=(x:=1.0, y:=0.0): Pos2D;
Var c:=(x:=1.0, y:=0.0, z:=1.0): Pos3D;

Begin
    a.x:=0.0;
    a.y:=1.0;
    writeln('Struct a = ', a);
    writeln('Struct b = ', b);

    writeln('Struct c = ', c);
End.
```

12.2.4 Interface

It is possible to declare interfaces in a program. This must be done if it is impossible to model the interface or if the interface is abstract. These interfaces can be used to interact with other agents, e.g., to request a skill to start and send variable values. The following interface types exist:

- Abstract
- Define
- Static

All abstract interfaces expose some static and some asynchronous functions. The asynchronous functions starts with an underscore _. For more details see the section interfaces and negotiation.

12.2.4.1 Asynchronous functions

12.2.4.1.1 _book

abstractinterface._book() will try to book the abstract interface. See _book()

12.2.4.1.2 _unbook

abstractinterface._unbook() win unbook the abstract interface. See _unbook()

12.2.4.1.3 _isbookedbyme

abstractinterface.isbookedbyme() will return done if the remote interface is booked by current agent. See _isbookedbyme()

12.2.4.1.4 _isbooked

abstractinterface._isbooked() will return done if the remote interface is booked by anyone. See _isbooked()

12.2.4.2 Static functions

12.2.4.2.1 negotiate

abstractinterface.negotiate() will make a single attempt to locate a compatible remote interface from the accessible resources. See negotiate()

12.2.4.2.2 negotiateandbook

abstractinterface.negotiateandbook() will make a single attempt to locate and book a compatible remote interface from the accessible resources. See negotiateandbook()

12.2.4.2.3 agree

The *abstractinterface*.agree() will try forever to locate a compatible remote interface from the accessible resources. See agree()

12.2.4.2.4 agreeandbook

The *abstractinterface*.agreeeandbook() will try forever to locate and book a compatible remote interface from the accessible resources. See agreeandbook()

12.2.4.3 Abstract interface declarations

Section: Abstract interface declarations
Syntax: `abstract [handle] [local] <name1> [, <name2>] [, ...] : interface;`
Description: Declaration of abstract interfaces. Abstract interfaces are mapped during runtime through a negotiation process. The negotiation process consider variables and used skills. If there are many interfaces that accept the negotiation the one with the lowest cost is selected. The negotiation process starts automatically when an item of an abstract interface is accessed for the first time. It is possible to force a negotiation for an interface by calling the negotiation() action on the abstract interface. It is possible to add the modifier handle for an abstract interface, see interface details for more information. If the *local* modifier is specified the search will be done on the agent it self to find a local interface that is compatible with the abstract interface.
Example 1:

```
Program AbstractExample;
{ Abstract remote interfaces }
abstract Robot, Resource : interface;

Begin
    Robot.MoveTo();

    Resource.Use();
    Resource.Signal:=TRUE;
    Resource.Motor();
End.
```

Example 2:

```
Program AbstractHandleExample;
{ Abstract remote interfaces that is handled by onRun() }
abstract handle transport : interface;
var returnValue:integer;

function onRun() : integer;
begin
    transport.negotiateandbook();
    transport.moveRight(returnValue);

    // Wait until moveRight skill call is done
    // this is necessary since the call is done with an result variable
    repeat
        sleep(100);
    until returnValue=processplan.running;

    transport._unbook();
    onRun:=returnValue;
end.
```

Note

The negotiate() call will make sure that the abstract interface is renegotiated on each run. If negotiate() is not called the system will make sure that a negotiation is made the first time and then the interface will remain static and no further negotiations are made. The repeat loop will wait for the skill moveRight() to end.

12.2.4.4 Define interface declaration

Section: Define a new local interface syntax 1
Syntax1: `define [handle] [local] <name> : interface; Begin <variable declarations> <function definitions>End;`
Description: Declaration of a new local interface for use to find compatible remote or local interfaces. This can be used when it is not possible define it in the configuration. The declared name of the interface is the name used for matching with remote interfaces. All declared variables together with all skill calls used on the abstract interface is automatically added to the requirement list for matching. It is also possible to add skills manually by defining function names in the interface. If *local* is specified the search will be done on the agent it self to find a local interface that is compatible with the defined interface. The defined name of the interface is used during negotiation to find a matching interface. It is possible to override the name by defining name inside the begin end block.
Example:

```
Program DefineExample1;

{ Define a new local interface }
define aRobot : interface;
begin
    name robot;
    var x, y, z : real;
    function move;
end;

function onRun() : integer;
begin
    aRobot.x:=2;
    aRobot.y:=3;
    aRobot.z:=0;

    // Will search for a remote interface aRobot and run the skill move() on the remote interface.
    aRobot.move();

    // Return value - running, done, failed or interrupted
    onRun:=processplan.done;
end.
```

Section: Define a reference to an existing local interface syntax 2
Syntax2: `define [handle] <name> :='<interface name>' interface;`
Description: Define a reference to of an existing local interface, named <interface name> for use to find compatible remote interfaces. This can be used when you do not want to search locally for an interface. Note, <name> will be used as interface name during the negotiation process.
Example:

```
Program DefineExample2;

{ Define an existing local interface robot to use as an abstract interface }
define aRobot:='robot' : interface;

function onRun() : integer;
begin
    aRobot.x:=2;
    aRobot.y:=3;
    aRobot.z:=0;

    // Will use the local interface 'robot' to search for a remote interface named aRobot and run the skill
      move() on the remote interface.
    aRobot.move();

    // Return value - running, done, failed or interrupted
    onRun:=processplan.done;
end.
```

12.2.4.5 Static interface declarations

Section: Static interface declarations
Syntax: `static [handle] <name1:='ID1'> [, <name2:='ID2'>] [, ...] : interface;`

Description: Declaration of static interfaces. The remote interface is mapped based on the interface entity ID.
Example:

```
Program StaticExample;
{ Abstract remote interfaces }
static Robot:='interfaceID', Resource:='124' : interface;

Begin
    Robot.Position:=1;
    Robot.MoveTo();
    Resource.Signal:=TRUE;
    Resource.Motor();
End.
```

12.2.5 Global variable declarations

Section: Global variable declarations
Syntax:
`var [constant] <name1> [, <name2>] [, ...] : {real | integer | boolean | string | structtype};`
or
`var [constant] <name1> [, <name2>] [, ...] : array [<startindex> .. <endindex>] of {real | integer | boolean | string | structtype };`
Description: Declaration of variables or arrays that are accessible in the entire program and all functions. It is possible to declare several variables based on the same type on one line or it is possible to use several lines. During declaration is also possible to assign a default value. For structures the default value must be surrounded by () and all values bust be parried with variable names. For arrays the default value must be surrounded by [].
Example:

```
var Ready : boolean;
var Counter, Number : integer;
var constant PI:=3.1415 : real;
var pos:=(x:=1.0, y:=2.0) of Position;
```

12.2.6 Arrays

Description: An array is an indexed collection of elements of the same type. Each element is accessed using its index, which must lie within the declared index range. The index range is explicitly defined at declaration time using a start and end index. Arrays may be initialized with default values, specified in index order and enclosed in square brackets []. The number of default values must match the size of the declared index range.
Example:

```
var Data:=[3,7,8,2,1]  : array [1 .. 5] of integer;
var Index : integer;
begin
    // Write default values
    writeln(data);

    // Access all elements in the array with a for loop and a index
    for Index:=startindex(Data) to endindex(Data) do
        Data[Index]:=random(10);

    writeln(data);

    // Access all elements in the array with a foreach loop
    foreach item in Data do
        item:=random(10);

    writeln(data);
end.
```

12.2.7 Comment

Section: Comment
Syntax 1: `{ <multi line text> } | (* <multi line text> *)`
Syntax 2: `// <single line text>`
Description: Comments are ignored by the compiler and are optional but can improve the readability of the code. Comment can be single line // that ends when the line ends or multiline { } that end at the end of the comment character }. Note that comments cannot be nested, i.e. a comment cannot contain another comment. The comment type { } is equivalent to (* *).

Example:

```
{  Program to write Hello World on the console
  The program will show the basic idea of structured text
}

// This comment is one line only!

(* Multiline comment again *)
```

12.2.8 Function declaration

Section: Function declaration
Syntax 1 - Parameters:

```
function <name>([<parameter 1> [; <parameter n>]:<parameter type>] [;
...]) :  { real | integer | boolean | string | structtype};
[local variable declaration]
begin
[function code]
end.
```

Syntax 2 – No parameters:

```
function <name>() :  {real | integer | boolean | string | structtype};
[local variable declaration]
begin
[function code]
end.
```

Description: Functions can be declared to encapsulate a block of code to perform a specific action. Functions can be called from other functions or procedures and have a return value. The return value is handled by assigning the function name a value. A function can accept 0 or n number of parameters where <parameterx>\ represents parameter x name and <parameter typ> is a known type or struct. A function with no parameters can be called in two ways, <name>; or by <name>(); .e.g., f; or f();.
Example 1:

```
Program SquareFunction;
{ Function with return value }
function Square(Value: integer) : integer;
begin
    Square:=Value*Value; // Calculate and set return value
end.

Begin
    writeln('The square of 4 is ', Square(4));
End.
```

Example 2:

```
Program PI;
{ Function with no parameters }
function Pi : real;
begin
    Pi:=3.1415; // Constant expression
end.

Begin
    writeln('Pi=', Pi);
End.
```

12.2.9 Procedure declaration

Section: Procedure declaration
Syntax 1 - Parameters:

```
procedure <name>([<parameter 1>[; <parameter 2>]:<parameter type>] [;
<parameter n>:<parameter type>] [; ...]);
[local variable declaration]
begin
[procedure code]
end.
```

Syntax 2 - No parameters:

```
procedure <name>;
[local variable declaration]
```

```
begin
[procedure code]
end.
```

Description: Procedures can be declared to encapsulate a block of code to perform a specific action. Procedures can be called from other functions or procedures but have no return value. A procedure can accept 0 or n number of parameters where <parameter x> represents parameter x name and <parameter type> is a known type or struct. A procedure with no parameters can be called in two ways, <name>; or by <name>(); .e.g., f; or f();.

Example:

```
program ProcedureExample;

procedure WriteSquare(Value: integer);
var Squared : integer;
begin
    writeln('The value is ', Value);
    Squared:=Value*Value;
    writeln('and the value squared is ', Squared);
end.

Begin
    WriteSquare(5);  // Print the square of 5
End.
```

12.2.10 Overriding ontology functions and procedures

Some objects in the ontology have predefined functions (actions) that can be overridden, either as processplan actions, interface actions or as agent actions.
To override a function and implement your own behavior, simply define a procedure or function with the same name in your code.

Processplan action

- onNegotiate
- onRun (processplan)
- onFinished
- onOverlap

Interface action

- onAttach
- onDetach
- onNegotiate
- onOverlap
- onBook
- onUnbook

Agent action

- Constructor
- onDepot
- onDeploy
- Running
- onRun (agent)
- onFail
- onUndeploy
- onFinished
- Activate
- Failed
- Deactivate

12.2.11 Main program block

Section: Main program block
Syntax:
```
begin
[code]
end.
```
Description: Each program can have a main program block and for most objects this is the constructor that will be called when the object is initialised. The main program block must be defined at the end of the program file. The main program block starts at `begin` and ends at `end`. Note that the main program block ends with a period (.).
Example:

```
Program HelloWorld;
{ Program to write Hello World! on the console }
Begin
  writeln('Hello World!');
End.
```

12.3 Control structures

The following control structures exist:

- while
- repeat until
- for
- foreach
- for downto
- if ifelse else
- case
- exit
- break
- error
- label
- parallel

12.3.1 while

Control iterator: while
Syntax 1 - block:
```
while <expression> do
begin
[code block]
end;
```
Syntax 2 – no block:
```
while <expression> do
[single statement]
```
Description: Iterates the code block or statement as long as the expression evaluates to true.
Example:

```
Program WhileIterator;
var Counter :integer;
Begin
    Counter:=5;
    While Counter>0 do
    Begin
        writeln('Countdown ', Counter);
        Counter:=Counter-1;
    end;
End.
```

12.3.2 repeat

Control iterator: repeat
Syntax:
```
repeat
[code block]
until <expression>;
```
Description: Iterates the code block as long as the expression evaluates to true.
Note, the expression is evaluated after the code block is executed.
Example:
```
Program WhileIterator;
var Counter :integer;
Begin
    Counter:=5;
    repeat
        writeln('Countdown ', Counter);
        Counter:=Counter-1;
    until Counter>0;
End.
```

12.3.3 for

Control iterator: for to
Syntax 1 - block:
```
for <variable>:=<fromexpression> to <toexpression> do
begin
[code block]
end;
```
Syntax 2 – no block:
```
for <variable>:=<fromexpression> to <toexpression> do
[single statement]
```

Description: Iterates the code block or the statement n times with an integer variable as a counter. The iteration starts from <fromexpression> and counts up to <toexpression>.
Example:
```
Program ForIterator;
var Counter :integer;
Begin
    for Counter:=1 to 5 do
    begin
        writeln('Count up ', Counter);
    end;
End.
```

12.3.4 foreach

Control iterator: foreach in
Syntax 1 - block:
```
foreach <variable_name> in <iterablevariable> do
begin
[code block]
end;
```
Description: Iterates the code block or the statement over all elements in the <iterablevariable>. The <variable↩
_name> will be a name that can be used to reference each element in the <iterablevariable>.

Note

The <iterablevariable> must be of a type that can be iterable, e.g, arrays, structs or strings.

Example:
```
Program ForEachIterator;
var anArray: array [ 1 .. 4] of integer;

procedure onRun;
begin
    { Initialize the array to -1}
    foreach i in anArray do
```

```
    begin
        i:=-1;
    end;

    writeln(anArray);
end.
```

12.3.5 for dwonto

Control iterator: for dwonto
Syntax 1 - block:

```
for <variable>:=<fromexpression> downto <toexpression> do
begin
[code block]
end;
```

Syntax 2 - no block:

```
for <variable>:=<fromexpression> downto <toexpression> do
[single statement]
```

Description: Iterates the code block or the statement n times with an integer variable as a counter. The iteration starts from fromexpression and countdown to toexpression.
Example:

```
Program ForDowntoIterator;
var Counter :integer;
Begin
    for Counter:=5 downto 1 do
    begin
        writeln('Countdown ', Counter);
    end;
End.
```

12.3.6 if else if else

Control selection: if ifelse else
Syntax 1:

```
if <expression1> then
begin
[code block]
end;
[else if <expression2> then
begin
[code block]
end;]
[else if <expression_n> then
...
]
[else
begin
[code block]
end;]
```

Syntax 2:

```
if <expression1> then
<single statement>
[else if <expression2> then
<single statement>]
[else if <expression_n> then
...
]
[else
<single statement>]
```

Description: Selection control.
Example:

```
Program IfElseIFElse;
var a, b : integer;
```

```
Begin
    writeln('Enter two integers');
    readln(a, b);
    if a<b then
    begin
      writeln('a<b');
    end;
    else if a>b then
    begin
        writeln('a>b');
    end;
    else
    begin
        writeln('a=b');
    end;
End.
```

Note

For syntax 2 each single line expression ends with a semicolon ;
However, it can be safer to work with blocks as in syntax 1 for complex or nested if/else sections.

```
if a>0 then
    writeln('Greater');
else if a=0 then
    writeln('Zero');
else
    writeln('Less');
```

12.3.7 case

Control selection: case
Syntax:

```
case <expression> of
label_1:  [code block]
...
[label_n:] [code block]
[else:] [code block]
end;
```

Description: Case executes a block of code based on the value of an `<expression>`.
The **else** clause is optional and can be used to handle values not matched by any label.
Note: `<expression>` must be of type **integer**, **boolean**, or **string**. Labels must be constants, such as `1`, `2`, `3`, etc. Alternatively, a label can reference a constant variable whose value will be used.

Example:

```
program CaseExample;

var Input:=0: integer;

begin
    writeln('Press 1 to start and 2 to stop:');
    readln(Input);
    case Input of
    1:
        writeln('Start');
    2:
        writeln('Stop');
    else:
        writeln('Unknown option');
    end;
end.
```

12.3.8 exit

Control sequence: exit
Description: Exit exits the current procedure or function and returns control to the caller. If invoked in the main program block, exit stops the program.
Example:

```
Program ExitEarly;
var a : integer;
Begin
    for a:=1 to 10 do
    begin
        if a=5 then
```

```
        begin
            exit;
        end;
        WriteLn('a = ', a);
    end;
End.
```

12.3.9 break

Control sequence: break
Description: Break breaks the current loop and continues to execute after the end of the loop.
Example:

```
Program BreakEarly;
var a : integer;
Begin
    for a:=1 to 10 do
    begin
        if a=5 then
        begin
            break;
        end;
        WriteLn('a = ', a);
    end;
    WriteLn('Loop is ready');
End.
```

12.3.10 error

Control sequence: error
Syntax: error <message>
Description: Error will generate a user error and stops the current program to display a message.
Example:

```
Program Error_Example;

Var a, b : Integer;

Begin
    a:=random(5);
    b:=random(5);

    if b=0 then
        error 'cannot divide by zero';

    writeln(a,'/',b,'=',inttoreal(a)/inttoreal(b));
End.
```

12.3.11 label

Control sequence: label
Syntax: <name>:
Description: A label is a constant value used in a case statement to identify a specific code block. Labels must be constants, such as 1, 2, 3, etc. Alternatively, a label can reference a constant variable, in which case its value is used.
Example:

```
program CaseExample;
var Input:=0: integer;
var CONSTANT SEQ_1:=1, SEQ_2:=2: Integer;
begin
   Input=getValue();

    case Input of
    SEQ_1:
        writeln('Will start sequence ', SEQ_1);
    SEQ_2:
        writeln('Will start sequence ', SEQ_2);
    else:
        writeln('Unknown sequence');
    end;
end.
```

12.3.12 parallel

Control statement: parallel

Syntax: < <statement>; | <statement>; | ... | <statement n>; >
Description: The parallel statement executes all n statements in parallel and waits until all statements have completed. A statement can be a single line or a block enclosed by begin and end, allowing multiple statements to be executed as one parallel branch. If execution is interrupted or the enclosing block requests termination, all running parallel statements are cancelled. Parallel statements are separated by a |

Note

Shared variables and resources accessed by parallel statements are not synchronized automatically. The user is responsible for ensuring safe concurrent access and for avoiding race conditions or conflicting operations on shared resources.

Example:

```
program ParallelExample;
begin

    < writeln('statment 1'); | writeln('statment 2'); | begin writeln('statment 3a'); writeln('statment
      3b'); end. >

end.
```

12.4 Operators

Operators define computations, comparisons, boolean logic, and state updates in expressions.
The following operators exist:

12.4.1 Arithmetic

Arithmetic operators perform numeric computations on numeric operands and produce a numeric result. However, it is also possible to use addition (+) to merge strings.

Operator	Notation
Division	/
Addition	+
Subtraction	-
Multiplication	*
Negative (unaray minus)	-
Modulus	mod

12.4.2 Assignment

Assign values to variables.

Operator	Notation
Assignment	↩ :=

```
var arr : array [ 1 .. 2, 1 .. 2] of integer;
var str :string;
var int : integer;
begin
    str:='Hellow' + ' ' + ' World';
```

```
    writeln(str);

    arr:=[[1, 2], [3, 4]];
    writeln(arr);

    int:=1;
    writeln(int);
end.
```

Note

Assignment is an expression and yields the assigned value. This allows chained assignments such as a := b := c := 1.

12.4.3 Relations

Compare two values and produce a Boolean result.

Operator	Notation
Equeal	=
Less equal	<=
Less	<
More equal	>=
More	>
Not equal	<>

12.4.4 Logical

Combine or negate Boolean expressions and produce a Boolean result.

Operator	Notation
Or	or
Xor	xor
And	and
Not	not

Note

Will only operate on booleans.

Bitwise
Perform bit-level operations on integer operands and produce an integer result.

Operator	Notation
Or	or
Xor	xor
And	and
Complement	not
Shift left	<<
Shift right	>>

Note

Will only operate on integers.

12.4.5 Object

Operator	Notation
Access member operator	.

12.4.6 Array

Operator	Notation
Index operator	[ ]

12.5 Types

The following types exist:

- REAL
- INTEGER
- BOOLEAN
- STRING
- STRUCT
- OBJECT
- REFERENCE

12.5.1 Real

Real syntax: `[-]number[.number]`
Example:

```
Var aVariable : real;
aVariable := -123.456;
```

12.5.2 Integer

Integer 1 syntax: `[-]number`
Description: Represents a standard integer value in decimal format. **Example:**

```
Var aVariable: integer;
aVariable := -1234;
```

Integer 2 syntax: `[-]base#number`
Description: Represents a standard integer in other bases, such as binary (base 2), octal (base 8), decimal (base 10), or hexadecimal (base 16). **Example:**

```
Var aVariable: integer;
aVariable := 2#1001;   // bin 1001 or 9 in decimal
aVariable := 8#10;     // oct 10 or 8 in decimal
aVariable := 16#ff;    // hex ff or 255 in decimal
```

12.5.3 Boolean

Boolean syntax: `[true | false]`
Example:

```
Var aVariable: boolean;
aVariable:=true;
```

12.5.4 String

String syntax: `'text'`
Example:

```
var aVariable : string;
avariable:='Text message';
```

12.5.5 Struct

Struct type variables are data structures based on a user defined type. User defined types are global and act as templates to be used when variables are declared. A Struct type consist of base types (integer, real, boolean or string) or other structs.

12.5.6 Objects

Objects cannot be defined within a program. They exist from the start and they are instances of structs. All agents have an object defined called agent. All interfaces are defined as objects one per interface and share the name of the interface. It is possible to refer to interfaces on other agents by declaring abstract interfaces. Use the dot operator to access any member of an object, variables and skills. If the agent has goals the goal variables are defined under the object called goal.
The following objects exists:

12.5.6.1 this

The entity representing the program. Example of an entity is a goal. See the ontology overview for object definitions.

12.5.6.2 agent

Agent is an object representing the agent running the code. The agent is typically a resource or a part, see the ontology overview for object definitions.

12.5.6.3 interface

Interface is an object that represents an interface on the current agent (local). Interface can be any of the interface objects in the ontology.

12.5.6.4 remoteinterface

Remoteinterface is an object that represents an interface on an remote agent during negotiation.

12.5.6.5 Goal

Goal is an object that represents a goal, only process plans can reach a goal objects

12.5.6.6 ProcessPlan

Processplan is an object that contains possible return values for an onRun() function. Processplan has the following members:

Member	Value	Description
running	1	Function is still running and must be called next cycle
done	2	Function is done

Member	Value	Description
failed	3	Function has failed and cannot continue
interrupted	4	Function was interrupted during the execution
replan	5	Function has run in to a problem and the entire solution must be replanned might trigger a new negotiation
redo	6	Function failed and want to be restarted

Example:

```
Abstract Interface anInterface;

{ will change the value of variable MyVariable defined on agent }
agent.MyVariable:=1;

{ Will change the value of variable A defined on a local interface }
interface.A:=true;

{ will change the value of variable Run defined on a remote abstract interface referred to as anInterface }
anInterface.Run:=true;

{ Will run the skill Execute on a remote abstract interface and at the same time set the Run variable to
      true }
anInterface.Execute();

{ Will print the name of the entity }
writeln(this.Name);

{ For process plans it is possible to access variables defined for goals }
WriteLn('Goal position is ', goal.Pos);

{ Will set return value to done }
onRun:=processplan.done;
```

12.5.7 Reference

CMAS model objects cannot be defined within a program. They exist from the start and they are instances of structs. However, it is possible to get a reference to an object from the system through the override functions or by the getReference function. All reference objects are read only.

Example:

```
function onNegotiate(Planning : boolean; agentObject:reference; interfaceObject:reference): integer;
begin
    writeln('Negotiate with agent ', agentObject.name, ' on interface ', interfaceObject.name);
end.
```

12.6 Procedures

12.6.1 Input and output

12.6.1.1 writeln

Procedure: writeln
Syntax: `writeln([<expression>] [, <expression>] [, ...])`
Description: Write with new line
All expressions will be written to the console and a new line will be added to the end. **Example:**

```
{ Hello World to the console }
writeln('Hello World!');

{ Empty will only output a new line }
writeln();

{ Multi expressions }
writeln('The value of a is ', a, ' and b is ', b);
```

12.6.1.2 write

Procedure: write
Syntax: `write([<expression>] [, <expression>] [, ...])`

Description: Write without a new line
All expressions will be written to the console. **Example:**

```
{ The two wite() outputs will be written on the same line }
write('The value of a is ', a);
write(' and b is ', b);
```

12.6.1.3 readln

Procedure: readln
Syntax: `readln([<variable_1>] [, <variable_2>] [, ...])`
Description: Will read input from the console window and store the input in the given variables.
Example:

```
program Readln_Example;

function Menu() : integer;
var input:string;
begin
    writeln(' 1 - Write hello');
    writeln(' 2 - Write world');
    write('Selection: ');

    // Repeat until a valid integer is read
    repeat
        readln(input); // Read user input from console
    until not stringisinteger(input);
    Menu:=stringtoint(input);
end.

var m:integer;
begin
    m:=Menu();
    if m=1 then
        writeln('hello');
    else if m=2 then
        writeln('world');
    else
        writeln('wrong menu');
end.
```

12.6.2 Abstract Interface Functions

12.6.2.1 abstractname

Procedure: abstractname
Syntax: `abstractname(<interface>] [, <string_name>]) : string`
Description: Will get the <interface> name and if <string_name> is given also change the name.
Return value: the new name if changed otherwise current name.
Example:

```
define handle example : interface;
begin
    var x, y : integer;
end;

begin

    example.negotiate();  // Will negotiate with the name example
    example.run();

    abstractname(example, 'process'); Will negotiate with the name process
    example.run();
end.
```

12.6.2.2 abstractcost

Procedure: abstractcost
Syntax: `abstractcost(<interface>]) : integer`
Description: Will get the total cost to call all planed skills on the abstract interface <interface>. The cost will only include the skills used by the abstract <interface>. If the <interface> is not negotiated the cost returned is 0.
Return value: the total cost.
Example:

```
abstract aResource : interface;

begin
```

```
    aResource.S1();
    aResource.S2();
    writeln('The cost to execute S1 and S2 is ', abstractcost(aResource));

end.
```

12.6.2.3 abstractshare

Procedure: abstractshare
Syntax: `abstractshare(<string_name>, <interface>) : boolean`
Description: Will add the <interface> as a global abstract interface available for all skills on the same agent. The global abstract interface can be found with the help of the <string_name>
Return value: true if it was possible to add the abstract interface.
Example:

```
define handle example : interface;
begin
    var x, y : integer;
end;

begin

    example.negotiateandbook();
    example.run();
    abstractshare('example', example);  // Share the interface

end.
```

12.6.2.4 abstractfind

Procedure: abstractfind
Syntax: `abstractfind(<string_name>, <interface>) : boolean`
Description: Will search for a shared abstract interface by the name <string_name>. If an interface is found it is associated with the <interface>. All variables and functions for the global interface will be available on the local defined interface after a successful find.
Return value: true if it was possible to find a global interface.
Example:

```
define handle example : interface;
begin
end;

begin
    // Find an existing abstract interface
    if not abstractfind('example', example) then error 'unable to find interface';

    example.x:=1;
    example.y:=2;
    example.run();
end.
```

12.6.2.5 ignorebooked

Procedure: ignorebooked
Syntax: `ignorebooked(<interface>] [, <boolean_value>]) : boolean`
Description: Will get the <interface> property IgnoreBooked and if <boolean_value> is given also change the value. This property affects negotiation process and how to handle remote interfaces.
Return value: the new vaule if changed otherwise current value.
Example:

```
define handle example : interface;
begin
    var x, y : integer;
end;

begin

    example.ignoreBooked(example, false);
    example.negotiate();  // Will negotiate with the name example
    example.run();
end.
```

12.6.2.6 ignorebusy

Procedure: ignorebusy
Syntax: `ignorebusy(<interface>] [, <boolean_value>]) : boolean`
Description: Will get the <interface> property IgnoreBusy and if <boolean_value> is given also change the value. This property affects negotiation process and how to handle remote interfaces.
Return value: the new vaule if changed otherwise current value.
Example:

```
define handle example : interface;
begin
    var x, y : integer;
end;

begin

    example.ignoreBusy(example, false);
    example.negotiate();  // Will negotiate with the name example
    example.run();
end.
```

12.6.3 Test Functions

12.6.3.1 isreal

Procedure: isreal
Syntax: `isreal(<expression>) : boolean`
Description: Will test if the <expression> is of the type real.
Return value: true if <expression> is of specific type otherwise false.
Example:

```
Var a:=3.14: real;

begin
    { Will write true }
    writeln(isreal(7.3));
    writeln(isreal(a));
    writeln(isstring('Hello'));

    { Will write false }
    writeln(isreal('Hello'));
    writeln(isstring(7.3));
end.
```

12.6.3.2 isboolean

Procedure: isboolean
Syntax: `isboolean(<expression>) : boolean`
Description: Will test if the <expression> is of the type boolean.
Return value: true if <expression> is of specific type otherwise false.

12.6.3.3 isinteger

Procedure: isinteger
Syntax: `isinteger(<expression>) : boolean`
Description: Will test if the <expression> is of the type integer.
Return value: true if <expression> is of specific type otherwise false.

12.6.3.4 isstring

Procedure: isstring
Syntax: `isstring(<expression>) : boolean`
Description: Will test if the <expression> is of the type string.
Return value: true if <expression> is of specific type otherwise false.

12.6.3.5 stringisreal

Procedure: stringisreal
Syntax: `stringisreal(<string>) : boolean`
Description: Will test if the content of the <string> can be interpreted and converted to real.
Return value: true if <string> can be converted otherwise false.
Example:

```
{ Will write true }
writeln(stringisreal('7.3'));
writeln(stringisreal('-7'));

{ Will write false }
writeln(stringisinteger('Hello'));
writeln(stringisinteger('7.3'));
```

12.6.3.6 stringisinteger

Procedure: stringisinteger
Syntax: `stringisinteger(<string>) : boolean`
Description: Will test if the content of the <string> can be interpreted and converted to integer.
Return value: true if <string> can be converted otherwise false.

12.6.3.7 stringisboolean

Procedure: stringisboolean
Syntax: `stringisboolean(<string>) : boolean`
Description: Will test if the content of the <string> can be interpreted and converted to boolean.
Return value: true if <string> can be converted otherwise false.

12.6.3.8 stringisempty

Procedure: stringisempty
Syntax: `stringisempty(<string>) : boolean`
Description: Will test if the <string> is empty or not.
Return value: true if \<string> is empty otherwise false .
Example:

```
{ Will write true }
writeln(stringisempty(''));

{ Will write false }
writeln(stringisempty('Hello'));
```

12.6.4 Converting Functions

This section documents a set of type conversion procedures available, and they all follow the same general pattern, as described below. Since Structured Text is type-sensitive, it is important to use the correct type. These procedures can be used to assist in converting values or variables to the required types.
Procedure: <type_from>to<type_to>
Syntax: `<type_from>to<type_to>(<expression>) : <type_to>`
Description: Will convert the <expression> value from the <type_from> to <type_to>.
Return value: The <expression> value converted to <type_to>.

Example:

```
Program Example_Convert;

Var a:=1:Integer;
Var b:=3.1:Real;
Var c:=false:Boolean;
```

```
Var d:='Hello':String;

Begin
    WriteLn('a is ', a,
            ' b is ', b,
            ' c is ', c,
            ' and d is ', d);

    c:=inttoboolean(a);
    d:=realtostring(b);
    a:=realtoint(b);
    b:=booleantoreal(c);

    WriteLn('and now a is ', a,
            ' b is ', b,
            ' c is ', c,
            ' and d is ', d);
End.
```

12.6.4.1 realtoint

Procedure: realtoint
Syntax: `realtoint(<expression>) : int`
Description: Will convert the <expression> value from the real to int.
Return value: The <expression> value converted to int.

12.6.4.2 inttoint

Procedure: inttoint
Syntax: `inttoint(<expression>) : int`
Description: Will convert the <expression> value from the int to int.
Return value: The <expression> value converted to int.

12.6.4.3 booleantoint

Procedure: booleantoint
Syntax: `booleantoint(<expression>) : int`
Description: Will convert the <expression> value from the boolean to int.
Return value: The <expression> value converted to int.

12.6.4.4 stringtoint

Procedure: stringtoint
Syntax: `stringtoint(<expression>) : int`
Description: Will convert the <expression> value from the string to int.
Return value: The <expression> value converted to int.

12.6.4.5 realtoreal

Procedure: realtoreal
Syntax: `realtoreal(<expression>) : real`
Description: Will convert the <expression> value from the real to real.
Return value: The <expression> value converted to real.

12.6.4.6 inttoreal

Procedure: inttoreal
Syntax: `inttoreal(<expression>) : real`
Description: Will convert the <expression> value from the int to real.

Return value: The <expression> value converted to real.

12.6.4.7 booleantoreal

Procedure: booleantoreal
Syntax: `booleantoreal(<expression>) : real`
Description: Will convert the <expression> value from the boolean to real.
Return value: The <expression> value converted to real.

12.6.4.8 stringtoreal

Procedure: stringtoreal
Syntax: `stringtoreal(<expression>) : real`
Description: Will convert the <expression> value from the string to real.
Return value: The <expression> value converted to real.

12.6.4.9 realtoboolean

Procedure: realtoboolean
Syntax: `realtoboolean(<expression>) : boolean`
Description: Will convert the <expression> value from the real to boolean.
Return value: The <expression> value converted to boolean.

12.6.4.10 inttoboolean

Procedure: inttoboolean
Syntax: `inttoboolean(<expression>) : boolean`
Description: Will convert the <expression> value from the int to boolean.
Return value: The <expression> value converted to boolean.

12.6.4.11 booleantoboolean

Procedure: booleantoboolean
Syntax: `booleantoboolean(<expression>) : boolean`
Description: Will convert the <expression> value from the boolean to boolean.
Return value: The <expression> value converted to boolean.

12.6.4.12 stringtoboolean

Procedure: stringtoboolean
Syntax: `stringtoboolean(<expression>) : boolean`
Description: Will convert the <expression> value from the string to boolean.
Return value: The <expression> value converted to boolean.

12.6.4.13 realtostring

Procedure: realtostring
Syntax: `realtostring(<expression>) : string`
Description: Will convert the <expression> value from the real to string.
Return value: The <expression> value converted to string.

12.6.4.14 inttostring

Procedure: inttostring
Syntax: `inttostring(<expression>) : string`
Description: Will convert the <expression> value from the int to string.
Return value: The <expression> value converted to string.

12.6.4.15 booleantostring

Procedure: booleantostring
Syntax: `booleantostring(<expression>) : string`
Description: Will convert the <expression> value from the boolean to string.
Return value: The <expression> value converted to string.

12.6.4.16 stringtostring

Procedure: stringtostring
Syntax: `stringtostring(<expression>) : string`
Description: Will convert the <expression> value from the string to string.
Return value: The <expression> value converted to string.

12.6.5 String Functions

12.6.5.1 len

Procedure: len
Syntax: `len(<string>) : integer`
Description: Will return the length of the <string>.
Return value: <string> length as an integer.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln('string s is of length ', len(s));          // Will print 11
    writeln('the first 5 letters of s ', left(s, 5));   // Will print Hello
    writeln('the last 5 letters of s ', right(s, 5));   // Will print World
    writeln('the mid part of s ', mid(s, 5 ,3));        // Will print lo Wo
End.
```

12.6.5.2 left

Procedure: left
Syntax: `left(<string>, <integer>) : string`
Description: Will return a new string based on <string> containing <integer> number of characters from the left side of the <string>.
Return value: A new string.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln('string s is of length ', len(s));          // Will print 11
    writeln('the first 5 letters of s ', left(s, 5));   // Will print Hello
    writeln('the last 5 letters of s ', right(s, 5));   // Will print World
    writeln('the mid part of s ', mid(s, 5 ,3));        // Will print lo Wo
End.
```

12.6.5.3 right

Procedure: right
Syntax: `right(<string>, <integer>) : string`

Description: Will return a new string based on <string> containing <integer> number of characters from the right side of the <string>.
Return value: A new string.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln('string s is of length ', len(s));          // Will print 11
    writeln('the first 5 letters of s ', left(s, 5));   // Will print Hello
    writeln('the last 5 letters of s ', right(s, 5));   // Will print World
    writeln('the mid part of s ', mid(s, 5 ,3));        // Will print lo Wo
End.
```

12.6.5.4 mid

Procedure: mid
Syntax: `mid(<string>, <integer_start>, <integer_length>) : string`
Description: Will return a new string based on <string> containing <integer_length> number of characters starting from <integer_start> of the <string> (zero based index).
Return value: A new string.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln('string s is of length ', len(s));          // Will print 11
    writeln('the first 5 letters of s ', left(s, 5));   // Will print Hello
    writeln('the last 5 letters of s ', right(s, 5));   // Will print World
    writeln('the mid part of s ', mid(s, 5 ,3));        // Will print lo Wo
End.
```

12.6.5.5 trim

Procedure: trim
Syntax: `trim(<string>) : string`
Description: Will return a new string without any leading or trailing whitespace based on <string>.
Return value: a trimmed string without leading or trailing whitespace.
Example:

```
Program Example_String_Functions;

Var s:='   Hello World   ' : String;
Begin
    writeln('-', s, '-');          // Will print -   Hello World   -
    writeln('-', trim(s), '-');    // Will print - Hello World -
End.
```

12.6.5.6 replace

Procedure: replace
Syntax: `replace(<string_input>, <string_find>, <string_replace>) : string`
Description: Will return a new string based on <string_input> where all occurrences of <string_find> is replaced with <string_replace>. If <string_replace> is empty all occurrences of <tring_find> will be deleted.
Return value: a new string that does not contain <string_find>.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln(replace(s, 'l', '.'));          // Will print He..o Wor.d
    writeln(replace(s, 'l', ''));           // Will print Heo Word
End.
```

12.6.5.7 contains

Procedure: contains
Syntax: `contains(<string_input>, <string_find>) : boolean`
Description: Check if <string_input> contains <string_find>.
Return value: True if <string_find> is a part of <string_input>.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln(contains(s, 'Hello'));          // Will print true
    writeln(contains(s, 'hello'));          // Will print false
End.
```

12.6.5.8 tolowercase

Procedure: tolowercase
Syntax: `tolowercase(<string>) : string`
Description: Converts all of the characters in <string> to lowercase.
Return value: a new string with all character in lowercase.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln(tolowercase(s));          // Will print hello world
End.
```

12.6.5.9 touppercase

Procedure: touppercase
Syntax: `touppercase(<string>) : string`
Description: Converts all of the characters in <string> to uppercase.
Return value: a new string with all character in uppercase.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln(touppercase(s));          // Will print HELLO WORLD
End.
```

12.6.5.10 split

Procedure: split
Syntax: `split(<string>, <string_find>) : string`
Description: Will split <string> if the <string_find> is found and return the first part.
Return value: a new string.
Example:

```
Program Example_String_Functions;

Var s:='Hello World' : String;
Begin
    writeln(split(s, ' '));          // Will print Hello
End.
```

12.6.5.11 tostring

Procedure: tostring
Syntax: `tostring(<expression>) : string`
Description: Will return a string based on the <expression>.

Return value: <expression> converted to a string.
Example:

```
Program Example_String_Functions;

Var value:=4 : Integer;
Var strvalue : String;
Begin
    strvalue:=toString(value*2);     // Convert expression to a string
    writeln('strvalue=', strvalue);  // Will print 8
End.
```

12.6.5.12 chr

Procedure: chr
Syntax: `chr(<integer>) : string`
Description: Converts an <integer> representing an ASCII code into the corresponding character and returns it as a single-character <string>.
Return value: A <string> containing the character corresponding to the given ASCII value.
Example:

```
Program Example_Chr;

Begin
    writeln(chr(65));  // Writes 'A' to the console.
End.
```

12.6.5.13 ord

Procedure: ord
Syntax: `ord(<string>) : integer`
Description: Converts a single-character <string> into its corresponding ASCII code and returns the result as an <integer>.
Return value: An <integer> representing the ASCII value of the given character.
Example:

```
Program Example_Ord;

Begin
    writeln(ord('A'));  // Writes 65 to the console.
End.
```

Note

Chr and Ord procedure operates on ASCII character encoding only. Unicode, UTF-8, and other multibyte character encodings are not supported.

12.6.5.14 space

Procedure: space
Syntax 1: `space(<integer>) : string`
Description: Will return a string with <integer> spaces.
Return value: a string.
Example:

```
Program Example_String_Functions;

Begin
    writeln('Hello', space(4), 'world!');  // Will print 'Hello    world!'
End.
```

Procedure: space
Syntax 2: `space(<integer>, <string>) : string`
Description: Will return a string based on <string> padded with spaces. The string is always <integer> long.
Return value: Returns a left-aligned string padded with trailing spaces to the specified <integer> width.
Example:

```
Program Example_String_Functions_2;

var number, counter : integer;
Begin
    for counter := 1 to 10 do begin
        number:=random(100);
        // Will print random number in a table format
        writeln(space(4, inttostring(number)), '#', counter);
    end;
End.
```

12.6.6 JSON functions

12.6.6.1 GetJElement

Procedure: getjelement
Syntax: `getjelement(<string_json>, <string_element>) : string`
Description: Will get <string_elemnt> value from a JSON string <string_json>.
Return value: the value as a string.
Example:

```
Program Example_JSON;
Var JSONString:='{"value": "Hello World"}' : String;

Begin
     WriteLn(getjelement(JSONString, 'value'));             // Will print Hello World
End.
```

12.6.6.2 SetJElement

Procedure: setjelement
Syntax: `setjelement(<string_json>, <string_element>, <string_value>) : string`
Description: Will set <string_element> to <string_value> in the JSON string <string_json>.
Return value: a new JSON string with the new value.
Example:

```
Program Example_JSON;
Var JSONString:='{"value": "Hello World"}' : String;

Begin
     WriteLn(setjelement(JSONString, 'value', 'HI!'));             // Will print {"value": "HI!"}
End.
```

12.6.6.3 JToStruct

Function: jtostruct
Syntax: `jtostruct(<string_json>) : struct`
Description: Will return a new struct based on the JSON string <string_json>.
Return value: a new struct with the values based on <string_json>.
Example:

```
Program Example_JSON;
Var JSONString:='{"value": "Hello World"}' : String;

struct object;
begin
    var value : string;
end;

var data:object;

Begin
    data:=jtostruct(JSONString);
    WriteLn(data);             // Will print {"value":"Hello World!"}
End.
```

12.6.6.4 StructToJ

Function: structtoj
Syntax: `structtoj(<struct>) : string`
Description: Will return a new JSON string based on the <struct>.
Return value: a new JSON string.
Example:

```
Program Example_JSON;
struct object;
begin
    var value : string;
end;

var data:=(value:='hello world!'):object;
var str:string;

Begin
    str:=structtoj(data);
    WriteLn(str);             // Will print {"value":"Hello World!"}
End.
```

12.6.6.5 GetJArray

Procedure: getjarray
Syntax: `getjarray(<string_json>, <integer_index>) : string`
Description: Will get index <integer_index> from a JSON array <string_json>. Index is zero based.
Return value: the value of as a string.
Example:

```
Program Example_JSON;
Var JSONString:='[{"x":1}, {"x":2}]' : String;

Begin
     WriteLn(getjarray(JSONString, 0));             // Will print {x:1}
End.
```

12.6.6.6 IsJson

Function: isjson
Syntax: `isjson(<string>) : boolean`
Description: Will return true if string is a valid JSON object description.
Return value: true if string is valid otherwise false.
Example:

```
Program Example_JSON;

Begin
    writeln(isjson('{"x":1}'));             // Will print TRUE
End.
```

12.6.6.7 IsJsonArray

Function: isjsonarray
Syntax: `isjsonarray(<string>) : boolean`
Description: Will return true if string is a valid JSON array description.
Return value: true if string is a valid array otherwise false.
Example:

```
Program Example_JSON;

Begin
    writeln(isjsonarray('[{"x":1}, {"x":2}]'));             // Will print TRUE
End.
```

12.6.6.8 JsonWrite

Function: jsonwrite

Syntax: `jsonwrite(<string>) : string`
Description: Will write the string as a valid JSON description in the console.
Return value: return a string.
Example:

```
Program Example_JSON;

Begin
    jsonwrite(('[{"x":1}, {"x":2}]'));
End.
```

12.6.7 Math functions

Procedure: <math_function>
Syntax: `<math_function>(<expression>) : integer | real`
Description: The <mat function> accept a real or integer <expression>. The math functions will return a value based on the input type, i.e., if the <expression> input type is integer the output will be of type integer. Note that, the min and max functions requires two expressions that must match by type.
Return value: An integer or a real number.
Example:

```
Program Math;

Begin
    WriteLn( abs( -1) );   { Will write 1 }
    WriteLn( abs( -1.0) ); { Will write 1.0 }

    WriteLn( random(10) ); { Will write a random number between 0-9 }
    WriteLn( random(1.0) ); { Will write a random number between 0.0 and 1.0 }

    WriteLn( min(3, 2) );  { Will write 2 }
    WriteLn( max(3, 2) );  { Will write 3 }

End.
```

List of all math functions:

Function name	Description
abs	Absolute value of <expression>
ceil	Round <expression> up to nearest integer value.
floor	Round <expression> down to nearest integer value.
round	Round <expression> to the nearest integer value.
max	Returns the greater value of <expression1> and <expression2>
min	Returns the smaller value of <expression1> and <expression2>

Function name	Description
exp	e raised to the power of <expression>
log	natural logarithm of <expression>
log10	10 logarithm of <expression>
sqrt	square root of <expression>

Function name	Description
sin	trigonometric sine of an angle in radians

Function name	Description
cos	trigonometric cosine of an angle in radians
tan	trigonometric tangent of an angle in radians
asin	arc sine in radians
acos	arc cosine in radians
atan	arc tangent in radians
sinh	hyperbolic sine
cosh	hyperbolic cosine
tanh	hyperbolic tangent
todegrees	convert radians to degrees
toradians	convert degrees to radians

Function name	Description
random	generate a pseudo random number between 0 and <expression>

12.6.7.1 abs

Procedure: abs
Syntax: `abs(<expression>) : <type>`
Description: Absolute value of <expression>.
Return value: Will return the non-negative value of the input.

12.6.7.2 ceil

Procedure: ceil
Syntax: `ceil(<expression>) : <type>`
Description: Round <expression> up to nearest integer value.
Return value: Will return the smallest integer greater than or equal to the input.

12.6.7.3 floor

Procedure: floor
Syntax: `floor(<expression>) : <type>`
Description: Round <expression> down to nearest integer value.
Return value: Will return the largest integer less than or equal to the input.

12.6.7.4 round

Procedure: round
Syntax: `round(<expression>) : <type>`
Description: Round <expression> to the nearest integer value.
Return value: Will return the closest integer to the input.

12.6.7.5 max

Procedure: max
Syntax: `max(<expression1>, <expression2>) : <type>`
Description: Returns the greater value of <expression1> and <expression2>. Note, <expression1> and <expression2> must be of the same type.
Return value: Will return the larger of the two input values.

12.6.7.6 min

Procedure: min
Syntax: `min(<expression1>, <expression2>) : <type>`
Description: Returns the smaller value of <expression1> and <expression2>. Note, <expression1> and <expression2> must be of the same type.
Return value: Will return the smaller of the two input values.

12.6.7.7 exp

Procedure: exp
Syntax: `exp(<expression>) : <type>`
Description: e raised to the power of <expression>.
Return value: Will return the result of e^<expression>.

12.6.7.8 log

Procedure: log
Syntax: `log(<expression>) : <type>`
Description: Natural logarithm of <expression>.
Return value: Will return the base-e logarithm of the input.

12.6.7.9 log10

Procedure: log10
Syntax: `log10(<expression>) : <type>`
Description: 10 logarithm of <expression>.
Return value: Will return the base-10 logarithm of the input.

12.6.7.10 sqrt

Procedure: sqrt
Syntax: `sqrt(<expression>) : <type>`
Description: Square root of <expression>.
Return value: Will return the square root of the input.

12.6.7.11 sin

Procedure: sin
Syntax: `sin(<expression>) : <type>`
Description: Trigonometric sine of an angle in radians.
Return value: Will return the sine of the input angle.

12.6.7.12 cos

Procedure: cos
Syntax: `cos(<expression>) : <type>`
Description: Trigonometric cosine of an angle in radians.
Return value: Will return the cosine of the input angle.

12.6.7.13 tan

Procedure: tan
Syntax: `tan(<expression>) : <type>`
Description: Trigonometric tangent of an angle in radians.
Return value: Will return the tangent of the input angle.

12.6.7.14 asin

Procedure: asin
Syntax: `asin(<expression>) : <type>`
Description: Arc sine in radians.
Return value: Will return the angle in radians whose sine is the input.

12.6.7.15 acos

Procedure: acos
Syntax: `acos(<expression>) : <type>`
Description: Arc cosine in radians.
Return value: Will return the angle in radians whose cosine is the input.

12.6.7.16 atan

Procedure: atan
Syntax: `atan(<expression>) : <type>`
Description: Arc tangent in radians.
Return value: Will return the angle in radians whose tangent is the input.

12.6.7.17 sinh

Procedure: sinh
Syntax: `sinh(<expression>) : <type>`
Description: Hyperbolic sine.
Return value: Will return the hyperbolic sine of the input.

12.6.7.18 cosh

Procedure: cosh
Syntax: `cosh(<expression>) : <type>`
Description: Hyperbolic cosine.
Return value: Will return the hyperbolic cosine of the input.

12.6.7.19 tanh

Procedure: tanh
Syntax: `tanh(<expression>) : <type>`
Description: Hyperbolic tangent.
Return value: Will return the hyperbolic tangent of the input.

12.6.7.20 todegrees

Procedure: todegrees
Syntax: `todegrees(<expression>) : <type>`
Description: Convert radians to degrees.
Return value: Will return the input angle converted from radians to degrees.

12.6.7.21 toradians

Procedure: toradians
Syntax: `toradians(<expression>) : <type>`
Description: Convert degrees to radians.
Return value: Will return the input angle converted from degrees to radians.

12.6.7.22 random

Procedure: random
Syntax: `random(<expression>) : <type>`
Description: Generate a pseudo random number between 0 and <expression>.
Return value: Will return a pseudo random number in the range [0, <expression>).

12.6.8 Array Functions

12.6.8.1 length

Procedure: length
Syntax: `length(<any_array>) : integer`
Description: Will return the length of an array.
Return value: Array length.
Example:

```
Program Example_Array;

var MyArray : array [1 .. 3] of real;
var Sum:=0.0 : real;
```

```
var i : integer;
var Input : string;

Begin
    for i:=startindex(MyArray) to endindex(MyArray) do
    begin
       Input:=InputDialog('Give input for index ' + inttostring(i), '1');
       if stringisreal(Input) then
            MyArray[i]:=stringtoreal(Input);
    end;

    for i:=startindex(MyArray) to endindex(MyArray) do
    begin
        Sum:=Sum+MyArray[i];
    end;

    writeln('The sum of the array is ', Sum, ' and the average value ', Sum/inttoreal(length(MyArray)));
End.
```

12.6.8.2 startindex

Procedure: startindex
Syntax: `startindex(<any_array>) : integer`
Description: Will return the first index of an array.
Return value: First index.
Example: see length.

12.6.8.3 endindex

Procedure: endindex
Syntax: `endindex(<any_array>) : integer`
Description: Will return the last index of an array.
Return value: Last index.
Example: see length.

12.6.9 HMI

12.6.9.1 messagebox

Procedure: messagebox
Syntax: `messagebox(<string_message>, [<string_title>], [<integer_options>]) : integer`
Description: Will show an message box with a user message <string_message>. It is possible, but optional, to give the dialog a title <string_title>. <integer_option> gives a possibility to change the type of message and number of buttons, see example for the values.
Return value: The button pressed, see example for values.

Note

The constants used in the example, e.g, DIALOG_SHOW_YES_NO, are also defined in the system file messagebox.inc. An easier approach than defining all constants is to included all by the use of $include 'system:messagebox.inc'

Example:

```
{ An example program how to show HMI messages }
Program MessageBox_Example;

{ Return value constants for messagebox }
Var CONSTANT
    DIALOG_ANSWER_YES:=0,
    DIALOG_ANSWER_NO:=1,
    DIALOG_ANSWER_CANCEL:=2,
    DIALOG_ANSWER_CLOSED:=-1 :Integer;


{ Option constants for type of messagebox to show }
```

```
Var CONSTANT
    DIALOG_SHOW_YES_NO:=0,
    DIALOG_SHOW_YES_NO_CANCEL:=1,
    DIALOG_SHOW_OK_CANCEL:=2,
    DIALOG_ERROR_MESSAGE:=3,
    DIALOG_INFORMATION_MESSAGE:=4,
    DIALOG_WARNING_MESSAGE:=5,
    DIALOG_QUESTION_MESSAGE:=6,
    DIALOG_PLAIN_MESSAGE:=-1 :Integer;


{ Main procedure }
Var Result : Integer;
Begin
    Result:=MessageBox('Press a button', 'Title', DIALOG_SHOW_YES_NO_CANCEL);

    if Result=DIALOG_ANSWER_YES then
    begin
      WriteLn('You pressed Yes');
    end
    else if Result=DIALOG_ANSWER_NO then
    begin
      WriteLn('You pressed No');
    end
    else if Result=DIALOG_ANSWER_CANCEL then
    begin
      WriteLn('You pressed Cancel');
    end
    else
    begin
      WriteLn('You closed the window');
    end;
End.
```

12.6.9.2 inputdialog

Procedure: inputdialog
Syntax: `inputdialog(<string_prompt>, [<string_default_value>]) : string`
Description: Will show an input dialog window containing a message <string_prompt> and
an input filed with a default value of <string_default_value>. The user can input any string value in the dialog.
Return value: The value entered by the user or an empty string if the user clicks cancel.
Example:

```
{ An example program how to get user input strings from HMI }
Program InputDialog_Example;

Var Result : String;
Var DefaultValue:='3' : String;
Var Iterator: Integer;

{ Main procedure }
Begin
    Result:=InputDialog('How many iterations?', DefaultValue);

    if not StringIsEmpty(Result) and StringIsInteger(Result) then
    begin
       Iterator:=StringToInt(Result);

       while Iterator>0 do
       begin
         WriteLn('Loop - ', Iterator);
         Iterator:=Iterator-1;
       end;
    end;
End.
```

12.6.9.3 optiondialog

Procedure: optiondialog
Syntax 1: `optiondialog(<integer_handle>, <integer_value>, <string_message>, <string_title>, <string_option_1> [,<string option_2>] [, ...]) : boolean`

Syntax 2: `optiondialog(<integer_handle>, <boolean_condition>) : boolean`
Description: Will show a message box with a user message <string_message> and a title <string_title>. <string_option_1> is a string associated with a button 1. It is possible to give many <string_options> to create many buttons.The integer handle must be unique to create a new dialog. A handle can be reused if the dialog is

closed in between or if you want to refer to an already open dialog.

Note

The OptionDialog call is MODELESS, i.e., the execution of the code will not stop and wait for an answer. The code must handle this by constantly checking the <integer_value> to get the result.

For syntax 2 it is possible to close an open OptionDialog by setting the <boolean_condition> to false. Further, if the handle is -1 all open OptionDialogs will be closed. If <boolean_condition> is true it is possible to check the return value to see if the dialog is open or closed.

Note

An attempt to close an already closed OptionDialog will generate an error.

Return value: For syntax 1 True if the dialog was create as a new dialog, false if not. For syntax 2 True if the dialog is open or false if closed.
Example:

```
Program ExampleOptionDialog;
Var Selection : Integer;
{ Main procedure }
Begin
    Selection:=-1;
    OptionDialog(1, Selection, 'Press a button', 'Selection', 'Button 1', 'Button 2', 'Button 3');

    While Selection=-1 do
    Begin
        sleep(50);
    End;

    writeln('You selected button number ', Selection);

    // Check if dialog is still open
    if optiondialog(1, true) then
        optiondialog(1, false); // Close the dialog
End.
```

12.7 System Functions

12.7.1 Time

12.7.1.1 sleep

Procedure: sleep
Syntax 1: `sleep(<integer_time>)`
Syntax 2: `sleep(<string_time>)`
Description: Will causes the execution to sleep for the specified number of milliseconds <integer_time>. For syntax 2 a string is accepted that must include the time with an optional suffix. Valid suffixes include: h (hour), m (minutes), s (seconds), and ms (milliseconds). For example the string, '2s' represents 2 seconds.
Example:

```
Program Sleep_Example;

Begin
  WriteLn('Wait for it ...');
  sleep(3000);
  WriteLn('wait for it ...');
  sleep('3s');
  WriteLn('legendary!');
End.
```

12.7.1.2 time

Procedure: time
Syntax: `time() : integer`

Description: Will return the time since start as an integer.
Return value: <integer>
Example:

```
Program Time_Example;
Var t:integer;
Begin
    t=time();
    sleep(100);
    writeln(time()-t); // Will write aprox. 100
End.
```

12.7.1.3 beep

Procedure: beep
Syntax: `beep()`
Description: Will make a system beep.
Example:

```
Program Beep_Example;
Begin
    beep();
    sleep(1000);
    beep();
End.
```

12.7.1.4 memoryfree

Procedure: meoryfree
Syntax: `memoryfree() : integer`
Description: Will give the amount of free memory.
Return value: <integer> as an approximation of the total amount of memory currently available, measured in MB.
Example:

```
Program Memory_Example;
Begin
    writeln('Free memory in MB = ', memoryFree());
End.
```

12.7.1.5 memorytotal

Procedure: meorytotal
Syntax: `memorytotal() : integer`
Description: Will give the total memory size.
Return value: <integer> as the total amount of memory available, measured in MB.

Note

The memory functions gives an approximation of the memory handled by the underlying virtual machine. For each call to a memory function the value might change even if the amount of memory used by CMAS is fixed.

Example:

```
Program Memory_Example;
Begin
    writeln('Total memory in MB = ', memoryTotal());
End.
```

12.7.1.6 memoryused

Procedure: meoryused
Syntax: `memoryused() : integer`

Description: Will give the used memory size as memoryTotal() - memoryFree()().
Return value: <integer> the amount of used memory, measured in MB.
Example:

```
Program Memory_Example;
Begin
    writeln('Used memory in MB = ', memoryUsed());
End.
```

12.7.2 Preferences

12.7.2.1 preferencevalue

Procedure: preferencevalue
Syntax: `preferencevalue(<string_name>, [<value>>]) : string`
Description: Will set or get a global preference value. All available values can be found in the Preferences. **Return value:** <string>
Example:

```
Program Example;

Begin
    writeln('RetryOnBusy=', preferencevalue('RetryOnBusy', false));
End.
```

12.7.3 Debug

12.7.3.1 threadmessage

Procedure: threadmessage
Syntax: `threadmessage([<string_message>]) : string`
Description: Will set or get the user message for current agent thread. A thread user message can be seen in the Agent thread viewer during runtime and the user message is ignored by the system. It can be use to display various information for e.g. debug purposes.
Return value: <string>
Example:

```
Program Example;

Begin
    threadmessage('Init of agent - running constructor');
End.
```

12.7.4 File Functions

12.7.4.1 open

Procedure: open
Syntax: `open(<string_filename>, <integer_filemode>) : integer`
Description: Will open the <string_filename> file. The <integer_filemode> decides if the file is opened for reading or writing.
Return value: a file handle (integer) that can be used to access the file. If file handle<0 an error occurred and the file cannot be opened.
Example:

```
program FileExample;
$include 'system:file.inc'

var fileHandle:integer;

begin
    fileHandle:=open('test.txt', MODE.WRITE_TEXT);
    fwriteln(fileHandle, 'Hello World!');
    fwriteln(fileHandle, 1);
    fwriteln(fileHandle, true);
    close(fileHandle);
end.
```

12.7.4.2 MODE

Mode is an object that contains possible values for open() and is defined in the system include file file.inc. Use $include 'system:file.inc' to include the object. Mode has the following members:

Member	Value	Description
WRITE_TEXT	1	Create a new text file for writing.
READ_TEXT	2	Open an existing text file for reading.

12.7.4.3 close

Procedure: close
Syntax: `close(<integer_filehandle>) : boolean`
Description: Will close the <integer_filehandle> file. The <integer_handle> must be a valid file handle returned from an open.
Return value: true if the file was closed otherwise false.
Example:

```
program FileExample;
$include 'system:file.inc'

var fileHandle:integer;

begin
    fileHandle:=open('test.txt', MODE.WRITE_TEXT);
    fwriteln(fileHandle, 'Hello World!');
    fwriteln(fileHandle, 1);
    fwriteln(fileHandle, true);
    close(fileHandle);
end.
```

12.7.4.4 fwriteln

Procedure: fwriteln
Procedure: fwrite
Syntax: `fwriteln(<integer_filehandle>, [<expression>] [, <expression>] [, ...])`
Description: Will print all <expression> to the <integer_filehandle> file. The <integer_handle> must be a valid file handle. fwriteln will add a new line after each write and fwrite will not add any new line.
Example:

```
program FileExample;
$include 'system:file.inc'

var fileHandle:integer;

begin
    fileHandle:=open('test.txt', MODE.WRITE_TEXT);
    fwriteln(fileHandle, 'Hello World!');
    fwriteln(fileHandle, 1);
    fwriteln(fileHandle, true);
    close(fileHandle);
end.
```

12.7.4.5 freadln

Procedure: freadln
Syntax: `freadln(<integer_filehandle>, [<variable_1>] [, <variable_2>] [, ...]) : boolean`
Description: Will read values from the <integer_filehandle> file and store the result in <variable>. The <integer_handle> must be a valid file handle returned from an open.
Return value: Number of <variable> assigned a value.
Example:

```
program FileExample;
```

```
$include 'system:file.inc'

var fileHandle : integer;
var s:string;
var i:integer;
var b:boolean;

procedure readFromFile1;
begin
    fileHandle:=open('test.txt', MODE.READ_TEXT);

    // Read each line in the file and convert to a specific variable type
    freadln(fileHandle, s);
    freadln(fileHandle, i);
    freadln(fileHandle, b);

    writeln(s);
    writeln(i);
    writeln(b);
    close(fileHandle);
end.

procedure readFromFile2;
begin
    fileHandle:=open('test.txt', MODE.READ_TEXT);

    // Treat each line in the file as a string
    while freadln(fileHandle, s)>0 do
    begin
        writeln(s);
    end;

    close(fileHandle);
end.

begin
    // Will generate the same output on the screen
    readFromFile1();
    readFromFile2();
end.
```

12.8 Server API Functions

12.8.1 Pause

12.8.1.1 pauseAll

Procedure: pauseAll
Syntax: `pauseAll()`
Description: Will pause all running agents.
Example:

```
Begin
    pauseAll();
End.
```

12.8.1.2 pausebyname

Procedure: pauseByName
Procedure: pauseById
Procedure: pauseByInstanceId

Syntax: `pauseByName(<string_ID)`
Syntax: `pauseByID(<string_ID)`
Syntax: `pauseByInstanceID(<string_ID)`
Description: Will pause all running agents that match <string_ID>.
Example:

```
Begin
    // Will pause all drill agents
    pauseByName('drill');

    // Will pause all agent with a ID=1
    pauseByID('1');
```

```
End.
```

12.8.2 Console and log

12.8.2.1 clearconsole

Procedure: clearconsole
Syntax: `clearconsole()`
Description: Will clear the console window.
Example:

```
Begin
    clearconsole();
    writeln('Hello World!')
End.
```

12.8.2.2 clearlog

Procedure: clearlog
Syntax: `clearlog()`
Description: Will clear the log window.
Example:

```
Begin
    clearlog();
End.
```

12.8.3 Runtime control

12.8.3.1 isruntime

Procedure: isruntime
Syntax: `isruntime()`
Description: Will check if runtime environment is running. This is only valid if you test run inside the editor.
Example:

```
Begin
    if not isruntime() then error 'not in runtime';
End.
```

12.8.3.2 stopsystem

Procedure: stopsystem
Syntax: `stopsystem()`
Description: Will stop the runtime environment.
Example:

```
Begin
    stopsystem();
End.
```

12.8.3.3 restartsystem

Procedure: restartsystem
Syntax: `restartsystem()`
Description: Will restart the runtime environment.
Example:

```
Begin
    restartsystem();
End.
```

12.8.3.4 getagentsbyname

Procedure: getagentsbyname
Syntax: `getagentsbyname(<list>, <string_id>)`
Description: Will get all running agents that match <string_ID> and store all instance ids in the string array <list>.
Example:

```
program Example_GetAgents;

Var list: array [ 1 .. 2] of String;

procedure onDeploy;
begin
    getagentsbyname(list, 'drill');
    writeln(list);
End.
```

12.8.3.5 getagentsbyid

Procedure: getagentsbyid
Syntax: `getagentsbyid(<list>, <string_id>)`
Description: Will get all running agents that match <string_ID> and store all instance ids in the string array <list>.
Example:

```
program Example_GetAgents;

Var list: array [ 1 .. 2] of String;

procedure onDeploy;
begin
    getagentsByID(list, 'drill');
    writeln(list);
End.
```

12.8.3.6 deployandattach

Procedure: deployandattach
Syntax 1: `deployandattach(<string_name>, <string_deployinterfacename>, <string_localinterfaceid>) : boolean`
Description: Will deploy an agent that match the name with <string_name> and attach the newly deployed agent interface named <string_deployinterfacename> to a local interface with an ID of <string_localinterfaceid>.
Syntax 2: `deployandattach(<string_name>) : boolean`
Description: Will deploy an agent that match <string_name> and attach the new agent interface to an compatible interface of the calling agent. If the deployed agent is a part the goals are also searched to check for compatible interfaces.
Return value: true if any agent was deployed and attached otherwise **false**.
Example:

```
program Example;

begin
    DeployAndAttach('drill');
End.
```

12.8.3.7 deploybyinstanceid

12.8.3.8 deploybyid

12.8.3.9 deploybyname

Procedure: deploybyname
Syntax: `deploybyname(<string_id>) : boolean`
Syntax: `deploybyinstanceid(<string_id>) : boolean`
Syntax: `deploybyid(<string_id>) : boolean`
Description: Will deploy agents that match <string_ID>.

Return value: true if any agent was deployed otherwise **false**.
Example:

```
program Example;

begin
    DeployByName('drill');
End.
```

12.8.3.10 deployall

Procedure: deployall
Syntax: `deployall() : integer`
Description: Will deploy all agents in the depot.
Return value: number of agents deployed.
Example:

```
program Example;

var n:integer;

begin
    n:=DeployAll();
    writeln('deployed ', n, ' agents.');
End.
```

12.8.3.11 undeploybyname

Procedure: undeploybyname
Syntax: `undeploybyname(<string_id>) : boolean`
Description: Will undeploy agents that match <string_ID>.
Return value: true if any agent was undeployed otherwise **false**.
Example:

```
program Example;

begin
    UnDeployByName('drill');
End.
```

12.8.3.12 undeploybyid

Procedure: undeploybyid
Syntax: `undeploybyid(<string_id>) : boolean`
Description: Will undeploy agents that match <string_ID>.
Return value: true if any agent was undeployed otherwise **false**.
Example:

```
program Example;

begin
    UnDeployByID('idResource');
End.
```

12.8.3.13 undeploybyinstanceid

Procedure: undeploybyinstanceid
Syntax: `undeploybyinstanceid(<string_id>) : boolean`
Description: Will undeploy agents that match <string_ID>.
Return value: true if any agent was undeployed otherwise **false**.
Example:

```
program Example;

begin
    UnDeployByInstanceID(agent.InstanceID);
End.
```

12.8.3.14 undeployall

Procedure: undeployall
Syntax: `undeployall() : integer`
Description: Will undeploy all agents.
Return value: number of agents undeployed.
Example:

```
program Example;

var n:integer;

begin
    n:=UnDeployAll();
    writeln('undeployed ', n, ' agents.');
End.
```

12.8.3.15 getnumberofrunningparts

Procedure: getnumberofrunningparts
Syntax: `getnumberofrunningparts() : integer`
Description: Will count number of running part agents.
Return value: number of running parts.
Example:

```
program Example;
var n:integer;
begin
    n:=getnumberofrunningparts();
    writeln('Number of parts ', n);
End.
```

12.8.3.16 getnumberofrunningresources

Procedure: getnumberofrunningresources
Syntax: `getnumberofrunningresources() : integer`
Description: Will count number of running resource agents.
Return value: number of running resources.
Example:

```
program Example;
var n:integer;
begin
    n:=getnumberofrunningresources();
    writeln('Number of resources ', n);
End.
```

12.8.3.17 attach

Procedure: attach
Syntax 1: `attach(string_id1, string_id2) : boolean`
Description: Will attach two interfaces together based on the IDs.
Note: Two interface that can be attached will always be attache by this method regardless of their position. **Return value:** true on a successful attachment.
Syntax 2: `attach(abstract_interface, string_target) : boolean`
Description: Will try to attach two interfaces together by invoking the planner to find a solution on how to do the attachment. string_target is a name of a remote interface.
Return value: true on a successful attachment.

Example:

```
program Example;
begin
    attach('Interface1ID', 'Interfcae2ID');
End.
```

12.8.3.18 assembly

Procedure: assembly
Syntax: `assembly(abstract_interface, string_name) : integer`
Description: Will try to assemble *string_name* through and abstract interface *abstract_interface* by invoking a planner to find a way to do assembly.
Return value: process return value.
Example:

```
program Example;
abstract product : interface;

begin
     { Will search for Product2 and if compatible it will be assembled through product interface with us.
     Note, that after assembly Product2 will be marked FINISHED and removed. }
    assembly(product, 'Product2');
End.
```

12.8.3.19 getmodelstates

Procedure: getmodelstates
Syntax : `getmodelstates(<string_name> [, <boolean_mode>]) : string`
Description: Will get the model state from the ModelState object that is named string_name and return it as a JSON array string (default). If boolean_mode is omitted or set to to true an array is generated, if boolean_mode is set to false an JSON object is generated.
Return value: The states as a JSON array/object or an empty string on failure.
Example:

```
program Example;

var s: string;

begin
    s=getmodelstates('modelstate');
    writeln(s);
End.
```

12.8.4 Object access

12.8.4.1 getobject

Procedure: getobject
Syntax: `getobject(string_id, struct_var) : boolean`
Description: Will map the CMAS object identified by string_id and map that to the struct_var. Note, most of the ontology object are defined in the ontology.inc that can included.
Return value: true if mapped.
Example:

```
{ An example program }
Program Example;

struct Entity;
begin
    var Name : String;
    var Id : String;
    var InstanceId : String;
    var Description : String;
end;

var anObject : Entity;

procedure onRun;
begin
    if not getobject('id', anObject) then
        error 'unable to map object';

    // Set a new run time name
    anObject.name:='New name';
    writeln(anObject);
end.
```

12.8.4.2 getrelation

Procedure: getrelation
Syntax: `getrelation(string_id, array_var, string_relation) : boolean`
Description: Will map a relation named string_relation in a CMAS object and associate that with the array array_var. The CMAS object is identified by string_id.
Return value: true if mapped.
Example:

```
{ An example program }
Program Example;
{ Include ontology to get the defenition of Hazrad }
$include 'system:ontology.inc'
var items: array [ 1 .. 2 ] of Hazard;
var risk :integer;
procedure onRun;
begin
   // Get my own relation Hazards and map that to the items array
    getrelation(this.ID, items, 'Hazards');

    // Loop for all Hazards
    foreach i in items do
        risk:=risk+i.risk;
    writeln('Total risk as a sum = ', risk);
end.
```

12.8.4.3 getreference

Procedure: getreference
Syntax1: `getreference(string_id, reference_var) : boolean`
Description: Will get a reference to the CMAS object identified by string_id and map that to the reference_var.
Return value: true if mapped.
Example:

```
{ An example program }
Program Example;

var anObject : reference;

procedure onRun;
begin
    if not getreference('id_robot', anObject) then
        error 'unable to get a reference to object';

    // Write object name to screen
    writeln(anObject.name);
end.
```

Procedure: getreference
Syntax2: `getreference(object, reference_var) : boolean`
Description: Will get a reference to the object and map that to the reference_var.
Return value: true if mapped.
Example:

```
{ An example program }
Program Example;

var anObject : reference;

procedure onRun;
begin
    // Will get a reference to the object this
    if not getreference(thiss, anObject) then
        error 'unable to get a reference to object';

    // Write object name to screen (this.name)
    writeln(anObject.name);
end.
```

12.9 Examples

12.9.1 Example 1 - Arrays

```
{ An example program how to handle arrays }
Program Example_array;
{Global variables }
var MyArray : array [1 .. 10] of integer;
var i : integer;
Procedure PrintArray(Size: integer);
var Counter : integer;
Begin
  for Counter:=1 to Size do
  begin
    writeln('Value ', Counter, '=', MyArray[Counter] );
  end;
End.
{ Main procedure }
Begin
  for i:=startindex(MyArray) to endindex(MyArray) do
  begin
    MyArray[i]:=11 - i;
  end;  { End for }

  { Print array with a procedure }
  PrintArray(length(MyArray));

  { Print array with writeln }
  writeln(MyArray);
End.
```

12.9.2 Example 2 - Interfaces and objects

```
{ An example program how to handle interfaces and agent objects }
Program Example_skill;

{ Abstract must be global }
abstract Interface1 : interface;

{ Main procedure }
Begin
  { Set global agent variable }
  agent.running:=true;

  {Execute skill on remote interface }
  Interface1.Start:=true;
  Interface1.RemoteSkill(); // Call remote skill

End.
```

12.9.3 Example 3 - Structures

```
{ An example program how to use structures }
Program Pose_Example;
{ Define a new struct for a Pose in 3D }
struct Pose3D;
Begin
    Var  x,  y,  z: real;  // Location
    Var rx, ry, rz: real;  // Rotation
End;
function Diff(a, b : Pose3D) : Pose3D;
begin
    Diff.x:= a.x-b.x;
    Diff.y:= a.y-b.y;
    Diff.z:= a.z-b.z;
end.
{ Declaration of Pose3D variables Pos1 and Pos2}
Var Pos1:=(x:=1.0, y:=2.0,z:=3.0, rx:=0.0, ry:=0.0, rz:=0.0): Pose3D; // Init. of x, y and z in structure.
Var Pos2: Pose3D;
{ Main procedure }
Begin
    Pos2.x:=3.0;
    Pos2.y:=2.0;
    Pos2.z:=1.0;
    WriteLn('Pos diff=', Diff(Pos1, Pos2));
End.
```

12.9.4 Example 4 - MessageBox User Information

```
{ An example program how to show HMI messages }
```

```
Program MessageBox_Example;

$include 'system:messagebox.inc'

{ Main procedure }
Var Result : Integer;
Begin
    Result:=MessageBox('Press a button', 'Title', DIALOG_SHOW_YES_NO_CANCEL);

    if Result=DIALOG_ANSWER_YES then
    begin
      WriteLn('You pressed Yes');
    end;
    else if Result=DIALOG_ANSWER_NO then
    begin
      WriteLn('You pressed No');
    end;
    else if Result=DIALOG_ANSWER_CANCEL then
    begin
      WriteLn('You pressed Cancel');
    end;
    else
    begin
      WriteLn('You closed the window');
    end;
End.
```

12.9.5 Example 5 - InputDialog

```
{ An example program how to get user input strings from HMI }
Program InputDialog_Example;

Var Result : String;
Var DefaultValue:='3' : String;
Var Iterator: Integer;

{ Main procedure }
Begin
    Result:=InputDialog('How many iterations?', DefaultValue);

    if not StringIsEmpty(Result) and StringIsInteger(Result) then
    begin
       Iterator:=StringToInt(Result);

       while Iterator>0 do
       begin
         WriteLn('Loop - ', Iterator);
         Iterator:=Iterator-1;
       end;
    end;
End.
```

12.9.6 Example 6 - OptionDialog

```
{ An example program how to create a HMI stop button for a calculation.
  The program will calculate PI by a known series. The series is handled by a for loop
  that will end if the user press the stop button or when the Counter reach MAX_COUNT.
  }
Program ExampleOptionDialog;
Var Selection:=-1, Counter:=1 : Integer;
Var Odd:=false : Boolean;
Var PI:=1.0 : Real;
Var Constant MAX_COUNT:=10000000 :Integer;
{ Main procedure }
Begin
    { Create an OptionDialog with one stop button associated with handle 1 }
    OptionDialog(1, Selection, 'Calculating PI', 'Selection', 'Stop');
    While Selection=-1 AND Counter<MAX_COUNT do
    Begin
        if Odd then
            PI := PI + 1.0 / ( 2.0 * inttoreal(Counter) + 1.0);
        else
            PI := PI - 1.0 / ( 2.0 * inttoreal(Counter) + 1.0);

        Counter:=Counter+1;
        Odd:=not Odd;
    End;

    { Check if the OptionDialog is still open, if open then close it }
    if OptionDialog(1, true) = true then
    Begin
      OptionDialog(1, false);
    End;
```

```
    WriteLn('PI is ', PI*4.0, ' after ', Counter , ' number of iterations.');
End.
```

12.9.7 Example 7 - JSON Get

```
Program Example_JSON;

Var JSONString:='
{
  "array": [1, 2, 3],
  "string": "Hello World",
  "boolean": true,
  "real": 1.23,
  "integer": 123,
  "object": {
             "a": "b",
             "c": "d"
             }
  }' : String;

Var a : String;
Var b : Real;
Var c : Integer;
Var d : Boolean;
Var e : array [1 .. 3] of String;
Var f : String;

Begin
    a:=getjelement(JSONString, 'string');
    WriteLn('a=', a);                  // Will print Hello World

    b:=stringtoreal(getjelement(JSONString, 'real'));
    writeln('b=', b);              // Will print 1.23

    c:=stringtoint(getjelement(JSONString, 'integer'));
    writeln('c=',c);               // Will print 123

    d:=stringtoboolean(getjelement(JSONString, 'boolean'));
    writeln('d=', d);              // Will print true

    e:=getjelement(JSONString, 'array');
    writeln('e=', e[2]);           // Will print 2

    f:=getjelement(JSONString, 'object');
    writeln('f=', f);              // Will print {"a":"b","c":"d"}

    writeln('f.a=', getjelement(f, 'a')); // Will print b
End.
```

12.9.8 Example 8 - JSON and Struct mapping

```
Program Example_JSON_Struct;
Var JSONString:='
{
  "a": "Hello World",
  "b": true,
  "c": 1.23,
  "d": 123,
  "e": [1, 2, 3]
  }' : String;
Struct JObj;
Begin
    Var  a : String;
    Var  b : Boolean;
    Var  c : Real;
    Var  d : Integer;
    Var e : array [1 .. 3] of Integer;
End;

Var Data : JObj;

Begin
  Data:=jtostruct(JSONString);    // Convert JSON to struct
  WriteLn(Data);       // Print struct as a JSON

  Data.c:=3.21;        // Change value in struct
  Data.d:=4;
  Data.b:=false;
  Data.e[1]:=3;

  JSONString:=structtoj(Data);    // Convert Struct back to JSON
  Writeln(JSONString); // Print the canged JSON
End.
```

12.9.9 Example 9 - JSON Set

```
Program Example_JSON;
Var JSONString:='
{
  "array": [1, 2, 3],
  "string": "Hello World",
  "boolean": true,
  "real": 1.23,
  "integer": 123,
  "object": {
             "a": "b",
             "c": "d"
             }
  }' : String;

Struct JObj;
Begin
    Var  a : String;
    Var  c : Real;
End;

Var a : Integer;
Var b : array [1 .. 3] of Integer;
Var c : JObj;

Begin
    { Get value for a from JSON object and add 1 }
    a:=StringToInt(GetJElement(JSONString, 'integer')) + 1;

    b[1]:=3;
    b[2]:=2;
    b[3]:=1;

    c.a:='HI!';
    c.c:=3.14;

    { Write new values to JSON object }
    JSONString:=SetJElement(JSONString, 'integer', a);
    JSONString:=SetJElement(JSONString, 'boolean', false);
    JSONString:=SetJElement(JSONString, 'object', c);
    JSONString:=SetJElement(JSONString, 'array', b);

    { Write JSON object to screen }
    WriteLn(JSONString);
End.
```

12.9.10 Example 10 - Sort

```
{ An example program }
Program Example;

{ Array size }
Var Constant Size:=5 : Integer;

{ Data structure type }
Struct tData;
Begin
    Var Number:=0 : Integer;
    Var Index:=0  : Integer;
End;

{ Global data array }
Var data : Array [1 .. Size] of tData;

{ Procedure to initialize the data with random numbers }
Procedure Init;
    Var Counter : Integer;
Begin
    For Counter:=StartIndex(data) to EndIndex(data) do
     Begin
       Data[Counter].Number:=random(100);
       Data[Counter].Index:=Counter;
     End;
End.

{ Procedure to write all data to screen }
Procedure Display(Message : String);
    Var Counter : Integer;
Begin
    writeln();
    writeln(Message);

    For Counter:=StartIndex(data) to EndIndex(data) do
     Begin
```

```
       Write('Item ', Counter, ' ');
       Write('Number = ', Data[Counter].Number, ' ');
       Writeln('Index = ', Data[Counter].Index);
     End;
End.

{ Sorting algorithm }
Procedure BubbleSort;
Var  Counter        : Integer;
Var  UnSorted:=true : Boolean;
Var  SwapTmp        : tData;
Begin

   { Iterate until sorted }
   While UnSorted=true do
   Begin
         { Iterate over the data array }
        UnSorted:=false;
        For Counter:=StartIndex(data)+1 to EndIndex(data) do
        Begin

            { Check if unsorted data }
            if data[Counter].Number < data[Counter-1].Number then
            Begin
                SwapTmp:=data[Counter];
                data[Counter]:=data[Counter-1];
                data[Counter-1]:=SwapTmp;
                UnSorted:=true;
            End;
        End;
    End;
End.

{ Main procedure }
Begin
    Init;
    Display('Random Data');
    BubbleSort;
    Display('Sorted Data');
End.
```

12.9.11 Example 11 - Foreach

```
{ An example program }
Program Example;
var anArray: array [ 1 .. 2, 1 .. 2] of integer;
Var aString:='hello world': string;
Struct Object;
Begin
    Var x, y : Real;
End;
var anObject:=(x:=1.0, y:=1.0):object;

procedure onRun;
begin
    { Will iterate over the array }
    foreach sub in anArray  do
    begin
        foreach ii in sub do
            ii:=-1;
    end;
    writeln(anArray);

    { Will iterate over the string.
     Note that si is read only when it comes to strings
    }
    foreach si in aString do
    begin
        write(si);
    end;
    writeln();

    { Iterate over an object note that all fields in the
    object must match oi to be able to use foreach }
    foreach oi in anObject do
    begin
        oi:=1.1;
    end;
    writeln(anObject);
end.
```

12.10 System include files

CMAS provides a set of include files that can be used in any structured text program. These include files can be accessed using the $include directive. To access system include files, the filename must begin with **system:**, e.g. *$include 'system:ontology.inc'*.

Name	Description
ontology.inc	The most common CMAS objects
file.inc	Constants for file access
messagebox.inc	Constants for messageboxes

12.10.1 ontology.inc

```
//
// Include file for the ontology objects in CMAS
//

struct Entity;
begin
    var Name :          String;
    var Id :            String;
    var InstanceId :    String;
    var Description :   String;
end;

struct ProcessPlan extends Entity;
begin
    var Cost :              Integer;
    var TargetType :        String;
end;

struct Agent extends Entity;
begin
    var PhysicalID :        String;
    var AgentSpecificType:  String;
    var AutoDeploy :        Boolean;
end;

struct DeploymentAgent extends Agent;
begin
    var DeploymentTime:     String;
    var NumberOfItems :     Integer;
    var VariableTrigger:    String;
end;

struct Part extends Agent;
begin
end;

struct Resource extends Agent;
begin
end;

struct Material extends Agent;
begin
end;

struct InterfaceBase extends Entity;
begin
    var Booked : Boolean;
    var BookedBy : String;
    var Busy : Boolean;
    var AttachedTo : String;
end;

struct InterfaceSpecialisation extends InterfaceBase;
begin
    var SpecialisationName :    String;
end;

struct Goal extends InterfaceBase;
begin
    var InterfaceName :     String;
    var Priority :          Integer;
    var ProcessPlanName :   String;
end;

struct Value extends Entity;
```

```
begin
    var UpperBound :    String;
    var LowerBound :    String;
    var Unit :          String;
end;

struct Variable extends Value;
begin
    var Source :        String;
    var Address :       String;
    var ReadOnly :      Boolean;
    var StoreLongTerm : Boolean;
end;

struct VariableInteger extends Variable;
begin
    var Value :         Integer;
end;

struct VariableString extends Variable;
begin
    var Value :         String;
end;

struct VariableBoolean extends Variable;
begin
    var Value :         Boolean;
end;

struct VariableReal extends Variable;
begin
    var Value :         Real;
end;

struct VariableObject extends Variable;
begin
    var Size :          Integer;
end;

struct VariableArray extends Variable;
begin
    var LowerRange :    Integer;
    var UpperRange: Integer;
end;

struct Hazard extends Entity;
begin
    var Risk :          Integer;
    var Frequency :     Integer;
    var TargetType :    String;
end;
```

12.10.2 file.inc

```
// Include file for file handling

struct typeMODE;
begin
    var constant WRITE_TEXT:=1 : integer;
    var constant READ_TEXT:=2 : integer;
end;

var MODE:typeMODE;
```

12.10.3 messagebox.inc

```
// Include file for messagebox

// Return value constants for messagebox
Var CONSTANT
    DIALOG_ANSWER_YES:=0,
    DIALOG_ANSWER_NO:=1,
    DIALOG_ANSWER_CANCEL:=2,
    DIALOG_ANSWER_CLOSED:=-1 :Integer;

// Option constants for type of messagebox to show
Var CONSTANT
    DIALOG_SHOW_YES_NO:=0,
    DIALOG_SHOW_YES_NO_CANCEL:=1,
    DIALOG_SHOW_OK_CANCEL:=2,
```

```
    DIALOG_ERROR_MESSAGE:=3,
    DIALOG_INFORMATION_MESSAGE:=4,
    DIALOG_WARNING_MESSAGE:=5,
    DIALOG_QUESTION_MESSAGE:=6,
    DIALOG_PLAIN_MESSAGE:=-1 :Integer;
```

12.10.4 queue.inc

```
// Include file to handle queues of booking

struct queueItem;
begin
    var agentreference : reference;
    var interfacereference : reference;
    var time : integer;
    var value : integer;
end;

var queue : array [1 .. 10] of queueItem;
```

Chapter 13

Variables

13.1 Overview

Variables are used to represent values within agents, interfaces, goals and other CMAS objects. A variable may represent a single value, an array, or a structured object. The actual data stored by a variable is handled by a value object (see Variable types). Different variable types extend the basic variable concept to support primitive values, arrays, or structured objects. Variables are used to represent values within agents, interfaces, goals, and other CMAS objects. A variable may represent a single value, an array, or a structured object. The underlying data of a variable is represented by a value object.

Note

Ontology vs Structured Text Variables The variables described in this section are **ontology variables**, created and managed through the **modeling tree**.
These are part of the CMAS ontological model and exist in the system independently of any runtime code.
Variables declared in **Structured Text** (local or global variables within procedures) are different and not covered in this section, although they behave similarly in terms of read/write operations.

The following simplified diagram illustrates the relationships between the main variable types in CMAS. For the complete ontology see Ontology.

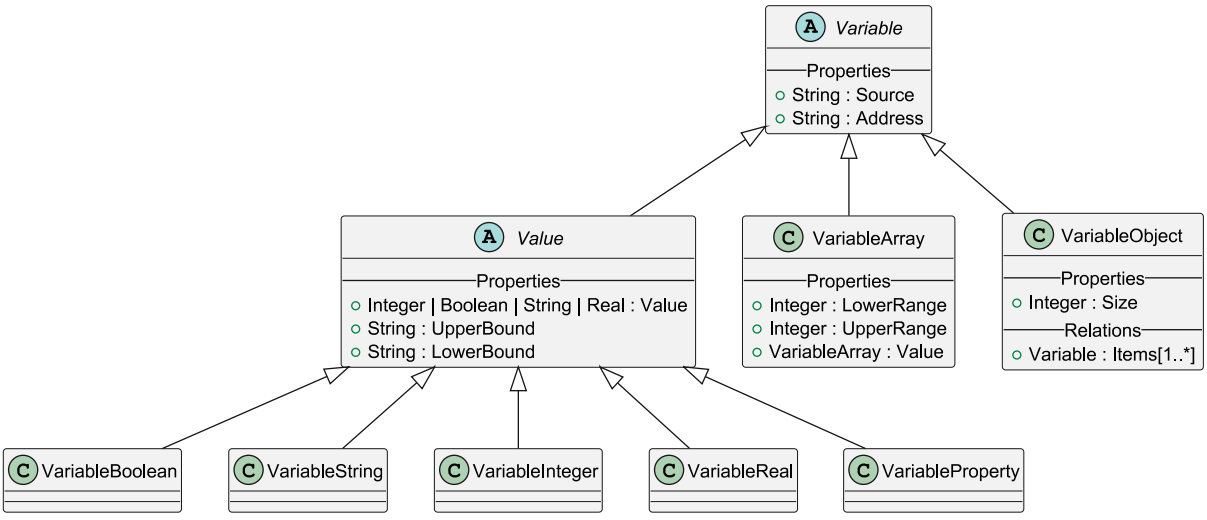


Figure CMAS variable ontology.

For additional details see:

- Variable adapters
- Variable bounds

13.2 Variable types

CMAS supports several variable types depending on the structure of the data. A variable may represent a **primitive value**, an **array**, or a **structured object**.
Primitive values can be of the following types:

- integer
- string
- real
- boolean

In addition to primitive values, CMAS also supports:

- array variables
- object variables

Array variables store an indexed collection of values, while object variables store a structured set of variables.

13.2.1 Example: Variable Value

The Agent object has a relation called Variables that can host variables of any type. Variables can be added to an agent in the modeling tree by using the Add new button, or by selecting the Variable commands in the Modeling tab of the ribbon. The following example demonstrates how a variable called counter can be used to count time during execution.

```
procedure onRun;
begin
  agent.counter := 0; // Initialize counter when the agent starts
end.

procedure Running;
begin
  // Increase the counter value once every second
  agent.counter := agent.counter + 1;
  sleep(1000);
end.
```

In this example, the variable counter is stored in the agent and updated continuously while the agent is running.

13.2.2 Example: Array Variable

The following example demonstrates how to use an array variable in an agent. It shows two common ways to access elements: by index and using a foreach loop. For an array variable, the **LowerRange** and **UpperRange** properties can be modified in the property editor during modeling to change the array size.

```
procedure init;
var i: integer;
begin
  // Access array elements by index
  for i := startindex(agent.a) to endindex(agent.a) do
  begin
    agent.a[i] := i;
  end;
end.

procedure onRun;
begin
  init();  // Initialize array values

  // Iterate over array elements using foreach
  foreach item in agent.a do
  begin
    writeln(item);
  end;
end.
```

In this example:

- `startindex(agent.a)` and `endindex(agent.a)` return the lower and upper bounds of the array.
- `agent.a[i]` accesses an individual element by its index.
- `foreach item in agent.a` iterates over all elements in the array, which is convenient for processing all values without explicit indexing.

13.2.3 Example: Object Variable

An object variable is used to group several related variables into a structured object.
Specifically, an **Object Variable** hosts other variables inside its **Items** relation.

It is possible to add variables of different types, including new **Object** or **Array** variables, to the **Items** relation.
This allows you to build complex nested structures within a single object variable.

```
procedure onRun;
begin
    agent.object.a:=1;
    agent.object.b:=true;
    agent.object.c[1]:=0;

    writeln(agent.object);
end.
```

Note

Using a variable adapter, a value can be connected to external data sources. A value can also have bounds which limit the allowed value range.

13.3 Functions

A value exposes optional functions that can be overridden to customize the behavior of value access.

Function	Description
onBeforeRead(value : <type>) : <type>	Called before the value is read.
onAfterRead(value : <type>) : <type>	Called after the value has been read.
onBeforeWrite(oldValue, newValue : <type>) : <type>	Called before the value is written.
onAfterWrite(oldValue, newValue : <type>) : <type>	Called after the value has been written.

These functions allow the user to intercept and modify the value or other properties before the value is returned.

13.3.1 onBeforeRead

The `onBeforeRead` function is called when a variable value is about to be read.
The purpose of `onBeforeRead` is to be able to modify properties related to the read operation before the value is retrieved or processed.
Typical use cases include:

- Modifying the address used by an adapter
- Adjusting value bounds or scaling parameters
- Providing a default value in case of communication errors

For example, an adapter may dynamically update the address that will be used to retrieve the value from an external system.

Note

`onBeforeRead` must accept a parameter of the same type as the variable declared in CMAS (integer, string, real or boolean).
The function must also return a value of the same type.

The returned value will be used as the initial value for the read operation.

```
function onBeforeRead(v : integer) : integer;
begin
  this.Address := '123';  // Set a valid address (overrides the configured address)
  onBeforeRead := -1;     // Provide a default value to catch communication errors
end.
```

13.3.2 onAfterRead

The `onAfterRead` function is called after the variable value has been read and before it is returned to the caller. The purpose of `onAfterRead` is to allow post-processing of the retrieved value. This makes it possible to validate, transform or filter the value before it is used by the system.
Typical use cases include:

- Converting units (for example raw sensor values)
- Applying scaling or calibration
- Clamping the value to a valid range
- Detecting invalid or unexpected values

Note

`onAfterRead` must accept a parameter of the same type as the variable declared in CMAS (integer, string, real or boolean).
The function must also return a value of the same type.

The returned value will be the final value delivered to the caller.

```
function onAfterRead(v : integer) : integer;
begin
  if v < 0 then
    onAfterRead := 0     // Clamp invalid values
  else
    onAfterRead := v;
end.
```

13.3.3 onBeforeWrite

The `onBeforeWrite` function is called when a value is about to be written to a variable.
The purpose of `onBeforeWrite` is to allow preprocessing or validation of the new value before it is stored or propagated.
This can be used to enforce constraints, modify metadata, or adjust the value dynamically.
Typical use cases include:

- Validating the value against bounds or business rules
- Transforming or scaling the value before storage
- Updating related properties such as timestamps or addresses
- Rejecting invalid writes by modifying the value

Note

`onBeforeWrite` must accept two parameters of the same type as the variable declared in CMAS (integer, string, real, or boolean).
The function must also return a value of the same type.
The returned value will be the one actually written to the variable.

```
function onBeforeWrite(oldValue, newValue : integer) : integer;
begin
  if newValue < 0 then
    onBeforeWrite := 0   // Clamp to minimum value
  else
    onBeforeWrite := newValue;

end.
```

13.3.4 onAfterWrite

The `onAfterWrite` function is called after a value has been written to a variable.
The purpose of `onAfterWrite` is to allow post-processing or notifications after the write operation has completed.
This can be used to update dependent variables, trigger events, or perform logging.
Typical use cases include:

- Logging the written value for audit or debug purposes
- Updating dependent or linked variables
- Triggering external notifications (e.g., adapters, agents)
- Validating that the write was successful and correcting errors

Note

`onAfterWrite` must accept two parameters of the same type as the variable declared in CMAS (integer, string, real, or boolean).
The function must also return a value of the same type.
The returned value will replace the stored value if necessary (e.g., to correct errors or normalize data).

```
function onAfterWrite(oldValue, newValue  : integer) : integer;
begin
    agent.Log:=newValue;  // Update logged value
end.
```

13.4 Variable adapters

13.4.1 Introduction

Variables can be used to handle values inside an agent. With a variable adapter it is possible to access external data. Example of external data might be a sensor value from a transducer. An Adapter, is a low level implementation, to be able to handle different communication protocols.
Supported adapters are:

- OPC UA
- REST
- FILE
- MODBUS
- SOCKET
- WEBSOCKET
- ZEROMQ
- SQL

Any variable can be connected to an adapter by specifying the **Source** and the **Address** property. **Source** is the path to the device to connect to and has a format of `[ADAPTER TYPE]:{optional parameters}`

Note

An adapter is shared among many variables if they have the same **Source**.
Address is the address to the specific data to read from a **Source**. An address shall not be reused or shared between variables.

If the first parameter before adapter type is **DEBUG:**, e.g. debug:opc.tcp:localhost, it is possible to get more debug information. This is useful if a connection is not working correctly.

13.4.2 OPC UA

For an OPC UA adapter the specific format for **Source** is:
`opc.tcp:HOST:PORT:NAME`

- opc.tcp is static for OPC UA
- HOST is the name or TCP/IP address to the server
- PORT is the port number to the server
- NAME is the server name instance

OPC UA Source example:

```
opc.tcp:localhost:53530:OPCUA/SimulationServer
```

The **Address** indicates which variable to read. If **Address** is empty or set to `AUTO` the name of the variable will be used to find the variable inside the `Source`. Otherwise, **Address** must be a valid node name.
OPC UA address example:

```
ns=3;i=1006
```

13.4.3 REST

For REST adapters, the source string must follow this format: REST[.type]:HTTP:HOST:PORT:[LIMIT]:[AUTH]↩ :[USER]:[PASW] eg, REST:http:localhost:80:-1:DIGEST:Default

- REST static prefix indicating the REST protocol.
- TYPE adapter mode is optional. Default is NON can be set to .ASSOCIATE
- HTTP transport protocol: http or https.
- HOST is the host name or TCP/IP address to the server
- PORT is the port number to the server
- LIMIT minimum time in milliseconds between consecutive read operations, default value is -1 (ignore).
- AUTH authorization method, BASIC, DIGEST or NONE (default value NONE)
- USER user name
- PASW password

By default, a REST adapter uses static references for the Source and Address properties. This means that read operations do not depend on the current value of the variable.
When using the REST.ASSOCIATE type, the adapter associates the current value of the variable with the REST request before performing a read operation. This is useful for REST APIs that expect a request body (for example, a JSON payload in POST requests).

REST type	HTTP	Body
ASSOCIATE	POST	JSON
NONE	GET	None

Use the LIMIT parameter to avoid overloading REST servers when variables are read frequently inside CMAS. The default value of LIMIT is -1, which means that CMAS will read from the REST server as fast as possible. If LIMIT is set to a value greater than 0 (for example, 10), CMAS will only perform a read operation if the last value was read more than 10 ms ago. If a new read is not performed due to the LIMIT constraint, the adapter returns the most recently cached value.
Example 1 - source: Set the source to connect to an ABB robot

```
REST:http:localhost:80:DIGEST:Default User:robotics
```

The **Address** indicates which variable to read and should normally start with a slash /
Example 1 - address:

```
/rw/rapid/symbol/data/RAPID/T_ROB1/MainModule/
```

Example 2: In this example, CMAS connects to an Ollama server via REST to access a large language model. A string variable, named llama, exists in the Agent Variable relation with the following source and address settings:

```
Source = REST.ASSOCIATED:HTTP:localhost:11434
Address = /api/generate
```

To use the ASSOCIATE behavior, we need to set the variable's value. An onRun procedure for the agent can look like this:

```
{ Rest associated variable example with ollama REST server }
Program Example;

// Create a struct type for ollama
struct REQUEST;
begin
    var model  : string;
    var prompt : string;
    var stream : boolean;
end;

// Initialize a variable by the REQUEST type
var aRequest:=(model:='llama3.2', prompt:='', stream:=false) : REQUEST;

procedure onRun;
var
  result, response: string;
begin
  // Set prompt value
  aRequest.prompt:='Hello World!';

  // Associate a JSON object to the llama variable
  // This associated value will be used when reading from the REST adapter later on
  agent.llama := structtoj(aRequest);

  // Read the value from the REST server by associating the value
  // This value is a JSON object
  result := agent.llama;

  // Get the response value from the JSON object
  response := GetJElement(result, 'response');

  if len(response) = 0 then
    writeln('Response is empty: ' + result);
  else
    writeln(response);
end.
```

13.4.4 FILE

For a FILE adapter the specific format for **Source** is:
`file:PATH`

- file is static
- PATH is a directory

Example:

```
file:C:\Users\Foo\Desktop\communication\
```

The **address** indicates which file to open without any path information given
Example:

```
forwardA.qx
```

13.4.5 MODBUS

For a MODBUS adapter the specific format for **Source** is:
`modbus.<TYPE>:HOST|SERIALPARAMETERS:[PORT]`
<TYPE> can be:

- tcp
- udp
- serial

HOST is the name or TCP/IP address to the server (default 127.0.0.1)
PORT is the port number to the server (default 502)
SERIALPARAMETERS is the configuration parameters for a modbus serial connection type. It has the following format: <SERIALPORT>,<BAUDRATE>,<DATABITS>,<STOPBITS>,<PARITY>,<FLOWCONTROLIN>,<FLOWCONTROLOU

- <SERIALPORT> is the name of the serial port.
- <BAUDRATE> (default 9600) is the baud rate for the connection.
- <DATABITS> (default 8) number of data bits.
- <STOPBITS> (default 1) number of stop bits.
- <PARITY> (default 0) 0 (non) / 1 (odd) / 2 (even) / 3 (mark) / 4 (space)
- <FLOWCONTROLIN> (default) 0 (disabled)
- <FLOWCONTROLOUT> (default) 0 (disabled)

MODBUS Source example for a TCP connection:

```
modbus.tcp:127.0.0.1:502
```

MODBUS Source example for a serial port:

```
modbus.serial:com1,9600,8,1
```

The **Address** indicates which variable to read. <unit>:<address>:<type>
<unit> is optional (default 1) and is the device unit number.
<address> is the address number to read or write.
<type> is optional (default holdingregister) and state which type of register to read <type> can be:

- holdingregistersigned or 5
- holdingregister or 4
- inputregister or 3
- discreteinput or 2
- coil or 1

MODBUS address example to read a holding register from unit 1 at address 4:

```
1:4:holdingregister
```

By the use of default values the same register as above can be simplified to:

```
4
```

13.4.6 SOCKET

For a SOCKET adapter the specific format for **Source** is:
`socket.<PROTOCOL>.<TYPE>.<RW>:SERVER:[PORT]:[TIMEOUT]`
<PROTOCOL> can be:

- tcp
- udp

<TYPE> can be:

- ascii (default value)
- binary

<RW> can be: RW for read and write (default value) R for read only W for write only

SERVER is the name or address to the server (default 127.0.0.1)
PORT is the port number to the server (default 80)

TIMEOUT is the time out in ms for reading value from the socket (default 2000)
\
SOCKET Source example for a TCP connection with read and write access:

```
socket.tcp.ascii:127.0.0.1:80
```

The **Address** indicates how to read and write values and is a formating string. Note, the address field is only used for ASCII based sockets.

Format Specifier	Data Type	Output
a	floating point	Hex output of floating point number
b	Any type	True or False
c	character	Unicode character
d	integer	Decimal Integer
e	floating point	a decimal number in scientific notation
f	floating point	decimal number
g	floating point	decimal number, possibly in scientific notation depending on the precision and value
o	integer	Octal number
s	Any type	String value
t	Date/Time	t is the prefix for Date/Time conversions.
x	integer	Hex string

The format string receives two values: Name and Value, derived from the variable object.
Example 1: Integer Variable For an integer variable A, if the address is:

```
set %1$s = %2$i
```

And the variable A is set to 1, the following will be sent:

```
set A = 1
```

A read operation will generate a request, provided the address is not empty. The request is assembled in the same way as for a write operation, and it is sent before the read begins. Note, The read operation will always wait until a response string is received or a timeout occurs. Therefore, a read operation may cause the application to freeze if no response is received.
Example 2:

```
get %2$s
```

13.4.7 WEBSOCKET

Web Socket - not implemented

13.4.8 ZeroMQ

For adapter type of ZeroMQ the specific format is: ZMQ.type:TCP://host:port eg, ZMQ.req:tcp://∗:5555
Type can one of: DEALER, PAIR, PUB, PULL, PUSH, REP, REQ, ROUTER, STREAM, SUB, XPUB, XSUB

13.4.9 SQL

The SQL adapter allows CMAS to read and write values from a JDBC-compliant database. It supports static queries and parameterized queries.
SQL:JDBC:HOST:PORT:DATABASE:[LIMIT]:[USER]:[PASSWORD]

- **SQL** Static prefix indicating the SQL adapter.
- **JDBC** JDBC protocol driver. Supported drivers are, SQLite: org.sqlite.JDBC, MySQL: com.mysql.cj.jdbc.↩ Driver, MariaDB: org.mariadb.jdbc.Driver, PostgreSQL: org.postgresql.Driver.
- **HOST** Database host name or IP address. For SQLite on window this is the drive that host the database, e.g., C:.
- **PORT** TCP port number of the database. For SQLite leave it blank.
- **DATABASE** Name of the target database. For SQLite it is the path to the database.
- **LIMIT** Minimum time (in milliseconds) between consecutive read operations. Default -1 means read as fast as requested.If set to a value >0, the adapter will return the cached last value if the last read is more recent than LIMIT milliseconds.
- **USER** Database user name.
- **PASSWORD** Database password.

Static vs Parameterized Queries

- Static queries are executed exactly as provided in the Address property.
- Parameterized queries contain one placeholder (?). The value of the variable is automatically bound to the placeholder.

LIMIT Controls how frequently the adapter actually queries the database. If the last read occurred less than LIMIT milliseconds ago, the adapter returns the cached value instead of executing a new query. Cached values are returned when a read is skipped due to LIMIT, or when a database error occurs.
Writes are executed immediately. LIMIT does not apply to writes.
Example 1 - static **read**: `Source = SQL:jdbc:org.mariadb.jdbc.Driver:localhost:3306↩ :production:-1:robot:secret Address = SELECT temperature FROM sensors WHERE id = 42`
Example 2 - parameterized **read** (database located at C:\SQLite\TestSQL.db): `Source = SQL:org.↩ sqlite.JDBC:c::/SQLite/TestSQL.db Address = SELECT temperature FROM sensors WHERE id = ?`
Example 3 - parameterized **write** (database located at C:\SQLite\TestSQL.db): `Source = SQL:org.↩ sqlite.JDBC:c::/SQLite/TestSQL.db Address = UPDATE sensors SET temperature = ?  WHERE id = 1`

Note

It is not possible to use the same CMAS variable for both reading and writing simultaneously, because the SQL address (query) format for reads and writes is different. If needed, the variable's Address can be changed at runtime from Structured Text to switch between read and write queries. Best practice for this case is to create two separate variables in CMAS on for read and one for write.

13.5 Variable bounds

Variables can be used to handle values inside an agent. With variable bounds it is possible to limit the values of a variable of type integer, real and string.
Any variable in the modelling tree can have a range expressed as upper and lower bounds. This behaviour is useful when the variable is part of an interface since the range is part of the negotiation process. If two agents negotiate a variable value must match the remote variable range.

13.5.1 Integer and Real

If upper bound is set to 5 and lower bound to 1 the value can only be within the range 1-5, i.e. 1<= x <=5, and the variable will only accept a value in that range during negotiation.

Note

At creation of a variable, the lower and upper bounds are empty and there is no limitation on the variable.

It is possible to write more complex expressions for upper and lower bound.
The basic syntax for a range expression:

Example	Name	Description
1	fixed value	only 1 is accepted

Example	Name	Description
<100	open range lt	all values lower than 100 are accepted
>50	open range gt	all values greater than 50 are accepted
!2	not	all values except 2 is accepted
[>100 <200]	AND	the set of all the elements inside [] must be true
1 | 2 | 3 | >10	OR	one of the element in the set separate by | must be true

Example 1:

```
[<100 >50 !70]      Will give a range less than 100 AND greater than 50 AND NOT the value 70)
```

Example 2:

```
[<10 >5] | [<20 >15]  A dual range example, value must be between 5 AND 10 OR 15 AND 20
```

13.5.2 String

An string can only handle the or | operator when it comes to range expressions.
Example:

```
start | stop  | pause      Will only accept strings with a value of 'start', 'stop' or 'pause'
```

13.5.3 Boolean

Boolean does not implement complex expressions, however it is possible to limit the value by setting upper or lower bound to true or false.

Chapter 14

Hierarchical Index

14.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- cmas.ontology.interfaces.AbstractInterface.AbstractType . . . 195
- cmas.ontology.agentcommunication.ACL.ACLMessage . . . 195
- cmas.ontology.action.Action . . . 196
- cmas.ontology.action.ActionInterface . . . 197
  - cmas.ontology.agents.AgentAction . . . 206
    - cmas.ontology.agents.Component . . . 224
      - cmas.ontology.agents.Material . . . 327
      - cmas.ontology.agents.Part . . . 352
    - cmas.ontology.agents.DeploymentAgent . . . 242
    - cmas.ontology.agents.Resource . . . 412
  - cmas.ontology.processplan.ProcessPlanJava . . . 378
  - cmas.ontology.processplan.ProcessPlanStructuredText . . . 387
- cmas.ontology.action.ActionJava . . . 197
- cmas.ontology.adapter.Adapter . . . 198
  - cmas.ontology.adapter.FILE.FileClient . . . 260
  - cmas.ontology.adapter.Modbus.ModbusClient . . . 336
  - cmas.ontology.adapter.OPCUA.OPCUAClientBase . . . 350
    - cmas.ontology.adapter.OPCUA.OPCUAClient . . . 346
  - cmas.ontology.adapter.REST.RESTClient . . . 422
  - cmas.ontology.adapter.SQL.SQLClient . . . 445
  - cmas.ontology.adapter.ZeroMQ.ZMQClient . . . 557
  - cmas.ontology.adapter.socket.SocketTCPClient . . . 440
  - cmas.ontology.adapter.socket.SocketUDPClient . . . 442
  - cmas.ontology.adapter.websocket.WebSocketClient . . . 539
- cmas.ontology.adapter.AdapterFactory . . . 199
- cmas.ontology.adapter.Modbus.ModbusClient.ADDRESS_TYPE . . . 199
- cmas.ontology.agents.Agent.AgentBaseTypes . . . 214

AgentRunnable
- cmas.ontology.agents.AgentAction . . . 206
- cmas.ontology.interfaces.InterfaceBase . . . 295
  - cmas.ontology.interfaces.Goal . . . 262
  - cmas.ontology.interfaces.Interface . . . 278
    - cmas.ontology.interfaces.InterfaceSpecialisation . . . 311
      - cmas.ontology.interfaces.InterfaceAssembly . . . 287
      - cmas.ontology.interfaces.InterfaceLocation . . . 303
      - cmas.ontology.interfaces.InterfaceTransport . . . 319
- cmas.ontology.agents.AgentAction.AgentState . . . 215
- cmas.ontology.interfaces.Attachment . . . 215
- cmas.ontology.batch.BatchItem . . . 223

cmas.ontology.agentcommunication.CommunicativeActs . . . 224
cmas.ontology.agents.AgentAction . . . 206
cmas.ontology.interfaces.AbstractInterface . . . 193
cmas.ontology.interfaces.InterfaceBase . . . 295
cmas.ontology.processplan.ProcessPlan . . . 360
cmas.ontology.processplan.ProcessPlanGoals . . . 368
cmas.ontology.processplan.ProcessPlanJava . . . 378
cmas.ontology.processplan.ProcessPlanStructuredText . . . 387
cmas.ontology.agentcommunication.jsoncontent.Content . . . 233
cmas.ontology.agentcommunication.jsoncontent.template.ContentBusy . . . 234
cmas.ontology.agentcommunication.jsoncontent.template.ContentDone . . . 235
cmas.ontology.agentcommunication.jsoncontent.template.ContentError . . . 236
cmas.ontology.agentcommunication.jsoncontent.template.ContentInformation . . . 237
cmas.ontology.agentcommunication.jsoncontent.template.ContentNegotiate . . . 238
cmas.ontology.agentcommunication.jsoncontent.template.ContentProgress . . . 239
cmas.ontology.agentcommunication.jsoncontent.template.ContentReplan . . . 240
cmas.ontology.agentcommunication.jsoncontent.template.ContentResult . . . 241
cmas.ontology.agentcommunication.jsoncontent.ContentParameter . . . 239
cmas.ontology.agents.DeploymentAgent.DeploymentCriteria . . . 250
cmas.ontology.base.Entity.EntityType . . . 258
Event
cmas.ontology.agentcommunication.ACL.MessageHandler . . . 335
cmas.ontology.base.Container . . . 232
cmas.ontology.base.Visualisation . . . 538
cmas.ontology.base.Entity . . . 251
cmas.ontology.Hazard . . . 270
cmas.ontology.Note . . . 338
cmas.ontology.agents.Agent . . . 200
cmas.ontology.agents.AgentAction . . . 206
cmas.ontology.base.Reference . . . 398
cmas.ontology.batch.Batch . . . 216
cmas.ontology.interfaces.InterfaceBase . . . 295
cmas.ontology.model.ModelAspect . . . ??
cmas.ontology.model.ModelSettings . . . ??
cmas.ontology.model.ModelState . . . ??
cmas.ontology.processplan.ProcessPlan . . . 360
cmas.ontology.processplan.SkillCall . . . 433
cmas.ontology.variables.Variable . . . 459
cmas.ontology.variables.Value . . . 450
cmas.ontology.variables.VariableBoolean . . . 475
cmas.ontology.variables.VariableInteger . . . 484
cmas.ontology.variables.VariableProperty . . . 502
cmas.ontology.variables.VariableReal . . . 511
cmas.ontology.variables.VariableString . . . 529
cmas.ontology.variables.VariableArray . . . 466
cmas.ontology.variables.VariableObject . . . 493
cmas.ontology.variables.VariableReference . . . 520
cmas.ontology.workflow.WorkFlow . . . 542
cmas.ontology.workflow.Requirement . . . 406
cmas.ontology.workflow.WorkFlowLog . . . 549
cmas.ontology.base.Property< T > . . . 396
EventListener
cmas.ontology.base.Relations.RelationObject . . . 404
events.Event.EventListener
cmas.ontology.base.Reference . . . 398
ExceptionBase
cmas.ontology.exceptions.ExceptionConstant . . . 258

cmas.ontology.exceptions.ExceptionType . . . 259
cmas.ontology.exceptions.ExceptionVariable . . . 259
cmas.ontology.interfaces.InterfaceBase.InterfaceState . . . 318
cmas.ontology.variables.Iterator . . . 327
cmas.ontology.adapter.Modbus.ModbusClient.MODBUS_TYPE . . . 335
cmas.ontology.base.Ontology . . . 345
cmas.ontology.base.Ontology.OntologyObject . . . 345
cmas.ontology.base.Ontology.OntologyRelation . . . 346
cmas.ontology.adapter.OPCUA.OPCUANode . . . 352
cmas.ontology.agentcommunication.ACL.ACLMessage.Performative . . . 360
cmas.ontology.processplan.ProcessPlanInterface . . . 377
cmas.ontology.processplan.ProcessPlan . . . 360
cmas.ontology.processplan.ProcessPlan.ProcessPlanReturnValue . . . 386
cmas.ontology.processplan.ProcessPlan.ProcessPlanType . . . 395
cmas.ontology.base.Properties . . . 395
cmas.ontology.variables.ValueRange.Range . . . 397
cmas.ontology.base.Relations . . . 405
cmas.ontology.adapter.REST.RESTClient.REST_TYPE . . . 422
cmas.ontology.adapter.socket.SocketTCPClient.SERVER_TYPE . . . 424
cmas.ontology.adapter.socket.SocketUDPClient.SERVER_TYPE . . . 424
cmas.ontology.base.Property< T >.SpecificPropertyType . . . 444
cmas.ontology.workflow.WorkFlow.SpecificTypes . . . 444
cmas.ontology.variables.Variable.TypeOfVariable . . . 447
cmas.ontology.Types . . . 447
cmas.ontology.base.Entity.UID . . . 450
URLClassLoader
cmas.ontology.action.IndependentClassLoader . . . 277
cmas.ontology.variables.ValueRange . . . 458
cmas.ontology.variables.Value.ValueTypes . . . 459
cmas.ontology.interfaces.InterfaceBase.VariableMatchingType . . . 493
cmas.ontology.workflow.WorkFlow.WorkFlowTypes . . . 556

Chapter 15

Class Index

15.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

cmas.ontology.interfaces.AbstractInterface . . . 193
cmas.ontology.interfaces.AbstractInterface.AbstractType . . . 195
cmas.ontology.agentcommunication.ACL.ACLMessage . . . 195
cmas.ontology.action.Action . . . 196
cmas.ontology.action.ActionInterface . . . 197
cmas.ontology.action.ActionJava . . . 197
cmas.ontology.adapter.Adapter . . . 198
cmas.ontology.adapter.AdapterFactory . . . 199
cmas.ontology.adapter.Modbus.ModbusClient.ADDRESS_TYPE . . . 199
cmas.ontology.agents.Agent . . . 200
cmas.ontology.agents.AgentAction . . . 206
cmas.ontology.agents.Agent.AgentBaseTypes . . . 214
cmas.ontology.agents.AgentAction.AgentState . . . 215
cmas.ontology.interfaces.Attachment . . . 215
cmas.ontology.batch.Batch . . . 216
cmas.ontology.batch.BatchItem . . . 223
cmas.ontology.agentcommunication.CommunicativeActs . . . 224
cmas.ontology.agents.Component . . . 224
cmas.ontology.base.Container . . . 232
cmas.ontology.agentcommunication.jsoncontent.Content . . . 233
cmas.ontology.agentcommunication.jsoncontent.template.ContentBusy . . . 234
cmas.ontology.agentcommunication.jsoncontent.template.ContentDone . . . 235
cmas.ontology.agentcommunication.jsoncontent.template.ContentError . . . 236
cmas.ontology.agentcommunication.jsoncontent.template.ContentInformation . . . 237
cmas.ontology.agentcommunication.jsoncontent.template.ContentNegotiate . . . 238
cmas.ontology.agentcommunication.jsoncontent.ContentParameter . . . 239
cmas.ontology.agentcommunication.jsoncontent.template.ContentProgress . . . 239
cmas.ontology.agentcommunication.jsoncontent.template.ContentReplan . . . 240
cmas.ontology.agentcommunication.jsoncontent.template.ContentResult . . . 241
cmas.ontology.agents.DeploymentAgent . . . 242
cmas.ontology.agents.DeploymentAgent.DeploymentCriteria . . . 250
cmas.ontology.base.Entity . . . 251
cmas.ontology.base.Entity.EntityType . . . 258
cmas.ontology.exceptions.ExceptionConstant . . . 258
cmas.ontology.exceptions.ExceptionType . . . 259
cmas.ontology.exceptions.ExceptionVariable . . . 259
cmas.ontology.adapter.FILE.FileClient . . . 260
cmas.ontology.interfaces.Goal . . . 262
cmas.ontology.Hazard . . . 270
cmas.ontology.action.IndependentClassLoader . . . 277

cmas.ontology.interfaces.Interface . . . 278
cmas.ontology.interfaces.InterfaceAssembly . . . 287
cmas.ontology.interfaces.InterfaceBase . . . 295
cmas.ontology.interfaces.InterfaceLocation . . . 303
cmas.ontology.interfaces.InterfaceSpecialisation . . . 311
cmas.ontology.interfaces.InterfaceBase.InterfaceState . . . 318
cmas.ontology.interfaces.InterfaceTransport . . . 319
cmas.ontology.variables.Iterator . . . 327
cmas.ontology.agents.Material . . . 327
cmas.ontology.agentcommunication.ACL.MessageHandler . . . 335
cmas.ontology.adapter.Modbus.ModbusClient.MODBUS_TYPE . . . 335
cmas.ontology.adapter.Modbus.ModbusClient . . . 336
cmas.ontology.model.ModelAspect . . . ??
cmas.ontology.model.ModelSettings . . . ??
cmas.ontology.model.ModelState . . . ??
cmas.ontology.Note . . . 338
cmas.ontology.base.Ontology . . . 345
cmas.ontology.base.Ontology.OntologyObject . . . 345
cmas.ontology.base.Ontology.OntologyRelation . . . 346
cmas.ontology.adapter.OPCUA.OPCUAClient . . . 346
cmas.ontology.adapter.OPCUA.OPCUAClientBase . . . 350
cmas.ontology.adapter.OPCUA.OPCUANode . . . 352
cmas.ontology.agents.Part . . . 352
cmas.ontology.agentcommunication.ACL.ACLMessage.Performative . . . 360
cmas.ontology.processplan.ProcessPlan . . . 360
cmas.ontology.processplan.ProcessPlanGoals . . . 368
cmas.ontology.processplan.ProcessPlanInterface . . . 377
cmas.ontology.processplan.ProcessPlanJava . . . 378
cmas.ontology.processplan.ProcessPlan.ProcessPlanReturnValue . . . 386
cmas.ontology.processplan.ProcessPlanStructuredText . . . 387
cmas.ontology.processplan.ProcessPlan.ProcessPlanType . . . 395
cmas.ontology.base.Properties . . . 395
cmas.ontology.base.Property< T > . . . 396
cmas.ontology.variables.ValueRange.Range . . . 397
cmas.ontology.base.Reference . . . 398
cmas.ontology.base.Relations.RelationObject . . . 404
cmas.ontology.base.Relations . . . 405
cmas.ontology.workflow.Requirement . . . 406
cmas.ontology.agents.Resource . . . 412
cmas.ontology.adapter.REST.RESTClient.REST_TYPE . . . 422
cmas.ontology.adapter.REST.RESTClient . . . 422
cmas.ontology.adapter.socket.SocketTCPClient.SERVER_TYPE . . . 424
cmas.ontology.adapter.socket.SocketUDPClient.SERVER_TYPE . . . 424
cmas.ontology.processplan.SkillCall . . . 433
cmas.ontology.adapter.socket.SocketTCPClient . . . 440
cmas.ontology.adapter.socket.SocketUDPClient . . . 442
cmas.ontology.base.Property< T >.SpecificPropertyType . . . 444
cmas.ontology.workflow.WorkFlow.SpecificTypes . . . 444
cmas.ontology.adapter.SQL.SQLClient . . . 445
cmas.ontology.variables.Variable.TypeOfVariable . . . 447
cmas.ontology.Types . . . 447
cmas.ontology.base.Entity.UID . . . 450
cmas.ontology.variables.Value . . . 450
cmas.ontology.variables.ValueRange . . . 458
cmas.ontology.variables.Value.ValueTypes . . . 459
cmas.ontology.variables.Variable . . . 459
cmas.ontology.variables.VariableArray . . . 466
cmas.ontology.variables.VariableBoolean . . . 475

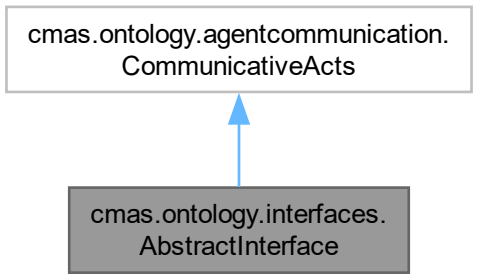
cmas.ontology.variables.VariableInteger 484
cmas.ontology.interfaces.InterfaceBase.VariableMatchingType 493
cmas.ontology.variables.VariableObject 493
cmas.ontology.variables.VariableProperty 502
cmas.ontology.variables.VariableReal 511
cmas.ontology.variables.VariableReference 520
cmas.ontology.variables.VariableString 529
cmas.ontology.base.Visualisation 538
cmas.ontology.adapter.websocket.WebSocketClient 539
cmas.ontology.workflow.WorkFlow 542
cmas.ontology.workflow.WorkFlowLog 549
cmas.ontology.workflow.WorkFlow.WorkFlowTypes 556
cmas.ontology.adapter.ZeroMQ.ZMQClient 557

Chapter 16

Class Documentation

16.1 cmas.ontology.interfaces.AbstractInterface Class Reference

Inheritance diagram for cmas.ontology.interfaces.AbstractInterface:



Classes

- enum AbstractType

Public Member Functions

- **AbstractInterface** (String AbstractName)
- **AbstractInterface** (String AbstractName, String Interface, AbstractType type)
- **AbstractInterface** (String AbstractName, VariableObject Object)
- ArrayList< String > **getAllSkills** ()
- String **toString** ()
- void **addSkill** (String Name)
- void **addSkills** (ArrayList< String > Names)
- String **getName** ()
- void **setName** (String newName)
- void **addVariable** (String Name)
- void **addVariables** (ArrayList< String > variables)
- void **setRemoteInterface** (Pair< Agent, InterfaceBase > bestInterface)
- Pair< Agent, InterfaceBase > **getRemoteInterface** ()
- Pair< Agent, InterfaceBase > **getLocalInterface** ()
- void **setLocalInterface** (Pair< Agent, InterfaceBase > localInterface)

- int **getCost** ()
- boolean **isStatic** ()
- boolean **isDefined** ()
- void **setIsDefined** ()
- boolean **isLocal** ()
- void **setReference** (AbstractInterface refInterface)
- InterfaceBase **getDefinedInterface** ()
- void **setDefinedInterface** (InterfaceBase localInterface)
- ProcessPlanReturnValue **agree** (boolean book) throws ExceptionRuntime, ExceptionFailedToExecute↩ Statement
- ProcessPlanReturnValue **negotiate** (boolean book) throws ExceptionRuntime, ExceptionFailedToExecute↩ Statement
- ArrayList< Requirement > **getRequirement** ()
- void **unbookOverlapInterfaces** ()
- HashSet< String > **getInterfaceTargetList** (InterfaceBase remoteInterface, ArrayList< String > skillList, boolean CheckOnSkillsOnly)
- synchronized ProcessPlanReturnValue **skillCall** (String functionName, VariableInteger result) throws InterruptedException, ExceptionRuntime, ExceptionNullValue, ExceptionType, ExceptionConstant
- synchronized ProcessPlanReturnValue **skillCall** (String functionName) throws InterruptedException, ExceptionRuntime
- synchronized ProcessPlanReturnValue **actionCall** (String functionName) throws InterruptedException, ExceptionRuntime
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- void **setReplyToInterface** (Agent agent, InterfaceBase replyInterface)
- boolean **getThrowOnError** ()
- void **setThrowOnError** (boolean throwOnError)
- AgentAction **getInitAgent** ()
- void **setInitAgent** (AgentAction initAgent)
- InterfaceBase **getInitGoal** ()
- void **setInitGoal** (InterfaceBase goal)
- boolean **isRunning** ()
- boolean **isNegotiated** ()
- void **setNegotiated** (boolean value)
- void **copyVariable** (Block globalBlock)
- boolean **getIgnoreBusy** ()
- void **setIgnoreBusy** (boolean ignoreBusy)
- boolean **getIgnoreBooked** ()
- void **setIgnoreBooked** (boolean ignoreBooked)
- ArrayList< Pair< Agent, InterfaceBase > > **getCompatibleInterfaceList** (DataTree root, Agent agent)
- MessageHandler getMessageHandler ()
- ImageView **getEntityIcon** ()
- ProcessPlanReturnValue **fulfilLocalRequirements** (Requirement requirement) throws ExceptionFailedTo↩ ExecuteStatement, ExceptionRuntime, InterruptedException
- boolean **isLocalSearch** ()
- void **setLocalSearch** (boolean localSearch)
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

16.1.1 Member Function Documentation

16.1.1.1 getCommunicative()

`Entity cmas.ontology.interfaces.AbstractInterface.getCommunicative ()`

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.1.1.2 getMessageHandler()

```
MessageHandler cmas.ontology.interfaces.AbstractInterface.getMessageHandler ()
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.1.1.3 postMessage()

```
void cmas.ontology.interfaces.AbstractInterface.postMessage (
            ACLMessage message)
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.
The documentation for this class was generated from the following file:

- AbstractInterface.java

16.2 cmas.ontology.interfaces.AbstractInterface.AbstractType Enum Reference

Public Attributes

- **DYNAMIC**
- **STATIC**
- **LOCAL**
- **DEFINED**

The documentation for this enum was generated from the following file:

- AbstractInterface.java

16.3 cmas.ontology.agentcommunication.ACL.ACLMessage Class Reference

Classes

- enum Performative

Public Member Functions

- **ACLMessage** (Performative performative, CommunicativeActs sender, Content content)
- **ACLMessage** (ACLMessage message)
- ACLMessage **copy** ()
- Performative **getPerformative** ()
- String **getConversationID** ()
- void **setConversationID** (String conversationID)
- String **getOntology** ()
- void **setOntology** (String ontology)
- Content **getContent** ()
- void **setContent** (String content)
- CommunicativeActs **getSender** ()
- StringProperty **senderProperty** ()
- void **setSender** (CommunicativeActs sender)
- CommunicativeActs **getReceiver** ()
- StringProperty **receiverProperty** ()
- void **setReceiver** (CommunicativeActs receiver)
- String **getLanguage** ()
- void **setLanguage** (String language)
- CommunicativeActs **getReplyTo** ()
- StringProperty **replyToProperty** ()

- void **setReplyTo** (CommunicativeActs replyTo)
- void **setPerformative** (Performative performative)
- String **getTimestampSend** ()
- void **setTimestampSend** (String timestampSend)
- String **getTimestampReceived** ()
- void **setTimestampReceived** (String timestampReceived)
- CommunicativeActs **getDelegateTo** ()
- void **setDelegateTo** (CommunicativeActs delegate)
- boolean **isAnswer** ()

Static Public Member Functions

- static StringProperty **getFullName** (CommunicativeActs item)

The documentation for this class was generated from the following file:

- ACLMessage.java

16.4 cmas.ontology.action.Action Class Reference

Public Member Functions

- **Action** (Entity Owner)
- String **getProgram** ()
- void **setProgram** (String actions)
- Parser **getCompiledProgram** ()
- void **compile** () throws ExceptionParameters, Exception
- void **compile** (Agent agent, Entity target, Variable ... variables) throws ExceptionRuntime, ExceptionParser, ExceptionBase, ExceptionExit
- void **compile** (Agent agent, Entity target, ArrayList< Variable > variables, ArrayList< AbstractInterface > abstractInterfaces) throws ExceptionRuntime, ExceptionParser, ExceptionBase, ExceptionExit
- boolean **isCompiled** ()
- void **run** () throws ExceptionRuntime, ExceptionParser, ExceptionBase, ExceptionExit
- void **run** (Agent agent, Entity target, Variable... variables) throws ExceptionRuntime, ExceptionParser, ExceptionBase, ExceptionExit
- void **replaceVariableObject** (VariableObject variable)
- void **replaceVariableArray** (VariableArray variable)
- void **addVariable** (Variable variable)
- void **closeDialogs** ()
- void **closeFiles** ()
- DebugResult **BreakPointCallBack** (StatementBase globalBlock, StatementBase localBlock, StatementBase statement, BreakPointBase breakpoint)
- Object **callFunction** (String Name) throws ExceptionParameters, Exception
- Object **callFunction** (String Name, Agent agent, Entity target, Vector< Expression > param, Variable... variables) throws ExceptionRuntime, ExceptionParser, ExceptionBase, ExceptionExit
- boolean **callProcedure** (String Name) throws ExceptionParameters, Exception
- boolean **callProcedure** (String Name, Agent agent, Entity target, Variable... variables) throws Exception↩ Runtime, ExceptionParser, ExceptionBase, ExceptionExit
- boolean **callProcedure** (String Name, Agent agent, Entity target, Vector< Expression > param, Variable... variables) throws ExceptionRuntime, ExceptionParser, ExceptionBase, ExceptionExit
- boolean **callProcedureForce** (String Name, Agent agent, Entity target) throws ExceptionParameters, Exception
- void **showRuntimeError** (ExceptionStructuredText e, Entity entity)
- void **stopEditor** (ExceptionStructuredText e, Entity entity)
- void **copyParam** (ContentParameter[ ] param, InterfaceBase localInterface)
- VariableObject **createRemoteInterfaceVariable** (InterfaceBase remoteInterface) throws ExceptionVariable, ExceptionType

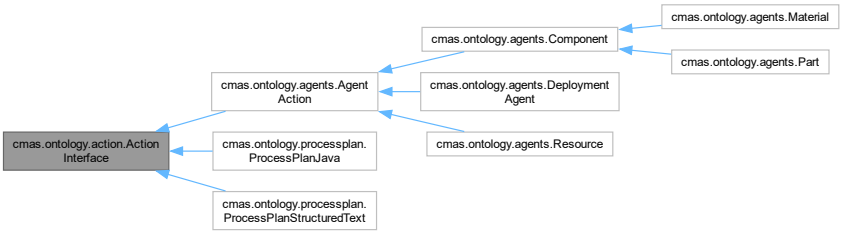
- VariableObject **createRemoteInterfaceVariable** (ContentParameter[ ] param, InterfaceBase localInterface) throws ExceptionBase, ExceptionRuntime
- VariableObject **createObject** (String varName, Entity entity) throws ExceptionType, ExceptionConstant, ExceptionVariable
- boolean **setFunctionValue** (String functionName, Object value) throws ExceptionParser, ExceptionType, ExceptionConstant, ExceptionVariable, ExceptionNullValue

The documentation for this class was generated from the following file:

- Action.java

16.5 cmas.ontology.action.ActionInterface Interface Reference

Inheritance diagram for cmas.ontology.action.ActionInterface:



Public Member Functions

- void **gotoLine** (ExceptionStructuredText e)
- void **stopEditor** (ExceptionStructuredText e)

The documentation for this interface was generated from the following file:

- ActionInterface.java

16.6 cmas.ontology.action.ActionJava Class Reference

Public Member Functions

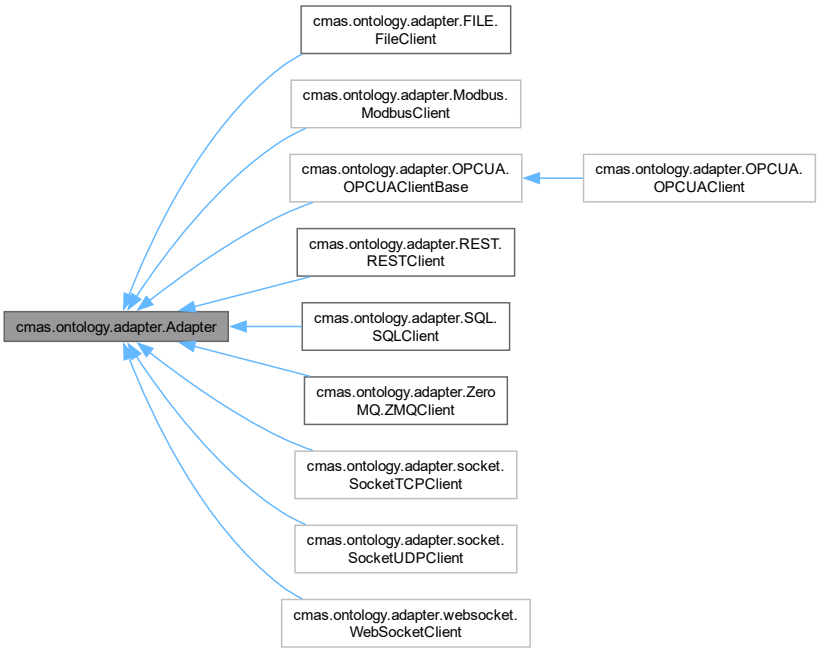
- **ActionJava** (String fileName, String dependencies, Property<?> program)
- ProcessPlanInterface **getObject** () throws Exception
- boolean **compile** () throws ExceptionParser
- String **getFileName** ()
- void **setFileName** (String fileName)
- String **getDependencies** ()
- void **setDependencies** (String dependencies)

The documentation for this class was generated from the following file:

- ActionJava.java

16.7 cmas.ontology.adapter.Adapter Interface Reference

Inheritance diagram for cmas.ontology.adapter.Adapter:



Public Member Functions

- abstract boolean **close** ()
- abstract String ReadData (String Address, String Name, Object Value) throws Exception
- abstract void WriteData (String Address, String Name, Boolean bValue) throws Exception
- abstract void **WriteData** (String Address, String Name, String sValue) throws Exception
- abstract void **WriteData** (String Address, String Name, Integer iValue) throws Exception
- abstract void **WriteData** (String Address, String Name, Double dValue) throws Exception
- abstract void setDebug (boolean bValue)
- abstract String **getDefaultValue** ()
- abstract String **getDescription** ()
- abstract boolean **isValid** (String Source)
- abstract Adapter **createNew** (String Source)

16.7.1 Member Function Documentation

16.7.1.1 ReadData()

```
abstract String cmas.ontology.adapter.Adapter.ReadData (
            String Address,
            String Name,
            Object Value) throws Exception  [abstract]
```

Implemented in cmas.ontology.adapter.SQL.SQLClient.

16.7.1.2 setDebug()

```
abstract void cmas.ontology.adapter.Adapter.setDebug (
            boolean bValue)  [abstract]
```

Implemented in cmas.ontology.adapter.OPCUA.OPCUAClientBase.

16.7.1.3 WriteData()

```
abstract void cmas.ontology.adapter.Adapter.WriteData (
            String Address,
            String Name,
            Boolean bValue) throws Exception  [abstract]
```

Implemented in cmas.ontology.adapter.SQL.SQLClient.
The documentation for this interface was generated from the following file:

- Adapter.java

16.8 cmas.ontology.adapter.AdapterFactory Class Reference

Static Public Member Functions

- static Adapter **getClient** (String Source)
- static String **getType** (String Source)
- static ArrayList< Pair< String, Adapter > > **getAllAdapters** ()
- static void **closeClients** ()

Package Attributes

- OPCUAClient **opcuaClients**

Static Package Attributes

- static ArrayList< Pair< String, Adapter > > **allAdapterInstances** = new ArrayList<Pair<String, Adapter>>()
- static ArrayList< Pair< String, Adapter > > adapterObjects

16.8.1 Member Data Documentation

16.8.1.1 adapterObjects

```
ArrayList<Pair<String, Adapter> > cmas.ontology.adapter.AdapterFactory.adapterObjects  [static],
[package]
```

Initial value:

```
= new ArrayList<>() {
        private static final long serialVersionUID = 1L;
        {
            add(new Pair<>("OPCUA", new OPCUAClient()));
            add(new Pair<>("REST", new RESTClient()));
            add(new Pair<>("FILE", new FileClient()));
            add(new Pair<>("MODBUS", new ModbusClient()));
            add(new Pair<>("SOCKET TCP", new SocketTCPClient()));
            add(new Pair<>("SOCKET UDP", new SocketUDPClient()));
            add(new Pair<>("ZMQ", new ZMQClient()));
            add(new Pair<>("WebSocket", new WebSocketClient()));
            add(new Pair<>("SQL", new SQLClient()));
        }
    }
```

The documentation for this class was generated from the following file:

- AdapterFactory.java

16.9 cmas.ontology.adapter.Modbus.ModbusClient.ADDRESS_TYPE Enum Reference

Public Attributes

- **COIL**
- **DISCRETE_INPUT**

- **INPUT_REGISTER**
- **HOLDING_REGISTER**
- **HOLDING_REGISTER_SIGNED**

The documentation for this enum was generated from the following file:

- ModbusClient.java

16.10 cmas.ontology.agents.Agent Class Reference

Inheritance diagram for cmas.ontology.agents.Agent:



Classes

- enum AgentBaseTypes

Public Member Functions

- **Agent** (AgentBaseTypes basetype)
- **Agent** (Agent entity, boolean instance)
- String **getPhysicalID** ()
- void **setPhysicalID** (String physicalID)
- String **getAgentSpecificType** ()
- void **setAgentSpecificType** (String agentSpecificType)
- AgentBaseTypes **getAgentBaseType** ()
- void **addAbstractInterface** (AbstractInterface abstractInterface, String Name)
- void **clearAbstractInterfaces** ()
- AbstractInterface **getAbstractInterface** (String Name)
- RuntimeTree **getRuntimeTree** ()
- void **setRuntimeTree** (RuntimeTree runtimeTree)
- Domain **getRuntimeDomain** ()
- void **setRuntimeDomain** (Domain runtimeDomain)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- LinkedBlockingQueue< WorkFlowLog > **getHistory** ()
- void **setHistory** (LinkedBlockingQueue< WorkFlowLog > p)
- void **addToHistory** (WorkFlowLog log)
- Boolean **getAutoDeploy** ()
- void **setAutoDeploy** (Boolean autoDeploy)
- abstract Entity copy (boolean instance)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()

- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- abstract void **openEntityEditor** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static AgentBaseTypes **getAgentType** (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.10.1 Member Function Documentation

16.10.1.1 copy()

```
abstract Entity cmas.ontology.base.Entity.copy (
            boolean instance)  [abstract], [inherited]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented in cmas.ontology.agents.DeploymentAgent, cmas.ontology.agents.Material, cmas.ontology.agents.Part, cmas.ontology.agents.Resource, cmas.ontology.base.Reference, cmas.ontology.batch.Batch, cmas.ontology.Hazard, cmas.ontology.interfaces.Goal, cmas.ontology.interfaces.Interface, cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, cmas.ontology.interfaces.InterfaceTrans cmas.ontology.model.ModelSettings, cmas.ontology.model.ModelState, cmas.ontology.Note, cmas.ontology.processplan.ProcessPlar cmas.ontology.processplan.ProcessPlanJava, cmas.ontology.processplan.ProcessPlanStructuredText, cmas.ontology.processplan.Sk cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.VariableInteger, cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, cmas.ontology.variables.VariableString, cmas.ontology.workflow.Requirement, and cmas.ontology.workflow.WorkFlowLog.
References Entity().

16.10.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.10.1.3 DeSerialize()

```
void cmas.ontology.agents.Agent.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.base.Entity.

16.10.1.4 Factory()

```
Entity cmas.ontology.agents.Agent.Factory (
            JsonObject job) throws ExceptionType  [static]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.10.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.10.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.10.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.10.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.10.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.10.1.10 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.10.1.11 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.10.1.12 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.10.1.13 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.10.1.14 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.10.1.15 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.10.1.16 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.10.1.17 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.10.1.18 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.10.1.19 Serialize()

```
JsonObject cmas.ontology.agents.Agent.Serialize ()
```

Reimplemented from cmas.ontology.base.Entity.

16.10.1.20 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.10.1.21 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.10.1.22 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.10.1.23 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

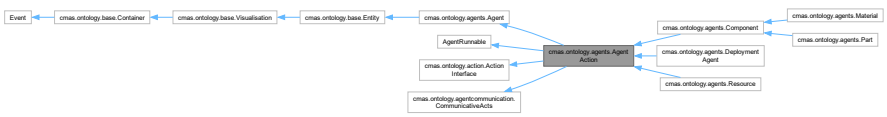
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Agent.java

16.11 cmas.ontology.agents.AgentAction Class Reference

Inheritance diagram for cmas.ontology.agents.AgentAction:



Classes

- enum AgentState

Public Member Functions

- **AgentAction** (AgentBaseTypes type)
- **AgentAction** (AgentAction entity, boolean instance)
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- synchronized Action **getActionObject** ()
- synchronized void **setState** (AgentState newState)
- synchronized AgentState **getState** ()
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- synchronized void **setPaused** (boolean value)
- synchronized boolean **getPaused** ()
- synchronized void postMessage (ACLMessage message)
- Entity getCommunicative ()
- void **onInit** ()
- void **onDepot** ()
- void **onDeploy** ()
- void **onRun** ()
- void **Running** ()
- void **onUndeploy** ()
- void **onFinished** ()
- void **onFailed** ()
- void **onStop** ()
- void **onMessage** (ACLMessage message)
- void **run** ()
- MessageHandler getMessageHandler ()
- void gotoLine (ExceptionStructuredText e)
- void stopEditor (ExceptionStructuredText e)
- void **compile** () throws ExceptionParameters, Exception
- String **getPhysicalID** ()
- void **setPhysicalID** (String physicalID)
- String **getAgentSpecificType** ()
- void **setAgentSpecificType** (String agentSpecificType)
- AgentBaseTypes **getAgentBaseType** ()
- void **addAbstractInterface** (AbstractInterface abstractInterface, String Name)
- void **clearAbstractInterfaces** ()
- AbstractInterface **getAbstractInterface** (String Name)
- RuntimeTree **getRuntimeTree** ()
- void **setRuntimeTree** (RuntimeTree runtimeTree)
- Domain **getRuntimeDomain** ()
- void **setRuntimeDomain** (Domain runtimeDomain)
- LinkedBlockingQueue< WorkFlowLog > **getHistory** ()
- void **setHistory** (LinkedBlockingQueue< WorkFlowLog > p)
- void **addToHistory** (WorkFlowLog log)
- Boolean **getAutoDeploy** ()
- void **setAutoDeploy** (Boolean autoDeploy)
- abstract Entity copy (boolean instance)
- void delete ()
- String **toString** ()
- String getID ()

- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- abstract void **openEntityEditor** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static AgentBaseTypes **getAgentType** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Attributes

- Action **action**
- MessageHandler **messageHandler**
- Properties **properties**
- Relations **relations**

16.11.1 Member Function Documentation

16.11.1.1 copy()

```
abstract Entity cmas.ontology.base.Entity.copy (
            boolean instance)  [abstract], [inherited]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented in cmas.ontology.agents.DeploymentAgent, cmas.ontology.agents.Material, cmas.ontology.agents.Part, cmas.ontology.agents.Resource, cmas.ontology.base.Reference, cmas.ontology.batch.Batch, cmas.ontology.Hazard, cmas.ontology.interfaces.Goal, cmas.ontology.interfaces.Interface, cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, cmas.ontology.interfaces.InterfaceTrans cmas.ontology.model.ModelSettings, cmas.ontology.model.ModelState, cmas.ontology.Note, cmas.ontology.processplan.ProcessPlar cmas.ontology.processplan.ProcessPlanJava, cmas.ontology.processplan.ProcessPlanStructuredText, cmas.ontology.processplan.Sk cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.VariableInteger, cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, cmas.ontology.variables.VariableString, cmas.ontology.workflow.Requirement, and cmas.ontology.workflow.WorkFlowLog.
References Entity().

16.11.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.11.1.3 DeSerialize()

```
void cmas.ontology.agents.AgentAction.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.agents.Agent.

16.11.1.4 Factory()

```
Entity cmas.ontology.agents.Agent.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.11.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.11.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.11.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.11.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.11.1.9 getCommunicative()

```
Entity cmas.ontology.agents.AgentAction.getCommunicative ()
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.11.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.11.1.11 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.11.1.12 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.11.1.13 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.11.1.14 getMessageHandler()

```
MessageHandler cmas.ontology.agents.AgentAction.getMessageHandler ()
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.11.1.15 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.11.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.11.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.11.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.11.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.11.1.20 gotoLine()

```
void cmas.ontology.agents.AgentAction.gotoLine (
            ExceptionStructuredText e)
```

Implements cmas.ontology.action.ActionInterface.

16.11.1.21 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.11.1.22 postMessage()

```
synchronized void cmas.ontology.agents.AgentAction.postMessage (
            ACLMessage message)
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.11.1.23 Serialize()

```
JsonObject cmas.ontology.agents.AgentAction.Serialize ()
```

Reimplemented from cmas.ontology.agents.Agent.

16.11.1.24 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.11.1.25 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.11.1.26 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.11.1.27 setName()

```
void cmas.ontology.base.Entity.setName (
             String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.11.1.28 stopEditor()

```
void cmas.ontology.agents.AgentAction.stopEditor (
             ExceptionStructuredText e)
```

Implements cmas.ontology.action.ActionInterface.
The documentation for this class was generated from the following file:

- AgentAction.java

16.12 cmas.ontology.agents.Agent.AgentBaseTypes Enum Reference

Public Attributes

- **PART**
- **MATERIAL**
- **RESOURCE**
- **DEPLOYMENTAGENT**

The documentation for this enum was generated from the following file:

- Agent.java

16.13 cmas.ontology.agents.AgentAction.AgentState Enum Reference

Public Attributes

- **INIT**
- **DEPOT**
- **DEPLOY**
- **RUNNING**
- **UNDEPLOY**
- **FINISHED**
- **FAILED**
- **STOPPED**
- **ABORT**

The documentation for this enum was generated from the following file:

- AgentAction.java

16.14 cmas.ontology.interfaces.Attachment Class Reference

Public Member Functions

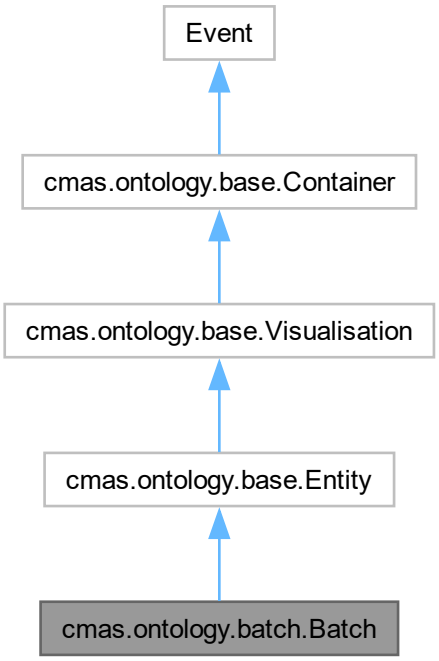
- InterfaceBase **getLocal** ()
- void **setLocal** (InterfaceBase local)
- InterfaceBase **getRemote** ()
- void **setRemote** (InterfaceBase remote)
- boolean **isAttached** ()
- boolean **isSource** ()
- String **getAttachedTo** ()
- void **makeConnection** (InterfaceBase localInterface, InterfaceBase remoteInterface, boolean source) throws ExceptionRuntime
- String **toString** ()

The documentation for this class was generated from the following file:

- Attachment.java

16.15 cmas.ontology.batch.Batch Class Reference

Inheritance diagram for cmas.ontology.batch.Batch:



Public Member Functions

- **Batch** (Batch deployBatch, boolean instance)
- ArrayList< BatchItem > **getBatch** ()
- ArrayList< BatchItem > **getBatchCopy** ()
- void **setBatch** (ArrayList< BatchItem > value)

- void **clear** ()
- int **size** ()
- Integer **getNumberOfItems** (int i)
- BatchItem **getDeploymentItem** (int i)
- Pair< Agent, InterfaceBase > **getDeploymentInstances** (int i)
- JsonArray **SerializeBatch** ()
- ArrayList< BatchItem > **DeSerializeBatch** (String value)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- void **add** (BatchItem batchItem)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)

- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.15.1 Member Function Documentation

16.15.1.1 copy()

```
Entity cmas.ontology.batch.Batch.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().

16.15.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.15.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.15.1.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.15.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.15.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.15.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.15.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.15.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.15.1.10 getEntityDescription()

```
String cmas.ontology.batch.Batch.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.15.1.11 getEntityIconFile()

```
String cmas.ontology.batch.Batch.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.15.1.12 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.15.1.13 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.15.1.14 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.15.1.15 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.15.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.15.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.15.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.15.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.15.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.15.1.21 openEntityEditor()

```
void cmas.ontology.batch.Batch.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.15.1.22 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.15.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.15.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.15.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.15.1.26 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Batch.java

16.16 cmas.ontology.batch.BatchItem Class Reference

Public Member Functions

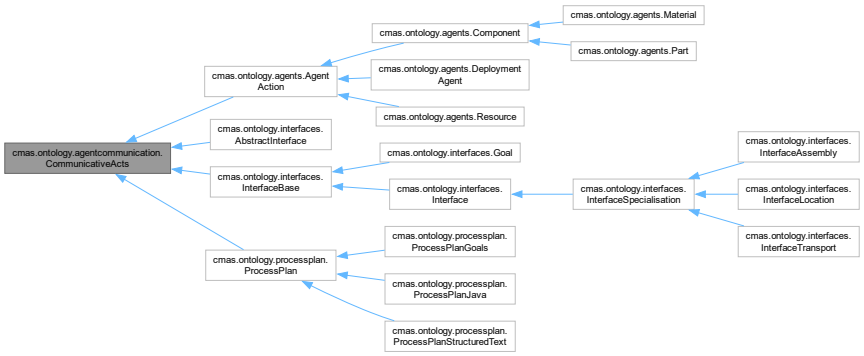
- **BatchItem** (String agentID, String attachToID)
- **BatchItem** (BatchItem batchItem)
- **BatchItem** (JsonObject asJsonObject)
- BatchItem **copy** ()
- String **getAgentID** ()
- void **setAgentID** (String agentID)
- String **getAttachToID** ()
- void **setAttachToID** (String attachToID)
- Integer **getNumberOfItems** ()
- void **setNumberOfItems** (Integer numberOfItems)
- String **getDescription** ()
- void **setDescription** (String description)
- JsonObject **serialize** ()
- void **deSerialize** (JsonObject obj)

The documentation for this class was generated from the following file:

- BatchItem.java

16.17 cmas.ontology.agentcommunication.CommunicativeActs Interface Reference

Inheritance diagram for cmas.ontology.agentcommunication.CommunicativeActs:



Public Member Functions

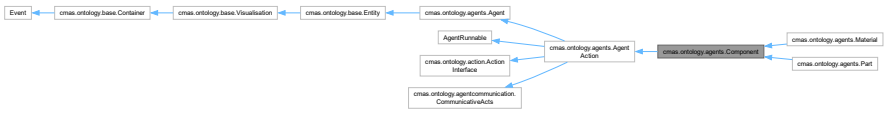
- Entity **getCommunicative** ()
- MessageHandler **getMessageHandler** ()
- void **postMessage** (ACLMessage message)
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

The documentation for this interface was generated from the following file:

- CommunicativeActs.java

16.18 cmas.ontology.agents.Component Class Reference

Inheritance diagram for cmas.ontology.agents.Component:



Public Member Functions

- **Component** (AgentBaseTypes type)
- **Component** (Component entity, boolean instance)
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- synchronized Action **getActionObject** ()
- synchronized void **setState** (AgentState newState)
- synchronized AgentState **getState** ()
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- synchronized void **setPaused** (boolean value)
- synchronized boolean **getPaused** ()
- synchronized void postMessage (ACLMessage message)
- Entity getCommunicative ()
- void **onInit** ()
- void **onDepot** ()
- void **onDeploy** ()
- void **onRun** ()
- void **Running** ()
- void **onUndeploy** ()
- void **onFinished** ()
- void **onFailed** ()
- void **onStop** ()
- void **onMessage** (ACLMessage message)
- void **run** ()
- MessageHandler getMessageHandler ()
- void gotoLine (ExceptionStructuredText e)
- void stopEditor (ExceptionStructuredText e)
- void **compile** () throws ExceptionParameters, Exception
- String **getPhysicalID** ()
- void **setPhysicalID** (String physicalID)
- String **getAgentSpecificType** ()

- void **setAgentSpecificType** (String agentSpecificType)
- AgentBaseTypes **getAgentBaseType** ()
- void **addAbstractInterface** (AbstractInterface abstractInterface, String Name)
- void **clearAbstractInterfaces** ()
- AbstractInterface **getAbstractInterface** (String Name)
- RuntimeTree **getRuntimeTree** ()
- void **setRuntimeTree** (RuntimeTree runtimeTree)
- Domain **getRuntimeDomain** ()
- void **setRuntimeDomain** (Domain runtimeDomain)
- LinkedBlockingQueue< WorkFlowLog > **getHistory** ()
- void **setHistory** (LinkedBlockingQueue< WorkFlowLog > p)
- void **addToHistory** (WorkFlowLog log)
- Boolean **getAutoDeploy** ()
- void **setAutoDeploy** (Boolean autoDeploy)
- abstract Entity copy (boolean instance)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- abstract void **openEntityEditor** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)

- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static AgentBaseTypes **getAgentType** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Attributes

- Action **action**
- MessageHandler **messageHandler**
- Properties **properties**
- Relations **relations**

16.18.1 Member Function Documentation

16.18.1.1 copy()

```
abstract Entity cmas.ontology.base.Entity.copy (
            boolean instance)  [abstract], [inherited]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented in cmas.ontology.agents.DeploymentAgent, cmas.ontology.agents.Material, cmas.ontology.agents.Part, cmas.ontology.agents.Resource, cmas.ontology.base.Reference, cmas.ontology.batch.Batch, cmas.ontology.Hazard, cmas.ontology.interfaces.Goal, cmas.ontology.interfaces.Interface, cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, cmas.ontology.interfaces.InterfaceTrans cmas.ontology.model.ModelSettings, cmas.ontology.model.ModelState, cmas.ontology.Note, cmas.ontology.processplan.ProcessPlar cmas.ontology.processplan.ProcessPlanJava, cmas.ontology.processplan.ProcessPlanStructuredText, cmas.ontology.processplan.Sk cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.VariableInteger, cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, cmas.ontology.variables.VariableString, cmas.ontology.workflow.Requirement, and cmas.ontology.workflow.WorkFlowLog.
References Entity().

16.18.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.18.1.3 DeSerialize()

```
void cmas.ontology.agents.AgentAction.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.agents.Agent.

16.18.1.4 Factory()

```
Entity cmas.ontology.agents.Agent.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.18.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.18.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.18.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.18.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.18.1.9 getCommunicative()

```
Entity cmas.ontology.agents.AgentAction.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.18.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.18.1.11 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.18.1.12 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.18.1.13 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.18.1.14 getMessageHandler()

`MessageHandler cmas.ontology.agents.AgentAction.getMessageHandler () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.18.1.15 getName()

`String cmas.ontology.base.Entity.getName () [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.18.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity) [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.18.1.17 getRelations()

`LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations () [inherited]`

Returns

a HashMap of all relation categories and their linked names.

16.18.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.18.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.18.1.20 gotoLine()

```
void cmas.ontology.agents.AgentAction.gotoLine (
            ExceptionStructuredText e)  [inherited]
```

Implements cmas.ontology.action.ActionInterface.

16.18.1.21 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.18.1.22 postMessage()

```
synchronized void cmas.ontology.agents.AgentAction.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.18.1.23 Serialize()

```
JsonObject cmas.ontology.agents.AgentAction.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.agents.Agent.

16.18.1.24 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.18.1.25 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.18.1.26 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.18.1.27 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.18.1.28 stopEditor()

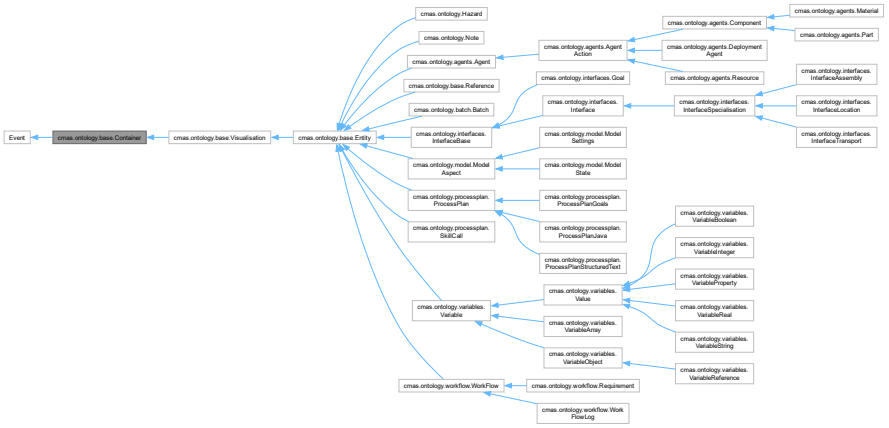
```
void cmas.ontology.agents.AgentAction.stopEditor (
            ExceptionStructuredText e)  [inherited]
```

Implements cmas.ontology.action.ActionInterface.
The documentation for this class was generated from the following file:

- Component.java

16.19 cmas.ontology.base.Container Class Reference

Inheritance diagram for cmas.ontology.base.Container:



Public Member Functions

- **Container** (EventType TypeOfEvent)
- **Container** (JsonObject job) throws ExceptionType
- **Container** (Container container, boolean instance)
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **delete** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- JsonObject **Serialize** ()
- void **DeSerialize** (JsonObject job) throws ExceptionType
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.19.1 Member Function Documentation

16.19.1.1 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()
```

Get the modeling entity instance id from a runnable instanced object.

Returns

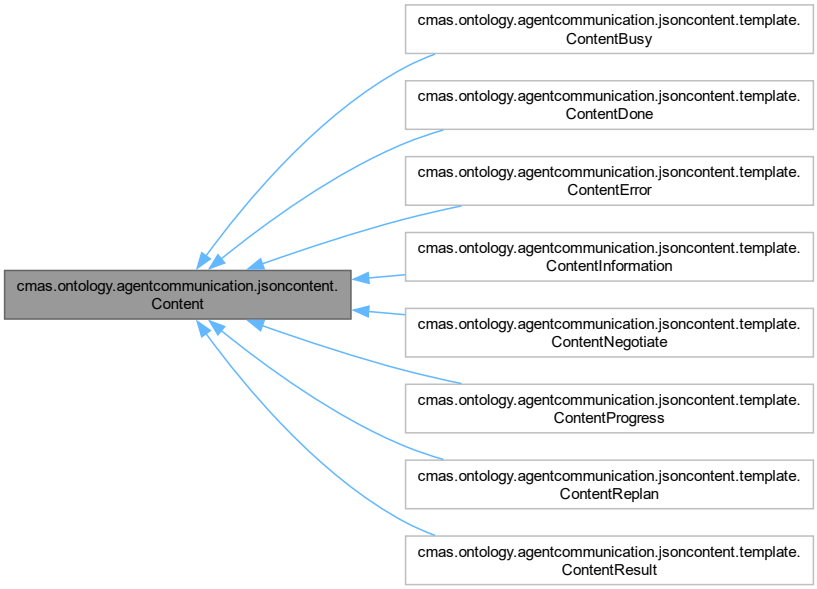
Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

The documentation for this class was generated from the following file:

- Container.java

16.20 cmas.ontology.agentcommunication.jsoncontent.Content Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.Content:



Public Member Functions

- **Content** (String method)
- **Content** (String method, String name, String value)
- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static Content **deserialize** (String stringJSON)

16.20.1 Member Function Documentation

16.20.1.1 createParameters()

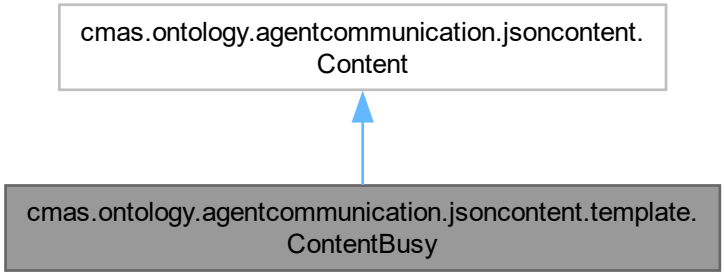
```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String...  values)
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- Content.java

16.21 cmas.ontology.agentcommunication.jsoncontent.template.ContentBusy Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.template.ContentBusy:



Public Member Functions

- **ContentBusy** (String Message)
- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static String **deserialize** (Content content)
- static Content **deserialize** (String stringJSON)

Static Public Attributes

- static final String **METHOD** ="Busy"

16.21.1 Member Function Documentation

16.21.1.1 createParameters()

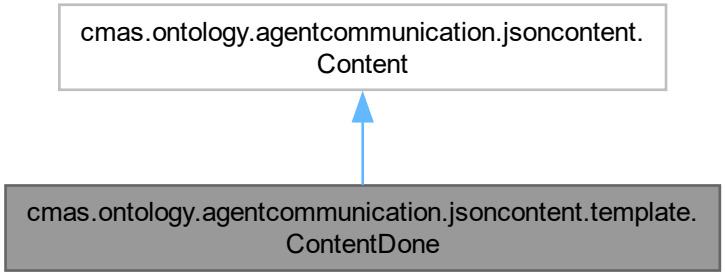
```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String... values)  [inherited]
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- ContentBusy.java

16.22 cmas.ontology.agentcommunication.jsoncontent.template.↩ ContentDone Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.template.ContentDone:



Public Member Functions

- **ContentDone** (ProcessPlanReturnValue value)
- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static ProcessPlanReturnValue **deserialize** (Content content)
- static Content **deserialize** (String stringJSON)

Static Public Attributes

- static final String **METHOD** ="Done"

16.22.1 Member Function Documentation

16.22.1.1 createParameters()

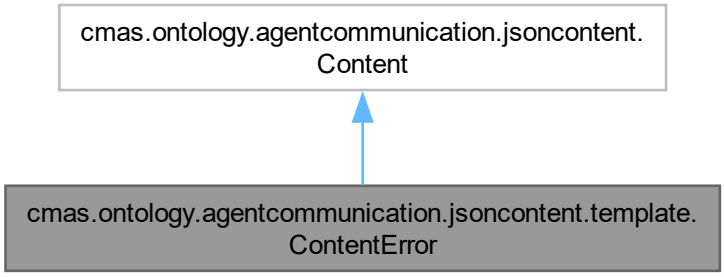
```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String...  values)  [inherited]
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- ContentDone.java

16.23 cmas.ontology.agentcommunication.jsoncontent.template.↩ ContentError Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.template.ContentError:



Public Member Functions

- **ContentError** (String Message, Integer ReturnValue)
- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static Content **deserialize** (String stringJSON)

Static Public Attributes

- static final String **METHOD** ="Error"

16.23.1 Member Function Documentation

16.23.1.1 createParameters()

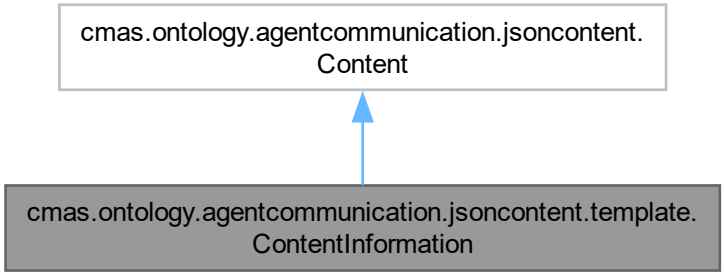
```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String... values)  [inherited]
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- ContentError.java

16.24 cmas.ontology.agentcommunication.jsoncontent.template.↩ ContentInformation Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.template.ContentInformation:



Public Member Functions

- **ContentInformation** (String Message)
- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static String **deserialize** (Content content)
- static Content **deserialize** (String stringJSON)

Static Public Attributes

- static final String **METHOD** ="Information"

16.24.1 Member Function Documentation

16.24.1.1 createParameters()

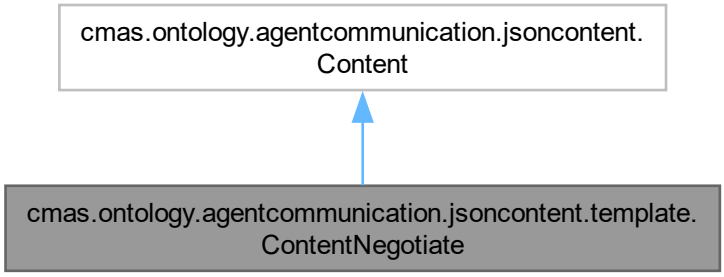
```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String...  values)  [inherited]
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- ContentInformation.java

16.25 cmas.ontology.agentcommunication.jsoncontent.template.↩ ContentNegotiate Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.template.ContentNegotiate:



Public Member Functions

- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static Content **deserialize** (String stringJSON)

Static Public Attributes

- static final String **METHOD** ="Negotiate"

16.25.1 Member Function Documentation

16.25.1.1 createParameters()

```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String... values) [inherited]
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- ContentNegotiate.java

16.26 cmas.ontology.agentcommunication.jsoncontent.Content↩ Parameter Class Reference

Public Member Functions

- **ContentParameter** (String name, String value)
- String **getName** ()
- String **getValue** ()
- void **setName** (String name)
- void **setValue** (String value)
- String **toString** ()

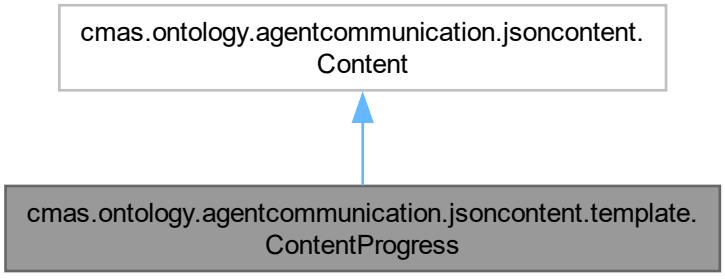
16.26.1 Detailed Description

This class define json method parameter.
The documentation for this class was generated from the following file:

- ContentParameter.java

16.27 cmas.ontology.agentcommunication.jsoncontent.template.↩ ContentProgress Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.template.ContentProgress:



Public Member Functions

- **ContentProgress** (Integer ProgressValue)
- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static Content **deserialize** (String stringJSON)

Static Public Attributes

- static final String **METHOD** ="Progress"

16.27.1 Member Function Documentation

16.27.1.1 createParameters()

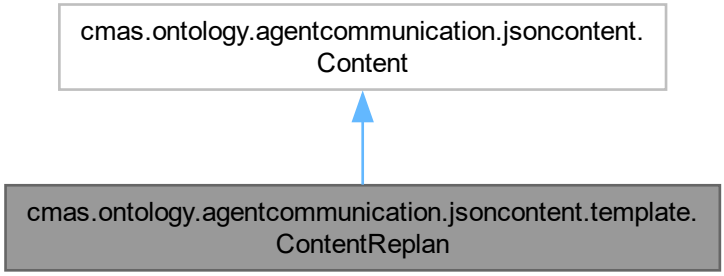
```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String...  values)  [inherited]
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- ContentProgress.java

16.28 cmas.ontology.agentcommunication.jsoncontent.template.ContentReplan Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.template.ContentReplan:



Public Member Functions

- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static Content **deserialize** (String stringJSON)

Static Public Attributes

- static final String **METHOD** ="Replan"

16.28.1 Member Function Documentation

16.28.1.1 createParameters()

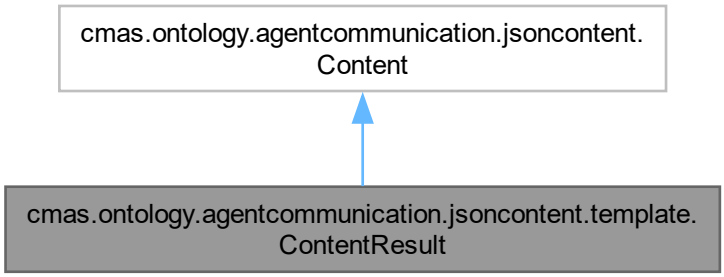
```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String...  values)  [inherited]
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- ContentReplan.java

16.29 cmas.ontology.agentcommunication.jsoncontent.template.↩ ContentResult Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.jsoncontent.template.ContentResult:



Public Member Functions

- String **getParameterValue** (String ParameterName)
- boolean **getParameterValueAsBoolean** (String ParameterName, boolean DefaultValue)
- void createParameters (String method, String... values)
- int **getNumberOfParameters** ()
- ContentParameter[ ] **getAllParameters** ()
- ContentParameter **getParameter** (int index)
- Content **addParameter** (ContentParameter parameter)
- String **toString** ()
- String **serialize** ()
- String **getMethod** ()
- void **setMethod** (String method)

Static Public Member Functions

- static Content **deserialize** (String stringJSON)

Static Public Attributes

- static final String **METHOD** ="ReturnValues"

16.29.1 Member Function Documentation

16.29.1.1 createParameters()

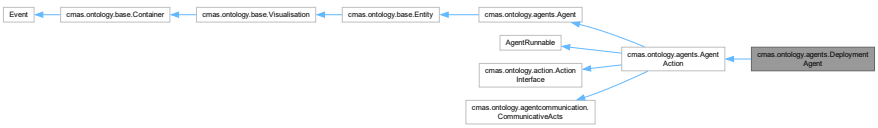
```
void cmas.ontology.agentcommunication.jsoncontent.Content.createParameters (
            String method,
            String...  values)  [inherited]
```

Create and add parameters based on string values. The string values must be in pair, i.e. the name of the parameter and the value. Example createParameters("myMethod", "Parameter1", "Value1", Parameter2", "Value2");
The documentation for this class was generated from the following file:

- ContentResult.java

16.30 cmas.ontology.agents.DeploymentAgent Class Reference

Inheritance diagram for cmas.ontology.agents.DeploymentAgent:



Classes

- enum DeploymentCriteria

Public Member Functions

- **DeploymentAgent** (DeploymentAgent entity, boolean instance)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- DeploymentCriteria **getDeployAt** ()

- String **getVariableTrigger** ()
- void **setVariableTrigger** (String variableName)
- void **setDeployAt** (DeploymentCriteria deployAt)
- long **getDeploymentTimeInMS** ()
- String **getDeploymentTime** ()
- void **setDeploymentTime** (String deployArgument)
- Integer **getNumberOfTimes** ()
- void **setNumberOfTimes** (Integer numberOfTimes)
- void onRun ()
- void onUndeploy ()
- void onFailed ()
- void onStop ()
- void onFinished ()
- String **getRetryDelay** ()
- void **setRetryDelay** (String retryDelay)
- long **getRetryDelayTimeInMS** ()
- void openEntityEditor ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- synchronized Action **getActionObject** ()
- synchronized void **setState** (AgentState newState)
- synchronized AgentState **getState** ()
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- synchronized void **setPaused** (boolean value)
- synchronized boolean **getPaused** ()
- synchronized void postMessage (ACLMessage message)
- Entity getCommunicative ()
- void **onInit** ()
- void **onDepot** ()
- void **onDeploy** ()
- void **Running** ()
- void **onMessage** (ACLMessage message)
- void **run** ()
- MessageHandler getMessageHandler ()
- void gotoLine (ExceptionStructuredText e)
- void stopEditor (ExceptionStructuredText e)
- void **compile** () throws ExceptionParameters, Exception
- String **getPhysicalID** ()
- void **setPhysicalID** (String physicalID)
- String **getAgentSpecificType** ()
- void **setAgentSpecificType** (String agentSpecificType)
- AgentBaseTypes **getAgentBaseType** ()
- void **addAbstractInterface** (AbstractInterface abstractInterface, String Name)
- void **clearAbstractInterfaces** ()
- AbstractInterface **getAbstractInterface** (String Name)
- RuntimeTree **getRuntimeTree** ()
- void **setRuntimeTree** (RuntimeTree runtimeTree)
- Domain **getRuntimeDomain** ()
- void **setRuntimeDomain** (Domain runtimeDomain)
- LinkedBlockingQueue< WorkFlowLog > **getHistory** ()
- void **setHistory** (LinkedBlockingQueue< WorkFlowLog > p)
- void **addToHistory** (WorkFlowLog log)
- Boolean **getAutoDeploy** ()
- void **setAutoDeploy** (Boolean autoDeploy)

- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static AgentBaseTypes **getAgentType** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Attributes

- Action **action**
- MessageHandler **messageHandler**
- Properties **properties**
- Relations **relations**

16.30.1 Member Function Documentation

16.30.1.1 copy()

```
Entity cmas.ontology.agents.DeploymentAgent.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.30.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.30.1.3 DeSerialize()

```
void cmas.ontology.agents.AgentAction.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.agents.Agent.

16.30.1.4 Factory()

```
Entity cmas.ontology.agents.Agent.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.30.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.30.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.30.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.30.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.30.1.9 getCommunicative()

`Entity cmas.ontology.agents.AgentAction.getCommunicative () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.30.1.10 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.30.1.11 getEntityDescription()

`String cmas.ontology.agents.DeploymentAgent.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.30.1.12 getEntityIconFile()

`String cmas.ontology.agents.DeploymentAgent.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.30.1.13 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.30.1.14 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.30.1.15 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.30.1.16 getMessageHandler()

`MessageHandler cmas.ontology.agents.AgentAction.getMessageHandler () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.30.1.17 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.30.1.18 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.30.1.19 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.30.1.20 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.30.1.21 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.30.1.22 gotoLine()

```
void cmas.ontology.agents.AgentAction.gotoLine (
            ExceptionStructuredText e)  [inherited]
```

Implements cmas.ontology.action.ActionInterface.

16.30.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.30.1.24 onFailed()

```
void cmas.ontology.agents.DeploymentAgent.onFailed ()
```

Reimplemented from cmas.ontology.agents.AgentAction.

16.30.1.25 onFinished()

```
void cmas.ontology.agents.DeploymentAgent.onFinished ()
```

Reimplemented from cmas.ontology.agents.AgentAction.

16.30.1.26 onRun()

```
void cmas.ontology.agents.DeploymentAgent.onRun ()
```

Reimplemented from cmas.ontology.agents.AgentAction.

16.30.1.27 onStop()

```
void cmas.ontology.agents.DeploymentAgent.onStop ()
```

Reimplemented from cmas.ontology.agents.AgentAction.

16.30.1.28 onUndeploy()

```
void cmas.ontology.agents.DeploymentAgent.onUndeploy ()
```

Reimplemented from cmas.ontology.agents.AgentAction.

16.30.1.29 openEntityEditor()

```
void cmas.ontology.agents.DeploymentAgent.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.30.1.30 postMessage()

```
synchronized void cmas.ontology.agents.AgentAction.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.30.1.31 Serialize()

```
JsonObject cmas.ontology.agents.AgentAction.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.agents.Agent.

16.30.1.32 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.30.1.33 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.30.1.34 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.30.1.35 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.30.1.36 stopEditor()

```
void cmas.ontology.agents.AgentAction.stopEditor (
            ExceptionStructuredText e)  [inherited]
```

Implements cmas.ontology.action.ActionInterface.
The documentation for this class was generated from the following file:

- DeploymentAgent.java

16.31 cmas.ontology.agents.DeploymentAgent.DeploymentCriteria Enum Reference

Public Attributes

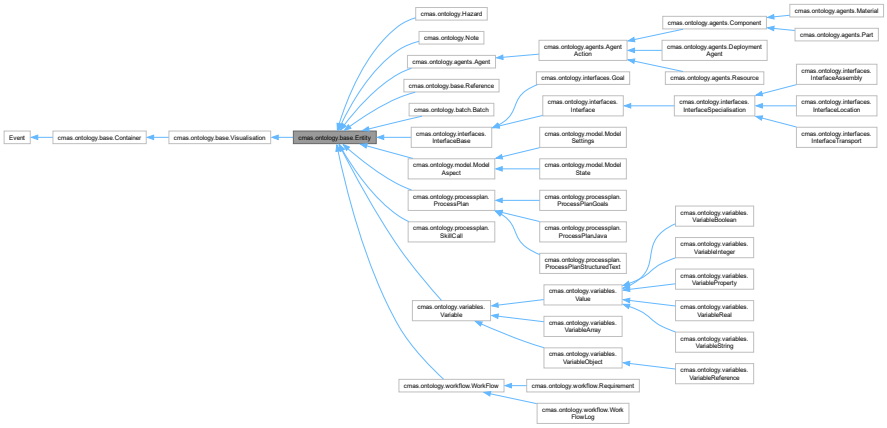
- **TIME**
- **INTERVAL**

The documentation for this enum was generated from the following file:

- DeploymentAgent.java

16.32 cmas.ontology.base.Entity Class Reference

Inheritance diagram for cmas.ontology.base.Entity:



Classes

- enum EntityType
- class UID

Public Member Functions

- Entity (EntityType Type)
- Entity (Entity entity, boolean instance)
- abstract Entity copy (boolean instance)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()

- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- abstract void **openEntityEditor** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.32.1 Constructor & Destructor Documentation

16.32.1.1 Entity() [1/2]

```
cmas.ontology.base.Entity.Entity (
            EntityType Type)
```

Entity constructor for all types based on Entity. Entity is an abstract class. Entity will be assigned an unique identifier for the entity i.e., ID that can be read by getID(). Default the name will be an empty string to change the name use setName().

Parameters

Type	the base type of entity to create. Note this type can not be changed after the Entity is created.

References setID(), and setName().
Referenced by copy(), cmas.ontology.base.Reference.copy(), cmas.ontology.batch.Batch.copy(), cmas.ontology.Hazard.copy(), cmas.ontology.Note.copy(), cmas.ontology.processplan.SkillCall.copy(), cmas.ontology.variables.Variable.copy(), Entity(), cmas.ontology.agents.Agent.Factory(), Factory(), cmas.ontology.processplan.ProcessPlan.Factory(), cmas.ontology.variables.Variable.Factory(), cmas.ontology.workflow.WorkFlow.Factory(), findRelationByName(), findRelationByNameIgnoreCase(), getRelationName(), and hasRelation().

16.32.1.2 Entity() [2/2]

```
cmas.ontology.base.Entity.Entity (
            Entity entity,
            boolean instance)
```

Copy constructor.

Parameters

entity	the entity to copy
instance	if instance is true a runtime entity will be create otherwise an modelling entity If instance is true all references will be replaces with instances of the referenced objects.

References Entity().

16.32.2 Member Function Documentation

16.32.2.1 copy()

```
abstract Entity cmas.ontology.base.Entity.copy (
            boolean instance)  [abstract]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented in cmas.ontology.agents.DeploymentAgent, cmas.ontology.agents.Material, cmas.ontology.agents.Part, cmas.ontology.agents.Resource, cmas.ontology.base.Reference, cmas.ontology.batch.Batch, cmas.ontology.Hazard, cmas.ontology.interfaces.Goal, cmas.ontology.interfaces.Interface, cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, cmas.ontology.interfaces.InterfaceTrans cmas.ontology.model.ModelSettings, cmas.ontology.model.ModelState, cmas.ontology.Note, cmas.ontology.processplan.ProcessPlar cmas.ontology.processplan.ProcessPlanJava, cmas.ontology.processplan.ProcessPlanStructuredText, cmas.ontology.processplan.Sk cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.VariableInteger, cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty,

cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, cmas.ontology.variables.VariableString, cmas.ontology.workflow.Requirement, and cmas.ontology.workflow.WorkFlowLog.
References Entity().

16.32.2.2 delete()

```
void cmas.ontology.base.Entity.delete ()
```
Reimplemented from cmas.ontology.base.Container.

16.32.2.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType
```
Reimplemented from cmas.ontology.base.Container.

16.32.2.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static]
```
Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.P
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.32.2.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)
```
Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.32.2.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.32.2.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.32.2.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()
```

Returns

a list of all relations categories

16.32.2.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()
```

Returns

the entity description.

16.32.2.10 getID()

```
String cmas.ontology.base.Entity.getID ()
```

Get the ID of the entity.

Returns

a string with current ID.

16.32.2.11 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.32.2.12 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.32.2.13 getName()

```
String cmas.ontology.base.Entity.getName ()
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.32.2.14 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.32.2.15 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()
```

Returns

a HashMap of all relation categories and their linked names.

16.32.2.16 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.32.2.17 getType()

```
EntityType cmas.ontology.base.Entity.getType ()
```

Get specific entity type.

Returns

entity type.

16.32.2.18 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.32.2.19 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()
```

Reimplemented from cmas.ontology.base.Container.

16.32.2.20 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.32.2.21 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.32.2.22 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.32.2.23 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Entity.java

16.33 cmas.ontology.base.Entity.EntityType Enum Reference

Public Attributes

- **ENTITY**
- **VARIABLE**
- **GOAL**
- **AGENT**
- **SKILL**
- **INTERFACE**
- **PROCESSPLAN**
- **REFERENCE**
- **WORKFLOW**
- **HAZARD**
- **BATCH**
- **NOTE**
- **SETTINGS**
- **STATE**
- **DATA_ROOT**

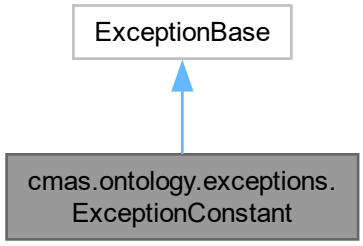
16.33.1 Detailed Description

Base type of the entity object.
The documentation for this enum was generated from the following file:

- Entity.java

16.34 cmas.ontology.exceptions.ExceptionConstant Class Reference

Inheritance diagram for cmas.ontology.exceptions.ExceptionConstant:



Public Member Functions

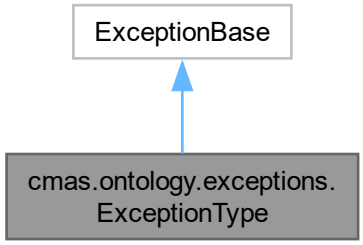
- **ExceptionConstant** (String Message)
- **ExceptionConstant** (String Message, String FileName, int LineNumber)

The documentation for this class was generated from the following file:

- ExceptionConstant.java

16.35 cmas.ontology.exceptions.ExceptionType Class Reference

Inheritance diagram for cmas.ontology.exceptions.ExceptionType:



Public Member Functions

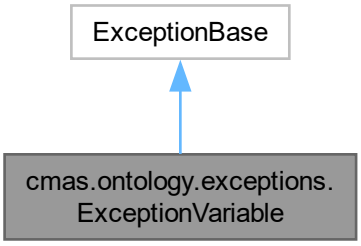
- **ExceptionType** (String Message)

The documentation for this class was generated from the following file:

- ExceptionType.java

16.36 cmas.ontology.exceptions.ExceptionVariable Class Reference

Inheritance diagram for cmas.ontology.exceptions.ExceptionVariable:



Public Member Functions

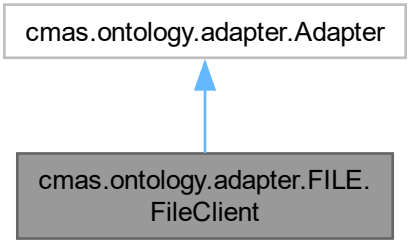
- **ExceptionVariable** (String Message)

The documentation for this class was generated from the following file:

- ExceptionVariable.java

16.37 cmas.ontology.adapter.FILE.FileClient Class Reference

Inheritance diagram for cmas.ontology.adapter.FILE.FileClient:



Public Member Functions

- **FileClient** (String Source)
- void **SetSource** (String Source)
- String ReadData (String Address, String Name, Object Value)
- void WriteData (String Address, String Name, Boolean bValue)
- void WriteData (String Address, String Name, String sValue)
- void WriteData (String Address, String Name, Integer iValue)
- void WriteData (String Address, String Name, Double dValue)
- boolean close ()

- void setDebug (boolean bValue)
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)

16.37.1 Detailed Description

File communications for variables

```
For adapter type of FILE the specific format is: FILE:[PATH]
eg, FILE:OUT:/tmp/outfile.txt
```

Author

Fredrik Danielsson, University West

16.37.2 Member Function Documentation

16.37.2.1 close()

```
boolean cmas.ontology.adapter.FILE.FileClient.close ()
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.2 createNew()

```
Adapter cmas.ontology.adapter.FILE.FileClient.createNew (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.3 getDefaultValue()

```
String cmas.ontology.adapter.FILE.FileClient.getDefaultValue ()
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.4 getDescription()

```
String cmas.ontology.adapter.FILE.FileClient.getDescription ()
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.5 isValid()

```
boolean cmas.ontology.adapter.FILE.FileClient.isValid (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.6 ReadData()

```
String cmas.ontology.adapter.FILE.FileClient.ReadData (
            String Address,
            String Name,
            Object Value)
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.7 setDebug()

```
void cmas.ontology.adapter.FILE.FileClient.setDebug (
            boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.8 WriteData() [1/4]

```
void cmas.ontology.adapter.FILE.FileClient.WriteData (
            String Address,
            String Name,
            Boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.9 WriteData() [2/4]

```
void cmas.ontology.adapter.FILE.FileClient.WriteData (
            String Address,
            String Name,
            Double dValue)
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.10 WriteData() [3/4]

```
void cmas.ontology.adapter.FILE.FileClient.WriteData (
            String Address,
            String Name,
            Integer iValue)
```

Implements cmas.ontology.adapter.Adapter.

16.37.2.11 WriteData() [4/4]

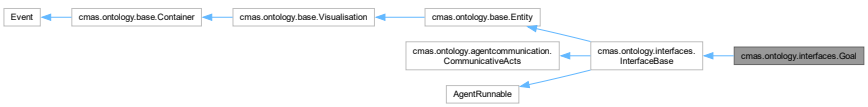
```
void cmas.ontology.adapter.FILE.FileClient.WriteData (
            String Address,
            String Name,
            String sValue)
```

Implements cmas.ontology.adapter.Adapter.
The documentation for this class was generated from the following file:

- FileClient.java

16.38 cmas.ontology.interfaces.Goal Class Reference

Inheritance diagram for cmas.ontology.interfaces.Goal:



Public Member Functions

- **Goal** (Goal goal, boolean instance)
- Entity copy (boolean instance)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- String **getPreCondition** ()
- void **setPreCondition** (String preCondition)
- Integer **getPriority** ()
- void **setPriority** (Integer priority)
- Boolean **getAssigned** ()
- void **setAssigned** (Boolean assigned)

- Boolean **isFulfilled** ()
- void **setFulfilled** (Boolean fulfilled)
- String **getProcessPlanName** ()
- void **setProcessPlanName** (String processPlan)
- boolean **evaluatePreCondition** (Agent agent) throws ExceptionRuntime, ExceptionBase, ExceptionParser
- Boolean **getFailed** ()
- void **setFailed** (Boolean failed)
- String **getFailedMessage** ()
- void **setFailedMessage** (String failedText)
- String **getInterface** ()
- void **setInterface** (String interface1)
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- String **getBookedByName** ()
- Boolean **isBusy** ()
- Boolean **isBooked** ()
- void **setBooked** (CommunicativeActs agent)
- CommunicativeActs **getBooked** ()
- Boolean **isAttachedTo** (InterfaceBase remoteInterface)
- Boolean **isAttached** ()
- String **getAttachedTo** ()
- Attachment **getAttachment** ()
- VariableMatchingType **getVariableMatching** ()
- ACLMessage **getOnGoingCommunication** ()
- void **setVariableMatching** (VariableMatchingType variableRequest)
- void **initInterface** (Agent agent)
- void **AttachTo** (InterfaceBase remote) throws ExceptionRuntime
- void **_AttachTo** (InterfaceBase remote, boolean source) throws ExceptionRuntime
- void **Detach** () throws ExceptionRuntime
- void **_Detach** ()
- void **Transfer** (InterfaceBase remoteInterface) throws ExceptionRuntime
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- void **run** ()
- String **getRequirements** (Entity skill, Agent callingAgent, InterfaceBase callingInterface) throws Exception↩ Runtime
- boolean **book** (InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **book** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **unbook** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface) throws Interrupted↩ Exception
- Agent **getRuntimeAgent** ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- Action **getActionObject** ()
- MessageHandler getMessageHandler ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)

- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- boolean addRequestToQueue (ACLMessage message, ArrayList< String > skillList, ContentParameter[] param)

Protected Attributes

- Properties **properties**
- Relations **relations**

16.38.1 Member Function Documentation

16.38.1.1 addRequestToQueue()

```
boolean cmas.ontology.interfaces.Goal.addRequestToQueue (
            ACLMessage message,
            ArrayList< String > skillList,
            ContentParameter[] param)  [protected]
```

Abstract method that should return true if it the request is fully handled and answered in the method or false if the base interface negotiation method should be used.

Parameters

message	
param	
skillList	

Returns

true or false

Reimplemented from cmas.ontology.interfaces.InterfaceBase.

16.38.1.2 copy()

```
Entity cmas.ontology.interfaces.Goal.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.38.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.38.1.4 DeSerialize()

```
void cmas.ontology.interfaces.Goal.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.base.Entity.

16.38.1.5 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.P
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.38.1.6 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.38.1.7 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.38.1.8 generateUniqueIDs()

`void cmas.ontology.base.Entity.generateUniqueIDs () [inherited]`
Will generate unique IDs for the entity and all relation objects
References setID().

16.38.1.9 getAllRelations()

`Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations () [inherited]`

Returns

a list of all relations categories

16.38.1.10 getCommunicative()

`Entity cmas.ontology.interfaces.InterfaceBase.getCommunicative () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.38.1.11 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.38.1.12 getEntityDescription()

`String cmas.ontology.interfaces.Goal.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.38.1.13 getEntityIconFile()

`String cmas.ontology.interfaces.Goal.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.38.1.14 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.38.1.15 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.38.1.16 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.38.1.17 getMessageHandler()

```
MessageHandler cmas.ontology.interfaces.InterfaceBase.getMessageHandler ()  [inherited]
```
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.38.1.18 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.38.1.19 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.38.1.20 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.38.1.21 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```
Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.38.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.38.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.38.1.24 openEntityEditor()

```
void cmas.ontology.interfaces.Goal.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.38.1.25 postMessage()

```
void cmas.ontology.interfaces.InterfaceBase.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.38.1.26 Serialize()

```
JsonObject cmas.ontology.interfaces.Goal.Serialize ()
```

Reimplemented from cmas.ontology.base.Entity.

16.38.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.38.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.38.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.38.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

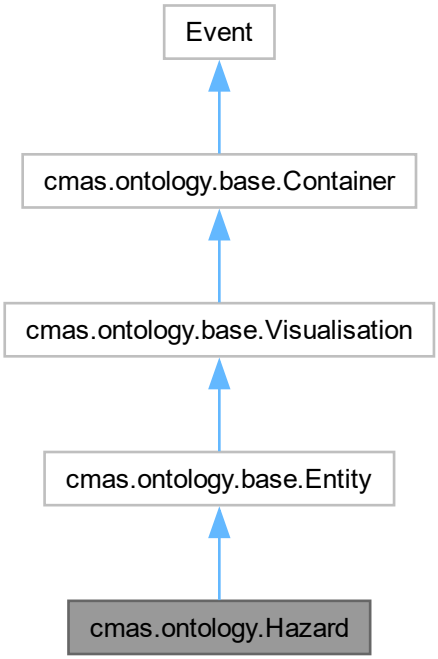
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Goal.java

16.39 cmas.ontology.Hazard Class Reference

Inheritance diagram for cmas.ontology.Hazard:



Public Member Functions

- **Hazard** (Hazard hazard, boolean instance)
- Entity copy (boolean instance)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- String getEntityDescription ()
- String getEntityIconFile ()
- Integer **getRisk** ()
- void **setRisk** (Integer risk)
- String **getTargetType** ()
- void **setTargetType** (String target)
- Integer **getFrequency** ()
- void **setFrequency** (Integer frequency)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)

- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.39.1 Member Function Documentation

16.39.1.1 copy()

```
Entity cmas.ontology.Hazard.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().

16.39.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.39.1.3 DeSerialize()

```
void cmas.ontology.Hazard.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.base.Entity.

16.39.1.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.P
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.39.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.39.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.39.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.39.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.39.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.39.1.10 getEntityDescription()

`String cmas.ontology.Hazard.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.39.1.11 getEntityIconFile()

`String cmas.ontology.Hazard.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.39.1.12 getID()

`String cmas.ontology.base.Entity.getID ()  [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.39.1.13 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID ()  [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.39.1.14 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.39.1.15 getName()

`String cmas.ontology.base.Entity.getName ()  [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.39.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.39.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.39.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.39.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.39.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.39.1.21 openEntityEditor()

```
void cmas.ontology.Hazard.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.39.1.22 Serialize()

```
JsonObject cmas.ontology.Hazard.Serialize ()
```

Reimplemented from cmas.ontology.base.Entity.

16.39.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.39.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.39.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.39.1.26 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

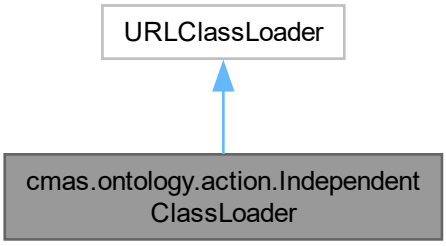
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Hazard.java

16.40 cmas.ontology.action.IndependentClassLoader Class Reference

Inheritance diagram for cmas.ontology.action.IndependentClassLoader:



Static Public Member Functions

- static ClassLoader getInstance ()

16.40.1 Member Function Documentation

16.40.1.1 getInstance()

```
ClassLoader cmas.ontology.action.IndependentClassLoader.getInstance ()  [static]
```

Returns

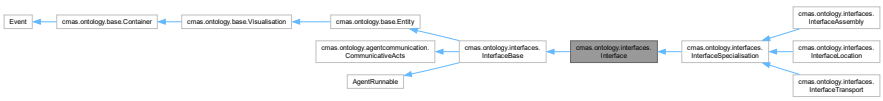
instance

The documentation for this class was generated from the following file:

- IndependentClassLoader.java

16.41 cmas.ontology.interfaces.Interface Class Reference

Inheritance diagram for cmas.ontology.interfaces.Interface:



Public Member Functions

- **Interface** (Interface interf, boolean instance)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- Integer **getParallelCapacity** ()
- void **setParallelCapacity** (Integer parallelCapacity)
- Boolean **getQueuRequests** ()
- void **setQueuRequests** (Boolean queuRequests)

- void openEntityEditor ()
- String **getBookedByName** ()
- Boolean **isBusy** ()
- Boolean **isBooked** ()
- void **setBooked** (CommunicativeActs agent)
- CommunicativeActs **getBooked** ()
- Boolean **isAttachedTo** (InterfaceBase remoteInterface)
- Boolean **isAttached** ()
- String **getAttachedTo** ()
- Attachment **getAttachment** ()
- VariableMatchingType **getVariableMatching** ()
- ACLMessage **getOnGoingCommunication** ()
- void **setVariableMatching** (VariableMatchingType variableRequest)
- void **initInterface** (Agent agent)
- void **AttachTo** (InterfaceBase remote) throws ExceptionRuntime
- void **_AttachTo** (InterfaceBase remote, boolean source) throws ExceptionRuntime
- void **Detach** () throws ExceptionRuntime
- void **_Detach** ()
- void **Transfer** (InterfaceBase remoteInterface) throws ExceptionRuntime
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- void **run** ()
- String **getRequirements** (Entity skill, Agent callingAgent, InterfaceBase callingInterface) throws Exception↩ Runtime
- boolean **book** (InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **book** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **unbook** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface) throws Interrupted↩ Exception
- Agent **getRuntimeAgent** ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- Action **getActionObject** ()
- MessageHandler getMessageHandler ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()

- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Interface Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- boolean addRequestToQueue (ACLMessage message, ArrayList< String > skillList, ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, ExceptionParser, ExceptionExit

Protected Attributes

- Properties **properties**
- Relations **relations**

16.41.1 Member Function Documentation

16.41.1.1 addRequestToQueue()

```
boolean cmas.ontology.interfaces.Interface.addRequestToQueue (
            ACLMessage message,
            ArrayList< String > skillList,
            ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, Exception↩
Parser, ExceptionExit  [protected]
```

Abstract method that should return true if it the request is fully handled and answered in the method or false if the base interface negotiation method should be used.

Parameters

message	
param	
skillList	

Returns

true or false

Reimplemented from cmas.ontology.interfaces.InterfaceBase.
References cmas.ontology.base.Entity.findRelationByName().

16.41.1.2 copy()

```
Entity cmas.ontology.interfaces.Interface.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.
Reimplemented in cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, and cmas.ontology.interfaces.InterfaceTransport.

16.41.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.41.1.4 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.41.1.5 Factory()

```
Interface cmas.ontology.interfaces.Interface.Factory (
            JsonObject job) throws ExceptionType  [static]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
Referenced by cmas.ontology.base.Entity.Factory().

16.41.1.6 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.41.1.7 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.41.1.8 generateUniqueIDs()

`void cmas.ontology.base.Entity.generateUniqueIDs () [inherited]`
Will generate unique IDs for the entity and all relation objects
References setID().

16.41.1.9 getAllRelations()

`Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations () [inherited]`

Returns

a list of all relations categories

16.41.1.10 getCommunicative()

`Entity cmas.ontology.interfaces.InterfaceBase.getCommunicative () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.41.1.11 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.41.1.12 getEntityDescription()

`String cmas.ontology.interfaces.Interface.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.41.1.13 getEntityIconFile()

`String cmas.ontology.interfaces.Interface.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.41.1.14 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.41.1.15 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.41.1.16 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.41.1.17 getMessageHandler()

```
MessageHandler cmas.ontology.interfaces.InterfaceBase.getMessageHandler ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.41.1.18 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.41.1.19 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.41.1.20 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.41.1.21 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.41.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```
Get specific entity type.

Returns

entity type.

16.41.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```
Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.41.1.24 openEntityEditor()

```
void cmas.ontology.interfaces.Interface.openEntityEditor ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.41.1.25 postMessage()

```
void cmas.ontology.interfaces.InterfaceBase.postMessage (
            ACLMessage message)  [inherited]
```
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.41.1.26 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.41.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.41.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.41.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.41.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

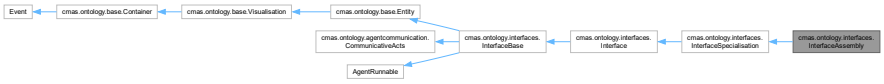
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Interface.java

16.42 cmas.ontology.interfaces.InterfaceAssembly Class Reference

Inheritance diagram for cmas.ontology.interfaces.InterfaceAssembly:



Public Member Functions

- **InterfaceAssembly** (InterfaceSpecialisation interf, boolean instance)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- String **getSpecialisationName** ()
- void **setSpecialisationName** (String specialisationName)
- Integer **getParallelCapacity** ()
- void **setParallelCapacity** (Integer parallelCapacity)
- Boolean **getQueuRequests** ()
- void **setQueuRequests** (Boolean queuRequests)
- void openEntityEditor ()
- String **getBookedByName** ()
- Boolean **isBusy** ()
- Boolean **isBooked** ()
- void **setBooked** (CommunicativeActs agent)
- CommunicativeActs **getBooked** ()

- Boolean **isAttachedTo** (InterfaceBase remoteInterface)
- Boolean **isAttached** ()
- String **getAttachedTo** ()
- Attachment **getAttachment** ()
- VariableMatchingType **getVariableMatching** ()
- ACLMessage **getOnGoingCommunication** ()
- void **setVariableMatching** (VariableMatchingType variableRequest)
- void **initInterface** (Agent agent)
- void **AttachTo** (InterfaceBase remote) throws ExceptionRuntime
- void **_AttachTo** (InterfaceBase remote, boolean source) throws ExceptionRuntime
- void **Detach** () throws ExceptionRuntime
- void **_Detach** ()
- void **Transfer** (InterfaceBase remoteInterface) throws ExceptionRuntime
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- void **run** ()
- String **getRequirements** (Entity skill, Agent callingAgent, InterfaceBase callingInterface) throws Exception↩ Runtime
- boolean **book** (InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **book** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **unbook** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface) throws Interrupted↩ Exception
- Agent **getRuntimeAgent** ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- Action **getActionObject** ()
- MessageHandler getMessageHandler ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)

- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Interface Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- boolean addRequestToQueue (ACLMessage message, ArrayList< String > skillList, ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, ExceptionParser, ExceptionExit

Protected Attributes

- Properties **properties**
- Relations **relations**

16.42.1 Member Function Documentation

16.42.1.1 addRequestToQueue()

```
boolean cmas.ontology.interfaces.Interface.addRequestToQueue (
            ACLMessage message,
            ArrayList< String > skillList,
            ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, Exception↩
Parser, ExceptionExit  [protected], [inherited]
```

Abstract method that should return true if it the request is fully handled and answered in the method or false if the base interface negotiation method should be used.

Parameters

message	
param	
skillList	

Returns

true or false

Reimplemented from cmas.ontology.interfaces.InterfaceBase.
References cmas.ontology.base.Entity.findRelationByName().

16.42.1.2 copy()

```
Entity cmas.ontology.interfaces.InterfaceAssembly.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.interfaces.InterfaceSpecialisation.

16.42.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.42.1.4 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.42.1.5 Factory()

```
Interface cmas.ontology.interfaces.Interface.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	

ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
Referenced by cmas.ontology.base.Entity.Factory().

16.42.1.6 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.42.1.7 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.42.1.8 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.42.1.9 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.42.1.10 getCommunicative()

`Entity cmas.ontology.interfaces.InterfaceBase.getCommunicative () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.42.1.11 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.42.1.12 getEntityDescription()

`String cmas.ontology.interfaces.InterfaceAssembly.getEntityDescription ()`
Reimplemented from cmas.ontology.interfaces.InterfaceSpecialisation.

16.42.1.13 getEntityIconFile()

`String cmas.ontology.interfaces.InterfaceSpecialisation.getEntityIconFile () [inherited]`
Reimplemented from cmas.ontology.interfaces.Interface.

16.42.1.14 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.42.1.15 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.42.1.16 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.42.1.17 getMessageHandler()

`MessageHandler cmas.ontology.interfaces.InterfaceBase.getMessageHandler () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.42.1.18 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.42.1.19 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.42.1.20 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.42.1.21 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.42.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.42.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.42.1.24 openEntityEditor()

```
void cmas.ontology.interfaces.Interface.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.42.1.25 postMessage()

```
void cmas.ontology.interfaces.InterfaceBase.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.42.1.26 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.42.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.42.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.42.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.42.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

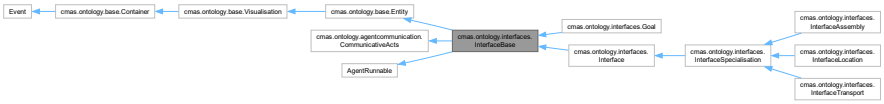
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- InterfaceAssembly.java

16.43 cmas.ontology.interfaces.InterfaceBase Class Reference

Inheritance diagram for cmas.ontology.interfaces.InterfaceBase:



Classes

- enum VariableMatchingType
- enum InterfaceState

Public Member Functions

- **InterfaceBase** (EntityType type)
- **InterfaceBase** (InterfaceBase interf, boolean instance)
- String **getBookedByName** ()
- Boolean **isBusy** ()
- Boolean **isBooked** ()
- void **setBooked** (CommunicativeActs agent)
- CommunicativeActs **getBooked** ()
- Boolean **isAttachedTo** (InterfaceBase remoteInterface)
- Boolean **isAttached** ()
- String **getAttachedTo** ()
- Attachment **getAttachment** ()
- VariableMatchingType **getVariableMatching** ()
- ACLMessage **getOnGoingCommunication** ()
- void **setVariableMatching** (VariableMatchingType variableRequest)
- void **initInterface** (Agent agent)
- void **AttachTo** (InterfaceBase remote) throws ExceptionRuntime

- void **_AttachTo** (InterfaceBase remote, boolean source) throws ExceptionRuntime
- void **Detach** () throws ExceptionRuntime
- void **_Detach** ()
- void **Transfer** (InterfaceBase remoteInterface) throws ExceptionRuntime
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- void **run** ()
- String **getRequirements** (Entity skill, Agent callingAgent, InterfaceBase callingInterface) throws Exception↩ Runtime
- boolean **book** (InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **book** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **unbook** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface) throws Interrupted↩ Exception
- Agent **getRuntimeAgent** ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- Action **getActionObject** ()
- MessageHandler getMessageHandler ()
- abstract Entity copy (boolean instance)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType

- JsonObject Serialize ()
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- abstract void **openEntityEditor** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- abstract boolean addRequestToQueue (ACLMessage message, ArrayList< String > skillList, ContentParameter[] param) throws ExceptionParser, ExceptionVariable, ExceptionType, ExceptionConstant, ExceptionBase, ExceptionRuntime, ExceptionExit

Protected Attributes

- Properties **properties**
- Relations **relations**

16.43.1 Member Function Documentation

16.43.1.1 addRequestToQueue()

```
abstract boolean cmas.ontology.interfaces.InterfaceBase.addRequestToQueue (
            ACLMessage message,
            ArrayList< String > skillList,
            ContentParameter[] param) throws ExceptionParser, ExceptionVariable, ExceptionType,
ExceptionConstant, ExceptionBase, ExceptionRuntime, ExceptionExit  [abstract], [protected]
```

Abstract method that should return true if it the request is fully handled and answered in the method or false if the base interface negotiation method should be used.

Parameters

message	
param	
skillList	

Returns

true or false

Reimplemented in cmas.ontology.interfaces.Goal, and cmas.ontology.interfaces.Interface.

16.43.1.2 copy()

```
abstract Entity cmas.ontology.base.Entity.copy (
            boolean instance)  [abstract], [inherited]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented in cmas.ontology.agents.DeploymentAgent, cmas.ontology.agents.Material, cmas.ontology.agents.Part, cmas.ontology.agents.Resource, cmas.ontology.base.Reference, cmas.ontology.batch.Batch, cmas.ontology.Hazard, cmas.ontology.interfaces.Goal, cmas.ontology.interfaces.Interface, cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, cmas.ontology.interfaces.InterfaceTrans cmas.ontology.model.ModelSettings, cmas.ontology.model.ModelState, cmas.ontology.Note, cmas.ontology.processplan.ProcessPlar cmas.ontology.processplan.ProcessPlanJava, cmas.ontology.processplan.ProcessPlanStructuredText, cmas.ontology.processplan.Sk cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.VariableInteger, cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, cmas.ontology.variables.VariableString, cmas.ontology.workflow.Requirement, and cmas.ontology.workflow.WorkFlowLog.
References Entity().

16.43.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.43.1.4 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.43.1.5 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.43.1.6 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.43.1.7 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.43.1.8 generateUniqueIDs()

`void cmas.ontology.base.Entity.generateUniqueIDs () [inherited]`
Will generate unique IDs for the entity and all relation objects
References setID().

16.43.1.9 getAllRelations()

`Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations () [inherited]`

Returns

a list of all relations categories

16.43.1.10 getCommunicative()

`Entity cmas.ontology.interfaces.InterfaceBase.getCommunicative ()`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.43.1.11 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.43.1.12 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.43.1.13 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.43.1.14 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.43.1.15 getMessageHandler()

`MessageHandler cmas.ontology.interfaces.InterfaceBase.getMessageHandler ()`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.43.1.16 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.43.1.17 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.43.1.18 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.43.1.19 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.43.1.20 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.43.1.21 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.43.1.22 postMessage()

```
void cmas.ontology.interfaces.InterfaceBase.postMessage (
            ACLMessage message)
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.43.1.23 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.43.1.24 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.43.1.25 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.43.1.26 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.43.1.27 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

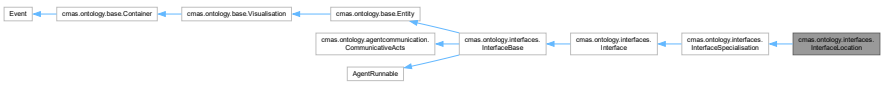
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- InterfaceBase.java

16.44 cmas.ontology.interfaces.InterfaceLocation Class Reference

Inheritance diagram for cmas.ontology.interfaces.InterfaceLocation:



Public Member Functions

- **InterfaceLocation** (InterfaceSpecialisation interf, boolean instance)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- String **getSpecialisationName** ()
- void **setSpecialisationName** (String specialisationName)
- Integer **getParallelCapacity** ()
- void **setParallelCapacity** (Integer parallelCapacity)
- Boolean **getQueuRequests** ()
- void **setQueuRequests** (Boolean queuRequests)
- void openEntityEditor ()
- String **getBookedByName** ()
- Boolean **isBusy** ()
- Boolean **isBooked** ()
- void **setBooked** (CommunicativeActs agent)
- CommunicativeActs **getBooked** ()
- Boolean **isAttachedTo** (InterfaceBase remoteInterface)
- Boolean **isAttached** ()
- String **getAttachedTo** ()
- Attachment **getAttachment** ()
- VariableMatchingType **getVariableMatching** ()
- ACLMessage **getOnGoingCommunication** ()
- void **setVariableMatching** (VariableMatchingType variableRequest)
- void **initInterface** (Agent agent)
- void **AttachTo** (InterfaceBase remote) throws ExceptionRuntime
- void **_AttachTo** (InterfaceBase remote, boolean source) throws ExceptionRuntime
- void **Detach** () throws ExceptionRuntime
- void **_Detach** ()

- void **Transfer** (InterfaceBase remoteInterface) throws ExceptionRuntime
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- void **run** ()
- String **getRequirements** (Entity skill, Agent callingAgent, InterfaceBase callingInterface) throws Exception↩ Runtime
- boolean **book** (InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **book** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **unbook** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface) throws Interrupted↩ Exception
- Agent **getRuntimeAgent** ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- Action **getActionObject** ()
- MessageHandler getMessageHandler ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()

- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Interface Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- boolean addRequestToQueue (ACLMessage message, ArrayList< String > skillList, ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, ExceptionParser, ExceptionExit

Protected Attributes

- Properties **properties**
- Relations **relations**

16.44.1 Member Function Documentation

16.44.1.1 addRequestToQueue()

```
boolean cmas.ontology.interfaces.Interface.addRequestToQueue (
            ACLMessage message,
            ArrayList< String > skillList,
            ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, Exception↩
Parser, ExceptionExit  [protected], [inherited]
```

Abstract method that should return true if it the request is fully handled and answered in the method or false if the base interface negotiation method should be used.

Parameters

message	
param	
skillList	

Returns

true or false

Reimplemented from cmas.ontology.interfaces.InterfaceBase.
References cmas.ontology.base.Entity.findRelationByName().

16.44.1.2 copy()

```
Entity cmas.ontology.interfaces.InterfaceLocation.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.interfaces.InterfaceSpecialisation.

16.44.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.44.1.4 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.44.1.5 Factory()

```
Interface cmas.ontology.interfaces.Interface.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
Referenced by cmas.ontology.base.Entity.Factory().

16.44.1.6 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.44.1.7 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.44.1.8 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.44.1.9 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.44.1.10 getCommunicative()

```
Entity cmas.ontology.interfaces.InterfaceBase.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.44.1.11 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.44.1.12 getEntityDescription()

`String cmas.ontology.interfaces.InterfaceLocation.getEntityDescription ()`
Reimplemented from cmas.ontology.interfaces.InterfaceSpecialisation.

16.44.1.13 getEntityIconFile()

`String cmas.ontology.interfaces.InterfaceSpecialisation.getEntityIconFile ()  [inherited]`
Reimplemented from cmas.ontology.interfaces.Interface.

16.44.1.14 getID()

`String cmas.ontology.base.Entity.getID ()  [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.44.1.15 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID ()  [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.44.1.16 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.44.1.17 getMessageHandler()

`MessageHandler cmas.ontology.interfaces.InterfaceBase.getMessageHandler ()  [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.44.1.18 getName()

`String cmas.ontology.base.Entity.getName ()  [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.44.1.19 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.44.1.20 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.44.1.21 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.44.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.44.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.44.1.24 openEntityEditor()

```
void cmas.ontology.interfaces.Interface.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.44.1.25 postMessage()

```
void cmas.ontology.interfaces.InterfaceBase.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.44.1.26 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.44.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.44.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.44.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.44.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

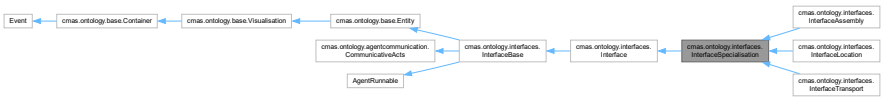
Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

The documentation for this class was generated from the following file:

- InterfaceLocation.java

16.45 cmas.ontology.interfaces.InterfaceSpecialisation Class Reference

Inheritance diagram for cmas.ontology.interfaces.InterfaceSpecialisation:



Public Member Functions

- **InterfaceSpecialisation** (InterfaceSpecialisation interf, boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- String **getSpecialisationName** ()
- void **setSpecialisationName** (String specialisationName)
- Entity copy (boolean instance)
- Integer **getParallelCapacity** ()
- void **setParallelCapacity** (Integer parallelCapacity)
- Boolean **getQueuRequests** ()
- void **setQueuRequests** (Boolean queuRequests)
- void openEntityEditor ()
- String **getBookedByName** ()
- Boolean **isBusy** ()
- Boolean **isBooked** ()
- void **setBooked** (CommunicativeActs agent)
- CommunicativeActs **getBooked** ()
- Boolean **isAttachedTo** (InterfaceBase remoteInterface)
- Boolean **isAttached** ()
- String **getAttachedTo** ()
- Attachment **getAttachment** ()
- VariableMatchingType **getVariableMatching** ()
- ACLMessage **getOnGoingCommunication** ()
- void **setVariableMatching** (VariableMatchingType variableRequest)
- void **initInterface** (Agent agent)
- void **AttachTo** (InterfaceBase remote) throws ExceptionRuntime
- void **_AttachTo** (InterfaceBase remote, boolean source) throws ExceptionRuntime
- void **Detach** () throws ExceptionRuntime
- void **_Detach** ()
- void **Transfer** (InterfaceBase remoteInterface) throws ExceptionRuntime
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- void **run** ()
- String **getRequirements** (Entity skill, Agent callingAgent, InterfaceBase callingInterface) throws Exception↩ Runtime
- boolean **book** (InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **book** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException

- boolean **unbook** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface) throws Interrupted↩ Exception
- Agent **getRuntimeAgent** ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- Action **getActionObject** ()
- MessageHandler getMessageHandler ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)

- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Interface Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- boolean addRequestToQueue (ACLMessage message, ArrayList< String > skillList, ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, ExceptionParser, ExceptionExit

Protected Attributes

- Properties **properties**
- Relations **relations**

16.45.1 Member Function Documentation

16.45.1.1 addRequestToQueue()

```
boolean cmas.ontology.interfaces.Interface.addRequestToQueue (
            ACLMessage message,
            ArrayList< String > skillList,
            ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, Exception↩
Parser, ExceptionExit  [protected], [inherited]
```

Abstract method that should return true if it the request is fully handled and answered in the method or false if the base interface negotiation method should be used.

Parameters

message	
param	
skillList	

Returns

true or false

Reimplemented from cmas.ontology.interfaces.InterfaceBase.
References cmas.ontology.base.Entity.findRelationByName().

16.45.1.2 copy()

```
Entity cmas.ontology.interfaces.InterfaceSpecialisation.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.interfaces.Interface.
Reimplemented in cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, and cmas.ontology.interfaces.InterfaceTransport.

16.45.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.45.1.4 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.45.1.5 Factory()

```
Interface cmas.ontology.interfaces.Interface.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```
Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
Referenced by cmas.ontology.base.Entity.Factory().

16.45.1.6 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```
Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.45.1.7 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.45.1.8 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.45.1.9 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.45.1.10 getCommunicative()

```
Entity cmas.ontology.interfaces.InterfaceBase.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.45.1.11 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.45.1.12 getEntityDescription()

```
String cmas.ontology.interfaces.InterfaceSpecialisation.getEntityDescription ()
```

Reimplemented from cmas.ontology.interfaces.Interface.

16.45.1.13 getEntityIconFile()

```
String cmas.ontology.interfaces.InterfaceSpecialisation.getEntityIconFile ()
```
Reimplemented from cmas.ontology.interfaces.Interface.

16.45.1.14 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```
Get the ID of the entity.

Returns

a string with current ID.

16.45.1.15 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.45.1.16 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.45.1.17 getMessageHandler()

```
MessageHandler cmas.ontology.interfaces.InterfaceBase.getMessageHandler ()  [inherited]
```
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.45.1.18 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.45.1.19 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.45.1.20 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.45.1.21 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.45.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.45.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.45.1.24 openEntityEditor()

```
void cmas.ontology.interfaces.Interface.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.45.1.25 postMessage()

```
void cmas.ontology.interfaces.InterfaceBase.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.45.1.26 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.45.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.45.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.45.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.45.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

The documentation for this class was generated from the following file:

- InterfaceSpecialisation.java

16.46 cmas.ontology.interfaces.InterfaceBase.InterfaceState Enum Reference

Public Attributes

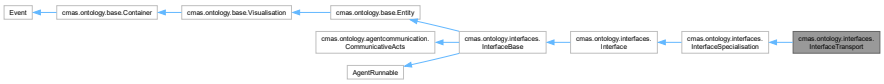
- **IDLE**
- **BUSY**

The documentation for this enum was generated from the following file:

- InterfaceBase.java

16.47 cmas.ontology.interfaces.InterfaceTransport Class Reference

Inheritance diagram for cmas.ontology.interfaces.InterfaceTransport:



Public Member Functions

- **InterfaceTransport** (InterfaceSpecialisation interf, boolean instance)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- String **getSpecialisationName** ()
- void **setSpecialisationName** (String specialisationName)
- Integer **getParallelCapacity** ()
- void **setParallelCapacity** (Integer parallelCapacity)
- Boolean **getQueuRequests** ()
- void **setQueuRequests** (Boolean queuRequests)
- void openEntityEditor ()
- String **getBookedByName** ()
- Boolean **isBusy** ()
- Boolean **isBooked** ()
- void **setBooked** (CommunicativeActs agent)
- CommunicativeActs **getBooked** ()
- Boolean **isAttachedTo** (InterfaceBase remoteInterface)
- Boolean **isAttached** ()
- String **getAttachedTo** ()
- Attachment **getAttachment** ()
- VariableMatchingType **getVariableMatching** ()
- ACLMessage **getOnGoingCommunication** ()
- void **setVariableMatching** (VariableMatchingType variableRequest)
- void **initInterface** (Agent agent)
- void **AttachTo** (InterfaceBase remote) throws ExceptionRuntime
- void **_AttachTo** (InterfaceBase remote, boolean source) throws ExceptionRuntime
- void **Detach** () throws ExceptionRuntime
- void **_Detach** ()

- void **Transfer** (InterfaceBase remoteInterface) throws ExceptionRuntime
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- void **run** ()
- String **getRequirements** (Entity skill, Agent callingAgent, InterfaceBase callingInterface) throws Exception↩ Runtime
- boolean **book** (InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **book** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface, ArrayList< String > desiredSkills, ArrayList< String > desiredVariables) throws InterruptedException
- boolean **unbook** (CommunicativeActs delegateAnswer, InterfaceBase remoteInterface) throws Interrupted↩ Exception
- Agent **getRuntimeAgent** ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- Action **getActionObject** ()
- MessageHandler getMessageHandler ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()

- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Interface Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- boolean addRequestToQueue (ACLMessage message, ArrayList< String > skillList, ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, ExceptionParser, ExceptionExit

Protected Attributes

- Properties **properties**
- Relations **relations**

16.47.1 Member Function Documentation

16.47.1.1 addRequestToQueue()

```
boolean cmas.ontology.interfaces.Interface.addRequestToQueue (
            ACLMessage message,
            ArrayList< String > skillList,
            ContentParameter[] param) throws ExceptionBase, ExceptionRuntime, Exception↩
Parser, ExceptionExit  [protected], [inherited]
```

Abstract method that should return true if it the request is fully handled and answered in the method or false if the base interface negotiation method should be used.

Parameters

message	
param	
skillList	

Returns

true or false

Reimplemented from cmas.ontology.interfaces.InterfaceBase.
References cmas.ontology.base.Entity.findRelationByName().

16.47.1.2 copy()

```
Entity cmas.ontology.interfaces.InterfaceTransport.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.interfaces.InterfaceSpecialisation.

16.47.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.47.1.4 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.47.1.5 Factory()

```
Interface cmas.ontology.interfaces.Interface.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
Referenced by cmas.ontology.base.Entity.Factory().

16.47.1.6 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.47.1.7 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.47.1.8 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.47.1.9 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.47.1.10 getCommunicative()

```
Entity cmas.ontology.interfaces.InterfaceBase.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.47.1.11 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.47.1.12 getEntityDescription()

```
String cmas.ontology.interfaces.InterfaceTransport.getEntityDescription ()
```
Reimplemented from cmas.ontology.interfaces.InterfaceSpecialisation.

16.47.1.13 getEntityIconFile()

```
String cmas.ontology.interfaces.InterfaceSpecialisation.getEntityIconFile ()  [inherited]
```
Reimplemented from cmas.ontology.interfaces.Interface.

16.47.1.14 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```
Get the ID of the entity.

Returns

a string with current ID.

16.47.1.15 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.47.1.16 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.47.1.17 getMessageHandler()

```
MessageHandler cmas.ontology.interfaces.InterfaceBase.getMessageHandler ()  [inherited]
```
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.47.1.18 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.47.1.19 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.47.1.20 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.47.1.21 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.47.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.47.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.47.1.24 openEntityEditor()

```
void cmas.ontology.interfaces.Interface.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.47.1.25 postMessage()

```
void cmas.ontology.interfaces.InterfaceBase.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.47.1.26 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.47.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.47.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.47.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.47.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

The documentation for this class was generated from the following file:

- InterfaceTransport.java

16.48 cmas.ontology.variables.Iterator Class Reference

Public Member Functions

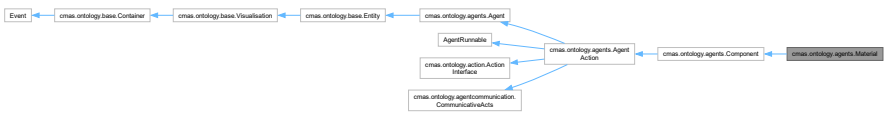
- **Iterator** (Variable variable, int index)
- boolean **hasNext** ()
- Object **getNext** () throws ExceptionVariable, ExceptionType, ExceptionConstant
- int **increaseIndex** ()
- int **getIndex** ()
- void **setIndex** (int index)
- Variable **getVariable** ()
- void **setVariable** (Variable variable)
- Object **getProperty** ()
- void **setProperty** (Object property)
- Variable **getIteratorObject** ()

The documentation for this class was generated from the following file:

- Iterator.java

16.49 cmas.ontology.agents.Material Class Reference

Inheritance diagram for cmas.ontology.agents.Material:



Public Member Functions

- **Material** (Material entity, boolean instance)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- synchronized Action **getActionObject** ()
- synchronized void **setState** (AgentState newState)
- synchronized AgentState **getState** ()
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- synchronized void **setPaused** (boolean value)
- synchronized boolean **getPaused** ()
- synchronized void postMessage (ACLMessage message)
- Entity getCommunicative ()
- void **onInit** ()
- void **onDepot** ()
- void **onDeploy** ()

- void **onRun** ()
- void **Running** ()
- void **onUndeploy** ()
- void **onFinished** ()
- void **onFailed** ()
- void **onStop** ()
- void **onMessage** (ACLMessage message)
- void **run** ()
- MessageHandler getMessageHandler ()
- void gotoLine (ExceptionStructuredText e)
- void stopEditor (ExceptionStructuredText e)
- void **compile** () throws ExceptionParameters, Exception
- String **getPhysicalID** ()
- void **setPhysicalID** (String physicalID)
- String **getAgentSpecificType** ()
- void **setAgentSpecificType** (String agentSpecificType)
- AgentBaseTypes **getAgentBaseType** ()
- void **addAbstractInterface** (AbstractInterface abstractInterface, String Name)
- void **clearAbstractInterfaces** ()
- AbstractInterface **getAbstractInterface** (String Name)
- RuntimeTree **getRuntimeTree** ()
- void **setRuntimeTree** (RuntimeTree runtimeTree)
- Domain **getRuntimeDomain** ()
- void **setRuntimeDomain** (Domain runtimeDomain)
- LinkedBlockingQueue< WorkFlowLog > **getHistory** ()
- void **setHistory** (LinkedBlockingQueue< WorkFlowLog > p)
- void **addToHistory** (WorkFlowLog log)
- Boolean **getAutoDeploy** ()
- void **setAutoDeploy** (Boolean autoDeploy)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)

- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static AgentBaseTypes **getAgentType** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Attributes

- Action **action**
- MessageHandler **messageHandler**
- Properties **properties**
- Relations **relations**

16.49.1 Member Function Documentation

16.49.1.1 copy()

```
Entity cmas.ontology.agents.Material.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.49.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.49.1.3 DeSerialize()

```
void cmas.ontology.agents.AgentAction.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.agents.Agent.

16.49.1.4 Factory()

```
Entity cmas.ontology.agents.Agent.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.49.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.49.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.49.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.49.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.49.1.9 getCommunicative()

```
Entity cmas.ontology.agents.AgentAction.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.49.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.49.1.11 getEntityDescription()

```
String cmas.ontology.agents.Material.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.49.1.12 getEntityIconFile()

```
String cmas.ontology.agents.Material.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.49.1.13 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.49.1.14 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.49.1.15 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.49.1.16 getMessageHandler()

`MessageHandler cmas.ontology.agents.AgentAction.getMessageHandler () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.49.1.17 getName()

`String cmas.ontology.base.Entity.getName () [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.49.1.18 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity) [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.49.1.19 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.49.1.20 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.49.1.21 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.49.1.22 gotoLine()

```
void cmas.ontology.agents.AgentAction.gotoLine (
            ExceptionStructuredText e) [inherited]
```

Implements cmas.ontology.action.ActionInterface.

16.49.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.49.1.24 openEntityEditor()

```
void cmas.ontology.agents.Material.openEntityEditor ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.49.1.25 postMessage()

```
synchronized void cmas.ontology.agents.AgentAction.postMessage (
            ACLMessage message)  [inherited]
```
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.49.1.26 Serialize()

```
JsonObject cmas.ontology.agents.AgentAction.Serialize ()  [inherited]
```
Reimplemented from cmas.ontology.agents.Agent.

16.49.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.49.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.49.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```
Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.49.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```
set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.49.1.31 stopEditor()

```
void cmas.ontology.agents.AgentAction.stopEditor (
            ExceptionStructuredText e)  [inherited]
```

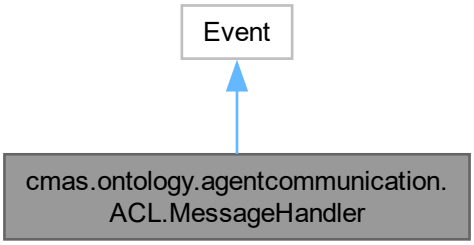
Implements cmas.ontology.action.ActionInterface.

The documentation for this class was generated from the following file:

- Material.java

16.50 cmas.ontology.agentcommunication.ACL.MessageHandler Class Reference

Inheritance diagram for cmas.ontology.agentcommunication.ACL.MessageHandler:



Public Member Functions

- void **addMessage** (ACLMessage newMessage)
- ACLMessage **getNextMessage** ()
- ACLMessage **getNextMessage** (long timeout) throws InterruptedException
- boolean **isEmpty** ()

Static Package Attributes

- static final long **INTERRUPT_RESOLUTION** =500L

The documentation for this class was generated from the following file:

- MessageHandler.java

16.51 cmas.ontology.adapter.Modbus.ModbusClient.MODBUS_TYPE Enum Reference

Public Attributes

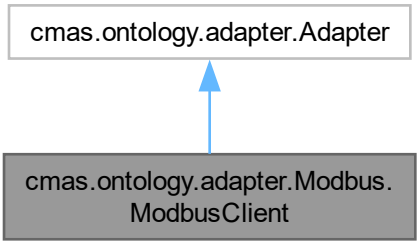
- **TCP**
- **UDP**
- **SERIAL**

The documentation for this enum was generated from the following file:

- ModbusClient.java

16.52 cmas.ontology.adapter.Modbus.ModbusClient Class Reference

Inheritance diagram for cmas.ontology.adapter.Modbus.ModbusClient:



Classes

- enum MODBUS_TYPE
- enum ADDRESS_TYPE

Public Member Functions

- **ModbusClient** (String Source)
- void **setSource** (String Source)
- void **connectToServer** ()
- SerialParameters **getSerialParameters** ()
- String **getServerHost** ()
- void **setServerHost** (String serverHost)
- int **getServerPort** ()
- void **setServerPort** (int serverPort)
- MODBUS_TYPE **getServerType** ()
- void **setServerType** (MODBUS_TYPE serverType)
- String ReadData (String Address, String Name, Object Value) throws Exception
- void WriteData (String Address, String Name, Boolean bValue) throws Exception
- void WriteData (String Address, String Name, String sValue) throws Exception
- void WriteData (String Address, String Name, Integer iValue) throws Exception
- void WriteData (String Address, String Name, Double dValue) throws Exception
- boolean close ()
- void setDebug (boolean bValue)
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)

Static Package Attributes

- static final int **DEFAULT_ID** =1

16.52.1 Member Function Documentation

16.52.1.1 close()

```
boolean cmas.ontology.adapter.Modbus.ModbusClient.close ()
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.2 createNew()

```
Adapter cmas.ontology.adapter.Modbus.ModbusClient.createNew (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.3 getDefaultValue()

```
String cmas.ontology.adapter.Modbus.ModbusClient.getDefaultValue ()
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.4 getDescription()

```
String cmas.ontology.adapter.Modbus.ModbusClient.getDescription ()
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.5 isValid()

```
boolean cmas.ontology.adapter.Modbus.ModbusClient.isValid (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.6 ReadData()

```
String cmas.ontology.adapter.Modbus.ModbusClient.ReadData (
            String Address,
            String Name,
            Object Value) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.7 setDebug()

```
void cmas.ontology.adapter.Modbus.ModbusClient.setDebug (
            boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.8 WriteData() [1/4]

```
void cmas.ontology.adapter.Modbus.ModbusClient.WriteData (
            String Address,
            String Name,
            Boolean bValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.9 WriteData() [2/4]

```
void cmas.ontology.adapter.Modbus.ModbusClient.WriteData (
            String Address,
            String Name,
            Double dValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.10 WriteData() [3/4]

```
void cmas.ontology.adapter.Modbus.ModbusClient.WriteData (
            String Address,
            String Name,
            Integer iValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.52.1.11 WriteData() [4/4]

```
void cmas.ontology.adapter.Modbus.ModbusClient.WriteData (
            String Address,
            String Name,
            String sValue) throws Exception
```

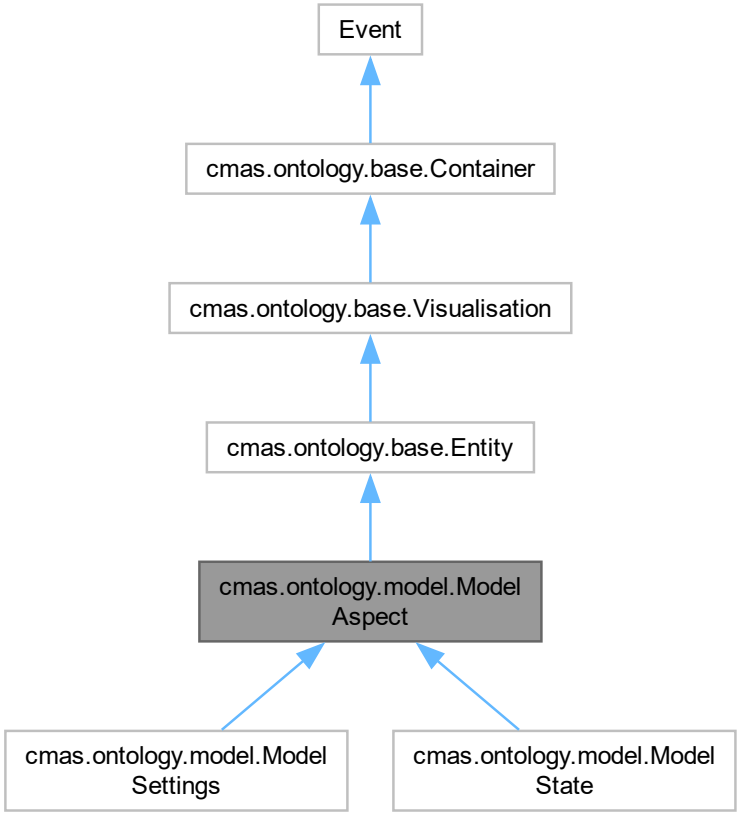
Implements cmas.ontology.adapter.Adapter.

The documentation for this class was generated from the following file:

- ModbusClient.java

16.53 cmas.ontology.model.ModelAspect Class Reference

Inheritance diagram for cmas.ontology.model.ModelAspect:



Public Member Functions

- JsonArray **SerializePropertySet** ()
- Collection< Property<?> > **DeSerializePropertySet** (String value)
- Collection< Property<?> > **getPropertySetItems** ()
- Properties **getPropertySet** ()
- void **setPropertySet** (Collection< Property<?> > list)
- JsonArray **getPropertySetAsJSONArray** ()
- JsonObject **getPropertySetAsJSONObject** ()
- void openEntityEditor ()

- abstract Entity copy (boolean instance)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- **ModelAspect** (EntityType type)
- **ModelAspect** (ModelAspect aspect, boolean instance)

Protected Attributes

- Properties **properties**
- Relations **relations**

16.53.1 Member Function Documentation

16.53.1.1 copy()

```
abstract Entity cmas.ontology.base.Entity.copy (
            boolean instance)  [abstract], [inherited]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented in cmas.ontology.agents.DeploymentAgent, cmas.ontology.agents.Material, cmas.ontology.agents.Part, cmas.ontology.agents.Resource, cmas.ontology.base.Reference, cmas.ontology.batch.Batch, cmas.ontology.Hazard, cmas.ontology.interfaces.Goal, cmas.ontology.interfaces.Interface, cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, cmas.ontology.interfaces.InterfaceTrans cmas.ontology.model.ModelSettings, cmas.ontology.model.ModelState, cmas.ontology.Note, cmas.ontology.processplan.ProcessPlar cmas.ontology.processplan.ProcessPlanJava, cmas.ontology.processplan.ProcessPlanStructuredText, cmas.ontology.processplan.Sk cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.VariableInteger, cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, cmas.ontology.variables.VariableString, cmas.ontology.workflow.Requirement, and cmas.ontology.workflow.WorkFlowLog.
References Entity().

16.53.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.53.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.53.1.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.53.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.53.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.53.1.7 generateUniqueIDs()

`void cmas.ontology.base.Entity.generateUniqueIDs () [inherited]`
Will generate unique IDs for the entity and all relation objects
References setID().

16.53.1.8 getAllRelations()

`Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations () [inherited]`

Returns

a list of all relations categories

16.53.1.9 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.53.1.10 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.53.1.11 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.53.1.12 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.53.1.13 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.53.1.14 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.53.1.15 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.53.1.16 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.53.1.17 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.53.1.18 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.53.1.19 openEntityEditor()

```
void cmas.ontology.model.ModelAspect.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.53.1.20 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.53.1.21 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.53.1.22 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.53.1.23 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.53.1.24 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

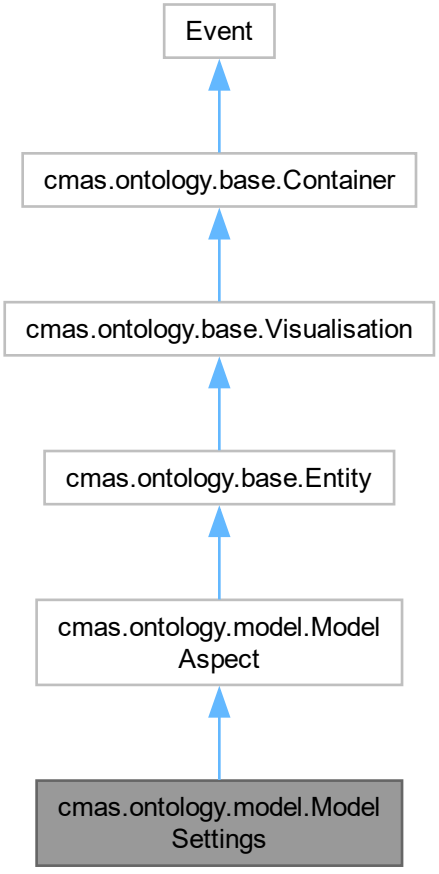
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- ModelAspect.java

16.54 cmas.ontology.model.ModelSettings Class Reference

Inheritance diagram for cmas.ontology.model.ModelSettings:



Public Member Functions

- **ModelSettings** (ModelSettings settings, boolean instance)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- JsonArray **SerializePropertySet** ()
- Collection< Property<?> > **DeSerializePropertySet** (String value)
- Collection< Property<?> > **getPropertySetItems** ()
- Properties **getPropertySet** ()
- void **setPropertySet** (Collection< Property<?> > list)
- JsonArray **getPropertySetAsJSONArray** ()
- JsonObject **getPropertySetAsJSONObject** ()
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)

- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.54.1 Member Function Documentation

16.54.1.1 copy()

```
Entity cmas.ontology.model.ModelSettings.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.54.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.54.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.54.1.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.54.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.54.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.54.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.54.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.54.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.54.1.10 getEntityDescription()

```
String cmas.ontology.model.ModelSettings.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.54.1.11 getEntityIconFile()

```
String cmas.ontology.model.ModelSettings.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.54.1.12 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.54.1.13 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.54.1.14 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.54.1.15 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.54.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.54.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.54.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.54.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.54.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.54.1.21 openEntityEditor()

```
void cmas.ontology.model.ModelAspect.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.54.1.22 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.54.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.54.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.54.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.54.1.26 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

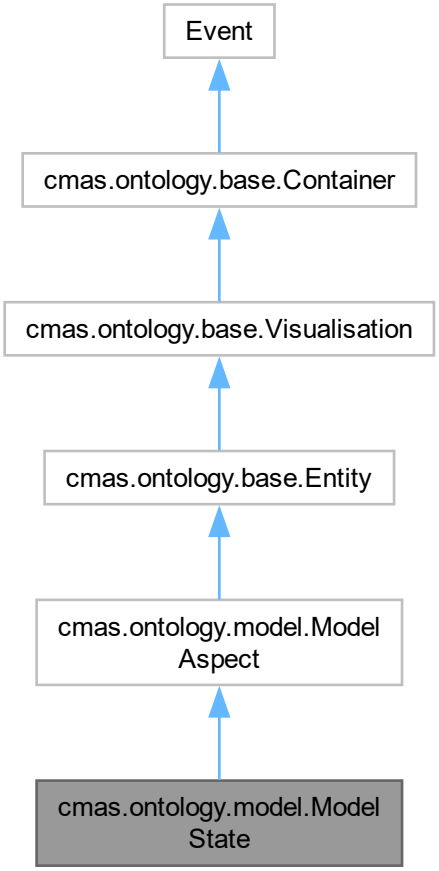
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- ModelSettings.java

16.55 cmas.ontology.model.ModelState Class Reference

Inheritance diagram for cmas.ontology.model.ModelState:



Public Member Functions

- **ModelState** (ModelState settings, boolean instance)
- String **getStateAsJSON** ()
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- JsonArray **SerializePropertySet** ()
- Collection< Property<?> > **DeSerializePropertySet** (String value)
- Collection< Property<?> > **getPropertySetItems** ()
- Properties **getPropertySet** ()
- void **setPropertySet** (Collection< Property<?> > list)
- JsonArray **getPropertySetAsJSONArray** ()
- JsonObject **getPropertySetAsJSONObject** ()
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()

- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.55.1 Member Function Documentation

16.55.1.1 copy()

```
Entity cmas.ontology.model.ModelState.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.55.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.55.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.55.1.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.55.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.55.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.55.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.55.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.55.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.55.1.10 getEntityDescription()

```
String cmas.ontology.model.ModelState.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.55.1.11 getEntityIconFile()

```
String cmas.ontology.model.ModelState.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.55.1.12 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.55.1.13 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.55.1.14 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.55.1.15 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.55.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.55.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.55.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.55.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.55.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.55.1.21 openEntityEditor()

```
void cmas.ontology.model.ModelAspect.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.55.1.22 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.55.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.55.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.55.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.55.1.26 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

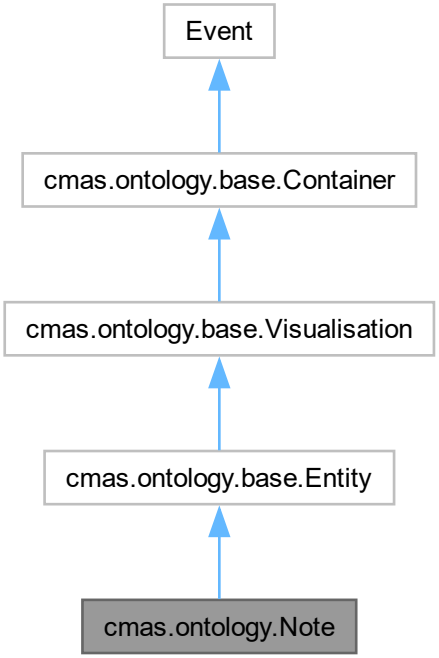
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- ModelState.java

16.56 cmas.ontology.Note Class Reference

Inheritance diagram for cmas.ontology.Note:



Public Member Functions

- **Note** (Note note, boolean instance)
- Entity copy (boolean instance)
- Boolean **getShowInModel** ()
- void **setShowInModel** (Boolean showInModel)
- Boolean **getShowOnLoad** ()
- void **setShowOnLoad** (Boolean showOnLoad)

- Boolean **getShowBeforeRun** ()
- void **setShowBeforeRun** (Boolean showBeforeRun)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.56.1 Member Function Documentation

16.56.1.1 copy()

```
Entity cmas.ontology.Note.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().

16.56.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.56.1.3 DeSerialize()

```
void cmas.ontology.Note.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.base.Entity.

16.56.1.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.56.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.56.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.56.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.56.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.56.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.56.1.10 getEntityDescription()

```
String cmas.ontology.Note.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.56.1.11 getEntityIconFile()

```
String cmas.ontology.Note.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.56.1.12 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.56.1.13 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.56.1.14 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.56.1.15 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.56.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.56.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.56.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.56.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.56.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.56.1.21 openEntityEditor()

```
void cmas.ontology.Note.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.56.1.22 Serialize()

```
JsonObject cmas.ontology.Note.Serialize ()
```

Reimplemented from cmas.ontology.base.Entity.

16.56.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.56.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.56.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.56.1.26 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Note.java

16.57 cmas.ontology.base.Ontology Class Reference

Classes

- class OntologyObject
- class OntologyRelation

Static Public Member Functions

- static void **Read** (String fileName)

The documentation for this class was generated from the following file:

- Ontology.java

16.58 cmas.ontology.base.Ontology.OntologyObject Class Reference

Package Attributes

- String **Name**
- String **Extends**
- String **Category**
- String **Description**
- List<?> **Property**
- List<?> **OntologyRelation**

The documentation for this class was generated from the following file:

- Ontology.java

16.59 cmas.ontology.base.Ontology.OntologyRelation Class Reference

Package Attributes

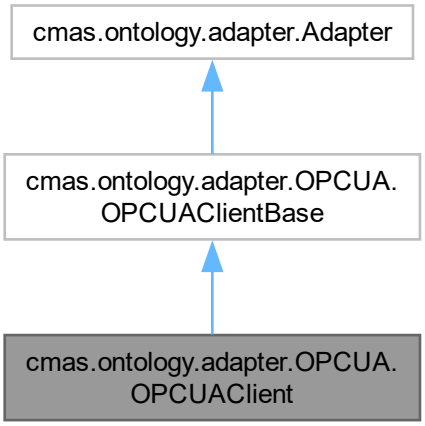
- String **Name**
- List< String > **AcceptedObjects**

The documentation for this class was generated from the following file:

- Ontology.java

16.60 cmas.ontology.adapter.OPCUA.OPCUAClient Class Reference

Inheritance diagram for cmas.ontology.adapter.OPCUA.OPCUAClient:



Public Member Functions

- **OPCUAClient** (String Source)
- void **connectToServer** (String Address, String Name)
- String ReadData (String Address, String Name, Object Value)
- void WriteData (String Address, String Name, String Value)
- void WriteData (String Address, String Name, Double Value)
- void WriteData (String Address, String Name, Integer Value)
- void WriteData (String Address, String Name, Boolean Value)
- boolean close ()
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)
- OpcUaClient **createClient** () throws InterruptedException, ExecutionException, UaException
- String **browseNode** (OpcUaClient client, NodeId browseRoot, String NodeName, String OutPutId)
- boolean **getAllNodes** (TreeItem< OPCUANode > rootNode, OpcUaClient client, NodeId browseRoot)
- void **setSource** (String Source)
- String **getServerEndpointUrl** ()
- String getServerName ()
- void setServerName (String ServerName)
- String getServerPort ()
- void setServerPort (String serverPort)
- String getServerHost ()
- void setServerHost (String serverHost)
- boolean **getDebug** ()
- void setDebug (boolean bValue)

16.60.1 Member Function Documentation

16.60.1.1 close()

```
boolean cmas.ontology.adapter.OPCUA.OPCUAClient.close ()
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.2 createNew()

```
Adapter cmas.ontology.adapter.OPCUA.OPCUAClient.createNew (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.3 getDefaultValue()

```
String cmas.ontology.adapter.OPCUA.OPCUAClient.getDefaultValue ()
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.4 getDescription()

```
String cmas.ontology.adapter.OPCUA.OPCUAClient.getDescription ()
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.5 getServerHost()

```
String cmas.ontology.adapter.OPCUA.OPCUAClientBase.getServerHost ()  [inherited]
```

Returns

the serverHost

16.60.1.6 getServerName()

```
String cmas.ontology.adapter.OPCUA.OPCUAClientBase.getServerName ()  [inherited]
```

Returns

the serverName

16.60.1.7 getServerPort()

```
String cmas.ontology.adapter.OPCUA.OPCUAClientBase.getServerPort ()  [inherited]
```

Returns

the serverPort

16.60.1.8 isValid()

```
boolean cmas.ontology.adapter.OPCUA.OPCUAClient.isValid (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.9 ReadData()

```
String cmas.ontology.adapter.OPCUA.OPCUAClient.ReadData (
            String Address,
            String Name,
            Object Value)
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.10 setDebug()

```
void cmas.ontology.adapter.OPCUA.OPCUAClientBase.setDebug (
            boolean bValue)  [inherited]
```

Parameters

bValue	turn adapter debug on or off

Implements cmas.ontology.adapter.Adapter.

16.60.1.11 setServerHost()

```
void cmas.ontology.adapter.OPCUA.OPCUAClientBase.setServerHost (
            String serverHost)  [inherited]
```

Parameters

serverHost	the serverHost to set

16.60.1.12 setServerName()

```
void cmas.ontology.adapter.OPCUA.OPCUAClientBase.setServerName (
            String ServerName)  [inherited]
```

Parameters

ServerName	the server name to set

16.60.1.13 setServerPort()

```
void cmas.ontology.adapter.OPCUA.OPCUAClientBase.setServerPort (
            String serverPort)  [inherited]
```

Parameters

serverPort	the serverPort to set

16.60.1.14 WriteData() [1/4]

```
void cmas.ontology.adapter.OPCUA.OPCUAClient.WriteData (
            String Address,
            String Name,
            Boolean Value)
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.15 WriteData() [2/4]

```
void cmas.ontology.adapter.OPCUA.OPCUAClient.WriteData (
            String Address,
            String Name,
            Double Value)
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.16 WriteData() [3/4]

```
void cmas.ontology.adapter.OPCUA.OPCUAClient.WriteData (
            String Address,
            String Name,
            Integer Value)
```

Implements cmas.ontology.adapter.Adapter.

16.60.1.17 WriteData() [4/4]

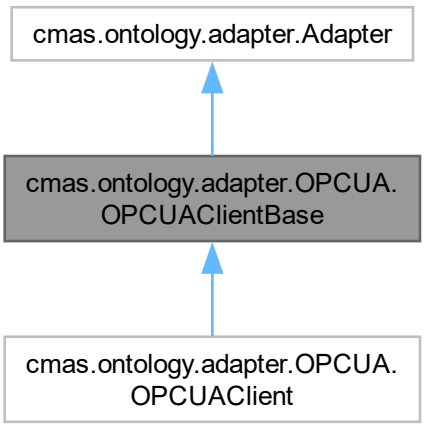
```
void cmas.ontology.adapter.OPCUA.OPCUAClient.WriteData (
            String Address,
            String Name,
            String Value)
```

Implements cmas.ontology.adapter.Adapter.
The documentation for this class was generated from the following file:

- OPCUAClient.java

16.61 cmas.ontology.adapter.OPCUA.OPCUAClientBase Class Reference

Inheritance diagram for cmas.ontology.adapter.OPCUA.OPCUAClientBase:



Public Member Functions

- OpcUaClient **createClient** () throws InterruptedException, ExecutionException, UaException
- String **browseNode** (OpcUaClient client, NodeId browseRoot, String NodeName, String OutPutId)
- boolean **getAllNodes** (TreeItem< OPCUANode > rootNode, OpcUaClient client, NodeId browseRoot)
- void **setSource** (String Source)
- String **getServerEndpointUrl** ()
- String getServerName ()
- void setServerName (String ServerName)
- String getServerPort ()
- void setServerPort (String serverPort)
- String getServerHost ()
- void setServerHost (String serverHost)

- boolean **getDebug** ()
- void setDebug (boolean bValue)
- abstract boolean **close** ()
- abstract String ReadData (String Address, String Name, Object Value) throws Exception
- abstract void WriteData (String Address, String Name, Boolean bValue) throws Exception
- abstract void **WriteData** (String Address, String Name, String sValue) throws Exception
- abstract void **WriteData** (String Address, String Name, Integer iValue) throws Exception
- abstract void **WriteData** (String Address, String Name, Double dValue) throws Exception
- abstract String **getDefaultValue** ()
- abstract String **getDescription** ()
- abstract boolean **isValid** (String Source)
- abstract Adapter **createNew** (String Source)

16.61.1 Member Function Documentation

16.61.1.1 getServerHost()

```
String cmas.ontology.adapter.OPCUA.OPCUAClientBase.getServerHost ()
```

Returns

the serverHost

16.61.1.2 getServerName()

```
String cmas.ontology.adapter.OPCUA.OPCUAClientBase.getServerName ()
```

Returns

the serverName

16.61.1.3 getServerPort()

```
String cmas.ontology.adapter.OPCUA.OPCUAClientBase.getServerPort ()
```

Returns

the serverPort

16.61.1.4 ReadData()

```
abstract String cmas.ontology.adapter.Adapter.ReadData (
            String Address,
            String Name,
            Object Value) throws Exception  [abstract], [inherited]
```

Implemented in cmas.ontology.adapter.SQL.SQLClient.

16.61.1.5 setDebug()

```
void cmas.ontology.adapter.OPCUA.OPCUAClientBase.setDebug (
            boolean bValue)
```

Parameters

bValue	turn adapter debug on or off

Implements cmas.ontology.adapter.Adapter.

16.61.1.6 setServerHost()

```
void cmas.ontology.adapter.OPCUA.OPCUAClientBase.setServerHost (
            String serverHost)
```

Parameters

serverHost	the serverHost to set

16.61.1.7 setServerName()

```
void cmas.ontology.adapter.OPCUA.OPCUAClientBase.setServerName (
            String ServerName)
```

Parameters

ServerName	the server name to set

16.61.1.8 setServerPort()

```
void cmas.ontology.adapter.OPCUA.OPCUAClientBase.setServerPort (
            String serverPort)
```

Parameters

serverPort	the serverPort to set

16.61.1.9 WriteData()

```
abstract void cmas.ontology.adapter.Adapter.WriteData (
            String Address,
            String Name,
            Boolean bValue) throws Exception  [abstract], [inherited]
```

Implemented in cmas.ontology.adapter.SQL.SQLClient.
The documentation for this class was generated from the following file:

- OPCUAClientBase.java

16.62 cmas.ontology.adapter.OPCUA.OPCUANode Class Reference

Public Member Functions

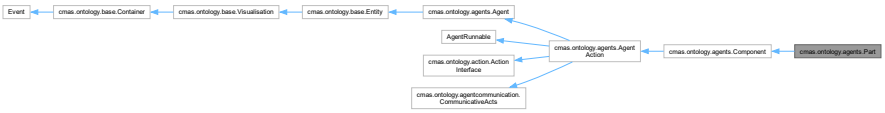
- **OPCUANode** (Adapter adapter, ReferenceDescription rd)
- String **getName** ()
- String **getDescription** ()
- String **getId** ()
- String **getDataType** ()
- String **getValue** ()

The documentation for this class was generated from the following file:

- OPCUANode.java

16.63 cmas.ontology.agents.Part Class Reference

Inheritance diagram for cmas.ontology.agents.Part:



Public Member Functions

- **Part** (Part entity, boolean instance)
- void onUndeploy ()
- void onFailed ()
- void onStop ()
- void onFinished ()
- void onRun ()
- Entity copy (boolean instance)
- BlockSequential **getSFC** ()
- void **setSFC** (BlockSequential sfc)
- void DeSerialize (JsonObject job) throws ExceptionType
- Integer **getInstances** ()
- void **setInstances** (Integer counter)
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- synchronized Action **getActionObject** ()
- synchronized void **setState** (AgentState newState)
- synchronized AgentState **getState** ()
- JsonObject Serialize ()
- synchronized void **setPaused** (boolean value)
- synchronized boolean **getPaused** ()
- synchronized void postMessage (ACLMessage message)
- Entity getCommunicative ()
- void **onInit** ()
- void **onDepot** ()
- void **onDeploy** ()
- void **Running** ()
- void **onMessage** (ACLMessage message)
- void **run** ()
- MessageHandler getMessageHandler ()
- void gotoLine (ExceptionStructuredText e)
- void stopEditor (ExceptionStructuredText e)
- void **compile** () throws ExceptionParameters, Exception
- String **getPhysicalID** ()
- void **setPhysicalID** (String physicalID)
- String **getAgentSpecificType** ()
- void **setAgentSpecificType** (String agentSpecificType)
- AgentBaseTypes **getAgentBaseType** ()
- void **addAbstractInterface** (AbstractInterface abstractInterface, String Name)
- void **clearAbstractInterfaces** ()
- AbstractInterface **getAbstractInterface** (String Name)

- RuntimeTree **getRuntimeTree** ()
- void **setRuntimeTree** (RuntimeTree runtimeTree)
- Domain **getRuntimeDomain** ()
- void **setRuntimeDomain** (Domain runtimeDomain)
- LinkedBlockingQueue< WorkFlowLog > **getHistory** ()
- void **setHistory** (LinkedBlockingQueue< WorkFlowLog > p)
- void **addToHistory** (WorkFlowLog log)
- Boolean **getAutoDeploy** ()
- void **setAutoDeploy** (Boolean autoDeploy)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)

- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static AgentBaseTypes **getAgentType** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Attributes

- Action **action**
- MessageHandler **messageHandler**
- Properties **properties**
- Relations **relations**

16.63.1 Member Function Documentation

16.63.1.1 copy()

```
Entity cmas.ontology.agents.Part.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.63.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.63.1.3 DeSerialize()

```
void cmas.ontology.agents.Part.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.agents.AgentAction.

16.63.1.4 Factory()

```
Entity cmas.ontology.agents.Agent.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.63.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.63.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.63.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.63.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.63.1.9 getCommunicative()

```
Entity cmas.ontology.agents.AgentAction.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.63.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.63.1.11 getEntityDescription()

```
String cmas.ontology.agents.Part.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.63.1.12 getEntityIconFile()

```
String cmas.ontology.agents.Part.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.63.1.13 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.63.1.14 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.63.1.15 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.63.1.16 getMessageHandler()

```
MessageHandler cmas.ontology.agents.AgentAction.getMessageHandler ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.63.1.17 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.63.1.18 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.63.1.19 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.63.1.20 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.63.1.21 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```
Get specific entity type.

Returns

entity type.

16.63.1.22 gotoLine()

```
void cmas.ontology.agents.AgentAction.gotoLine (
            ExceptionStructuredText e)  [inherited]
```
Implements cmas.ontology.action.ActionInterface.

16.63.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```
Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.63.1.24 onFailed()

```
void cmas.ontology.agents.Part.onFailed ()
```
Reimplemented from cmas.ontology.agents.AgentAction.

16.63.1.25 onFinished()

```
void cmas.ontology.agents.Part.onFinished ()
```
Reimplemented from cmas.ontology.agents.AgentAction.

16.63.1.26 onRun()

```
void cmas.ontology.agents.Part.onRun ()
```
Reimplemented from cmas.ontology.agents.AgentAction.

16.63.1.27 onStop()

```
void cmas.ontology.agents.Part.onStop ()
```
Reimplemented from cmas.ontology.agents.AgentAction.

16.63.1.28 onUndeploy()

```
void cmas.ontology.agents.Part.onUndeploy ()
```
Reimplemented from cmas.ontology.agents.AgentAction.

16.63.1.29 openEntityEditor()

```
void cmas.ontology.agents.Part.openEntityEditor ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.63.1.30 postMessage()

```
synchronized void cmas.ontology.agents.AgentAction.postMessage (
            ACLMessage message)  [inherited]
```
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.63.1.31 Serialize()

```
JsonObject cmas.ontology.agents.AgentAction.Serialize ()  [inherited]
```
Reimplemented from cmas.ontology.agents.Agent.

16.63.1.32 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.63.1.33 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.63.1.34 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```
Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.63.1.35 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```
set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.63.1.36 stopEditor()

```
void cmas.ontology.agents.AgentAction.stopEditor (
            ExceptionStructuredText e)  [inherited]
```

Implements cmas.ontology.action.ActionInterface.

The documentation for this class was generated from the following file:

- Part.java

16.64 cmas.ontology.agentcommunication.ACL.ACLMessage.↩ Performative Enum Reference

Public Attributes

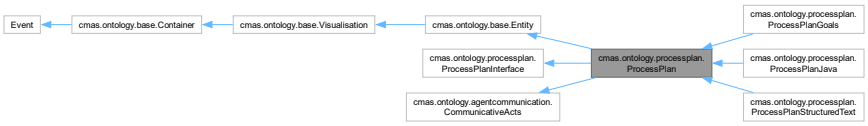
- **REQUEST**
- **QUERY**
- **INFORM**
- **CANCEL**
- **PROGRESS**
- **REFUSE**
- **AGREE**
- **REPLAN**
- **FAILURE**
- **DONE**
- **RESULT**

The documentation for this enum was generated from the following file:

- ACLMessage.java

16.65 cmas.ontology.processplan.ProcessPlan Class Reference

Inheritance diagram for cmas.ontology.processplan.ProcessPlan:



Classes

- enum ProcessPlanType
- enum ProcessPlanReturnValue

Public Member Functions

- **ProcessPlan** (ProcessPlanType type)
- **ProcessPlan** (ProcessPlan processplan, boolean instance)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- Integer **getCost** ()
- void **setCost** (Integer cost)
- ProcessPlanType **getProcessPlanType** ()
- WorkFlowTypes **getPostAction** ()

- void **setPostAction** (WorkFlowTypes action)
- void **setProcessPlanType** (ProcessPlanType processPlanType)
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- MessageHandler getMessageHandler ()
- String **getTargetType** ()
- void **setTargetType** (String targetType)
- abstract Entity copy (boolean instance)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- abstract void **openEntityEditor** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()

- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- abstract void **resetAllAbstractInterfaces** () throws ExceptionParser, ExceptionRuntime
- abstract int **onNegotiate** (Agent agent, Entity target, boolean planning, ContentParameter[ ] abstractValues)
- abstract boolean **onOverlap** (Agent agent, Entity target, boolean planning, InterfaceBase remoteInterface) throws ExceptionParser, ExceptionRuntime
- abstract void run (Agent agent, Entity target, ACLMessage message, ContentParameter[ ] abstractValues)
- abstract ProcessPlanReturnValue **execute** (Agent agent, Entity target, ACLMessage aclMessage, ContentParameter[ ] abstractValues) throws ExceptionParameters, ExceptionRuntime, ExceptionBase, ExceptionParser
- abstract void **abort** ()
- default void **loadClasspath** (URLClassLoader classLoader, String classpath)
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static ProcessPlanType **getProcessPlanType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.65.1 Member Function Documentation

16.65.1.1 copy()

```
abstract Entity cmas.ontology.base.Entity.copy (
            boolean instance)  [abstract], [inherited]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented in cmas.ontology.agents.DeploymentAgent, cmas.ontology.agents.Material, cmas.ontology.agents.Part, cmas.ontology.agents.Resource, cmas.ontology.base.Reference, cmas.ontology.batch.Batch, cmas.ontology.Hazard, cmas.ontology.interfaces.Goal, cmas.ontology.interfaces.Interface, cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, cmas.ontology.interfaces.InterfaceTrans cmas.ontology.model.ModelSettings, cmas.ontology.model.ModelState, cmas.ontology.Note, cmas.ontology.processplan.ProcessPlar cmas.ontology.processplan.ProcessPlanJava, cmas.ontology.processplan.ProcessPlanStructuredText, cmas.ontology.processplan.Sk cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.VariableInteger, cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, cmas.ontology.variables.VariableString, cmas.ontology.workflow.Requirement, and cmas.ontology.workflow.WorkFlowLog.
References Entity().

16.65.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.65.1.3 DeSerialize()

```
void cmas.ontology.processplan.ProcessPlan.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.base.Entity.

16.65.1.4 Factory()

```
Entity cmas.ontology.processplan.ProcessPlan.Factory (
            JsonObject job) throws ExceptionType  [static]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.65.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.65.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.65.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.65.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.65.1.9 getCommunicative()

```
Entity cmas.ontology.processplan.ProcessPlan.getCommunicative ()
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.65.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.65.1.11 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.65.1.12 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.65.1.13 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.65.1.14 getMessageHandler()

`MessageHandler cmas.ontology.processplan.ProcessPlan.getMessageHandler ()`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.65.1.15 getName()

`String cmas.ontology.base.Entity.getName () [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.65.1.16 getRelationName()

`String cmas.ontology.base.Entity.getRelationName (`
`Entity entity) [inherited]`
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.65.1.17 getRelations()

`LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations () [inherited]`

Returns

a HashMap of all relation categories and their linked names.

16.65.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.65.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.65.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.65.1.21 postMessage()

```
void cmas.ontology.processplan.ProcessPlan.postMessage (
            ACLMessage message)
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.65.1.22 run()

```
abstract void cmas.ontology.processplan.ProcessPlanInterface.run (
            Agent agent,
            Entity target,
            ACLMessage message,
            ContentParameter[] abstractValues)  [abstract], [inherited]
```

Implemented in cmas.ontology.processplan.ProcessPlanStructuredText.

16.65.1.23 Serialize()

```
JsonObject cmas.ontology.processplan.ProcessPlan.Serialize ()
```
Reimplemented from cmas.ontology.base.Entity.

16.65.1.24 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.65.1.25 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.65.1.26 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```
Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.65.1.27 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```
set entity name.

Parameters

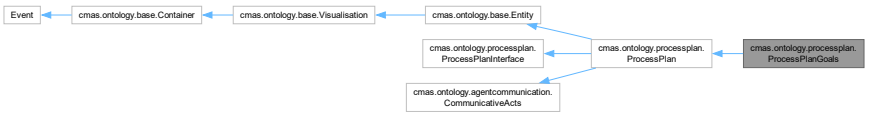
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- ProcessPlan.java

16.66 cmas.ontology.processplan.ProcessPlanGoals Class Reference

Inheritance diagram for cmas.ontology.processplan.ProcessPlanGoals:



Public Member Functions

- **ProcessPlanGoals** (ArrayList< RelationObject > relation)
- **ProcessPlanGoals** (ProcessPlanGoals processplan, ArrayList< RelationObject > relation, boolean instance)
- void run (Agent agent, Entity target, ACLMessage message, ContentParameter[ ] abstractValues)
- ProcessPlanReturnValue execute (Agent agent, Entity target, ACLMessage aclMessage, ContentParameter[ ] abstractValues) throws ExceptionRuntime, ExceptionBase, ExceptionParser
- void abort ()
- Entity copy (boolean instance)
- Entity **copy** (ArrayList< RelationObject > relation, boolean instance)
- DebugResult **BreakPointCallBack** (StatementBase globalBlock, StatementBase localBlock, StatementBase statement, BreakPointBase breakpoint)
- BlockSequential **getSFC** ()
- void **setSFC** (BlockSequential sfc)
- void **setOwner** (Entity owner)
- Entity **getOwner** ()
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- int onNegotiate (Agent agent, Entity target, boolean planning, ContentParameter[ ] abstractValues)
- boolean onOverlap (Agent agent, Entity target, boolean planning, InterfaceBase remoteInterface)
- void resetAllAbstractInterfaces ()
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- Integer **getCost** ()
- void **setCost** (Integer cost)
- ProcessPlanType **getProcessPlanType** ()
- WorkFlowTypes **getPostAction** ()
- void **setPostAction** (WorkFlowTypes action)
- void **setProcessPlanType** (ProcessPlanType processPlanType)
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- MessageHandler getMessageHandler ()
- String **getTargetType** ()
- void **setTargetType** (String targetType)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()

- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default void **loadClasspath** (URLClassLoader classLoader, String classpath)
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static ProcessPlanType **getProcessPlanType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.66.1 Member Function Documentation

16.66.1.1 abort()

```
void cmas.ontology.processplan.ProcessPlanGoals.abort ()
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.66.1.2 copy()

```
Entity cmas.ontology.processplan.ProcessPlanGoals.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.66.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.66.1.4 DeSerialize()

```
void cmas.ontology.processplan.ProcessPlanGoals.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.processplan.ProcessPlan.

16.66.1.5 execute()

```
ProcessPlanReturnValue cmas.ontology.processplan.ProcessPlanGoals.execute (
            Agent agent,
            Entity target,
            ACLMessage aclMessage,
            ContentParameter[] abstractValues) throws ExceptionRuntime, ExceptionBase, Exception↩
Parser
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.66.1.6 Factory()

```
Entity cmas.ontology.processplan.ProcessPlan.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.66.1.7 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.66.1.8 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.66.1.9 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.66.1.10 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.66.1.11 getCommunicative()

```
Entity cmas.ontology.processplan.ProcessPlan.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.66.1.12 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.66.1.13 getEntityDescription()

```
String cmas.ontology.processplan.ProcessPlanGoals.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.66.1.14 getEntityIconFile()

```
String cmas.ontology.processplan.ProcessPlanGoals.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.66.1.15 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.66.1.16 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.66.1.17 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.66.1.18 getMessageHandler()

```
MessageHandler cmas.ontology.processplan.ProcessPlan.getMessageHandler ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.66.1.19 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.66.1.20 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.66.1.21 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.66.1.22 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.66.1.23 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.66.1.24 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.66.1.25 onNegotiate()

```
int cmas.ontology.processplan.ProcessPlanGoals.onNegotiate (
            Agent agent,
            Entity target,
            boolean planning,
            ContentParameter[] abstractValues)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.66.1.26 onOverlap()

```
boolean cmas.ontology.processplan.ProcessPlanGoals.onOverlap (
            Agent agent,
            Entity target,
            boolean planning,
            InterfaceBase remoteInterface)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.66.1.27 openEntityEditor()

```
void cmas.ontology.processplan.ProcessPlanGoals.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.66.1.28 postMessage()

```
void cmas.ontology.processplan.ProcessPlan.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.66.1.29 resetAllAbstractInterfaces()

```
void cmas.ontology.processplan.ProcessPlanGoals.resetAllAbstractInterfaces ()
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.66.1.30 run()

```
void cmas.ontology.processplan.ProcessPlanGoals.run (
            Agent agent,
            Entity target,
            ACLMessage message,
            ContentParameter[] abstractValues)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.66.1.31 Serialize()

```
JsonObject cmas.ontology.processplan.ProcessPlanGoals.Serialize ()
```

Reimplemented from cmas.ontology.processplan.ProcessPlan.

16.66.1.32 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.66.1.33 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.66.1.34 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.66.1.35 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- ProcessPlanGoals.java

16.67 cmas.ontology.processplan.ProcessPlanInterface Interface Reference

Inheritance diagram for cmas.ontology.processplan.ProcessPlanInterface:



Public Member Functions

- abstract void **resetAllAbstractInterfaces** () throws ExceptionParser, ExceptionRuntime
- abstract int **onNegotiate** (Agent agent, Entity target, boolean planning, ContentParameter[ ] abstractValues)
- abstract boolean **onOverlap** (Agent agent, Entity target, boolean planning, InterfaceBase remoteInterface) throws ExceptionParser, ExceptionRuntime
- abstract void run (Agent agent, Entity target, ACLMessage message, ContentParameter[ ] abstractValues)
- abstract ProcessPlanReturnValue **execute** (Agent agent, Entity target, ACLMessage aclMessage, ContentParameter[ ] abstractValues) throws ExceptionParameters, ExceptionRuntime, ExceptionBase, ExceptionParser
- abstract void **abort** ()
- default void **loadClasspath** (URLClassLoader classLoader, String classpath)

16.67.1 Member Function Documentation

16.67.1.1 run()

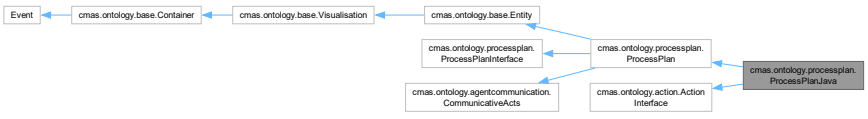
```
abstract void cmas.ontology.processplan.ProcessPlanInterface.run (
            Agent agent,
            Entity target,
            ACLMessage message,
            ContentParameter[] abstractValues)  [abstract]
```

Implemented in cmas.ontology.processplan.ProcessPlanStructuredText.
The documentation for this interface was generated from the following file:

- ProcessPlanInterface.java

16.68 cmas.ontology.processplan.ProcessPlanJava Class Reference

Inheritance diagram for cmas.ontology.processplan.ProcessPlanJava:



Public Member Functions

- **ProcessPlanJava** (ProcessPlanJava processplan, boolean instance)
- synchronized void **setFileName** (String fileName)
- synchronized String **getFileName** ()
- void resetAllAbstractInterfaces () throws ExceptionParser, ExceptionRuntime
- int onNegotiate (Agent agent, Entity target, boolean planning, ContentParameter[ ] abstractValues)
- boolean onOverlap (Agent agent, Entity target, boolean planning, InterfaceBase remoteInterface) throws ExceptionParser, ExceptionRuntime
- void run (Agent agent, Entity target, ACLMessage message, ContentParameter[ ] abstractValues)
- ProcessPlanReturnValue execute (Agent agent, Entity target, ACLMessage aclMessage, ContentParameter[ ] abstractValues)
- ProcessPlanInterface **getObject** () throws ExceptionParser, ExceptionRuntime
- void abort ()
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- String **getDependencies** ()
- void **setDependencies** (String dependencies)
- void gotoLine (ExceptionStructuredText e)
- void stopEditor (ExceptionStructuredText e)
- void openEntityEditor ()
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- Integer **getCost** ()
- void **setCost** (Integer cost)
- ProcessPlanType **getProcessPlanType** ()
- WorkFlowTypes **getPostAction** ()
- void **setPostAction** (WorkFlowTypes action)
- void **setProcessPlanType** (ProcessPlanType processPlanType)
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- MessageHandler getMessageHandler ()
- String **getTargetType** ()
- void **setTargetType** (String targetType)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()

- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default void **loadClasspath** (URLClassLoader classLoader, String classpath)
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static ProcessPlanType **getProcessPlanType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Static Public Attributes

- static final boolean **ENABLED** = false

Protected Attributes

- Properties **properties**
- Relations **relations**

16.68.1 Member Function Documentation

16.68.1.1 abort()

```
void cmas.ontology.processplan.ProcessPlanJava.abort ()
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.68.1.2 copy()

```
Entity cmas.ontology.processplan.ProcessPlanJava.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.68.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.68.1.4 DeSerialize()

```
void cmas.ontology.processplan.ProcessPlan.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Entity.

16.68.1.5 execute()

```
ProcessPlanReturnValue cmas.ontology.processplan.ProcessPlanJava.execute (
            Agent agent,
            Entity target,
            ACLMessage aclMessage,
            ContentParameter[] abstractValues)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.68.1.6 Factory()

```
Entity cmas.ontology.processplan.ProcessPlan.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.68.1.7 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.68.1.8 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.68.1.9 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.68.1.10 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.68.1.11 getCommunicative()

```
Entity cmas.ontology.processplan.ProcessPlan.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.68.1.12 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.68.1.13 getEntityDescription()

`String cmas.ontology.processplan.ProcessPlanJava.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.68.1.14 getEntityIconFile()

`String cmas.ontology.processplan.ProcessPlanJava.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.68.1.15 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.68.1.16 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.68.1.17 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.68.1.18 getMessageHandler()

`MessageHandler cmas.ontology.processplan.ProcessPlan.getMessageHandler () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.68.1.19 getName()

`String cmas.ontology.base.Entity.getName () [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.68.1.20 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.68.1.21 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.68.1.22 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.68.1.23 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.68.1.24 gotoLine()

```
void cmas.ontology.processplan.ProcessPlanJava.gotoLine (
            ExceptionStructuredText e)
```

Implements cmas.ontology.action.ActionInterface.

16.68.1.25 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.68.1.26 onNegotiate()

```
int cmas.ontology.processplan.ProcessPlanJava.onNegotiate (
            Agent agent,
            Entity target,
            boolean planning,
            ContentParameter[] abstractValues)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.68.1.27 onOverlap()

```
boolean cmas.ontology.processplan.ProcessPlanJava.onOverlap (
            Agent agent,
            Entity target,
            boolean planning,
            InterfaceBase remoteInterface) throws ExceptionParser, ExceptionRuntime
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.68.1.28 openEntityEditor()

```
void cmas.ontology.processplan.ProcessPlanJava.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.68.1.29 postMessage()

```
void cmas.ontology.processplan.ProcessPlan.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.68.1.30 resetAllAbstractInterfaces()

```
void cmas.ontology.processplan.ProcessPlanJava.resetAllAbstractInterfaces () throws Exception↩
Parser, ExceptionRuntime
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.68.1.31 run()

```
void cmas.ontology.processplan.ProcessPlanJava.run (
            Agent agent,
            Entity target,
```

```
            ACLMessage message,
            ContentParameter[] abstractValues)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.68.1.32 Serialize()

```
JsonObject cmas.ontology.processplan.ProcessPlan.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Entity.

16.68.1.33 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.68.1.34 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.68.1.35 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.68.1.36 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.68.1.37 stopEditor()

```
void cmas.ontology.processplan.ProcessPlanJava.stopEditor (
            ExceptionStructuredText e)
```

Implements cmas.ontology.action.ActionInterface.
The documentation for this class was generated from the following file:

- ProcessPlanJava.java

16.69 cmas.ontology.processplan.ProcessPlan.ProcessPlanReturnValue Enum Reference

Public Member Functions

- int **toInt** ()

Static Public Member Functions

- static ProcessPlanReturnValue **fromInt** (int val)

Public Attributes

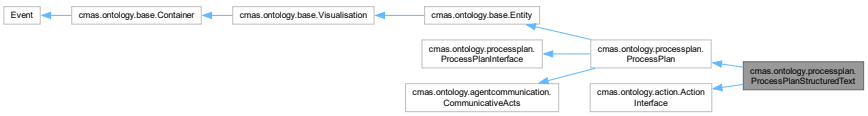
- **RUNNING** =(1)
- **DONE** =(2)
- **FAILED** =(3)
- **INTERRUPTED** =(4)
- **REPLAN** =(5)
- **REDO** =(6)

The documentation for this enum was generated from the following file:

- ProcessPlan.java

16.70 cmas.ontology.processplan.ProcessPlanStructuredText Class Reference

Inheritance diagram for cmas.ontology.processplan.ProcessPlanStructuredText:



Public Member Functions

- **ProcessPlanStructuredText** (ProcessPlanStructuredText processplan, boolean instance)
- Entity copy (boolean instance)
- int onNegotiate (Agent agent, Entity target, boolean planning, ContentParameter[ ] abstractValues)
- boolean onOverlap (Agent agent, Entity target, boolean planning, InterfaceBase remoteInterface)
- void run (Agent agent, Entity target, ACLMessage message, ContentParameter[ ] abstractValues)
- ProcessPlanReturnValue execute (Agent agent, Entity target, ACLMessage aclMessage, ContentParameter[ ] abstractValues)
- void gotoLine (ExceptionStructuredText e)
- void stopEditor (ExceptionStructuredText e)
- synchronized boolean **getAbort** ()
- synchronized void abort ()
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- synchronized String **getActions** ()
- void **compile** (Agent agent, Entity target) throws ExceptionRuntime, ExceptionParser, ExceptionBase, ExceptionExit

- void **compile** (Agent agent, Entity target, ArrayList< Variable > variables, ArrayList< AbstractInterface > abstractInterfaces) throws ExceptionRuntime, ExceptionParser, ExceptionBase, ExceptionExit
- void **addVariable** (VariableObject variableObject)
- synchronized void **setActions** (String actions)
- LinkedHashMap< String, AbstractInterface > **getAllAbstractInterfaces** ()
- void resetAllAbstractInterfaces ()
- String getEntityDescription ()
- String getEntityIconFile ()
- void **setConstructorCalled** (boolean b)
- void openEntityEditor ()
- Integer **getCost** ()
- void **setCost** (Integer cost)
- ProcessPlanType **getProcessPlanType** ()
- WorkFlowTypes **getPostAction** ()
- void **setPostAction** (WorkFlowTypes action)
- void **setProcessPlanType** (ProcessPlanType processPlanType)
- Entity getCommunicative ()
- void postMessage (ACLMessage message)
- MessageHandler getMessageHandler ()
- String **getTargetType** ()
- void **setTargetType** (String targetType)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()

- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default void **loadClasspath** (URLClassLoader classLoader, String classpath)
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static ProcessPlanType **getProcessPlanType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.70.1 Member Function Documentation

16.70.1.1 abort()

```
synchronized void cmas.ontology.processplan.ProcessPlanStructuredText.abort ()
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.70.1.2 copy()

```
Entity cmas.ontology.processplan.ProcessPlanStructuredText.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.70.1.3 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.70.1.4 DeSerialize()

```
void cmas.ontology.processplan.ProcessPlanStructuredText.DeSerialize (
            JsonObject job) throws ExceptionType
```

Reimplemented from cmas.ontology.processplan.ProcessPlan.

16.70.1.5 execute()

```
ProcessPlanReturnValue cmas.ontology.processplan.ProcessPlanStructuredText.execute (
            Agent agent,
            Entity target,
            ACLMessage aclMessage,
            ContentParameter[] abstractValues)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.70.1.6 Factory()

```
Entity cmas.ontology.processplan.ProcessPlan.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.70.1.7 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.70.1.8 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.70.1.9 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.70.1.10 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.70.1.11 getCommunicative()

```
Entity cmas.ontology.processplan.ProcessPlan.getCommunicative ()  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.70.1.12 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.70.1.13 getEntityDescription()

```
String cmas.ontology.processplan.ProcessPlanStructuredText.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.70.1.14 getEntityIconFile()

```
String cmas.ontology.processplan.ProcessPlanStructuredText.getEntityIconFile ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.70.1.15 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```
Get the ID of the entity.

Returns

a string with current ID.

16.70.1.16 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.70.1.17 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.70.1.18 getMessageHandler()

```
MessageHandler cmas.ontology.processplan.ProcessPlan.getMessageHandler ()  [inherited]
```
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.70.1.19 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.70.1.20 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.70.1.21 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.70.1.22 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.70.1.23 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.70.1.24 gotoLine()

```
void cmas.ontology.processplan.ProcessPlanStructuredText.gotoLine (
            ExceptionStructuredText e)
```

Implements cmas.ontology.action.ActionInterface.

16.70.1.25 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.70.1.26 onNegotiate()

```
int cmas.ontology.processplan.ProcessPlanStructuredText.onNegotiate (
            Agent agent,
            Entity target,
            boolean planning,
            ContentParameter[] abstractValues)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.70.1.27 onOverlap()

```
boolean cmas.ontology.processplan.ProcessPlanStructuredText.onOverlap (
            Agent agent,
            Entity target,
            boolean planning,
            InterfaceBase remoteInterface)
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.70.1.28 openEntityEditor()

```
void cmas.ontology.processplan.ProcessPlanStructuredText.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.70.1.29 postMessage()

```
void cmas.ontology.processplan.ProcessPlan.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.70.1.30 resetAllAbstractInterfaces()

```
void cmas.ontology.processplan.ProcessPlanStructuredText.resetAllAbstractInterfaces ()
```

Implements cmas.ontology.processplan.ProcessPlanInterface.

16.70.1.31 run()

```
void cmas.ontology.processplan.ProcessPlanStructuredText.run (
            Agent agent,
            Entity target,
            ACLMessage message,
            ContentParameter[] abstractValues)
```

Will run the process plan in a separate thread and does not wait for it to finish. The result will be sent to the remote caller and a DONE signal is sent back to our own local interface.
Implements cmas.ontology.processplan.ProcessPlanInterface.
References cmas.ontology.base.Entity.findRelationByNameIgnoreCase(), cmas.ontology.base.Entity.getName(), and run().
Referenced by run().

16.70.1.32 Serialize()

```
JsonObject cmas.ontology.processplan.ProcessPlanStructuredText.Serialize ()
```

Reimplemented from cmas.ontology.processplan.ProcessPlan.

16.70.1.33 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.70.1.34 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.70.1.35 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.70.1.36 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.70.1.37 stopEditor()

```
void cmas.ontology.processplan.ProcessPlanStructuredText.stopEditor (
            ExceptionStructuredText e)
```

Implements cmas.ontology.action.ActionInterface.
The documentation for this class was generated from the following file:

- ProcessPlanStructuredText.java

16.71 cmas.ontology.processplan.ProcessPlan.ProcessPlanType Enum Reference

Public Attributes

- **STRUCTUREDTEXT**

- **SFC_GOALS**
- **JAVA**

The documentation for this enum was generated from the following file:

- ProcessPlan.java

16.72 cmas.ontology.base.Properties Class Reference

Public Member Functions

- **Properties** (Properties properties, boolean instance)
- Properties **copy** (boolean instance)
- Collection< Property<?> > getPropertiesList ()
- void **delete** ()
- void **clear** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- boolean **propertyExist** (Property<?> p)
- void **addWithUniqueKey** (String key, Property<?> Item)
- void **add** (Property<?> Item)
- void **addAll** (Collection< Property<?> > list)
- void **remove** (String Name)
- void **setParent** (Entity ent)
- void **setOverride** (Boolean value)
- JsonArray **Serialize** ()

16.72.1 Member Function Documentation

16.72.1.1 getPropertiesList()

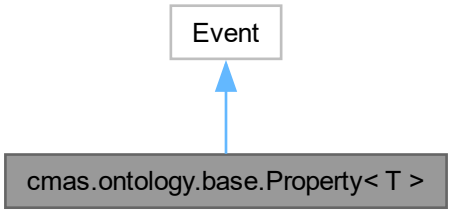
```
Collection< Property<?> > cmas.ontology.base.Properties.getPropertiesList ()
```

Returns a readonly list
The documentation for this class was generated from the following file:

- Properties.java

16.73 cmas.ontology.base.Property< T > Class Template Reference

Inheritance diagram for cmas.ontology.base.Property< T >:



Classes

- enum SpecificPropertyType

Public Member Functions

- **Property** (String name)
- **Property** (String name, String classname, String description, Supplier< Object > getter, Consumer< T > setter, Function< String, Object > deserializer, Function< Object, String > serializer, Boolean readonly, Boolean caninherit, Entity owner)
- **Property** (String name, String classname, String description, Supplier< Object > getter, Consumer< T > setter, Function< String, Object > deserializer, Function< Object, String > serializer, Boolean readonly, Boolean caninherit, EnumSet< SpecificPropertyType > PropertyType, Entity owner)
- **Property** (Property<?> property, boolean instance)
- **Property** (JsonObject job) throws Exception
- Property< T > **copy** (boolean instance)
- void **delete** ()
- Entity **getParent** ()
- void **setParent** (Entity parent)
- String **getParentID** ()
- String **getName** ()
- void **setName** (String name)
- String **getDescription** ()
- void **setDescription** (String description)
- Object **getValue** ()
- void **setValueUndoable** (Object value)
- void **setValue** (Object value)
- Object **getLocalValue** ()
- void **setLocalValue** (Object value)
- Boolean **getCanInherit** ()
- void **setCanInherit** (Boolean canInherit)
- Boolean **getOverride** ()
- void **setOverride** (Boolean overide)
- Supplier< Object > **getGetter** ()
- void **setGetter** (Supplier< Object > getter)
- Consumer< T > **getSetter** ()
- void **setSetter** (Consumer<?> setter)
- String **getClassName** ()
- void **setClassName** (String className)
- Boolean **getReadOnly** ()
- void **setReadOnly** (Boolean readOnly)
- boolean **isEditable** ()
- EnumSet< SpecificPropertyType > **getPropertyType** ()
- void **setPropertyType** (EnumSet< SpecificPropertyType > PropertyType)
- JsonObject **Serialize** ()
- void **DeSerialize** (JsonObject propObject, boolean loadAll)
- String **getHelpTopic** ()
- String **getHelpWord** ()
- boolean **isInstance** ()
- Entity **getOwner** ()
- void **setOwner** (Entity owner)
- ImageView **getIcon** ()

Static Public Member Functions

- static Boolean **dezBoolean** (String str)
- static Double **dezReal** (String str)
- static Integer **dezInteger** (String str)

The documentation for this class was generated from the following file:

- Property.java

16.74 cmas.ontology.variables.ValueRange.Range Class Reference

Public Member Functions

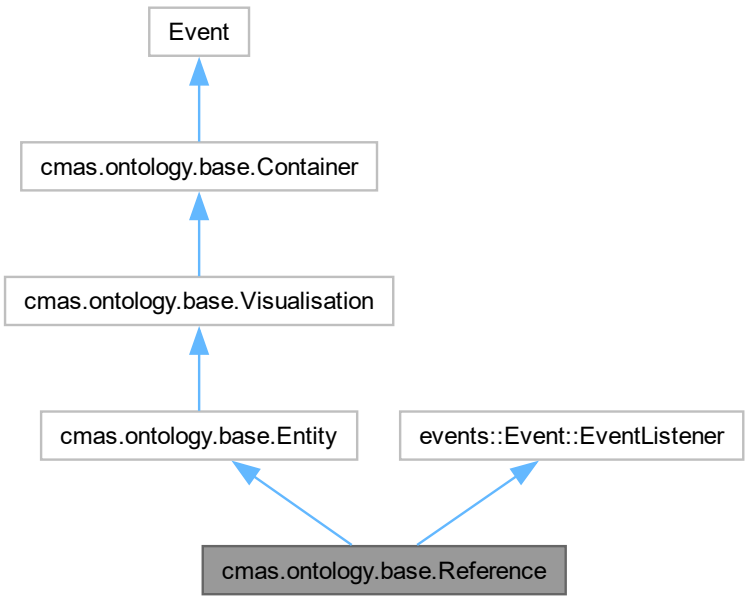
- **Range** (Number value)
- **Range** (Range[ ] ranges)
- boolean **Check** (Number testValue) throws ExceptionType
- boolean **CheckRanges** (Number testValue) throws ExceptionType
- boolean **CheckValues** (Number testValue) throws ExceptionType
- boolean **isNot** ()
- void **setNot** (boolean not)
- boolean **isGreaterThan** ()
- void **setGreaterThan** (boolean greaterThan)
- boolean **isLessThan** ()
- void **setLessThan** (boolean lessThan)

The documentation for this class was generated from the following file:

- ValueRange.java

16.75 cmas.ontology.base.Reference Class Reference

Inheritance diagram for cmas.ontology.base.Reference:



Public Member Functions

- **Reference** (Entity owner, Entity referencedObject)
- **Reference** (Reference reference, boolean instance)
- Entity copy (boolean instance)
- void delete ()
- String **getReferenceName** ()
- String getName ()
- void setName (String Name)
- String **getReferenceID** ()
- Entity **getObject** ()
- void **setObject** (Entity object)
- Entity **getOwner** ()
- void **setOwner** (Entity owner)
- void **onEvent** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object Owner)
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()

- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.75.1 Member Function Documentation

16.75.1.1 copy()

```
Entity cmas.ontology.base.Reference.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().

16.75.1.2 delete()

```
void cmas.ontology.base.Reference.delete ()
```

Reimplemented from cmas.ontology.base.Entity.

16.75.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.75.1.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.
cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.75.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.75.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.75.1.7 generateUniqueIDs()

`void cmas.ontology.base.Entity.generateUniqueIDs () [inherited]`
Will generate unique IDs for the entity and all relation objects
References setID().

16.75.1.8 getAllRelations()

`Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations () [inherited]`

Returns

a list of all relations categories

16.75.1.9 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.75.1.10 getEntityDescription()

`String cmas.ontology.base.Reference.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.75.1.11 getEntityIconFile()

`String cmas.ontology.base.Reference.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.75.1.12 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.75.1.13 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.75.1.14 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.75.1.15 getName()

```
String cmas.ontology.base.Reference.getName ()
```
get current name of the entity.

Returns

current name.

Reimplemented from cmas.ontology.base.Entity.

16.75.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.75.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.75.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```
Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.75.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```
Get specific entity type.

Returns

entity type.

16.75.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.75.1.21 openEntityEditor()

```
void cmas.ontology.base.Reference.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.75.1.22 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.75.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.75.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.75.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.75.1.26 setName()

```
void cmas.ontology.base.Reference.setName (
            String Name)
```

set entity name.

Parameters

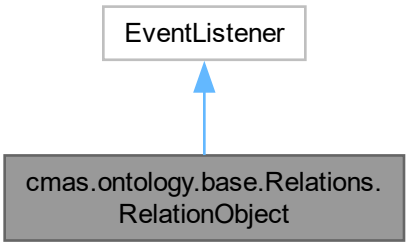
Name	the new name.

Reimplemented from cmas.ontology.base.Entity.
The documentation for this class was generated from the following file:

- Reference.java

16.76 cmas.ontology.base.Relations.RelationObject Class Reference

Inheritance diagram for cmas.ontology.base.Relations.RelationObject:



Public Member Functions

- **RelationObject** (Entity relationentity, Entity owner)
- **RelationObject** (RelationObject relationobject)
- RelationObject **copy** ()
- void **delete** ()
- void **setOwner** (Entity entity)
- Entity **getOwner** ()
- Entity **getObject** ()
- void **onEvent** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object anOwner)
- JsonObject **Serialize** ()

The documentation for this class was generated from the following file:

- Relations.java

16.77 cmas.ontology.base.Relations Class Reference

Classes

- class RelationObject

Public Member Functions

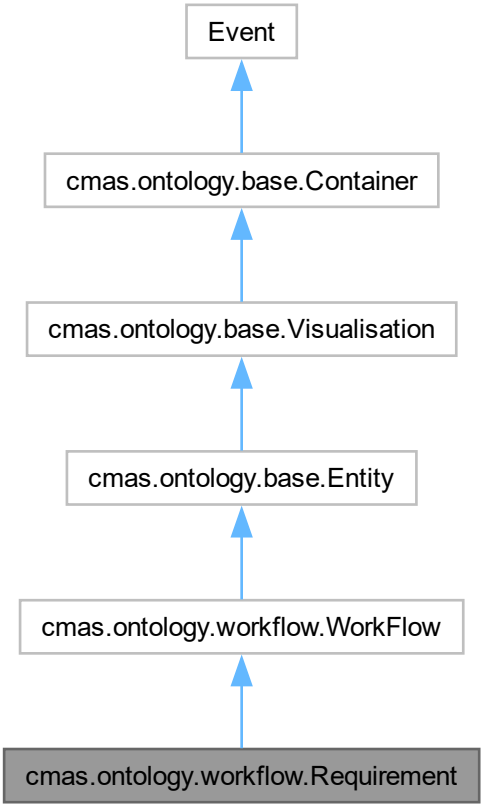
- **Relations** (Relations relations, Container owner, boolean instance)
- Relations **copy** (Container owner, boolean instance)
- void **delete** ()
- LinkedHashMap< String, ArrayList< RelationObject > > **getRelations** ()
- Collection< ArrayList< RelationObject > > **getRelationList** ()
- ArrayList< RelationObject > **getRelation** (String Name)
- Entity **getSpecificRelationEntity** (String RelationName, String Name)
- void **createRelation** (String Name, ArrayList<? extends Entity > AcceptedObjects)
- void **createRelation** (String Name, ArrayList< RelationObject > relation, boolean notused)
- void **addRelationType** (String Name, Entity e)
- ArrayList< Entity > **getRelationTypes** (String Name)
- void **addRelations** (String Name, ArrayList<? extends Entity > ve, Entity Owner)
- void **addItem** (String Name, Entity Item, Entity Owner)
- boolean **removeItem** (String Name, Entity Item)
- boolean **isCompatible** (String Name, Entity ent)
- String **getRealtionName** (Entity ent)
- boolean **hasRelation** (Entity ent, boolean deep)
- void **setParent** (String Name, Entity ent)
- JsonArray **Serialize** ()
- void **DeSerialize** (Entity Owner, JsonArray relationVector) throws ExceptionVariable

The documentation for this class was generated from the following file:

- Relations.java

16.78 cmas.ontology.workflow.Requirement Class Reference

Inheritance diagram for cmas.ontology.workflow.Requirement:



Public Member Functions

- **Requirement** (WorkFlowTypes type, String Name, String Target)
- **Requirement** (Requirement entity, boolean instance)
- String **getTarget** ()
- void **setTarget** (String target)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- SpecificTypes **getSpecificType** ()
- WorkFlowTypes **getWorkFlowType** ()
- void **setWorkFlowType** (WorkFlowTypes workFlowType)
- JsonObject Serialize ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()

- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static SpecificTypes **getSpecificType** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.78.1 Member Function Documentation

16.78.1.1 copy()

```
Entity cmas.ontology.workflow.Requirement.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.78.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.78.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.78.1.4 Factory()

```
Entity cmas.ontology.workflow.WorkFlow.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.78.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.78.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.78.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.78.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.78.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.78.1.10 getEntityDescription()

```
String cmas.ontology.workflow.Requirement.getEntityDescription ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.78.1.11 getEntityIconFile()

```
String cmas.ontology.workflow.Requirement.getEntityIconFile ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.78.1.12 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```
Get the ID of the entity.

Returns

a string with current ID.

16.78.1.13 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.78.1.14 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.78.1.15 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.78.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.78.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.78.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.78.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.78.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.78.1.21 openEntityEditor()

```
void cmas.ontology.workflow.Requirement.openEntityEditor ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.78.1.22 Serialize()

```
JsonObject cmas.ontology.workflow.WorkFlow.Serialize ()  [inherited]
```
Reimplemented from cmas.ontology.base.Entity.

16.78.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.78.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.78.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```
Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.78.1.26 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```
set entity name.

Parameters

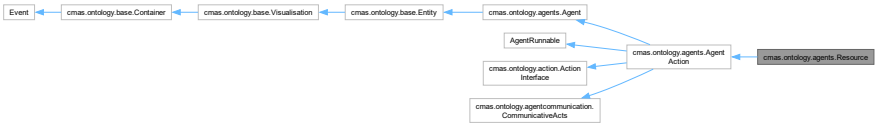
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Requirement.java

16.79 cmas.ontology.agents.Resource Class Reference

Inheritance diagram for cmas.ontology.agents.Resource:



Public Member Functions

- **Resource** (Resource entity, boolean instance)
- Entity copy (boolean instance)
- void Running ()
- void onMessage (ACLMessage message)
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- synchronized Action **getActionObject** ()
- synchronized void **setState** (AgentState newState)
- synchronized AgentState **getState** ()
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- synchronized void **setPaused** (boolean value)
- synchronized boolean **getPaused** ()
- synchronized void postMessage (ACLMessage message)
- Entity getCommunicative ()
- void **onInit** ()
- void **onDepot** ()
- void **onDeploy** ()
- void **onRun** ()
- void **onUndeploy** ()
- void **onFinished** ()
- void **onFailed** ()
- void **onStop** ()
- void **run** ()
- MessageHandler getMessageHandler ()
- void gotoLine (ExceptionStructuredText e)
- void stopEditor (ExceptionStructuredText e)
- void **compile** () throws ExceptionParameters, Exception
- String **getPhysicalID** ()
- void **setPhysicalID** (String physicalID)
- String **getAgentSpecificType** ()
- void **setAgentSpecificType** (String agentSpecificType)
- AgentBaseTypes **getAgentBaseType** ()
- void **addAbstractInterface** (AbstractInterface abstractInterface, String Name)
- void **clearAbstractInterfaces** ()
- AbstractInterface **getAbstractInterface** (String Name)
- RuntimeTree **getRuntimeTree** ()
- void **setRuntimeTree** (RuntimeTree runtimeTree)
- Domain **getRuntimeDomain** ()

- void **setRuntimeDomain** (Domain runtimeDomain)
- LinkedBlockingQueue< WorkFlowLog > **getHistory** ()
- void **setHistory** (LinkedBlockingQueue< WorkFlowLog > p)
- void **addToHistory** (WorkFlowLog log)
- Boolean **getAutoDeploy** ()
- void **setAutoDeploy** (Boolean autoDeploy)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()
- default ACLMessage **waitForPerformative** (MessageHandler messageHandler, ACLMessage message, Performative answerType) throws InterruptedException

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static AgentBaseTypes **getAgentType** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Attributes

- Action **action**
- MessageHandler **messageHandler**
- Properties **properties**
- Relations **relations**

16.79.1 Member Function Documentation

16.79.1.1 copy()

```
Entity cmas.ontology.agents.Resource.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.79.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.79.1.3 DeSerialize()

```
void cmas.ontology.agents.AgentAction.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.agents.Agent.

16.79.1.4 Factory()

```
Entity cmas.ontology.agents.Agent.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.79.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.79.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.79.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.79.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.79.1.9 getCommunicative()

`Entity cmas.ontology.agents.AgentAction.getCommunicative () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.79.1.10 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.79.1.11 getEntityDescription()

`String cmas.ontology.agents.Resource.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.79.1.12 getEntityIconFile()

`String cmas.ontology.agents.Resource.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.79.1.13 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.79.1.14 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.79.1.15 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.79.1.16 getMessageHandler()

`MessageHandler cmas.ontology.agents.AgentAction.getMessageHandler () [inherited]`
Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.79.1.17 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.79.1.18 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.79.1.19 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.79.1.20 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.79.1.21 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.79.1.22 gotoLine()

```
void cmas.ontology.agents.AgentAction.gotoLine (
            ExceptionStructuredText e)  [inherited]
```

Implements cmas.ontology.action.ActionInterface.

16.79.1.23 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.79.1.24 onMessage()

```
void cmas.ontology.agents.Resource.onMessage (
            ACLMessage message)
```

Reimplemented from cmas.ontology.agents.AgentAction.

16.79.1.25 openEntityEditor()

```
void cmas.ontology.agents.Resource.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.79.1.26 postMessage()

```
synchronized void cmas.ontology.agents.AgentAction.postMessage (
            ACLMessage message)  [inherited]
```

Implements cmas.ontology.agentcommunication.CommunicativeActs.

16.79.1.27 Running()

```
void cmas.ontology.agents.Resource.Running ()
```

Reimplemented from cmas.ontology.agents.AgentAction.

16.79.1.28 Serialize()

```
JsonObject cmas.ontology.agents.AgentAction.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.agents.Agent.

16.79.1.29 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.79.1.30 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.79.1.31 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.79.1.32 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.79.1.33 stopEditor()

```
void cmas.ontology.agents.AgentAction.stopEditor (
            ExceptionStructuredText e)  [inherited]
```

Implements cmas.ontology.action.ActionInterface.
The documentation for this class was generated from the following file:

- Resource.java

16.80 cmas.ontology.adapter.REST.RESTClient.REST_TYPE Enum Reference

Public Attributes

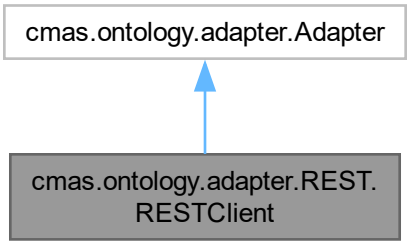
- **NON**
- **ASSOCIATE**

The documentation for this enum was generated from the following file:

- RESTClient.java

16.81 cmas.ontology.adapter.REST.RESTClient Class Reference

Inheritance diagram for cmas.ontology.adapter.REST.RESTClient:



Classes

- enum REST_TYPE

Public Member Functions

- **RESTClient** (String Source)
- void **SetSource** (String Source)
- void **setServerType** (REST_TYPE mode)
- REST_TYPE **getServerType** ()
- String ReadData (String Address, String Name, Object Value)
- void WriteData (String Address, String Name, Boolean bValue) throws Exception
- void WriteData (String Address, String Name, String sValue) throws Exception
- void WriteData (String Address, String Name, Integer iValue) throws Exception
- void WriteData (String Address, String Name, Double dValue) throws Exception
- boolean close ()
- void setDebug (boolean bValue)
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)

16.81.1 Detailed Description

Main class for REST communication using POST and GET
Ideas from: https://stackoverflow.com/questions/32586626/java-equivalent-of-networkcredentia
Examples in ABB Robot Web Services: https://developercenter.robotstudio.com/api/rws↩
Api/_hello_controller_json_8cs-example.html

```
For adapter type of REST the specific format is: REST:[HTTP]:[HOST]:[PORT]:[AUTH]:[USER]:[PASW]
eg, REST:http:localhost:80:/rw/rapid/symbol/data/RAPID/T_ROB1/MainModule/:DIGEST:Default User:robotics
opc.tcp is static for OPC UA
HTTP http or https
HOST is the host name or TCP/IP address to the server
PORT is the port number to the server
AUTH  is the name of the authentication type to use it can be DIGEST, BASIC or NONE
USER User name
PASW Password
```

Author

Anders Nilsson, University West

16.81.2 Member Function Documentation

16.81.2.1 close()

```
boolean cmas.ontology.adapter.REST.RESTClient.close ()
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.2 createNew()

```
Adapter cmas.ontology.adapter.REST.RESTClient.createNew (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.3 getDefaultValue()

```
String cmas.ontology.adapter.REST.RESTClient.getDefaultValue ()
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.4 getDescription()

```
String cmas.ontology.adapter.REST.RESTClient.getDescription ()
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.5 isValid()

```
boolean cmas.ontology.adapter.REST.RESTClient.isValid (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.6 ReadData()

```
String cmas.ontology.adapter.REST.RESTClient.ReadData (
            String Address,
            String Name,
            Object Value)
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.7 setDebug()

```
void cmas.ontology.adapter.REST.RESTClient.setDebug (
            boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.8 WriteData() [1/4]

```
void cmas.ontology.adapter.REST.RESTClient.WriteData (
            String Address,
            String Name,
            Boolean bValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.9 WriteData() [2/4]

```
void cmas.ontology.adapter.REST.RESTClient.WriteData (
            String Address,
```

```
            String Name,
            Double dValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.10 WriteData() [3/4]

```
void cmas.ontology.adapter.REST.RESTClient.WriteData (
            String Address,
            String Name,
            Integer iValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.81.2.11 WriteData() [4/4]

```
void cmas.ontology.adapter.REST.RESTClient.WriteData (
            String Address,
            String Name,
            String sValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.
The documentation for this class was generated from the following file:

- RESTClient.java

16.82 cmas.ontology.adapter.socket.SocketTCPClient.SERVER_TYPE Enum Reference

Public Attributes

- **ASCII**
- **BINARY**

The documentation for this enum was generated from the following file:

- SocketTCPClient.java

16.83 cmas.ontology.adapter.socket.SocketUDPClient.SERVER_TYPE Enum Reference

Public Attributes

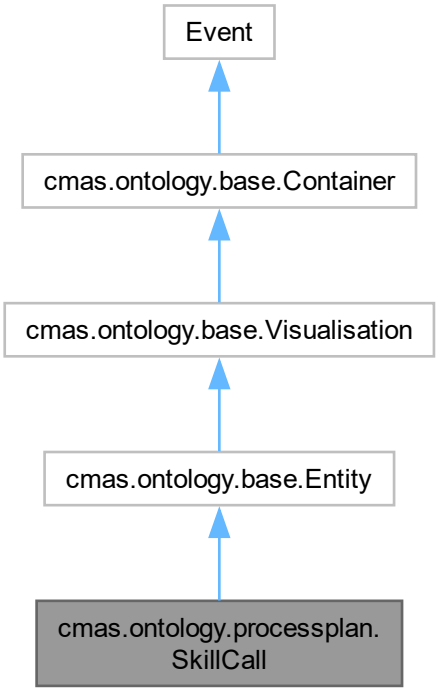
- **ASCII**
- **BINARY**

The documentation for this enum was generated from the following file:

- SocketUDPClient.java

16.84 cmas.ontology.processplan.SkillCall Class Reference

Inheritance diagram for cmas.ontology.processplan.SkillCall:



Public Member Functions

- **SkillCall** (String Name)
- **SkillCall** (SkillCall skillCall, boolean instance)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)

- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- JsonObject Serialize ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.84.1 Member Function Documentation

16.84.1.1 copy()

```
Entity cmas.ontology.processplan.SkillCall.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().

16.84.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.84.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.84.1.4 Factory()

```
Entity cmas.ontology.base.Entity.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```
Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented in cmas.ontology.agents.Agent, cmas.ontology.interfaces.Interface, cmas.ontology.processplan.ProcessPlan, cmas.ontology.variables.Value, cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, and cmas.ontology.workflow.WorkFlow.
References Entity(), cmas.ontology.agents.Agent.Factory(), cmas.ontology.interfaces.Interface.Factory(), cmas.ontology.processplan.P cmas.ontology.variables.Value.Factory(), cmas.ontology.variables.VariableArray.Factory(), and cmas.ontology.workflow.WorkFlow.Fact

16.84.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```
Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	

EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.84.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.84.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.84.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.84.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.84.1.10 getEntityDescription()

```
String cmas.ontology.processplan.SkillCall.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.84.1.11 getEntityIconFile()

```
String cmas.ontology.processplan.SkillCall.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.84.1.12 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.84.1.13 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.84.1.14 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.84.1.15 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.84.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.84.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.84.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.84.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.84.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.84.1.21 openEntityEditor()

```
void cmas.ontology.processplan.SkillCall.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.84.1.22 Serialize()

```
JsonObject cmas.ontology.base.Entity.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.84.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.84.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.84.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.84.1.26 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

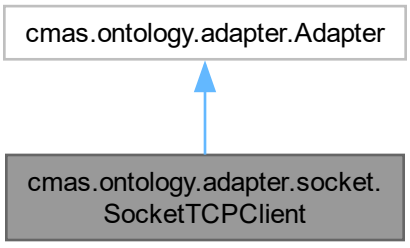
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- SkillCall.java

16.85 cmas.ontology.adapter.socket.SocketTCPClient Class Reference

Inheritance diagram for cmas.ontology.adapter.socket.SocketTCPClient:



Classes

- enum SERVER_TYPE

Public Member Functions

- **SocketTCPClient** (String Source)
- void **setSource** (String Source)
- void **connectToServer** ()
- String **getServerHost** ()
- void **setServerHost** (String serverHost)
- int **getServerPort** ()
- void **setServerPort** (int serverPort)
- SERVER_TYPE **getServerType** ()
- void **setServerType** (SERVER_TYPE serverType)
- int **getTimeout** ()
- void **setTimeout** (int timeout)
- String ReadData (String Address, String Name, Object Value) throws Exception
- void WriteData (String Address, String Name, Boolean bValue) throws Exception
- void WriteData (String Address, String Name, String sValue) throws Exception
- void WriteData (String Address, String Name, Integer iValue) throws Exception
- void WriteData (String Address, String Name, Double dValue) throws Exception
- boolean close ()
- void setDebug (boolean bValue)
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)

16.85.1 Member Function Documentation

16.85.1.1 close()

```
boolean cmas.ontology.adapter.socket.SocketTCPClient.close ()
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.2 createNew()

```
Adapter cmas.ontology.adapter.socket.SocketTCPClient.createNew (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.3 getDefaultValue()

```
String cmas.ontology.adapter.socket.SocketTCPClient.getDefaultValue ()
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.4 getDescription()

```
String cmas.ontology.adapter.socket.SocketTCPClient.getDescription ()
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.5 isValid()

```
boolean cmas.ontology.adapter.socket.SocketTCPClient.isValid (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.6 ReadData()

```
String cmas.ontology.adapter.socket.SocketTCPClient.ReadData (
            String Address,
            String Name,
            Object Value) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.7 setDebug()

```
void cmas.ontology.adapter.socket.SocketTCPClient.setDebug (
            boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.8 WriteData() [1/4]

```
void cmas.ontology.adapter.socket.SocketTCPClient.WriteData (
            String Address,
            String Name,
            Boolean bValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.9 WriteData() [2/4]

```
void cmas.ontology.adapter.socket.SocketTCPClient.WriteData (
            String Address,
            String Name,
            Double dValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.10 WriteData() [3/4]

```
void cmas.ontology.adapter.socket.SocketTCPClient.WriteData (
            String Address,
            String Name,
            Integer iValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.85.1.11 WriteData() [4/4]

```
void cmas.ontology.adapter.socket.SocketTCPClient.WriteData (
            String Address,
            String Name,
            String sValue) throws Exception
```

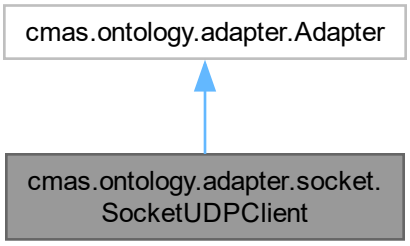
Implements cmas.ontology.adapter.Adapter.

The documentation for this class was generated from the following file:

- SocketTCPClient.java

16.86 cmas.ontology.adapter.socket.SocketUDPClient Class Reference

Inheritance diagram for cmas.ontology.adapter.socket.SocketUDPClient:



Classes

- enum SERVER_TYPE

Public Member Functions

- **SocketUDPClient** (String Source)
- void **setSource** (String Source)
- void **connectToServer** ()
- String **getServerHost** ()
- void **setServerHost** (String serverHost)
- int **getServerPort** ()
- void **setServerPort** (int serverPort)
- SERVER_TYPE **getServerType** ()
- void **setServerType** (SERVER_TYPE serverType)
- int **getTimeout** ()
- void **setTimeout** (int timeout)
- String ReadData (String Address, String Name, Object Value) throws Exception
- void WriteData (String Address, String Name, Boolean bValue) throws Exception
- void WriteData (String Address, String Name, String sValue) throws Exception
- void WriteData (String Address, String Name, Integer iValue) throws Exception
- void WriteData (String Address, String Name, Double dValue) throws Exception
- boolean close ()
- void setDebug (boolean bValue)
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)

16.86.1 Member Function Documentation

16.86.1.1 close()

```
boolean cmas.ontology.adapter.socket.SocketUDPClient.close ()
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.2 createNew()

```
Adapter cmas.ontology.adapter.socket.SocketUDPClient.createNew (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.3 getDefaultValue()

```
String cmas.ontology.adapter.socket.SocketUDPClient.getDefaultValue ()
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.4 getDescription()

```
String cmas.ontology.adapter.socket.SocketUDPClient.getDescription ()
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.5 isValid()

```
boolean cmas.ontology.adapter.socket.SocketUDPClient.isValid (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.6 ReadData()

```
String cmas.ontology.adapter.socket.SocketUDPClient.ReadData (
            String Address,
            String Name,
            Object Value) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.7 setDebug()

```
void cmas.ontology.adapter.socket.SocketUDPClient.setDebug (
            boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.8 WriteData() [1/4]

```
void cmas.ontology.adapter.socket.SocketUDPClient.WriteData (
            String Address,
            String Name,
            Boolean bValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.9 WriteData() [2/4]

```
void cmas.ontology.adapter.socket.SocketUDPClient.WriteData (
            String Address,
            String Name,
            Double dValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.10 WriteData() [3/4]

```
void cmas.ontology.adapter.socket.SocketUDPClient.WriteData (
            String Address,
            String Name,
            Integer iValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.86.1.11 WriteData() [4/4]

```
void cmas.ontology.adapter.socket.SocketUDPClient.WriteData (
            String Address,
            String Name,
            String sValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.
The documentation for this class was generated from the following file:

- SocketUDPClient.java

16.87 cmas.ontology.base.Property< T >.SpecificPropertyType Enum Reference

Public Attributes

- **INHERITANCE**
- **REFERENCEFILE**
- **PARENTEDITOR**
- **IDEDIT**
- **TEXTEDITOR**
- **STRINGEDIT**
- **STRINGEDITMULTILINE**
- **VARIABLEEDITOR**
- **SFCEDITOR**
- **PROCESSPLANEDITOR**
- **BATCHEDITOR**
- **INTERFACEBROWSER**
- **ADAPTEREDIT**
- **ADDRESSBROWSE**
- **HISTORYSHOW**
- **PROPERTYSETEDITOR**
- **HIDE**
- **RUNTIME**
- **MODELLING**
- **HIDEBASIC**

The documentation for this enum was generated from the following file:

- Property.java

16.88 cmas.ontology.workflow.WorkFlow.SpecificTypes Enum Reference

Public Attributes

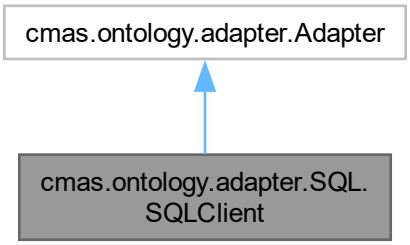
- **REQUIREMENTS**
- **WORKFLOWLOG**

The documentation for this enum was generated from the following file:

- WorkFlow.java

16.89 cmas.ontology.adapter.SQL.SQLClient Class Reference

Inheritance diagram for cmas.ontology.adapter.SQL.SQLClient:



Public Member Functions

- **SQLClient** (String Source)
- void **setSource** (String source)
- String ReadData (String Address, String Name, Object Value)
- void WriteData (String Address, String Name, Boolean bValue) throws Exception
- void WriteData (String Address, String Name, String sValue) throws Exception
- void WriteData (String Address, String Name, Integer iValue) throws Exception
- void WriteData (String Address, String Name, Double dValue) throws Exception
- boolean close ()
- void setDebug (boolean bValue)
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)

16.89.1 Detailed Description

SQL Adapter for CMAS - Fredrik Danielsson
Source format: SQL:JDBC:HOST:PORT:DATABASE:LIMIT:USER:PASSWORD Example: SQL:org.mariadb.↩
jdbc.Driver:localhost:3306:production:100:robot:secret

16.89.2 Member Function Documentation

16.89.2.1 close()

```
boolean cmas.ontology.adapter.SQL.SQLClient.close ()
```

Implements cmas.ontology.adapter.Adapter.

16.89.2.2 createNew()

```
Adapter cmas.ontology.adapter.SQL.SQLClient.createNew (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.89.2.3 getDefaultValue()

```
String cmas.ontology.adapter.SQL.SQLClient.getDefaultValue ()
```

Implements cmas.ontology.adapter.Adapter.

16.89.2.4 getDescription()

```
String cmas.ontology.adapter.SQL.SQLClient.getDescription ()
```

Implements cmas.ontology.adapter.Adapter.

16.89.2.5 isValid()

```
boolean cmas.ontology.adapter.SQL.SQLClient.isValid (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.89.2.6 ReadData()

```
String cmas.ontology.adapter.SQL.SQLClient.ReadData (
            String Address,
            String Name,
            Object Value)
```

Read a value from the database

Implements cmas.ontology.adapter.Adapter.

16.89.2.7 setDebug()

```
void cmas.ontology.adapter.SQL.SQLClient.setDebug (
            boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.89.2.8 WriteData() [1/4]

```
void cmas.ontology.adapter.SQL.SQLClient.WriteData (
            String Address,
            String Name,
            Boolean bValue) throws Exception
```

Write value to database using prepared statement

Implements cmas.ontology.adapter.Adapter.

16.89.2.9 WriteData() [2/4]

```
void cmas.ontology.adapter.SQL.SQLClient.WriteData (
            String Address,
            String Name,
            Double dValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.89.2.10 WriteData() [3/4]

```
void cmas.ontology.adapter.SQL.SQLClient.WriteData (
            String Address,
            String Name,
            Integer iValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.89.2.11 WriteData() [4/4]

```
void cmas.ontology.adapter.SQL.SQLClient.WriteData (
            String Address,
            String Name,
            String sValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

The documentation for this class was generated from the following file:

- SQLClient.java

16.90 cmas.ontology.variables.Variable.TypeOfVariable Enum Reference

Public Attributes

- **PRIMITIVE**
- **ARRAY**
- **OBJECT**

The documentation for this enum was generated from the following file:

- Variable.java

16.91 cmas.ontology.Types Class Reference

Static Public Member Functions

- static Entity **getType** (String Name)
- static Entity **Factory** (String Name) throws InstantiationException, IllegalAccessException, IllegalArgument↩ Exception, InvocationTargetException, NoSuchMethodException, SecurityException
- static void **addEntity** (String Category, Entity newObject)

Static Public Attributes

- static final ArrayList< Variable > ValueTypes
- static final ArrayList< ProcessPlan > SkillTypes
- static final ArrayList< Interface > InterfaceTypes
- static final ArrayList< Goal > GoalTypes
- static final ArrayList< Hazard > HazardTypes
- static final ArrayList< ProcessPlan > ProcessPlanTypes
- static final ArrayList< Agent > AgentTypes
- static final ArrayList< Batch > BatchTypes
- static final ArrayList< DeploymentAgent > DeploymentTypes
- static final ArrayList< WorkFlowLog > WorkFlowTypes
- static final ArrayList< Requirement > RequirementTypes
- static final ArrayList< Entity > FolderTypes
- static final ArrayList< Entity > EntityTypes

16.91.1 Member Data Documentation

16.91.1.1 AgentTypes

```
final ArrayList<Agent> cmas.ontology.Types.AgentTypes  [static]
```

Initial value:

```
=new ArrayList<Agent>() {
            private static final long serialVersionUID = 1L;
            {
                    add(new Part());
                    add(new Material());
                    add(new Resource());
            }}
```

16.91.1.2 BatchTypes

```
final ArrayList<Batch> cmas.ontology.Types.BatchTypes  [static]
```

Initial value:

```
=new ArrayList<Batch>() {
            private static final long serialVersionUID = 1L;
            {
                    add(new Batch());
            }}
```

16.91.1.3 DeploymentTypes

final ArrayList<DeploymentAgent> cmas.ontology.Types.DeploymentTypes [static]

Initial value:

```
=new ArrayList<DeploymentAgent>() {
                private static final long serialVersionUID = 1L;
                {
                        add(new DeploymentAgent());
                }}
```

16.91.1.4 EntityTypes

final ArrayList<Entity> cmas.ontology.Types.EntityTypes [static]

Initial value:

```
=new ArrayList<Entity>() {
            private static final long serialVersionUID = 1L;
            {
                try {
                    addAll(AgentTypes);
                    addAll(InterfaceTypes);
                    addAll(GoalTypes);
                    addAll(ValueTypes);

                    addAll(WorkFlowTypes);
                    addAll(RequirementTypes);
                    addAll(ProcessPlanTypes);
                    addAll(HazardTypes);

                    addAll(DeploymentTypes);
                    addAll(BatchTypes);
                    add(new Folder());
                    add(new Note());
                    add(new ModelSettings());
                    add(new ModelState());
                } catch(Exception e) {
                     if(Main.mainPreferences.getValueAsBoolean("Debug"))
                         System.out.println("unable to initialise static Types (3):" + e.getMessage());

                };
            }}
```

16.91.1.5 FolderTypes

final ArrayList<Entity> cmas.ontology.Types.FolderTypes [static]

Initial value:

```
=new ArrayList<Entity>() {
                private static final long serialVersionUID = 1L;
                {
                    try {
                        addAll(AgentTypes);
                        addAll(InterfaceTypes);
                        addAll(GoalTypes);
                        addAll(ValueTypes);

                        addAll(WorkFlowTypes);
                        addAll(RequirementTypes);
                        addAll(ProcessPlanTypes);

                        addAll(BatchTypes);
                        addAll(DeploymentTypes);
                    } catch(Exception e) {
                         if(Main.mainPreferences.getValueAsBoolean("Debug"))
                             System.out.println("unable to initialise static Types (2):" + e.getMessage());

                    };
                }}
```

16.91.1.6 GoalTypes

final ArrayList<Goal> cmas.ontology.Types.GoalTypes [static]

Initial value:

```
=new ArrayList<Goal>() {
            private static final long serialVersionUID = 1L;
            {
                    add(new Goal());
            }}
```

16.91.1.7 HazardTypes

final ArrayList<Hazard> cmas.ontology.Types.HazardTypes [static]

Initial value:

```
=new ArrayList<Hazard>() {
            private static final long serialVersionUID = 1L;
            {
                    add(new Hazard());
            }}
```

16.91.1.8 InterfaceTypes

final ArrayList<Interface> cmas.ontology.Types.InterfaceTypes [static]

Initial value:

```
=new ArrayList<Interface>() {
            private static final long serialVersionUID = 1L;
            {
                    add(new Interface());
                    add(new InterfaceSpecialisation());
                    add(new InterfaceTransport());
                    add(new InterfaceLocation());
                    add(new InterfaceAssembly());
            }}
```

16.91.1.9 ProcessPlanTypes

final ArrayList<ProcessPlan> cmas.ontology.Types.ProcessPlanTypes [static]

Initial value:

```
=new ArrayList<ProcessPlan>() {
            private static final long serialVersionUID = 1L;
            {
                    add(new ProcessPlanStructuredText());
                    add(new ProcessPlanGoals());

                    if(ProcessPlanJava.ENABLED)
                        add(new ProcessPlanJava());
            }}
```

16.91.1.10 RequirementTypes

final ArrayList<Requirement> cmas.ontology.Types.RequirementTypes [static]

Initial value:

```
=new ArrayList<Requirement>() {
                private static final long serialVersionUID = 1L;
                {
                        add(new Requirement());
                }}
```

16.91.1.11 SkillTypes

final ArrayList<ProcessPlan> cmas.ontology.Types.SkillTypes [static]

Initial value:

```
=new ArrayList<ProcessPlan>() {
            private static final long serialVersionUID = 1L;
            {
                ProcessPlan p=new ProcessPlanStructuredText();
                p.setName("Skill Structured Text");
                add(p);

                p=new ProcessPlanGoals();
                p.setName("Skill SFC Goals");
                add(p);

                if(ProcessPlanJava.ENABLED) {
                    p=new ProcessPlanJava();
                    p.setName("Skill Java");
                    add(p);
                }
            }}
```

16.91.1.12 ValueTypes

final ArrayList<Variable> cmas.ontology.Types.ValueTypes [static]

Initial value:

```
=new ArrayList<Variable>() {
        private static final long serialVersionUID = 1L;
        {
            try {
                add(new VariableInteger());
                add(new VariableReal());
                add(new VariableBoolean());
                add(new VariableString());

                add(new VariableArray("VariableArray", new VariableInteger(),new Integer[] {1}, new
      Integer[] {2}));
                add(new VariableObject("VariableObject"));
            }catch(Exception e) {
                 if(Main.mainPreferences.getValueAsBoolean("Debug"))
                     System.out.println("unable to initialise static Types (1):" + e.getMessage());
            };
        }}
```

16.91.1.13 WorkFlowTypes

```
final ArrayList<WorkFlowLog> cmas.ontology.Types.WorkFlowTypes  [static]
```

Initial value:

```
=new ArrayList<WorkFlowLog>() {
                private static final long serialVersionUID = 1L;
                {


                }}
```

The documentation for this class was generated from the following file:

- Types.java

16.92 cmas.ontology.base.Entity.UID Class Reference

Static Public Member Functions

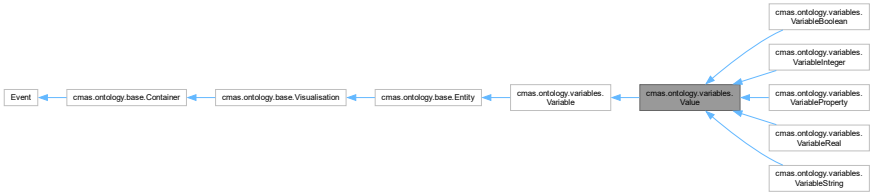
- static synchronized String **Generate** ()

The documentation for this class was generated from the following file:

- Entity.java

16.93 cmas.ontology.variables.Value Class Reference

Inheritance diagram for cmas.ontology.variables.Value:



Classes

- enum ValueTypes

Public Member Functions

- **Value** (ValueTypes type) throws ExceptionType
- **Value** (Value variable, boolean instance) throws ExceptionType
- Object getCopyOfValue () throws ExceptionType

- String getStringValue () throws ExceptionType
- void **setReferenceProperty** (Property<?> prop)
- Property<?> **getReferenceProperty** ()
- boolean **isReferenceProperty** ()
- ValueTypes **getValueType** ()
- Boolean **isBoolean** ()
- Boolean **isInteger** ()
- Boolean **isReal** ()
- Boolean **isString** ()
- synchronized Object getValue ()
- void setValue (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void setValueUndoable (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void **setValue** (Object newValue, boolean undoable) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- Object valueOf (String s)
- String **getUnit** ()
- void **setUnit** (String value)
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- String **getUpperBound** ()
- void **setUpperBound** (String upperBound)
- String **getLowerBound** ()
- void **setLowerBound** (String lowerBound)
- boolean **isWithinRange** (Object testValue) throws ExceptionType
- Integer **parseInteger** (String Value, Integer Default)
- boolean **parseBoolean** (String Value, Boolean Default)
- Double **parseReal** (String Value, Double Default)
- JsonObject Serialize ()
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- abstract Entity copy (boolean instance)
- abstract Iterator **getIterator** ()
- Variable **getIteratorObject** ()
- boolean **hasNext** (Iterator iterator)
- Object **getNext** (Iterator iterator) throws ExceptionVariable, ExceptionType, ExceptionConstant
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()
- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)

- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static ValueTypes **getValueType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Member Functions

- void **setIgnoreReadOnly** (boolean value)

Protected Attributes

- Property<?> **ReferenceProperty**
- ValueTypes **ValueType**
- String **UpperBound**
- String **LowerBound**
- String **Unit**
- Action **Action**
- ValueRange **upperRangeExpression**
- ValueRange **lowerRangeExpression**
- Boolean **ignoreReadOnly** = true
- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

Static Protected Attributes

- static final Boolean **ignoreTypeCheck** = false
- static final Boolean **ignoreRangeCheck** = false

16.93.1 Member Function Documentation

16.93.1.1 copy()

```
abstract Entity cmas.ontology.variables.Variable.copy (
            boolean instance)  [abstract], [inherited]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.
Reimplemented in cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.Variable cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, and cmas.ontology.variables.VariableString.
References cmas.ontology.base.Entity.Entity().

16.93.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.93.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.93.1.4 Factory()

```
Entity cmas.ontology.variables.Value.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.variables.Variable.
Referenced by cmas.ontology.base.Entity.Factory(), and cmas.ontology.variables.Variable.Factory().

16.93.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.93.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.93.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.93.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.93.1.9 getCopyOfValue()

```
Object cmas.ontology.variables.Value.getCopyOfValue () throws ExceptionType
```

Reimplemented from cmas.ontology.variables.Variable.

16.93.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.93.1.11 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.93.1.12 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.93.1.13 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.93.1.14 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.93.1.15 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.93.1.16 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.93.1.17 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.93.1.18 getStringValue()

```
String cmas.ontology.variables.Value.getStringValue () throws ExceptionType
```
Reimplemented from cmas.ontology.variables.Variable.

16.93.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```
Get specific entity type.

Returns

entity type.

16.93.1.20 getValue()

```
synchronized Object cmas.ontology.variables.Value.getValue ()
```
Reimplemented from cmas.ontology.variables.Variable.

16.93.1.21 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```
Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.93.1.22 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```
Reimplemented from cmas.ontology.base.Visualisation.

16.93.1.23 Serialize()

```
JsonObject cmas.ontology.variables.Value.Serialize ()
```
Reimplemented from cmas.ontology.variables.Variable.

16.93.1.24 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.93.1.25 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.93.1.26 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.93.1.27 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.93.1.28 setValue()

```
void cmas.ontology.variables.Value.setValue (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
```

Reimplemented from cmas.ontology.variables.Variable.

16.93.1.29 setValueUndoable()

```
void cmas.ontology.variables.Value.setValueUndoable (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
```

Reimplemented from cmas.ontology.variables.Variable.

16.93.1.30 valueOf()

```
Object cmas.ontology.variables.Value.valueOf (
            String s)
```

Reimplemented from cmas.ontology.variables.Variable.
The documentation for this class was generated from the following file:

- Value.java

16.94 cmas.ontology.variables.ValueRange Class Reference

Classes

- class Range

Public Member Functions

- boolean **Evaluate** (Number value) throws ExceptionType
- void **compile** (String expression) throws ExceptionType

Static Public Member Functions

- static boolean **isARangeExpression** (String expression)

The documentation for this class was generated from the following file:

- ValueRange.java

16.95 cmas.ontology.variables.Value.ValueTypes Enum Reference

Public Attributes

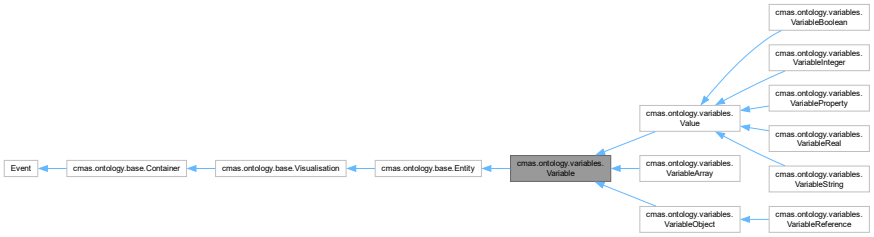
- **NON**
- **INTEGER**
- **REAL**
- **BOOLEAN**
- **STRING**

The documentation for this enum was generated from the following file:

- Value.java

16.96 cmas.ontology.variables.Variable Class Reference

Inheritance diagram for cmas.ontology.variables.Variable:



Classes

- enum TypeOfVariable

Public Member Functions

- **Variable** (TypeOfVariable type)
- **Variable** (Variable variable, boolean instance)
- abstract Entity copy (boolean instance)
- abstract Object **getCopyOfValue** () throws ExceptionType
- abstract String **getStringValue** () throws ExceptionType, ExceptionVariable

- abstract Object **getValue** ()
- abstract void **setValueUndoable** (Object newValue) throws ExceptionType, ExceptionNullValue, ExceptionConstant, ExceptionVariable
- abstract void **setValue** (Object newValue) throws ExceptionType, ExceptionNullValue, ExceptionConstant, ExceptionVariable
- abstract Object **valueOf** (String value) throws ExceptionVariable, ExceptionType, ExceptionNullValue, ExceptionConstant
- abstract Iterator **getIterator** ()
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- Variable **getIteratorObject** ()
- boolean **hasNext** (Iterator iterator)
- Object **getNext** (Iterator iterator) throws ExceptionVariable, ExceptionType, ExceptionConstant
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()
- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- JsonObject Serialize ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()

- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

16.96.1 Member Function Documentation

16.96.1.1 copy()

```
abstract Entity cmas.ontology.variables.Variable.copy (
            boolean instance)  [abstract]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.
Reimplemented in cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.Variable cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, and cmas.ontology.variables.VariableString.
References cmas.ontology.base.Entity.Entity().

16.96.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.96.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.96.1.4 Factory()

```
Entity cmas.ontology.variables.Variable.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
Reimplemented in cmas.ontology.variables.Value, and cmas.ontology.variables.VariableArray.
References cmas.ontology.base.Entity.Entity(), cmas.ontology.variables.Value.Factory(), and cmas.ontology.variables.VariableArray.Fa

16.96.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.96.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.96.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.96.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.96.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.96.1.10 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.96.1.11 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.96.1.12 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.96.1.13 getName()

`String cmas.ontology.base.Entity.getName () [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.96.1.14 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity) [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.96.1.15 getRelations()

`LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations () [inherited]`

Returns

a HashMap of all relation categories and their linked names.

16.96.1.16 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.96.1.17 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.96.1.18 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.96.1.19 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.96.1.20 Serialize()

```
JsonObject cmas.ontology.variables.Variable.Serialize ()
```

Reimplemented from cmas.ontology.base.Entity.

16.96.1.21 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.96.1.22 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.96.1.23 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.96.1.24 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

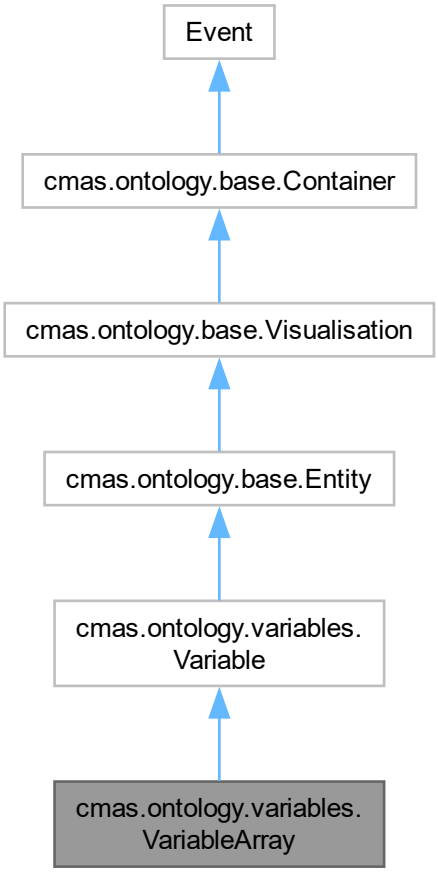
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- Variable.java

16.97 cmas.ontology.variables.VariableArray Class Reference

Inheritance diagram for cmas.ontology.variables.VariableArray:



Public Member Functions

- **VariableArray** (String Name, Object initValue, Integer[ ] LowerRange, Integer[ ] UpperRange) throws ExceptionVariable, ExceptionType
- **VariableArray** (VariableArray array, boolean instance) throws ExceptionType
- void **resize** (Object initValue, Integer[ ] LowerRange, Integer[ ] UpperRange) throws ExceptionVariable, ExceptionType
- void setName (String name)
- Object getValue ()
- Object **getValue** (Integer Index) throws ExceptionVariable
- void setValueUndoable (Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- void setValue (Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- void **replaceValue** (VariableArray newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- void **setValue** (Integer Index, Object newValue) throws ExceptionVariable, ExceptionType, ExceptionNull↩ Value, ExceptionConstant

- void **swap** (int Index1, int Index2) throws ExceptionVariable, ExceptionConstant
- String getStringValue () throws ExceptionType, ExceptionVariable
- Object valueOf (String value) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- Object getCopyOfValue () throws ExceptionType
- Integer **getSize** ()
- Integer **getLowerRange** ()
- Integer **getUpperRange** ()
- void **setUpperRange** (Integer range)
- void **setLowerRange** (Integer range)
- ValueTypes **getValueType** ()
- void **setValueType** (ValueTypes type)
- JsonObject Serialize ()
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- Iterator getIterator ()
- boolean hasNext (Iterator iterator)
- Object getNext (Iterator iterator) throws ExceptionVariable
- Variable getIteratorObject ()
- boolean **isRelationReference** ()
- void **setRelationReference** (ArrayList< RelationObject > relation) throws ExceptionVariable, ExceptionType
- ArrayList< RelationObject > **getRelationReference** ()
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()
- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)

- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

16.97.1 Member Function Documentation

16.97.1.1 copy()

```
Entity cmas.ontology.variables.VariableArray.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.variables.Variable.

16.97.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.97.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.97.1.4 Factory()

```
Entity cmas.ontology.variables.VariableArray.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.variables.Variable.
Referenced by cmas.ontology.base.Entity.Factory(), and cmas.ontology.variables.Variable.Factory().

16.97.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.97.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.97.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.97.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.97.1.9 getCopyOfValue()

`Object cmas.ontology.variables.VariableArray.getCopyOfValue () throws ExceptionType`
Reimplemented from cmas.ontology.variables.Variable.

16.97.1.10 getDescription()

`String cmas.ontology.base.Entity.getDescription ()  [inherited]`

Returns

the entity description.

16.97.1.11 getEntityDescription()

`String cmas.ontology.variables.VariableArray.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.97.1.12 getEntityIconFile()

`String cmas.ontology.variables.VariableArray.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.97.1.13 getID()

`String cmas.ontology.base.Entity.getID ()  [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.97.1.14 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID ()  [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.97.1.15 getIterator()

`Iterator cmas.ontology.variables.VariableArray.getIterator ()`
Reimplemented from cmas.ontology.variables.Variable.

16.97.1.16 getIteratorObject()

`Variable cmas.ontology.variables.VariableArray.getIteratorObject ()`
Reimplemented from cmas.ontology.variables.Variable.

16.97.1.17 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.97.1.18 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.97.1.19 getNext()

```
Object cmas.ontology.variables.VariableArray.getNext (
            Iterator iterator) throws ExceptionVariable
```

Reimplemented from cmas.ontology.variables.Variable.

16.97.1.20 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.97.1.21 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.97.1.22 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.97.1.23 getStringValue()

```
String cmas.ontology.variables.VariableArray.getStringValue () throws ExceptionType, ExceptionVariable
```
Reimplemented from cmas.ontology.variables.Variable.

16.97.1.24 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```
Get specific entity type.

Returns

entity type.

16.97.1.25 getValue()

```
Object cmas.ontology.variables.VariableArray.getValue ()
```
Reimplemented from cmas.ontology.variables.Variable.

16.97.1.26 hasNext()

```
boolean cmas.ontology.variables.VariableArray.hasNext (
            Iterator iterator)
```
Reimplemented from cmas.ontology.variables.Variable.

16.97.1.27 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```
Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.97.1.28 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```
Reimplemented from cmas.ontology.base.Visualisation.

16.97.1.29 Serialize()

```
JsonObject cmas.ontology.variables.VariableArray.Serialize ()
```
Reimplemented from cmas.ontology.variables.Variable.

16.97.1.30 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.97.1.31 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.97.1.32 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```
Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.97.1.33 setName()

```
void cmas.ontology.variables.VariableArray.setName (
            String Name)
```
set entity name.

Parameters

Name	the new name.

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.getName(), and setName().
Referenced by setName().

16.97.1.34 setValue()

```
void cmas.ontology.variables.VariableArray.setValue (
            Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue,
ExceptionConstant
```
Reimplemented from cmas.ontology.variables.Variable.

16.97.1.35 setValueUndoable()

```
void cmas.ontology.variables.VariableArray.setValueUndoable (
            Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue,
ExceptionConstant
```

Reimplemented from cmas.ontology.variables.Variable.

16.97.1.36 valueOf()

```
Object cmas.ontology.variables.VariableArray.valueOf (
            String value) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
```

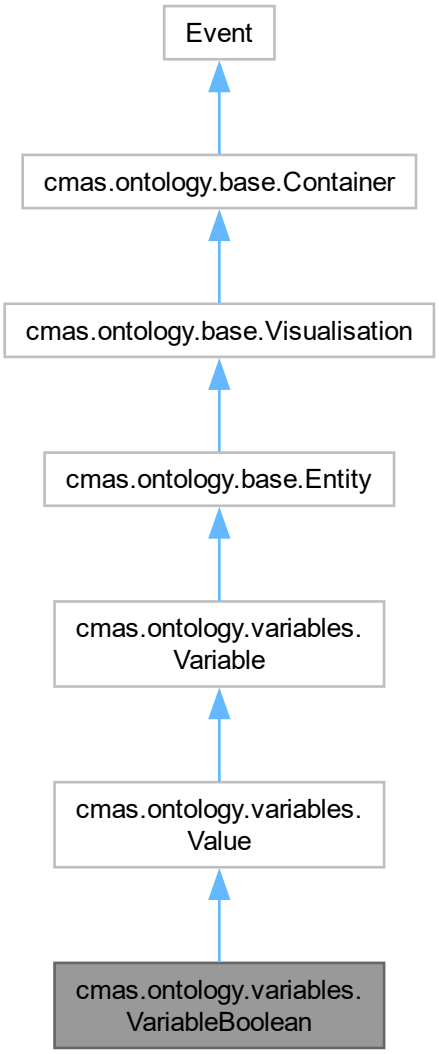
Reimplemented from cmas.ontology.variables.Variable.

The documentation for this class was generated from the following file:

- VariableArray.java

16.98 cmas.ontology.variables.VariableBoolean Class Reference

Inheritance diagram for cmas.ontology.variables.VariableBoolean:



Public Member Functions

- **VariableBoolean** (String Name) throws ExceptionType, ExceptionConstant
- **VariableBoolean** (String Name, Boolean DefaultValue) throws ExceptionType, ExceptionConstant
- **VariableBoolean** (VariableBoolean variable, boolean instance) throws ExceptionType
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- Iterator getIterator ()
- Object getCopyOfValue () throws ExceptionType
- String getStringValue () throws ExceptionType
- ValueTypes **getValueType** ()
- void **setReferenceProperty** (Property<?> prop)

- Property<?> **getReferenceProperty** ()
- boolean **isReferenceProperty** ()
- Boolean **isBoolean** ()
- Boolean **isInteger** ()
- Boolean **isReal** ()
- Boolean **isString** ()
- synchronized Object getValue ()
- void setValue (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void **setValue** (Object newValue, boolean undoable) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void setValueUndoable (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- Object valueOf (String s)
- String **getUnit** ()
- void **setUnit** (String value)
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- String **getUpperBound** ()
- void **setUpperBound** (String upperBound)
- String **getLowerBound** ()
- void **setLowerBound** (String lowerBound)
- boolean **isWithinRange** (Object testValue) throws ExceptionType
- Integer **parseInteger** (String Value, Integer Default)
- boolean **parseBoolean** (String Value, Boolean Default)
- Double **parseReal** (String Value, Double Default)
- JsonObject Serialize ()
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- Variable **getIteratorObject** ()
- boolean **hasNext** (Iterator iterator)
- Object **getNext** (Iterator iterator) throws ExceptionVariable, ExceptionType, ExceptionConstant
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()
- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)

- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static ValueTypes **getValueType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Member Functions

- void **setIgnoreReadOnly** (boolean value)

Protected Attributes

- Property<?> **ReferenceProperty**
- ValueTypes **ValueType**
- String **UpperBound**
- String **LowerBound**
- String **Unit**
- Action **Action**
- ValueRange **upperRangeExpression**
- ValueRange **lowerRangeExpression**
- Boolean **ignoreReadOnly** = true
- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

Static Protected Attributes

- static final Boolean **ignoreTypeCheck** = false
- static final Boolean **ignoreRangeCheck** = false

16.98.1 Member Function Documentation

16.98.1.1 copy()

```
Entity cmas.ontology.variables.VariableBoolean.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.variables.Variable.

16.98.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.98.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.98.1.4 Factory()

```
Entity cmas.ontology.variables.Value.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.variables.Variable.
Referenced by cmas.ontology.base.Entity.Factory(), and cmas.ontology.variables.Variable.Factory().

16.98.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.98.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	

EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.98.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.98.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.98.1.9 getCopyOfValue()

```
Object cmas.ontology.variables.Value.getCopyOfValue () throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.98.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.98.1.11 getEntityDescription()

```
String cmas.ontology.variables.VariableBoolean.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.98.1.12 getEntityIconFile()

```
String cmas.ontology.variables.VariableBoolean.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.98.1.13 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.98.1.14 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.98.1.15 getIterator()

```
Iterator cmas.ontology.variables.VariableBoolean.getIterator ()
```

Reimplemented from cmas.ontology.variables.Variable.

16.98.1.16 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.98.1.17 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.98.1.18 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.98.1.19 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.98.1.20 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.98.1.21 getStringValue()

```
String cmas.ontology.variables.Value.getStringValue () throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.98.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.98.1.23 getValue()

```
synchronized Object cmas.ontology.variables.Value.getValue ()  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.98.1.24 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.98.1.25 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.98.1.26 Serialize()

```
JsonObject cmas.ontology.variables.Value.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.98.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.98.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.98.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.98.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.98.1.31 setValue()

```
void cmas.ontology.variables.Value.setValue (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
[inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.98.1.32 setValueUndoable()

```
void cmas.ontology.variables.Value.setValueUndoable (
```

```
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
[inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.98.1.33 valueOf()

```
Object cmas.ontology.variables.Value.valueOf (
            String s)  [inherited]
```

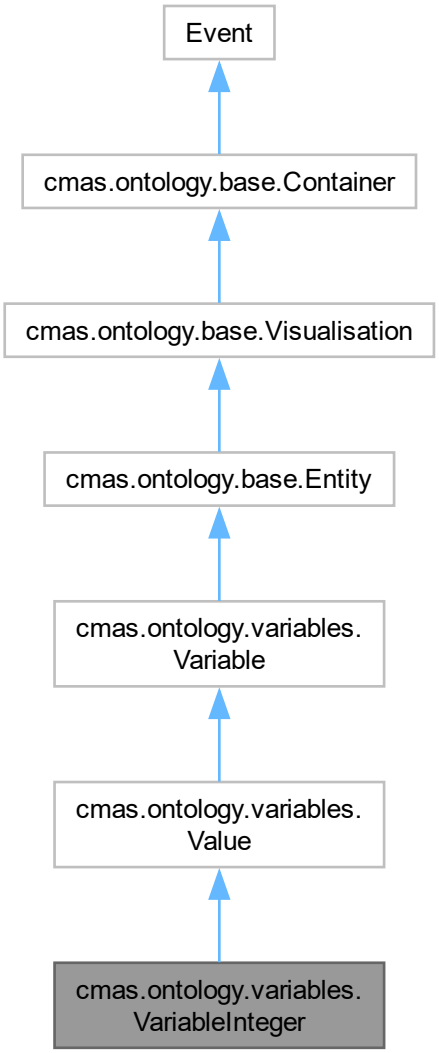
Reimplemented from cmas.ontology.variables.Variable.

The documentation for this class was generated from the following file:

- VariableBoolean.java

16.99 cmas.ontology.variables.VariableInteger Class Reference

Inheritance diagram for cmas.ontology.variables.VariableInteger:



Public Member Functions

- **VariableInteger** (String Name) throws ExceptionType, ExceptionConstant
- **VariableInteger** (String Name, Integer DefaultValue) throws ExceptionType, ExceptionConstant
- **VariableInteger** (VariableInteger variable, boolean instance) throws ExceptionType
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- Iterator getIterator ()
- Object getCopyOfValue () throws ExceptionType
- String getStringValue () throws ExceptionType
- ValueTypes **getValueType** ()
- void **setReferenceProperty** (Property<?> prop)
- Property<?> **getReferenceProperty** ()
- boolean **isReferenceProperty** ()
- Boolean **isBoolean** ()
- Boolean **isInteger** ()
- Boolean **isReal** ()
- Boolean **isString** ()
- synchronized Object getValue ()
- void setValue (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void **setValue** (Object newValue, boolean undoable) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void setValueUndoable (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- Object valueOf (String s)
- String **getUnit** ()
- void **setUnit** (String value)
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- String **getUpperBound** ()
- void **setUpperBound** (String upperBound)
- String **getLowerBound** ()
- void **setLowerBound** (String lowerBound)
- boolean **isWithinRange** (Object testValue) throws ExceptionType
- Integer **parseInteger** (String Value, Integer Default)
- boolean **parseBoolean** (String Value, Boolean Default)
- Double **parseReal** (String Value, Double Default)
- JsonObject Serialize ()
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- Variable **getIteratorObject** ()
- boolean **hasNext** (Iterator iterator)
- Object **getNext** (Iterator iterator) throws ExceptionVariable, ExceptionType, ExceptionConstant
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()

- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()

- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static ValueTypes **getValueType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Member Functions

- void **setIgnoreReadOnly** (boolean value)

Protected Attributes

- Property<?> **ReferenceProperty**
- ValueTypes **ValueType**
- String **UpperBound**
- String **LowerBound**
- String **Unit**
- Action **Action**
- ValueRange **upperRangeExpression**
- ValueRange **lowerRangeExpression**
- Boolean **ignoreReadOnly** = true
- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

Static Protected Attributes

- static final Boolean **ignoreTypeCheck** = false
- static final Boolean **ignoreRangeCheck** = false

16.99.1 Member Function Documentation

16.99.1.1 copy()

```
Entity cmas.ontology.variables.VariableInteger.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.variables.Variable.

16.99.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.99.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.99.1.4 Factory()

```
Entity cmas.ontology.variables.Value.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```
Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.variables.Variable.
Referenced by cmas.ontology.base.Entity.Factory(), and cmas.ontology.variables.Variable.Factory().

16.99.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```
Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.99.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.99.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.99.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.99.1.9 getCopyOfValue()

```
Object cmas.ontology.variables.Value.getCopyOfValue () throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.99.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.99.1.11 getEntityDescription()

```
String cmas.ontology.variables.VariableInteger.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.99.1.12 getEntityIconFile()

```
String cmas.ontology.variables.VariableInteger.getEntityIconFile ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.99.1.13 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```
Get the ID of the entity.

Returns

a string with current ID.

16.99.1.14 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.99.1.15 getIterator()

```
Iterator cmas.ontology.variables.VariableInteger.getIterator ()
```
Reimplemented from cmas.ontology.variables.Variable.

16.99.1.16 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.99.1.17 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.99.1.18 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.99.1.19 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.99.1.20 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.99.1.21 getStringValue()

```
String cmas.ontology.variables.Value.getStringValue () throws ExceptionType [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.99.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.99.1.23 getValue()

```
synchronized Object cmas.ontology.variables.Value.getValue () [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.99.1.24 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.99.1.25 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.99.1.26 Serialize()

```
JsonObject cmas.ontology.variables.Value.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.99.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.99.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.99.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.99.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.99.1.31 setValue()

```
void cmas.ontology.variables.Value.setValue (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
[inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.99.1.32 setValueUndoable()

```
void cmas.ontology.variables.Value.setValueUndoable (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
[inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.99.1.33 valueOf()

```
Object cmas.ontology.variables.Value.valueOf (
            String s)  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.
The documentation for this class was generated from the following file:

- VariableInteger.java

16.100 cmas.ontology.interfaces.InterfaceBase.VariableMatchingType Enum Reference

Public Attributes

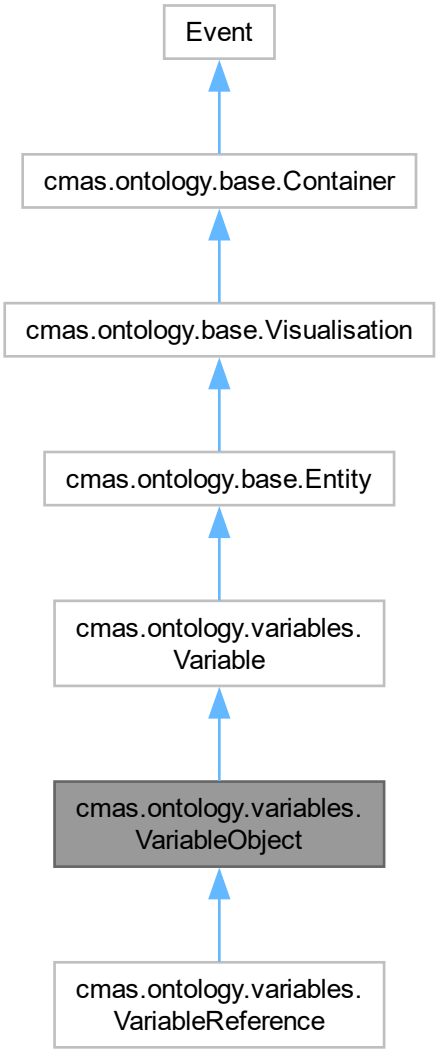
- **REQUIRED**
- **OPTIONAL**
- **IGNORE**

The documentation for this enum was generated from the following file:

- InterfaceBase.java

16.101 cmas.ontology.variables.VariableObject Class Reference

Inheritance diagram for cmas.ontology.variables.VariableObject:



Public Member Functions

- **VariableObject** (String Name) throws ExceptionVariable, ExceptionType
- **VariableObject** (String Name, VariableObject DefaultValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- **VariableObject** (VariableObject variableobject, boolean instance) throws ExceptionType
- Object getValue ()
- Integer **getSize** ()
- void **clear** ()
- void **addItem** (Variable newVar)
- void setValueUndoable (Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant

- void setValue (Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- void **replaceValue** (Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- String getStringValue () throws ExceptionType, ExceptionVariable
- Container makeInstance ()
- String **getJSONStringValue** () throws ExceptionType, ExceptionVariable
- Object valueOf (String value) throws ExceptionVariable, ExceptionType, ExceptionNullValue, ExceptionConstant
- Object getCopyOfValue () throws ExceptionType
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- Iterator getIterator ()
- boolean hasNext (Iterator iterator)
- Object getNext (Iterator iterator) throws ExceptionVariable
- Variable getIteratorObject ()
- Entity **getOntologyObjectReference** ()
- void **setOntologyObjectReference** (Entity ontologyObjectReference) throws ExceptionParameters
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()
- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- JsonObject Serialize ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()

- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- String getLinkedInstanceID ()

Static Public Member Functions

- static Variable **fromJson** (String Name, JsonElement json) throws ExceptionVariable, ExceptionType, ExceptionConstant, ExceptionNullValue
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

16.101.1 Member Function Documentation

16.101.1.1 copy()

```
Entity cmas.ontology.variables.VariableObject.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.variables.Variable.
Reimplemented in cmas.ontology.variables.VariableReference.

16.101.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.101.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.101.1.4 Factory()

```
Entity cmas.ontology.variables.Variable.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
Reimplemented in cmas.ontology.variables.Value, and cmas.ontology.variables.VariableArray.
References cmas.ontology.base.Entity.Entity(), cmas.ontology.variables.Value.Factory(), and cmas.ontology.variables.VariableArray.Fa

16.101.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.101.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.101.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.101.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.101.1.9 getCopyOfValue()

`Object cmas.ontology.variables.VariableObject.getCopyOfValue () throws ExceptionType`
Reimplemented from cmas.ontology.variables.Variable.

16.101.1.10 getDescription()

`String cmas.ontology.base.Entity.getDescription ()  [inherited]`

Returns

the entity description.

16.101.1.11 getEntityDescription()

`String cmas.ontology.variables.VariableObject.getEntityDescription ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.101.1.12 getEntityIconFile()

`String cmas.ontology.variables.VariableObject.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.101.1.13 getID()

`String cmas.ontology.base.Entity.getID ()  [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.101.1.14 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID ()  [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.101.1.15 getIterator()

`Iterator cmas.ontology.variables.VariableObject.getIterator ()`
Reimplemented from cmas.ontology.variables.Variable.

16.101.1.16 getIteratorObject()

`Variable cmas.ontology.variables.VariableObject.getIteratorObject ()`
Reimplemented from cmas.ontology.variables.Variable.

16.101.1.17 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.101.1.18 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.101.1.19 getNext()

```
Object cmas.ontology.variables.VariableObject.getNext (
            Iterator iterator) throws ExceptionVariable
```
Reimplemented from cmas.ontology.variables.Variable.

16.101.1.20 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.101.1.21 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.101.1.22 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```
Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.101.1.23 getStringValue()

```
String cmas.ontology.variables.VariableObject.getStringValue () throws ExceptionType, ExceptionVariable
```
Reimplemented from cmas.ontology.variables.Variable.

16.101.1.24 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```
Get specific entity type.

Returns

entity type.

16.101.1.25 getValue()

```
Object cmas.ontology.variables.VariableObject.getValue ()
```
Reimplemented from cmas.ontology.variables.Variable.

16.101.1.26 hasNext()

```
boolean cmas.ontology.variables.VariableObject.hasNext (
            Iterator iterator)
```
Reimplemented from cmas.ontology.variables.Variable.

16.101.1.27 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```
Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.101.1.28 makeInstance()

```
Container cmas.ontology.variables.VariableObject.makeInstance ()
```
Reimplemented from cmas.ontology.base.Container.

16.101.1.29 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```
Reimplemented from cmas.ontology.base.Visualisation.

16.101.1.30 Serialize()

```
JsonObject cmas.ontology.variables.Variable.Serialize ()  [inherited]
```
Reimplemented from cmas.ontology.base.Entity.

16.101.1.31 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.101.1.32 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.101.1.33 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```
Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.101.1.34 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```
set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.101.1.35 setValue()

```
void cmas.ontology.variables.VariableObject.setValue (
            Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue,
ExceptionConstant
```

Reimplemented from cmas.ontology.variables.Variable.

16.101.1.36 setValueUndoable()

```
void cmas.ontology.variables.VariableObject.setValueUndoable (
            Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue,
ExceptionConstant
```

Reimplemented from cmas.ontology.variables.Variable.

16.101.1.37 valueOf()

```
Object cmas.ontology.variables.VariableObject.valueOf (
            String value) throws ExceptionVariable, ExceptionType, ExceptionNullValue, ExceptionConstant
```

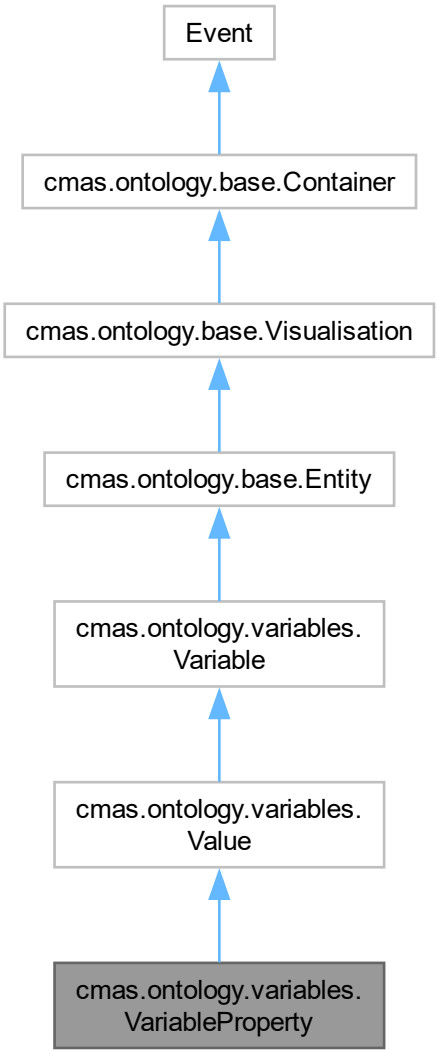
Reimplemented from cmas.ontology.variables.Variable.

The documentation for this class was generated from the following file:

- VariableObject.java

16.102 cmas.ontology.variables.VariableProperty Class Reference

Inheritance diagram for cmas.ontology.variables.VariableProperty:



Public Member Functions

- **VariableProperty** (Property<?> Prop) throws ExceptionType, ExceptionConstant
- String getStringValue () throws ExceptionType
- Object getCopyOfValue () throws ExceptionType
- Object getValue ()
- void setValue (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- Iterator getIterator ()
- ValueTypes **getValueType** ()
- void **setReferenceProperty** (Property<?> prop)

- Property<?> **getReferenceProperty** ()
- boolean **isReferenceProperty** ()
- Boolean **isBoolean** ()
- Boolean **isInteger** ()
- Boolean **isReal** ()
- Boolean **isString** ()
- void **setValue** (Object newValue, boolean undoable) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void setValueUndoable (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- Object valueOf (String s)
- String **getUnit** ()
- void **setUnit** (String value)
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- String **getUpperBound** ()
- void **setUpperBound** (String upperBound)
- String **getLowerBound** ()
- void **setLowerBound** (String lowerBound)
- boolean **isWithinRange** (Object testValue) throws ExceptionType
- Integer **parseInteger** (String Value, Integer Default)
- boolean **parseBoolean** (String Value, Boolean Default)
- Double **parseReal** (String Value, Double Default)
- JsonObject Serialize ()
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- Variable **getIteratorObject** ()
- boolean **hasNext** (Iterator iterator)
- Object **getNext** (Iterator iterator) throws ExceptionVariable, ExceptionType, ExceptionConstant
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()
- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()

- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static ValueTypes **getValueType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Member Functions

- void **setIgnoreReadOnly** (boolean value)

Protected Attributes

- Property<?> **property**
- Property<?> **ReferenceProperty**
- ValueTypes **ValueType**
- String **UpperBound**
- String **LowerBound**
- String **Unit**
- Action **Action**
- ValueRange **upperRangeExpression**
- ValueRange **lowerRangeExpression**
- Boolean **ignoreReadOnly** = true
- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

Static Protected Attributes

- static final Boolean **ignoreTypeCheck** = false
- static final Boolean **ignoreRangeCheck** = false

16.102.1 Member Function Documentation

16.102.1.1 copy()

```
Entity cmas.ontology.variables.VariableProperty.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.variables.Variable.

16.102.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.102.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.102.1.4 Factory()

```
Entity cmas.ontology.variables.Value.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.variables.Variable.
Referenced by cmas.ontology.base.Entity.Factory(), and cmas.ontology.variables.Variable.Factory().

16.102.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.102.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	

EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.102.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.102.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.102.1.9 getCopyOfValue()

```
Object cmas.ontology.variables.VariableProperty.getCopyOfValue () throws ExceptionType
```

Reimplemented from cmas.ontology.variables.Value.

16.102.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.102.1.11 getEntityDescription()

```
String cmas.ontology.variables.VariableProperty.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.102.1.12 getEntityIconFile()

```
String cmas.ontology.variables.VariableProperty.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.102.1.13 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.102.1.14 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.102.1.15 getIterator()

`Iterator cmas.ontology.variables.VariableProperty.getIterator ()`
Reimplemented from cmas.ontology.variables.Variable.

16.102.1.16 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.102.1.17 getName()

`String cmas.ontology.base.Entity.getName () [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.102.1.18 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity) [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.102.1.19 getRelations()

`LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations () [inherited]`

Returns

a HashMap of all relation categories and their linked names.

16.102.1.20 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.102.1.21 getStringValue()

```
String cmas.ontology.variables.VariableProperty.getStringValue () throws ExceptionType
```

Reimplemented from cmas.ontology.variables.Value.

16.102.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.102.1.23 getValue()

```
Object cmas.ontology.variables.VariableProperty.getValue ()
```

Reimplemented from cmas.ontology.variables.Value.

16.102.1.24 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.102.1.25 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.102.1.26 Serialize()

```
JsonObject cmas.ontology.variables.Value.Serialize ()  [inherited]
```
Reimplemented from cmas.ontology.variables.Variable.

16.102.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.102.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.102.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```
Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.102.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```
set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.102.1.31 setValue()

```
void cmas.ontology.variables.VariableProperty.setValue (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
```
Reimplemented from cmas.ontology.variables.Value.

16.102.1.32 setValueUndoable()

```
void cmas.ontology.variables.Value.setValueUndoable (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
```

[inherited]
Reimplemented from cmas.ontology.variables.Variable.

16.102.1.33 valueOf()

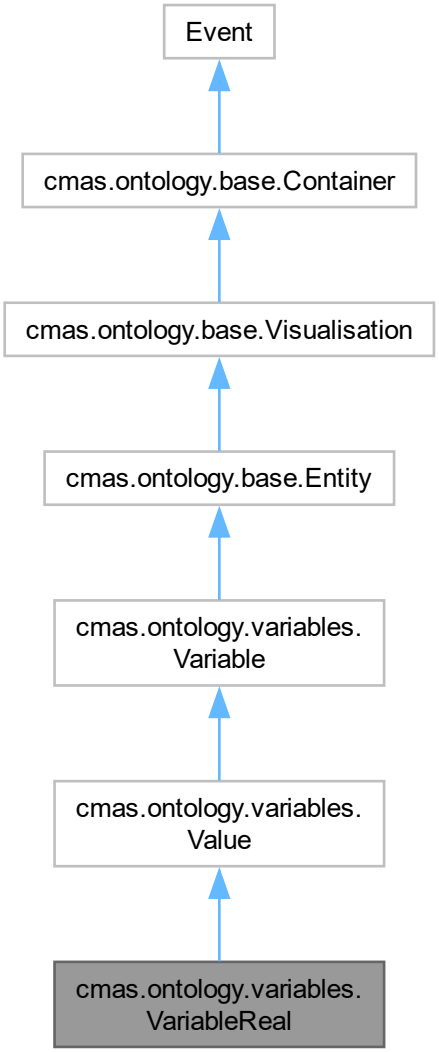
```
Object cmas.ontology.variables.Value.valueOf (
            String s)  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.
The documentation for this class was generated from the following file:

- VariableProperty.java

16.103 cmas.ontology.variables.VariableReal Class Reference

Inheritance diagram for cmas.ontology.variables.VariableReal:



Public Member Functions

- **VariableReal** (String Name) throws ExceptionType, ExceptionConstant
- **VariableReal** (String Name, Double DefaultValue) throws ExceptionType, ExceptionConstant
- **VariableReal** (VariableReal variable, boolean instance) throws ExceptionType
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- Iterator getIterator ()
- Object getCopyOfValue () throws ExceptionType
- String getStringValue () throws ExceptionType
- ValueTypes **getValueType** ()
- void **setReferenceProperty** (Property<?> prop)
- Property<?> **getReferenceProperty** ()
- boolean **isReferenceProperty** ()
- Boolean **isBoolean** ()
- Boolean **isInteger** ()
- Boolean **isReal** ()
- Boolean **isString** ()
- synchronized Object getValue ()
- void setValue (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void **setValue** (Object newValue, boolean undoable) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void setValueUndoable (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- Object valueOf (String s)
- String **getUnit** ()
- void **setUnit** (String value)
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- String **getUpperBound** ()
- void **setUpperBound** (String upperBound)
- String **getLowerBound** ()
- void **setLowerBound** (String lowerBound)
- boolean **isWithinRange** (Object testValue) throws ExceptionType
- Integer **parseInteger** (String Value, Integer Default)
- boolean **parseBoolean** (String Value, Boolean Default)
- Double **parseReal** (String Value, Double Default)
- JsonObject Serialize ()
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- Variable **getIteratorObject** ()
- boolean **hasNext** (Iterator iterator)
- Object **getNext** (Iterator iterator) throws ExceptionVariable, ExceptionType, ExceptionConstant
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()

- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()

- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static ValueTypes **getValueType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Member Functions

- void **setIgnoreReadOnly** (boolean value)

Protected Attributes

- Property<?> **ReferenceProperty**
- ValueTypes **ValueType**
- String **UpperBound**
- String **LowerBound**
- String **Unit**
- Action **Action**
- ValueRange **upperRangeExpression**
- ValueRange **lowerRangeExpression**
- Boolean **ignoreReadOnly** = true
- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

Static Protected Attributes

- static final Boolean **ignoreTypeCheck** = false
- static final Boolean **ignoreRangeCheck** = false

16.103.1 Member Function Documentation

16.103.1.1 copy()

```
Entity cmas.ontology.variables.VariableReal.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.variables.Variable.

16.103.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.103.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.103.1.4 Factory()

```
Entity cmas.ontology.variables.Value.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.variables.Variable.
Referenced by cmas.ontology.base.Entity.Factory(), and cmas.ontology.variables.Variable.Factory().

16.103.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.103.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.103.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.103.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.103.1.9 getCopyOfValue()

```
Object cmas.ontology.variables.Value.getCopyOfValue () throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.103.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.103.1.11 getEntityDescription()

```
String cmas.ontology.variables.VariableReal.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.103.1.12 getEntityIconFile()

```
String cmas.ontology.variables.VariableReal.getEntityIconFile ()
```
Reimplemented from cmas.ontology.base.Visualisation.

16.103.1.13 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```
Get the ID of the entity.

Returns

a string with current ID.

16.103.1.14 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.103.1.15 getIterator()

```
Iterator cmas.ontology.variables.VariableReal.getIterator ()
```
Reimplemented from cmas.ontology.variables.Variable.

16.103.1.16 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.103.1.17 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.103.1.18 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.103.1.19 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.103.1.20 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name) [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.103.1.21 getStringValue()

```
String cmas.ontology.variables.Value.getStringValue () throws ExceptionType [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.103.1.22 getType()

```
EntityType cmas.ontology.base.Entity.getType () [inherited]
```

Get specific entity type.

Returns

entity type.

16.103.1.23 getValue()

```
synchronized Object cmas.ontology.variables.Value.getValue () [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.103.1.24 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep) [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.103.1.25 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.103.1.26 Serialize()

```
JsonObject cmas.ontology.variables.Value.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.103.1.27 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.103.1.28 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.103.1.29 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.103.1.30 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.103.1.31 setValue()

```
void cmas.ontology.variables.Value.setValue (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
[inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.103.1.32 setValueUndoable()

```
void cmas.ontology.variables.Value.setValueUndoable (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
[inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.103.1.33 valueOf()

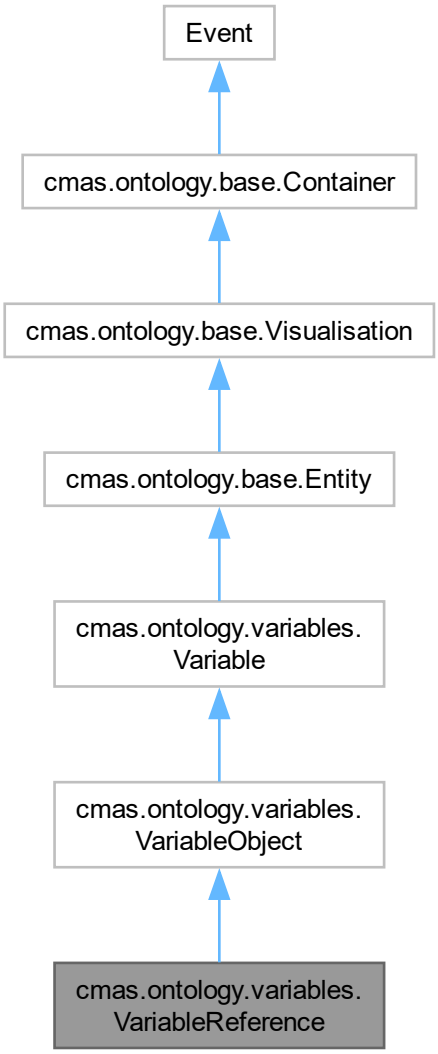
```
Object cmas.ontology.variables.Value.valueOf (
            String s)  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.
The documentation for this class was generated from the following file:

- VariableReal.java

16.104 cmas.ontology.variables.VariableReference Class Reference

Inheritance diagram for cmas.ontology.variables.VariableReference:



Public Member Functions

- **VariableReference** (String Name) throws ExceptionVariable, ExceptionType
- **VariableReference** (String Name, Entity reference) throws ExceptionVariable, ExceptionType
- **VariableReference** (VariableReference reference, boolean instance) throws ExceptionType
- Entity **getReferenceObject** ()
- void **setReferenceObject** (Entity referenceObject)
- Container makeInstance ()
- void setValue (Object newValue) throws ExceptionVariable
- Object getValue ()
- Integer getSize ()
- String getStringValue () throws ExceptionType, ExceptionVariable
- String getJSONStringValue () throws ExceptionType, ExceptionVariable

- Object valueOf (String value) throws ExceptionVariable, ExceptionType, ExceptionNullValue, ExceptionConstant
- Object getCopyOfValue () throws ExceptionType
- Entity copy (boolean instance)
- String getEntityDescription ()
- void **clear** ()
- void **addItem** (Variable newVar)
- void setValueUndoable (Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- void **replaceValue** (Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue, ExceptionConstant
- Container **makeInstance** (Entity linkedEntity)
- String getEntityIconFile ()
- Iterator getIterator ()
- boolean hasNext (Iterator iterator)
- Object getNext (Iterator iterator) throws ExceptionVariable
- Variable getIteratorObject ()
- Entity **getOntologyObjectReference** ()
- void **setOntologyObjectReference** (Entity ontologyObjectReference) throws ExceptionParameters
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()
- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- JsonObject Serialize ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()

- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Variable **fromJson** (String Name, JsonElement json) throws ExceptionVariable, ExceptionType, ExceptionConstant, ExceptionNullValue
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**

- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

16.104.1 Member Function Documentation

16.104.1.1 copy()

```
Entity cmas.ontology.variables.VariableReference.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.104.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.104.1.4 Factory()

```
Entity cmas.ontology.variables.Variable.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
Reimplemented in cmas.ontology.variables.Value, and cmas.ontology.variables.VariableArray.
References cmas.ontology.base.Entity.Entity(), cmas.ontology.variables.Value.Factory(), and cmas.ontology.variables.VariableArray.Fa

16.104.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.104.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.104.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.104.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.104.1.9 getCopyOfValue()

```
Object cmas.ontology.variables.VariableReference.getCopyOfValue () throws ExceptionType
```

Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.10 getDescription()

`String cmas.ontology.base.Entity.getDescription () [inherited]`

Returns

the entity description.

16.104.1.11 getEntityDescription()

`String cmas.ontology.variables.VariableReference.getEntityDescription ()`
Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.12 getEntityIconFile()

`String cmas.ontology.variables.VariableObject.getEntityIconFile () [inherited]`
Reimplemented from cmas.ontology.base.Visualisation.

16.104.1.13 getID()

`String cmas.ontology.base.Entity.getID () [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.104.1.14 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID () [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.104.1.15 getIterator()

`Iterator cmas.ontology.variables.VariableObject.getIterator () [inherited]`
Reimplemented from cmas.ontology.variables.Variable.

16.104.1.16 getIteratorObject()

`Variable cmas.ontology.variables.VariableObject.getIteratorObject () [inherited]`
Reimplemented from cmas.ontology.variables.Variable.

16.104.1.17 getJSONStringValue()

`String cmas.ontology.variables.VariableReference.getJSONStringValue () throws ExceptionType, ExceptionVariable`
Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.18 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.104.1.19 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.104.1.20 getNext()

```
Object cmas.ontology.variables.VariableObject.getNext (
            Iterator iterator) throws ExceptionVariable  [inherited]
```
Reimplemented from cmas.ontology.variables.Variable.

16.104.1.21 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```
Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.104.1.22 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.104.1.23 getSize()

```
Integer cmas.ontology.variables.VariableReference.getSize ()
```
Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.24 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```
Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.104.1.25 getStringValue()

```
String cmas.ontology.variables.VariableReference.getStringValue () throws ExceptionType, ExceptionVariable
```
Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.26 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```
Get specific entity type.

Returns

entity type.

16.104.1.27 getValue()

```
Object cmas.ontology.variables.VariableReference.getValue ()
```
Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.28 hasNext()

```
boolean cmas.ontology.variables.VariableObject.hasNext (
            Iterator iterator)  [inherited]
```
Reimplemented from cmas.ontology.variables.Variable.

16.104.1.29 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```
Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.104.1.30 makeInstance()

```
Container cmas.ontology.variables.VariableReference.makeInstance ()
```
Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.31 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```
Reimplemented from cmas.ontology.base.Visualisation.

16.104.1.32 Serialize()

```
JsonObject cmas.ontology.variables.Variable.Serialize ()  [inherited]
```
Reimplemented from cmas.ontology.base.Entity.

16.104.1.33 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```
Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.104.1.34 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```
setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.104.1.35 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```
Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.104.1.36 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```
set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.104.1.37 setValue()

```
void cmas.ontology.variables.VariableReference.setValue (
            Object newValue) throws ExceptionVariable
```
Reimplemented from cmas.ontology.variables.VariableObject.

16.104.1.38 setValueUndoable()

```
void cmas.ontology.variables.VariableObject.setValueUndoable (
            Object newValue) throws ExceptionType, ExceptionVariable, ExceptionNullValue,
```

ExceptionConstant [inherited]
Reimplemented from cmas.ontology.variables.Variable.

16.104.1.39 valueOf()

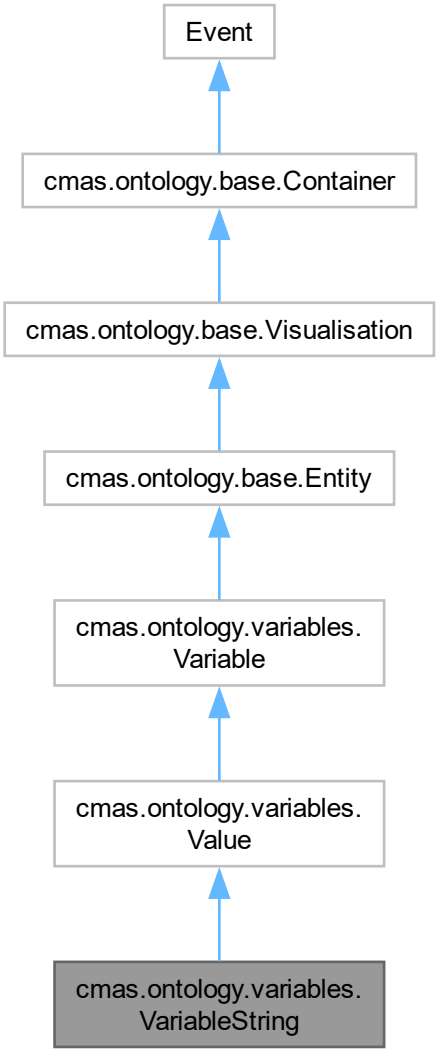
```
Object cmas.ontology.variables.VariableReference.valueOf (
            String value) throws ExceptionVariable, ExceptionType, ExceptionNullValue, ExceptionConstant
```

Reimplemented from cmas.ontology.variables.VariableObject.
The documentation for this class was generated from the following file:

- VariableReference.java

16.105 cmas.ontology.variables.VariableString Class Reference

Inheritance diagram for cmas.ontology.variables.VariableString:



Public Member Functions

- **VariableString** (String Name) throws ExceptionType, ExceptionConstant
- **VariableString** (String Name, String DefaultValue) throws ExceptionType, ExceptionConstant
- **VariableString** (VariableString variable, boolean instance) throws ExceptionType
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- Iterator getIterator ()
- boolean hasNext (Iterator iterator)
- Object getNext (Iterator iterator) throws ExceptionVariable, ExceptionType, ExceptionConstant
- Variable getIteratorObject ()
- Object getCopyOfValue () throws ExceptionType
- String getStringValue () throws ExceptionType
- ValueTypes **getValueType** ()
- void **setReferenceProperty** (Property<?> prop)
- Property<?> **getReferenceProperty** ()
- boolean **isReferenceProperty** ()
- Boolean **isBoolean** ()
- Boolean **isInteger** ()
- Boolean **isReal** ()
- Boolean **isString** ()
- synchronized Object getValue ()
- void setValue (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void **setValue** (Object newValue, boolean undoable) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- void setValueUndoable (Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
- Object valueOf (String s)
- String **getUnit** ()
- void **setUnit** (String value)
- synchronized String **getActions** ()
- synchronized void **setActions** (String actions)
- String **getUpperBound** ()
- void **setUpperBound** (String upperBound)
- String **getLowerBound** ()
- void **setLowerBound** (String lowerBound)
- boolean **isWithinRange** (Object testValue) throws ExceptionType
- Integer **parseInteger** (String Value, Integer Default)
- boolean **parseBoolean** (String Value, Boolean Default)
- Double **parseReal** (String Value, Double Default)
- JsonObject Serialize ()
- boolean **isIteratorObject** ()
- void **isIteratorObject** (boolean v)
- TypeOfVariable **getVariableType** ()
- Boolean **isArray** ()
- Boolean **isPrimitive** ()
- Boolean **isObject** ()
- Boolean **getReadOnly** ()
- Boolean **hasAdapter** ()
- void **setReadOnly** (Boolean ReadOnly)
- Boolean **getStoreLongTerm** ()
- void **setStoreLongTerm** (Boolean storeLongTerm)
- Boolean **getError** ()
- void **setError** (Boolean error)
- String **getAddress** ()

- void **setAddress** (String address)
- String **getSource** ()
- void **setSource** (String source)
- void **setReference** (Variable ref)
- Variable **getReference** ()
- boolean **isReference** ()
- boolean **compare** (Object o)
- boolean **isVariableObjectMatching** (VariableObject variable1, VariableObject variable2)
- void openEntityEditor ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()

- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static ValueTypes **getValueType** (JsonObject job) throws ExceptionType
- static Entity Factory (JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static TypeOfVariable **getVariableType** (JsonObject job) throws ExceptionType
- static TypeOfVariable **getTypeOfVariable** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Member Functions

- void **setIgnoreReadOnly** (boolean value)

Protected Attributes

- Property<?> **ReferenceProperty**
- ValueTypes **ValueType**
- String **UpperBound**
- String **LowerBound**
- String **Unit**
- Action **Action**
- ValueRange **upperRangeExpression**
- ValueRange **lowerRangeExpression**
- Boolean **ignoreReadOnly** = true
- TypeOfVariable **VariableType**
- Boolean **ReadOnly**
- Boolean **StoreLongTerm**
- Boolean **Error**
- String **Address**
- String **Source**
- Adapter **VariableAdapter**
- Variable **Reference**
- Properties **properties**
- Relations **relations**

Static Protected Attributes

- static final Boolean **ignoreTypeCheck** = false
- static final Boolean **ignoreRangeCheck** = false

16.105.1 Member Function Documentation

16.105.1.1 copy()

```
Entity cmas.ontology.variables.VariableString.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.variables.Variable.

16.105.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.105.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```
Reimplemented from cmas.ontology.base.Container.

16.105.1.4 Factory()

```
Entity cmas.ontology.variables.Value.Factory (
            JsonObject job) throws ExceptionType, ExceptionConstant, ExceptionVariable  [static],
[inherited]
```
Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.variables.Variable.
Referenced by cmas.ontology.base.Entity.Factory(), and cmas.ontology.variables.Variable.Factory().

16.105.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```
Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.105.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.105.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.105.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.105.1.9 getCopyOfValue()

```
Object cmas.ontology.variables.Value.getCopyOfValue () throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.105.1.10 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.105.1.11 getEntityDescription()

```
String cmas.ontology.variables.VariableString.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.105.1.12 getEntityIconFile()

`String cmas.ontology.variables.VariableString.getEntityIconFile ()`
Reimplemented from cmas.ontology.base.Visualisation.

16.105.1.13 getID()

`String cmas.ontology.base.Entity.getID ()  [inherited]`
Get the ID of the entity.

Returns

a string with current ID.

16.105.1.14 getInstanceID()

`String cmas.ontology.base.Entity.getInstanceID ()  [inherited]`
A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.105.1.15 getIterator()

`Iterator cmas.ontology.variables.VariableString.getIterator ()`
Reimplemented from cmas.ontology.variables.Variable.

16.105.1.16 getIteratorObject()

`Variable cmas.ontology.variables.VariableString.getIteratorObject ()`
Reimplemented from cmas.ontology.variables.Variable.

16.105.1.17 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]`
Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.105.1.18 getName()

`String cmas.ontology.base.Entity.getName ()  [inherited]`
get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.105.1.19 getNext()

```
Object cmas.ontology.variables.VariableString.getNext (
            Iterator iterator) throws ExceptionVariable, ExceptionType, ExceptionConstant
```
Reimplemented from cmas.ontology.variables.Variable.

16.105.1.20 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.105.1.21 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.105.1.22 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.105.1.23 getStringValue()

```
String cmas.ontology.variables.Value.getStringValue () throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.105.1.24 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.105.1.25 getValue()

```
synchronized Object cmas.ontology.variables.Value.getValue ()  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.105.1.26 hasNext()

```
boolean cmas.ontology.variables.VariableString.hasNext (
            Iterator iterator)
```

Reimplemented from cmas.ontology.variables.Variable.

16.105.1.27 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.105.1.28 openEntityEditor()

```
void cmas.ontology.variables.Variable.openEntityEditor ()  [inherited]
```

Reimplemented from cmas.ontology.base.Visualisation.

16.105.1.29 Serialize()

```
JsonObject cmas.ontology.variables.Value.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.105.1.30 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.105.1.31 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.105.1.32 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.105.1.33 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.105.1.34 setValue()

```
void cmas.ontology.variables.Value.setValue (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
[inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.105.1.35 setValueUndoable()

```
void cmas.ontology.variables.Value.setValueUndoable (
            Object newValue) throws ExceptionNullValue, ExceptionType, ExceptionConstant
[inherited]
```

Reimplemented from cmas.ontology.variables.Variable.

16.105.1.36 valueOf()

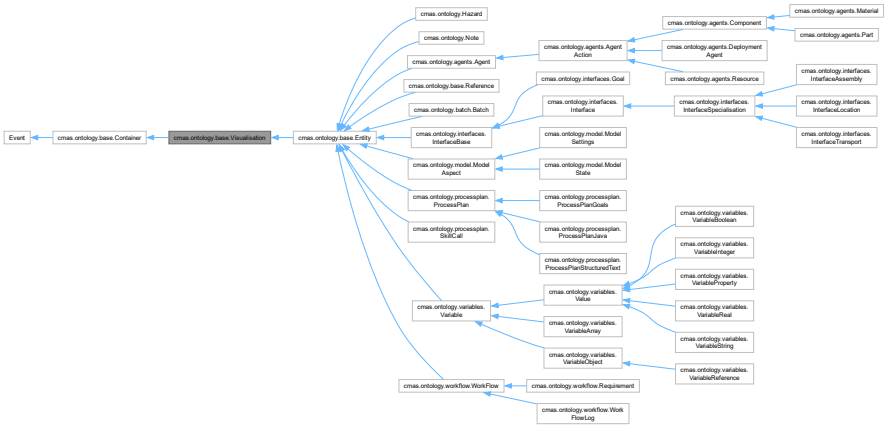
```
Object cmas.ontology.variables.Value.valueOf (
            String s)  [inherited]
```

Reimplemented from cmas.ontology.variables.Variable.
The documentation for this class was generated from the following file:

- VariableString.java

16.106 cmas.ontology.base.Visualisation Class Reference

Inheritance diagram for cmas.ontology.base.Visualisation:



Public Member Functions

- **Visualisation** (EventType TypeOfEvent)
- **Visualisation** (JsonObject job) throws ExceptionType
- **Visualisation** (Visualisation visualisation, boolean instance)
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- abstract void **openEntityEditor** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **delete** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- JsonObject **Serialize** ()
- void **DeSerialize** (JsonObject job) throws ExceptionType
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.106.1 Member Function Documentation

16.106.1.1 getLinkedInstanceID()

`String cmas.ontology.base.Container.getLinkedInstanceID () [inherited]`

Get the modeling entity instance id from a runnable instanced object.

Returns

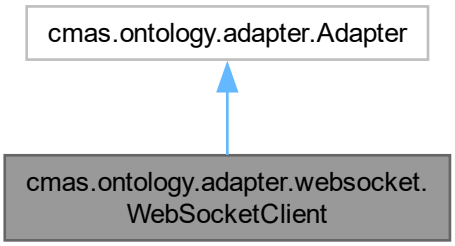
Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

The documentation for this class was generated from the following file:

- Visualisation.java

16.107 cmas.ontology.adapter.websocket.WebSocketClient Class Reference

Inheritance diagram for cmas.ontology.adapter.websocket.WebSocketClient:



Public Member Functions

- **WebSocketClient** (String Source)
- void **setSource** (String Source)
- void **connectToServer** ()
- void **onMessage** (String message)
- void **onOpen** (Session session)
- void **onClose** ()
- String ReadData (String Address, String Name, Object Value) throws Exception
- void WriteData (String Address, String Name, Boolean bValue) throws Exception
- void WriteData (String Address, String Name, String sValue) throws Exception
- void WriteData (String Address, String Name, Integer iValue) throws Exception
- void WriteData (String Address, String Name, Double dValue) throws Exception
- boolean close ()
- void setDebug (boolean bValue)
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)

16.107.1 Member Function Documentation

16.107.1.1 close()

```
boolean cmas.ontology.adapter.websocket.WebSocketClient.close ()
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.2 createNew()

```
Adapter cmas.ontology.adapter.websocket.WebSocketClient.createNew (
            String Source)
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.3 getDefaultValue()

```
String cmas.ontology.adapter.websocket.WebSocketClient.getDefaultValue ()
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.4 getDescription()

```
String cmas.ontology.adapter.websocket.WebSocketClient.getDescription ()
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.5 isValid()

```
boolean cmas.ontology.adapter.websocket.WebSocketClient.isValid (
            String Source)
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.6 ReadData()

```
String cmas.ontology.adapter.websocket.WebSocketClient.ReadData (
            String Address,
            String Name,
            Object Value) throws Exception
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.7 setDebug()

```
void cmas.ontology.adapter.websocket.WebSocketClient.setDebug (
            boolean bValue)
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.8 WriteData() [1/4]

```
void cmas.ontology.adapter.websocket.WebSocketClient.WriteData (
            String Address,
            String Name,
            Boolean bValue) throws Exception
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.9 WriteData() [2/4]

```
void cmas.ontology.adapter.websocket.WebSocketClient.WriteData (
            String Address,
            String Name,
            Double dValue) throws Exception
```
Implements cmas.ontology.adapter.Adapter.

16.107.1.10 WriteData() [3/4]

```
void cmas.ontology.adapter.websocket.WebSocketClient.WriteData (
            String Address,
            String Name,
            Integer iValue) throws Exception
```

Implements cmas.ontology.adapter.Adapter.

16.107.1.11 WriteData() [4/4]

```
void cmas.ontology.adapter.websocket.WebSocketClient.WriteData (
            String Address,
            String Name,
            String sValue) throws Exception
```

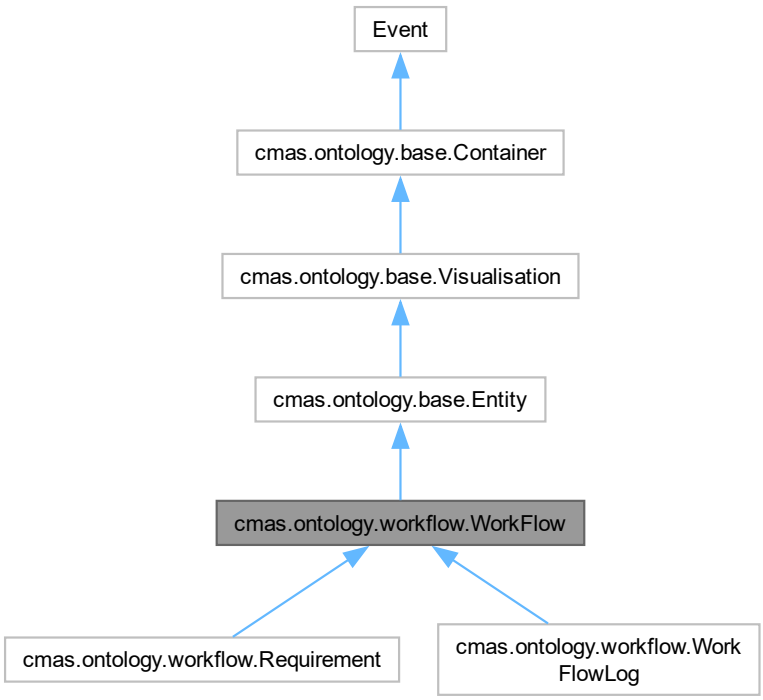
Implements cmas.ontology.adapter.Adapter.

The documentation for this class was generated from the following file:

- WebSocketClient.java

16.108 cmas.ontology.workflow.WorkFlow Class Reference

Inheritance diagram for cmas.ontology.workflow.WorkFlow:



Classes

- enum SpecificTypes
- enum WorkFlowTypes

Public Member Functions

- **WorkFlow** (SpecificTypes type)
- **WorkFlow** (WorkFlow entity, boolean instance)
- WorkFlowTypes **getWorkFlowType** ()
- void **setWorkFlowType** (WorkFlowTypes workFlowType)
- SpecificTypes **getSpecificType** ()
- JsonObject Serialize ()
- abstract Entity copy (boolean instance)
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- abstract String **getEntityDescription** ()
- abstract String **getEntityIconFile** ()
- abstract void **openEntityEditor** ()
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()

- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()
- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static SpecificTypes **getSpecificType** (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.108.1 Member Function Documentation

16.108.1.1 copy()

```
abstract Entity cmas.ontology.base.Entity.copy (
            boolean instance)  [abstract], [inherited]
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented in cmas.ontology.agents.DeploymentAgent, cmas.ontology.agents.Material, cmas.ontology.agents.Part, cmas.ontology.agents.Resource, cmas.ontology.base.Reference, cmas.ontology.batch.Batch, cmas.ontology.Hazard, cmas.ontology.interfaces.Goal, cmas.ontology.interfaces.Interface, cmas.ontology.interfaces.InterfaceAssembly, cmas.ontology.interfaces.InterfaceLocation, cmas.ontology.interfaces.InterfaceSpecialisation, cmas.ontology.interfaces.InterfaceTrans cmas.ontology.model.ModelSettings, cmas.ontology.model.ModelState, cmas.ontology.Note, cmas.ontology.processplan.ProcessPlar cmas.ontology.processplan.ProcessPlanJava, cmas.ontology.processplan.ProcessPlanStructuredText, cmas.ontology.processplan.Sk cmas.ontology.variables.Variable, cmas.ontology.variables.VariableArray, cmas.ontology.variables.VariableBoolean, cmas.ontology.variables.VariableInteger, cmas.ontology.variables.VariableObject, cmas.ontology.variables.VariableProperty, cmas.ontology.variables.VariableReal, cmas.ontology.variables.VariableReference, cmas.ontology.variables.VariableString, cmas.ontology.workflow.Requirement, and cmas.ontology.workflow.WorkFlowLog.
References Entity().

16.108.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.108.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.108.1.4 Factory()

```
Entity cmas.ontology.workflow.WorkFlow.Factory (
            JsonObject job) throws ExceptionType  [static]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.108.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.108.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	

EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.108.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.108.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.108.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.108.1.10 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.108.1.11 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.108.1.12 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.108.1.13 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.108.1.14 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.108.1.15 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
() [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.108.1.16 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.108.1.17 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.108.1.18 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.108.1.19 Serialize()

```
JsonObject cmas.ontology.workflow.WorkFlow.Serialize ()
```

Reimplemented from cmas.ontology.base.Entity.

16.108.1.20 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.108.1.21 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.108.1.22 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.108.1.23 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

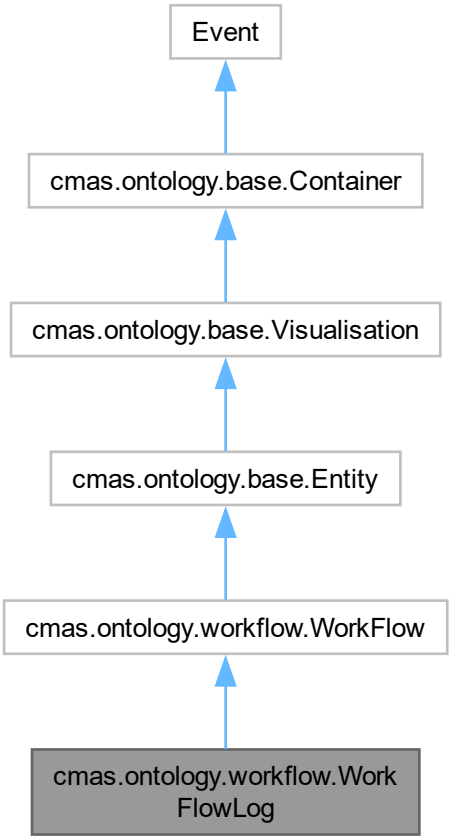
Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().
The documentation for this class was generated from the following file:

- WorkFlow.java

16.109 cmas.ontology.workflow.WorkFlowLog Class Reference

Inheritance diagram for cmas.ontology.workflow.WorkFlowLog:



Public Member Functions

- **WorkFlowLog** (WorkFlowTypes type, String Name, String Target, String Information)
- **WorkFlowLog** (WorkFlowLog entity, boolean instance)
- String **getTime** ()
- void **setTime** (String time)
- void setTime ()
- String **getTarget** ()
- void **setTarget** (String target)

- String **getInformation** ()
- void **setInformation** (String information)
- Entity copy (boolean instance)
- String getEntityDescription ()
- String getEntityIconFile ()
- void openEntityEditor ()
- SpecificTypes **getSpecificType** ()
- WorkFlowTypes **getWorkFlowType** ()
- void **setWorkFlowType** (WorkFlowTypes workFlowType)
- JsonObject Serialize ()
- void delete ()
- String **toString** ()
- String getID ()
- void setID (String ID)
- void setID ()
- String getName ()
- void setName (String Name)
- String **getParentName** ()
- void **setParent** (Entity ent)
- String getDescription ()
- void setDescription (String Description)
- String getInstanceID ()
- EntityType getType ()
- Properties **getProperties** ()
- Property<?> **getProperty** (String Name)
- Property<?> **getPropertyIgnoreCase** (String Name)
- void **addProperty** (Property<?> Item)
- boolean **isCompatibleRelation** (String Name, Entity ent)
- boolean hasRelation (Entity relationEntity, boolean deep)
- LinkedHashMap< String, ArrayList< RelationObject > > getRelations ()
- Collection< ArrayList< RelationObject > > getAllRelations ()
- String getRelationName (Entity entity)
- ArrayList< RelationObject > getSpecificRelations (String Name)
- Entity findRelationByName (String RelationName, String EntityName)
- Entity findRelationByNameIgnoreCase (String RelationName, String EntityName)
- ArrayList< Entity > **getSpecificRelationType** (String Name)
- void **addSpecificRelation** (String Name, Entity ent)
- void **addSpecificRelation** (String Name, ArrayList<? extends Entity > list)
- void **removeSpecificRelation** (String Name, Entity ent)
- void **removeSpecificRelation** (Entity ent)
- void generateUniqueIDs ()
- void **eventReflection** (EventType TypeOfEvent, EventCause CauseOfEvent, Object Argument, Object an↩ Owner)
- void DeSerialize (JsonObject job) throws ExceptionType
- ImageView **getEntityIcon** ()
- void **copyProperties** (Container container, boolean instance)
- Long **getDebugId** ()
- void **setOverride** (Boolean value)
- Boolean **getProtected** ()
- void **setProtected** (Boolean protectedValue)
- Object **getUserData** ()
- void **setUserData** (Object userData)
- String **getClassName** ()
- void **setClassName** (String newName)
- Relations **getRealtionsObject** ()

- ArrayList< BreakPoint > **getBreakPointList** ()
- void **setBreakPointList** (ArrayList< BreakPoint > breakPointList)
- Boolean **isInstance** ()
- Container **makeInstance** (Entity linkedEntity)
- Container **makeInstance** ()
- String getLinkedInstanceID ()

Static Public Member Functions

- static Entity Factory (JsonObject job) throws ExceptionType
- static Entity **Factory** (String job) throws ExceptionType, ExceptionConstant, ExceptionVariable
- static SpecificTypes **getSpecificType** (JsonObject job) throws ExceptionType
- static synchronized void **resetDebugID** ()

Protected Attributes

- Properties **properties**
- Relations **relations**

16.109.1 Member Function Documentation

16.109.1.1 copy()

```
Entity cmas.ontology.workflow.WorkFlowLog.copy (
            boolean instance)
```

Must be override by all classes extending Entity if instance is true Properties will be created without a link to parent (parent value will be used) and relations without a link.

Parameters

instance	if true a runtime entity is create otherwise a modelling entity

Reimplemented from cmas.ontology.base.Entity.

16.109.1.2 delete()

```
void cmas.ontology.base.Entity.delete ()  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.109.1.3 DeSerialize()

```
void cmas.ontology.base.Entity.DeSerialize (
            JsonObject job) throws ExceptionType  [inherited]
```

Reimplemented from cmas.ontology.base.Container.

16.109.1.4 Factory()

```
Entity cmas.ontology.workflow.WorkFlow.Factory (
            JsonObject job) throws ExceptionType  [static], [inherited]
```

Static factory method to create a an object from CMAS ontology based on a JsonObject

Parameters

job	is a JsonObject

Returns

the new object or null

Exceptions

ExceptionType	
ExceptionConstant	
ExceptionVariable	

Reimplemented from cmas.ontology.base.Entity.
References cmas.ontology.base.Entity.Entity().
Referenced by cmas.ontology.base.Entity.Factory().

16.109.1.5 findRelationByName()

```
Entity cmas.ontology.base.Entity.findRelationByName (
            String RelationName,
            String EntityName)  [inherited]
```

Will search for a specific name of an Entity in a specific relation category based on a relation name

Parameters

RelationName	
EntityName	

Returns

Entity or null

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.interfaces.Interface.addRequestToQueue().

16.109.1.6 findRelationByNameIgnoreCase()

```
Entity cmas.ontology.base.Entity.findRelationByNameIgnoreCase (
            String RelationName,
            String EntityName)  [inherited]
```

The same as findRelationByName() but will ignore case of the EntityName

Parameters

RelationName	
EntityName	

Returns

References Entity(), and getSpecificRelations().
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run().

16.109.1.7 generateUniqueIDs()

```
void cmas.ontology.base.Entity.generateUniqueIDs ()  [inherited]
```

Will generate unique IDs for the entity and all relation objects
References setID().

16.109.1.8 getAllRelations()

```
Collection< ArrayList< RelationObject > > cmas.ontology.base.Entity.getAllRelations ()  [inherited]
```

Returns

a list of all relations categories

16.109.1.9 getDescription()

```
String cmas.ontology.base.Entity.getDescription ()  [inherited]
```

Returns

the entity description.

16.109.1.10 getEntityDescription()

```
String cmas.ontology.workflow.WorkFlowLog.getEntityDescription ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.109.1.11 getEntityIconFile()

```
String cmas.ontology.workflow.WorkFlowLog.getEntityIconFile ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.109.1.12 getID()

```
String cmas.ontology.base.Entity.getID ()  [inherited]
```

Get the ID of the entity.

Returns

a string with current ID.

16.109.1.13 getInstanceID()

```
String cmas.ontology.base.Entity.getInstanceID ()  [inherited]
```

A unique runtime ID. The InstanceID is generated automatically by the system when an entity is instantiated.

Returns

runtime instance id

16.109.1.14 getLinkedInstanceID()

```
String cmas.ontology.base.Container.getLinkedInstanceID ()  [inherited]
```

Get the modeling entity instance id from a runnable instanced object.

Returns

Will return null if there is no link, otherwise a string to the unique instanceid of the modeling object.

16.109.1.15 getName()

```
String cmas.ontology.base.Entity.getName ()  [inherited]
```

get current name of the entity.

Returns

current name.

Reimplemented in cmas.ontology.base.Reference.
Referenced by cmas.ontology.processplan.ProcessPlanStructuredText.run(), and cmas.ontology.variables.VariableArray.setName().

16.109.1.16 getRelationName()

```
String cmas.ontology.base.Entity.getRelationName (
            Entity entity)  [inherited]
```

Get the name of the specific relation that host an object.

Parameters

A	relation entity onjecty

Returns

A string with the relation name that host the entity object or null if not found.

References Entity().

16.109.1.17 getRelations()

```
LinkedHashMap< String, ArrayList< RelationObject > > cmas.ontology.base.Entity.getRelations
()  [inherited]
```

Returns

a HashMap of all relation categories and their linked names.

16.109.1.18 getSpecificRelations()

```
ArrayList< RelationObject > cmas.ontology.base.Entity.getSpecificRelations (
            String Name)  [inherited]
```

Get a specific relation category by name

Parameters

Name	of the category

Returns

The specific category or null

Referenced by findRelationByName(), and findRelationByNameIgnoreCase().

16.109.1.19 getType()

```
EntityType cmas.ontology.base.Entity.getType ()  [inherited]
```

Get specific entity type.

Returns

entity type.

16.109.1.20 hasRelation()

```
boolean cmas.ontology.base.Entity.hasRelation (
            Entity relationEntity,
            boolean deep)  [inherited]
```

Checks whether the specified relationEntity is related to this entity.

Parameters

relationEntity	the entity to check for a relation
deep	if `true`, the search is performed recursively and includes relations of all child entities

Returns

`true` if a relation exists; `false` otherwise

References Entity().

16.109.1.21 openEntityEditor()

```
void cmas.ontology.workflow.WorkFlowLog.openEntityEditor ()
```

Reimplemented from cmas.ontology.base.Visualisation.

16.109.1.22 Serialize()

```
JsonObject cmas.ontology.workflow.WorkFlow.Serialize ()  [inherited]
```

Reimplemented from cmas.ontology.base.Entity.

16.109.1.23 setDescription()

```
void cmas.ontology.base.Entity.setDescription (
            String Description)  [inherited]
```

Description is a way for the user to document the entity.

Parameters

Description	a user defined description

16.109.1.24 setID() [1/2]

```
void cmas.ontology.base.Entity.setID ()  [inherited]
```

setID will set a new ID by generating a world" unique identifier.
Referenced by Entity().

16.109.1.25 setID() [2/2]

```
void cmas.ontology.base.Entity.setID (
            String ID)  [inherited]
```

Will set a specific ID.

Parameters

ID	the new ID value.

Referenced by generateUniqueIDs().

16.109.1.26 setName()

```
void cmas.ontology.base.Entity.setName (
            String Name)  [inherited]
```

set entity name.

Parameters

Name	the new name.

Reimplemented in cmas.ontology.base.Reference, and cmas.ontology.variables.VariableArray.
Referenced by Entity().

16.109.1.27 setTime()

```
void cmas.ontology.workflow.WorkFlowLog.setTime ()
```

Will use the local time to set the time with the following format yyyy-MM-dd HH:mm:ss:SSS
The documentation for this class was generated from the following file:

- WorkFlowLog.java

16.110 cmas.ontology.workflow.WorkFlow.WorkFlowTypes Enum Reference

Public Attributes

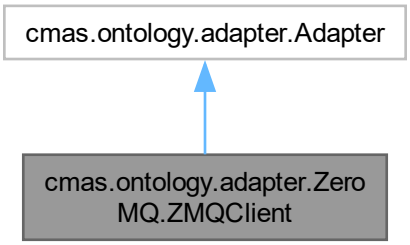
- **NONE**
- **SKILL**
- **CALL**
- **GOAL**
- **ATTACHED**
- **DETACHED**
- **ASSEMBLED**
- **TRANSFERED**
- **CREATED**
- **DEPLOYED**
- **FINISHED**
- **FAILED**
- **UNDEPLOYED**
- **INDEPOT**

The documentation for this enum was generated from the following file:

- WorkFlow.java

16.111 cmas.ontology.adapter.ZeroMQ.ZMQClient Class Reference

Inheritance diagram for cmas.ontology.adapter.ZeroMQ.ZMQClient:



Public Member Functions

- **ZMQClient** (String Source)
- void **SetSource** (String Source)
- String ReadData (String Address, String Name, Object Value)
- void WriteData (String Address, String Name, Boolean bValue)
- void WriteData (String Address, String Name, String sValue)
- void WriteData (String Address, String Name, Integer iValue)
- void WriteData (String Address, String Name, Double dValue)
- void **Write** (String string)
- boolean close ()
- void setDebug (boolean bValue)
- String getDefaultValue ()
- String getDescription ()
- boolean isValid (String Source)
- Adapter createNew (String Source)

16.111.1 Detailed Description

ZeroMQ communications for variables

```
For adapter type of ZeroMQ the specific format is: ZMQ.type:[TCP]:[//host]:[port]
eg, ZMQ:tcp://*:5555
```

Type can one of: DEALER, PAIR, PUB, PULL, PUSH, REP, REQ, ROUTER, STREAM, SUB, XPUB, XSUB

Author

Fredrik Danielsson, University West

16.111.2 Member Function Documentation

16.111.2.1 close()

```
boolean cmas.ontology.adapter.ZeroMQ.ZMQClient.close ()
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.2 createNew()

```
Adapter cmas.ontology.adapter.ZeroMQ.ZMQClient.createNew (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.3 getDefaultValue()

```
String cmas.ontology.adapter.ZeroMQ.ZMQClient.getDefaultValue ()
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.4 getDescription()

```
String cmas.ontology.adapter.ZeroMQ.ZMQClient.getDescription ()
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.5 isValid()

```
boolean cmas.ontology.adapter.ZeroMQ.ZMQClient.isValid (
            String Source)
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.6 ReadData()

```
String cmas.ontology.adapter.ZeroMQ.ZMQClient.ReadData (
            String Address,
            String Name,
            Object Value)
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.7 setDebug()

```
void cmas.ontology.adapter.ZeroMQ.ZMQClient.setDebug (
            boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.8 WriteData() [1/4]

```
void cmas.ontology.adapter.ZeroMQ.ZMQClient.WriteData (
            String Address,
            String Name,
            Boolean bValue)
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.9 WriteData() [2/4]

```
void cmas.ontology.adapter.ZeroMQ.ZMQClient.WriteData (
            String Address,
            String Name,
            Double dValue)
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.10 WriteData() [3/4]

```
void cmas.ontology.adapter.ZeroMQ.ZMQClient.WriteData (
            String Address,
            String Name,
            Integer iValue)
```

Implements cmas.ontology.adapter.Adapter.

16.111.2.11 WriteData() [4/4]

```
void cmas.ontology.adapter.ZeroMQ.ZMQClient.WriteData (
            String Address,
            String Name,
            String sValue)
```

Implements cmas.ontology.adapter.Adapter.

The documentation for this class was generated from the following file:

- ZMQClient.java

Index

abort
    cmas.ontology.processplan.ProcessPlanGoals, 370
    cmas.ontology.processplan.ProcessPlanJava, 380
    cmas.ontology.processplan.ProcessPlanStructuredText, 389
Abstract Interfaces in CMAS, 71
adapterObjects
    cmas.ontology.adapter.AdapterFactory, 199
addRequestToQueue
    cmas.ontology.interfaces.Goal, 264
    cmas.ontology.interfaces.Interface, 280
    cmas.ontology.interfaces.InterfaceAssembly, 290
    cmas.ontology.interfaces.InterfaceBase, 297
    cmas.ontology.interfaces.InterfaceLocation, 306
    cmas.ontology.interfaces.InterfaceSpecialisation, 313
    cmas.ontology.interfaces.InterfaceTransport, 321
Advanced Interface Concepts, 73
Agent action, 7
AgentTypes
    cmas.ontology.Types, 447

BatchTypes
    cmas.ontology.Types, 447

close
    cmas.ontology.adapter.FILE.FileClient, 260
    cmas.ontology.adapter.Modbus.ModbusClient, 336
    cmas.ontology.adapter.OPCUA.OPCUAClient, 347
    cmas.ontology.adapter.REST.RESTClient, 423
    cmas.ontology.adapter.socket.SocketTCPClient, 440
    cmas.ontology.adapter.socket.SocketUDPClient, 443
    cmas.ontology.adapter.SQL.SQLClient, 445
    cmas.ontology.adapter.websocket.WebSocketClient, 540
    cmas.ontology.adapter.ZeroMQ.ZMQClient, 558
CMAS Command Line Arguments, 13
CMAS Examples, 23
CMAS Main Page, 1
cmas.ontology.action.Action, 196
cmas.ontology.action.ActionInterface, 197
cmas.ontology.action.ActionJava, 197
cmas.ontology.action.IndependentClassLoader, 277
    getInstance, 278
cmas.ontology.adapter.Adapter, 198
    ReadData, 198
    setDebug, 198
    WriteData, 198
cmas.ontology.adapter.AdapterFactory, 199
    adapterObjects, 199
cmas.ontology.adapter.FILE.FileClient, 260
    close, 260
    createNew, 260
    getDefaultValue, 260
    getDescription, 261
    isValid, 261
    ReadData, 261
    setDebug, 261
    WriteData, 261
cmas.ontology.adapter.Modbus.ModbusClient, 336
    close, 336
    createNew, 336
    getDefaultValue, 337
    getDescription, 337
    isValid, 337
    ReadData, 337
    setDebug, 337
    WriteData, 337
cmas.ontology.adapter.Modbus.ModbusClient.ADDRESS_TYPE, 199
cmas.ontology.adapter.Modbus.ModbusClient.MODBUS_TYPE, 335
cmas.ontology.adapter.OPCUA.OPCUAClient, 346
    close, 347
    createNew, 347
    getDefaultValue, 347
    getDescription, 347
    getServerHost, 347
    getServerName, 347
    getServerPort, 347
    isValid, 348
    ReadData, 348
    setDebug, 348
    setServerHost, 348
    setServerName, 348
    setServerPort, 348
    WriteData, 349
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 350
    getServerHost, 351
    getServerName, 351
    getServerPort, 351
    ReadData, 351
    setDebug, 351
    setServerHost, 351
    setServerName, 351
    setServerPort, 352

WriteData, 352
cmas.ontology.adapter.OPCUA.OPCUANode, 352
cmas.ontology.adapter.REST.RESTClient, 422
close, 423
createNew, 423
getDefaultValue, 423
getDescription, 423
isValid, 423
ReadData, 423
setDebug, 423
WriteData, 423, 424
cmas.ontology.adapter.REST.RESTClient.REST_TYPE, 422
cmas.ontology.adapter.socket.SocketTCPClient, 440
close, 440
createNew, 440
getDefaultValue, 441
getDescription, 441
isValid, 441
ReadData, 441
setDebug, 441
WriteData, 441
cmas.ontology.adapter.socket.SocketTCPClient.SERVER_TYPE, 424
cmas.ontology.adapter.socket.SocketUDPClient, 442
close, 443
createNew, 443
getDefaultValue, 443
getDescription, 443
isValid, 443
ReadData, 443
setDebug, 443
WriteData, 443, 444
cmas.ontology.adapter.socket.SocketUDPClient.SERVER_TYPE, 424
cmas.ontology.adapter.SQL.SQLClient, 445
close, 445
createNew, 445
getDefaultValue, 445
getDescription, 445
isValid, 446
ReadData, 446
setDebug, 446
WriteData, 446
cmas.ontology.adapter.websocket.WebSocketClient, 539
close, 540
createNew, 540
getDefaultValue, 540
getDescription, 540
isValid, 540
ReadData, 540
setDebug, 540
WriteData, 540, 541
cmas.ontology.adapter.ZeroMQ.ZMQClient, 557
close, 558
createNew, 558
getDefaultValue, 558
getDescription, 558
isValid, 558
ReadData, 558
setDebug, 558
WriteData, 558, 559
cmas.ontology.agentcommunication.ACL.ACLMessage, 195
cmas.ontology.agentcommunication.ACL.ACLMessage.Performative, 360
cmas.ontology.agentcommunication.ACL.MessageHandler, 335
cmas.ontology.agentcommunication.CommunicativeActs, 224
cmas.ontology.agentcommunication.jsoncontent.Content, 233
createParameters, 234
cmas.ontology.agentcommunication.jsoncontent.ContentParameter, 239
cmas.ontology.agentcommunication.jsoncontent.template.ContentBusy, 234
createParameters, 235
cmas.ontology.agentcommunication.jsoncontent.template.ContentDone, 235
createParameters, 236
cmas.ontology.agentcommunication.jsoncontent.template.ContentError, 236
createParameters, 237
cmas.ontology.agentcommunication.jsoncontent.template.ContentInformat 237
createParameters, 238
cmas.ontology.agentcommunication.jsoncontent.template.ContentNegotiat 238
createParameters, 239
cmas.ontology.agentcommunication.jsoncontent.template.ContentProgres 239
createParameters, 240
cmas.ontology.agentcommunication.jsoncontent.template.ContentReplan, 240
createParameters, 241
cmas.ontology.agentcommunication.jsoncontent.template.ContentResult, 241
createParameters, 242
cmas.ontology.agents.Agent, 200
copy, 202
delete, 202
DeSerialize, 202
Factory, 202
findRelationByName, 203
findRelationByNameIgnoreCase, 203
generateUniqueIDs, 203
getAllRelations, 203
getDescription, 203
getID, 204
getInstanceID, 204
getLinkedInstanceID, 204
getName, 204
getRelations, 204
getSpecificRelations, 204

getType, 205
hasRelation, 205
Serialize, 205
setDescription, 205
setID, 205
setName, 206
cmas.ontology.agents.Agent.AgentBaseTypes, 214
cmas.ontology.agents.AgentAction, 206
copy, 209
delete, 210
DeSerialize, 210
Factory, 210
findRelationByName, 210
findRelationByNameIgnoreCase, 211
generateUniqueIDs, 211
getAllRelations, 211
getCommunicative, 211
getDescription, 211
getID, 211
getInstanceID, 212
getLinkedInstanceID, 212
getMessageHandler, 212
getName, 212
getRelations, 212
getSpecificRelations, 212
getType, 213
gotoLine, 213
hasRelation, 213
postMessage, 213
Serialize, 213
setDescription, 213
setID, 214
setName, 214
stopEditor, 214
cmas.ontology.agents.AgentAction.AgentState, 215
cmas.ontology.agents.Component, 224
copy, 226
delete, 227
DeSerialize, 227
Factory, 227
findRelationByName, 227
findRelationByNameIgnoreCase, 228
generateUniqueIDs, 228
getAllRelations, 228
getCommunicative, 228
getDescription, 228
getID, 228
getInstanceID, 229
getLinkedInstanceID, 229
getMessageHandler, 229
getName, 229
getRelations, 229
getSpecificRelations, 229
getType, 230
gotoLine, 230
hasRelation, 230
postMessage, 230
Serialize, 230
setDescription, 230
setID, 231
setName, 231
stopEditor, 231
cmas.ontology.agents.DeploymentAgent, 242
copy, 245
delete, 245
DeSerialize, 245
Factory, 245
findRelationByName, 246
findRelationByNameIgnoreCase, 246
generateUniqueIDs, 246
getAllRelations, 246
getCommunicative, 247
getDescription, 247
getEntityDescription, 247
getEntityIconFile, 247
getID, 247
getInstanceID, 247
getLinkedInstanceID, 247
getMessageHandler, 247
getName, 247
getRelations, 248
getSpecificRelations, 248
getType, 248
gotoLine, 248
hasRelation, 248
onFailed, 249
onFinished, 249
onRun, 249
onStop, 249
onUndeploy, 249
openEntityEditor, 249
postMessage, 249
Serialize, 249
setDescription, 249
setID, 250
setName, 250
stopEditor, 250
cmas.ontology.agents.DeploymentAgent.DeploymentCriteria, 250
cmas.ontology.agents.Material, 327
copy, 330
delete, 330
DeSerialize, 330
Factory, 330
findRelationByName, 330
findRelationByNameIgnoreCase, 331
generateUniqueIDs, 331
getAllRelations, 331
getCommunicative, 331
getDescription, 331
getEntityDescription, 332
getEntityIconFile, 332
getID, 332
getInstanceID, 332
getLinkedInstanceID, 332
getMessageHandler, 332

getName, 332
getRelations, 332
getSpecificRelations, 333
getType, 333
gotoLine, 333
hasRelation, 333
openEntityEditor, 333
postMessage, 334
Serialize, 334
setDescription, 334
setID, 334
setName, 334
stopEditor, 334
cmas.ontology.agents.Part, 352
copy, 355
delete, 355
DeSerialize, 355
Factory, 355
findRelationByName, 355
findRelationByNameIgnoreCase, 356
generateUniqueIDs, 356
getAllRelations, 356
getCommunicative, 356
getDescription, 356
getEntityDescription, 356
getEntityIconFile, 357
getID, 357
getInstanceID, 357
getLinkedInstanceID, 357
getMessageHandler, 357
getName, 357
getRelations, 357
getSpecificRelations, 357
getType, 358
gotoLine, 358
hasRelation, 358
onFailed, 358
onFinished, 358
onRun, 358
onStop, 359
onUndeploy, 359
openEntityEditor, 359
postMessage, 359
Serialize, 359
setDescription, 359
setID, 359
setName, 359
stopEditor, 360
cmas.ontology.agents.Resource, 412
copy, 415
delete, 416
DeSerialize, 416
Factory, 416
findRelationByName, 416
findRelationByNameIgnoreCase, 417
generateUniqueIDs, 417
getAllRelations, 417
getCommunicative, 417
getDescription, 417
getEntityDescription, 417
getEntityIconFile, 417
getID, 418
getInstanceID, 418
getLinkedInstanceID, 418
getMessageHandler, 418
getName, 418
getRelations, 418
getSpecificRelations, 418
getType, 420
gotoLine, 420
hasRelation, 420
onMessage, 420
openEntityEditor, 420
postMessage, 420
Running, 420
Serialize, 421
setDescription, 421
setID, 421
setName, 421
stopEditor, 421
cmas.ontology.base.Container, 232
getLinkedInstanceID, 233
cmas.ontology.base.Entity, 251
copy, 253
delete, 253
DeSerialize, 253
Entity, 252, 253
Factory, 253
findRelationByName, 254
findRelationByNameIgnoreCase, 254
generateUniqueIDs, 255
getAllRelations, 255
getDescription, 255
getID, 255
getInstanceID, 255
getLinkedInstanceID, 255
getName, 255
getRelations, 256
getSpecificRelations, 256
getType, 256
hasRelation, 256
Serialize, 257
setDescription, 257
setID, 257
setName, 257
cmas.ontology.base.Entity.EntityType, 258
cmas.ontology.base.Entity.UID, 450
cmas.ontology.base.Ontology, 345
cmas.ontology.base.Ontology.OntologyObject, 345
cmas.ontology.base.Ontology.OntologyRelation, 346
cmas.ontology.base.Properties, 395
getPropertiesList, 396
cmas.ontology.base.Property< T >, 396
cmas.ontology.base.Property< T >.SpecificPropertyType, 444
cmas.ontology.base.Reference, 398

copy, 400
delete, 400
DeSerialize, 400
Factory, 400
findRelationByName, 400
findRelationByNameIgnoreCase, 401
generateUniqueIDs, 401
getAllRelations, 401
getDescription, 401
getEntityDescription, 401
getEntityIconFile, 402
getID, 402
getInstanceID, 402
getLinkedInstanceID, 402
getName, 402
getRelations, 402
getSpecificRelations, 402
getType, 403
hasRelation, 403
openEntityEditor, 403
Serialize, 403
setDescription, 403
setID, 403, 404
setName, 404
cmas.ontology.base.Relations, 405
cmas.ontology.base.Relations.RelationObject, 404
cmas.ontology.base.Visualisation, 538
getLinkedInstanceID, 539
cmas.ontology.batch.Batch, 216
copy, 218
delete, 218
DeSerialize, 218
Factory, 218
findRelationByName, 218
findRelationByNameIgnoreCase, 219
generateUniqueIDs, 219
getAllRelations, 219
getDescription, 219
getEntityDescription, 219
getEntityIconFile, 220
getID, 220
getInstanceID, 220
getLinkedInstanceID, 220
getName, 220
getRelations, 220
getSpecificRelations, 220
getType, 222
hasRelation, 222
openEntityEditor, 222
Serialize, 222
setDescription, 222
setID, 222, 223
setName, 223
cmas.ontology.batch.BatchItem, 223
cmas.ontology.exceptions.ExceptionConstant, 258
cmas.ontology.exceptions.ExceptionType, 259
cmas.ontology.exceptions.ExceptionVariable, 259
cmas.ontology.Hazard, 270
copy, 272
delete, 272
DeSerialize, 272
Factory, 272
findRelationByName, 272
findRelationByNameIgnoreCase, 273
generateUniqueIDs, 273
getAllRelations, 273
getDescription, 273
getEntityDescription, 273
getEntityIconFile, 274
getID, 274
getInstanceID, 274
getLinkedInstanceID, 274
getName, 274
getRelations, 274
getSpecificRelations, 274
getType, 276
hasRelation, 276
openEntityEditor, 276
Serialize, 276
setDescription, 276
setID, 276, 277
setName, 277
cmas.ontology.interfaces.AbstractInterface, 193
getCommunicative, 194
getMessageHandler, 194
postMessage, 195
cmas.ontology.interfaces.AbstractInterface.AbstractType, 195
cmas.ontology.interfaces.Attachment, 215
cmas.ontology.interfaces.Goal, 262
addRequestToQueue, 264
copy, 264
delete, 265
DeSerialize, 265
Factory, 265
findRelationByName, 265
findRelationByNameIgnoreCase, 266
generateUniqueIDs, 266
getAllRelations, 266
getCommunicative, 266
getDescription, 266
getEntityDescription, 266
getEntityIconFile, 267
getID, 267
getInstanceID, 267
getLinkedInstanceID, 267
getMessageHandler, 267
getName, 267
getRelations, 267
getSpecificRelations, 267
getType, 268
hasRelation, 268
openEntityEditor, 268
postMessage, 268
Serialize, 268
setDescription, 268

setID, 269
setName, 269
cmas.ontology.interfaces.Interface, 278
addRequestToQueue, 280
copy, 280
delete, 282
DeSerialize, 282
Factory, 282
findRelationByName, 282
findRelationByNameIgnoreCase, 283
generateUniqueIDs, 283
getAllRelations, 283
getCommunicative, 283
getDescription, 283
getEntityDescription, 283
getEntityIconFile, 283
getID, 284
getInstanceID, 284
getLinkedInstanceID, 284
getMessageHandler, 284
getName, 284
getRelations, 284
getSpecificRelations, 284
getType, 286
hasRelation, 286
openEntityEditor, 286
postMessage, 286
Serialize, 286
setDescription, 286
setID, 287
setName, 287
cmas.ontology.interfaces.InterfaceAssembly, 287
addRequestToQueue, 290
copy, 290
delete, 290
DeSerialize, 290
Factory, 290
findRelationByName, 291
findRelationByNameIgnoreCase, 291
generateUniqueIDs, 291
getAllRelations, 292
getCommunicative, 292
getDescription, 292
getEntityDescription, 292
getEntityIconFile, 292
getID, 292
getInstanceID, 292
getLinkedInstanceID, 292
getMessageHandler, 292
getName, 293
getRelations, 293
getSpecificRelations, 293
getType, 293
hasRelation, 293
openEntityEditor, 294
postMessage, 294
Serialize, 294
setDescription, 294
setID, 294
setName, 294
cmas.ontology.interfaces.InterfaceBase, 295
addRequestToQueue, 297
copy, 297
delete, 298
DeSerialize, 298
Factory, 298
findRelationByName, 299
findRelationByNameIgnoreCase, 299
generateUniqueIDs, 299
getAllRelations, 299
getCommunicative, 299
getDescription, 300
getID, 300
getInstanceID, 300
getLinkedInstanceID, 300
getMessageHandler, 300
getName, 300
getRelations, 300
getSpecificRelations, 300
getType, 301
hasRelation, 301
postMessage, 301
Serialize, 301
setDescription, 301
setID, 303
setName, 303
cmas.ontology.interfaces.InterfaceBase.InterfaceState, 318
cmas.ontology.interfaces.InterfaceBase.VariableMatchingType, 493
cmas.ontology.interfaces.InterfaceLocation, 303
addRequestToQueue, 306
copy, 306
delete, 306
DeSerialize, 306
Factory, 306
findRelationByName, 307
findRelationByNameIgnoreCase, 307
generateUniqueIDs, 307
getAllRelations, 308
getCommunicative, 308
getDescription, 308
getEntityDescription, 308
getEntityIconFile, 308
getID, 308
getInstanceID, 308
getLinkedInstanceID, 308
getMessageHandler, 308
getName, 309
getRelations, 309
getSpecificRelations, 309
getType, 309
hasRelation, 309
openEntityEditor, 310
postMessage, 310
Serialize, 310

setDescription, 310
setID, 310
setName, 310
cmas.ontology.interfaces.InterfaceSpecialisation, 311
addRequestToQueue, 313
copy, 313
delete, 314
DeSerialize, 314
Factory, 314
findRelationByName, 314
findRelationByNameIgnoreCase, 315
generateUniqueIDs, 315
getAllRelations, 315
getCommunicative, 315
getDescription, 315
getEntityDescription, 315
getEntityIconFile, 316
getID, 316
getInstanceID, 316
getLinkedInstanceID, 316
getMessageHandler, 316
getName, 316
getRelations, 316
getSpecificRelations, 316
getType, 317
hasRelation, 317
openEntityEditor, 317
postMessage, 317
Serialize, 317
setDescription, 317
setID, 318
setName, 318
cmas.ontology.interfaces.InterfaceTransport, 319
addRequestToQueue, 321
copy, 321
delete, 321
DeSerialize, 321
Factory, 322
findRelationByName, 322
findRelationByNameIgnoreCase, 322
generateUniqueIDs, 324
getAllRelations, 324
getCommunicative, 324
getDescription, 324
getEntityDescription, 324
getEntityIconFile, 324
getID, 324
getInstanceID, 324
getLinkedInstanceID, 325
getMessageHandler, 325
getName, 325
getRelations, 325
getSpecificRelations, 325
getType, 325
hasRelation, 326
openEntityEditor, 326
postMessage, 326
Serialize, 326
setDescription, 326
setID, 326
setName, 327
cmas.ontology.Note, 338
copy, 340
delete, 340
DeSerialize, 340
Factory, 340
findRelationByName, 341
findRelationByNameIgnoreCase, 341
generateUniqueIDs, 341
getAllRelations, 341
getDescription, 341
getEntityDescription, 342
getEntityIconFile, 342
getID, 342
getInstanceID, 342
getLinkedInstanceID, 342
getName, 342
getRelations, 342
getSpecificRelations, 342
getType, 343
hasRelation, 343
openEntityEditor, 343
Serialize, 343
setDescription, 343
setID, 345
setName, 345
cmas.ontology.processplan.ProcessPlan, 360
copy, 362
delete, 363
DeSerialize, 363
Factory, 363
findRelationByName, 363
findRelationByNameIgnoreCase, 364
generateUniqueIDs, 364
getAllRelations, 364
getCommunicative, 364
getDescription, 364
getID, 364
getInstanceID, 365
getLinkedInstanceID, 365
getMessageHandler, 365
getName, 365
getRelations, 365
getSpecificRelations, 365
getType, 366
hasRelation, 366
postMessage, 366
run, 366
Serialize, 366
setDescription, 366
setID, 368
setName, 368
cmas.ontology.processplan.ProcessPlan.ProcessPlanReturnValue, 386
cmas.ontology.processplan.ProcessPlan.ProcessPlanType, 395

cmas.ontology.processplan.ProcessPlanGoals, 368
abort, 370
copy, 370
delete, 370
DeSerialize, 371
execute, 371
Factory, 371
findRelationByName, 371
findRelationByNameIgnoreCase, 372
generateUniqueIDs, 372
getAllRelations, 372
getCommunicative, 372
getDescription, 372
getEntityDescription, 372
getEntityIconFile, 372
getID, 373
getInstanceID, 373
getLinkedInstanceID, 373
getMessageHandler, 373
getName, 373
getRelations, 373
getSpecificRelations, 373
getType, 375
hasRelation, 375
onNegotiate, 375
onOverlap, 375
openEntityEditor, 375
postMessage, 375
resetAllAbstractInterfaces, 376
run, 376
Serialize, 376
setDescription, 376
setID, 376
setName, 376
cmas.ontology.processplan.ProcessPlanInterface, 377
run, 377
cmas.ontology.processplan.ProcessPlanJava, 378
abort, 380
copy, 380
delete, 380
DeSerialize, 380
execute, 380
Factory, 380
findRelationByName, 381
findRelationByNameIgnoreCase, 381
generateUniqueIDs, 381
getAllRelations, 381
getCommunicative, 382
getDescription, 382
getEntityDescription, 382
getEntityIconFile, 382
getID, 382
getInstanceID, 382
getLinkedInstanceID, 382
getMessageHandler, 382
getName, 383
getRelations, 383
getSpecificRelations, 383
getType, 383
gotoLine, 383
hasRelation, 383
onNegotiate, 385
onOverlap, 385
openEntityEditor, 385
postMessage, 385
resetAllAbstractInterfaces, 385
run, 385
Serialize, 385
setDescription, 385
setID, 386
setName, 386
stopEditor, 386
cmas.ontology.processplan.ProcessPlanStructuredText, 387
abort, 389
copy, 389
delete, 389
DeSerialize, 389
execute, 389
Factory, 390
findRelationByName, 390
findRelationByNameIgnoreCase, 390
generateUniqueIDs, 391
getAllRelations, 391
getCommunicative, 391
getDescription, 391
getEntityDescription, 391
getEntityIconFile, 391
getID, 391
getInstanceID, 391
getLinkedInstanceID, 392
getMessageHandler, 392
getName, 392
getRelations, 392
getSpecificRelations, 392
getType, 392
gotoLine, 393
hasRelation, 393
onNegotiate, 393
onOverlap, 393
openEntityEditor, 393
postMessage, 393
resetAllAbstractInterfaces, 394
run, 394
Serialize, 394
setDescription, 394
setID, 394
setName, 394
stopEditor, 395
cmas.ontology.processplan.SkillCall, 433
copy, 435
delete, 435
DeSerialize, 435
Factory, 435
findRelationByName, 436
findRelationByNameIgnoreCase, 436

generateUniqueIDs, 436
getAllRelations, 436
getDescription, 437
getEntityDescription, 437
getEntityIconFile, 437
getID, 437
getInstanceID, 437
getLinkedInstanceID, 437
getName, 437
getRelations, 437
getSpecificRelations, 438
getType, 438
hasRelation, 438
openEntityEditor, 438
Serialize, 438
setDescription, 439
setID, 439
setName, 439
cmas.ontology.Settings, 425
copy, 427
delete, 428
DeSerialize, 428
Factory, 428
findRelationByName, 428
findRelationByNameIgnoreCase, 429
generateUniqueIDs, 429
getAllRelations, 429
getDescription, 429
getEntityDescription, 429
getEntityIconFile, 429
getID, 429
getInstanceID, 430
getLinkedInstanceID, 430
getName, 430
getRelations, 430
getSpecificRelations, 430
getType, 430
hasRelation, 431
openEntityEditor, 431
Serialize, 431
setDescription, 431
setID, 431
setName, 433
cmas.ontology.Types, 447
AgentTypes, 447
BatchTypes, 447
DeploymentTypes, 447
EntityTypes, 448
FolderTypes, 448
GoalTypes, 448
HazardTypes, 448
InterfaceTypes, 449
ProcessPlanTypes, 449
RequirementTypes, 449
SkillTypes, 449
ValueTypes, 449
WorkFlowTypes, 450
cmas.ontology.variables.Iterator, 327
cmas.ontology.variables.Value, 450
copy, 453
delete, 453
DeSerialize, 454
Factory, 454
findRelationByName, 454
findRelationByNameIgnoreCase, 454
generateUniqueIDs, 455
getAllRelations, 455
getCopyOfValue, 455
getDescription, 455
getID, 455
getInstanceID, 455
getLinkedInstanceID, 456
getName, 456
getRelations, 456
getSpecificRelations, 456
getStringValue, 456
getType, 456
getValue, 457
hasRelation, 457
openEntityEditor, 457
Serialize, 457
setDescription, 457
setID, 457
setName, 458
setValue, 458
setValueUndoable, 458
valueOf, 458
cmas.ontology.variables.Value.ValueTypes, 459
cmas.ontology.variables.ValueRange, 458
cmas.ontology.variables.ValueRange.Range, 397
cmas.ontology.variables.Variable, 459
copy, 461
delete, 462
DeSerialize, 462
Factory, 462
findRelationByName, 462
findRelationByNameIgnoreCase, 463
generateUniqueIDs, 463
getAllRelations, 463
getDescription, 463
getID, 463
getInstanceID, 463
getLinkedInstanceID, 464
getName, 464
getRelations, 464
getSpecificRelations, 464
getType, 464
hasRelation, 464
openEntityEditor, 465
Serialize, 465
setDescription, 465
setID, 465
setName, 465
cmas.ontology.variables.Variable.TypeOfVariable, 447
cmas.ontology.variables.VariableArray, 466
copy, 469

delete, 469
DeSerialize, 469
Factory, 469
findRelationByName, 470
findRelationByNameIgnoreCase, 470
generateUniqueIDs, 470
getAllRelations, 470
getCopyOfValue, 471
getDescription, 471
getEntityDescription, 471
getEntityIconFile, 471
getID, 471
getInstanceID, 471
getIterator, 471
getIteratorObject, 471
getLinkedInstanceID, 471
getName, 472
getNext, 472
getRelations, 472
getSpecificRelations, 472
getStringValue, 472
getType, 472
getValue, 473
hasNext, 473
hasRelation, 473
openEntityEditor, 473
Serialize, 473
setDescription, 473
setID, 473, 474
setName, 474
setValue, 474
setValueUndoable, 474
valueOf, 474
cmas.ontology.variables.VariableBoolean, 475
copy, 478
delete, 478
DeSerialize, 478
Factory, 478
findRelationByName, 479
findRelationByNameIgnoreCase, 479
generateUniqueIDs, 480
getAllRelations, 480
getCopyOfValue, 480
getDescription, 480
getEntityDescription, 480
getEntityIconFile, 480
getID, 480
getInstanceID, 480
getIterator, 481
getLinkedInstanceID, 481
getName, 481
getRelations, 481
getSpecificRelations, 481
getStringValue, 481
getType, 482
getValue, 482
hasRelation, 482
openEntityEditor, 482
Serialize, 482
setDescription, 482
setID, 482, 483
setName, 483
setValue, 483
setValueUndoable, 483
valueOf, 483
cmas.ontology.variables.VariableInteger, 484
copy, 487
delete, 487
DeSerialize, 487
Factory, 487
findRelationByName, 488
findRelationByNameIgnoreCase, 488
generateUniqueIDs, 489
getAllRelations, 489
getCopyOfValue, 489
getDescription, 489
getEntityDescription, 489
getEntityIconFile, 489
getID, 489
getInstanceID, 489
getIterator, 490
getLinkedInstanceID, 490
getName, 490
getRelations, 490
getSpecificRelations, 490
getStringValue, 490
getType, 491
getValue, 491
hasRelation, 491
openEntityEditor, 491
Serialize, 491
setDescription, 491
setID, 491, 492
setName, 492
setValue, 492
setValueUndoable, 492
valueOf, 492
cmas.ontology.variables.VariableObject, 493
copy, 496
delete, 496
DeSerialize, 496
Factory, 496
findRelationByName, 497
findRelationByNameIgnoreCase, 497
generateUniqueIDs, 497
getAllRelations, 497
getCopyOfValue, 498
getDescription, 498
getEntityDescription, 498
getEntityIconFile, 498
getID, 498
getInstanceID, 498
getIterator, 498
getIteratorObject, 498
getLinkedInstanceID, 498
getName, 499

getNext, 499
getRelations, 499
getSpecificRelations, 499
getStringValue, 499
getType, 499
getValue, 500
hasNext, 500
hasRelation, 500
makeInstance, 500
openEntityEditor, 500
Serialize, 500
setDescription, 500
setID, 501
setName, 501
setValue, 501
setValueUndoable, 501
valueOf, 501
cmas.ontology.variables.VariableProperty, 502
copy, 505
delete, 505
DeSerialize, 505
Factory, 505
findRelationByName, 506
findRelationByNameIgnoreCase, 506
generateUniqueIDs, 507
getAllRelations, 507
getCopyOfValue, 507
getDescription, 507
getEntityDescription, 507
getEntityIconFile, 507
getID, 507
getInstanceID, 507
getIterator, 508
getLinkedInstanceID, 508
getName, 508
getRelations, 508
getSpecificRelations, 508
getStringValue, 508
getType, 509
getValue, 509
hasRelation, 509
openEntityEditor, 509
Serialize, 509
setDescription, 509
setID, 509, 510
setName, 510
setValue, 510
setValueUndoable, 510
valueOf, 510
cmas.ontology.variables.VariableReal, 511
copy, 514
delete, 514
DeSerialize, 514
Factory, 514
findRelationByName, 515
findRelationByNameIgnoreCase, 515
generateUniqueIDs, 516
getAllRelations, 516
getCopyOfValue, 516
getDescription, 516
getEntityDescription, 516
getEntityIconFile, 516
getID, 516
getInstanceID, 516
getIterator, 517
getLinkedInstanceID, 517
getName, 517
getRelations, 517
getSpecificRelations, 517
getStringValue, 517
getType, 518
getValue, 518
hasRelation, 518
openEntityEditor, 518
Serialize, 518
setDescription, 518
setID, 518, 519
setName, 519
setValue, 519
setValueUndoable, 519
valueOf, 519
cmas.ontology.variables.VariableReference, 520
copy, 523
delete, 523
DeSerialize, 523
Factory, 523
findRelationByName, 523
findRelationByNameIgnoreCase, 524
generateUniqueIDs, 524
getAllRelations, 524
getCopyOfValue, 524
getDescription, 524
getEntityDescription, 524
getEntityIconFile, 525
getID, 525
getInstanceID, 525
getIterator, 525
getIteratorObject, 525
getJSONStringValue, 525
getLinkedInstanceID, 525
getName, 525
getNext, 525
getRelations, 526
getSize, 526
getSpecificRelations, 526
getStringValue, 526
getType, 526
getValue, 526
hasNext, 526
hasRelation, 526
makeInstance, 527
openEntityEditor, 527
Serialize, 527
setDescription, 527
setID, 527
setName, 528

setValue, 528
setValueUndoable, 528
valueOf, 528
cmas.ontology.variables.VariableString, 529
copy, 532
delete, 532
DeSerialize, 532
Factory, 532
findRelationByName, 533
findRelationByNameIgnoreCase, 533
generateUniqueIDs, 534
getAllRelations, 534
getCopyOfValue, 534
getDescription, 534
getEntityDescription, 534
getEntityIconFile, 534
getID, 534
getInstanceID, 534
getIterator, 535
getIteratorObject, 535
getLinkedInstanceID, 535
getName, 535
getNext, 535
getRelations, 535
getSpecificRelations, 535
getStringValue, 536
getType, 536
getValue, 536
hasNext, 536
hasRelation, 536
openEntityEditor, 536
Serialize, 536
setDescription, 537
setID, 537
setName, 537
setValue, 537
setValueUndoable, 537
valueOf, 538
cmas.ontology.workflow.Requirement, 406
copy, 408
delete, 408
DeSerialize, 408
Factory, 408
findRelationByName, 408
findRelationByNameIgnoreCase, 409
generateUniqueIDs, 409
getAllRelations, 409
getDescription, 409
getEntityDescription, 409
getEntityIconFile, 409
getID, 410
getInstanceID, 410
getLinkedInstanceID, 410
getName, 410
getRelations, 410
getSpecificRelations, 410
getType, 411
hasRelation, 411
openEntityEditor, 411
Serialize, 411
setDescription, 411
setID, 411, 412
setName, 412
cmas.ontology.workflow.WorkFlow, 542
copy, 544
delete, 544
DeSerialize, 544
Factory, 544
findRelationByName, 545
findRelationByNameIgnoreCase, 545
generateUniqueIDs, 545
getAllRelations, 545
getDescription, 545
getID, 546
getInstanceID, 546
getLinkedInstanceID, 546
getName, 546
getRelations, 546
getSpecificRelations, 546
getType, 547
hasRelation, 547
Serialize, 547
setDescription, 547
setID, 547
setName, 548
cmas.ontology.workflow.WorkFlow.SpecificTypes, 444
cmas.ontology.workflow.WorkFlow.WorkFlowTypes, 556
cmas.ontology.workflow.WorkFlowLog, 549
copy, 551
delete, 551
DeSerialize, 551
Factory, 551
findRelationByName, 551
findRelationByNameIgnoreCase, 552
generateUniqueIDs, 552
getAllRelations, 552
getDescription, 552
getEntityDescription, 552
getEntityIconFile, 553
getID, 553
getInstanceID, 553
getLinkedInstanceID, 553
getName, 553
getRelations, 553
getSpecificRelations, 553
getType, 555
hasRelation, 555
openEntityEditor, 555
Serialize, 555
setDescription, 555
setID, 555, 556
setName, 556
setTime, 556
copy
cmas.ontology.agents.Agent, 202
cmas.ontology.agents.AgentAction, 209

cmas.ontology.agents.Component, 226
cmas.ontology.agents.DeploymentAgent, 245
cmas.ontology.agents.Material, 330
cmas.ontology.agents.Part, 355
cmas.ontology.agents.Resource, 415
cmas.ontology.base.Entity, 253
cmas.ontology.base.Reference, 400
cmas.ontology.batch.Batch, 218
cmas.ontology.Hazard, 272
cmas.ontology.interfaces.Goal, 264
cmas.ontology.interfaces.Interface, 280
cmas.ontology.interfaces.InterfaceAssembly, 290
cmas.ontology.interfaces.InterfaceBase, 297
cmas.ontology.interfaces.InterfaceLocation, 306
cmas.ontology.interfaces.InterfaceSpecialisation, 313
cmas.ontology.interfaces.InterfaceTransport, 321
cmas.ontology.Note, 340
cmas.ontology.processplan.ProcessPlan, 362
cmas.ontology.processplan.ProcessPlanGoals, 370
cmas.ontology.processplan.ProcessPlanJava, 380
cmas.ontology.processplan.ProcessPlanStructuredText, 389
cmas.ontology.processplan.SkillCall, 435
cmas.ontology.Settings, 427
cmas.ontology.variables.Value, 453
cmas.ontology.variables.Variable, 461
cmas.ontology.variables.VariableArray, 469
cmas.ontology.variables.VariableBoolean, 478
cmas.ontology.variables.VariableInteger, 487
cmas.ontology.variables.VariableObject, 496
cmas.ontology.variables.VariableProperty, 505
cmas.ontology.variables.VariableReal, 514
cmas.ontology.variables.VariableReference, 523
cmas.ontology.variables.VariableString, 532
cmas.ontology.workflow.Requirement, 408
cmas.ontology.workflow.WorkFlow, 544
cmas.ontology.workflow.WorkFlowLog, 551

Crane Simulation Example, 28
Crane-3D Simulation Example, 36
createNew
cmas.ontology.adapter.FILE.FileClient, 260
cmas.ontology.adapter.Modbus.ModbusClient, 336
cmas.ontology.adapter.OPCUA.OPCUAClient, 347
cmas.ontology.adapter.REST.RESTClient, 423
cmas.ontology.adapter.socket.SocketTCPClient, 440
cmas.ontology.adapter.socket.SocketUDPClient, 443
cmas.ontology.adapter.SQL.SQLClient, 445
cmas.ontology.adapter.websocket.WebSocketClient, 540
cmas.ontology.adapter.ZeroMQ.ZMQClient, 558
createParameters
cmas.ontology.agentcommunication.jsoncontent.Content, 234
cmas.ontology.agentcommunication.jsoncontent.template.ContentBu 235
cmas.ontology.agentcommunication.jsoncontent.template.ContentDo 236
cmas.ontology.agentcommunication.jsoncontent.template.ContentErr 237
cmas.ontology.agentcommunication.jsoncontent.template.ContentInfo 238
cmas.ontology.agentcommunication.jsoncontent.template.ContentNe 239
cmas.ontology.agentcommunication.jsoncontent.template.ContentPro 240
cmas.ontology.agentcommunication.jsoncontent.template.ContentRe 241
cmas.ontology.agentcommunication.jsoncontent.template.ContentRe 242

delete
cmas.ontology.agents.Agent, 202
cmas.ontology.agents.AgentAction, 210
cmas.ontology.agents.Component, 227
cmas.ontology.agents.DeploymentAgent, 245
cmas.ontology.agents.Material, 330
cmas.ontology.agents.Part, 355
cmas.ontology.agents.Resource, 416
cmas.ontology.base.Entity, 253
cmas.ontology.base.Reference, 400
cmas.ontology.batch.Batch, 218
cmas.ontology.Hazard, 272
cmas.ontology.interfaces.Goal, 265
cmas.ontology.interfaces.Interface, 282
cmas.ontology.interfaces.InterfaceAssembly, 290
cmas.ontology.interfaces.InterfaceBase, 298
cmas.ontology.interfaces.InterfaceLocation, 306
cmas.ontology.interfaces.InterfaceSpecialisation, 314
cmas.ontology.interfaces.InterfaceTransport, 321
cmas.ontology.Note, 340
cmas.ontology.processplan.ProcessPlan, 363
cmas.ontology.processplan.ProcessPlanGoals, 370
cmas.ontology.processplan.ProcessPlanJava, 380
cmas.ontology.processplan.ProcessPlanStructuredText, 389
cmas.ontology.processplan.SkillCall, 435
cmas.ontology.Settings, 428
cmas.ontology.variables.Value, 453
cmas.ontology.variables.Variable, 462
cmas.ontology.variables.VariableArray, 469
cmas.ontology.variables.VariableBoolean, 478
cmas.ontology.variables.VariableInteger, 487
cmas.ontology.variables.VariableObject, 496
cmas.ontology.variables.VariableProperty, 505
cmas.ontology.variables.VariableReal, 514
cmas.ontology.variables.VariableReference, 523
cmas.ontology.variables.VariableString, 532
cmas.ontology.workflow.Requirement, 408
cmas.ontology.workflow.WorkFlow, 544
cmas.ontology.workflow.WorkFlowLog, 551

Deployment, 19
DeploymentTypes
cmas.ontology.Types, 447
DeSerialize
cmas.ontology.agents.Agent, 202
cmas.ontology.agents.AgentAction, 210
cmas.ontology.agents.Component, 227
cmas.ontology.agents.DeploymentAgent, 245
cmas.ontology.agents.Material, 330
cmas.ontology.agents.Part, 355
cmas.ontology.agents.Resource, 416
cmas.ontology.base.Entity, 253
cmas.ontology.base.Reference, 400
cmas.ontology.batch.Batch, 218
cmas.ontology.Hazard, 272
cmas.ontology.interfaces.Goal, 265
cmas.ontology.interfaces.Interface, 282
cmas.ontology.interfaces.InterfaceAssembly, 290
cmas.ontology.interfaces.InterfaceBase, 298
cmas.ontology.interfaces.InterfaceLocation, 306
cmas.ontology.interfaces.InterfaceSpecialisation, 314
cmas.ontology.interfaces.InterfaceTransport, 321
cmas.ontology.Note, 340
cmas.ontology.processplan.ProcessPlan, 363
cmas.ontology.processplan.ProcessPlanGoals, 371
cmas.ontology.processplan.ProcessPlanJava, 380
cmas.ontology.processplan.ProcessPlanStructuredText, 389
cmas.ontology.processplan.SkillCall, 435
cmas.ontology.Settings, 428
cmas.ontology.variables.Value, 454
cmas.ontology.variables.Variable, 462
cmas.ontology.variables.VariableArray, 469
cmas.ontology.variables.VariableBoolean, 478
cmas.ontology.variables.VariableInteger, 487
cmas.ontology.variables.VariableObject, 496
cmas.ontology.variables.VariableProperty, 505
cmas.ontology.variables.VariableReal, 514
cmas.ontology.variables.VariableReference, 523
cmas.ontology.variables.VariableString, 532
cmas.ontology.workflow.Requirement, 408
cmas.ontology.workflow.WorkFlow, 544
cmas.ontology.workflow.WorkFlowLog, 551

Entity
cmas.ontology.base.Entity, 252, 253
EntityTypes
cmas.ontology.Types, 448
Example 1 - Name tag, 23
execute
cmas.ontology.processplan.ProcessPlanGoals, 371
cmas.ontology.processplan.ProcessPlanJava, 380
cmas.ontology.processplan.ProcessPlanStructuredText, 389

Factory
cmas.ontology.agents.Agent, 202
cmas.ontology.agents.AgentAction, 210
cmas.ontology.agents.Component, 227
cmas.ontology.agents.DeploymentAgent, 245
cmas.ontology.agents.Material, 330
cmas.ontology.agents.Part, 355
cmas.ontology.agents.Resource, 416
cmas.ontology.base.Entity, 253
cmas.ontology.base.Reference, 400
cmas.ontology.batch.Batch, 218
cmas.ontology.Hazard, 272
cmas.ontology.interfaces.Goal, 265
cmas.ontology.interfaces.Interface, 282
cmas.ontology.interfaces.InterfaceAssembly, 290
cmas.ontology.interfaces.InterfaceBase, 298
cmas.ontology.interfaces.InterfaceLocation, 306
cmas.ontology.interfaces.InterfaceSpecialisation, 314
cmas.ontology.interfaces.InterfaceTransport, 322
cmas.ontology.Note, 340
cmas.ontology.processplan.ProcessPlan, 363
cmas.ontology.processplan.ProcessPlanGoals, 371
cmas.ontology.processplan.ProcessPlanJava, 380
cmas.ontology.processplan.ProcessPlanStructuredText, 390
cmas.ontology.processplan.SkillCall, 435
cmas.ontology.Settings, 428
cmas.ontology.variables.Value, 454
cmas.ontology.variables.Variable, 462
cmas.ontology.variables.VariableArray, 469
cmas.ontology.variables.VariableBoolean, 478
cmas.ontology.variables.VariableInteger, 487
cmas.ontology.variables.VariableObject, 496
cmas.ontology.variables.VariableProperty, 505
cmas.ontology.variables.VariableReal, 514
cmas.ontology.variables.VariableReference, 523
cmas.ontology.variables.VariableString, 532
cmas.ontology.workflow.Requirement, 408
cmas.ontology.workflow.WorkFlow, 544
cmas.ontology.workflow.WorkFlowLog, 551
findRelationByName
cmas.ontology.agents.Agent, 203
cmas.ontology.agents.AgentAction, 210
cmas.ontology.agents.Component, 227
cmas.ontology.agents.DeploymentAgent, 246
cmas.ontology.agents.Material, 330
cmas.ontology.agents.Part, 355
cmas.ontology.agents.Resource, 416
cmas.ontology.base.Entity, 254
cmas.ontology.base.Reference, 400
cmas.ontology.batch.Batch, 218
cmas.ontology.Hazard, 272
cmas.ontology.interfaces.Goal, 265
cmas.ontology.interfaces.Interface, 282
cmas.ontology.interfaces.InterfaceAssembly, 291
cmas.ontology.interfaces.InterfaceBase, 299
cmas.ontology.interfaces.InterfaceLocation, 307

cmas.ontology.interfaces.InterfaceSpecialisation, 314
cmas.ontology.interfaces.InterfaceTransport, 322
cmas.ontology.Note, 341
cmas.ontology.processplan.ProcessPlan, 363
cmas.ontology.processplan.ProcessPlanGoals, 371
cmas.ontology.processplan.ProcessPlanJava, 381
cmas.ontology.processplan.ProcessPlanStructuredText, 390
cmas.ontology.processplan.SkillCall, 436
cmas.ontology.Settings, 428
cmas.ontology.variables.Value, 454
cmas.ontology.variables.Variable, 462
cmas.ontology.variables.VariableArray, 470
cmas.ontology.variables.VariableBoolean, 479
cmas.ontology.variables.VariableInteger, 488
cmas.ontology.variables.VariableObject, 497
cmas.ontology.variables.VariableProperty, 506
cmas.ontology.variables.VariableReal, 515
cmas.ontology.variables.VariableReference, 523
cmas.ontology.variables.VariableString, 533
cmas.ontology.workflow.Requirement, 408
cmas.ontology.workflow.WorkFlow, 545
cmas.ontology.workflow.WorkFlowLog, 551

findRelationByNameIgnoreCase
cmas.ontology.agents.Agent, 203
cmas.ontology.agents.AgentAction, 211
cmas.ontology.agents.Component, 228
cmas.ontology.agents.DeploymentAgent, 246
cmas.ontology.agents.Material, 331
cmas.ontology.agents.Part, 356
cmas.ontology.agents.Resource, 417
cmas.ontology.base.Entity, 254
cmas.ontology.base.Reference, 401
cmas.ontology.batch.Batch, 219
cmas.ontology.Hazard, 273
cmas.ontology.interfaces.Goal, 266
cmas.ontology.interfaces.Interface, 283
cmas.ontology.interfaces.InterfaceAssembly, 291
cmas.ontology.interfaces.InterfaceBase, 299
cmas.ontology.interfaces.InterfaceLocation, 307
cmas.ontology.interfaces.InterfaceSpecialisation, 315
cmas.ontology.interfaces.InterfaceTransport, 322
cmas.ontology.Note, 341
cmas.ontology.processplan.ProcessPlan, 364
cmas.ontology.processplan.ProcessPlanGoals, 372
cmas.ontology.processplan.ProcessPlanJava, 381
cmas.ontology.processplan.ProcessPlanStructuredText, 390
cmas.ontology.processplan.SkillCall, 436
cmas.ontology.Settings, 429
cmas.ontology.variables.Value, 454
cmas.ontology.variables.Variable, 463
cmas.ontology.variables.VariableArray, 470
cmas.ontology.variables.VariableBoolean, 479
cmas.ontology.variables.VariableInteger, 488
cmas.ontology.variables.VariableObject, 497
cmas.ontology.variables.VariableProperty, 506
cmas.ontology.variables.VariableReal, 515
cmas.ontology.variables.VariableReference, 524
cmas.ontology.variables.VariableString, 533
cmas.ontology.workflow.Requirement, 409
cmas.ontology.workflow.WorkFlow, 545
cmas.ontology.workflow.WorkFlowLog, 552

FolderTypes
cmas.ontology.Types, 448

generateUniqueIDs
cmas.ontology.agents.Agent, 203
cmas.ontology.agents.AgentAction, 211
cmas.ontology.agents.Component, 228
cmas.ontology.agents.DeploymentAgent, 246
cmas.ontology.agents.Material, 331
cmas.ontology.agents.Part, 356
cmas.ontology.agents.Resource, 417
cmas.ontology.base.Entity, 255
cmas.ontology.base.Reference, 401
cmas.ontology.batch.Batch, 219
cmas.ontology.Hazard, 273
cmas.ontology.interfaces.Goal, 266
cmas.ontology.interfaces.Interface, 283
cmas.ontology.interfaces.InterfaceAssembly, 291
cmas.ontology.interfaces.InterfaceBase, 299
cmas.ontology.interfaces.InterfaceLocation, 307
cmas.ontology.interfaces.InterfaceSpecialisation, 315
cmas.ontology.interfaces.InterfaceTransport, 324
cmas.ontology.Note, 341
cmas.ontology.processplan.ProcessPlan, 364
cmas.ontology.processplan.ProcessPlanGoals, 372
cmas.ontology.processplan.ProcessPlanJava, 381
cmas.ontology.processplan.ProcessPlanStructuredText, 391
cmas.ontology.processplan.SkillCall, 436
cmas.ontology.Settings, 429
cmas.ontology.variables.Value, 455
cmas.ontology.variables.Variable, 463
cmas.ontology.variables.VariableArray, 470
cmas.ontology.variables.VariableBoolean, 480
cmas.ontology.variables.VariableInteger, 489
cmas.ontology.variables.VariableObject, 497
cmas.ontology.variables.VariableProperty, 507
cmas.ontology.variables.VariableReal, 516
cmas.ontology.variables.VariableReference, 524
cmas.ontology.variables.VariableString, 534
cmas.ontology.workflow.Requirement, 409
cmas.ontology.workflow.WorkFlow, 545
cmas.ontology.workflow.WorkFlowLog, 552

getAllRelations
cmas.ontology.agents.Agent, 203
cmas.ontology.agents.AgentAction, 211
cmas.ontology.agents.Component, 228
cmas.ontology.agents.DeploymentAgent, 246

cmas.ontology.agents.Material, 331
cmas.ontology.agents.Part, 356
cmas.ontology.agents.Resource, 417
cmas.ontology.base.Entity, 255
cmas.ontology.base.Reference, 401
cmas.ontology.batch.Batch, 219
cmas.ontology.Hazard, 273
cmas.ontology.interfaces.Goal, 266
cmas.ontology.interfaces.Interface, 283
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceBase, 299
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 315
cmas.ontology.interfaces.InterfaceTransport, 324
cmas.ontology.Note, 341
cmas.ontology.processplan.ProcessPlan, 364
cmas.ontology.processplan.ProcessPlanGoals, 372
cmas.ontology.processplan.ProcessPlanJava, 381
cmas.ontology.processplan.ProcessPlanStructuredText, 391
cmas.ontology.processplan.SkillCall, 436
cmas.ontology.Settings, 429
cmas.ontology.variables.Value, 455
cmas.ontology.variables.Variable, 463
cmas.ontology.variables.VariableArray, 470
cmas.ontology.variables.VariableBoolean, 480
cmas.ontology.variables.VariableInteger, 489
cmas.ontology.variables.VariableObject, 497
cmas.ontology.variables.VariableProperty, 507
cmas.ontology.variables.VariableReal, 516
cmas.ontology.variables.VariableReference, 524
cmas.ontology.variables.VariableString, 534
cmas.ontology.workflow.Requirement, 409
cmas.ontology.workflow.WorkFlow, 545
cmas.ontology.workflow.WorkFlowLog, 552

getCommunicative
cmas.ontology.agents.AgentAction, 211
cmas.ontology.agents.Component, 228
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 331
cmas.ontology.agents.Part, 356
cmas.ontology.agents.Resource, 417
cmas.ontology.interfaces.AbstractInterface, 194
cmas.ontology.interfaces.Goal, 266
cmas.ontology.interfaces.Interface, 283
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceBase, 299
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 315
cmas.ontology.interfaces.InterfaceTransport, 324
cmas.ontology.processplan.ProcessPlan, 364
cmas.ontology.processplan.ProcessPlanGoals, 372
cmas.ontology.processplan.ProcessPlanJava, 382
cmas.ontology.processplan.ProcessPlanStructuredText, 391

getCopyOfValue
cmas.ontology.variables.Value, 455
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableBoolean, 480
cmas.ontology.variables.VariableInteger, 489
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableProperty, 507
cmas.ontology.variables.VariableReal, 516
cmas.ontology.variables.VariableReference, 524
cmas.ontology.variables.VariableString, 534

getDefaultValue
cmas.ontology.adapter.FILE.FileClient, 260
cmas.ontology.adapter.Modbus.ModbusClient, 337
cmas.ontology.adapter.OPCUA.OPCUAClient, 347
cmas.ontology.adapter.REST.RESTClient, 423
cmas.ontology.adapter.socket.SocketTCPClient, 441
cmas.ontology.adapter.socket.SocketUDPClient, 443
cmas.ontology.adapter.SQL.SQLClient, 445
cmas.ontology.adapter.websocket.WebSocketClient, 540
cmas.ontology.adapter.ZeroMQ.ZMQClient, 558

getDescription
cmas.ontology.adapter.FILE.FileClient, 261
cmas.ontology.adapter.Modbus.ModbusClient, 337
cmas.ontology.adapter.OPCUA.OPCUAClient, 347
cmas.ontology.adapter.REST.RESTClient, 423
cmas.ontology.adapter.socket.SocketTCPClient, 441
cmas.ontology.adapter.socket.SocketUDPClient, 443
cmas.ontology.adapter.SQL.SQLClient, 445
cmas.ontology.adapter.websocket.WebSocketClient, 540
cmas.ontology.adapter.ZeroMQ.ZMQClient, 558
cmas.ontology.agents.Agent, 203
cmas.ontology.agents.AgentAction, 211
cmas.ontology.agents.Component, 228
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 331
cmas.ontology.agents.Part, 356
cmas.ontology.agents.Resource, 417
cmas.ontology.base.Entity, 255
cmas.ontology.base.Reference, 401
cmas.ontology.batch.Batch, 219
cmas.ontology.Hazard, 273
cmas.ontology.interfaces.Goal, 266
cmas.ontology.interfaces.Interface, 283
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceBase, 300
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 315
cmas.ontology.interfaces.InterfaceTransport, 324
cmas.ontology.Note, 341

cmas.ontology.processplan.ProcessPlan, 364
cmas.ontology.processplan.ProcessPlanGoals, 372
cmas.ontology.processplan.ProcessPlanJava, 382
cmas.ontology.processplan.ProcessPlanStructuredText, 391
cmas.ontology.processplan.SkillCall, 437
cmas.ontology.Settings, 429
cmas.ontology.variables.Value, 455
cmas.ontology.variables.Variable, 463
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableBoolean, 480
cmas.ontology.variables.VariableInteger, 489
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableProperty, 507
cmas.ontology.variables.VariableReal, 516
cmas.ontology.variables.VariableReference, 524
cmas.ontology.variables.VariableString, 534
cmas.ontology.workflow.Requirement, 409
cmas.ontology.workflow.WorkFlow, 545
cmas.ontology.workflow.WorkFlowLog, 552
getEntityDescription
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 332
cmas.ontology.agents.Part, 356
cmas.ontology.agents.Resource, 417
cmas.ontology.base.Reference, 401
cmas.ontology.batch.Batch, 219
cmas.ontology.Hazard, 273
cmas.ontology.interfaces.Goal, 266
cmas.ontology.interfaces.Interface, 283
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 315
cmas.ontology.interfaces.InterfaceTransport, 324
cmas.ontology.Note, 342
cmas.ontology.processplan.ProcessPlanGoals, 372
cmas.ontology.processplan.ProcessPlanJava, 382
cmas.ontology.processplan.ProcessPlanStructuredText, 391
cmas.ontology.processplan.SkillCall, 437
cmas.ontology.Settings, 429
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableBoolean, 480
cmas.ontology.variables.VariableInteger, 489
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableProperty, 507
cmas.ontology.variables.VariableReal, 516
cmas.ontology.variables.VariableReference, 524
cmas.ontology.variables.VariableString, 534
cmas.ontology.workflow.Requirement, 409
cmas.ontology.workflow.WorkFlowLog, 552
getEntityIconFile
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 332
cmas.ontology.agents.Part, 357
cmas.ontology.agents.Resource, 417
cmas.ontology.base.Reference, 402
cmas.ontology.batch.Batch, 220
cmas.ontology.Hazard, 274
cmas.ontology.interfaces.Goal, 267
cmas.ontology.interfaces.Interface, 283
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 316
cmas.ontology.interfaces.InterfaceTransport, 324
cmas.ontology.Note, 342
cmas.ontology.processplan.ProcessPlanGoals, 372
cmas.ontology.processplan.ProcessPlanJava, 382
cmas.ontology.processplan.ProcessPlanStructuredText, 391
cmas.ontology.processplan.SkillCall, 437
cmas.ontology.Settings, 429
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableBoolean, 480
cmas.ontology.variables.VariableInteger, 489
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableProperty, 507
cmas.ontology.variables.VariableReal, 516
cmas.ontology.variables.VariableReference, 525
cmas.ontology.variables.VariableString, 534
cmas.ontology.workflow.Requirement, 409
cmas.ontology.workflow.WorkFlowLog, 553
getID
cmas.ontology.agents.Agent, 204
cmas.ontology.agents.AgentAction, 211
cmas.ontology.agents.Component, 228
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 332
cmas.ontology.agents.Part, 357
cmas.ontology.agents.Resource, 418
cmas.ontology.base.Entity, 255
cmas.ontology.base.Reference, 402
cmas.ontology.batch.Batch, 220
cmas.ontology.Hazard, 274
cmas.ontology.interfaces.Goal, 267
cmas.ontology.interfaces.Interface, 284
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceBase, 300
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 316
cmas.ontology.interfaces.InterfaceTransport, 324
cmas.ontology.Note, 342
cmas.ontology.processplan.ProcessPlan, 364
cmas.ontology.processplan.ProcessPlanGoals, 373
cmas.ontology.processplan.ProcessPlanJava, 382
cmas.ontology.processplan.ProcessPlanStructuredText, 391
cmas.ontology.processplan.SkillCall, 437
cmas.ontology.Settings, 429

cmas.ontology.variables.Value, 455
cmas.ontology.variables.Variable, 463
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableBoolean, 480
cmas.ontology.variables.VariableInteger, 489
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableProperty, 507
cmas.ontology.variables.VariableReal, 516
cmas.ontology.variables.VariableReference, 525
cmas.ontology.variables.VariableString, 534
cmas.ontology.workflow.Requirement, 410
cmas.ontology.workflow.WorkFlow, 546
cmas.ontology.workflow.WorkFlowLog, 553
getInstance
cmas.ontology.action.IndependentClassLoader, 278
getInstanceID
cmas.ontology.agents.Agent, 204
cmas.ontology.agents.AgentAction, 212
cmas.ontology.agents.Component, 229
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 332
cmas.ontology.agents.Part, 357
cmas.ontology.agents.Resource, 418
cmas.ontology.base.Entity, 255
cmas.ontology.base.Reference, 402
cmas.ontology.batch.Batch, 220
cmas.ontology.Hazard, 274
cmas.ontology.interfaces.Goal, 267
cmas.ontology.interfaces.Interface, 284
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceBase, 300
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 316
cmas.ontology.interfaces.InterfaceTransport, 324
cmas.ontology.Note, 342
cmas.ontology.processplan.ProcessPlan, 365
cmas.ontology.processplan.ProcessPlanGoals, 373
cmas.ontology.processplan.ProcessPlanJava, 382
cmas.ontology.processplan.ProcessPlanStructuredText, 391
cmas.ontology.processplan.SkillCall, 437
cmas.ontology.Settings, 430
cmas.ontology.variables.Value, 455
cmas.ontology.variables.Variable, 463
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableBoolean, 480
cmas.ontology.variables.VariableInteger, 489
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableProperty, 507
cmas.ontology.variables.VariableReal, 516
cmas.ontology.variables.VariableReference, 525
cmas.ontology.variables.VariableString, 534
cmas.ontology.workflow.Requirement, 410
cmas.ontology.workflow.WorkFlow, 546
cmas.ontology.workflow.WorkFlowLog, 553
getIterator
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableBoolean, 481
cmas.ontology.variables.VariableInteger, 490
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableProperty, 508
cmas.ontology.variables.VariableReal, 517
cmas.ontology.variables.VariableReference, 525
cmas.ontology.variables.VariableString, 535
getIteratorObject
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableReference, 525
cmas.ontology.variables.VariableString, 535
getJSONStringValue
cmas.ontology.variables.VariableReference, 525
getLinkedInstanceID
cmas.ontology.agents.Agent, 204
cmas.ontology.agents.AgentAction, 212
cmas.ontology.agents.Component, 229
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 332
cmas.ontology.agents.Part, 357
cmas.ontology.agents.Resource, 418
cmas.ontology.base.Container, 233
cmas.ontology.base.Entity, 255
cmas.ontology.base.Reference, 402
cmas.ontology.base.Visualisation, 539
cmas.ontology.batch.Batch, 220
cmas.ontology.Hazard, 274
cmas.ontology.interfaces.Goal, 267
cmas.ontology.interfaces.Interface, 284
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceBase, 300
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 316
cmas.ontology.interfaces.InterfaceTransport, 325
cmas.ontology.Note, 342
cmas.ontology.processplan.ProcessPlan, 365
cmas.ontology.processplan.ProcessPlanGoals, 373
cmas.ontology.processplan.ProcessPlanJava, 382
cmas.ontology.processplan.ProcessPlanStructuredText, 392
cmas.ontology.processplan.SkillCall, 437
cmas.ontology.Settings, 430
cmas.ontology.variables.Value, 456
cmas.ontology.variables.Variable, 464
cmas.ontology.variables.VariableArray, 471
cmas.ontology.variables.VariableBoolean, 481
cmas.ontology.variables.VariableInteger, 490
cmas.ontology.variables.VariableObject, 498
cmas.ontology.variables.VariableProperty, 508
cmas.ontology.variables.VariableReal, 517
cmas.ontology.variables.VariableReference, 525
cmas.ontology.variables.VariableString, 535
cmas.ontology.workflow.Requirement, 410

cmas.ontology.workflow.WorkFlow, 546
cmas.ontology.workflow.WorkFlowLog, 553
getMessageHandler
cmas.ontology.agents.AgentAction, 212
cmas.ontology.agents.Component, 229
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 332
cmas.ontology.agents.Part, 357
cmas.ontology.agents.Resource, 418
cmas.ontology.interfaces.AbstractInterface, 194
cmas.ontology.interfaces.Goal, 267
cmas.ontology.interfaces.Interface, 284
cmas.ontology.interfaces.InterfaceAssembly, 292
cmas.ontology.interfaces.InterfaceBase, 300
cmas.ontology.interfaces.InterfaceLocation, 308
cmas.ontology.interfaces.InterfaceSpecialisation, 316
cmas.ontology.interfaces.InterfaceTransport, 325
cmas.ontology.processplan.ProcessPlan, 365
cmas.ontology.processplan.ProcessPlanGoals, 373
cmas.ontology.processplan.ProcessPlanJava, 382
cmas.ontology.processplan.ProcessPlanStructuredText, 392
getName
cmas.ontology.agents.Agent, 204
cmas.ontology.agents.AgentAction, 212
cmas.ontology.agents.Component, 229
cmas.ontology.agents.DeploymentAgent, 247
cmas.ontology.agents.Material, 332
cmas.ontology.agents.Part, 357
cmas.ontology.agents.Resource, 418
cmas.ontology.base.Entity, 255
cmas.ontology.base.Reference, 402
cmas.ontology.batch.Batch, 220
cmas.ontology.Hazard, 274
cmas.ontology.interfaces.Goal, 267
cmas.ontology.interfaces.Interface, 284
cmas.ontology.interfaces.InterfaceAssembly, 293
cmas.ontology.interfaces.InterfaceBase, 300
cmas.ontology.interfaces.InterfaceLocation, 309
cmas.ontology.interfaces.InterfaceSpecialisation, 316
cmas.ontology.interfaces.InterfaceTransport, 325
cmas.ontology.Note, 342
cmas.ontology.processplan.ProcessPlan, 365
cmas.ontology.processplan.ProcessPlanGoals, 373
cmas.ontology.processplan.ProcessPlanJava, 383
cmas.ontology.processplan.ProcessPlanStructuredText, 392
cmas.ontology.processplan.SkillCall, 437
cmas.ontology.Settings, 430
cmas.ontology.variables.Value, 456
cmas.ontology.variables.Variable, 464
cmas.ontology.variables.VariableArray, 472
cmas.ontology.variables.VariableBoolean, 481
cmas.ontology.variables.VariableInteger, 490
cmas.ontology.variables.VariableObject, 499
cmas.ontology.variables.VariableProperty, 508
cmas.ontology.variables.VariableReal, 517
cmas.ontology.variables.VariableReference, 525
cmas.ontology.variables.VariableString, 535
cmas.ontology.workflow.Requirement, 410
cmas.ontology.workflow.WorkFlow, 546
cmas.ontology.workflow.WorkFlowLog, 553
getNext
cmas.ontology.variables.VariableArray, 472
cmas.ontology.variables.VariableObject, 499
cmas.ontology.variables.VariableReference, 525
cmas.ontology.variables.VariableString, 535
getPropertiesList
cmas.ontology.base.Properties, 396
getRelations
cmas.ontology.agents.Agent, 204
cmas.ontology.agents.AgentAction, 212
cmas.ontology.agents.Component, 229
cmas.ontology.agents.DeploymentAgent, 248
cmas.ontology.agents.Material, 332
cmas.ontology.agents.Part, 357
cmas.ontology.agents.Resource, 418
cmas.ontology.base.Entity, 256
cmas.ontology.base.Reference, 402
cmas.ontology.batch.Batch, 220
cmas.ontology.Hazard, 274
cmas.ontology.interfaces.Goal, 267
cmas.ontology.interfaces.Interface, 284
cmas.ontology.interfaces.InterfaceAssembly, 293
cmas.ontology.interfaces.InterfaceBase, 300
cmas.ontology.interfaces.InterfaceLocation, 309
cmas.ontology.interfaces.InterfaceSpecialisation, 316
cmas.ontology.interfaces.InterfaceTransport, 325
cmas.ontology.Note, 342
cmas.ontology.processplan.ProcessPlan, 365
cmas.ontology.processplan.ProcessPlanGoals, 373
cmas.ontology.processplan.ProcessPlanJava, 383
cmas.ontology.processplan.ProcessPlanStructuredText, 392
cmas.ontology.processplan.SkillCall, 437
cmas.ontology.Settings, 430
cmas.ontology.variables.Value, 456
cmas.ontology.variables.Variable, 464
cmas.ontology.variables.VariableArray, 472
cmas.ontology.variables.VariableBoolean, 481
cmas.ontology.variables.VariableInteger, 490
cmas.ontology.variables.VariableObject, 499
cmas.ontology.variables.VariableProperty, 508
cmas.ontology.variables.VariableReal, 517
cmas.ontology.variables.VariableReference, 526
cmas.ontology.variables.VariableString, 535
cmas.ontology.workflow.Requirement, 410
cmas.ontology.workflow.WorkFlow, 546
cmas.ontology.workflow.WorkFlowLog, 553
getServerHost

cmas.ontology.adapter.OPCUA.OPCUAClient, 347
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 351
getServerName
cmas.ontology.adapter.OPCUA.OPCUAClient, 347
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 351
getServerPort
cmas.ontology.adapter.OPCUA.OPCUAClient, 347
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 351
getSize
cmas.ontology.variables.VariableReference, 526
getSpecificRelations
cmas.ontology.agents.Agent, 204
cmas.ontology.agents.AgentAction, 212
cmas.ontology.agents.Component, 229
cmas.ontology.agents.DeploymentAgent, 248
cmas.ontology.agents.Material, 333
cmas.ontology.agents.Part, 357
cmas.ontology.agents.Resource, 418
cmas.ontology.base.Entity, 256
cmas.ontology.base.Reference, 402
cmas.ontology.batch.Batch, 220
cmas.ontology.Hazard, 274
cmas.ontology.interfaces.Goal, 267
cmas.ontology.interfaces.Interface, 284
cmas.ontology.interfaces.InterfaceAssembly, 293
cmas.ontology.interfaces.InterfaceBase, 300
cmas.ontology.interfaces.InterfaceLocation, 309
cmas.ontology.interfaces.InterfaceSpecialisation, 316
cmas.ontology.interfaces.InterfaceTransport, 325
cmas.ontology.Note, 342
cmas.ontology.processplan.ProcessPlan, 365
cmas.ontology.processplan.ProcessPlanGoals, 373
cmas.ontology.processplan.ProcessPlanJava, 383
cmas.ontology.processplan.ProcessPlanStructuredText, 392
cmas.ontology.processplan.SkillCall, 438
cmas.ontology.Settings, 430
cmas.ontology.variables.Value, 456
cmas.ontology.variables.Variable, 464
cmas.ontology.variables.VariableArray, 472
cmas.ontology.variables.VariableBoolean, 481
cmas.ontology.variables.VariableInteger, 490
cmas.ontology.variables.VariableObject, 499
cmas.ontology.variables.VariableProperty, 508
cmas.ontology.variables.VariableReal, 517
cmas.ontology.variables.VariableReference, 526
cmas.ontology.variables.VariableString, 535
cmas.ontology.workflow.Requirement, 410
cmas.ontology.workflow.WorkFlow, 546
cmas.ontology.workflow.WorkFlowLog, 553
getStringValue
cmas.ontology.variables.Value, 456
cmas.ontology.variables.VariableArray, 472
cmas.ontology.variables.VariableBoolean, 481
cmas.ontology.variables.VariableInteger, 490
cmas.ontology.variables.VariableObject, 499
cmas.ontology.variables.VariableProperty, 508
cmas.ontology.variables.VariableReal, 517
cmas.ontology.variables.VariableReference, 526
cmas.ontology.variables.VariableString, 536
getType
cmas.ontology.agents.Agent, 205
cmas.ontology.agents.AgentAction, 213
cmas.ontology.agents.Component, 230
cmas.ontology.agents.DeploymentAgent, 248
cmas.ontology.agents.Material, 333
cmas.ontology.agents.Part, 358
cmas.ontology.agents.Resource, 420
cmas.ontology.base.Entity, 256
cmas.ontology.base.Reference, 403
cmas.ontology.batch.Batch, 222
cmas.ontology.Hazard, 276
cmas.ontology.interfaces.Goal, 268
cmas.ontology.interfaces.Interface, 286
cmas.ontology.interfaces.InterfaceAssembly, 293
cmas.ontology.interfaces.InterfaceBase, 301
cmas.ontology.interfaces.InterfaceLocation, 309
cmas.ontology.interfaces.InterfaceSpecialisation, 317
cmas.ontology.interfaces.InterfaceTransport, 325
cmas.ontology.Note, 343
cmas.ontology.processplan.ProcessPlan, 366
cmas.ontology.processplan.ProcessPlanGoals, 375
cmas.ontology.processplan.ProcessPlanJava, 383
cmas.ontology.processplan.ProcessPlanStructuredText, 392
cmas.ontology.processplan.SkillCall, 438
cmas.ontology.Settings, 430
cmas.ontology.variables.Value, 456
cmas.ontology.variables.Variable, 464
cmas.ontology.variables.VariableArray, 472
cmas.ontology.variables.VariableBoolean, 482
cmas.ontology.variables.VariableInteger, 491
cmas.ontology.variables.VariableObject, 499
cmas.ontology.variables.VariableProperty, 509
cmas.ontology.variables.VariableReal, 518
cmas.ontology.variables.VariableReference, 526
cmas.ontology.variables.VariableString, 536
cmas.ontology.workflow.Requirement, 411
cmas.ontology.workflow.WorkFlow, 547
cmas.ontology.workflow.WorkFlowLog, 555
getValue
cmas.ontology.variables.Value, 457
cmas.ontology.variables.VariableArray, 473
cmas.ontology.variables.VariableBoolean, 482
cmas.ontology.variables.VariableInteger, 491
cmas.ontology.variables.VariableObject, 500
cmas.ontology.variables.VariableProperty, 509
cmas.ontology.variables.VariableReal, 518
cmas.ontology.variables.VariableReference, 526

cmas.ontology.variables.VariableString, 536
Goal, 69
Goal Charts, 55
GoalTypes
cmas.ontology.Types, 448
gotoLine
cmas.ontology.agents.AgentAction, 213
cmas.ontology.agents.Component, 230
cmas.ontology.agents.DeploymentAgent, 248
cmas.ontology.agents.Material, 333
cmas.ontology.agents.Part, 358
cmas.ontology.agents.Resource, 420
cmas.ontology.processplan.ProcessPlanJava, 383
cmas.ontology.processplan.ProcessPlanStructuredText, 393

hasNext
cmas.ontology.variables.VariableArray, 473
cmas.ontology.variables.VariableObject, 500
cmas.ontology.variables.VariableReference, 526
cmas.ontology.variables.VariableString, 536
hasRelation
cmas.ontology.agents.Agent, 205
cmas.ontology.agents.AgentAction, 213
cmas.ontology.agents.Component, 230
cmas.ontology.agents.DeploymentAgent, 248
cmas.ontology.agents.Material, 333
cmas.ontology.agents.Part, 358
cmas.ontology.agents.Resource, 420
cmas.ontology.base.Entity, 256
cmas.ontology.base.Reference, 403
cmas.ontology.batch.Batch, 222
cmas.ontology.Hazard, 276
cmas.ontology.interfaces.Goal, 268
cmas.ontology.interfaces.Interface, 286
cmas.ontology.interfaces.InterfaceAssembly, 293
cmas.ontology.interfaces.InterfaceBase, 301
cmas.ontology.interfaces.InterfaceLocation, 309
cmas.ontology.interfaces.InterfaceSpecialisation, 317
cmas.ontology.interfaces.InterfaceTransport, 326
cmas.ontology.Note, 343
cmas.ontology.processplan.ProcessPlan, 366
cmas.ontology.processplan.ProcessPlanGoals, 375
cmas.ontology.processplan.ProcessPlanJava, 383
cmas.ontology.processplan.ProcessPlanStructuredText, 393
cmas.ontology.processplan.SkillCall, 438
cmas.ontology.Settings, 431
cmas.ontology.variables.Value, 457
cmas.ontology.variables.Variable, 464
cmas.ontology.variables.VariableArray, 473
cmas.ontology.variables.VariableBoolean, 482
cmas.ontology.variables.VariableInteger, 491
cmas.ontology.variables.VariableObject, 500
cmas.ontology.variables.VariableProperty, 509
cmas.ontology.variables.VariableReal, 518
cmas.ontology.variables.VariableReference, 526
cmas.ontology.variables.VariableString, 536
cmas.ontology.workflow.Requirement, 411
cmas.ontology.workflow.WorkFlow, 547
cmas.ontology.workflow.WorkFlowLog, 555
HazardTypes
cmas.ontology.Types, 448
How to use CMAS, 4

Interface action, 62
Interfaces, 61
InterfaceTypes
cmas.ontology.Types, 449
isValid
cmas.ontology.adapter.FILE.FileClient, 261
cmas.ontology.adapter.Modbus.ModbusClient, 337
cmas.ontology.adapter.OPCUA.OPCUAClient, 348
cmas.ontology.adapter.REST.RESTClient, 423
cmas.ontology.adapter.socket.SocketTCPClient, 441
cmas.ontology.adapter.socket.SocketUDPClient, 443
cmas.ontology.adapter.SQL.SQLClient, 446
cmas.ontology.adapter.websocket.WebSocketClient, 540
cmas.ontology.adapter.ZeroMQ.ZMQClient, 558

makeInstance
cmas.ontology.variables.VariableObject, 500
cmas.ontology.variables.VariableReference, 527

Negotiation, 64

onFailed
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Part, 358
onFinished
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Part, 358
onMessage
cmas.ontology.agents.Resource, 420
onNegotiate
cmas.ontology.processplan.ProcessPlanGoals, 375
cmas.ontology.processplan.ProcessPlanJava, 385
cmas.ontology.processplan.ProcessPlanStructuredText, 393
onOverlap
cmas.ontology.processplan.ProcessPlanGoals, 375
cmas.ontology.processplan.ProcessPlanJava, 385
cmas.ontology.processplan.ProcessPlanStructuredText, 393
onRun
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Part, 358
onStop
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Part, 359
Ontology, 77

onUndeploy
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Part, 359
openEntityEditor
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Material, 333
cmas.ontology.agents.Part, 359
cmas.ontology.agents.Resource, 420
cmas.ontology.base.Reference, 403
cmas.ontology.batch.Batch, 222
cmas.ontology.Hazard, 276
cmas.ontology.interfaces.Goal, 268
cmas.ontology.interfaces.Interface, 286
cmas.ontology.interfaces.InterfaceAssembly, 294
cmas.ontology.interfaces.InterfaceLocation, 310
cmas.ontology.interfaces.InterfaceSpecialisation, 317
cmas.ontology.interfaces.InterfaceTransport, 326
cmas.ontology.Note, 343
cmas.ontology.processplan.ProcessPlanGoals, 375
cmas.ontology.processplan.ProcessPlanJava, 385
cmas.ontology.processplan.ProcessPlanStructuredText, 393
cmas.ontology.processplan.SkillCall, 438
cmas.ontology.Settings, 431
cmas.ontology.variables.Value, 457
cmas.ontology.variables.Variable, 465
cmas.ontology.variables.VariableArray, 473
cmas.ontology.variables.VariableBoolean, 482
cmas.ontology.variables.VariableInteger, 491
cmas.ontology.variables.VariableObject, 500
cmas.ontology.variables.VariableProperty, 509
cmas.ontology.variables.VariableReal, 518
cmas.ontology.variables.VariableReference, 527
cmas.ontology.variables.VariableString, 536
cmas.ontology.workflow.Requirement, 411
cmas.ontology.workflow.WorkFlowLog, 555

Planner, 101
Plug & Produce and Multiagent, 2
postMessage
cmas.ontology.agents.AgentAction, 213
cmas.ontology.agents.Component, 230
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Material, 334
cmas.ontology.agents.Part, 359
cmas.ontology.agents.Resource, 420
cmas.ontology.interfaces.AbstractInterface, 195
cmas.ontology.interfaces.Goal, 268
cmas.ontology.interfaces.Interface, 286
cmas.ontology.interfaces.InterfaceAssembly, 294
cmas.ontology.interfaces.InterfaceBase, 301
cmas.ontology.interfaces.InterfaceLocation, 310
cmas.ontology.interfaces.InterfaceSpecialisation, 317
cmas.ontology.interfaces.InterfaceTransport, 326
cmas.ontology.processplan.ProcessPlan, 366
cmas.ontology.processplan.ProcessPlanGoals, 375
cmas.ontology.processplan.ProcessPlanJava, 385
cmas.ontology.processplan.ProcessPlanStructuredText, 393
Processplan action, 108
Processplans, 107
ProcessPlanTypes
cmas.ontology.Types, 449

ReadData
cmas.ontology.adapter.Adapter, 198
cmas.ontology.adapter.FILE.FileClient, 261
cmas.ontology.adapter.Modbus.ModbusClient, 337
cmas.ontology.adapter.OPCUA.OPCUAClient, 348
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 351
cmas.ontology.adapter.REST.RESTClient, 423
cmas.ontology.adapter.socket.SocketTCPClient, 441
cmas.ontology.adapter.socket.SocketUDPClient, 443
cmas.ontology.adapter.SQL.SQLClient, 446
cmas.ontology.adapter.websocket.WebSocketClient, 540
cmas.ontology.adapter.ZeroMQ.ZMQClient, 558
Requirements, 113
RequirementTypes
cmas.ontology.Types, 449
resetAllAbstractInterfaces
cmas.ontology.processplan.ProcessPlanGoals, 376
cmas.ontology.processplan.ProcessPlanJava, 385
cmas.ontology.processplan.ProcessPlanStructuredText, 394
run
cmas.ontology.processplan.ProcessPlan, 366
cmas.ontology.processplan.ProcessPlanGoals, 376
cmas.ontology.processplan.ProcessPlanInterface, 377
cmas.ontology.processplan.ProcessPlanJava, 385
cmas.ontology.processplan.ProcessPlanStructuredText, 394
Running
cmas.ontology.agents.Resource, 420

Serialize
cmas.ontology.agents.Agent, 205
cmas.ontology.agents.AgentAction, 213
cmas.ontology.agents.Component, 230
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Material, 334
cmas.ontology.agents.Part, 359
cmas.ontology.agents.Resource, 421
cmas.ontology.base.Entity, 257
cmas.ontology.base.Reference, 403
cmas.ontology.batch.Batch, 222
cmas.ontology.Hazard, 276

cmas.ontology.interfaces.Goal, 268
cmas.ontology.interfaces.Interface, 286
cmas.ontology.interfaces.InterfaceAssembly, 294
cmas.ontology.interfaces.InterfaceBase, 301
cmas.ontology.interfaces.InterfaceLocation, 310
cmas.ontology.interfaces.InterfaceSpecialisation, 317
cmas.ontology.interfaces.InterfaceTransport, 326
cmas.ontology.Note, 343
cmas.ontology.processplan.ProcessPlan, 366
cmas.ontology.processplan.ProcessPlanGoals, 376
cmas.ontology.processplan.ProcessPlanJava, 385
cmas.ontology.processplan.ProcessPlanStructuredText, 394
cmas.ontology.processplan.SkillCall, 438
cmas.ontology.Settings, 431
cmas.ontology.variables.Value, 457
cmas.ontology.variables.Variable, 465
cmas.ontology.variables.VariableArray, 473
cmas.ontology.variables.VariableBoolean, 482
cmas.ontology.variables.VariableInteger, 491
cmas.ontology.variables.VariableObject, 500
cmas.ontology.variables.VariableProperty, 509
cmas.ontology.variables.VariableReal, 518
cmas.ontology.variables.VariableReference, 527
cmas.ontology.variables.VariableString, 536
cmas.ontology.workflow.Requirement, 411
cmas.ontology.workflow.WorkFlow, 547
cmas.ontology.workflow.WorkFlowLog, 555

setDebug
cmas.ontology.adapter.Adapter, 198
cmas.ontology.adapter.FILE.FileClient, 261
cmas.ontology.adapter.Modbus.ModbusClient, 337
cmas.ontology.adapter.OPCUA.OPCUAClient, 348
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 351
cmas.ontology.adapter.REST.RESTClient, 423
cmas.ontology.adapter.socket.SocketTCPClient, 441
cmas.ontology.adapter.socket.SocketUDPClient, 443
cmas.ontology.adapter.SQL.SQLClient, 446
cmas.ontology.adapter.websocket.WebSocketClient, 540
cmas.ontology.adapter.ZeroMQ.ZMQClient, 558

setDescription
cmas.ontology.agents.Agent, 205
cmas.ontology.agents.AgentAction, 213
cmas.ontology.agents.Component, 230
cmas.ontology.agents.DeploymentAgent, 249
cmas.ontology.agents.Material, 334
cmas.ontology.agents.Part, 359
cmas.ontology.agents.Resource, 421
cmas.ontology.base.Entity, 257
cmas.ontology.base.Reference, 403
cmas.ontology.batch.Batch, 222
cmas.ontology.Hazard, 276
cmas.ontology.interfaces.Goal, 268
cmas.ontology.interfaces.Interface, 286
cmas.ontology.interfaces.InterfaceAssembly, 294
cmas.ontology.interfaces.InterfaceBase, 301
cmas.ontology.interfaces.InterfaceLocation, 310
cmas.ontology.interfaces.InterfaceSpecialisation, 317
cmas.ontology.interfaces.InterfaceTransport, 326
cmas.ontology.Note, 343
cmas.ontology.processplan.ProcessPlan, 366
cmas.ontology.processplan.ProcessPlanGoals, 376
cmas.ontology.processplan.ProcessPlanJava, 385
cmas.ontology.processplan.ProcessPlanStructuredText, 394
cmas.ontology.processplan.SkillCall, 439
cmas.ontology.Settings, 431
cmas.ontology.variables.Value, 457
cmas.ontology.variables.Variable, 465
cmas.ontology.variables.VariableArray, 473
cmas.ontology.variables.VariableBoolean, 482
cmas.ontology.variables.VariableInteger, 491
cmas.ontology.variables.VariableObject, 500
cmas.ontology.variables.VariableProperty, 509
cmas.ontology.variables.VariableReal, 518
cmas.ontology.variables.VariableReference, 527
cmas.ontology.variables.VariableString, 537
cmas.ontology.workflow.Requirement, 411
cmas.ontology.workflow.WorkFlow, 547
cmas.ontology.workflow.WorkFlowLog, 555

setID
cmas.ontology.agents.Agent, 205
cmas.ontology.agents.AgentAction, 214
cmas.ontology.agents.Component, 231
cmas.ontology.agents.DeploymentAgent, 250
cmas.ontology.agents.Material, 334
cmas.ontology.agents.Part, 359
cmas.ontology.agents.Resource, 421
cmas.ontology.base.Entity, 257
cmas.ontology.base.Reference, 403, 404
cmas.ontology.batch.Batch, 222, 223
cmas.ontology.Hazard, 276, 277
cmas.ontology.interfaces.Goal, 269
cmas.ontology.interfaces.Interface, 287
cmas.ontology.interfaces.InterfaceAssembly, 294
cmas.ontology.interfaces.InterfaceBase, 303
cmas.ontology.interfaces.InterfaceLocation, 310
cmas.ontology.interfaces.InterfaceSpecialisation, 318
cmas.ontology.interfaces.InterfaceTransport, 326
cmas.ontology.Note, 345
cmas.ontology.processplan.ProcessPlan, 368
cmas.ontology.processplan.ProcessPlanGoals, 376
cmas.ontology.processplan.ProcessPlanJava, 386
cmas.ontology.processplan.ProcessPlanStructuredText, 394
cmas.ontology.processplan.SkillCall, 439

cmas.ontology.Settings, 431
cmas.ontology.variables.Value, 457
cmas.ontology.variables.Variable, 465
cmas.ontology.variables.VariableArray, 473, 474
cmas.ontology.variables.VariableBoolean, 482, 483
cmas.ontology.variables.VariableInteger, 491, 492
cmas.ontology.variables.VariableObject, 501
cmas.ontology.variables.VariableProperty, 509, 510
cmas.ontology.variables.VariableReal, 518, 519
cmas.ontology.variables.VariableReference, 527
cmas.ontology.variables.VariableString, 537
cmas.ontology.workflow.Requirement, 411, 412
cmas.ontology.workflow.WorkFlow, 547
cmas.ontology.workflow.WorkFlowLog, 555, 556
setName
cmas.ontology.agents.Agent, 206
cmas.ontology.agents.AgentAction, 214
cmas.ontology.agents.Component, 231
cmas.ontology.agents.DeploymentAgent, 250
cmas.ontology.agents.Material, 334
cmas.ontology.agents.Part, 359
cmas.ontology.agents.Resource, 421
cmas.ontology.base.Entity, 257
cmas.ontology.base.Reference, 404
cmas.ontology.batch.Batch, 223
cmas.ontology.Hazard, 277
cmas.ontology.interfaces.Goal, 269
cmas.ontology.interfaces.Interface, 287
cmas.ontology.interfaces.InterfaceAssembly, 294
cmas.ontology.interfaces.InterfaceBase, 303
cmas.ontology.interfaces.InterfaceLocation, 310
cmas.ontology.interfaces.InterfaceSpecialisation, 318
cmas.ontology.interfaces.InterfaceTransport, 327
cmas.ontology.Note, 345
cmas.ontology.processplan.ProcessPlan, 368
cmas.ontology.processplan.ProcessPlanGoals, 376
cmas.ontology.processplan.ProcessPlanJava, 386
cmas.ontology.processplan.ProcessPlanStructuredText, 394
cmas.ontology.processplan.SkillCall, 439
cmas.ontology.Settings, 433
cmas.ontology.variables.Value, 458
cmas.ontology.variables.Variable, 465
cmas.ontology.variables.VariableArray, 474
cmas.ontology.variables.VariableBoolean, 483
cmas.ontology.variables.VariableInteger, 492
cmas.ontology.variables.VariableObject, 501
cmas.ontology.variables.VariableProperty, 510
cmas.ontology.variables.VariableReal, 519
cmas.ontology.variables.VariableReference, 528
cmas.ontology.variables.VariableString, 537
cmas.ontology.workflow.Requirement, 412
cmas.ontology.workflow.WorkFlow, 548
cmas.ontology.workflow.WorkFlowLog, 556
setServerHost
cmas.ontology.adapter.OPCUA.OPCUAClient, 348
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 351
setServerName
cmas.ontology.adapter.OPCUA.OPCUAClient, 348
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 351
setServerPort
cmas.ontology.adapter.OPCUA.OPCUAClient, 348
cmas.ontology.adapter.OPCUA.OPCUAClientBase, 352
setTime
cmas.ontology.workflow.WorkFlowLog, 556
setValue
cmas.ontology.variables.Value, 458
cmas.ontology.variables.VariableArray, 474
cmas.ontology.variables.VariableBoolean, 483
cmas.ontology.variables.VariableInteger, 492
cmas.ontology.variables.VariableObject, 501
cmas.ontology.variables.VariableProperty, 510
cmas.ontology.variables.VariableReal, 519
cmas.ontology.variables.VariableReference, 528
cmas.ontology.variables.VariableString, 537
setValueUndoable
cmas.ontology.variables.Value, 458
cmas.ontology.variables.VariableArray, 474
cmas.ontology.variables.VariableBoolean, 483
cmas.ontology.variables.VariableInteger, 492
cmas.ontology.variables.VariableObject, 501
cmas.ontology.variables.VariableProperty, 510
cmas.ontology.variables.VariableReal, 519
cmas.ontology.variables.VariableReference, 528
cmas.ontology.variables.VariableString, 537
SkillTypes
cmas.ontology.Types, 449
stopEditor
cmas.ontology.agents.AgentAction, 214
cmas.ontology.agents.Component, 231
cmas.ontology.agents.DeploymentAgent, 250
cmas.ontology.agents.Material, 334
cmas.ontology.agents.Part, 360
cmas.ontology.agents.Resource, 421
cmas.ontology.processplan.ProcessPlanJava, 386
cmas.ontology.processplan.ProcessPlanStructuredText, 395
Structured text, 117

valueOf
cmas.ontology.variables.Value, 458
cmas.ontology.variables.VariableArray, 474
cmas.ontology.variables.VariableBoolean, 483
cmas.ontology.variables.VariableInteger, 492
cmas.ontology.variables.VariableObject, 501
cmas.ontology.variables.VariableProperty, 510
cmas.ontology.variables.VariableReal, 519
cmas.ontology.variables.VariableReference, 528
cmas.ontology.variables.VariableString, 538
ValueTypes
cmas.ontology.Types, 449

Variable adapters, 177
Variable bounds, 69, 182
Variables, 173

WorkFlowTypes
    cmas.ontology.Types, 450
WriteData
    cmas.ontology.adapter.Adapter, 198
    cmas.ontology.adapter.FILE.FileClient, 261
    cmas.ontology.adapter.Modbus.ModbusClient, 337
    cmas.ontology.adapter.OPCUA.OPCUAClient, 349
    cmas.ontology.adapter.OPCUA.OPCUAClientBase, 352
    cmas.ontology.adapter.REST.RESTClient, 423, 424
    cmas.ontology.adapter.socket.SocketTCPClient, 441
    cmas.ontology.adapter.socket.SocketUDPClient, 443, 444
    cmas.ontology.adapter.SQL.SQLClient, 446
    cmas.ontology.adapter.websocket.WebSocketClient, 540, 541
    cmas.ontology.adapter.ZeroMQ.ZMQClient, 558, 559